Android Application Development

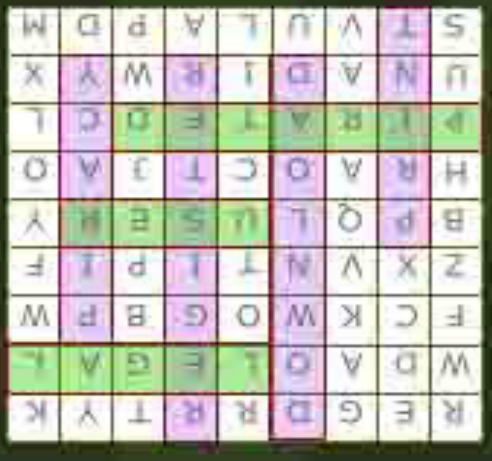
Onlinevarsity

Find these words

- REGISTER
- PRINT
- LEGAL
- PIRACY
- DOWNLOAD
- USER
- PIRATED

R	E	G	D	R	R	T	Y	K
W	D	A	O	L	E	G	A	L
F	C	K	W	O	G	B	P	W
Z	X	V	N	T	I	P	I	F
B	P	Q	L	U	S	E	R	Y
H	R	A	O	C	T	J	A	O
P	I	R	A	T	E	D	C	L
U	N	A	D	I	R	W	Y	X
S	T	V	U	L	A	P	D	M

Solution



PIRACY IS NOT GOOD

Register on **www.onlinevarsity.com**
to download a legal copy of this ebook

Android Application Development

© 2016 Aptech Limited

All rights reserved.

No part of this book may be reproduced or copied in any form or by any means – graphic, electronic or mechanical, including photocopying, recording, taping, or storing in information retrieval system or sent or transferred without the prior written permission of copyright owner Aptech Limited.

All trademarks acknowledged.

APTECH LIMITED
Contact E-mail: ov-support@onlinevarsity.com

Edition 1 - 2016

Preface

The book, Android Application Development, aims to teach the basics of developing Android applications for Android enabled devices. The book introduces the developer to the new features of Android Lollipop and Android Marshmallow 6.0 with a step-by-step approach. The book also includes example codes that are created using Android Software Development Kit (SDK).

Android programming has undergone several reformations since its inception. This book intends to familiarize the developer with new features and APIs of Android 5.0 and Android 6.0. The book begins with an introduction to the basic concepts of developing Android applications using Eclipse IDE as well as Android Studio. Further, the book explains the various new user interface components and views used to build the Android UI. It also teaches the different methods for saving and storing data in the application. The book enables a developer to create professional Android applications using connectivity APIs, Wi-Fi, and Google Maps. Additionally, the book covers an explanation of publishing an Android application to the Google Play Store. Further, Android Interface Definition Language (AIDL) and Android Native Development Kit (NDK) are also explained in the book.

The knowledge and information in this book is the result of the concentrated effort of the Design Team, which is continuously striving to bring to you the latest, the best and the most relevant subject matter in Information Technology. As a part of Aptech's quality drive, this team does intensive research and curriculum enrichment to keep it in line with industry trends and learner requirements.

We will be glad to receive your suggestions. Please send us your feedback, addressed to the Design Centre at Aptech's corporate office.

Design Team

Onlinevarsity

INDUSTRY BEST PRACTICES

SYNC WITH THE INDUSTRY

Table of Contents

Sessions

1. Introduction to Android
2. Getting Started with Android
3. Android System Overview
4. Android User Interface (UI)
5. More UI Elements
6. Media Handling
7. Data Handling and Content Providers
8. Services, Broadcast Receivers, and Intent Filters
9. Google API
10. Web Services in Android
11. Wireless and Networking
12. Telephony, SMS, and VoIP
13. Sensors
14. Google Play Store
15. Android Studio 2.1 and Android 6.0 Marshmallow
16. Material Design
17. Android Interface Definition Language
18. Android Native Development Kit (NDK)

Onlinevarsity

01 Introduction to Android

Welcome to the Session, **Introduction to Android**.

This session introduces Android and explains its history and advantages. It also describes the Android architecture, the layers, application framework, libraries, and the Linux Kernel. Further, it explains how to install the Android Software Development Kit (SDK).

In this Session, you will learn to:

- Explain Android
- Explain the history of Android
- Describe the architectural framework of Android
- Explain the downloading and installation process of Android SDK

Session

01

Introduction to Android

1.1 Introduction

An Operating System (OS) is a software program that enables communication and utilization of the hardware resources by the software programs. Each OS can be broadly divided into the Application layer, the Kernel, and the hardware and device driver layers. Windows 7 and Ubuntu are examples of OS.

The Kernel plays the most important role in the functionality of an OS. It manages the I/O and data processing requests from the software application layer. It is also responsible for translating those requests into instructions understandable to the CPU and other electronic devices. Windows NT and Linux are examples of Kernels.

Following are some of the features of an OS:

- Keeps track of files and folders, the peripheral devices such as disk drives and printers.
- Assists in security settings in a computer.
- Memory management.

Having understood the concept of an OS, imagine the advantages of having full-fledged OS with the feature of a desktop/PC on a mobile. The profitability list is endless.

The current mobile market is seeing a steady rise in the use of smart phones which come with a sophisticated OS. Some of these include the iOS, Android, Windows Phone, Symbian, and Blackberry OS. The current market is dominated by Android and iOS. Each OS has its own application market place and applications made for one OS rarely run on the other, without the need to rewrite the code for the other OS.

1.2 Introduction to Android

Android is an Operating System (OS) for mobile devices running on the Linux Kernel. Mobile devices include Smartphones and Tablets as shown in figure 1.1.

Figure 1.1: Smartphone and Tablet

Android can be defined as a software stack for mobile devices. Apart from the OS, it includes middleware and key applications.

In brief, Android is:

- A free open source Linux based mobile OS.
- A development platform used for creating mobile applications.
- A device used for executing the OS and applications created for it.

Android is being used by a large community of developers to create applications. The Android applications are written in Java programming language. However, Android does not support a Java Virtual Machine (JVM). Hence, a new virtual machine which is exclusively meant Android is used to compile the Java classes. Originally a virtual machine supported by Android called Dalvik Virtual Machine (DVM) was created for this purpose. A distinctive feature of the Dalvik VM is that it administers low level memory management. The Dalvik VM depends on Linux Kernel for security, memory, and process management. The files executed by the Dalvik VM are available in the .dex format.

Starting from Lollipop, Dalvik VM has been replaced with a new Runtime Environment called the Android Runtime Environment (ART). This was released as a technical preview in Android Kitkat(4.4) and launched officially with the release of 5.0 lollipop.

Android OS has been popularized to the point where Android currently owns the majority of the Smartphone market share. This is mainly due to the following reasons:

- The wide spread utilization by major device manufacturers.
- The stability of the underlying Linux Kernel.
- The official support from Google.
- The wide range of functionality offered.

It has become the most used and powerful tool in mobile systems today. All over the world, many device manufacturers have brought to the market devices that are capable of running Android OS.

1.2.1 History of Android

Android Inc. began in 2003 when its founders started the company with the intention of developing software for mobile phones. In 2005, Google acquired Android Inc.

The first version of the Android was released on November 5, 2007. On this eventful day, Google, in coalition with a number of other companies, including mobile phone manufacturers and wireless providers revealed Android, which was built on the Linux platform. This group included key players such as, Texas Instruments, High Tech Computer Corporation (HTC), Google, Intel, LG, Motorola, Samsung Electronics, Broadcom Corporation, Marvell Technology Group, Nvidia, QualComm, Sprint Nextel, and T-Mobile. This coalition, termed as the Open Handset Alliance (OHA) aimed at providing a free and open source platform for mobile devices. The OHA came out with the release of Android's first version, built on the Linux Kernel version 2.6 with an open source Apache License.

Evolution of Android Versions

It is important to learn about Android's history to better understand its growing popularity and demand. From the time of its market release in 2008, Android has since released a number of versions. With each version came a new set of features and options.

Table 1.1 lists the different Android versions and related features.

Android Version and Release	Description
Android 1.0 – 2008	The first commercial version of Android was released in 2008. It was significant for offering a wide variety of applications as opposed to Apple's iPhone. Android Cupcake Version 1.5 – May 2009. **Prominent Features:** ☐ A user could surf the Web quickly and enjoy the experience of Internet browsing as it was more user friendly. ☐ Google Maps came with this version of Android, making it a useful device for travel. ☐ It also allowed the user to sync e-mail and calendar with their Gmail account.
Android Cupcake Version 1.5 – May 2009	The subsequent versions of Android are based on the names of desserts. The Cupcake version was incorporated with significant number of new features. The most popular feature amongst the set of new features was its virtual keyboard. **Prominent Features:** ☐ A number of shortcuts and widgets on the home screen. ☐ Virtual onscreen keyboard made Android phones lighter and smarter. ☐ Video recording made it possible for the user to upload videos to YouTube. ☐ Stereo Bluetooth enabled wireless music. ☐ Copy and Paste function enabled on the Web browser.

Android Version and Release	Description
Android Éclair Versions 2.0 and 2.1 – November 2009 and January 2010	The main upgrade that Éclair offered was support for the Microsoft Exchange Server and HTML 5. Thus, it opened the Android out to more business users. **Prominent features:** ☐ Access to Outlook mail. ☐ Access to multiple Google accounts. ☐ Support for flash, digital zoom, and other features in Camera settings. ☐ Search for Text and Multimedia Messaging Services (MMS) messages. ☐ Auto complete with a dictionary that included names in the phone directory. ☐ Additional Web browser features.
Android Froyo Version 2.2 – May 2010	Android Froyo was a popular version because it supported Flash. This dissolved one of the significant differences between the Android and the iPhone. **Prominent features:** ☐ Flash Player enhanced the Web, videos, photo viewing, and streaming audio experience. ☐ Improved back-up storage for contacts and e-mail data. ☐ Improved Outlook features. ☐ Shared 3G Internet connection with other gadgets using Wi-Fi. ☐ Better Web surfing and Bluetooth support.
Android Gingerbread Version 2.3 – December 2010	Android Gingerbread did not have major upgrades from its earlier version. The major difference was that single-core phones could now run applications that were previously designed for dual-core phones.

Android Version and Release	Description
Android Honeycomb Version 3.0 – February 2011	With Honeycomb, Android could be used for bigger screens, including tablet PCs. This version was specifically meant for tablets and cannot be used for mobile phones. The main attraction in this release was the User Interface (UI). It was more interactive, and offered easier navigation. It also offered a more personalized experience. The highlight was that these enhanced features were provided for all applications, including the ones developed for earlier Android versions. For instance, the applications in Android version 3.0 had better capabilities in terms of graphics, UI, and multimedia features. **Prominent Features:** ☐ Improvements for a large range of tablets. ☐ Zoom feature for apps with a fixed size. ☐ Ability to download media files from a Secure Digital (SD) card – Users with devices that provide support for an SD card can download media files from the card. Apps in the device can then play the files.
Android Ice Cream Sandwich (ICS) Version 4.0 – October 2011	Android ICS was built to merge Gingerbread and Honeycomb versions. However, this version does not support Flash. **Prominent Features:** ☐ A faster, smoother browser. ☐ Data traffic monitor ☐ User-friendly action bar instead of the Menu button. ☐ Better storage space for applications. ☐ Pre-written text messaging feature to decline calls. ☐ Face recognition feature for unlocking phone. ☐ Live video effects.

Android Version and Release	Description
Android Version 4.1, 4.2 and 4.3 Jelly Bean – June 2012, October 2012 and July 2013	Android Jelly Bean 4.1, 4.2, and 4.3 aimed at a better and faster UI. The features are said to be based on 'Project Butter' which enable Jelly Bean to function like 'Butter'. This means that the touch related features are synchronized better and works faster. **Prominent Features of 4.1:** ☐ Touch Prediction, which attempts to predict where the user will touch the screen. ☐ Google Now, which tries to predict and provide the user with information based on the user's existing patterns. ☐ Voice Typing without Online Access – When the user has a bad Internet connection or is not connected, the user can still use the voice-to-text feature. Previously voice-to-text required the user to be connected to the Internet. ☐ Voice Search, which enables user to search for information through voice rather than text. This makes the search process faster and easier. ☐ Improvements in the Notifications feature. **Prominent Features of 4.2:** ☐ Developer Options added to the Settings menu, which was previously stand-alone. ☐ Lock Screen can now display widgets, which could earlier be viewed only on the home screen. ☐ Disable Lock Screen option is made available. ☐ Better image editing options. ☐ New screensavers named Daydreams. ☐ Photo Sphere, which allows user to take a 360-degree picture. **Prominent Features of 4.3:** ☐ Bluetooth, low energy support and AVRCP support. ☐ OpenGL ES 3.0 support allowing for improved game graphics.

Android Version and Release	Description
	☐ 4K resolution support. ☐ Dial pad, auto-complete. ☐ Minor UI tweaks. ☐ Experimental support for Android Wear devices.
Android 4.4 – Kitkat – October 2013	Kitkat brought Android to wear devices with UI improvements and a new Runtime environment. **Prominent Features:** ☐ UI overhaul. ☐ New dialer app. ☐ Technical preview of ART. ☐ Wireless printing support. ☐ Official Android Wear devices support introduced.
Android 5.0 - Lollipop November 2014	Android Lollipop aims to provide major improvements in terms of performance and 64 bit CPU support along with a brand new UI scheme. **Prominent Features:** ☐ Official release of ART. ☐ 64-bit CPU support. ☐ Better multi core utilization. ☐ Material design. ☐ OpenGL ES 3.1 support. ☐ Material Design.

Table 1.1: History of Android

1.2.2 Features of Android

Android provides several user and developer friendly features, some of which are described as follows:

- **Ease of Programming** - Java is the language used for Android applications. A large number of developers are familiar with Java, which makes it easier for them to develop applications in Android. Java also provides numerous developer tools.
- **2D and 3D Graphics and Animation** - One of the main advantages of Android is that it comes with several APIs for developing 2D and 3D graphics. It also has several features

for adding animation to the UI.

- **Multiple Languages** - Android provides multiple language support for apps powering voice and text features. The user can also customize the language for the UI.
- **Web Browsers** - Android provides support for most Web browsers, such as Firefox, Opera, SkyFire, XScope, and Dolphin in addition to the default browser Chrome.
- **Multi Touch** - Android support multi-touch detection up to 16 points and higher simultaneously to allow development of complex gestures and multimedia applications utilizing these features.
- **Connectivity** - Android provides support for multiple wireless connectivity options such as Wi-Fi, Bluetooth and NFC. In addition to these, Android also provides the capability to 'tether' the connection to allow sharing a data connection.

Note - Tethering allows sharing the data connection of a device with other devices which do not have a direct data connection. Android allows creation of portable Wi-Fi hotspots.

- **Media Processing** - Android supports several DSPs and hardware decoders to take full advantage of the hardware encoding and decoding for playing and recording high resolution audio and video in multiple formats.
- **Sensor Support** - Android natively supports several sensors such as the Gyroscope, the accelerometer, the compass, and so on.
- **Storage Capabilities** - Android supports the SQLite database natively to offer a lightweight storage option. Media storage capabilities include external storage device support including the SD card family, USB devices, and remote file system support in addition to the on board storage.
- **Large Market Share** - Android is currently the smart phone OS with the highest market share. Any application developed for Android is guaranteed to be targeted at the widest range of audience.
- **Abstraction of Hardware Differences** - Android nullifies the need to know about the hardware that the application will run on or to interact with it directly in any way.
- **Open Source** - Android is completely open source which allows innovation and mods and ROMs to be developed. This increases the life span of a device and the support and community for the operating system.
- **Free to Use** - Any manufacturer willing to use Android can do so completely free of charge which has greatly helped with market penetration.
- **Ease of Use** - Android was designed with the end user in mind which made the operating system extremely easy to use.
- **Google Now** - A digital assistant/Information retrieval system introduced in Android 4.1 Jelly Bean.

New Features of Android 5.0 Lollipop

- Improved Notification System with Notification grouping.

- Optimized battery consumption as a result of Google's RandD project 'Project Volta'.
- Better performance.
- OpenGL ES 3.1 support for gaming and multimedia applications.
- Replaced Just in Time (JIT) compilation of Dalvik VM with an Ahead of Time (AoT) compilation of ART.
- Complete UI overhaul to make it more user friendly.
- A digital personal assistant with voice recognition in the form of Google now.

1.2.3 Device Types Running Android

Android is deployed on multiple types of devices and it is no longer just a mobile OS. Some of them are as follows:

- **Smartphones** - The most wide spread utilization of Android is in the field of smart phones.
- **Tablets** - Tablets are meant to act as mobile computers with usually a larger display and omission of telephony functionality. In the current state, there is large feature overlap between smartphones and tablets.
- **Televisions** - With the rise of the smart TV market, Android is utilized in Televisions due to Android's stability and the large app library. Sony and Toshiba are some of the major Smart TV manufacturers using Android as the choice for OS for their smart televisions.
- **Cars and In-Vehicle Infotainment** - The media and navigation capabilities of Android have allowed it to be used inside cars as media and entertainment devices.
- **Android Wear** - This is a rising market which has been recently supported officially by Android with the release of Android KitKat (4.4). Motorola's Moto 360, LG's G watch, and Samsung's Galaxy Gear are some of the prime examples of wearable devices running Android. The Smart watch market is expected to rise as new solutions are being developed to increase battery life of the devices.
- **Google Glass** - Google Glass is an optical head mounted device launched by Google running on the Android 4.4.2 Kitkat version.

Figure 1.2 depicts the types of devices running the Android Operating System.

Figure 1.2: Devices Running Android

1.2.4 Challenges to Developing for Android

Even though there are great benefits to developing for Android devices, there are certain challenges that need to address prior to development.

Some of these are as follows:

- **Hardware Fragmentation**

  Android devices have varying hardware and no two devices are guaranteed to have the same hardware or perform in the same manner. The hardware and device configuration possibilities are infinite. This is contrary to the situation to developing for iOS where the developer can expect the exact same hardware for every device that they are targeting.

  This limits the extent the developer can harness the hardware capabilities as the same app which runs flawlessly on one device can struggle to even perform at a reasonable level on another.

- **Software Fragmentation**

  Unlike the OS market of a desktop platform, the users of mobile devices are not directly in control of the choice of the OS in use. Any updates to the OS need to be made and delivered to the users by the manufactures. This means that some users can lag behind in terms of the OS version they are using.

  This results in a situation where developing for a certain version of the OS can result in the app being unusable with the devices running an older version.

- **Lack of Hardware Software Integration Standards**

  Google does not provide specifications for the type or nature of the hardware that should be used with devices running Android. This means that what a button 'X' does on one device may not be the same as it does on another device. In addition to this, any modifications that a device manufacturer makes to their devices are not shared or standardized and hence, utilizing these features would make the app unusable on other devices. Examples of such scenarios are the Gaming button on Xperia Play and the hover touch feature of Xperia Sola.

- **Lack of Quality Control on Android devices**

  Google has no authority and does not try to regulate the devices running Android. This can lead to performance issues and other unexpected circumstances when deploying apps on these devices.

1.2.5 Android and the Open Source Platform

Android, as specified earlier is based on Linux platform. The code is open source as shown in figure 1.3, that is, the code for the product's design and implementation is freely distributed.

Figure 1.3: Open Source

Android is developed privately by Google until it is released, at which point its source code is made available to everyone. Any developer can make changes to the existing code. This feature enables a wide variety of Android enthusiasts, service providers, and mobile device providers to modify the code. It also makes a whole plethora of applications available to Android.

The open source code makes Android a low cost, high quality, and customizable platform, thus, turning it into a widely used and extremely popular option. Another advantage of modified code is that new features and updates are added at a rapid rate. Android has an active community of developers and enthusiasts who use the Android Open Source Project (AOSP). The community develops custom versions of Android OS for devices much faster than the manufacturers. The community has delivered several high quality distributions commonly termed as 'ROMs' such as CyanogenMOD.

1.3 Android Architecture

The Android architecture, as shown in figure 1.4, is made up of different layers. Each layer consists of several program components. All the layers are connected to each other. The Android architecture consists of the following four layers:

- Kernel
- Libraries and Runtime
- Application Framework
- Applications

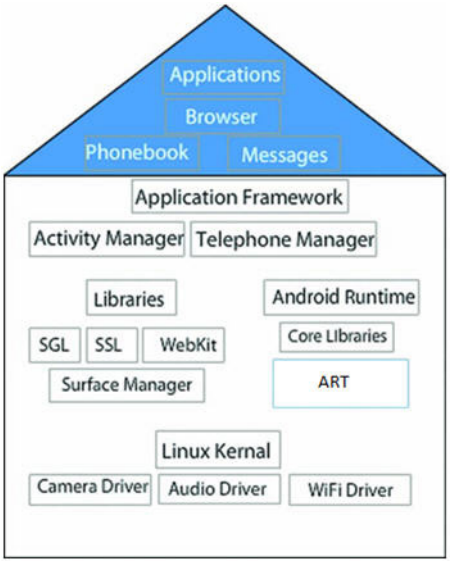


Figure 1.4: Android Architecture

Following is a brief explanation of the layers:

- **Linux Kernel**
  - The Linux Kernel is the base layer. This is the OS layer upon which the entire framework is built.
  - The layer communicates with the hardware and consists of the key hardware drivers. For example, for the camera to function on an Android phone, the Kernel must have a camera driver installed.
  - The Kernel is also used for important functionalities, such as, managing memory, processes, network, and security.
  - The Linux Kernel is responsible for managing the connected devices using the respective device drivers.
  - The Linux Kernel interacts with the Libraries layer. It also serves as an interface between the hardware and all other elements of the stack.
- **Libraries**
  - The layer immediately above the Linux Kernel consists of the libraries. In generic terms, a Library is nothing but a resource or information stack. For a developer, a library is a resource centre to assist in software development. The libraries contain data meant for specific hardware. The library consists of APIs and C/C++ core libraries. Different components make use of these libraries. Developers can access the libraries through the Application Framework layer.
  - Surface Manager – The Surface Manager enables combining different functions on a single frame. It helps in display management, and also determines how the applications should be layered.

- Open Graphics Libraries (OGL)/ES and Scalable Graphics Library (SGL) – Open Graphics libraries support 2D and 3D graphics implementation and related aspects.
  - **Media Framework** – Provides support for audio and video formats, recording, playback, and other related functions. It also helps to manage static images.
  - **FreeType** – Assists in bitmap and vector font rendering. Also supports fonts and font related functions.
  - **Secure Sockets Layer (SSL)** – Helps to provide security related features.
  - **SQLite** – SQLite is an open source storage database that manages all database related queries.
  - **WebKit** – Web browser provides the tools for building a Web browser.
- **Libc** - A small size custom library for the C compiler. This library needs to be loaded in all the processes.

Android Runtime

- The runtime is also part of the library layer. It is responsible for execution of applications.
- The ART runtime is similar to the Java Virtual Machine (JVM).
- The libraries layer interacts directly with the ART.
- With the introduction of ART as the replacement of Dalvik VM in Android Lollipop 5.0, there are some changes that are made in the runtime functionality of Android.
  - Just in Time (JIT) compilation is replaced with Ahead of Time (AoT) compilation. This brings several advantages such as faster load times for applications, better performance, and compatibility across varying hardware.
  - Applications are run in Native code instead of byte code.
  - .odex files replaced with ELF (Executable and Linkable) files.

Application Framework

- The Application Framework layer manages the applications for the basic functioning of the device. This layer enables important functions such as voice calls, data sharing, activities, and location management.
  - **Activity Manager** - Manages the lifecycle of all Android applications, such as Start, Run, Background, and Kill. It provides a common stack for applications running in various processes so that navigation between applications is synchronized.
  - **Packages Manager** - Monitors the applications installed on the mobile device. The user might download and install an application online. Also, the user may use a Compact Disc (CD) and package manager to track the application and its capabilities.

- **Windows Manager** - Manages active windows. It is more of a lower layer service offered by the Surface Manager.
- **Telephony Manager** - An important element that consists of the APIs to construct the phone application including voice SMS, and dialling services.
- **Content Providers** - The content providers are important to the Android platform, as they enable applications to share data with each other. For example, the messaging application may require information from the contacts application.
- **Resources Manager** - Stores the external elements of an application that do not include code, such as layout files, strings storage, and bitmaps.
- **View System** - Contains the basic applications for UI, such as lists and buttons. It also performs other tasks, such as dispatching events, drawing, and layouts.
- **Location Manager** - Manages GPS, Wi-Fi, and cell tower applications.
- **Notification Manager** - Manages notifications, such as reminders, task lists, alarms, and so on. Displays updates on the status bar.
- **Extensible Messaging and Presence Protocol (XMPP) Manager** - Manages XMPP service related applications. Provides support for Internet Messaging, Instant message, chat, offline messaging, and Voice over IP (VOIP).

- This layer interacts with the Applications layer.

☐ **Applications Layer:**

- The topmost layer consists of all the applications. A few applications, such as the Web browser and Contacts manager come with the standard installation. However, a variety of other applications can be added to the phone.
- The layer interacts with the other layers.

1.4 Dalvik VM vs. ART

Google introduced a new runtime called ART as a successor for the previous Dalvik VM in Android Kitkat (4.4). It was officially launched in the release of Android Lollipop (5.0). The main goal behind doing so was to make performance and battery usage improvements. ART replaces the Just in Time (JIT) compiler of Dalvik VM with an Ahead of Time (AoT) compiler. A JIT compiler runs the code only when necessary which can cause a performance bottle neck. An AoT does this well in advance fully utilizing the computational resources which would have been idle instead.

Garbage collection is another area of massive improvements in the new runtime environment. Garbage collection is the process of detecting and deleting unnecessary variables for better memory management. This results in better memory utilization.

With all these improvements, ART is considered objectively better than its predecessor.

1.5 Google Play Platform

Google Play (originally called Android Market) is Google's official market place for Android. Every Android device comes with Google Play and Google Play services applications pre-installed. It acts as the primary source of applications for Android devices. Installing applications from any other market place requires the user to explicitly provide permission to install application from other sources. This also makes the device vulnerable to malicious applications and as a result the number of users having this feature enabled is significantly less. Due to these circumstances, in order to target the maximum number of audience, deploying the application on the Google Play Store is considered a must.

All applications that are free of charge are available worldwide whereas paid applications are currently available in 135 countries. Google Play also provides the ability to remotely install applications on devices. Starting from 2013, Google has also added the facility to purchase media such as music, videos, movies, and books experimentally in a few countries. Regional pricing is introduced wherever possible.

In addition to acting as a marketplace, Google Play also provides device compatibility check so that the user is informed about the compatibility of the device with the application before installing. Google Play Services had been introduced in 2012 which acts as framework for retrieving usage information for providing facilities such as micro transactions and cross-device settings synchronization features.

1.6 Android Software Development Kit (SDK)

A Software Development Kit (SDK), as shown in figure 1.5, is a package that consists of all the tools required for developing applications.

Figure 1.5: Android SDK

The applications are typically developed for specific software, hardware, OS, or any other platform. The Android Software Development Kit (SDK) provides a comprehensive set of tools that help the developer to build applications for Android. The SDK also offers tools for testing and debugging apps. It includes libraries, sample code, an emulator and tutorials among other tools.

The advantage of using the SDK is that it is easier to download. Once the SDK is downloaded, developers can then login to the Android App Developer Web site and use the emulators to understand the application. The developer can avoid uploading, downloading, or reloading parts of software.

The 'Android Studio Bundle' comes with the Android Studio IDE and the Android SDK. Android Studio is Google's official IDE for developing applications for Android.

1.6.1 System Requirements for SDK Installation

Table 1.2 lists the system requirements for the Android SDK.

Requirement	Description
Operating Systems Supported and Requirements	☐ Windows 2003 (32-bit), Windows XP (32-bit), Vista (32- or 64-bit) or Windows 7 (32- or 64-bit). ☐ Mac OS X 10.5.8 or later (x86 only). ☐ Linux.
Development Environment Supported and Requirements	☐ Optional if using Android Studio - • Eclipse 4.0 (Juno) or higher. • Eclipse Java Development Tools (JDT) plug-in. • Several types of Eclipse packages are available for each platform. For developing Android applications, it is recommended that one of these packages be installed: ◆ Eclipse Integrated Development Environment (IDE) for Java Developers Eclipse Classic. ◆ Eclipse IDE for Java Enterprise Edition (EE) Developers. ◆ ADT plug-in. ☐ Mandatory - • JDK 7 or higher - Java Runtime Environment (JRE) alone is not sufficient.
Recommended Hardware Requirements	☐ Minimum 2 Gigabyte (GB) of Random Access Memory (RAM). ☐ Intel Core2Due or equal processor for x86 architecture. ☐ 4 GB hard disk space.

Table 1.2: Requirements for SDK

1.6.2 Downloading the SDK

Following are two options available for downloading the Android SDK.

- The Android Studio Bundle.
- The Android SDK standalone.

The Android Studio Bundle is recommended as it comes with the official IDE for developing Android application recommended by Google, the Android Studio. If Eclipse is your preferred IDE then the standalone package can be downloaded and used along with the Eclipse ADT plug-in.

Note - It is highly recommended to go with the Android Studio Bundle as Google has plans to drop support for the ADT plug-in in the future.

☐ Downloading and Installing JDK

1. Check system requirements for Windows OS mentioned earlier.
2. If the Java Development Kit is not installed, install it from http://www. oracle.com/technetwork/java/javase/downloads/index.html. Scroll down to locate Java SE 8 and click DOWNLOAD under JDK as shown in figure 1.6.

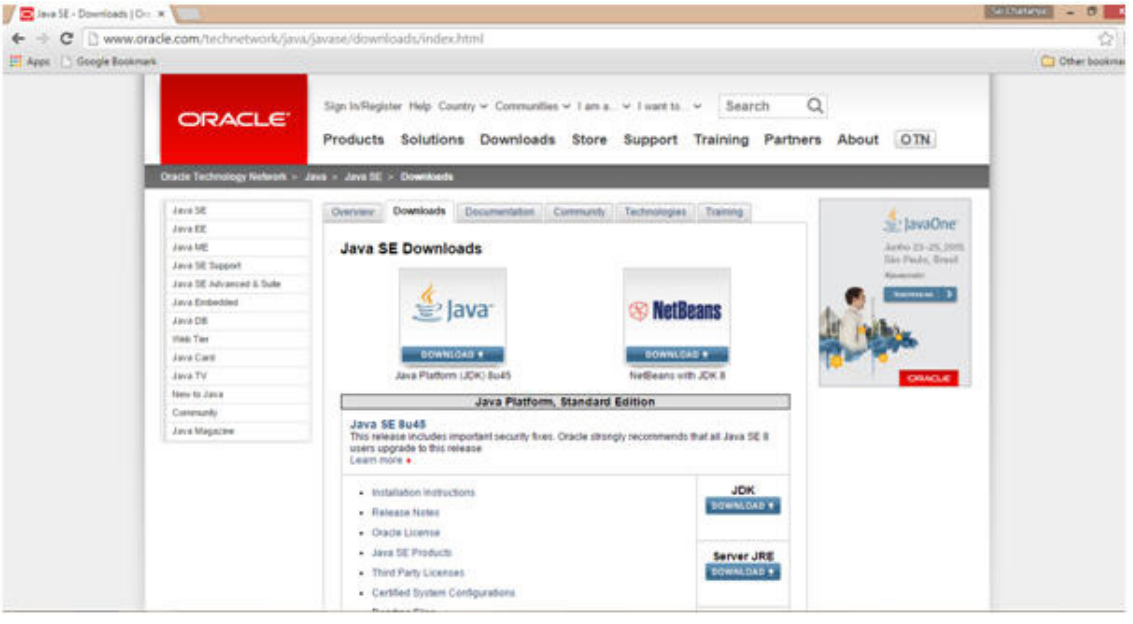


Figure 1.6: JDK Download Page

Note - Internet links are subject to change. If the link provided does not function, go to the http://www.oracle.com Web page and navigate to the Downloads page to find the updated link.

3. Start the installation running the downloaded setup file.
4. Click Next to proceed with the installation as shown in figure 1.7.

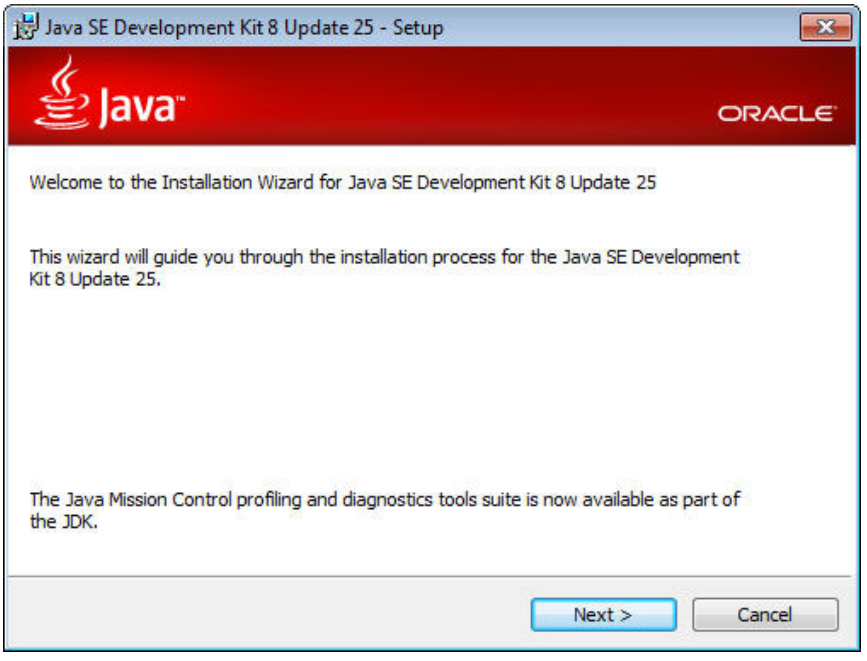


Figure 1.7: Installation Welcome Screen

5. Select the components to install and the destination directory as shown in figure 1.8.

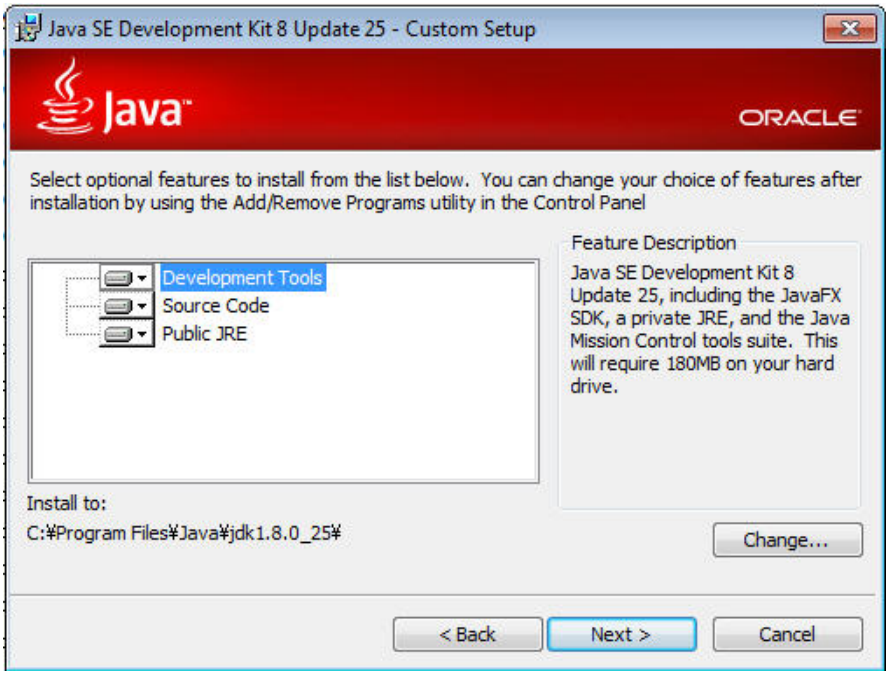


Figure 1.8: Installation Components Screen

6. Click Next.
7. The installation begins as shown in figure 1.9.

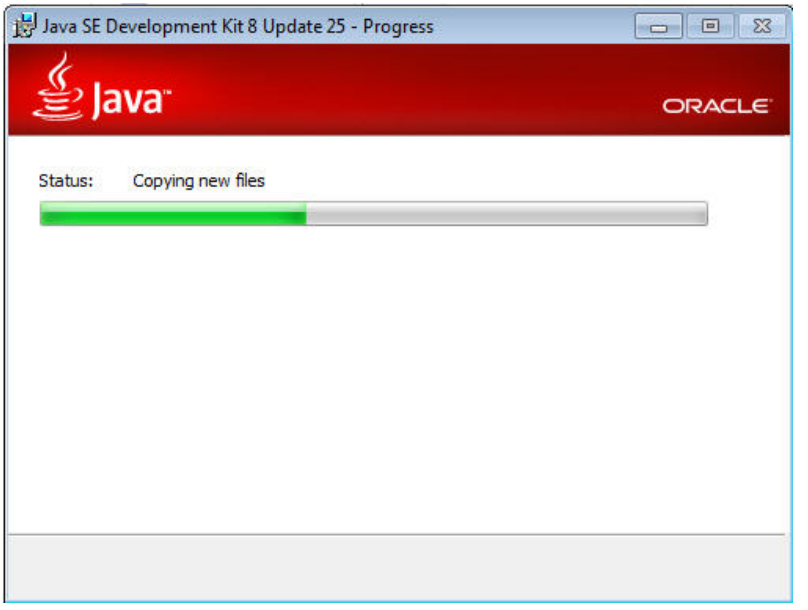


Figure 1.9: Installation Screen

8. Click Close to complete the installation as shown in figure 1.10.

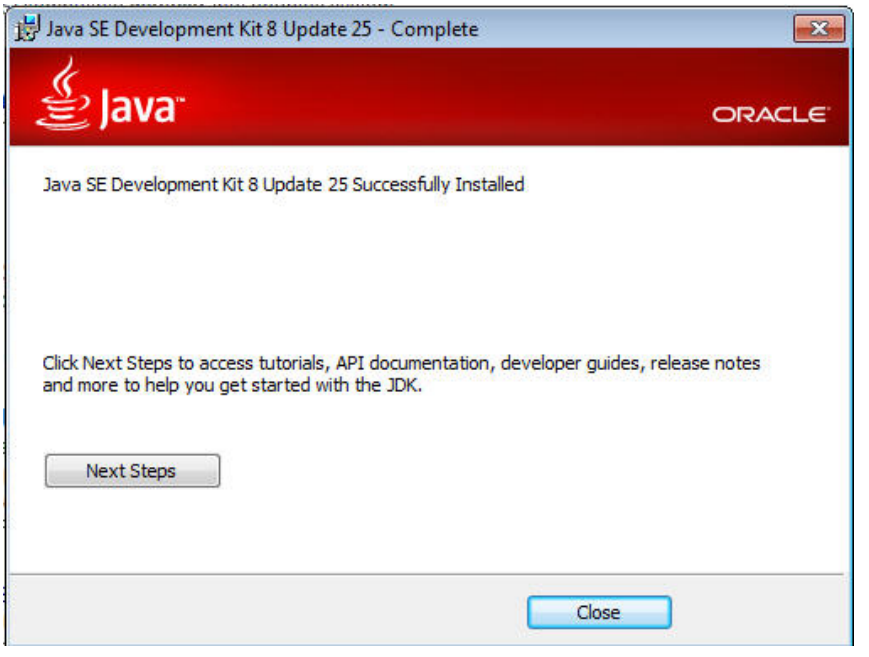


Figure 1.10: Installation Completion Screen

The installation folder can be opened to verify the installation as shown in figure 1.11.

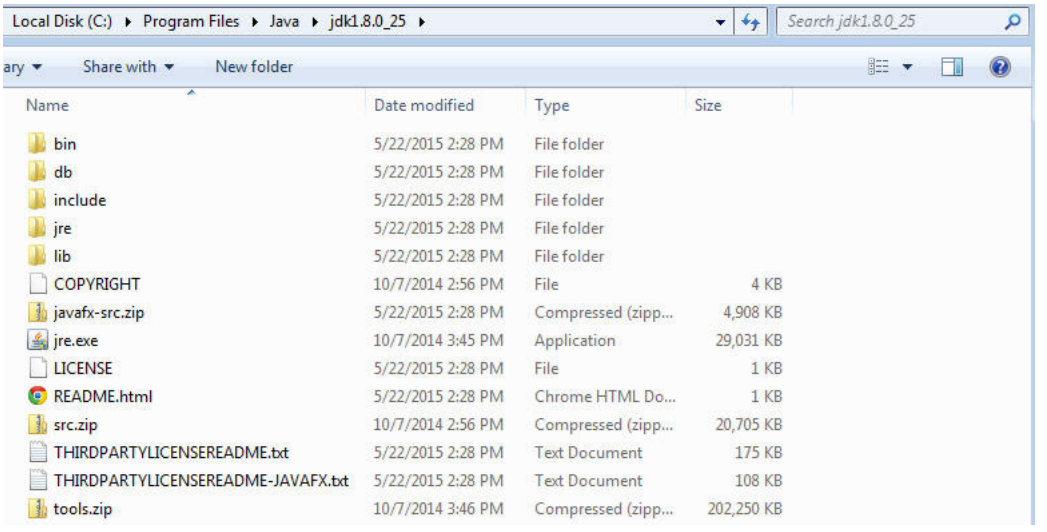


Figure 1.11: JDK Successfully Installed

☐ Downloading Android Studio Bundle

1. To download the Android SDK, go to https://developer.Android.com/sdk/index.html and click the Download Android Studio Bundle as shown in figure 1.12.

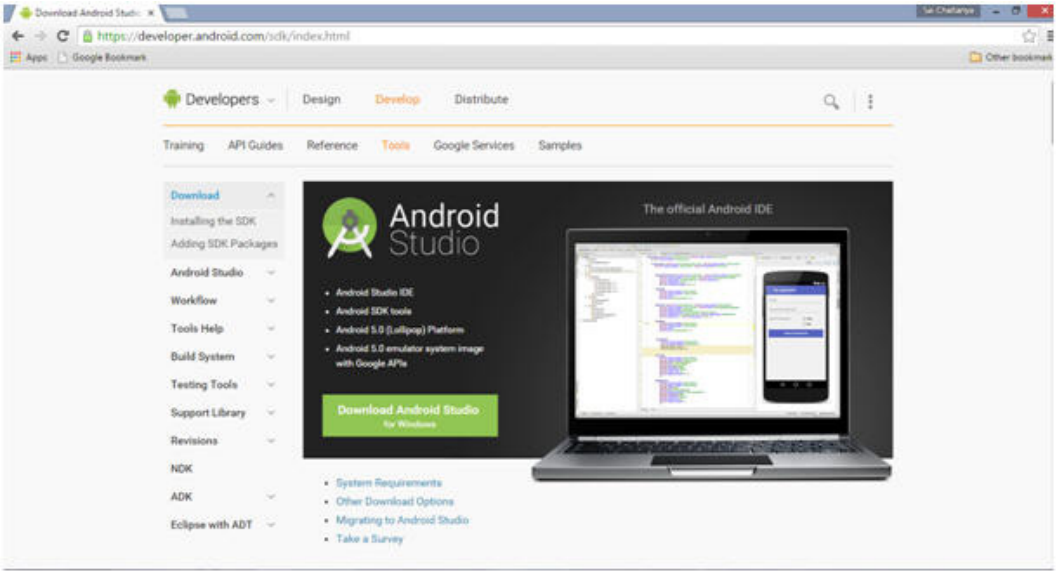


Figure 1.12: Android SDK Download Page

2. In the Terms and Conditions page as shown in figure 1.13, read the terms. Select the 'I have read and agree with the above terms and conditions' check box.

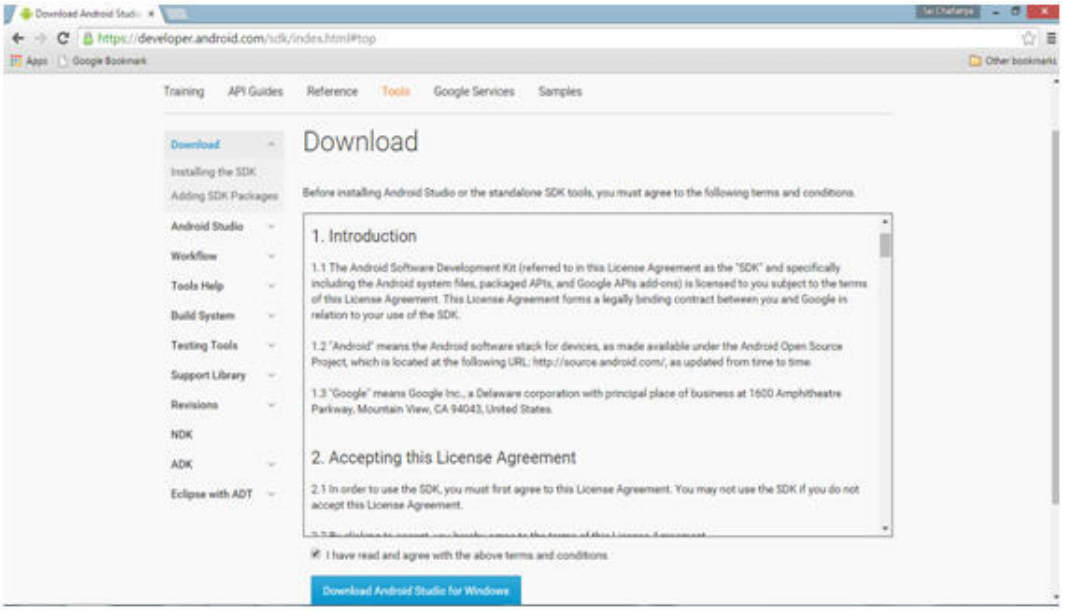


Figure 1.13: Android SDK Download Page

3. Click the Download Android Studio for Windows button.

The download begins as shown in figure 1.14. The file will be saved in the default download location, that is, the Downloads folder, unless specified otherwise.

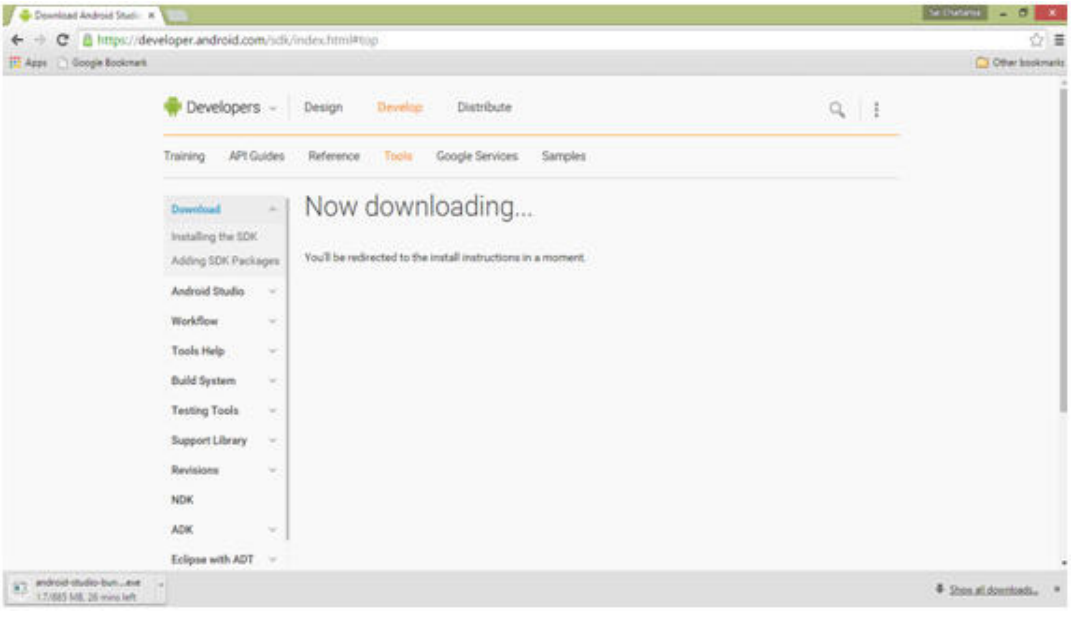


Figure 1.14: Android SDK Downloading Page

☐ Downloading Android SDK Standalone

1. To download the Android SDK, go to https://developer.Android.com/sdk/index.

html, and click the Download Android Studio Bundle as shown in figure 1.15.

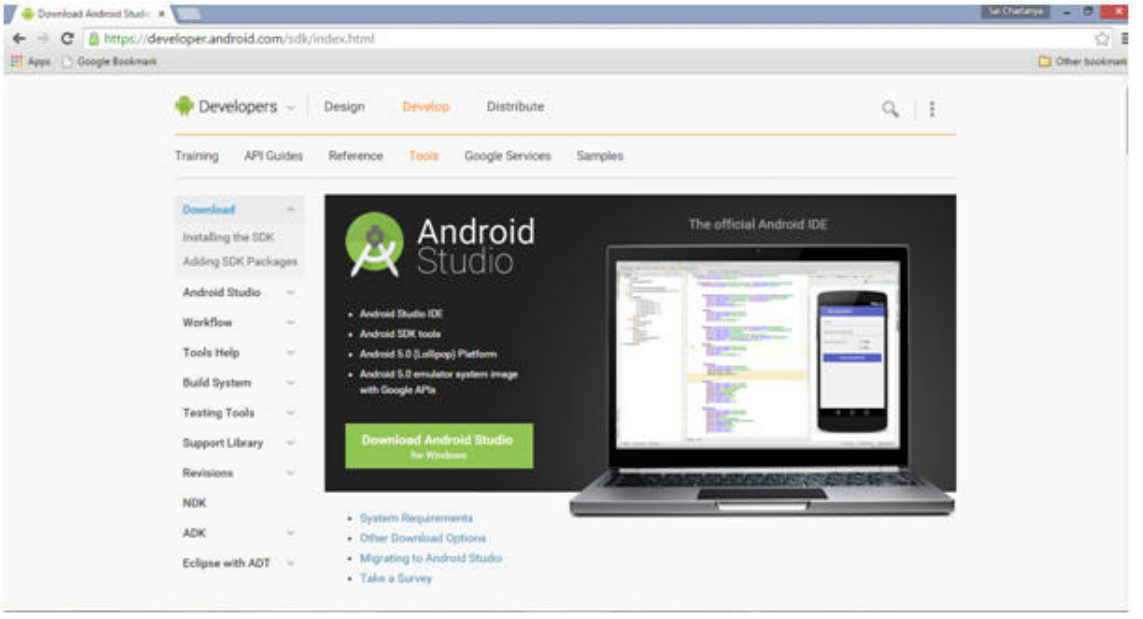


Figure 1.15: Android SDK Page

2. Scroll down to the Other Download Options section as shown in figure 1.16.

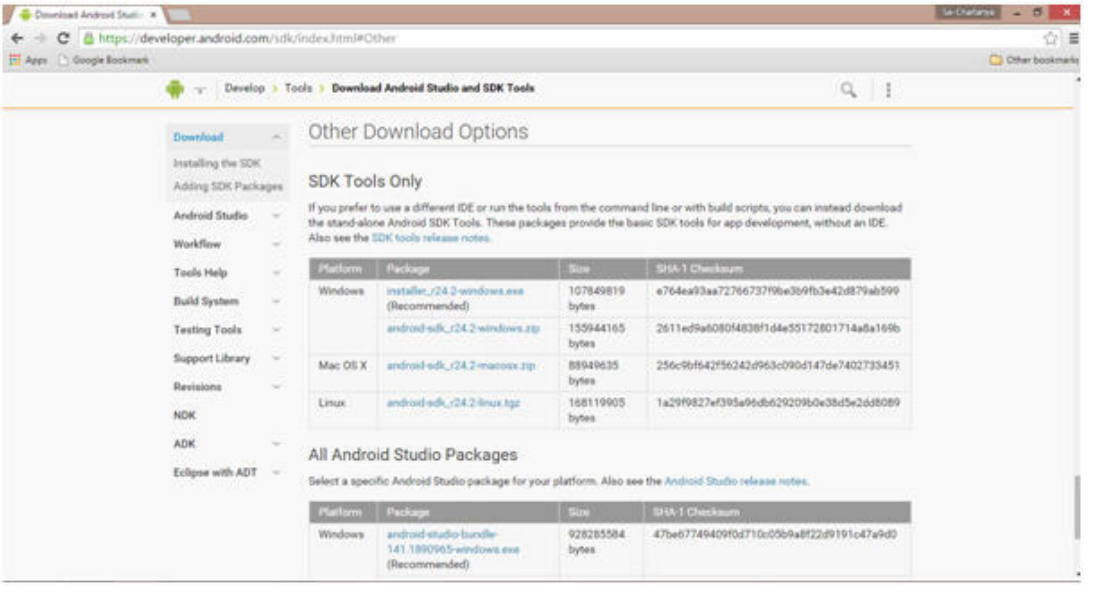


Figure 1.16: Android SDK Download Page

3. Click the package corresponding to your operating system.
4. In the Terms and Conditions page as shown in figure 1.17, read the terms. Select the 'I have read and agree with the above terms and conditions' check box.

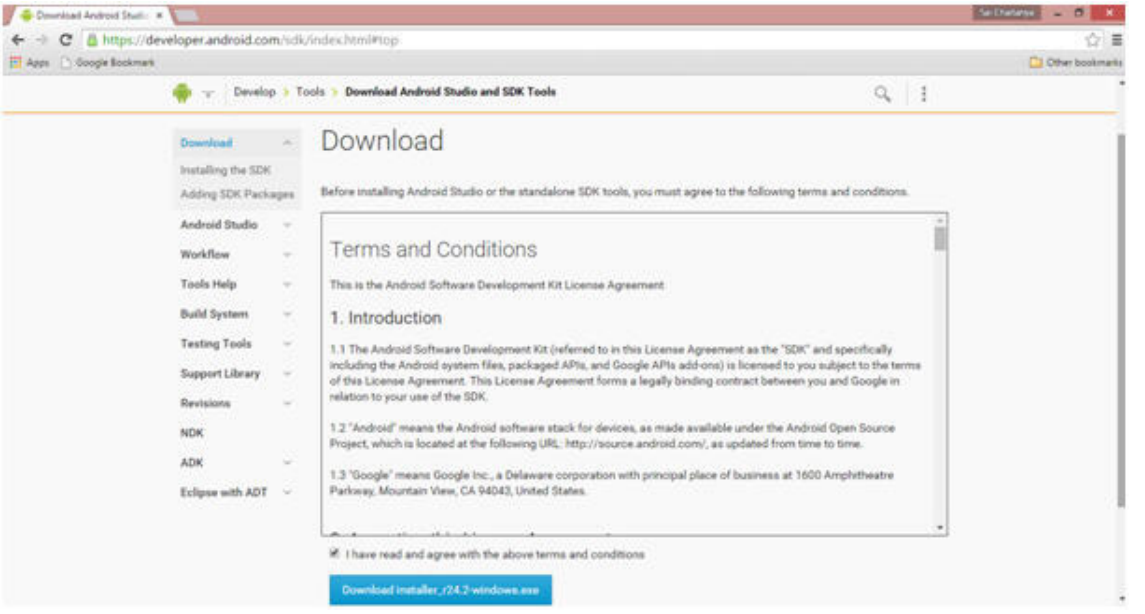


Figure 1.17: Terms and Conditions Page

5. Click the Download button at the bottom with the file name you have selected.
6. The download begins and the file will be saved in the default download location, that is, the Downloads folder, unless specified otherwise.

1.6.3 Installing Android SDK

In this section we will learn the process of installing the selected choice of SDK. The process of installation for both the Android Studio Bundle and the Android SDK standalone are similar.

☐ Android Studio Bundle/Android SDK Standalone

1. Go to the directory where the SDK has been downloaded as explained in the previous section.
2. Double-click to open the SDK installer. Select Yes on the UAC prompt if it appears. The installation wizard appears as shown in figure 1.18.

Figure 1.18: Android Setup Wizard

Note - The window may look different if the standalone SDK package is being installed.

3. Select all the components in the components screen as shown in figure 1.19 and click Next.

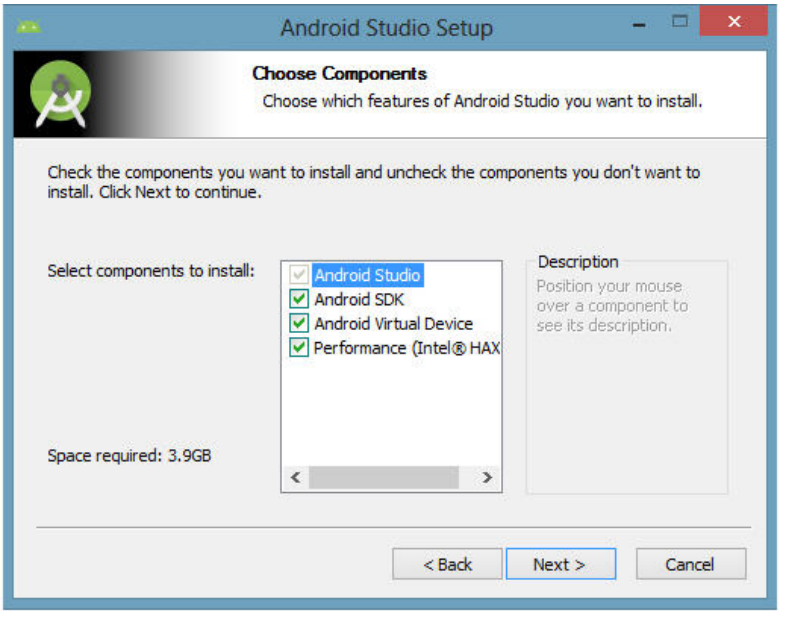


Figure 1.19: Android Setup Wizard Components

Note - The Android Studio option will not be available while installing the standalone SDK package.

4. The SDK license agreement appears. Click the I Agree button as shown in figure 1.20.

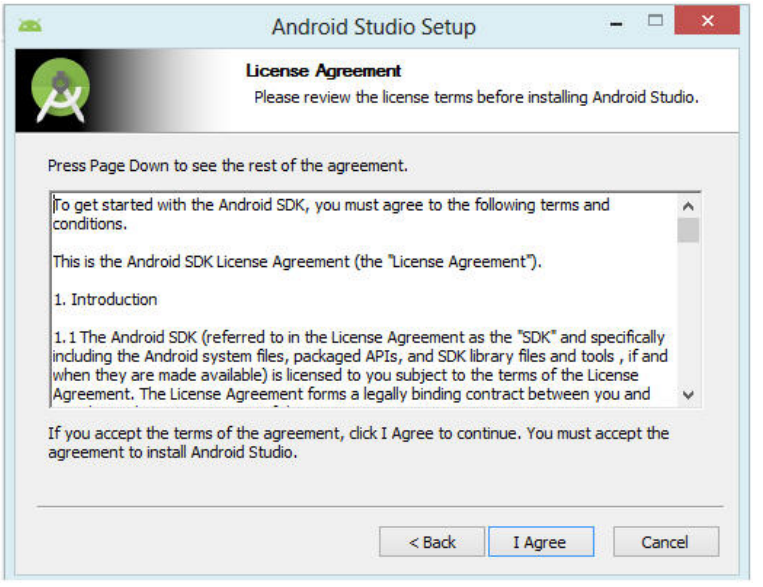


Figure 1.20: Android Setup Wizard License Agreement

The Intel Hardware Accelerated Execution Manager License Agreement appears if the component was selected in step 3 as shown in figure 1.21.

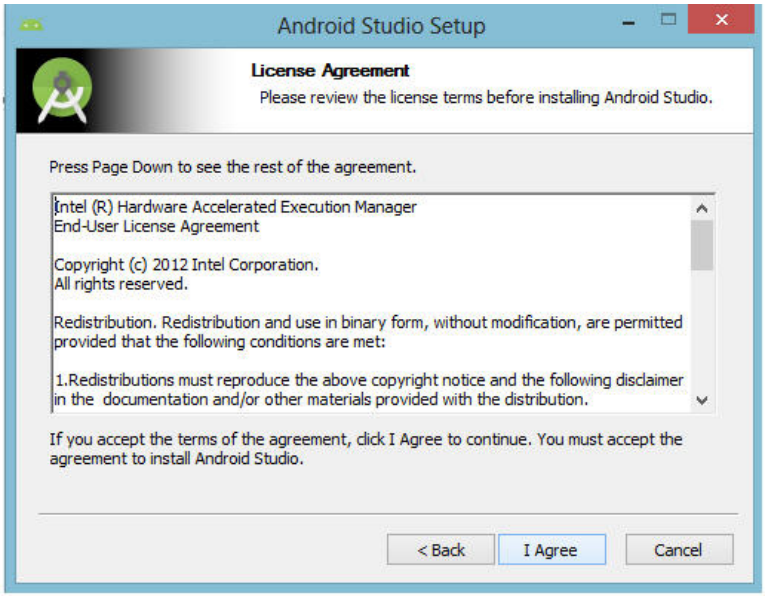


Figure 1.21: License Agreement

5. Select I Agree.
   - The installation path appears as shown in figure 1.22.

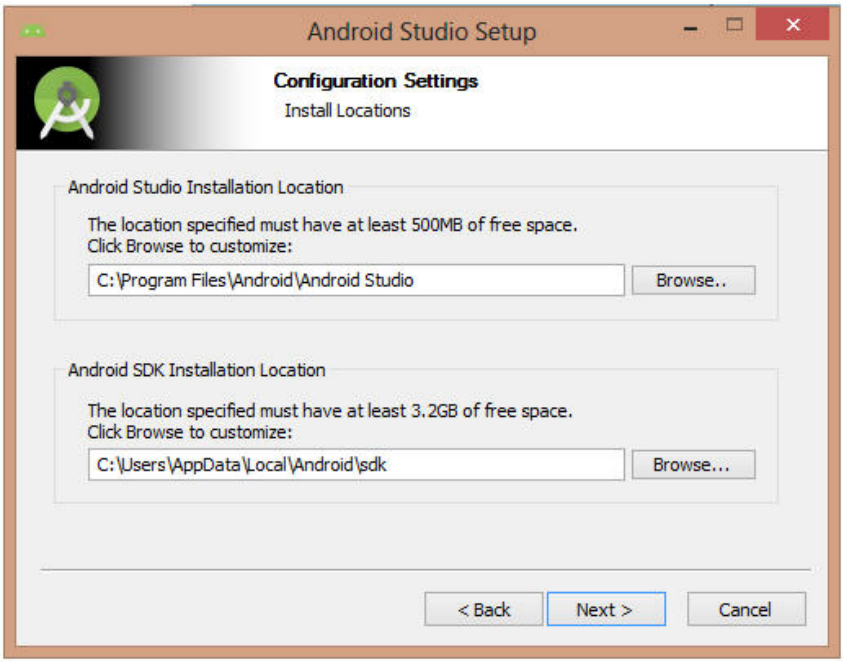


Figure 1.22: Android Configuration Settings

6. Select the path of installation and click Next. The AVD configuration page appears as shown in figure 1.23.

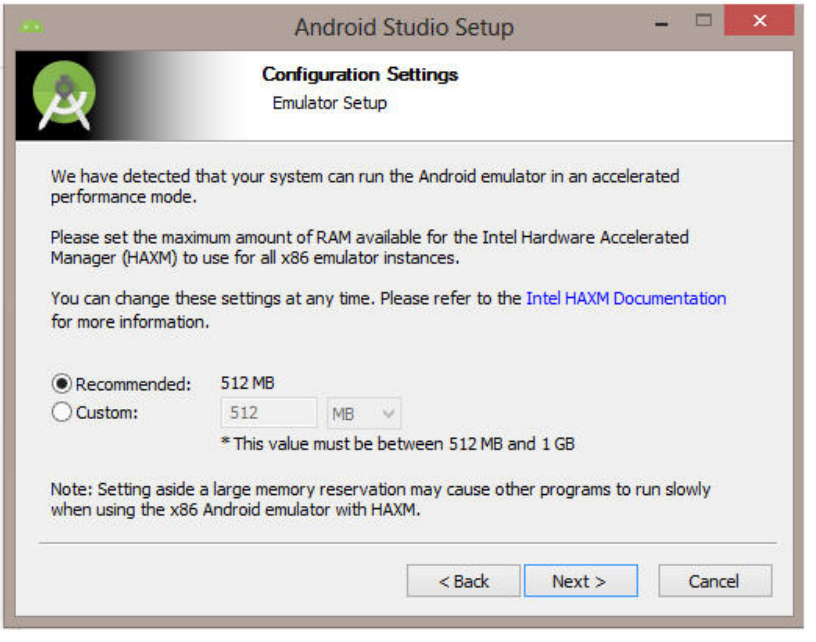


Figure 1.23: Android AVD Configuration

7. Select the default RAM size for the device and click Next.

The start menu selection screen appears as shown in figure 1.24.

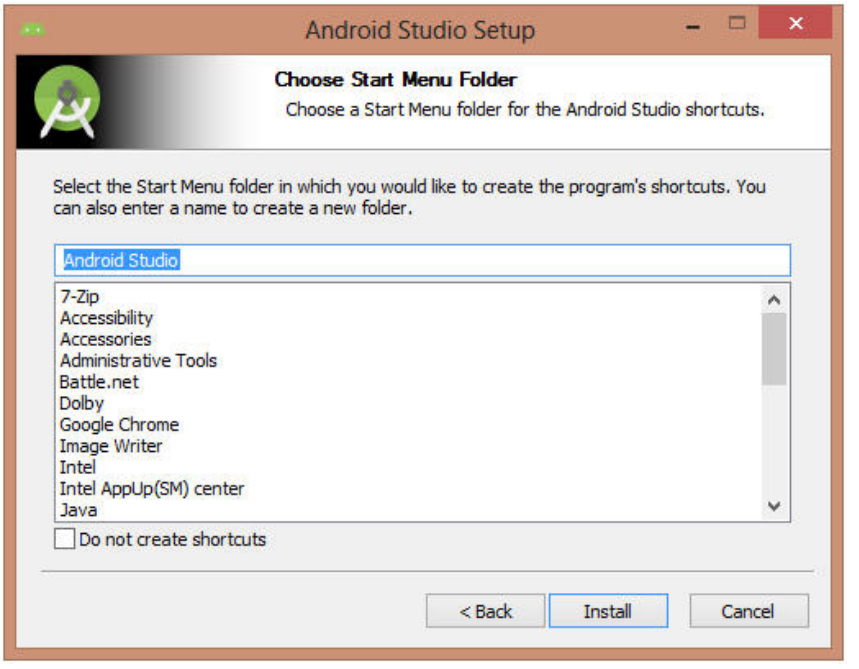


Figure 1.24: Android Setup - Start Menu Shortcut

8. Click Install. The installation begins as shown in figure 1.25.

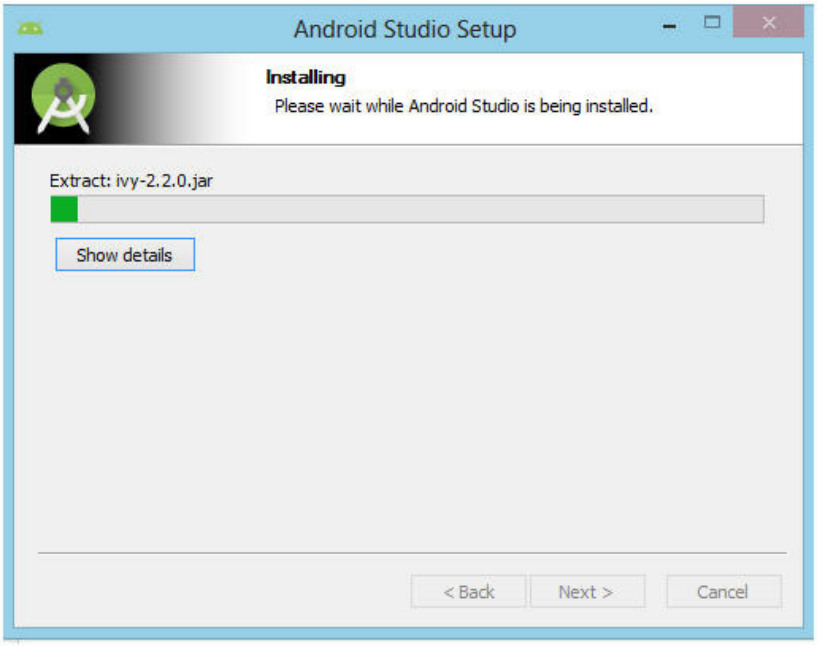


Figure 1.25: Android Setup – Installing Screen

9. Click Next after the installation completes as shown in figure 1.26.

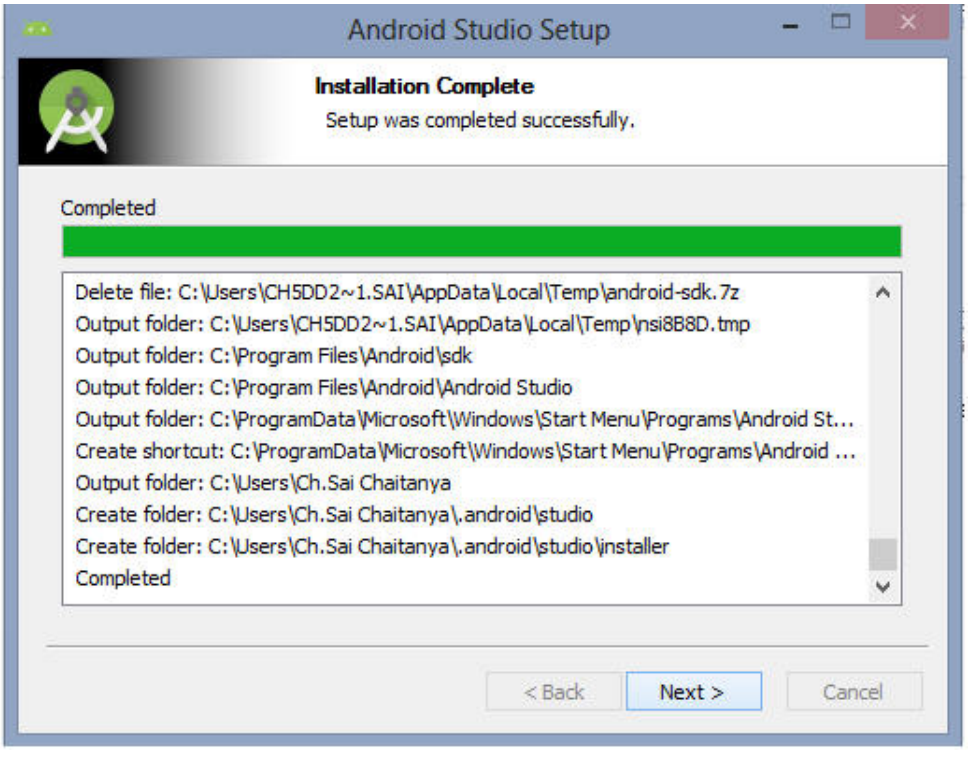


Figure 1.26: Android Setup - Installation Completed

The successfully installed screen appears as shown in figure 1.27.

Figure 1.27: Android Setup Completion

10. Click Finish.

1.6.4 Installing SDK from the Zip File

Installers are only supported for Windows OS and packages for other OS (and also Windows) are available in zip format. This section describes the steps involved in working with the zip files.

1. Go to the location where the zip file is downloaded.
2. Extract the contents of the zip file to a known location.
3. Make a note of the name and location of the SDK directory on your system.
4. Use the command line to use the SDK tools.
5. If the OS is Windows, the SDK manager can directly be launched and the GUI interface can be used instead of the command line.

1.7 Eclipse IDE

Eclipse IDE can also be used instead of Android Studio for developing Android applications. Eclipse 4.4 Luna is the latest addition to the Eclipse family. It is the first Eclipse version to fully support Java 8.

1.7.1 System Requirements

Table 1.3 lists the system requirements for the Eclipse IDE.

Requirement	Description
Operating Systems Supported and Requirements	☐ Windows XP (32-bit), Vista (32- or 64-bit), or Windows 7 (32- or 64-bit) ☐ Mac OS X 10.5.8 or later (x86 only) ☐ Linux
Development Environment Supported and Requirements	JDK 7 or higher - Java Runtime Environment (JRE) alone is not sufficient
Recommended Hardware Requirements	☐ Minimum 2 Gigabyte (GB) of Random Access Memory (RAM) ☐ Intel Core2Due or equal processor for x86 architecture ☐ 1 GB hard disk space

Table 1.3: Requirements for Eclipse IDE

1.7.2 Downloading Eclipse

Following are the steps for downloading the Eclipse IDE:

1. Check system requirements for Windows OS mentioned earlier.
2. Go to http://eclipse.org/downloads/ and click the 32-bit or the 64-bit download option for 'Eclipse IDE for Java Developers' or the 'Eclipse IDE for Java EE Developers' based on the target OS as shown in figure 1.28.

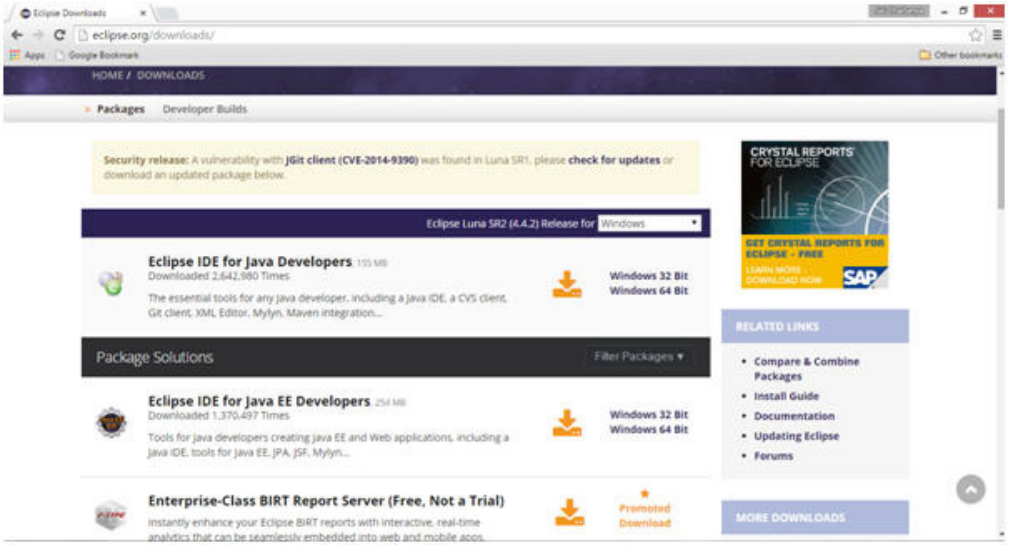


Figure 1.28: Eclipse Download Page

The server selection page is displayed.

3. Click the download server name displayed on the page as shown in figure 1.29.

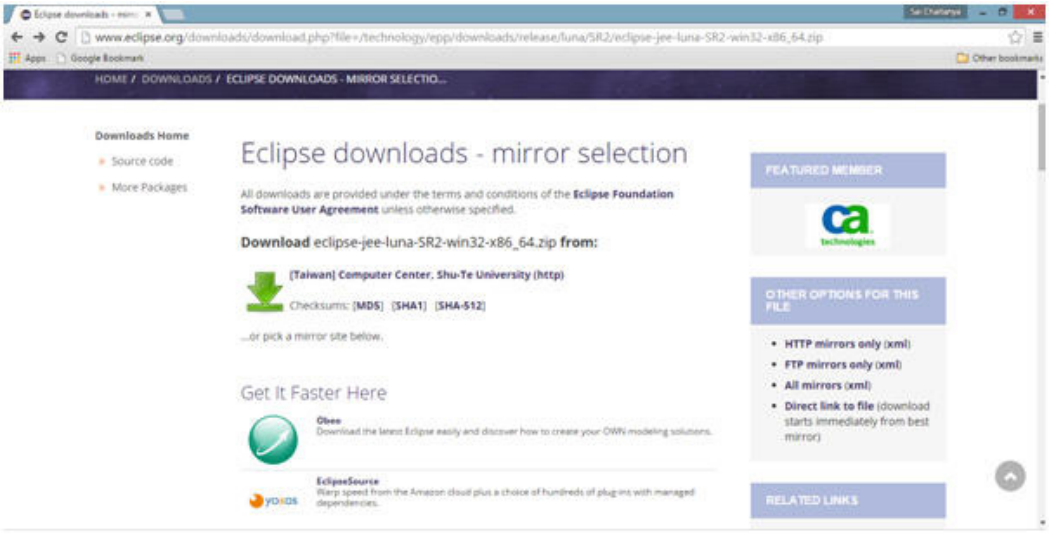


Figure 1.29: Server Selection Page

4. The Eclipse installation file begins to download. It is stored in the Downloads folder unless specified otherwise.

1.7.3 Installing Eclipse

To install Eclipse, perform the following steps:

1. Navigate to the location of the Eclipse setup downloaded earlier.
2. Right-click the file and Click Extract All as shown in figure 1.30.

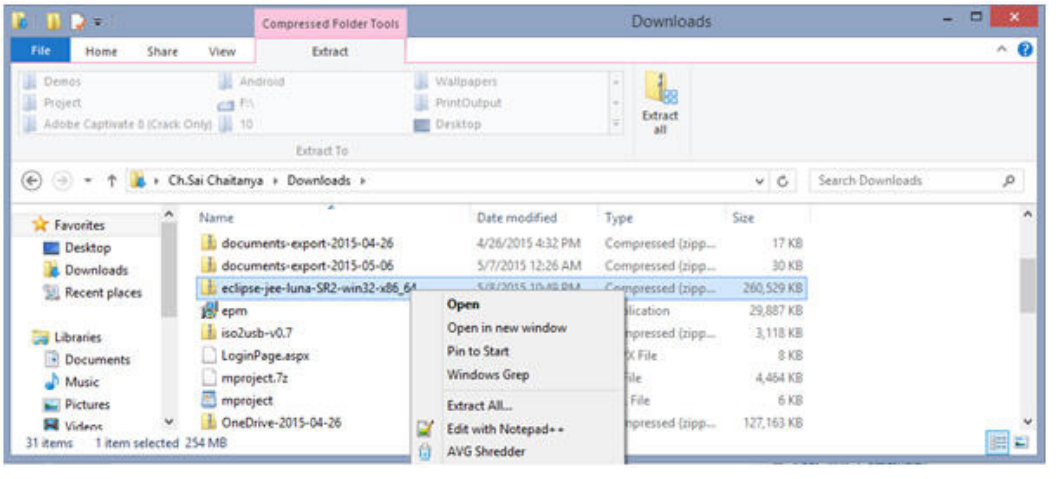


Figure 1.30: Extracting the Contents

3. Provide the path for extracting the files as shown in figure 1.31.

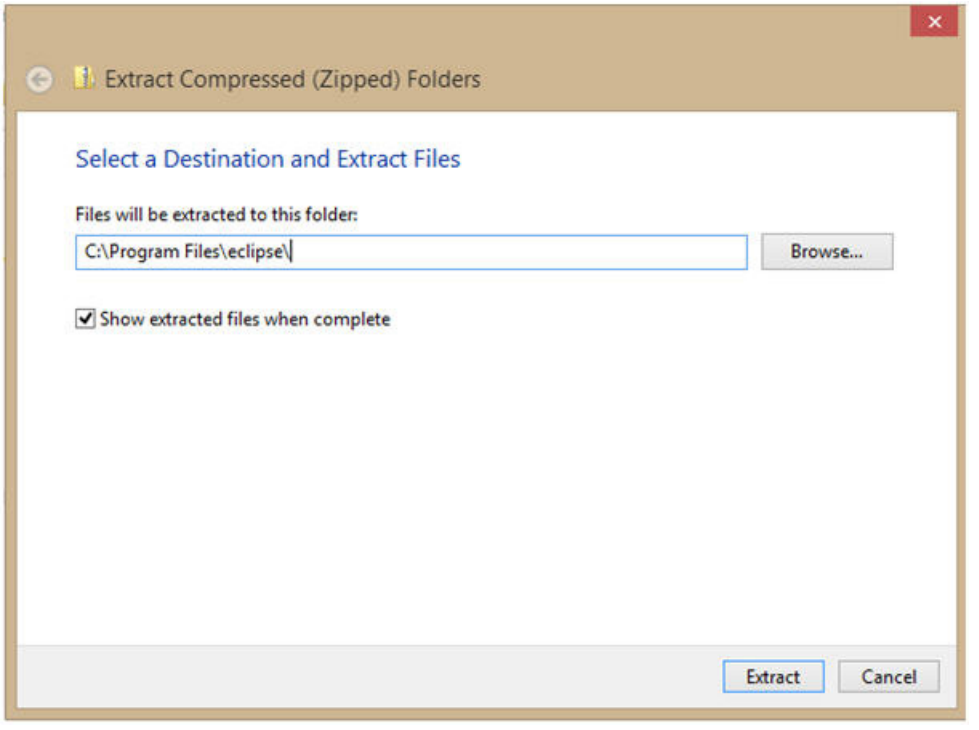


Figure 1.31: Selecting the Destination

4. Click Extract.

The contents are extracted in the mentioned folder as shown in figure 1.32.

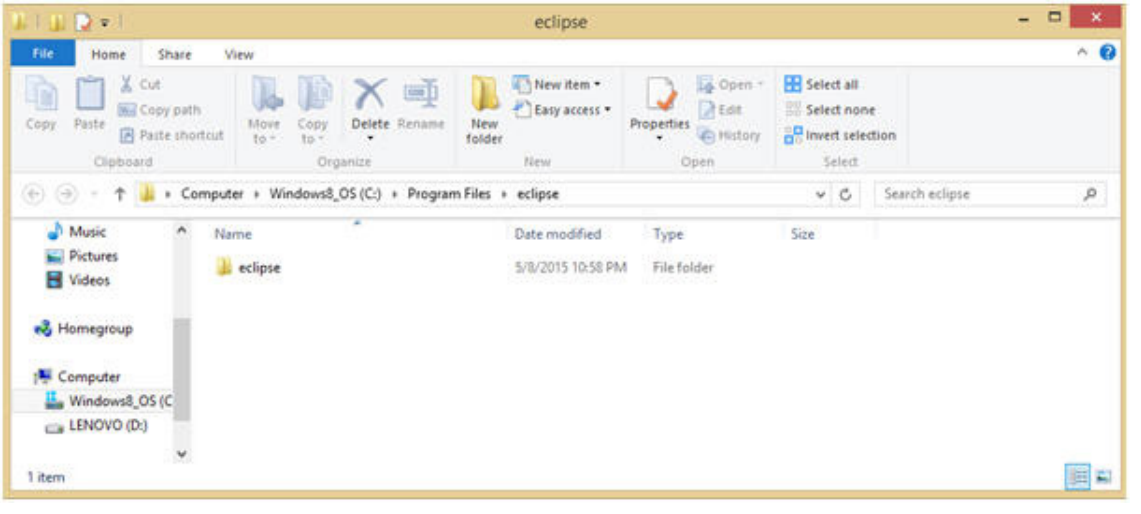


Figure 1.32: Extracted Eclipse Folder

1.7.4 Installing the Android Development Toolkit Plug-in

Android Development Toolkit (ADT) is Android's customized plugin for the Eclipse IDE. It offers a dynamic and all inclusive environment for developing Android apps. The ADT makes it easier to create new projects, construct a UI for the apps, debug, and export Application Packages (APK). The Eclipse ADT plugin is required if the developer has already installed the Eclipse IDE, but has not downloaded and installed the ADT Bundle.

1. Start Eclipse as shown in figure 1.33.

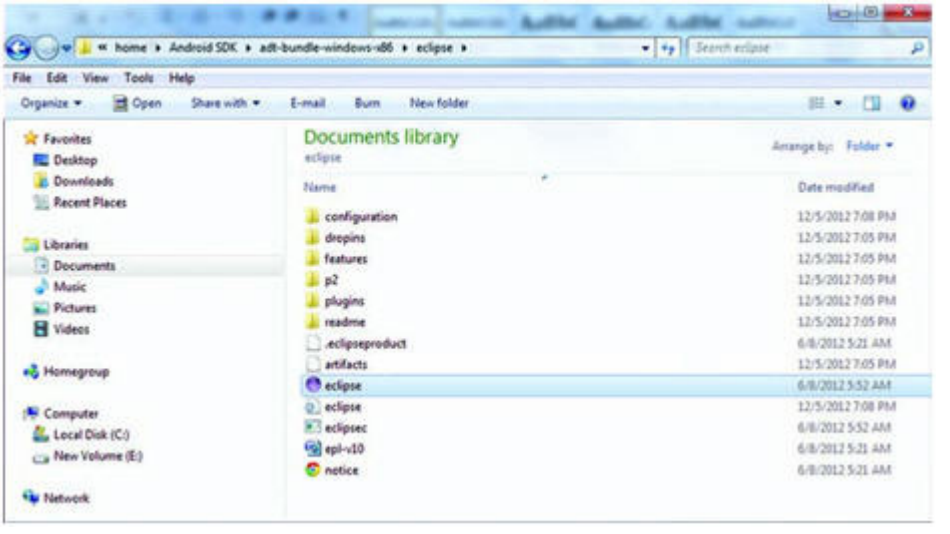


Figure 1.33: Eclipse Installation Folder

2. Click Help → Install New Software as shown in figure 1.34.

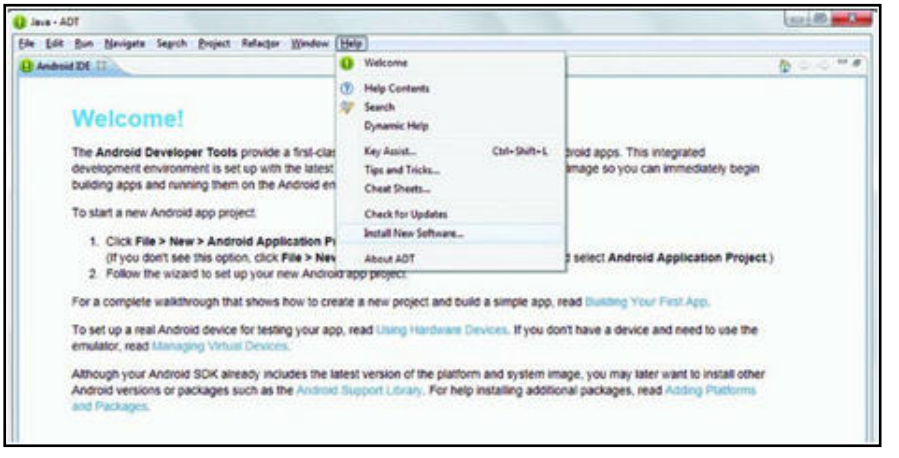


Figure 1.34: Help Menu

3. In the Available Software pane of the Install dialog box, click Add, next to the Work with list, as shown in figure 1.35.

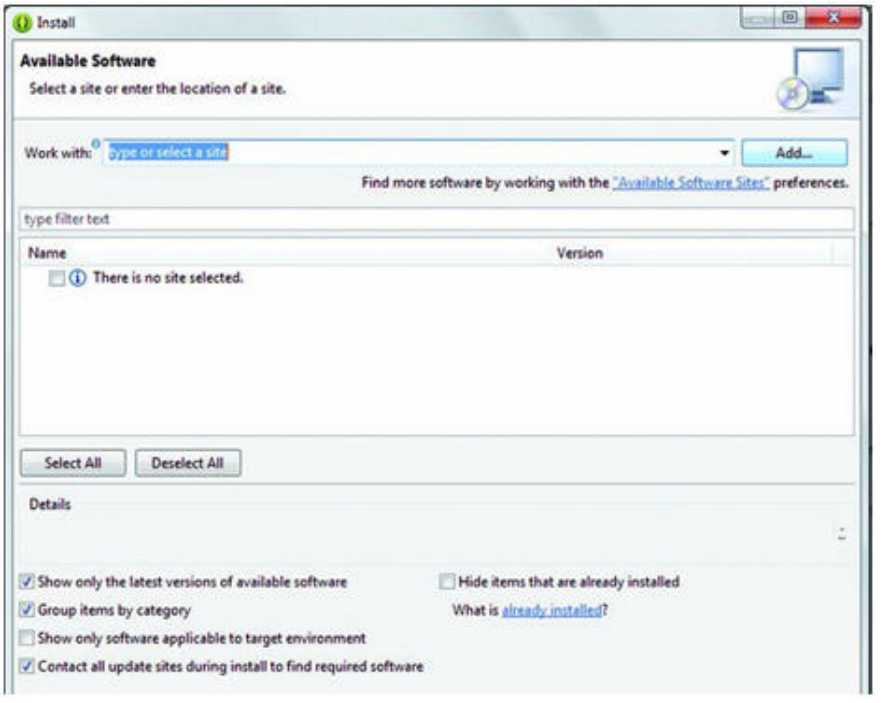


Figure 1.35: Install Available Software

4. In the Add Repository dialog box, in the Name box, type ADT Plugin as shown in figure 1.36.

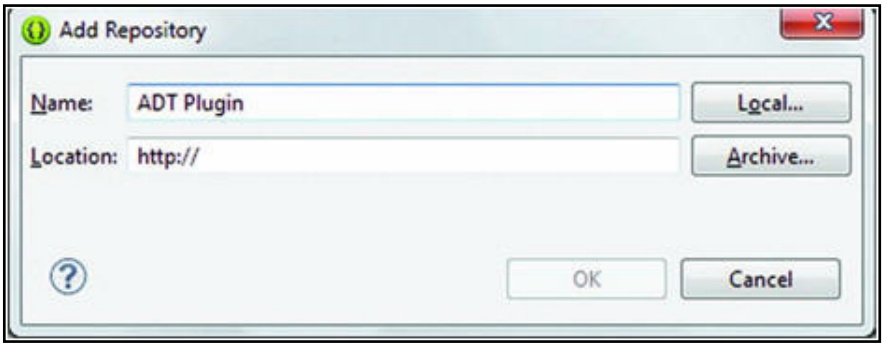


Figure 1.36: Add Repository

5. In the Location box, type the URL https://dl-ssl.google.com/android/eclipse/ and click OK as shown in figure 1.37.

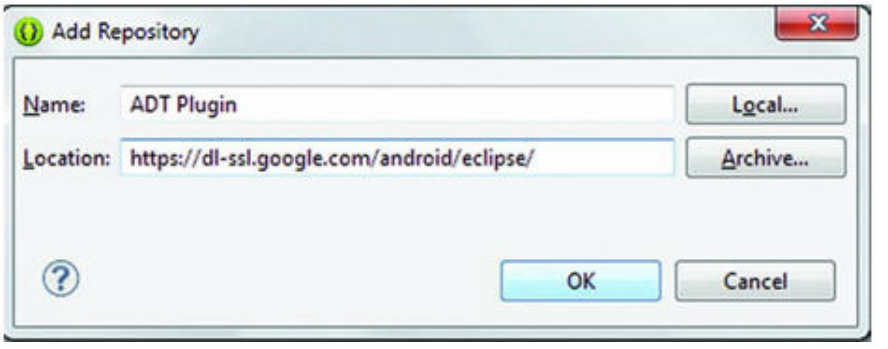


Figure 1.37: ADT Plug-in Details

6. In the Available Software pane of the Install dialog box, select Developer Tools as shown in figure 1.38.

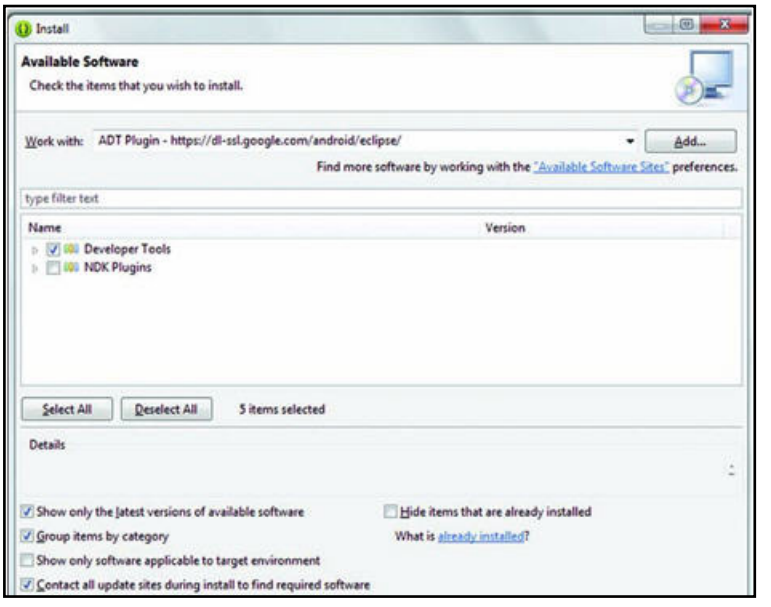


Figure 1.38: Available Software Dialog Box

7. In the Install Details pane of the Install dialog box, click Next as shown in figure 1.39.

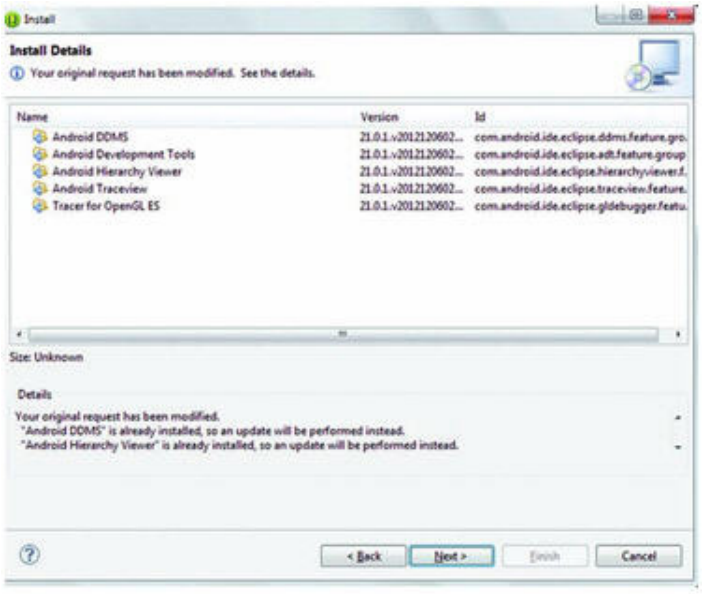


Figure 1.39: Install Details

8. Read and accept the license agreement and click Finish as shown in figure 1.40.

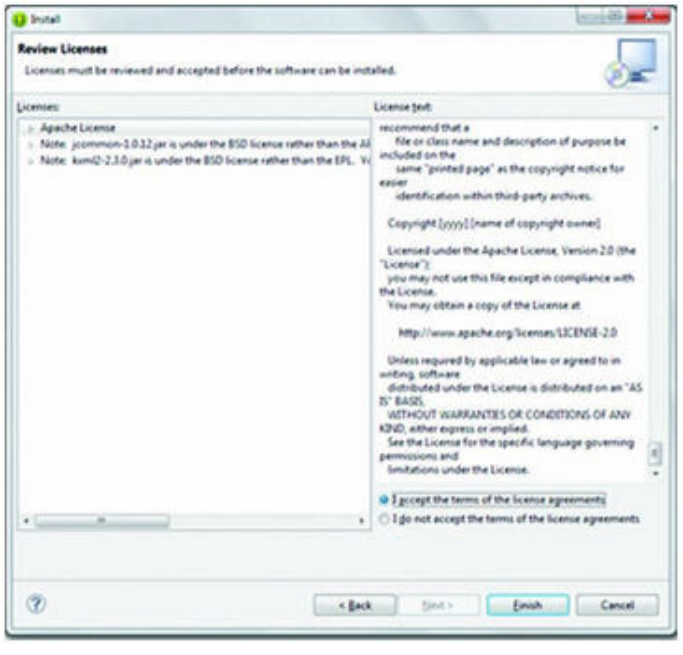


Figure 1.40: Review Licenses

9. After the installation, Eclipse restarts automatically. If Eclipse does not restart automatically, click File -> Restart as shown in figure 1.41.

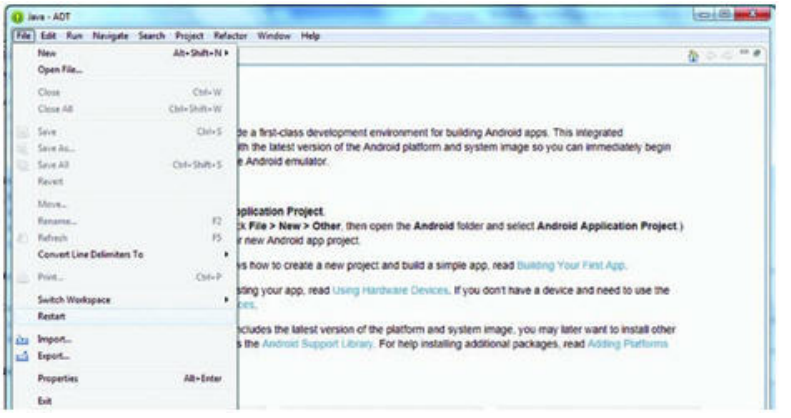


Figure 1.41: Restart Eclipse

10. In the Select a workspace pane in the Workspace Launcher dialog box browse and select the Android SDK directory where the SDK was downloaded and unpacked as shown in figure 1.42. Click OK.

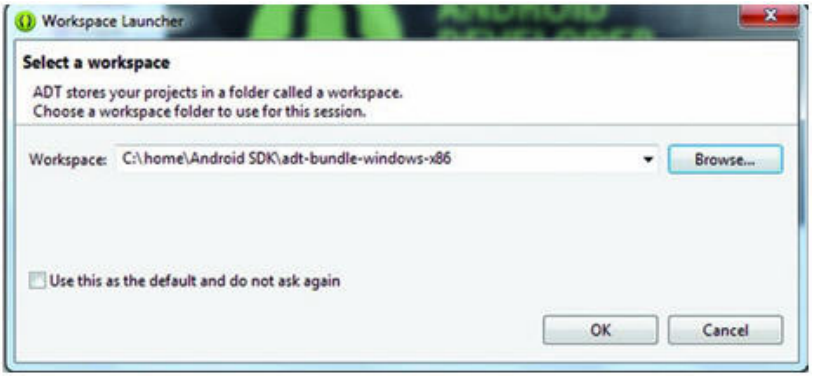


Figure 1.42: Select a Workspace

The installation is complete as shown in figure 1.43.

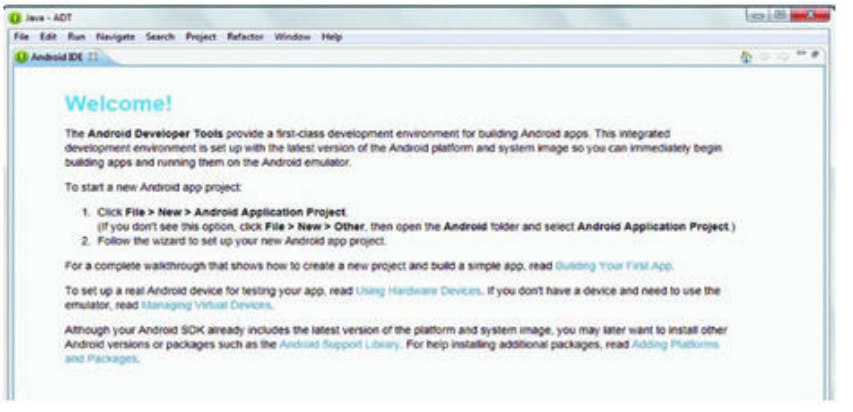


Figure 1.43: Installation Completed

Check Your Progress

1. Match the following Android versions with the corresponding nicknames.

	Version		Nickname
a.	2.0 and 2.1	1.	Jelly Bean
b.	3.0	2.	Kitkat
c.	4.1, 4.2, and 4.3	3.	Android Honeycomb
d.	4.4	4.	Android Éclair

(A)	a-2, b-3, c-4, d-1	(B)	a-3, b-1, c-4, d-2
(C)	a-4, b-3, c-1, d-2	(D)	a-4, b-1, c-2, d-3

2. Which of the following versions of Android was exclusive to Tablets?

(A)	Honeycomb	(B)	Kitkat
(C)	ICS	(D)	Froyo

3. What is the runtime used in the Android Jelly Bean?

(A)	JVM	(B)	ART
(C)	Dalvik VM	(D)	.Net CLR

4. Which of the following statements are true?

a.	Face unlock was introduced in Android Lollipop
b.	Android is a Mobile Only Operating System.
c.	Android Lollipop introduced a new UI philosophy.
d.	There are multiple IDEs to develop android applications.

(A)	a, d	(B)	b, d
(C)	b, c, d	(D)	c, d

5. Official support for Android wear devices was first introduced in which version of Android?

(A)	Jelly Bean	(B)	Kitkat
(C)	Lollipop	(D)	Ice Cream Sandwich

Answers

1.	C
2.	A
3.	C
4.	D
5.	B

Session

01

Introduction to Android

Summary

- An Operating System (OS) is a software program that enables communication and utilization of the hardware resources by the software programs.
- Android is an Operating System (OS) for mobile devices running on the Linux Kernel.
- Android Lollipop aims to provide major improvements in terms of performance and 64 bit CPU support along with a brand new UI scheme.
- The Android architecture consists four layers namely, Kernel, Libraries and Runtime, Application Framework, and Applications.
- Google Play (originally called Android Market) is Google's official market place for Android.
- The Linux Kernel is the base layer. This is the OS layer upon which the entire framework is built.
- The Android Studio Bundle comes with the official IDE for developing Android application recommended by Google, the Android Studio.

02 Getting Started with Android

Welcome to the Session, **Getting Started with Android.**

This session explains how developers can start using the Android platform with the Eclipse Integrated Development Environment (IDE) and recreating the project using the Android Studio IDE. It proceeds further to explain how to create an Android application, which includes creating an Android project, an Android Virtual Device (AVD), launch configurations, running and debugging the project, and understanding the project. It also describes the Android application fundamentals and its composition, communication components,and pre-existing components.

In this Session, you will learn to:

- Explain the process of creating an Android application
- Explain the fundamentals of Android application
- Explain the composition of Android applications framework
- Explain communication components
- Explain pre-existing components

Session

02

Getting Started with Android

2.1 Introduction

A new Android application developer needs to understand the basics of creating an Android application. In order to get started with Android, the developer should first learn to create a simple project.

An Android application can be created either using the Eclipse IDE, the Android Studio IDE, or any other IDE such as IntelliJ. Before executing the application, the developer needs to create an AVD or use a real device. The AVD represents an emulator or model of devices, such as the tablet or Smartphone. Creation of the AVD will generate a virtual device that the developer can use to test the application.

2.2 Creating an Android Application

Android applications are created using Java syntax. A majority of the features present in core Java Application Programming Interfaces (APIs) are found in Android's core libraries. This session describes the different components of Android applications which are bound together by the Android manifest file.

Creating an Android application first involves understanding an Android project.

2.2.1 What is an Android Project?

An Android project essentially consists of the entire source code for an Android application. The project helps to create an .apk file for installation of the application on any device. When the Android Software Development Kit (SDK) has been downloaded and installed, the process of creating a new Android project becomes simple and easy.

An Android project can be created in two ways:

- **With the Eclipse IDE** – For this, the Android Software Development Kit (SDK) should have been downloaded and installed along with Eclipse ADT plugin.
- **With the Android Studio IDE** – For this the Android Studio Bundle must be installed.

2.2.2 Creating a HelloWorld Android Project Using Eclipse

In order to create a project using Eclipse, first install the Eclipse IDE and ADT plugin.

Steps for creating a HelloWorld Android project using Eclipse IDE in the Windows OS are as follows:

1. Access the folder where the Eclipse has been installed/extracted and double-click the eclipse executable as shown in figure 2.1.

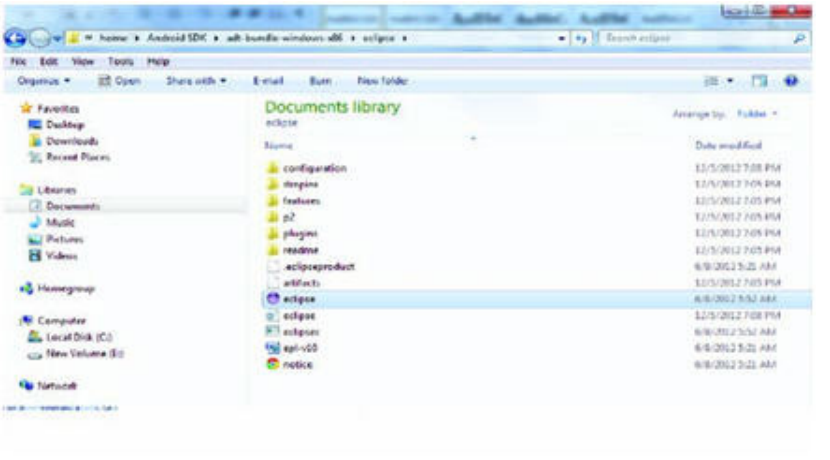


Figure 2.1: Eclipse Folder

2. In the Workspace Launcher dialog box, click Browse and select a folder where the project needs to be stored as shown in figure 2.2.

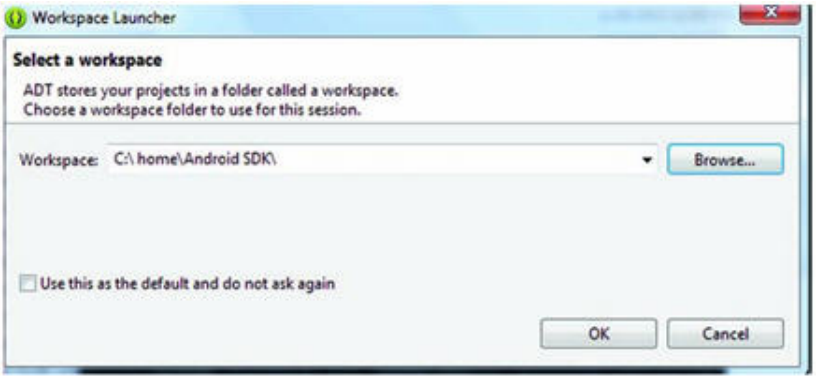


Figure 2.2: Workspace Selector

3. Click OK. Within Eclipse IDE, the Home Screen is displayed as shown in figure 2.3.

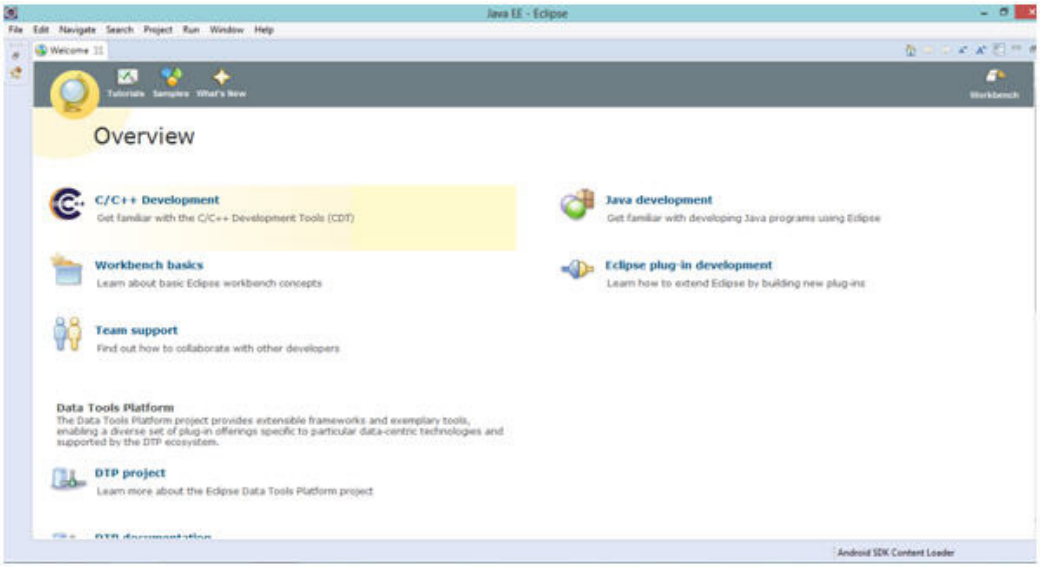


Figure 2.3: Eclipse Home Screen

4. Click File → New → Project as shown in figure 2.4.

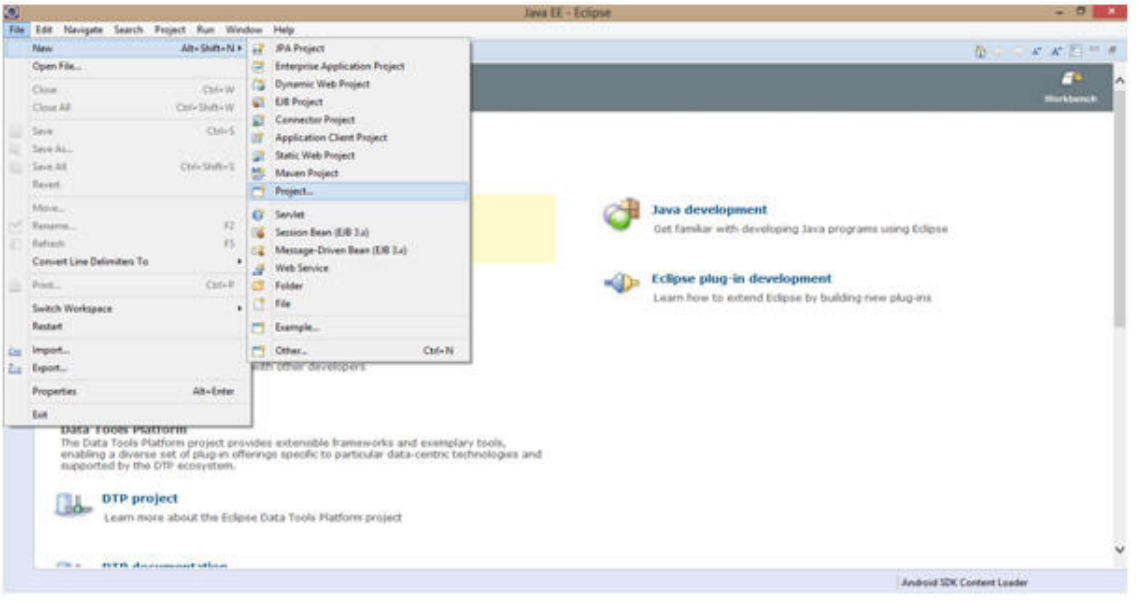


Figure 2.4: New Android Application Project

5. From the new Wizard that appears, select Android → Android Application and click Next as shown in figure 2.5.

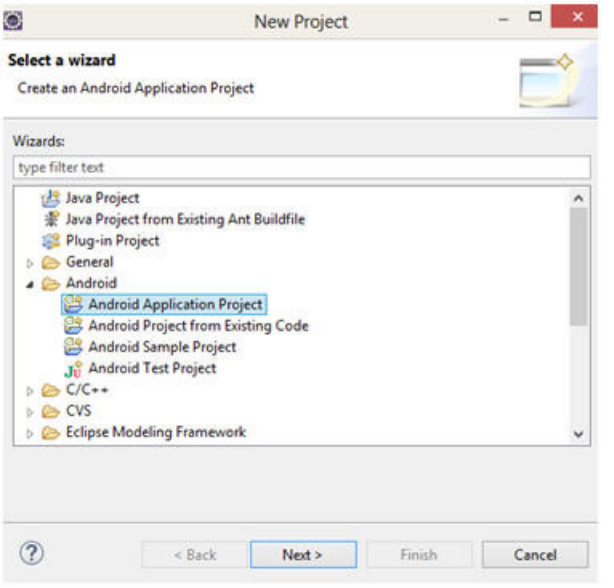


Figure 2.5: New Project Dialog Box

Note - Later, Android Application can be directly selected from File → New menu itself.

The New Android Application dialog box is displayed.

6. In the Application Name box, type HelloWorld as shown in figure 2.6.

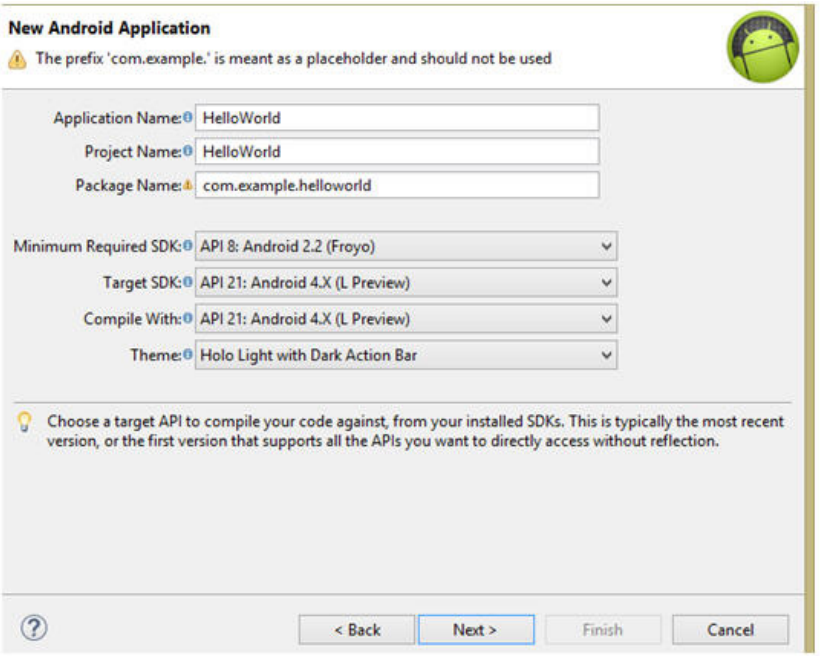


Figure 2.6: New Application Wizard

The application name indicates the name of the application that the users will view. When the application name is typed, the Project Name and Package Name boxes are automatically filled.

The project name is the directory in which the project files are saved and can be viewed in Eclipse.

The package name for the application has to be a unique name and cannot be the same as another package installed in Android. To ensure that the name is unique, usually the reversed domain name of the organization or other entity is provided.

The Minimum Required SDK, Target SDK, Compile With, and Theme boxes are filled by default.

The Minimum Required SDK refers to the lowest Android version that the application (app) can support. The Target SDK refers to the highest Android version.

Note - Android 5.0 shows up as 4.X L preview because ADT is abandoned.

The Compile With option is the version of the platform used to create the app. This is set by default to the highest Android version. Theme is the style of the User Interface (UI) for the app being created.

7. Click Next.
8. Click Next again in the Configure Project pane of the New Android Application dialog box as shown in figure 2.7.

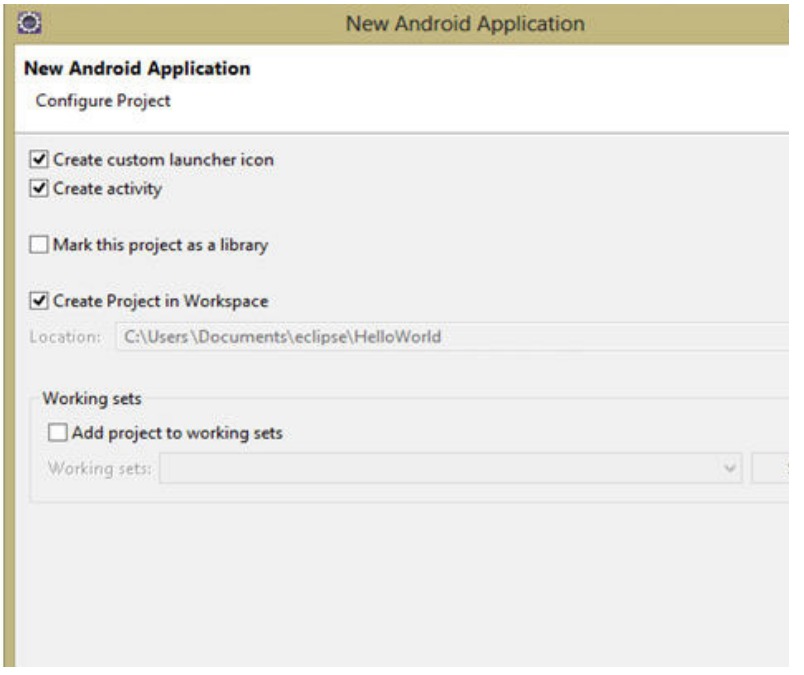


Figure 2.7: Project Configuration

9. In the Configure Launcher Icon screen that is displayed, customize the appearance of the launcher icon as shown in figure 2.8.

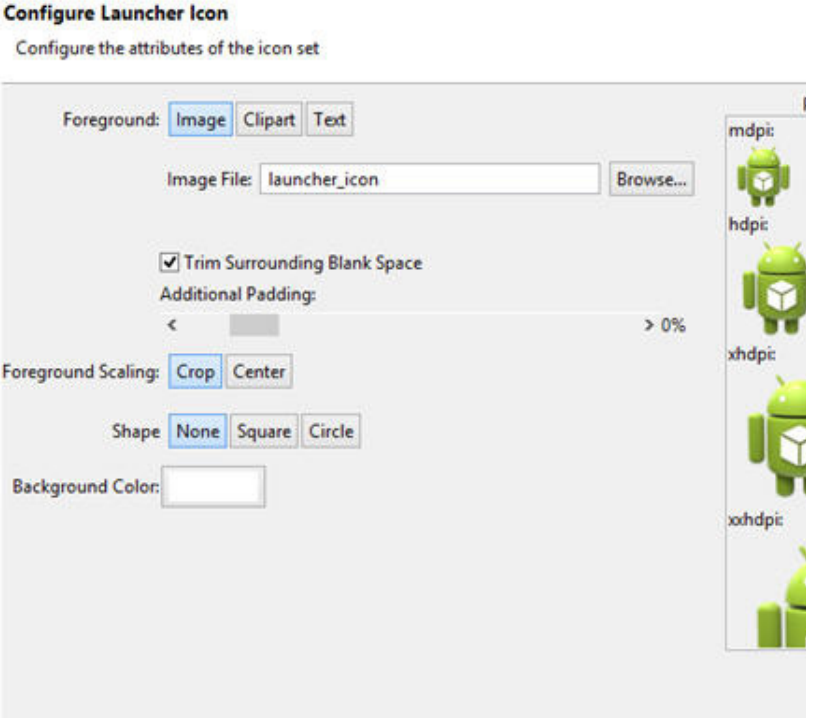


Figure 2.8: Launcher Configuration

10. Click Next. The Create Activity screen is displayed as shown in figure 2.9.

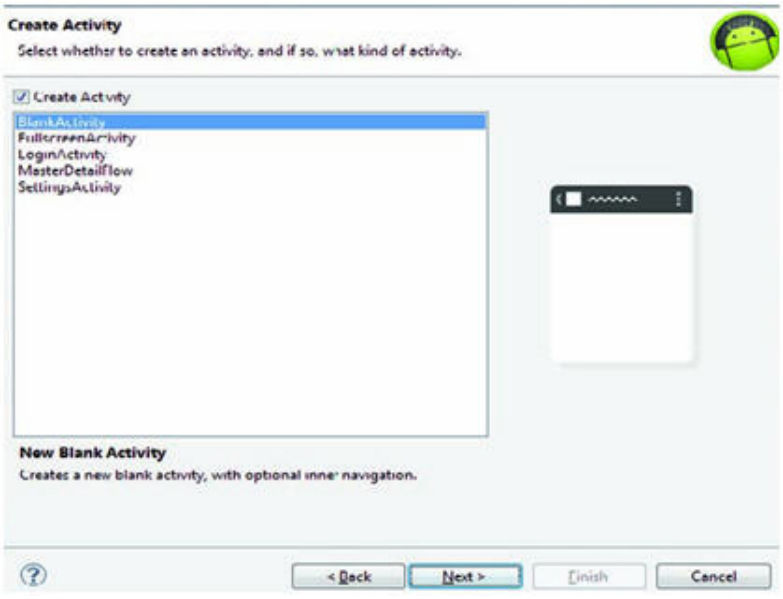


Figure 2.9: Create Activity

This dialog box provides the template for building the app. Blank Activity is selected by default.

11. Click Next in the Create Activity screen.

The New Blank Activity screen is displayed as shown in figure 2.10.

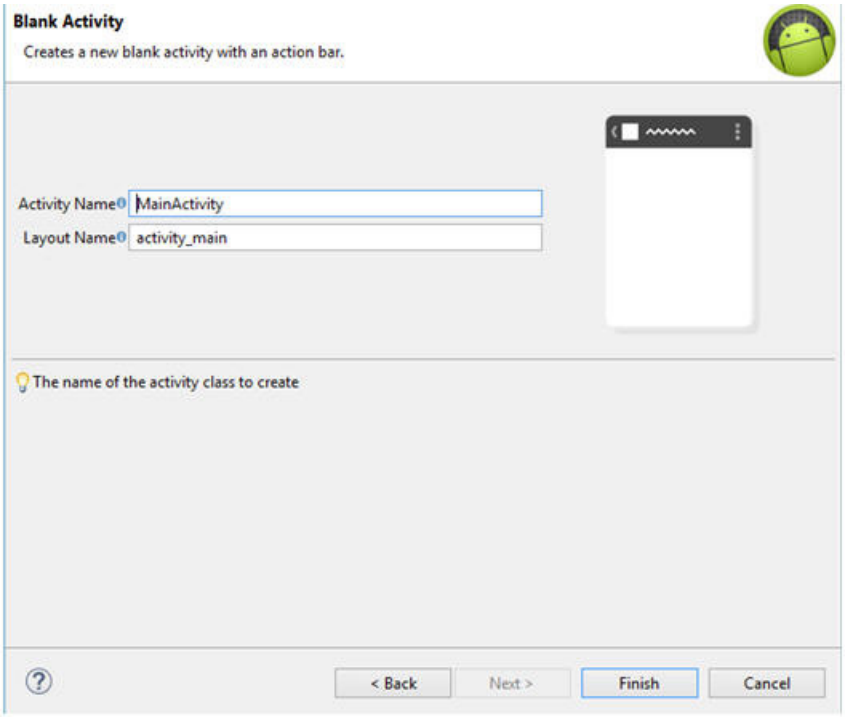


Figure 2.10: New Blank Activity

12. Click Finish.

The Eclipse IDE main window is displayed. The directory structure for the newly created HelloWorld project can be seen in the Package Explorer pane of Eclipse IDE as shown in figure 2.11.

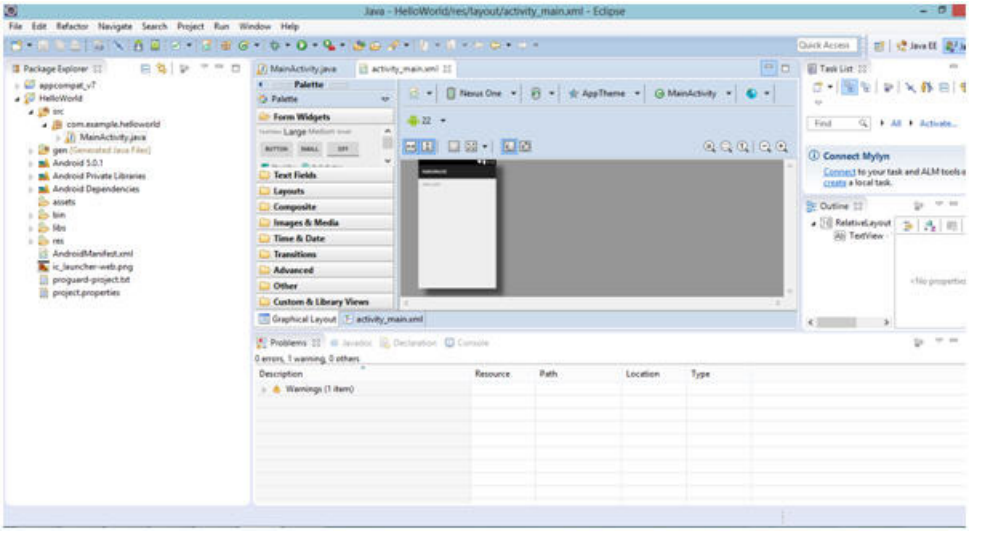


Figure 2.11: Eclipse IDE Main Window

2.2.3 Creating Android Virtual Device (AVD)

The AVD is a configuration that enables the developer to sample various devices using an emulator as shown in figure 2.12.

Figure 2.12: Android Emulator

The emulator functions exactly similar to a mobile device. The AVD identifies hardware and software options for the model device, which the Android emulator duplicates. The emulator is based on AVD configurations which the developer can also specify. AVD consists of:

- Hardware features for the device, such as keyboard, camera, processor, and storage.
- Version of Android OS to be used on the emulator.
- Software features for the device, such as games, social networking apps, e-mail, Internet, Microsoft Outlook, multimedia, and so on.
- The look of the screen, size, specific storage space for data, settings, and applications.

To use an emulator, the AVD configurations need to be created for the model device. Many AVDs can be created for different types of devices. Each AVD is independent of the others. Once the AVD configuration is created, the application can be run on single or multiple emulators.

- **Steps to create an Emulator**

  1. Click Window → Android Virtual Device Manager to start the AVD Manager as shown in figure 2.13.

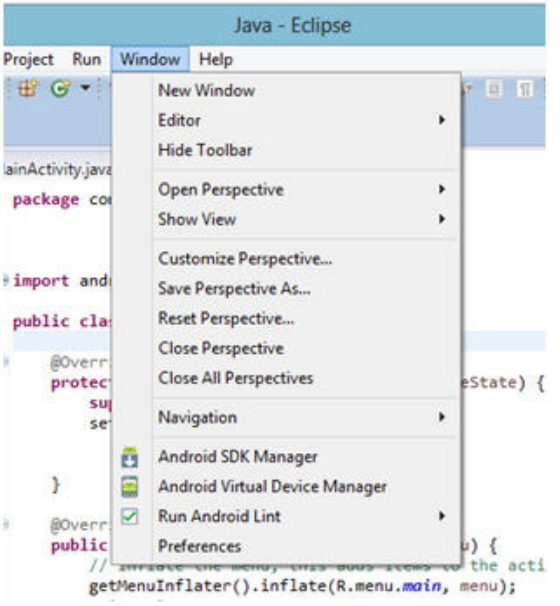


Figure 2.13: Start AVD Manager

Or navigate to the Android SDK installation directory and start the AVD Manager as shown in figure 2.14.

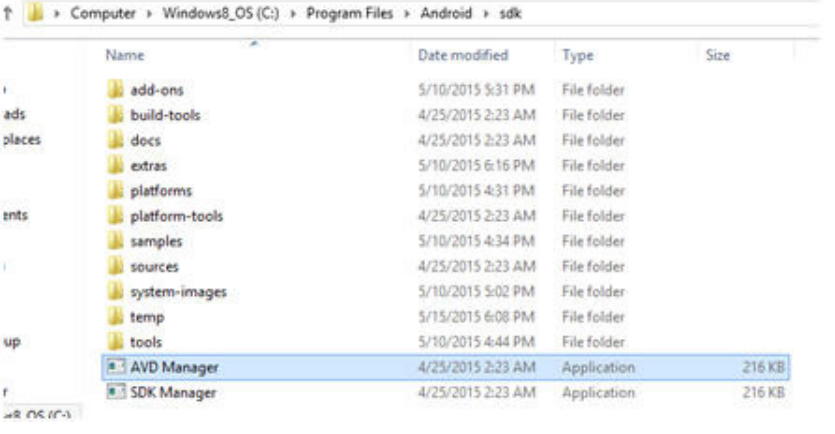


Figure 2.14: Android SDK Installation Directory

The Android Virtual Device Manager dialog box is displayed as shown in figure 2.15.

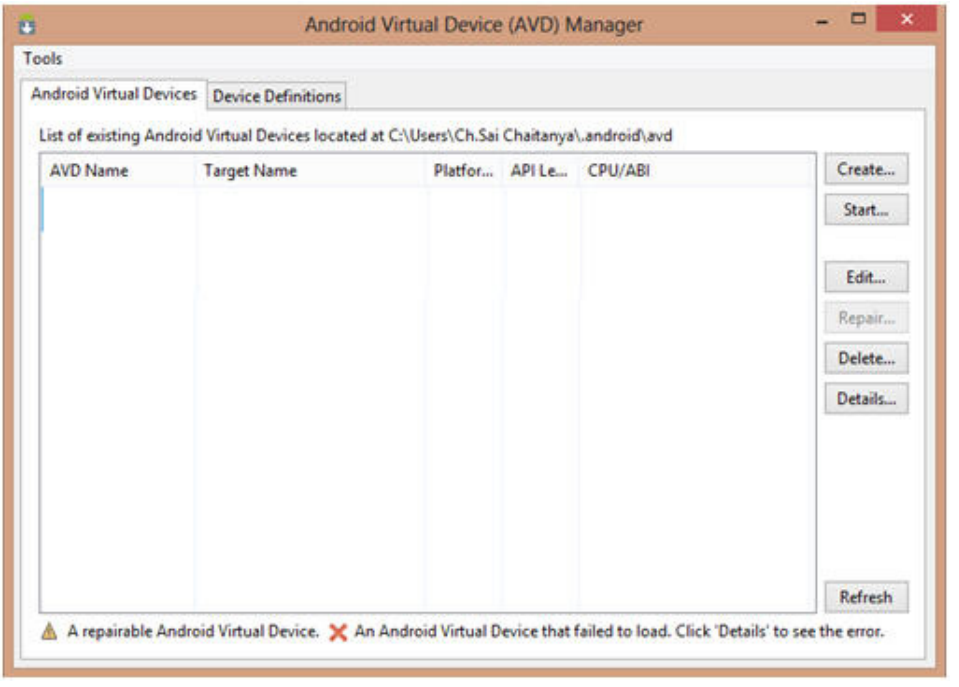


Figure 2.15: AVD Manager

2. Click Create to create a new AVD. The Create new Android Virtual Device (AVD) dialog box is displayed as shown in figure 2.16.

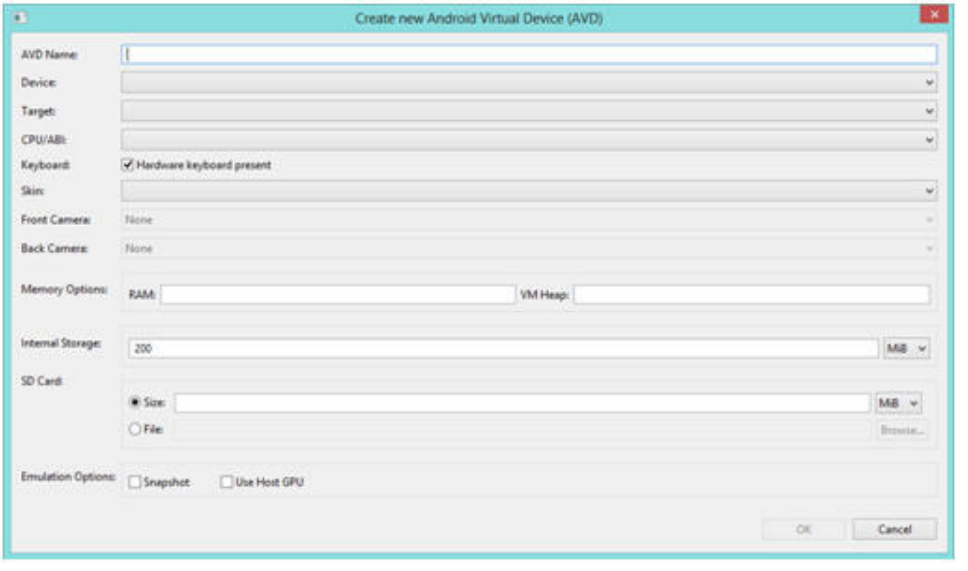


Figure 2.16: New AVD Screen

3. In the Create new Android Virtual Device dialog box, provide the details for the AVD as shown in figure 2.17.

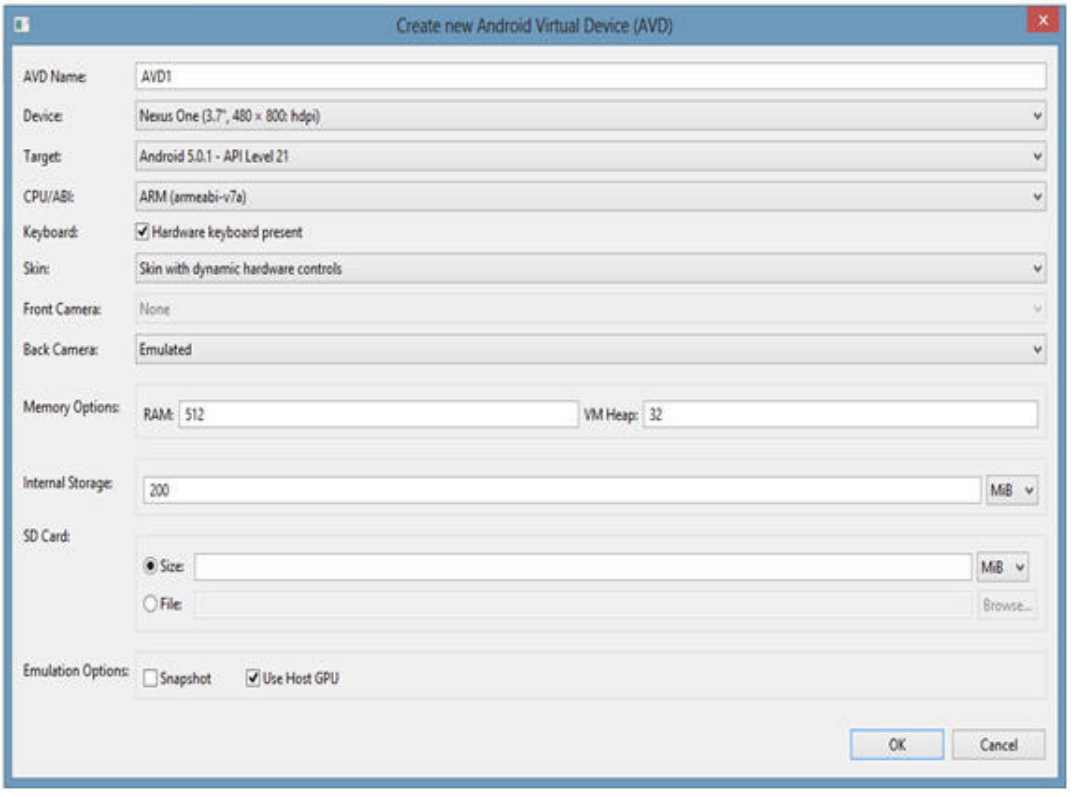


Figure 2.17: Create New AVD Details

Various fields are described as follows:

a. In the AVD Name box, type a name that is easily identifiable.

b. In the Device list, select the device that is being emulated.

c. When the device is selected, the Target is selected by default.

d. Complete other details as required, such as, Keyboard, Skin, Front and Back Camera, SD Card, and Emulation Options. The Memory Options field gets filled by default.

4. Click OK after all the requirements are complete.

The Android Virtual Device Manager dialog box appears displaying the newly created AVD as shown in figure 2.18.

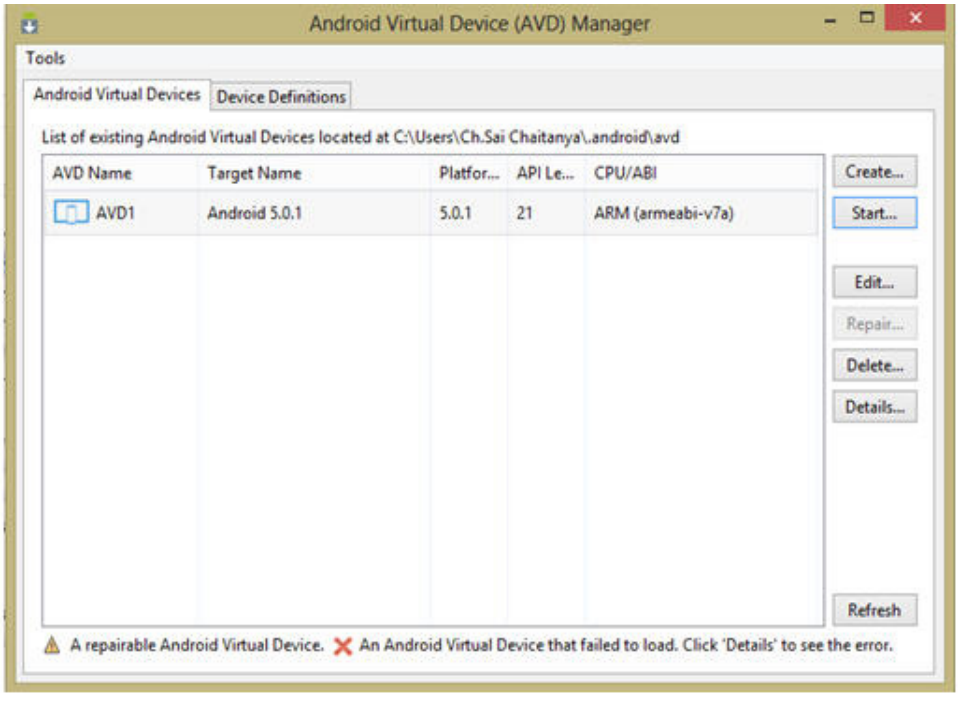


Figure 2.18: AVD Manager with New AVD

5. Select AVD1 and click Start to start the emulator.
6. Click Launch in the Launch Options dialog box as shown in figure 2.19.

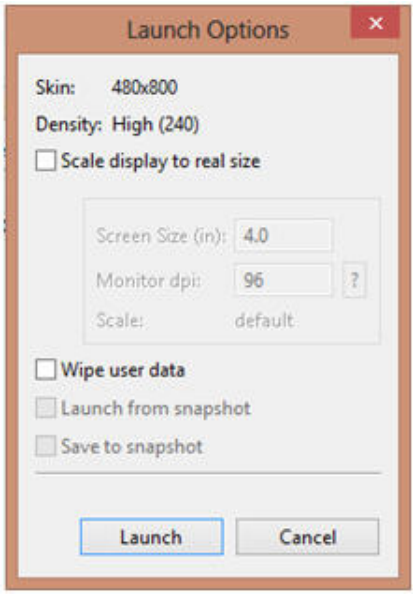


Figure 2.19: Launch Options

The emulator is launched as shown in figure 2.20.

Figure 2.20: Android Emulator

Note - The emulator may take several minutes to load.

2.2.4 Creating Launch Configurations

Although the emulator has been launched, a Run configuration is required to be set up for the application to run correctly. The Run configuration identifies the project that has been created and is required to be executed, the main activity that needs to be started and the emulator that is going to be used to run the application. When there is an actual device, the Run configuration identifies the device. A launch configuration can also include a debug configuration, which enables debugging the application as and when it is being run on a device.

☐ Steps to Create Run Configurations

1. In Eclipse, click Run → Run Configurations as shown in figure 2.21.

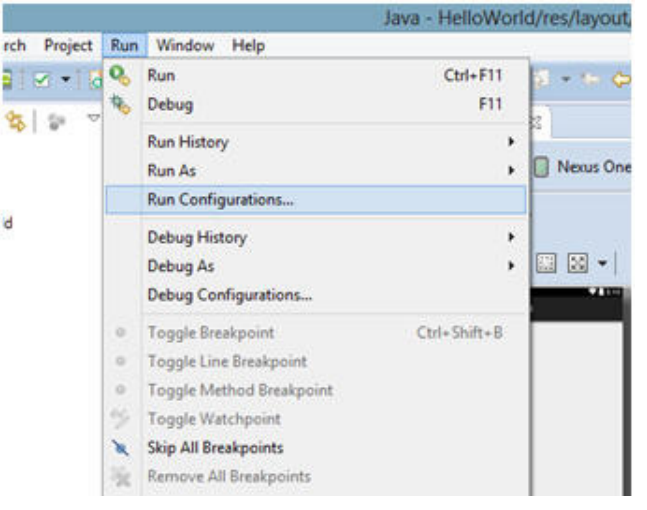


Figure 2.21: Selecting Run Configurations

2. In the Create, manage, and run configurations dialog box, select Android Application, and click New launch configuration icon as shown in figure 2.22.

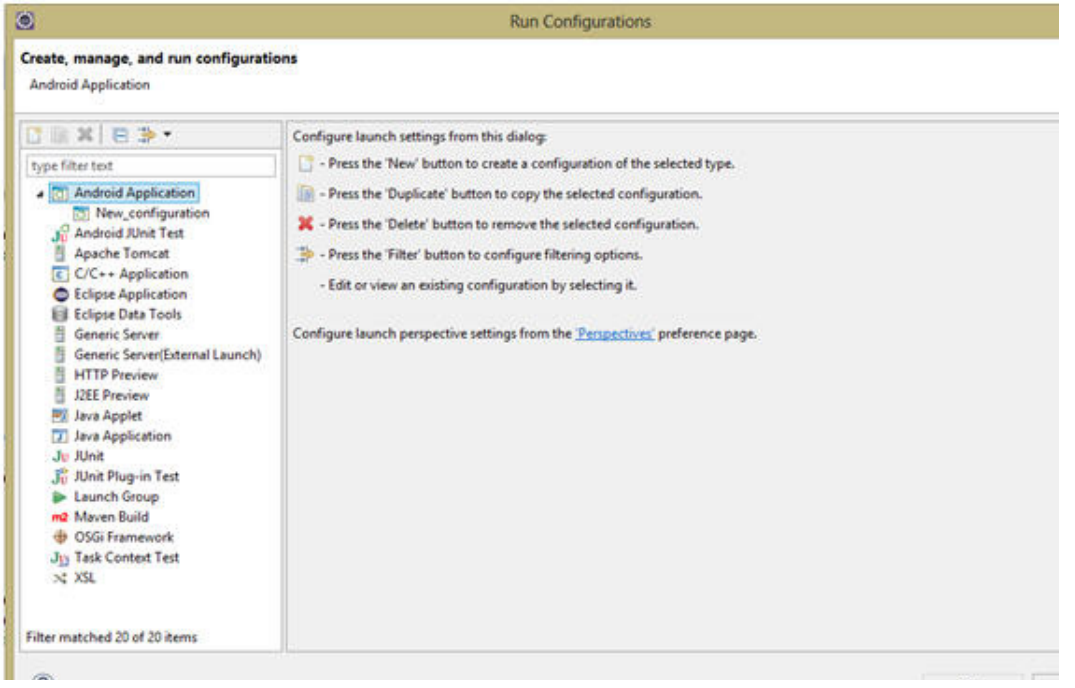


Figure 2.22: New Launch Configurations

3. In the Name box of the Create, manage, and run configurations dialog box, type the name of the new configuration as HelloWorld.
4. Click Browse to display the Project Selection dialog box as shown in figure 2.23.

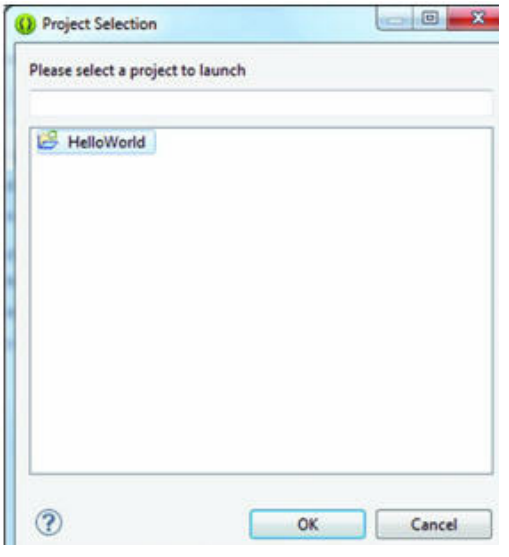


Figure 2.23: Project Selection Dialog Box

5. Select HelloWorld and click OK.

   The Create, manage, and run configurations dialog box now displays the project named, HelloWorld.

6. Click the Target tab and select the newly created AVD, as shown in figure 2.24.

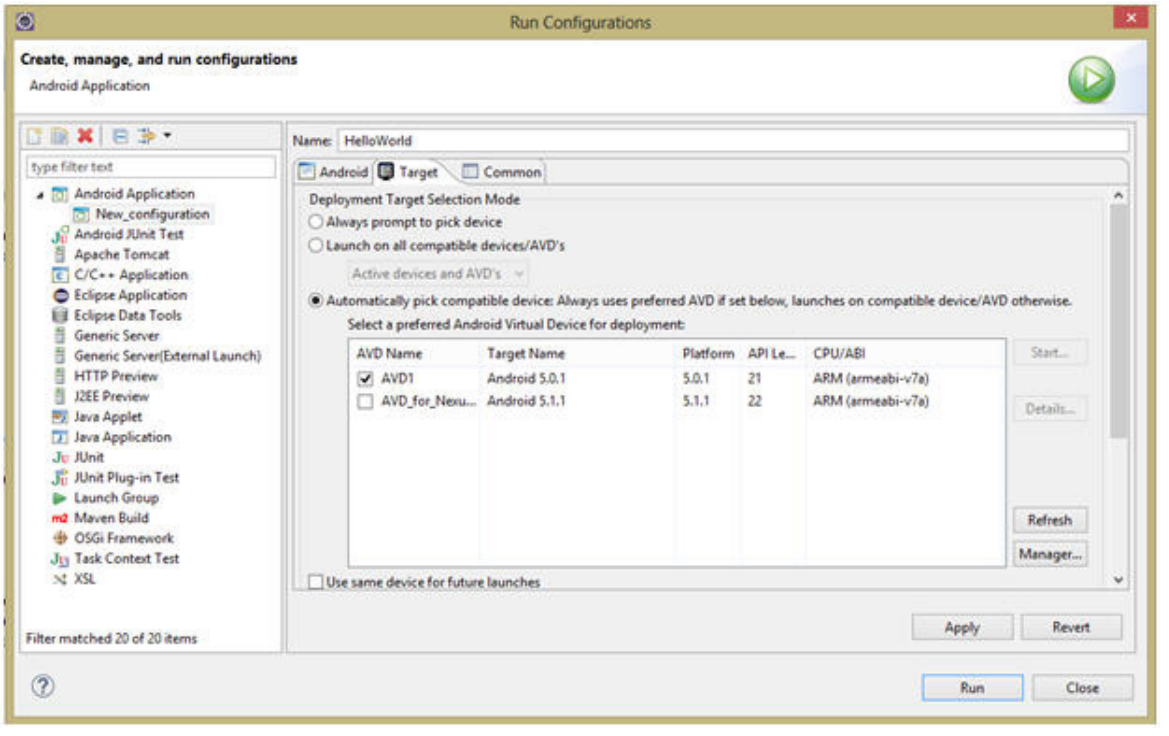


Figure 2.24: Target Tab Options

7. Click Apply.

   The launch configuration has been created.

8. Click Close to exit the Create, manage, and run configurations dialog box.

☐ Steps to Create Debug Configurations

The steps for creating a debug configuration are the same as for creating a run configuration. However, when a run configuration has been created for an activity, few steps need to be followed for creating a debug configuration for the same activity.

1. In Eclipse, click Run → Debug Configurations.
2. In the Debug Configurations dialog box, in the Android Application list, select HelloWorld as shown in figure 2.25.

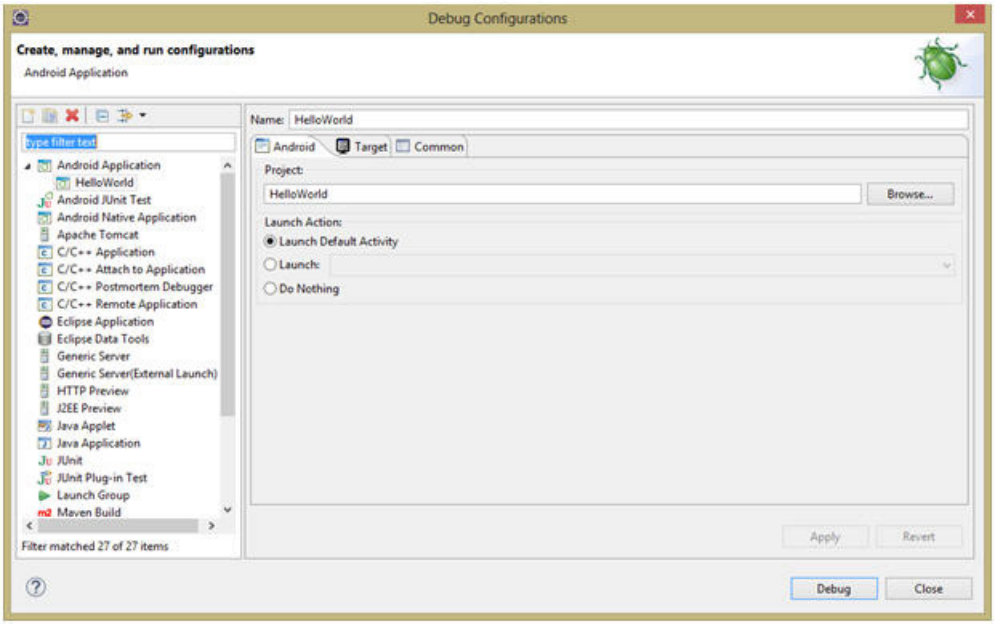


Figure 2.25: Debug Configurations Dialog Box

In the Android tab, the project name for the selected configuration is displayed by default.

3. Click the Target tab. The AVD for the configuration is selected by default as shown in figure 2.26.

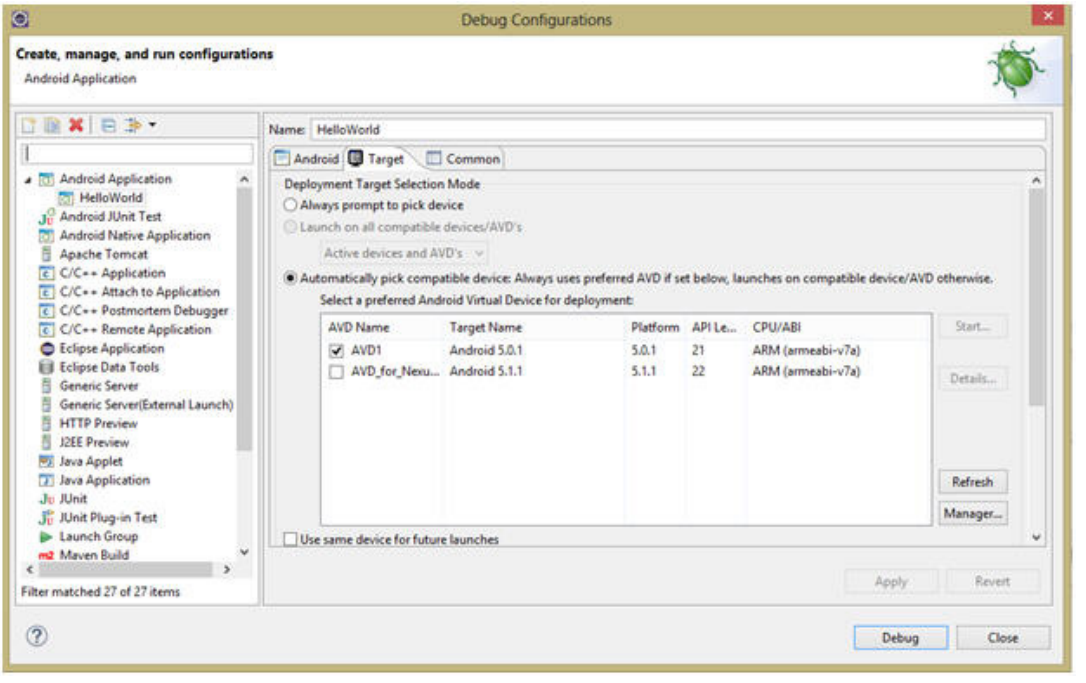


Figure 2.26: Target Tab

4. Click Debug.

   The launch configuration has been created.

5. Click Run → Run.

   The emulator is displayed as shown in figure 2.27.

Figure 2.27: Emulator

2.2.5 Running and Debugging the HelloWorld Project

There are two ways to run and debug the HelloWorld project; using an Android device or through the emulator.

Note - In order to run the app, ensure that the Android manifest file for the project is present. The file is very important for running all Android applications. In the file, verify that the highest version of Android is included. To do this, alter the element in the manifest file.

Ensure that the required directories and files for running the project are present. When the application is built, an .apk file is created, which contains the binary code for the application. The .apk file consists of all the files and resources required to run the application. The file includes ART byte code files, resource.arsc file that are compiled and uncompiled resources required by the application, and a binary form of the AndroidManifest.xml file.

In Eclipse, the .apk file is automatically created in the project's bin folder. To run an application using a device or an emulator, the application needs to be signed in the debug or release mode. This is because while building the application, a debug key is created with an identifiable password so that the developer need not keep providing the password while building the

application. For better security, however, the developer needs to sign the application using the release mode while releasing the application. This requires designating a private key.

- **Steps to Run the HelloWorld Project on the Emulator (Windows OS)**

1. In Eclipse, click Run → Run.

   The emulator is displayed.

   When the emulator first loads, the home screen is locked as shown in figure 2.28.

Figure 2.28: Emulator with Home Screen Locked

2. Click and drag the lock icon upwards to unlock the home screen.
3. On the home screen, click the circle and click OK as shown in figure 2.29.

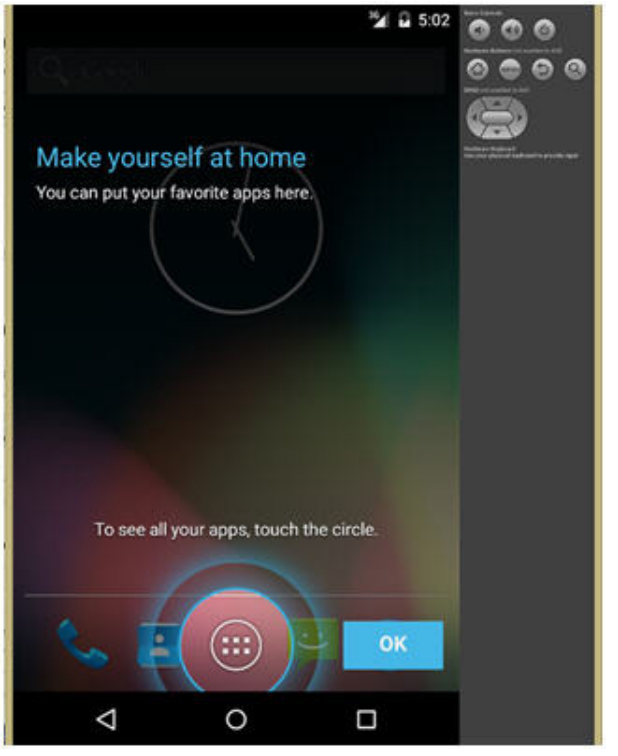


Figure 2.29: Home Screen Circle

4. When the apps load, click the HelloWorld app icon as shown in figure 2.30.

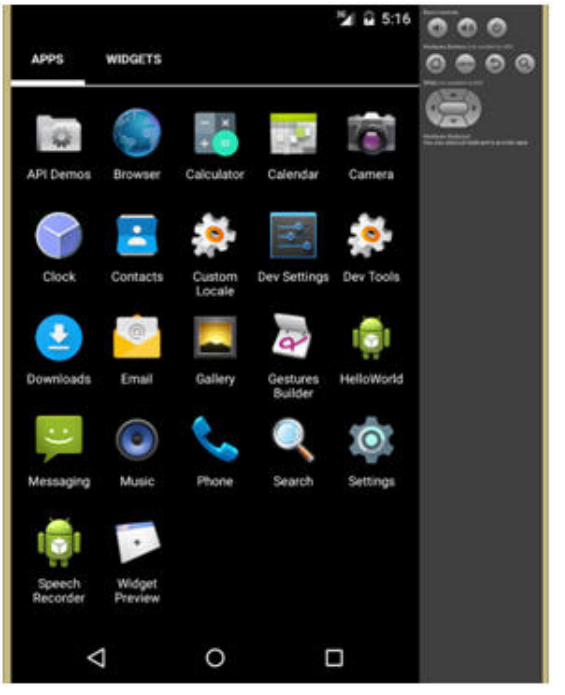


Figure 2.30: Hello World App Icon

The text Hello world! is displayed on the screen as shown in figure 2.31.

Figure 2.31: Hello World Project – Output

Steps to Debug the HelloWorld Project in Eclipse (Windows OS)

1. In Eclipse, click Run → Debug.

The line by line debug reports are displayed in eclipse as shown in figure 2.32.

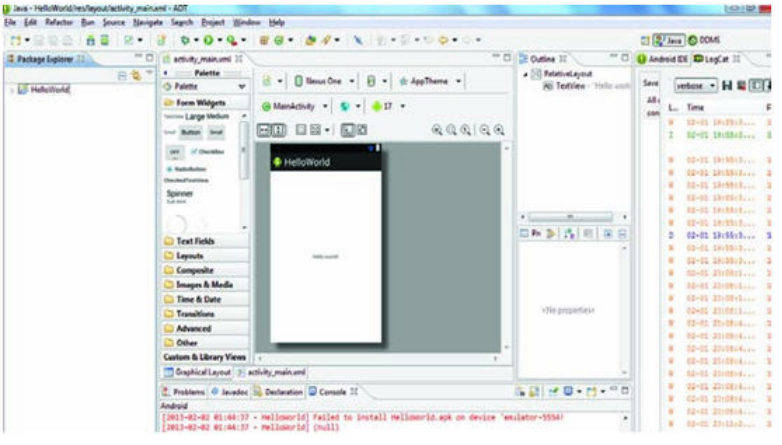


Figure 2.32: Eclipse Debugging Applications

Note - For debugging applications through the emulator, the developer can also go to Run → Debug Configurations.

2.3 Understanding the HelloWorld Project

In Eclipse IDE, the file structure of an Android project is created automatically. The HelloWorld project, when it is created, will consist of some main folders. In the Eclipse IDE, these can be viewed using the Package Explorer pane as shown in figure 2.33.

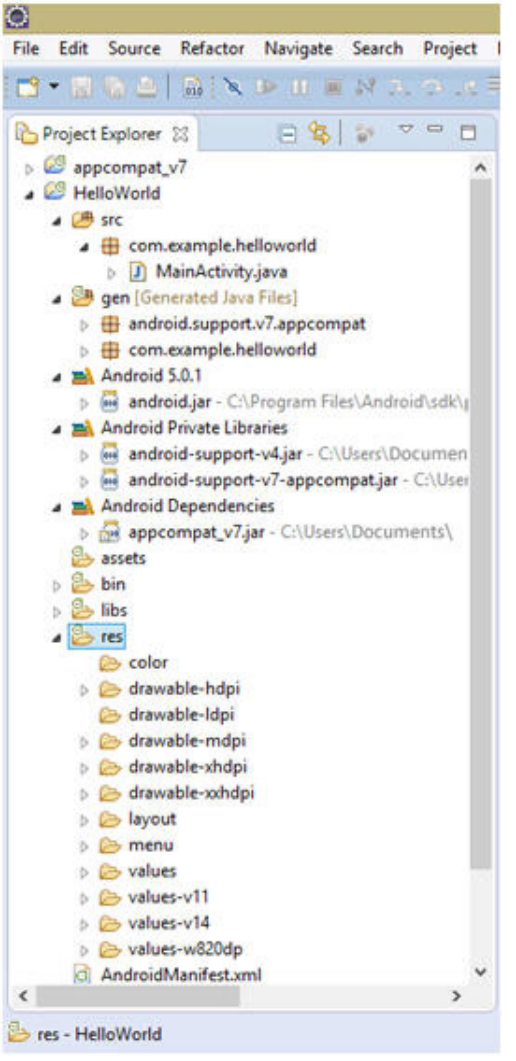


Figure 2.33: Hello World Project Folders

- **src** – This folder consists of the .java and .aidl source files of the HelloWorld project. It contains all the code required by the HelloWorld app. It consists of the stub MainActivity. java file by default.
- **gen** – This folder consists of both compiler generated Java files and other source code files required by the project. It is important not to directly edit the files in this folder. For instance, the folder consists of the R.java file which has references to resources present in the application. It acts as an index to all the resources in the project and the .gen files always need to be writable.

- The Android 5.0.1, Android Private Libraries and Android Dependencies folders contain all the .class libraries required by the application.
- **assets** – This folder is used to save raw asset files that contain assets required by the application to function properly. The folder is originally empty and asset files get added when assets are added for the application. The Asset Manager helps to read and organize asset files similar to a regular file system. It consists of HTML files, text files, databases, and so on.
- **bin** – This folder consists of several compiled resources and the final .apk file.
- **res** – This folder consists of all the applications resources, such as values, layouts and images. In other words, it consists of all the resources used in the application. Resources refer to files that are not part of the code files, but are used by the application code. Resources are worked into the application during build time. The res/layout/ folder consists of XML files that are compiled as screen layouts or parts of a screen. The res/color consists of XML file that describe colors. The res/drawable folders consists of bitmap files, image files, and other XML files for describing the draw able objects and shapes. The res or values folder consists of XML files that are compiled into different types of resources.
- **AndroidManifest.xml** – This is the most important file for each Android application. It is stored in the root folder of the application. It defines the components of the application, such as services and activities. Using this file, a developer can also define Intent filters, content providers, and broadcast receivers. The file contains the name of the application's Java package, which acts as a unique identifier for the application. It contains the permissions for the application and also lists the libraries for the application. It also provides information about the version code, version name, minSdkVersion, and targetSdkVersion as shown in figure 2.34.

MainActivity.j... | activity_main... | proguard-proj... | project.prope... | HelloWorld Ma...

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.helloworld"
    android:versionCode="1"
    android:versionName="1.0" >

    <uses-sdk
        android:minSdkVersion="8"
        android:targetSdkVersion="21" />

    <application
        android:allowBackup="true"
        android:icon="@drawable/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <activity
            android:name=".MainActivity"
            android:label="@string/app_name" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Figure 2.34: AndroidManifest.xml

- **project.properties** – This file sets the default properties for the Android project and consists of settings for the project.
- **proguard-project.txt** – This file contains settings related to the proguard tool. Proguard is a code cleanup tool which removes any unnecessary and unused parts of the code to shrink it. Proguard needs to be enabled in project.properties prior to use.
- **ic_Launcher-web.png** – The application icon for use in play store and the application launcher.
- **activity_main.xml File** – Expand the res → layout folder. It will show the activity_main.xml in Graphical Layout screen. Click the activity_main.xml tab at the bottom to view the code as shown in figure 2.35.

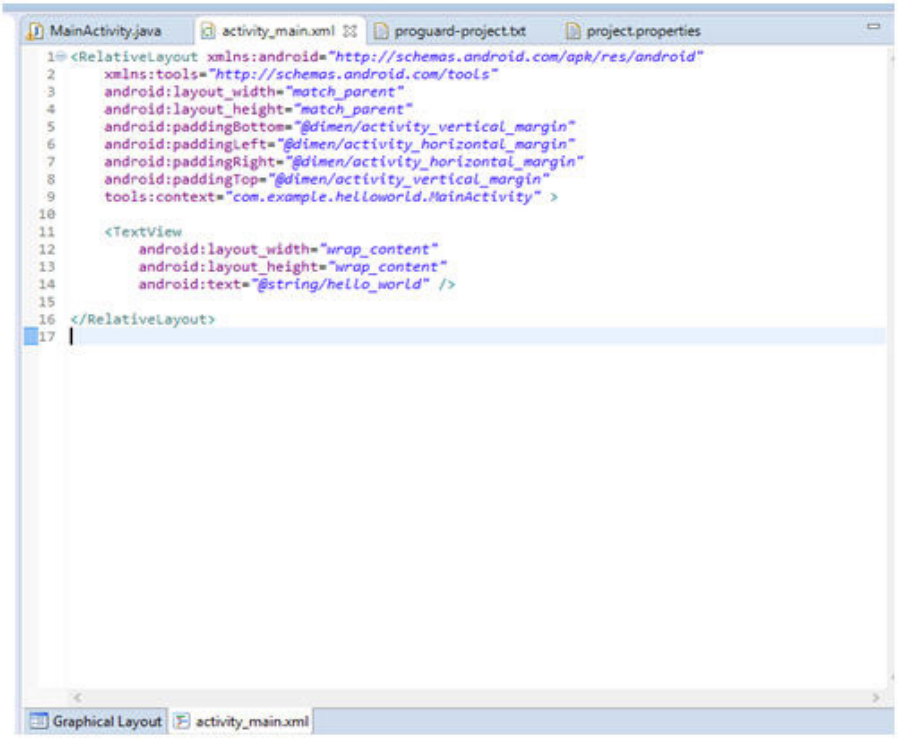


Figure 2.35: activity_main.xml

When the developer creates the new HelloWorld project, Android creates by default the activity_main.xml file. This file is stored in the res/layout/ folder and contains the UI code. It is an XML layout file and is a tree of XML elements. Each node of the tree represents a View class which is used to design a UI layout.

The different elements it contains and their description are as follows:

- **"<RelativeLayout ....." –**

  Specifies the type of layout used for the application. In this case RelativeLayout has

been used. There are many other layouts, such as linear layout, table layout, grid view, Web view, and so on. In the relative layout, the controls are relative to each other or to the parent elements.

- **android:layout_width="match_parent" –**

  Indicates that the width should be of same size as the parent node. In this case, the width will be of same size as screen size. This is the same for the height as well. It is used for specifying the height that will be used by the View. There can be several other properties for a layout.

- **"<TextView ..." –**

  Creates a view component that prints the text given in the android:text attribute. In other words, it specifies the text that will be displayed by the TextView. Here, we the "HelloWorld" text is read from another string value. The string value is defined in strings.xml file which is present in the res/value folder.

☐ **.java File(s)**

Double-click the MainActivity.java file in the /src folder to view the code as shown in figure 2.36.

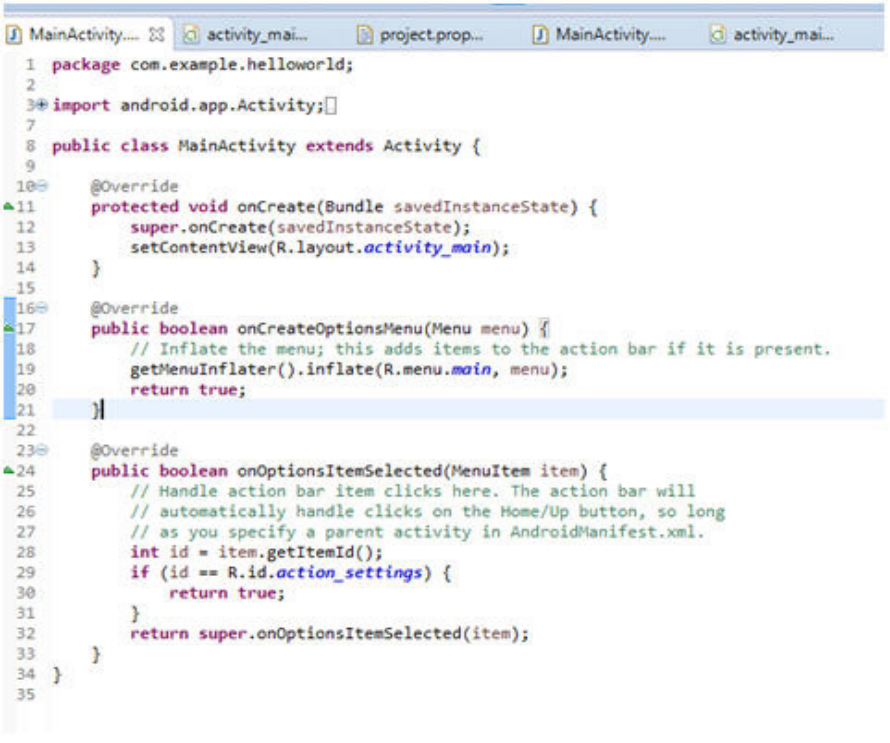


Figure 2.36: MainAcitivity.java – Default Code

Note – At the time of writing eclipse generates deprecated code by extending ActionBarActivity. The code has been modified to use Activity class instead.

MainActivity is a class which extends the Activity class. In the HelloWorld project, MainActivity is the starting activity. Here, the onCreate() method which is part of activity lifecycle has been overridden. The setContentView() method is defined in the Activity class through which the activity_main.xml for the UI is mapped. In other words, it accepts the reference of the layout resource which is identified as R.layout.activity_main. It actually refers to the layout defined in the /res/layout/activity_main.xml file. In Android, an R.java file gets auto-generated for the changes in the resources. So there will be a static entry in R.java for activity_ main.xml. This is the reason why activity_main.xml is accessible using R.layout.activity_main.

2.4 Android Application

Using the Android application layer, the developer can create a wide variety of applications. The Android components are reusable and can be used effectively to build applications.

2.4.1 Android Application Fundamentals

The Android application is written in Java programming language. The distinct feature of the Android application is its ability to live in its own 'security sandbox'. The reasons for this distinct feature are as follows:

- Every application has a different user (a different Linux User ID assigned by the system).
- The User ID is recognized by the system and permission is granted to the application files so that only valid users can access them.
- The presence of a Virtual Machine (VM) provides scope for an application to function independently.

This understanding leads to the concept of 'Principle of Least Privilege'. The 'Principle of Least Privilege' states that every application is dependent on components. Therefore, an application can access only those dependent components which it requires to perform the task. In this manner, a secure environment is maintained.

Exceptions to the 'Principle of Least Privilege' (shared access) are as follows:

- Two applications can function using the same Linux User ID and VM and thus, can access each other's files.
- At the time of installation, permission can be granted to the application to access data beyond its boundary.

This leads to the core framework components that define the application, the manifest file where the components are declared along with the device features required by the application and finally the resources that are separate from the application code and allows the application to optimize its behavior for different device configurations.

2.4.2 Android Application Components

Application components refer to the points that the system can use to enter an application. Each component also determines the way in which the overall application performs. Each component can be independently activated and affects the behavior of the application. All components must be identified in the Android Manifest file.

The four types of application components are activities, services, content providers, and broadcast receivers. Three of the application components are activated by Intents. Intent is a message used for activating components and are created using Intent Objects and can be explicit or implicit. Intents define the action that needs to be performed for activities and services. They define the announcement to be broadcasted for broadcast receivers. Intents do not activate content providers, which are activated by requests from a Content Resolver.

Android application components include:

- **Activities**

  An activity represents the presentation layer. In other words, it is a screen containing the User Interface (UI). It allows the user to interact with the application. An application can have one or more activities. These activities are loosely coupled. Every screen in an application is an activity.

  For example, a Phonebook application can have different activities, such as a Contacts list, Call log, Phone directory, SIM directory, and so on. The activities together form the Phonebook application, but are distinct from each other. Other applications, such as the Messaging application could start the Phonebook application as well.

  There are several activities that are tied together within each application. However, every activity is independent of the other. A specific activity in an application is designated as the main activity, which is typically viewed by the user when the application is first launched. Every activity in the application can initiate another activity. For every new activity that is initiated, the previous activity stops and waits in the background. The changes in activity states are managed by the activity lifecycle.

  It is similar to a form. To create the UI screen, the developer must create a Java class that extends the Activity class. Within this class the developer has to define the UI and implementation of the functionality. This class must implement the callback methods such as onCreate(). These callback methods are invoked by the system, when the transition of the Activity takes place between different lifecycle states.

  UI of the Activity class is implemented using Views. Views are the UI controls that display data and allow user interaction. The setContentView() method is used within the onCreate() method to assign an UI to an activity.

Activity Lifecycle

From the time an activity is initiated, it goes through several stages. The lifecycle of the activity changes depending on other related activities that are performed.

Essentially when the activity is first initiated, the system calls certain lifecycle processes for the activity. The lifecycle processes for the original activity change when the user initiates an action that triggers another activity or accesses another application. The lifecycle of an activity is managed by implementing the callback methods.

In the course of an activity's lifecycle, there are activity states. An Activity goes through four transition states during its lifetime that are as follows:

- **Active State**: When an Activity is present at the top of the stack, it is the currently visible and focused Activity and all the user inputs are provided to it. In order to keep this Activity active, Android provides it with all the required resources and also terminates the previous activities if required. When another Activity becomes active, this Activity will be pushed to the paused state.
- **Paused State**: An Activity in this state is visible, but another Activity will have the focus and is present in the foreground. An Activity in the paused state is treated in the same way as it was treated when it was in the active state. The only difference is that it will not receive any user input. In serious cases, Android terminates a paused Activity to ensure availability of resources.
- **Stopped State**: In this state, the Activity is not visible. It is however, present in the memory with all the state information. All such activities are now ready for termination when the system requires memory. When an Activity is stopped, its data and UI information needs to be saved. An Activity becomes inactive when it is closed or exited.
- **Inactive State**: When the Activity is no longer in the memory, it is said to be in the inactive state. An Activity goes to the inactive state when it is terminated. All such activities need to be restarted before they are used again.

As and when a new activity instance is created, callback methods move the activity state one step forward. When the user is in the process of completing or leaving the activity, callback methods move the activity state one step down. In some cases, callback methods move the activity state part way down when it is to be moved to the top. There are several methods that notify the activity of the different states. Some of the callback methods include onCreate(), onRestart(), onStart(), onResume(), onPause(), onStop(), and onDestroy() as shown in figure 2.37.

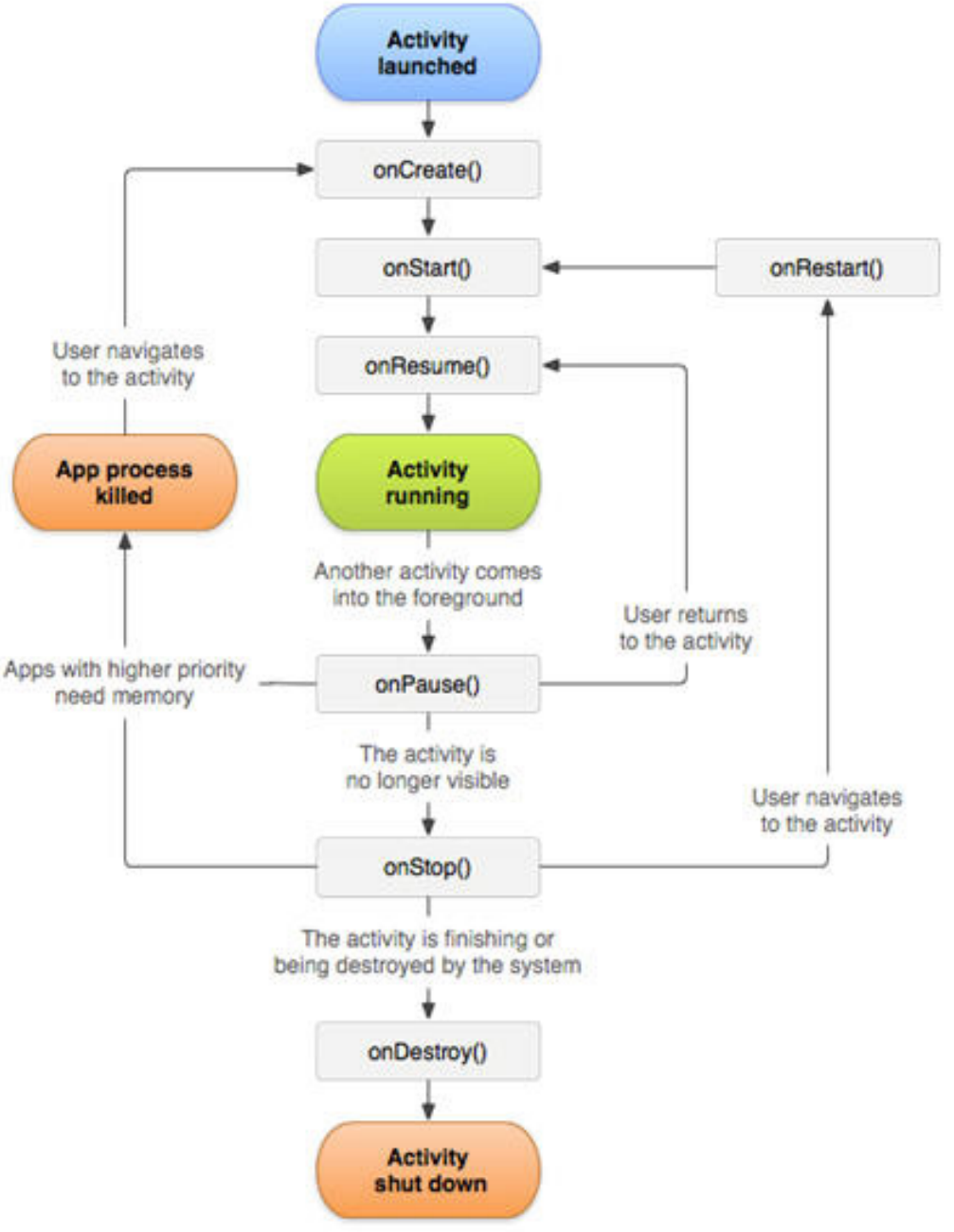


Figure 2.37: Activity Lifecycle

Following is the description of the lifecycle methods:

- **onCreate()**: This callback method that is invoked when the activity is first created. The developer should set up static tasks, such as creating views and binding data. This method is always followed by the onStart() method.
- **onStart()**: This callback method is invoked when the user is beginning to view the

activity. The method is followed by onResume() when the activity is in the foreground and by onStop() if the activity is in the background.

- **onResume()**: This callback method is invoked when the user begins to interact with the activity.
- **onPause()**: This callback method is invoked when the activity is receding to the background but has not been killed.
- **onStop()**: This callback method is invoked when the activity cannot be viewed by the user anymore. It could be followed by onRestart() or onDestroy method based on user activity.
- **onDestroy()**: This callback method is invoked when the activity is about to be destroyed. The method is called when the activity is about to end or when the activity is being terminated due to lack of memory or space.
- **onRestart()**: This callback method is invoked once the activity has stopped and before the activity is started again. This method is also always followed by onStart() method.

These methods define the entire lifecycle of an activity. Once this method is implemented, the developer can monitor three nested loops in the activity lifecycle. These are as follows:

- **Full Lifetime**: This lifetime occurs between onCreate() to onDestroy() method invocation. In this lifetime, the activity normally performs global activity and releasing of resources. For example, it can create a thread in the onCreate() method and stop the thread in the onDestroy() method.
- **Visible Lifetime**: This lifetime occurs between onstart() and onstop() callback methods. The developer maintains the resources between these methods which are required to be displayed to the user.
- **Active Lifetime**: This lifetime occurs between the onResume() and onPause() method invocation. In this lifetime, the activity is in the foreground and has the user input focus.

To see the working of the lifecycle methods, perform the following steps:

1. Open the HelloWorld project folder in Eclipse IDE created earlier.
2. Click the src folder and double-click the MainActivity.java file to view the code. Modify the code as shown in Code Snippet 1.

Code Snippet 1:

```
Code Snippet

…

public class MainActivity extends Activity {

@Override

protected void onCreate(Bundle savedInstanceState) {

      super.onCreate(savedInstanceState);    setContentView(R.
layout.activity_main);

      Toast.makeText(getApplicationContext(), "oncreate()
callback", Toast.LENGTH_SHORT).show();

      }

/** Called when the activity is about to become visible. */

      @Override

protected void onStart() {

super.onStart();


       Toast.makeText(getApplicationContext(), "The onStart()
callback", Toast.LENGTH_SHORT).show();

             }

            /** Called when the activity has become visible. */

            @Override

      protected void onResume() {       super.onResume();

       Toast.makeText(getApplicationContext(), "The onResume()
callback", Toast.LENGTH_SHORT).show();

             }

            /** Called when another activity is taking focus. */

            @Override

      protected void onPause() {   super.onPause();

       Toast.makeText(getApplicationContext(), "The onPause()
callback", Toast.LENGTH_SHORT).show();
```

```
/** Called when the activity is no longer visible. */
            @Override
      protected void onStop() {    super.onStop();
       Toast.makeText(getApplicationContext(), "The onStop()
callback", Toast.LENGTH_SHORT).show();
             }
            /** Called just before the activity is destroyed. */
            @Override
            public void onDestroy() {
             super.onDestroy();
       Toast.makeText(getApplicationContext(), "The onDestroy()
callback", Toast.LENGTH_SHORT).show();
            }
@Override  public boolean onCreateOptionsMenu(Menu menu) {
// Inflate the menu; this adds items to the action bar if it
is present.      getMenuInflater().inflate(R.menu.main, menu);
returntrue;
      }
}
```

Note - In the code, the Toast class is used to display a message to the user.

Figure 2.38 displays the output of the modified code.

Figure 2.38: Lifecycle Call Back Methods – Output

Services

A service as a component is not displayed on the UI. In other words, it does not help in the design of the UI. It is a background component that processes lengthy operations or is connected to processes that run remotely. Consider an example of music being played in the background while the user is surfing the net or checking emails. Music in the background is considered as a service. This will not affect user interaction with an activity.

Services include gathering data without interrupting another activity and running a process while the user is involved in another activity task. Other activities can start a service, run it, and get attached to the service as well.

Each service has two forms – Started and Bound.

A Started service has the following features:

- Starts when the startService() method is invoked by an application component, such as an Activity.
- Runs indefinitely in the background irrespective of the state of the invoking component.
- Stops once the operation is complete.

A Bound service has the following features:

- Starts when the bindService() method is invoked by the application component to bind to it.

- It interacts with the application components, such as by sending requests, obtaining results and performing Inter Process Communication (IPC).
- Runs as long as the application component is bound. Several components can bind to a single service.

Note -A service also needs to be identified in the AndroidManifest.xml file.

A new service class must extend the Service class. Some of the callback methods must be overridden in the implementation class to handle key aspects of the service lifecycle and provide a mechanism for the components to bind to the service.

Service methods include Context.startService(), onStartCommand, Context. stopService, and Context.bindService().

Following is the description of the Service class methods:

- **Context.startService**(): This method is used to start an application service.
- **onStartCommand**(): This method of the Service class is called once the service is requested to be started.
- **onCreate**(): This method is invoked to perform the setup procedures only once when the service is first created.
- **Context.stopService**() **or stopSelf**(): This method is used to stop the service. The method is also used to stop multiple instances of a service. The stopSelf (int) method can be used to prevent the service from stopping until all the intents that were started are processed.
- **Context.bindService**(): This method is used to keep the service connected until the connection remains. An IBinder object is received by the client when the service returns from the onBind() method. This will allow the client to make calls back to the client.
- **onBind**(): This method is invoked when another component wants to bind with the Service class.

The service lifecycle is shown in figure 2.39

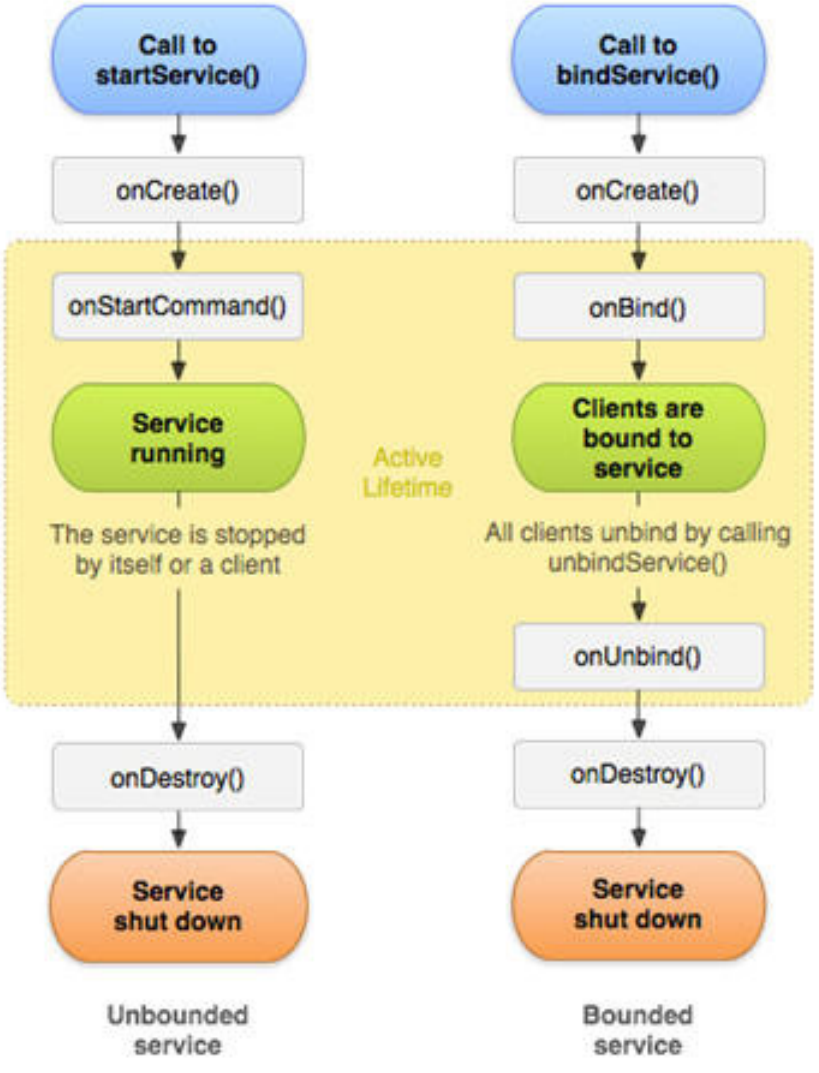


Figure 2.39: Service Lifecycle

Now, consider the example of music player where the music is played and stopped with the help of an Android service. The steps to be performed are as follows:

1. Open the HelloWorld Project in Eclipse which developer created earlier.
2. Navigate to src → com.example.helloworld.
3. To create a new class right-click the package, com.example.helloworld to display the context menu.
4. Select New → Class as shown in figure 2.40.

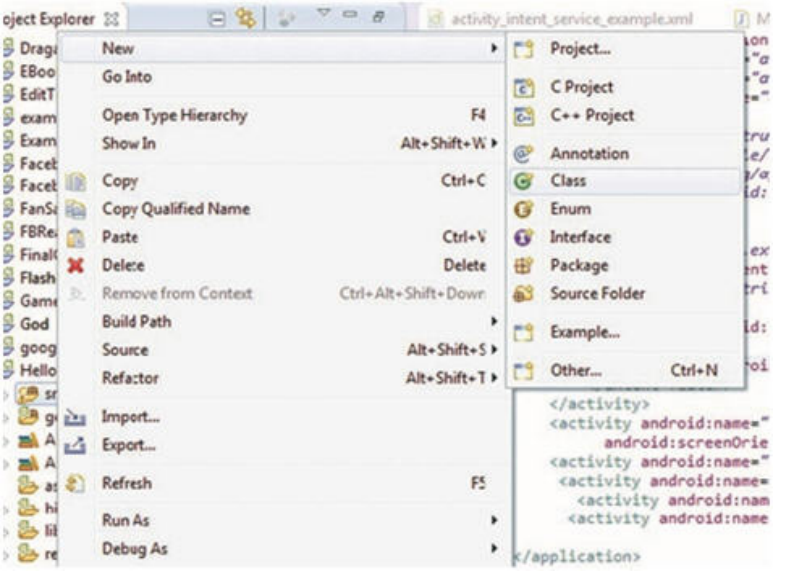


Figure 2.40: Create Class

The New Java Class dialog box is displayed as shown in figure 2.41.

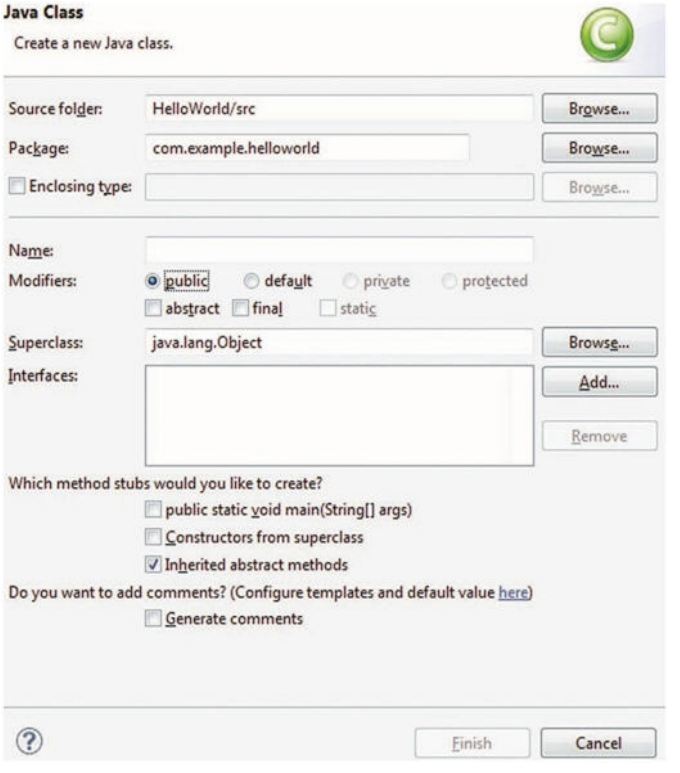


Figure 2.41: New Java Class

5. Type ServiceRun in the name box.
6. Click Finish.

7. Modify the code as shown in Code Snippet 2.

Code Snippet 2:

```
package com.example.helloworld; import android.app.Service; import
android.content.Intent; import android.media.MediaPlayer; import
android.os.IBinder;

public class ServiceRun extends Service {

      MediaPlayer mp_object;

      @Override

      public IBinder onBind(Intent argø) {     // TODO Auto-
                                               generated method stub

       returnnull;

      }

      @Override

      public void onCreate() {

       // TODO Auto-generated method stub

       super.onCreate();      mp_object = MediaPlayer.
                    create(getApplicationContext(), R.raw.flysong);

      }

      @Override

startId) { public int onStartCommand(Intent intent, int flags, int

       mp_object.start();

       return 0;

      }

      @Override


public void onDestroy() {

    mp_object.release();

       super.onDestroy();

      }

}
```

8. Create a new folder named 'raw' inside the /res directory by right-clicking the /raw folder and selecting New → Folder.

9. Copy a song inside it as shown in figure 2.42. For example, it is flysong.mp3.

Note - The song name should be in lowercase without space.

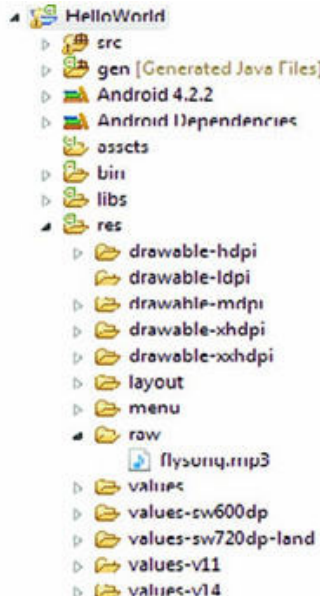


Figure 2.42: Folder Structure

10. Navigate to the res → layout folder.
11. Double-click activity_main.xml file to display the file in Graphical Layout mode and add two buttons by dragging the button as shown in figure 2.43.

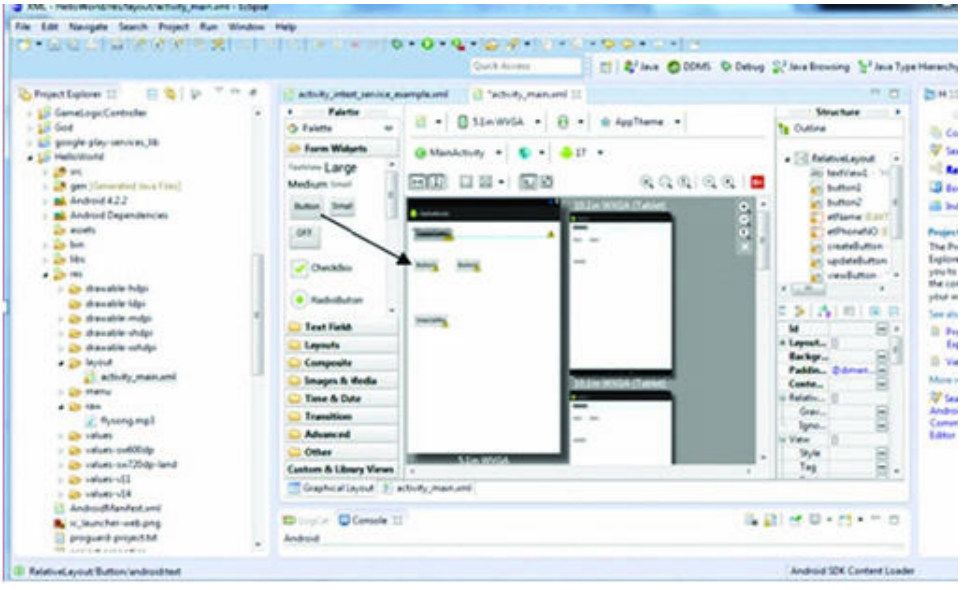


Figure 2.43: Create Buttons

12. Click activity_main.xml tab at the bottom to open the file in editable screen.
13. Modify the code as shown in Code Snippet 3.

Code Snippet 3:

```
<RelativeLayout xmlns:android=”http://schemas.android.com/
apk/res/android”
xmlns:tools=”http://schemas.android.com/tools”
android:layout_width=”match_parent”
android:layout_height=”match_parent”
tools:context=”.MainActivity”>
<TextView
android:id=”@+id/textView1”
android:layout_width=”wrap_content”
android:layout_height=”wrap_content”
android:text=”@string/hello_world”/>
<Button
android:id=”@+id/button1”
android:layout_width=”wrap_content”
android:layout_height=”wrap_content”
android:layout_alignLeft=”@+id/textView1”
android:layout_below=”@+id/textView1”
android:layout_marginTop=”80dp”
android:text=”Start_Music”/>
<Button
android:id=”@+id/button2”
android:layout_width=”wrap_content”
android:layout_height=”wrap_content”
android:layout_alignBaseline=”@+id/button1”
android:layout_alignBottom=”@+id/button1”
android:layout_marginLeft=”45dp”
android:layout_toRightOf=”@+id/button1”
android:text=”Stop_Music”/>
</RelativeLayout>
```

14. Navigate to res → layout folder.

15. Open and add the <service> tag in AndroidManifest.xml file as shown in Code Snippet 4.

Code Snippet 4:

```
<?xmlversion=”1.0”encoding=”utf-8”?>
<manifest xmlns:android=”http://schemas.android.com/apk/res/
android”
package=”com.example.helloworld”
android:versionCode=”1”
android:versionName=”1.0”>
<uses-sdk
android:minSdkVersion=”8”
android:targetSdkVersion=”17”/>
<application
android:allowBackup=”true”
android:icon=”@drawable/ic_launcher”
android:label=”@string/app_name”
android:theme=”@style/AppTheme”>
<activity
android:name=”com.example.helloworld.MainActivity”
android:label=”@string/app_name”>
<intent-filter>
<action android:name=”android.intent.action.MAIN”/>
<categoryandroid:name=”android.intent.category.LAUNCHER”/>
</intent-filter>
</activity>
<service android:name=”.ServiceRun”>
</service>
</application>
</manifest>
```

16. Open the MainActivity.java file and set the onClickListeners for both the buttons inside the code as shown in Code Snippet 5.

Code Snippet 5:

```
package com.example.helloworld;
import android.os.Bundle;
import android.app.Activity;
import android.content.Intent;
import android.view.Menu;
import android.view.View;
import android.widget.Button;
import android.widget.Toast;
public class MainActivity extends Activity {
 Button play_song, stop_song;
 @Override
 protected void onCreate(Bundle savedInstanceState)
                              (super.onCreate(savedInstanceState);
  setContentView(R.layout.activity_main);
  Toast.makeText(getApplicationContext(), "oncreate()}
                                          callback",
    Toast.LENGTH_SHORT).show();
  play_song = (Button) findViewById(R.id.button1);
  stop_song = (Button) findViewById(R.id.button2);
  play_song.setOnClickListener(new View. OnClickListener() {
   @Override
   public void onClick(View v) {
    /** way to call the service class */
      Intent service = new Intent(MainActivity.this, ServiceRun.
                                                      class);
    startService(service);
```

```
 }
  });
  stop_song.setOnClickListener(new View.OnClickListener() {
   @Override
   public void onClick(View v) {
    Intent name = new Intent(MainActivity.this, ServiceRun.class);
    stopService(name);
   }
  });
 }
/** Called when the activity is about to become visible. */
 @Override
 protected void onStart() {
  super.onStart();
  Toast.makeText(getApplicationContext(), "The onStart() callback",
    Toast.LENGTH_SHORT).show();
 }
 /** Called when the activity has become visible. */
 @Override
 protected void onResume() {
  super.onResume();
  Toast.makeText(getApplicationContext(), "The onResume() callback",
    Toast.LENGTH_SHORT).show();
 }
/** Called when another activity is taking focus. */
@Override
 protected void onPause() {
  super.onPause();
 Toast.makeText(getApplicationContext(), "The onPause() callback",
```

```
    Toast.LENGTH_SHORT).show();
 }
 /** Called when the activity is no longer visible. */
 @Override
 protected void onStop() {
  super.onStop();
  Toast.makeText(getApplicationContext(), "The onStop() callback",
    Toast.LENGTH_SHORT).show();
 }
/** Called just before the activity is destroyed. */
 @Override
 public void onDestroy() {
  super.onDestroy();
 Toast.makeText(getApplicationContext(), "The onDestroy() callback",
    Toast.LENGTH_SHORT).show();
 }
 @Override
 public boolean onCreateOptionsMenu(Menu menu) {
  // Inflate the menu; this adds items to the action bar if it
is present.
  getMenuInflater().inflate(R.menu.main, menu);
  returntrue;
 }
}
```

17. Build and execute the application by clicking Run → Run.

The output in the emulator will be as shown in figure 2.44.

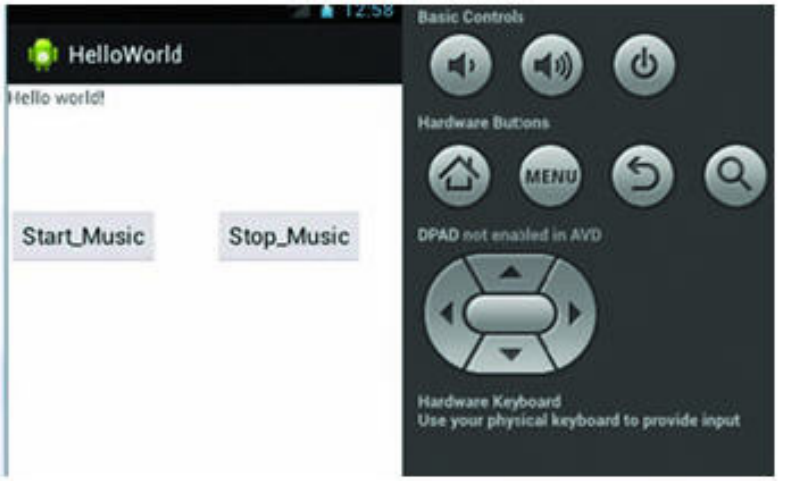


Figure 2.44: Start and Stop the Music

Click Start_Music and Stop_Music buttons to start and stop playing the song respectively.

- **Content Providers**

Content providers are used for managing shared data sets. The data can be stored in the SQLite Database, Web, or other easily accessible storage location. The content providers help other applications to query for data and make changes to the data for a specific application.

For example, if the content provider allows access, an application can query personal information, for instance, information in a phonebook. The content provider also enables reading and writing data that is not shared. A content provider is implemented as a subclass of ContentProvider. ContentResolver object present in the client provides access to data in a content provider.

Content providers are simple in their existence as they use the standard methods of insert(), update(), delete(), and query() to work with data. Some examples of native Android content providers are media store, settings provider, contacts provider, and so on.

Content providers are accessed using a URI model. The main feature of the content provider is its flexibility to extend so that the native Android databases can be modified. Access to data from content providers is managed by the ContentResolver class.

An object of the ContentResolver class is obtained using the getContentResolver() method.

The ContentResolver class has methods to query and modify content providers.

The query string used for querying a content provider is in the form of a URI. Content providers have two forms of URI. The first form will access all data and the second form will access a single row.

The expression of the two URI forms is as follows:

```
<standard_prefix>://<authority>/<data_path>/<id>
```

Parts of the URI are as follows:

- content:// is the standard prefix for all content providers.
- The name of the content provider will be specified by the authority. Contacts for the built-in contacts content provider can be stated as an example.
- The type of data requested will be specified by the data path. For example, if all the contacts are obtained from the contacts content provider, the URI would be content://contacts/people.
- The specific record that is requested will be specified by the id. If contact number 5 is being looked for, then the URI would be: content://contacts/people/5.

The methods that need to be used for content providers are as follows:

- **onCreate ()** – Is used to initialize the content provider for the first time.
- **query (Uri, String [], String, String [], String)** – Provides data to the caller. In other words, it is used to retrieve the records.
- **insert (Uri, ContentValues)** – Is used to insert data into the content provider.
- **update (Uri, ContentValues, String, String [])** – Is used to update data in the content provider.
- **delete (Uri, ContentValues, String, String [])** – Is used to delete data from the content provider.
- **getType (Uri)** – Is used to provide the MIME data type in the content provider.

Broadcast Receivers

The broadcast receiver, as the name suggests, is a component that receives or responds to announcements broadcast by the system. These could be announcements for low battery, locked screen, data downloaded, and so on. Broadcast receivers also inform other applications of data updates. Broadcast receivers do not display a UI as well. The broadcast receiver acts as an entry point for other components and is simply an announcement component. Most of the broadcasts are initiated by the system but an application can also initiate a broadcast.

Broadcasts are of two types namely, Normal and Ordered. Normal broadcasts are received and executed in an asynchronous order. Ordered broadcasts are received one at a time and are synchronous. With normal broadcasts however, at times, the system can deliver one broadcast at a time.

The Broadcast Receiver class' lifecycle includes security, receiver lifecycle, and process lifecycle. Developer can dynamically register an instance of this class with Context.registerReceiver() or statically with the help of <receiver> tag in AndroidManifest.xml file.

Receivers are used with Context APIs and interact with other applications. This makes them vulnerable to abuse by other applications. Some considerations with regard to security are as follows:

- When using registerReceiver (BroadcastReceiver, IntentFilter) the registered receiver is open to receive broadcasts from any application. Permissions need to be set for controlling the broadcasts that are sent. In the latest version the package name of the application that will receive the broadcast can be specified.
- Intent names should be written in a namespace owned by the developer to avoid applications conflict.
- When a receiver is published in the manifest file and intent filters are specified for the file, enter android:exported="false" to ensure security.

Receiver Lifecycle

This is important because a BroadcastReceiver is only active for the call for onReceive() method. Once the function is terminated, the receiver becomes inactive. A broadcast receiver extends the BroadcastReceiver abstract class which means that the developer has to implement the base class method onReceive(). Whenever an event occurs, Android invokes the onReceive() method on the registered broadcast receiver.

A process lifecycle keeps the BroadcastReceiver running until onReceive() method is active.

Methods for BroadcastReceiver are as follows:

- **abortBroadcast()** – This method call to the broadcast receiver indicates that the receiver should abort a current broadcast and work for broadcasts sent via Context.sendOrderedBroadcast.
- **clearAbortBroadcast()** – This method call to the broadcast receiver indicates that the current broadcast should be terminated.
- **getAbortBroadcast()** – This method returns boolean value indicating whether the receiver should abort the current broadcast.
- **getDebugUnregister()** – This method returns the last value from

setDebugUnregister(boolean) method.

- **getResultCode()** – This method provides the result code that is set by the previous receiver.
- **getResultData()** – This method receives the current data that is set by the previous owner.

Note - Avoid using long running task in the broadcast receiver. For performing any long task, start a service within a receiver.

Resources

An application consists of content that is generated programmatically as well as content that is static in nature such as images, icons, audio files, String labels. The static content of the application is isolated from the code and termed as 'Resources'.

Resources are stored in the 'res' folder in the code structure. The Resources need to be added to the appropriate subdirectory, depending upon the type of the Resource. Various types of Resources are explained in table 2.1.

Type	Description	Location	Access by
Animation Resources	Animations used within the application	/res/anim /res/drawable	R.anim R.drawable
Color State List Resource	A color resource which varies based on the view state	/res/color	R.color
Drawable Resources	Static Images used within the application	/res/drawable	R.drawable
Layout Resource	Defines layouts for use with the application UI	/res/layout	R.layout
Menu Resource	Defines Content of application menu	/res/menu	R.menu
String Resource	Defines static strings used within the application	/res/values	R.string R.array R.plurals
Style Resources	Themes and format for UI elements	/res/values	R.style
Other Resources	Other xml data	/res/xml	Resources.getXML()

Table 2.1: Resource Types

Each Resource can be accessed by referencing it by the R. access class, followed by its name.

For example, a string resource with the name labeltext can be access by R.string. labeltext.

- **Assets**

Assets are similar to resources in terms of functionality. That is, it can also be used to hold static data. The primary difference between the two appears in the areas of usage and the nature of data that can be stored.

Assets are stored in the /assets or /src/main/assets directory of the code structure.

While resources are used to store specific kind of data, assets do not have data type requirements. They can hold data of any format. Any files placed in the assets folder are added to the .apk as is. The assets directory acts like a file system. Any file added as an asset can be accessed in the same way as a file on the file system.

In the case of resources, every resource is given a precompiled id which can be used to access quickly by calling R.id.[id here]. Whereas, assets are accessed in the same way as file system files, which adds an I/O delay.

Resources have built-in support for multi-language functionality. Due to the ambiguity in the type of data store in assets, this is not possible. Assets need not be declared or indexed prior to compilation. Assets also allow for creating sub-directories and custom hierarchies to store data.

If the application project is added as library to another project, the resources are still accessible in the new project whereas assets are not.

- **Layouts**

A layout defines the visual structure for a UI interface. It acts as a basic building object or a View object.

Layouts can be declared in two ways:

- Within XML files
- programmatically in code

The developer can use any one or both of these methods. The XML file is preferable in cases where the UI is expected to be constant irrespective of the user interaction

up to that point.

The layout XML files can be accessed in the /res/layout/directory of the code structure. Android provides a set of default layouts. Google recommends the developer to use one of these layouts, for consistency of look and feel. Some of these are listed in the following list.

- Linear Layout
- Relative Layout
- Table Layout
- Absolute Layout
- Frame Layout
- List View
- Grid View

2.4.3 Android Application Components

The mode of communication between the components includes Intents and Intent filters.

Intent

- An Intent activates the three components - activities, services, and broadcast receivers. When several actions from different components are requested at runtime for an application, the Intent brings the components together.
- An Intent object creates an Intent. In other words, it is a gamut of information used for activating the components. The Intent object identifies the message that should activate a component or component type. There are two types of Intents – implicit and explicit.
- Explicit Intents are Intents that identify a component by the component's name. The Java class name is used as the identifier. Implicit Intents do not identify a component by the name. Implicit Intents are most commonly used for identifying components in other applications.
- In case of activities and services, an Intent determines the action that must be performed. For example, sending a message or viewing an e-mail.
- An Intent can also communicate a request for data. For instance, an Intent can communicate a request to display a file. For activities that involve a result, Intents can communicate the request for the data and the data result.
- In case of broadcast receivers, the Intent merely communicates the announcement, for instance, "data downloaded".

- Intents do not activate content providers. Content providers are activated by content resolvers.

Intent Filter

Activities, services, and broadcast receivers use Intent filters to inform the system about the Intents that they can manage.

Note - In an Intent Filter, two components are unaware of each other's existence. However, they work in partnership to deliver the expected result.

- Every filter provides information on a particular ability of the component and the Intents that the component can receive.
- An explicit Intent does not require a filter as it directly communicates with the component. An implicit Intent can communicate with a component only if the filters allow it to.
- Although implicit Intents are filtered, the Intent filter does not provide any security.
- A component contains a different filter for each activity.
- Intent filters are executed as an object of IntentFilter class.

Some of the points to remember about Intent filters are as follows:

- A component can claim any number of intent filters.
- If there is more than one component claiming the same intent filter, the user can decide to select the appropriate component.
- Priorities can be set for intent filters to track the order of responses

Pre-existing Components

Pre-existing components are determined by the Android manifest file. The pre-existing components are vital for the execution of implicit Intents. They include:

- **Intent Objects** – An Intent object consists of information of interest that is received by a component. It also contains information, such as category of the component. The Intent should be handled by the component name which is provided in the manifest file as a component name object. The component name is optional and the Intent object can deliver the information to the designated class or to another target it identifies.
- **Intent Matching** – Intents are matched with Intent filters to identify a target component that needs to be activated. They are also matched to identify more information on the device components. Intent matching can be specified using "android.intent.category.Launcher" and "android.intent.category.HOME".

- **Intent Resolution** – Intents are of two types, explicit and implicit. Explicit intents identify target components by their name. These are typically used for messages internal to an application. Implicit Intents do not identify components by name and are used for messages in other applications. An activity or service is identified that can best perform the specified action and the Intent is delivered accordingly. This is done by comparing the Intent objects to Intent filters. Filters display the abilities of components so that they can receive Intents. Intent resolution basically functions around matching Intent against all the <intent-filter> descriptions in the installed application packages.

Android Manifest File

The manifest file is important because all the components required for an application need to be present in this file. Before an application component is initiated, the system looks for the component in the AndroidManifest.xml file.

Every Android project contains the AndroidManifest.xml file which contains the detailed configuration information as shown in figure 2.45. It can be considered as a metadata file and is present in the root directory of the project. It contains the details of the components that exist in an application.

Figure 2.45: Android Manifest File

In figure 2.45,

- package – Specifies the package name of the application.
- android:versionCode – Specifies the internal version of the project.

- android:versionName – Specifies the version number shown to the user.
- android:minSdkVersion – Specifies an integer value indicating the minimum Android API version number.
- android:targetSdkVersion – Specifies an integer value indicating the target Android API version number.

The <application> tag is a container as it includes the tags for declaring Services, Activities, ContentProvider, and BroadcastReceiver present in the application.

There is only one application node present in the AndroidManifest.xml file. Android application related configuration information such as app icon, name, theme, and so on are also declared and form the metadata of information of the application.

Every activity in Android should be registered here in the <activity> tag. In the case of HelloWorld project, the MainActivity class is registered and also made as the launcher Activity class. In the code, the android:name attribute is used to specify the Activity class name. This displays the main launch Activity and any other screen/dialogs. A runtime exception is thrown when an Activity has not been defined but an attempt is made to start the Activity. The <activity> tag also specifies which Intents will launch the Activity and is specified by their child tags <intent-filter>.

Other node tags that can be used in AndroidManifest.xml file are as follows:

- **<service>**: The tag is used for declaring each Service class used in the Android application development. The <service> tag supports the <intent-filter> child tags.
- **<provider>**: The tag is used for declaring content provider.
- **<receiver>**: The tag is used for registering a broadcast receiver.
- **<uses-permission>**: The developer uses the tag to set permissions for applications so that it can function correctly. In other words, this tag is used to grant access to a feature that is protected by permission. During installation, the user is presented with the option of granting or denying permission to access certain applications. Even execution of native applications, such as receiving of SMS or dialing, requires permission.
- **<permission>**: The tag helps in defining permissions in the manifest file. These permissions refer to restricting access to certain shared application components. The android:name attribute specifies the name of the permission that will be referred in the code and the android:protectionLevel characterizes the level of access the permission grants.

  The values for android:protectionLevel are as follows:

  - **normal**: Signifies low risk and is the default value.

- **dangerous**: Signifies high risk.
- **signature**: Signifies that the requesting application and the application declaring permission should be signed with the same certificate.
- **signatureOrSystem**: Signifies that the requesting application should be signed with the same certificate as that which is present in the system image.

The android:label and the android:description explains the risk involved in granting this level of permission.

- <instrumentation>: The <instrumentation> tag is used for monitoring the application and its response to the system resources. The monitoring is done by conducting tests on activities and services at runtime.

Importance of the Manifest File

The important reasons of using manifest file are as follows:

- Assists in granting permission. An example could be permission to access the Internet.
- Helps in declaring hardware and software requirement for a particular application. For example, Bluetooth, multi-touch screen, and so forth.
- Declares application components.
- Helps in specifying the minimum API level required by the application.
- Helps in linking the application to the API libraries. For example, contains link to the Google Maps library.
- The manifest file contains a root <manifest> tag and a <package> attribute. This <package> attribute is associated to the project's package. The versionCode and versionName attributes are used to specify the application version number and the version number that will be displayed to the user respectively.
- Apart from the libraries in the Android architecture, the application needs to be connected to other application libraries. The manifest file determines the libraries as well.

2.5 Recreating the Project Using Android Studio

Android Studio is the Google's Official IDE for Android application development. This section demonstrates the procedure for creating and running projects using Android Studio.

2.5.1 Creating a New Project

1. Start Android Studio from the Desktop shortcut or the start menu shortcut, or by

navigating to the installation directory. The loading screen appears as shown in figure 2.46.

Figure 2.46: Android Studio Splash Screen

Once the IDE opens, the Welcome screen appears as shown in figure 2.47.

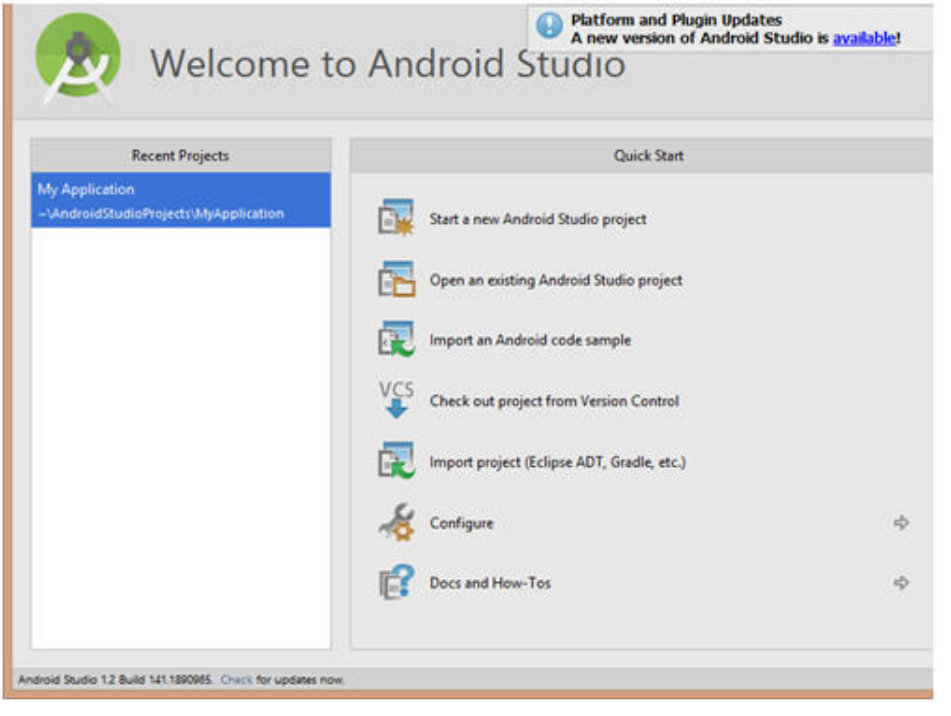


Figure 2.47: Android Welcome Screen

2. Click Start a new Android Studio project.

The new project window appears as shown in figure 2.48.

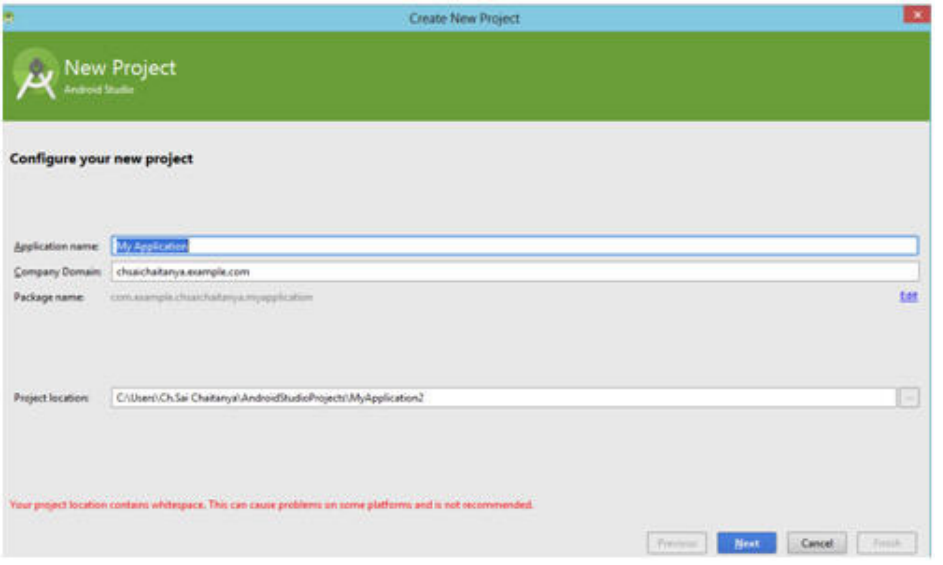


Figure 2.48: New Project Screen

3. Enter the Application name, Company Domain, and the Project location as shown in figure 2.49.

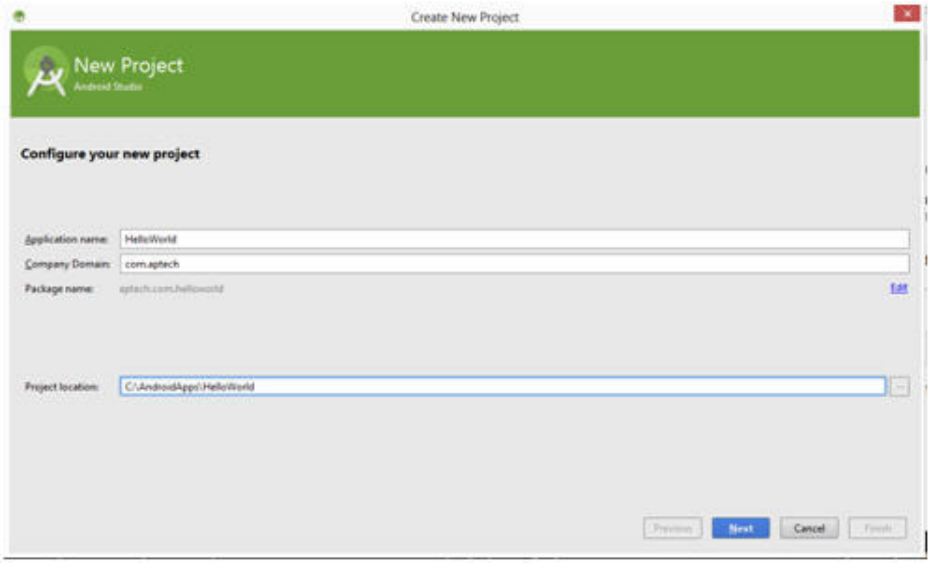


Figure 2.49: New Project Details

4. Click Next. The Target Android Devices screen appears.
5. Select the API level as API 21 as shown in figure 2.50.

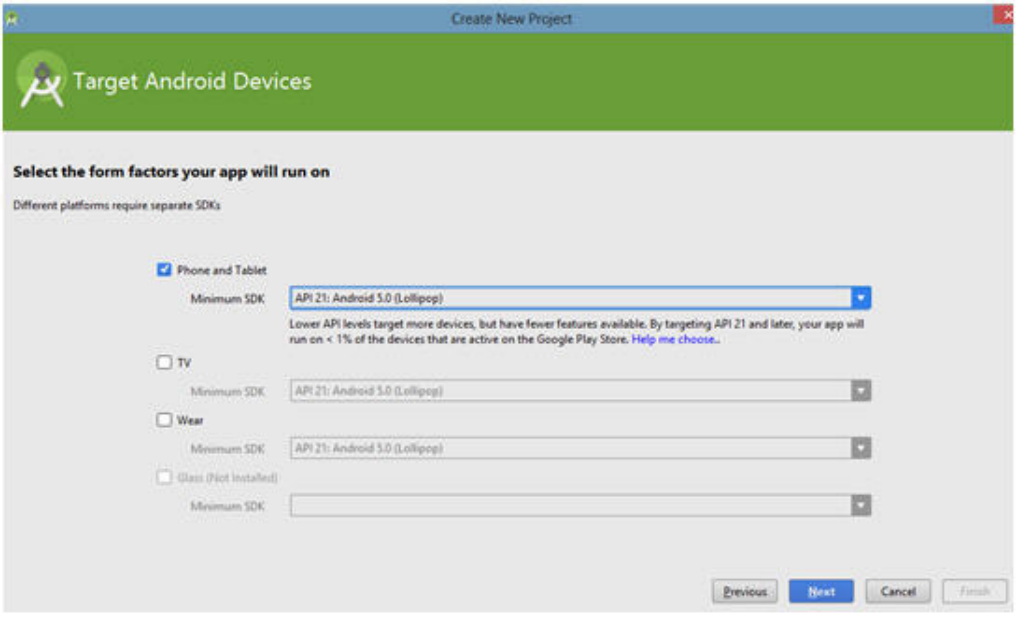


Figure 2.50: Target Devices Details

6. Click Next.

The Activity Screen appears as shown in figure 2.51.

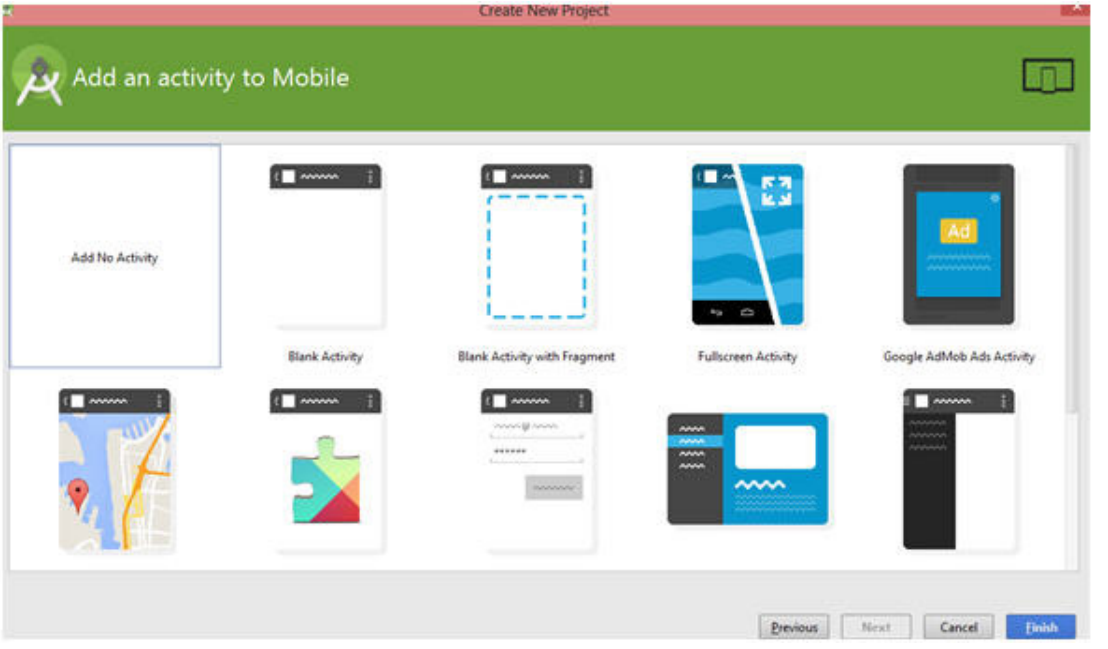


Figure 2.51: Activity Selection screen

7. Select Blank Activity and click Next.

The Customize activity screen appears as shown in figure 2.52.

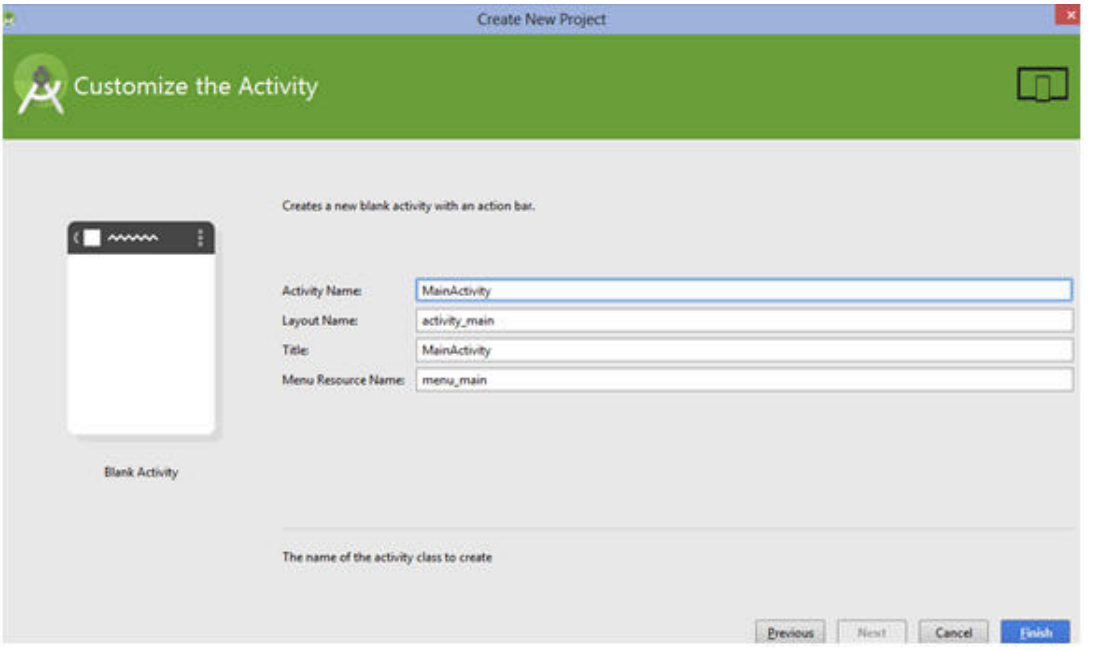


Figure 2.52: Activity Customization Details

8. Retain the default values and click Finish.

The project is created and opened in the IDE as shown in figure 2.53.

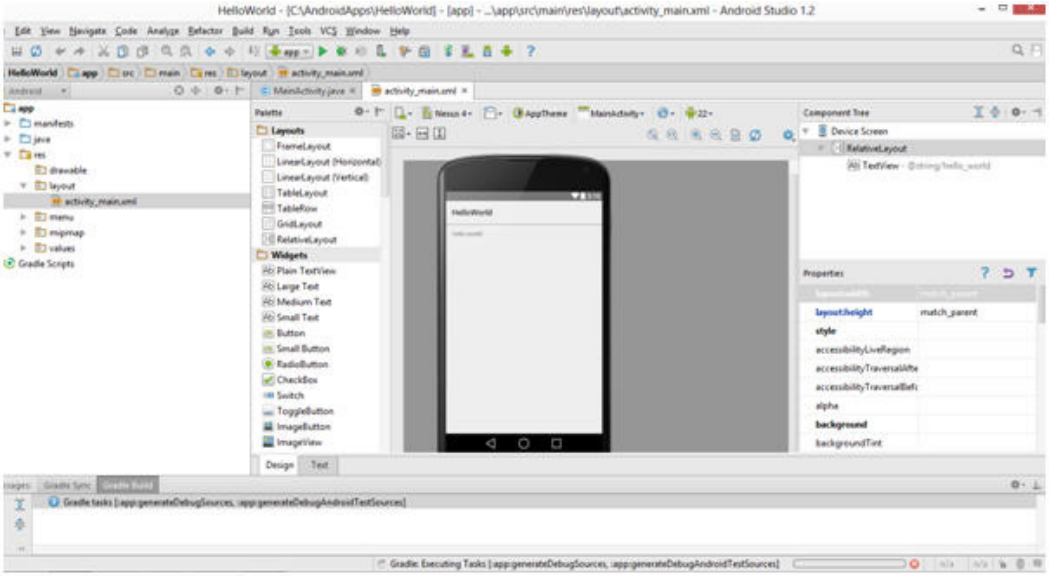


Figure 2.53: Project Created In Android Studio

By default the GUI editor is opened for editing the appearance of the activity screen.

2.5.2 Editing and Running the Project

There are two ways to display the test “Hello World!” message, both of which will be discussed here.

☐ Using the GUI Editor

1. Double-click the activity_main.xml, if it is not already opened as shown in figure 2.54.

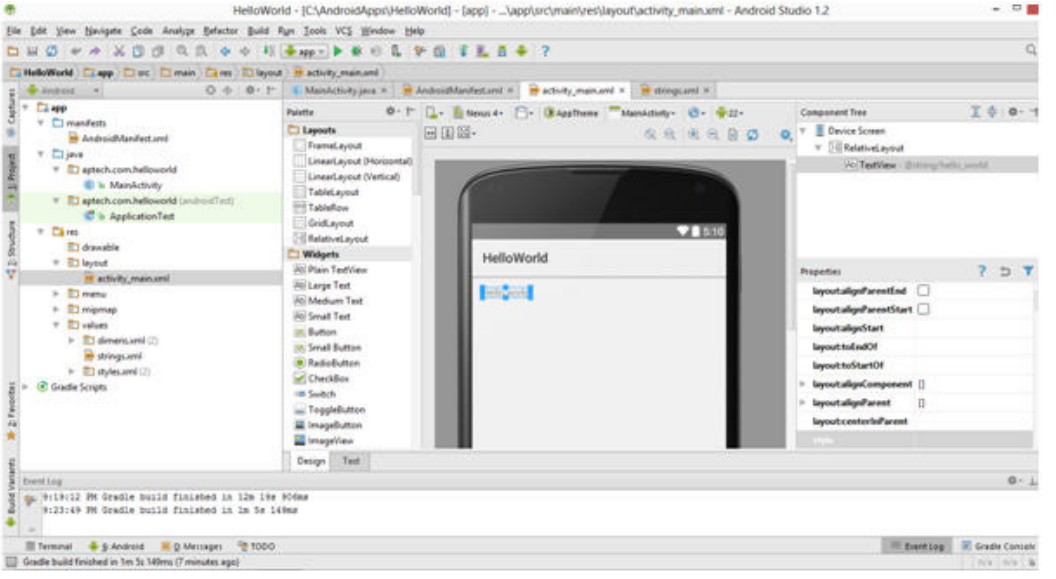


Figure 2.54: acitivity_main.xml – Design Tab

The Project comes with a default TextView with the content set to the string resource named hello_world.

2. Double-click the TextView object and type unique id ‘mainTextView’ in the id field as shown in figure 2.55.

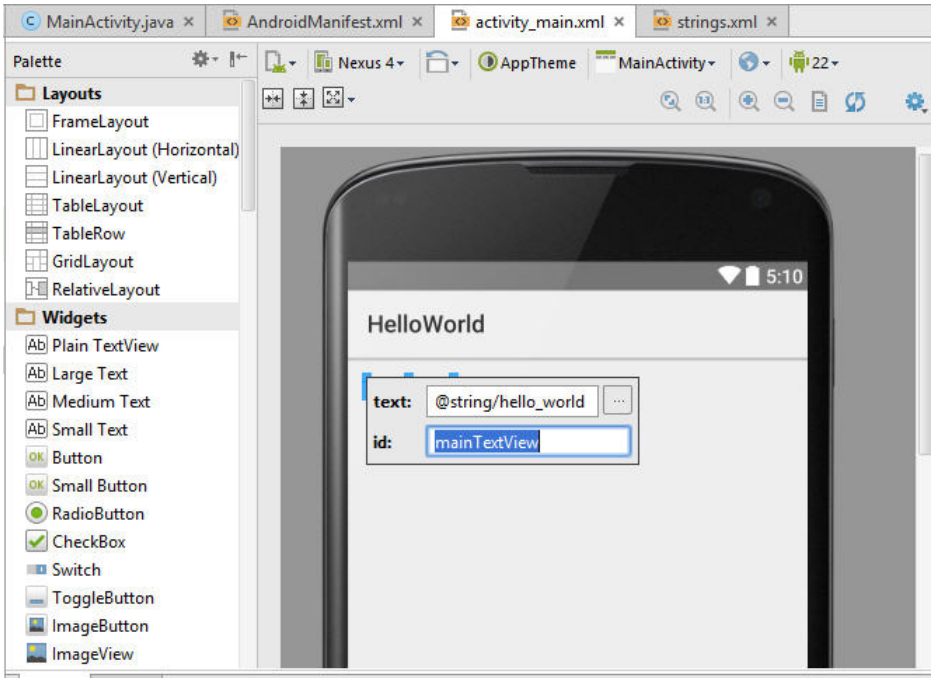


Figure 2.55: Edit Text View

3. Navigate to res → values → Strings.xml and double-click it to open for editing as shown in figure 2.56.

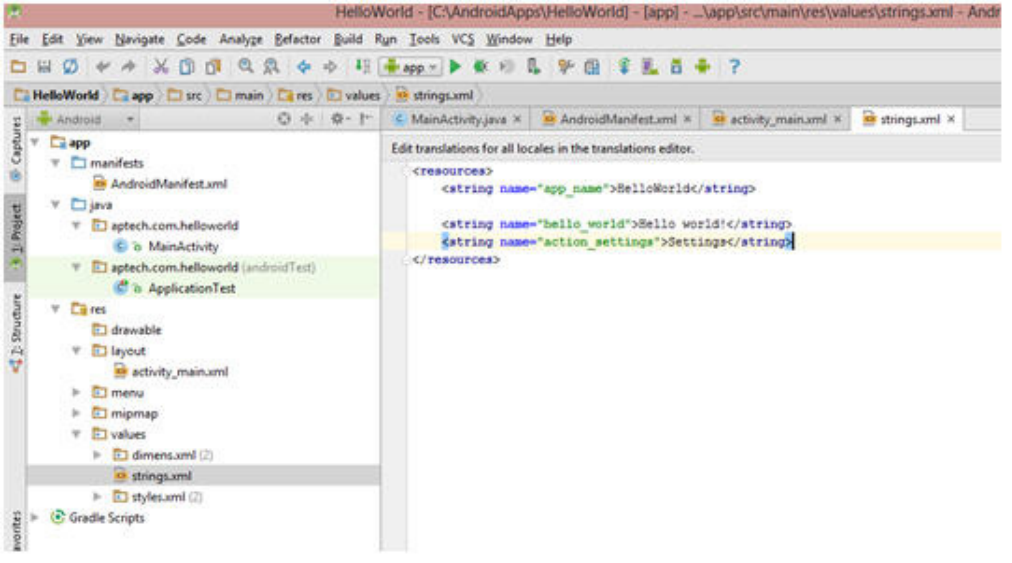


Figure 2.56: Strings Resource File

The file contains three string resources.

- **app_name** – The application name.

- **hello_world** – The default TextView text.
- **action_settings** - The settings text.

4. Modify the code as shown in Code Snippet 6.

Code Snippet 6:

```
<resources>
    <string name="app_name">HelloWorld</string>

   <string name="hello_world">Hello world! using the GUI Editor</string>
    <string name="action_settings">Settings</string>
</resources>
```

Using the Code Editor

1. Navigate to java → aptech.com.helloworld → MainActivity and double-click the file to open for editing as shown in figure 2.57.

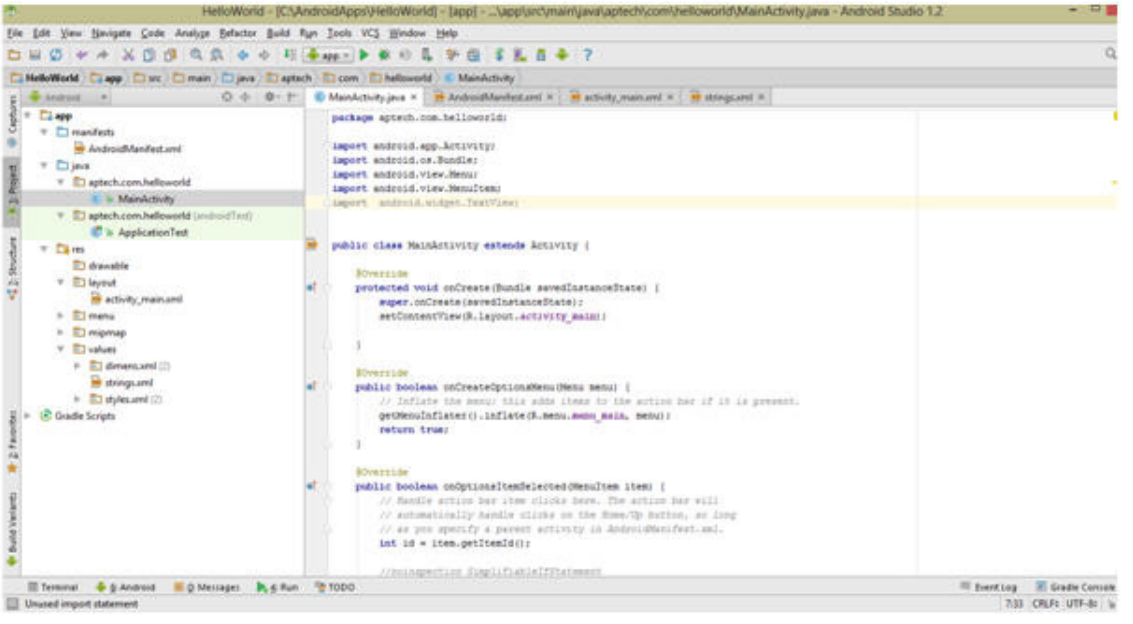


Figure 2.57: MainActivity.java

2. Modify the code as shown in Code Snippet 7.

Code Snippet 7:

```
package aptech.com.helloworld;

import android.app.Activity;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.widget.TextView;

public class MainActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        TextView view = (TextView) findViewById(R.id.mainTextView);
        view.setText("Hello World! using Code editor");
    }
@Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if it
is present.
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
        // Handle action bar item clicks here. The action bar will
        // automatically handle clicks the Home/Up button, so long
        // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
 //noinspection SimplifiableIfStatement
```

```
                if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
}
```

Note - The changes made in code will override the changes made in the GUI editor. So in this case, the text displayed will be the text set in the code editor. This is due to the fact that onCreate() method is called after the layout is initialized.

- Running the Application

1. Navigate to the Run menu and click Run 'app' as shown in figure 2.58.

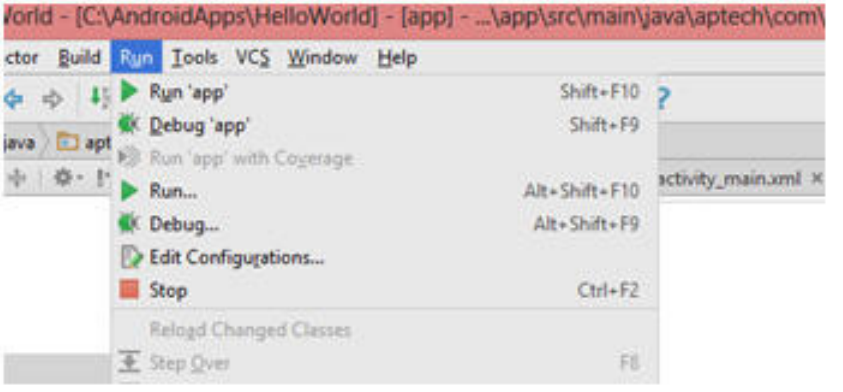


Figure 2.58: Run Menu

The Choose Device window appears as shown in figure 2.59.

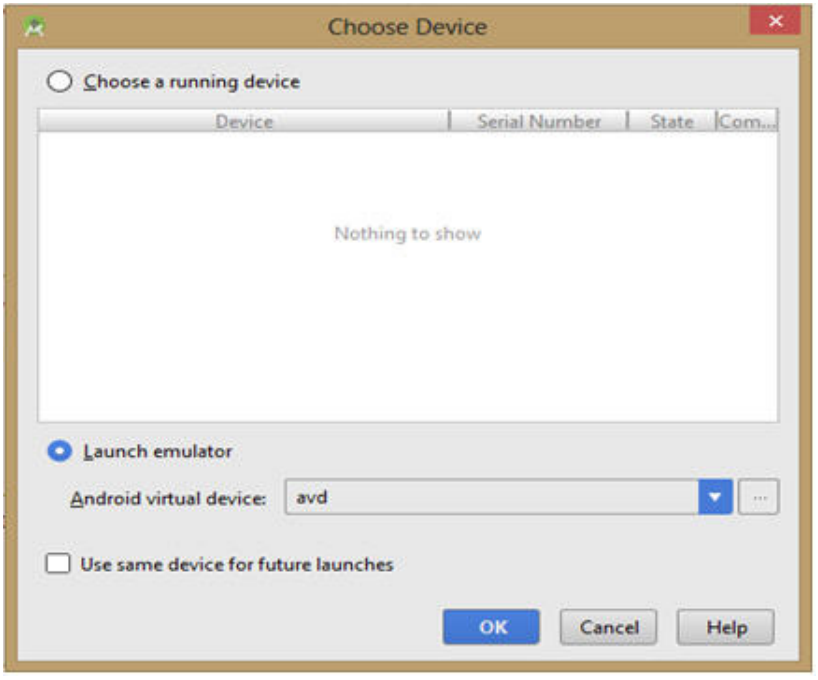


Figure 2.59: Target Device Selection

2. Select the device AVD1 that was created earlier as shown in figure 2.60.

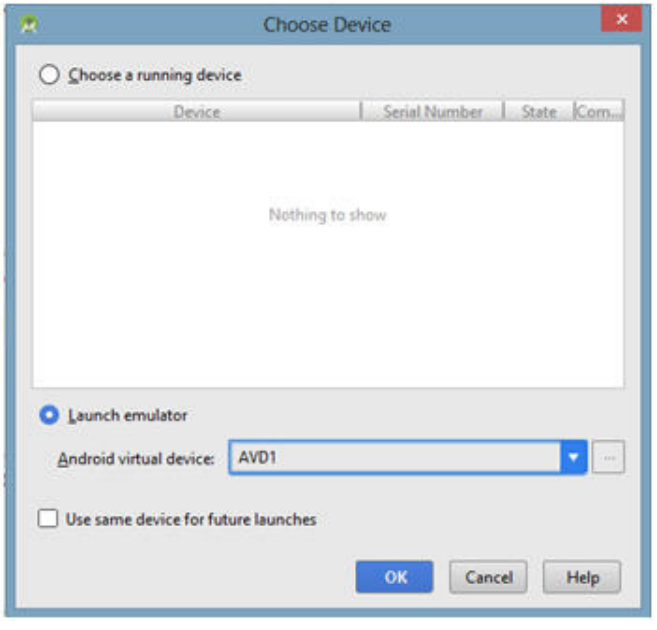


Figure 2.60: Target Device Selection – Details

Note - Here, the default configuration will be used without any changes. To change the Run and Debug configuration, follow the instructions given earlier in the session.

3. Click OK. The Android emulator launches with the application as shown in figure 2.61.

Figure 2.61: HelloWorld Application Output

2.5.3 Importing Eclipse Projects into Android Studio

Android Studio provides the facility to import existing Eclipse projects into Android Studio. The steps to be performed are as follows:

1. Navigate to File → New → Import Project or from the Home Screen select Import Project as shown in figures 2.62 and 2.63 respectively.

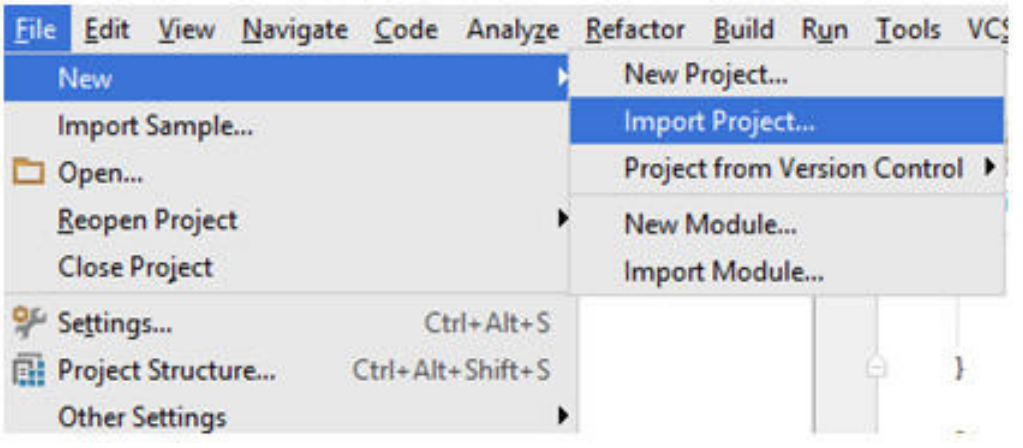


Figure 2.62: Import Project from File Menu

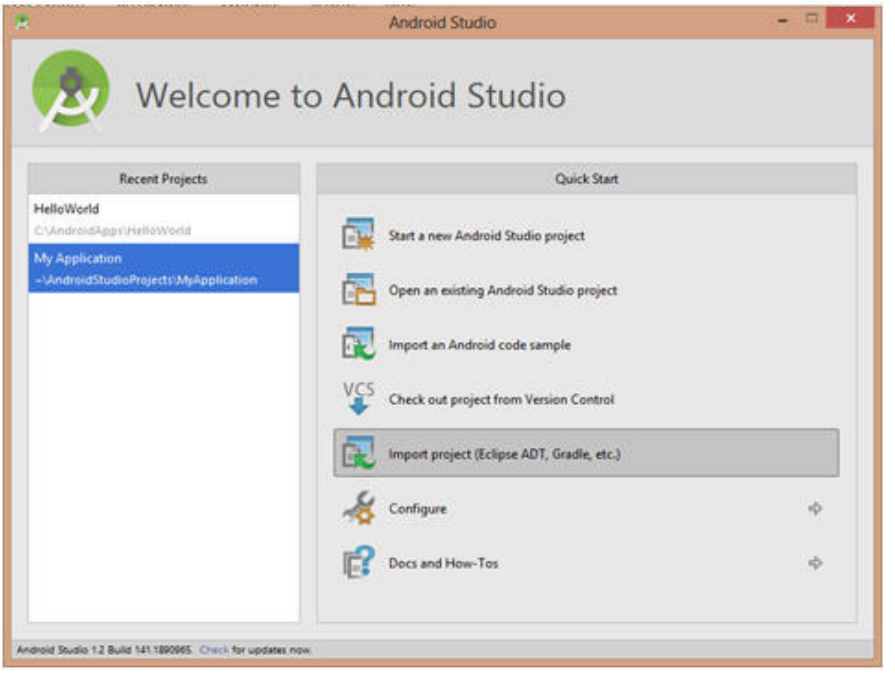


Figure 2.63: Import Project from Welcome Screen

The Project Folder selection screen appears as shown in figure 2.64.

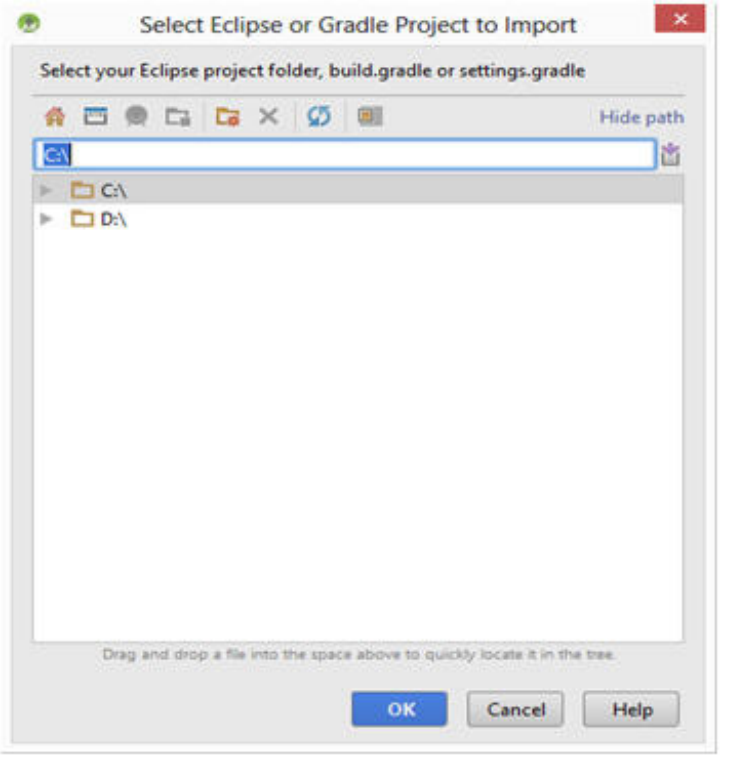


Figure 2.64: Directory Selection Screen

2. Navigate to the project directory and select the project folder as shown in figure 2.65.

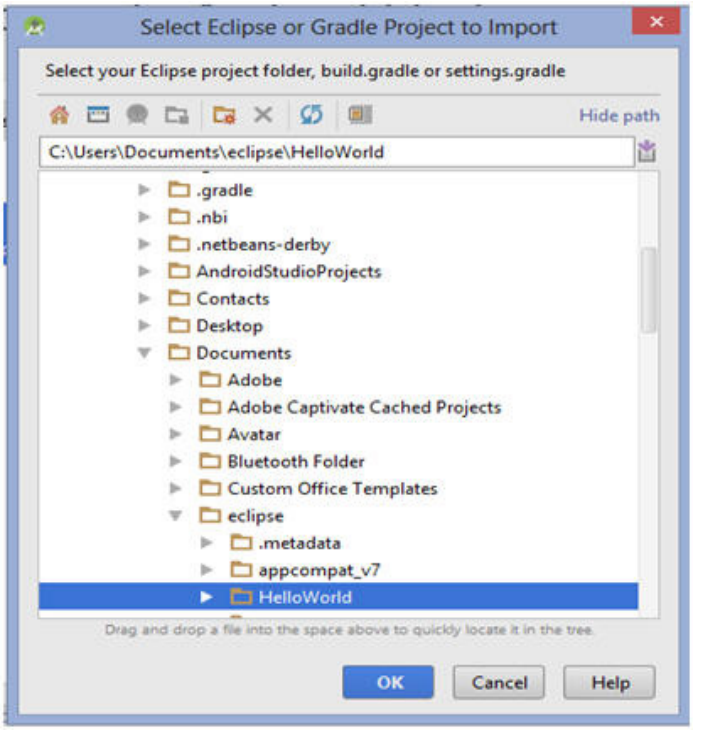


Figure 2.65: Directory Selection Screen - Details

3. Click OK.
4. The Destination Directory Window appears as shown in figure 2.66.

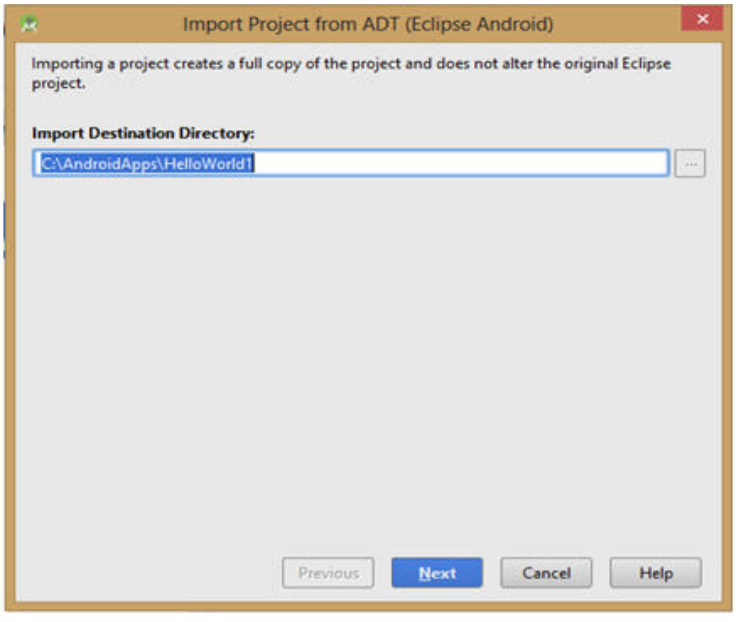


Figure 2.66: Destination Directory Screen

5. Select the destination directory and click Next as shown in figure 2.67.

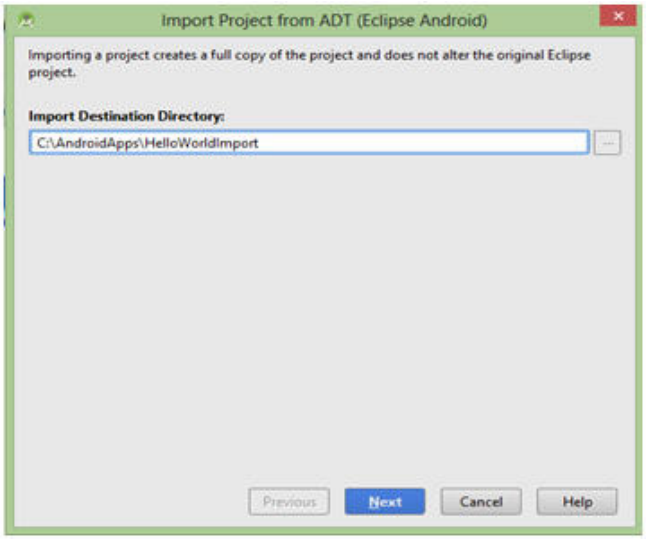


Figure 2.67: Destination Directory Screen - Details

The confirmation window appears as shown in figure 2.68.

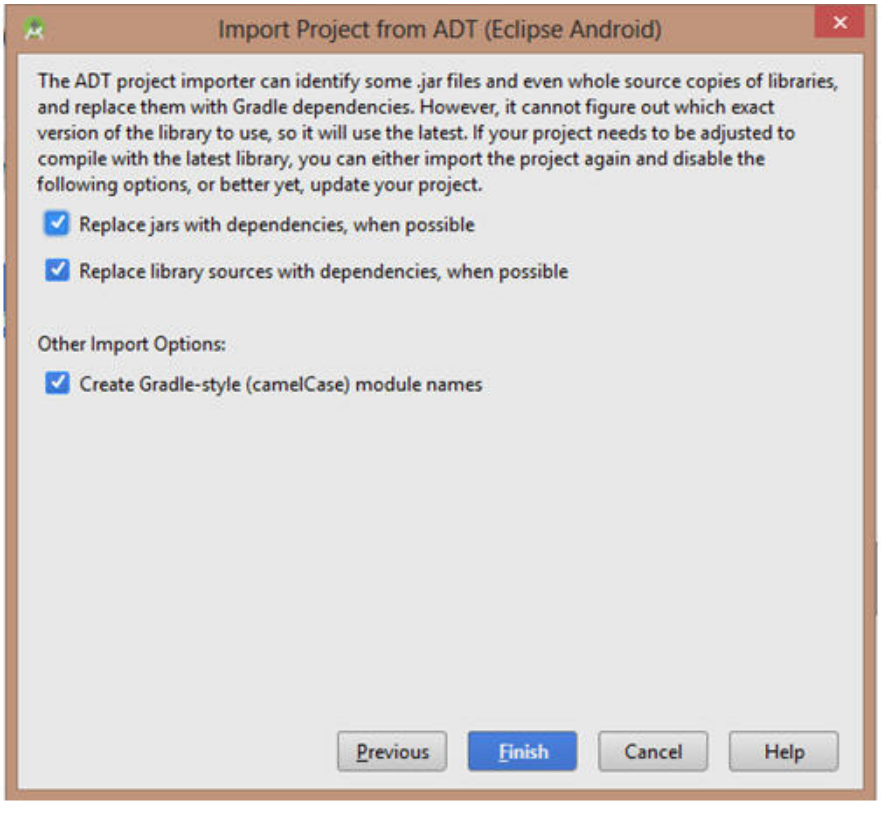


Figure 2.68: Import Project Confirmation Screen

6. Click Finish.

The project is imported and opened in Android Studio with the import summary opened as shown in figure 2.69.

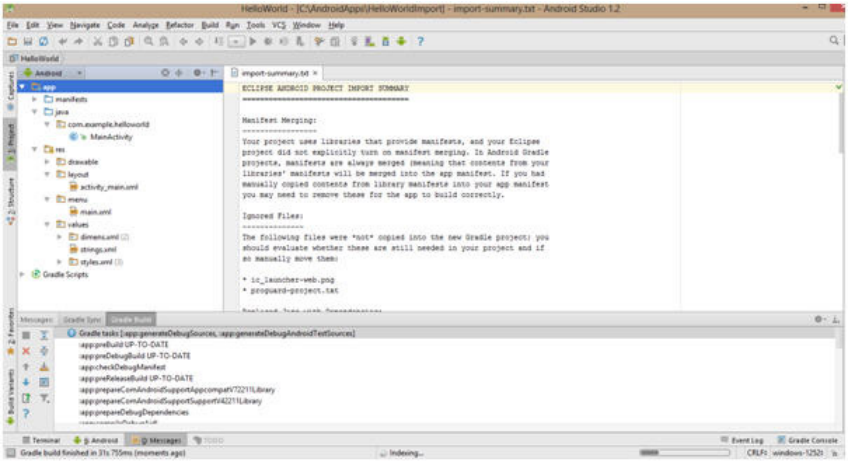


Figure 2.69: Eclipse Project Imported Into Android Studio

2.6 Deploying Applications on a Real Device

Until this point, the Android Virtual Device has been used to deploy and test the Android application. Following is the procedure to deploy the application on a real device:

Phone Setup

1. Navigate to Settings → About Phone.
2. Tap on the Build Number seven times.
3. Return back to the previous screen.
4. The Developer Options setting is available.
5. Click Developer Options.
6. Select the USB Debugging check box.

Note - On older versions of Android prior to 4.2, the Developer Options settings is directly available.

System Setup

1. Install the adb drivers for Windows.

These can be downloaded from the Website of the device manufacturer. A generic driver, intended to be used with Nexus devices is available from the installation

of the Android SDK.

2. Connect the device to your computer using a USB cable.

   The phone should popup a notification with the information that USB debugging has been enabled.

3. Start command prompt and Navigate to the Android SDK installation directory → platform-tools.
4. Use the command 'adb devices'.

   The connected Android Device should be listed here. If it is not found, repeat steps one to three.

5. Start Android Studio, open the project, and navigate to the Run menu.
6. Click Run 'app'.
7. From the device selector screen, select Choose a running device.
8. Select the connected Android Device.
9. Click OK.

   The application will be installed and started on the device automatically.

Check Your Progress

1. Match following descriptions with the corresponding storage location.

	Description		Storage location
a.	A folder where the images be placed.	1.	lib
b.	Files where the labels need to be stored.	2.	/res/values/strings.xml
c.	Folder from where the Menu entries loaded.	3.	/res/drawable/
d.	Folder where the dependent libraries are added.	4.	/res/menu//

(A)	a-1, b-3, c-4, c-2	(B)	a-3, b-1, c-4, d-2
(C)	a-4, b-2, c-3, d-1	(D)	a-1, b-4, c-3, d-2

2. Which of the following file or file(s) holds the target Android version information?

1.	AndroidManifext.xml
2.	BuildConfig.java
3.	Project.properties
4.	MainActivity.xml

(A)	2, 3	(B)	1, 3
(C)	1, 4	(D)	2, 4

3. The _____ component of Android is used to display the UI.

(A)	Services	(B)	Activities
(C)	Broadcast Receivers	(D)	Content Providers

Check Your Progress

4. Which of the file or file(s) is packaged into an apk as it is?

1.	Activity_main.xml
2.	asset
3.	String.xml
4.	AndroidManifest.xml

(A)	1, 2, 4	(B)	1, 2
(C)	1, 4	(D)	2, 4

5. Where are the layout files located in the Android project?

(A)	res	(B)	src
(C)	Lib	(D)	assets

Answers

1.	B
2.	B
3.	B
4.	D
5.	A

Summary

- An Android project consists of all the files and resources required to build the project into an .apk file for installation.
- Activities in Android are UI Screens. Layouts are used to define the UI of these screens.
- Services in Android are background processes with no UI.
- Content providers are used for managing shared data sets.
- Broadcast Receiver receives or responds to announcements broadcast by the system.
- Resources and assets are used to hold static content of the application.
- Applications can be tested on AVDs or Real Devices.

Onlinevarsity

INDUSTRY BEST PRACTICES

SYNC WITH THE INDUSTRY

03 Android System Overview

Welcome to the Session, **Android System Overview.**

This session provides an overview of the Android security system, preferences, file system, and notifications. These are important aspects which the developer has to consider for developing an Android application.

In this Session, you will learn to:

- Explain Preferences
- Explain Shared Preferences Storage Structure
- Explain the Android File System
- Explain Notifications

Session

03

Android System Overview

3.1 Introduction

An Android developer needs to be aware of setting preferences, saving files, sending different types of notifications, and Android security model. While developing applications, one also needs to provide the Android user with options to customize their applications. This can be done by setting preferences.

While building different types of applications, there might be a requirement to read and write a large amount of data to and from files. Further, one needs to know about the different types of notifications and the ways to display them. To implement security in the application, the Android security model is used which helps to comprehend concepts, such as user permissions, password protection, and application security.

3.2 Android System Overview

There are several options in Android to save persistent application data. The solution you choose depends upon the requirement of the project such as whether data should be private to the application or accessible to other applications also. Following are the options which can be used for storing the data:

- **Shared Preferences**: Store data as key-value pairs.
- **Network Connection**: Store data on the Web with your own network server.
- **Internal Storage**: Store data on the device memory. Data will not be accessed by other application.
- **External Storage**: Store public data on the shared external storage. Data will be accessed by other application.
- **SQLite Databases**: Store structured data in a private database.
- **Network Connection**: Store data on the Web with your own network server.

Shared preferences (android.content.SharedPreferences) can be used to easily store Application data. The android.preference package provides classes to manage and implement UI preferences. This ensures that the user experience with applications remains consistent. The Preference subclass can be used to determine settings that enable users to customize their applications. There are two ways in which a developer can specify preferences. The first way is to create an XML file. The developer can also specify preferences directly in the code. The Preference object is used as the building block for a single setting. Each of the preference settings is saved as a key-value pair in the default SharedPreferences file for the app's settings.

The three most commonly used Preferences are as follows:

- **CheckBoxPreference**: CheckBoxPreference is used to store data in preferences using CheckBox widget. In other words, it provides the CheckBox widget functionality. It returns a true or false value as shown in figure 3.1.

Figure 3.1: CheckBoxPreference

- **ListPreference**: ListPreference is used to display the list of entries as a dialog from which a user can choose a single selection. This displays a group of radio buttons as shown in figure 3.2 among which only one button can be selected.

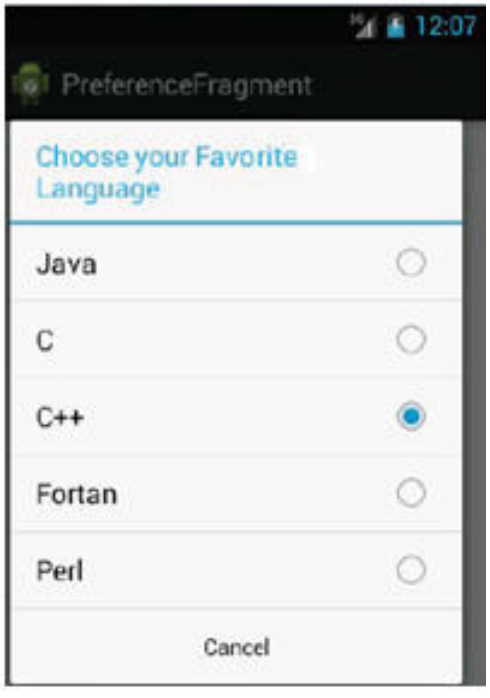


Figure 3.2: ListPreference

- **EditTextPreference**: This opens up the dialog box where user can enter a value as shown in figure 3.3. When text is typed into the field, it returns a string value.

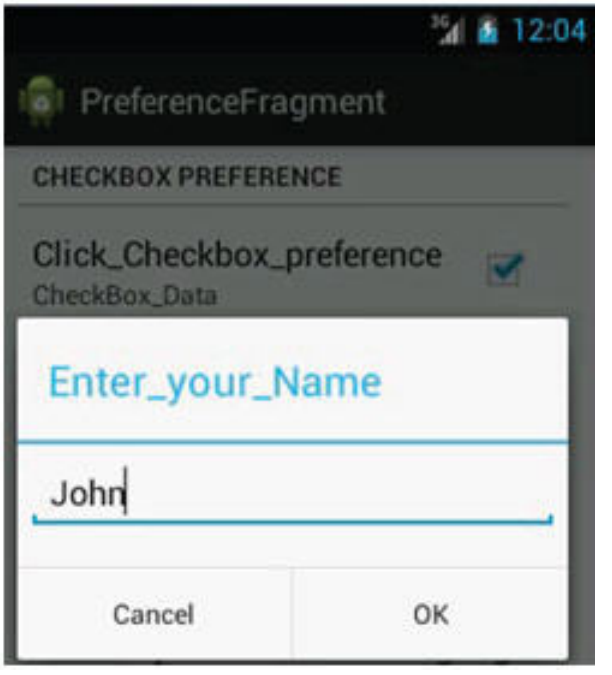


Figure 3.3: EditTextPreference

In the following example, an application is created where different types of preferences are integrated.

The steps to create the application are as follows:

1. Create a project named PreferenceFragment in Android Studio.
2. Navigate to app → res.
3. Right-click and select New → Android Resource Directory.
4. Select resource type as xml and select the name as xml.
5. Right-click the newly created xml directory. Select New → File.
6. Create an .xml file, to define the preference under/res/xml named preferences.xml.
7. Add the code to preferences.xml file as shown in Code Snippet 1.

Code Snippet 1:

```
<?xml version="1.0"encoding="UTF-8"?>
<PreferenceScreen xmlns:android="http://schemas.android.com/apk/
res/android">
<PreferenceCategory android:title="Checkbox Preference">
```

```
<CheckBoxPreference android:key="checkbox_preferencevalue"
android:summary="CheckBox_Data"
android:title="Click_Checkbox_preference"/>
<PreferenceCategory android:title="EditTextPreference">
<EditTextPreference android:dialogTitle="Enter_your_Name"
android:key="edittext_preferencevalue"
android:summary="summary_edittext_preference
android:title="title_edittext_preference"/>
</PreferenceCategory>
<PreferenceCategory android:title="ListView Preference">
<ListPreference android:entries="@array/array_lp_entries"
android:entryValues="@array/array_lp_entry_values"
android:key="lp_android_choice"
android:title="@string/str_lp_android_choice_title"/>
</PreferenceCategory>
</PreferenceScreen>
```

8. Right-click menu folder. Select New → Menu Resource File. Create an XML file to define the menu in the res → menu → main.xml file.
9. Modify the code as shown in Code Snippet 2 in main.xml file.

Code Snippet 2:

```
<menu xmlns:android="http://schemas.android.com/apk/res/
android">
<item
android:id="@+id/action_settings"
android:orderInCategory="100"
android:showAsAction="never"
android:title="@string/action_settings"/>
</menu>
```

10. Navigate to res → values → strings.xml file.

11. Modify the code in strings.xml file as shown in Code Snippet 3.

Code Snippet 3:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">PreferenceFragment</string>
     <string name="action_settings">Preference Example</
string>
    <string name="hello_world">Hello world!</string>
  <string name="str_lp_android_choice_title">Choose your
Favorite Language</string>
    <string-array name="array_lp_entries">
            <item>Java</item>
            <item>C</item>
            <item>C++</item>
            <item>Fortan</item>
            <item>Perl</item>
    </string-array>
    <string-array name="array_lp_entry_values">
<item>Java</item>
            <item>C</item>
            <item>C++</item>
            <item>Fortan</item>
            <item>Perl</item>
</string-array>
</resources>
```

12. Create a Java class named PrefsFragment.java under the src → com.example.preferencefragment folder.

13. Add the code to PrefsFragment.java file as shown in Code Snippet 4.

Code Snippet 4:

```
package com.example.PreferenceFragment;
import android.app.Fragment;
import android.os.Bundle;
import android.preference.PreferenceFragment;
public class PrefsFragment extends PreferenceFragment {
    @Override
    public void onCreate(Bundle savedInstanceState) {
    // TODO Auto-generated method stub
       super.onCreate(savedInstanceState);
    // Load the preferences from an XML resource
     addPreferencesFromResource(R.xml.preferences);
   }
}
```

14. Create a Java class named SetPreferenceActivity under src → com.example.preferencefragment folder.

15. Add the code to SetPreferenceActivity.java file as shown in Code Snippet 5.

Code Snippet 5:

```
package com.example.PreferenceFragment;
import android.app.Activity;
import android.os.Bundle;
public class SetPreferenceActivity extends Activity{
             @Override
            protected void onCreate(Bundle savedInstanceState) {
            // TODO Auto-generated method stub
            super.onCreate(savedInstanceState);
            getFragmentManager().beginTransaction().
replace(android.R.id.content, new
PrefsFragment()).commit();
}
}
```

16. Navigate to src → com.example.preferencefragment → MainActivity.java file.
17. Add the code to MainActivity.java file as shown in Code Snippet 6.

Code Snippet 6:

```
package com.example.PreferenceFragment;
import android.os.Bundle;
import android.preference.PreferenceManager;
import android.app.Activity;
import android.content.Intent;
import android.content.SharedPreferences;
import android.view.Menu;
import android.view.MenuItem;
import android.widget.CheckBox;
import android.widget.TextView;
public class MainActivity extends Activity {
CheckBox prefCheckBoxvalue;
 TextView prefEditTextvalue,prefEditTextvalueone;

@Override
protected void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
prefCheckBoxvalue = (CheckBox)findViewById(R.
id.prefCheckBoxvalue);
 prefEditTextvalue = (TextView)findViewById(R.
id.prefEditTextvalue);
prefEditTextvalueone=TextView)findViewById(R.id.prefEditTextvalueone);
loadPrefvalue();
}

@Override
public boolean onCreateOptionsMenu(Menu menu) {
```

```
// Inflate the menu; this adds items to the action bar if it
is present.
getMenuInflater().inflate(R.menu.main, menu);
 return true;
}
@Override
 public boolean onOptionsItemSelected(MenuItem item) {
 Intent intent = new Intent();
 intent.setClass(MainActivity.this,
 SetPreferenceActivity.class);
 startActivityForResult(intent, 0);
 return true;
 }
 @Override
 protected void onActivityResult(int requestCode, int
resultCode, Intent data) {
 loadPrefvalue();
 }
 private void loadPrefvalue(){
 SharedPreferences mySharedPreferences =
PreferenceManager.getDefaultSharedPreferences(this);
boolean my_checkbox_preference = mySharedPreferences.
getBoolean("checkbox_preferencevalue", false);
 prefCheckBoxvalue.setChecked(my_checkbox_preference);
String my_edittext_preference = mySharedPreferences.
getString("edittext_preferencevalue", "");
prefEditTextvalueone.setText("ListBox Preference:"+my_edittext_
preference1);
```

```
edittext_preference);
String my_edittext_preference1 = mySharedPreferences.
getString("lp_android_choice", "");
prefEditTextvalueone.setText("ListBox Preference:"+my_
edittext_preference1);
        }
}
```

18. Navigate to AndroidManifest.xml file.
19. Add the code to AndroidManifest.xml file as shown in Code Snippet 7.

Code Snippet 7:

```
<activity android:name="com.example.preferencefragment.
SetPreferenceActivity">
</activity>
```

AndroidManifest.xml file will contain the code as shown in Code Snippet 8.

Code Snippet 8:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/
apk/res/ android"
package="com.example.PreferenceFragment"
android:versionCode="1"
android:versionName="1.0" >
<uses-sdk
android:minSdkVersion="8"
android:targetSdkVersion="17" />
<application
android:allowBackup="true"
android:icon="@drawable/ic_launcher"
android:label="@string/app_name"
android:theme="@style/AppTheme" >
```

```
<activity
android:name="com.example.PreferenceFragment.
MainActivity"
android:label="@string/app_name" >
<intent-filter>
<action android:name="android.intent.action. MAIN" />
<category android:name="android.intent.category. LAUNCHER"/>
</intent-filter>
</activity>
<activity
android:name=".SetPreferenceActivity" >
</activity>
</application>
</manifest>
```

Execute the application to get the output as shown in figure 3.4.

Figure 3.4: Preference Fragment

Click Menu to display the output as shown in figure 3.5.

Figure 3.5: Different Types of Preferences

Click Preference Example to display the output as shown in figure 3.6.

Figure 3.6: Preference Options

Click 'Choose your Favorite Language' to display the output as shown in figure 3.7.

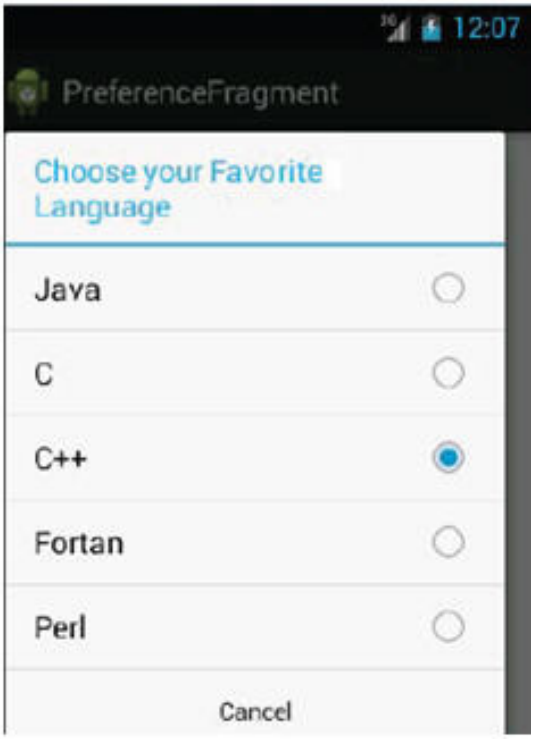


Figure 3.7: Choose Favorite Language ListPreference

Click title_edittext_preference, to display the output as shown in figure 3.8.

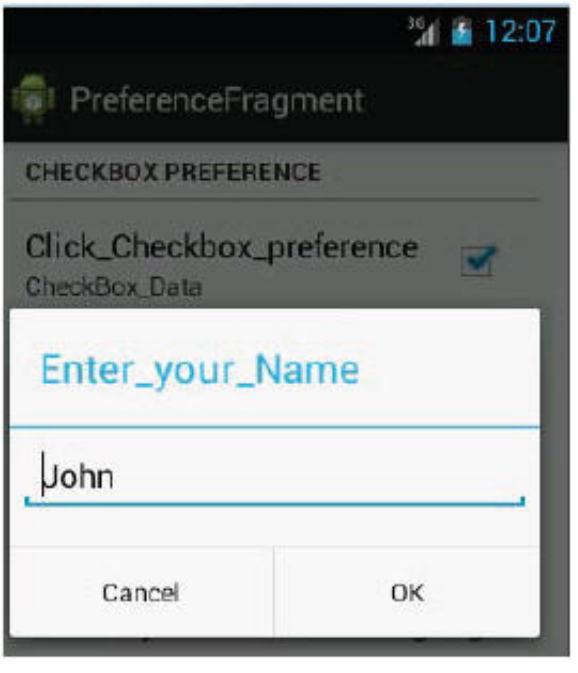


Figure 3.8: EditTextPreference

As seen in Code Snippet 1, the developer creates an XML file and specifies the preferences in the file. The XML file provides a simple way of specifying preferences with a structure that is easily understood and managed.

The XML contains a list of preference objects and preference subclasses.

The <PreferenceScreen> element refers to the top most level of a hierarchy and forms the basis of a preference hierarchy. In other words, the root element of the XML file is <PreferenceScreen> element. Each XML element can be matched to the class name and will have a preference subclass assigned to it, such as <CheckBoxPreference>. Every child that is added within the <PreferenceScreen> element exists as a single item within the list of settings.

Users may find it difficult to go through a list of 10 or more settings. In such cases, the settings can be grouped. Within each <PreferenceScreen> element, any combination of <PreferenceCategory> and Preference <control> elements can be included as shown in Code Snippet 9.

Code Snippet 9:

```
<?xml version="1.0" encoding="UTF-8"?>
<PreferenceScreen xmlns:android="http://schemas.android.
com/apk/ res/android" >
<PreferenceCategory android:title="CheckboxPreference" >
<CheckBoxPreference
android:key="checkbox_preferencevalue"
android:summary="CheckBox_Data"
android:title="Click_Checkbox_preference" />
</PreferenceCategory>
<PreferenceCategory android:title="EditTextPreference" >
<EditTextPreference
android:dialogTitle="Enter_your_Name"
android:key="edittext_preferencevalue"
android:summary="summary_edittext_preference"
android:title="title_edittext_preference" />
</PreferenceCategory>
<PreferenceCategory android:title="ListViewPreference">
<ListPreference
android:entries="@array/array_lp_entries"
```

```
android:entryValues="@array/array_lp_entry_ values"
android:key="lp_android_choice"
android:title="@string/str_lp_android_choice_ title" />
</PreferenceCategory>
</PreferenceScreen>
```

The XML file is typically named as preferences.xml and should be saved in the res/xml/ directory. For a single-panel layout, it is sufficient to create a single XML file. For multiple pane layouts, more than one XML file is required for specifying preferences. Code Snippet 9 shows settings for three preferences - Checkbox, Editbox, and List preference. The attributes that can be defined for these preferences are as follows:

- **android:key** - This attribute is mandatory data value. It provides a distinct key for the system to save the data value. The data value is saved in the SharedPreferences file. The key is the unique value used to access the Preference.
- **android:title** - This attribute specifies a name for the setting which is visible to the user.
- **android:defaultValue** - The default value that will be displayed (and selected) if no preference value has been assigned to this preference key.

3.2.1 Dividing Settings into Groups

When there are more than ten settings to specify, the developer can divide the settings into groups. There are three ways in which group settings can be specified. These are by using titles, Intents and sub screens.

- **Titles**: The developer can provide headings between groups of settings in the XML file so as to identify each group. To do this, each group of preference objects should be placed inside the <PreferenceCategory> element as shown in Code Snippet 10.

Code Snippet 10:

```
<PreferenceCategory android:title="ListViewPreference" >
<ListPreference
android:entries="@array/array_lp_entries"
android:entryValues="@array/array_lp_entry_values"
android:key="lp_android_choice"
android:title="@string/str_lp_android_choice_title" />
</PreferenceCategory>
```

- **Sub screen**: The developer can also place settings groups within a sub screen. To do this, the preference objects group needs to be inside the <PreferenceScreen> element. When a user makes changes to a preference, the changes are saved to the SharedPreferences file.

3.2.2 Providing Default Values

It is important to provide a default value for each preference object in the XML file. The SharedPreferences file is initialized using the default values specified when the user first accesses the application. The default value is specified using the android: default value attribute in the XML preference file. The value specified depends on the preference object as shown in Code Snippet 11.

Code Snippet 11:

```
<PreferenceCategory
android:title="CheckboxPreference">
<CheckBoxPreference
android:key="checkbox_preferencevalue"
android:title="Click_Checkbox_preference"
android:summary="CheckBox_Data"
android:defaultValue="true"/>
</PreferenceCategory>
```

The developer has to ensure that the default values are assigned by invoking the setDefaultValues() method when the user first accesses the application. Thus, the method is invoked in the onCreate() method of the main Activity class. The application will start with the default settings.

The setDefaultValues() method accepts three arguments. They are as follows:

- The context for the application.
- The resource identity for the XML file for which the default values are being set.
- A boolean value that determines the number of times the default values are set. If it is set to false, the default values are called only if the values have not been called before. If set to true, previous default values will be ignored. Code Snippet 12 demonstrates the use of the setDefaultValues() method.

Code Snippet 12:

```
PreferenceManager.setDefaultValues(this,R.xml.user_profile,
false);
```

3.2.3 Saving Data in Shared Preference

Following application shows how to create and store the shared preference data from the code. The steps to create the application are as follows:

1. Create a project named SharePreferenceExample in Android Studio.

2. Navigate to res → layout → activity_main.xml file.
3. Modify the code of activity_main.xml file as shown in Code Snippet 13.

Code Snippet 13:

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/
res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical" >
<EditText
android:id="@+id/editText"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:ems="10" >
<requestFocus />
</EditText>
<Button
android:id="@+id/button"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_below="@+id/editText"
android:layout_centerHorizontal="true"
android:layout_marginTop="18dp"
android:text="Submit" />
<Button
android:id="@+id/buttonShow"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_centerHorizontal="true"
android:layout_centerVertical="true"
android:text="Show Stored Data" />
```

```
<TextView
android:id="@+id/textViewData"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_below="@+id/button"
android:layout_centerHorizontal="true"
android:layout_marginTop="41dp"
android:text=""
android:textAppearance="?android:attr/textAppearanceLarge" />
</RelativeLayout>
```

4. Navigate to src → com.example.sharedpreferenceexample → MainActivity.java file.
5. Modify the code of MainActivity.java file as shown in Code Snippet 14.

Code Snippet 14:

```
package com.example.sharedpreferenceexample;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.content.SharedPreferences;
import android.preference.PreferenceManager;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;
public class MainActivity extends Activity {
SharedPreferences preferences;
SharedPreferences.Editor editor;
EditText editText;
Button submit;
```

```
Button show;
TextView dataText;
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
preferences = PreferenceManager
.getDefaultSharedPreferences(MainActivity.this);
editText = (EditText) findViewById(R.id.editText);
submit = (Button) findViewById(R.id.button);
show = (Button) findViewById(R.id.buttonShow);
dataText = (TextView) findViewById(R.id.textViewData);
submit.setOnClickListener(new OnClickListener() {
@Override
public void onClick(View arg0) {
// TODO Auto-generated method stub
editor = preferences.edit();
editor.putString("data", editText.getText(). toString());
editor.commit();
Toast.makeText(getApplicationContext(),
"Data saved in preferences", Toast.LENGTH_LONG). show();
}
});
show.setOnClickListener(new OnClickListener() {
@Override
public void onClick(View arg0) {
// TODO Auto-generated method stub
String data = preferences.getString("data", "");
```

```
if (!data.equals(“”)) {
dataText.setText(data);
} else {
Toast.makeText(getApplicationContext(),
“No Data Available”, Toast.LENGTH_ LONG).show();
}
}
});
}
@Override
public boolean onCreateOptionsMenu(Menu menu) {
// Inflate the menu; this adds items to the action bar if it
is present.
getMenuInflater().inflate(R.menu.main, menu);
return true;
}
}
```

When the application is executed the output will be as shown in figure 3.9.

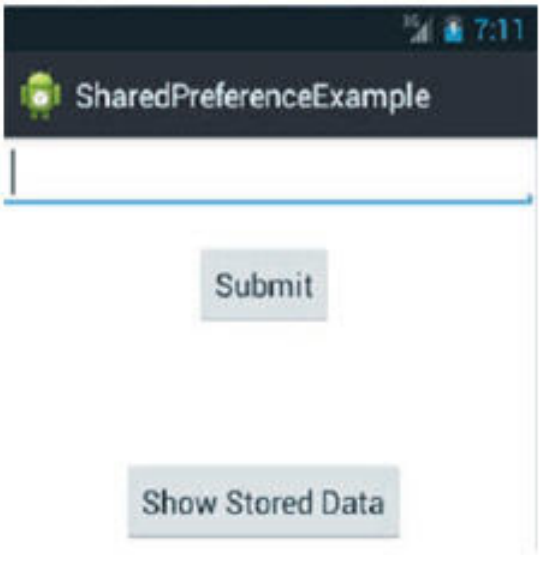


Figure 3.9: Shared Preference

On typing the content in the edit box and clicking Submit, the output will be as shown in figure 3.10.

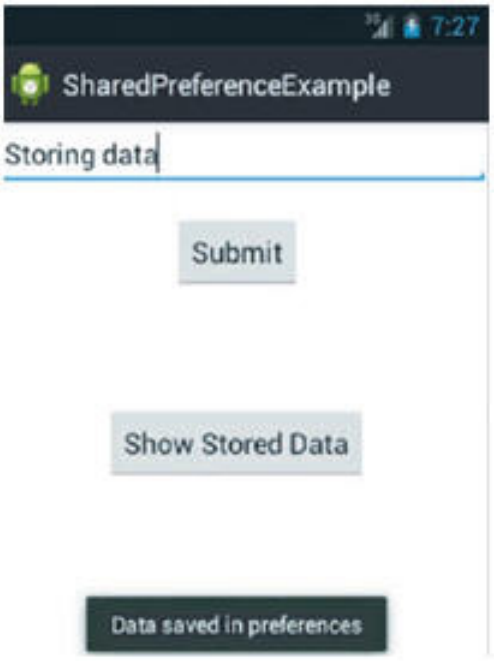


Figure 3.10: Enter Data to Store

On clicking the Show Stored Data button, the output will be as shown in figure 3.11.

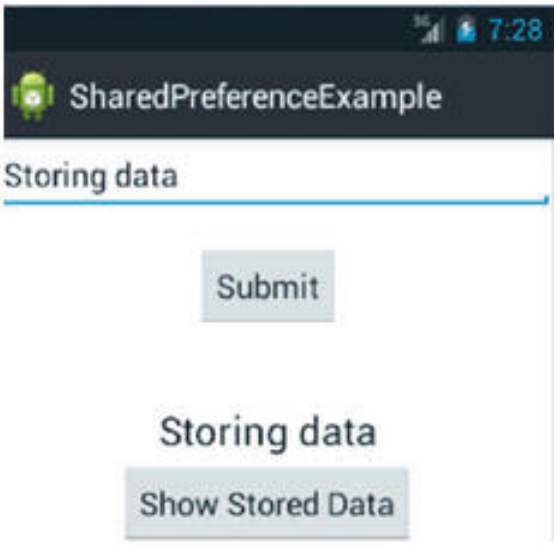


Figure 3.11: Displaying Data from SharedPreference

3.3 File System

Android is characterized by two storage options namely, internal and external. Android's internal file system utilizes the ext4 file system format by default. Android natively supports reading FAT32, ext3, and ext4 file systems. Third party tools have added support for NTFS and exFAT file systems.

The developer can read and write files using the File APIs. Large chunks of data can be viewed and written without interruption using a file object. Typically, the built-in memory is internal storage, while removable memory is external storage. The current market trend is moving away from external or expansion slot based store. The number of manufacturers providing devices with high internal storage and no scope for expansion is rising.

However, from a developer perspective, this does not affect how the applications are developed. Almost all implementations, the existing storage space is partitioned into internal and external storage. Irrespective of whether the removable storage exist or not. Hence, the API activities remain the same.

Each application has its own internal file system and the application does not need to ask for permission to read or write into the internal file system. Any files stored in the internal file system are only available to that app. All the data is removed once the application is removed or if the user selects clear app data from the Settings Menu → Applications.

The manifest file must explicitly specify permission to write to external storage. Any data stored here can be accessed by all the applications and the user. The system removes the application related files, when the application is removed, only if the files are saved in the application data directory. This directory can be retrieved using the getExternalFilesDir().

3.3.1 Internal Storage

Developers would sometimes like to store data in traditional file system. These files will be saved on the device's internal storage. The data saved by this method cannot be accessed by other applications. The data can be saved using the classes present in the java.io package.

Features of internal storage are as follows:

- More secure because only the related application can access files stored.
- Files are always accessible.
- When an application is uninstalled from the system, all related files are removed from internal storage.

In order to read and write to the internal storage, the user needs to reference file using the File class. Reading or writing can be done on the File reference using one of the Stream classes as explained here.

File Class

The internal storage directory for saving files can be obtained by using, the getFilesDir() and getCacheDir() methods. These getCacheDir() returns the path to the cache files whereas getFilesDir() returns the path to files created and stored in the internal storage of your app.

Code Snippet 15 explains the use of this method.

Code Snippet 15:

```
File srDir=newFile(getFilesDir(), "file_name");
srDir.mkdir();
```

Some of the methods of File class are as follows:

- **mkdir()**: Used to create a directory if the parent exists.
- **mkdirs()**: Used to create a new directory along with the parent path.
- **renameTo()**: Used to rename a file.
- **isFile()**: Returns true if the object references a File.
- **isDirectory()**: Returns true if the object references a directory.
- **delete()**: Deletes the file or directory.
- **exists()**: Used to check if the file or directory exists.
- **getAbsolutePath()**: Returns an absolute path to the reference.
- **list()**: Return lists of files and subdirectories if the reference is a directory.
- **canRead()/canWrite()**: Returns true if the reference is readable or writable respectively.

Stream Classes

To create and write data to a file saved in the internal storage, perform the following steps:

1. Obtain an object of the FileOutputStream class by invoking the openFileOutput() method which accepts the filename and the operating mode. The operating mode can be MODE_APPEND, MODE_WORLD_READABLE, MODE_WORLD_WRITABLE, or MODE_APPEND.
2. Write to a file using the write() method of the input stream.
3. Invoke the close() method to close the stream.

Code Snippet 16 explains the creation and writing of data to a private file which will be present in the internal storage of the Android system.

Code Snippet 16:

```
...
String FILENAME = "android.txt";
String string = "Learning Android programming!!";
FileOutputStream fos = openFileOutput(FILENAME, Context.MODE_ PRIVATE);
fos.write(string.getBytes());
fos.close();
 ...
```

To read data from a file saved in the internal storage, perform the following steps:

- Obtain an object of the FileInputStream class by invoking the openFileInput() method which accepts the filename.
- Read the data using the read() method.
- Invoke the close() method to close the stream as shown in Code Snippet 17.

Code Snippet 17:

```
...
File file=newFile(getAbsolutePath(), "android.txt");
FileInputStream fis = openFileinput(file);
String text = fis.read();
fis.close();
…
```

3.3.2 Working with Storing Files in Internal Storage

This section explains the procedure to store files to the internal storage of an Android device. In this example, a new project will be created with the ability to create new files, view existing files, and navigate the file system. Navigation is achieved through EditTexts. File contents are displayed in a TextView.

The steps to store files to the internal storage are as follows:

1. Create a project InternalFS in Android Studio.
2. Navigate to app → res → layout → activity_main.xml file in the Package Explorer.

3. Double-click activity_main.xml.
4. Modify the code of the activity_main.xml file as shown in Code Snippet 18.

Code Snippet 18:

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
    android:layout_height="match_parent" android:paddingLeft="@
dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
tools:context=".MainActivity">
    <TextView
       android:layout_width="wrap_content"
       android:layout_height="wrap_content"
       android:textAppearance="?android:attr/
                               textAppearanceLarge"
       android:text="Click a button to start"
       android:id="@+id/textView"
       android:layout_alignParentTop="true"
       android:layout_alignParentStart="true"
       android:layout_alignParentBottom="true"
       android:layout_alignParentEnd="true" />

   <Button
       android:layout_width="wrap_content"
       android:layout_height="wrap_content"
       android:text="List Files"
       android:id="@+id/button2"
```

```
        android:layout_alignBottom="@+id/textView
        android:onClick="listFiles"
        android:layout_alignParentStart="true" />

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="New File"
        android:id="@+id/button"
        android:onClick="newFile"
        android:layout_alignParentBottom="true"
        android:layout_alignEnd="@+id/textView" />

      <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Open File"
        android:id="@+id/button3"
        android:onClick="openFile"
        android:layout_alignParentBottom="true"
        android:layout_centerHorizontal="true" />
</RelativeLayout>
```

5. Navigate to app → java → aptech.com.internalfs and open MainActivity.java for editing.
6. Modify the code as shown in Code Snippet 19.

Code Snippet 19:

```
package aptech.com.internalfs;
import android.app.Activity;
import android.app.AlertDialog;
import android.content.DialogInterface;
import android.os.Bundle;
import android.text.InputType;
import android.util.Log;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.EditText;
import android.widget.LinearLayout;
import android.widget.TextView;

import java.io.BufferedInputStream;
import java.io.BufferedReader;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.FileReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.PrintWriter;
public class MainActivity extends Activity {
    String userInput;
    @Override
```

```
protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
  // Inflate the menu; this adds items to the action bar if it is
//present.
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
      // Handle action bar item clicks here. The action bar will
     // automatically handle clicks on the Home/Up button, so long
     // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();

        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
    public void listFiles(View view) {
      loadFileList();
    }
    public void newFile(View view) {
```

```
getUserInput();
}
public void openFile(View view) {
    getFileName();
}
private void writeFile(String fileName, String fileContent) {

    File file = new File(getFilesDir(),fileName);

    try {
        FileOutputStream fs = new FileOutputStream(file);
        PrintWriter pw = new PrintWriter(fs);
        pw.println(fileContent);
        pw.flush();
        pw.close();
        fs.close();
    } catch (FileNotFoundException e) {
        e.printStackTrace();
    }
    catch (IOException e) {
        e.printStackTrace();
    }
}

private String readFile(String fileName){
    StringBuilder content = new StringBuilder();
   File file = new File(getFilesDir(),fileName);
    try {
```

```
BufferedReader br = new BufferedReader(new FileReader(file));
            String line;
            while ((line = br.readLine()) != null) {
                content.append(line);
                content.append("\n");
            }
            br.close();
        }
        catch (FileNotFoundException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }

        return content.toString();
    }
    private void getUserInput(){
      final AlertDialog.Builder alert = new AlertDialog.
Builder(this);

        LinearLayout lila1= new LinearLayout(this);
        lila1.setOrientation(LinearLayout.VERTICAL);
        final EditText fileName = new EditText(this);
        final EditText fileContent = new EditText(this);
        lila1.addView(fileName);
        lila1.addView(fileContent);
        alert.setView(lila1);
        alert.setTitle("File Name and Content");
```

```
alert.setPositiveButton("Ok",new
DialogInterface.OnClickListener() {
            public void onClick(DialogInterface dialog, int
whichButton) {
                writeFile(fileName.getText().toString(),
fileContent.getText().toString());
                loadFileList();
            }
        });
        alert.setNegativeButton("Cancel",new
DialogInterface.OnClickListener() {
      public void onClick(DialogInterface dialog, int
whichButton) {
              dialog.cancel();
                    }
                });
        alert.show();
    }

    private void getFileName() {
        final AlertDialog.Builder alert = new AlertDialog.
Builder(this);
        final EditText fileName = new EditText(this);
        alert.setView(fileName);

        alert.setTitle("File Name to Open");
        alert.setPositiveButton("Ok",new
                            DialogInterface.OnClickListener() {
        public void onClick(DialogInterface dialog, int
                                                whichButton) {
```

```
                TextView tv = (TextView)findViewById(R.id.textView);
                tv.setText(readFile(fileName.getText().toString()))
               }
        });
        alert.setNegativeButton("Cancel",
                new DialogInterface.OnClickListener() {
        public void onClick(DialogInterface dialog, int
whichButton) {
                        dialog.cancel();
                    }
                });
        alert.show();
    }
    private void loadFileList(){
        TextView tv = (TextView)findViewById(R.id.textView);
        File currentDir = getFilesDir();

        tv.setText("");
        for(String fileName : currentDir.list()){
            tv.setText(tv.getText()+"\n "+fileName);
        }
    }
}
```

7. Navigate to Run menu and select Run 'app'.

The output is as shown in figure 3.12.

Figure 3.12: InternalFS Application

Click the NEW FILE button and enter the details as shown in figure 3.13.

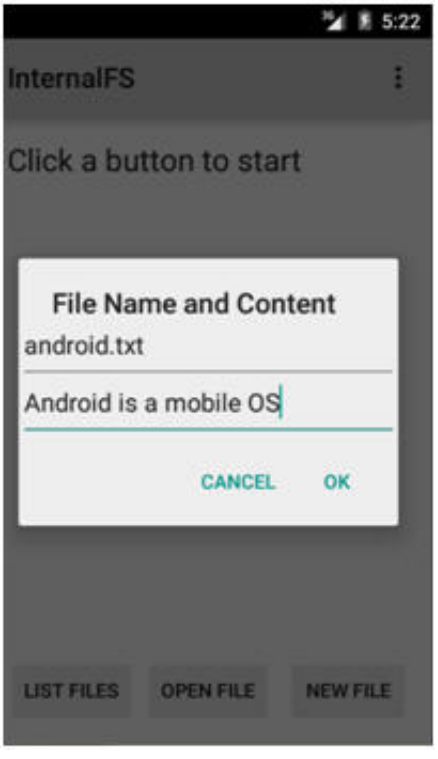


Figure 3.13: New File Creation

Click OK. The file is created and listed as shown in figure 3.14.

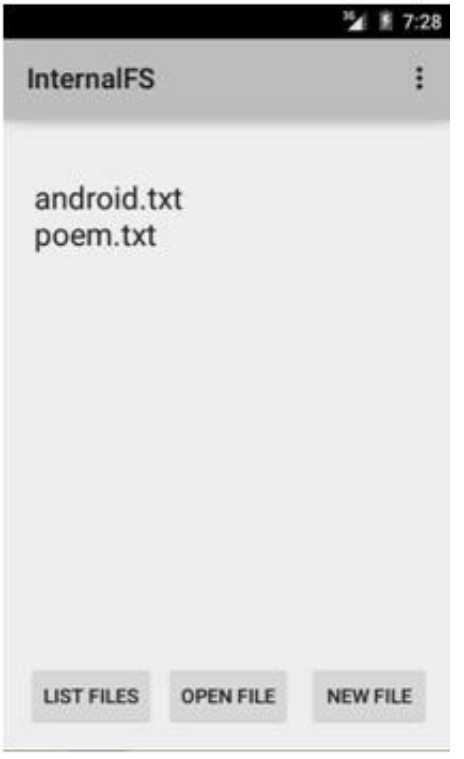


Figure 3.14: New File Created

Click the OPEN FILE button and enter the file name of the file created in the previous step as shown in figure 3.15.

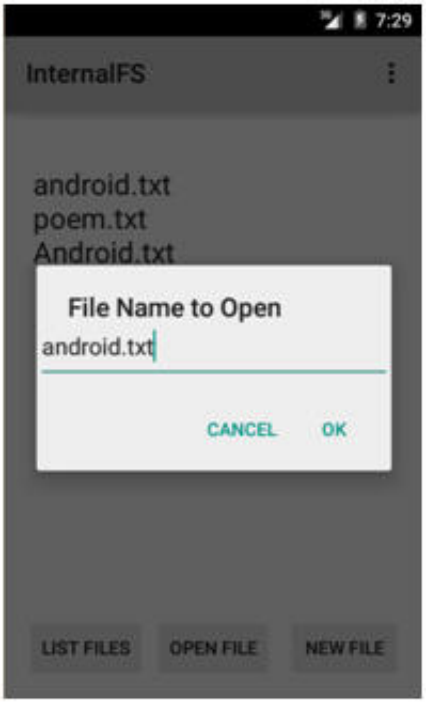


Figure 3.15: File Open Dialog

Click OK. The contents of the file are displayed as shown in figure 3.16.

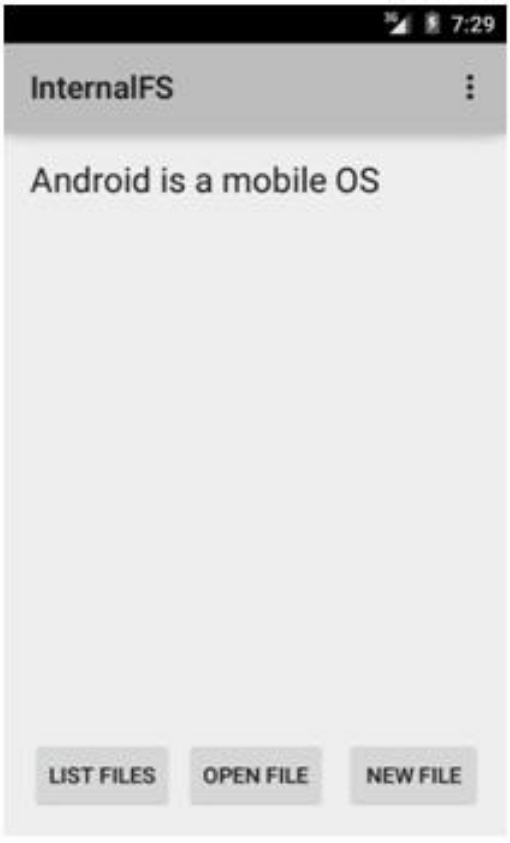


Figure 3.16: File Opened

3.3.3 External Storage

Developers also have the facility to save files to an external storage, such as an SD card. When a file is saved to an external storage, it can be modified and shared by other users.

Some of the features of external storage are as follows:

- It cannot be accessed all the time, because it is removable.
- Files are not very secure as they are accessible by others. All apps can be accessed without permissions in external storage.
- When an application is uninstalled, the related files are completely removed only if they are saved to the getFilesDir() folder.

It is best to use external storage for files without access restrictions and for files that can be shared without user permissions for access.

Writing and Saving to External Storage

To read and write data to an external storage directory, perform the following steps:

- Invoke the getExternalStorageDirectory() method to obtain the absolute path of the SD card.
- Create a directory in the SD card.
- Read or write data using the read() or write() method.
- Provide the WRITE_EXTERNAL_STORAGE permission in the AndroidManifest.xml file.

Code snippet 20 displays the use of WRITE_EXTERNAL_STORAGE permission in the Android manifest file.

Code Snippet 20:

```
<manifest ...>

      <uses-permission  android:name="android.permission.WRITE_
                                          EXTERNAL_STORAGE" />

    ...

</manifest>
```

To save to external storage, the developer has to first check if the storage medium is available. Using the getExternalStorageState() method the developer can determine if the media is mounted and then save files to the medium. External files can be classified as public and private files. Public files are files that can be accessed by all users and remain undisturbed even after the app is uninstalled. The getExternalStoragePublicDirectory() method can be used to save a file to external storage.

To get the absolute path of external storage, one can access as shown in Code Snippet 21.

Code Snippet 21:

```
String path = Environment.getExternalStorageDirectory().
                          getAbsolutePath();

File file = Environment.getExternalStorageDirectory();
```

The first line of code retrieves the path as a String. The second line returns a File object. The latter is more flexible and useful to the developer.

Private files are stored in a particular external directory when the app is installed and are automatically deleted when the app is uninstalled. The getExternalFilesDir() method can be used to save private files.

Deleting Files

It is a good practice to delete unwanted files to ensure the availability of disk space. The developer can use the delete() method. For instance, examplefile.delete(): can be used to directly delete the file. For files in internal storage, the deleteFile() method can be used to locate the file and then delete it.

Note -

- The developer should periodically delete all the files in the cache using the getCacheDir() option.
- If the developer is using the AVD for this project, then the AVD instance needs to have a SD card image associated with it.

Writing File in External SD Card

1. Create a project named ExternalFS in Android Studio.
2. Navigate to ExternalFS → res → layout → activity_main.xml file in the Project pane.
3. Double-click activity_main.xml.
4. Modify the code as shown in Code Snippet 22.

Code Snippet 22:

```
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
                          tools:context=".MainActivity">
<TextView
       android:layout_width="wrap_content"
       android:layout_height="wrap_content"
       android:textAppearance="?android:attr/textAppearanceLarge"
       android:text="Click a button to start"
       android:id="@+id/textView"
       android:layout_alignParentTop="true"
       android:layout_alignParentStart="true"
```

```
        android:layout_alignParentBottom="true"
        android:layout_alignParentEnd="true" />
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="List Files"
        android:id="@+id/button2"
        android:layout_alignBottom="@+id/textView"
        android:onClick="listFiles"
        android:layout_alignParentStart="true" />
    <Button
         android:layout_width="wrap_content"
         android:layout_height="wrap_content"
         android:text="New File"
         android:id="@+id/button"
         android:onClick="newFile"
         android:layout_alignBottom="@+id/textView"
         android:layout_alignStart="@+id/button4" />
    <Button
         android:layout_width="wrap_content"
         android:layout_height="wrap_content"
         android:text="Open"
         android:id="@+id/button3"
         android:onClick="openFile"
         android:layout_alignTop="@+id/button2"
         android:layout_toEndOf="@+id/button2" />
   <Button
         android:layout_width="wrap_content"
         android:layout_height="wrap_content"
         android:text="Up"
```

```
        android:id="@+id/button4"
        android:onClick="dirUP"
        android:layout_below="@+id/button6"
        android:layout_toEndOf="@+id/button7" />
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Rename"
        android:id="@+id/button5"
        android:onClick="rename"
        android:layout_above="@+id/button2"
        android:layout_alignParentStart="true"
        android:layout_alignEnd="@+id/button2" />
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Make Directory"
        android:id="@+id/button6"
        android:onClick="makeDir"
        android:layout_above="@+id/button5"
        android:layout_alignParentStart="true" />
   <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Delete"
        android:id="@+id/button7"
        android:onClick="delete"
        android:layout_below="@+id/button6"
       android:layout_alignStart="@+id/button3" />
</RelativeLayout>
```

5. Navigate to app → java → com.aptech.externalfs and open MainActivity.java.
6. Modify the code as shown in Code Snippet 23.

Code Snippet 23:

```
package aptech.com.externalfs;
import android.app.Activity;
import android.app.AlertDialog;
import android.content.DialogInterface;
import android.os.Bundle;
import android.os.Environment;
import android.text.Html;
import android.text.InputType;
import android.util.Log;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.EditText;
import android.widget.LinearLayout;
import android.widget.TextView;
import android.widget.Toast;
import java.io.BufferedInputStream;
import java.io.BufferedReader;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.FileReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.PrintWriter;
```

```
public class MainActivity extends Activity {
File currentDir= Environment.getExternalStorageDirectory();
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
   showToast("The current state of External Storage is : " +
             Environment.getExternalStorageState());
}
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    // Inflate the menu; this adds items to the action bar if it
is present.
    getMenuInflater().inflate(R.menu.menu_main, menu);
    return true;
}
@Override
public boolean onOptionsItemSelected(MenuItem item) {
 // Handle action bar item clicks here. The action bar will
 // automatically handle clicks on the Home/Up button, so long
 // as you specify a parent activity in AndroidManifest.xml.
    int id = item.getItemId();
    //noinspection SimplifiableIfStatement
    if (id == R.id.action_settings) {
        return true;
    }
    return super.onOptionsItemSelected(item);
}
```

```
public void listFiles(View view) {
    loadFileList();
}
public void newFile(View view) {
    getUserInput();
}
public void openFile(View view) {
    getFileName();
}
public void dirUP(View view) {
    currentDir = currentDir.getParentFile();
    loadFileList();
}
public void rename(View view) {
    getRenameFileName();
}
public void makeDir(View view) {
    getDirName();
}
public void delete(View view) {
    getDeleteFileName();
}
private void writeFile(String fileName, String fileContent) {
    File file = new File(currentDir,fileName);
    try {
        FileOutputStream fs = new FileOutputStream(file);
        PrintWriter pw = new PrintWriter(fs);
        pw.println(fileContent);
```

```
    pw.flush();
    pw.close();
    fs.close();
  }
    catch (FileNotFoundException e) {
          e.printStackTrace();
  }
    catch (IOException e) {
          e.printStackTrace();
      }
  }
  private String readFile(String fileName) {
      StringBuilder content = new StringBuilder();

      File file = new File(currentDir,fileName);
      try {
          BufferedReader br = new BufferedReader(new
                              FileReader(file));
          String line;
          while ((line = br.readLine()) != null) {
              content.append(line);
              content.append("\n");
          }
          br.close();
      }
      catch (FileNotFoundException e) {
          e.printStackTrace();
      } catch (IOException e) {
               e.printStackTrace();
```

```
        }
            return content.toString();
        }
        private void getUserInput() {
            final AlertDialog.Builder alert = new AlertDialog.
                                              Builder(this);
            LinearLayout lila1= new LinearLayout(this);
            lila1.setOrientation(LinearLayout.VERTICAL);
            final EditText fileName = new EditText(this);
            final EditText fileContent = new EditText(this);
            lila1.addView(fileName);
            lila1.addView(fileContent);
            alert.setView(lila1);

            alert.setTitle("File Name and Content");

            alert.setPositiveButton("Ok", new
                                DialogInterface.OnClickListener() {
              public void onClick(DialogInterface dialog, int
whichButton) {
                    writeFile(fileName.getText().toString(),
               fileContent.getText().toString());
               loadFileList();
            }
            });
               alert.setNegativeButton("Cancel", new
                                DialogInterface.OnClickListener() {
```

```
     public void onClick(DialogInterface dialog, int whichButton)
{
                dialog.cancel();
            }
        });
        alert.show();
    }
    private void getFileName(){
        final AlertDialog.Builder alert = new AlertDialog.
                                          Builder(this);
        final EditText fileName = new EditText(this);
        alert.setView(fileName);

        alert.setTitle("File/Dir Name to Open");

        alert.setPositiveButton("Ok", new DialogInterface.
OnClickListener() {
          public void onClick(DialogInterface dialog, int
whichButton)
             {
            File f = new File(currentDir,fileName.getText().
                              toString());
                if(f.isFile()) {
                  TextView tv = (TextView) findViewById(R.id.textView);
tv.setText(readFile(fileName.getText().toString()));
                }
                else {
```

```
                    currentDir = f;
                    loadFileList();
                }


            }
        });
        alert.setNegativeButton("Cancel",
                new DialogInterface.OnClickListener() {
          public void onClick(DialogInterface dialog, int
whichButton) {
                        dialog.cancel();
                    }
                });
        alert.show();
    }
    private void getDirName(){
        final AlertDialog.Builder alert = new AlertDialog.
                                          Builder(this);
        final EditText fileName = new EditText(this);
        alert.setView(fileName);
        alert.setTitle("Directory Name to create ");


        alert.setPositiveButton("Ok", new DialogInterface.
OnClickListener() {
          public void onClick(DialogInterface dialog, int
whichButton) {
                File newDir = new File(currentDir,fileName.
                              getText().toString());
                newDir.mkdir();
                loadFileList();
```

```
    }
        });
        alert.setNegativeButton("Cancel",
                new DialogInterface.OnClickListener() {
          public void onClick(DialogInterface dialog, int
                              whichButton) {
                        dialog.cancel();
                    }
                });
        alert.show();

    }
    void getDeleteFileName() {
        final AlertDialog.Builder alert = new AlertDialog.
                                          Builder(this);
        final EditText fileName = new EditText(this);
        alert.setView(fileName);
        alert.setTitle("Name to delete ");
        alert.setPositiveButton("Ok", new
        DialogInterface.OnClickListener() {
          public void onClick(DialogInterface dialog, int
whichButton) {
                File file = new File(currentDir,fileName.
                           getText().toString());
                file.delete();
                loadFileList();
            }
        });
```

```
     alert.setNegativeButton("Cancel",
                new DialogInterface.OnClickListener() {
           public void onClick(DialogInterface dialog, int
                              whichButton) {
                        dialog.cancel();
                    }
                });
        alert.show();
    }
    private void getRenameFileName() {
        final AlertDialog.Builder alert = new AlertDialog.
                                          Builder(this);
        LinearLayout lila1= new LinearLayout(this);
        lila1.setOrientation(LinearLayout.VERTICAL);
        final EditText oldName = new EditText(this);
        final EditText newName = new EditText(this);
        lila1.addView(oldName);
        lila1.addView(newName);
        alert.setView(lila1);
        alert.setTitle("Old name and New Name");
        alert.setPositiveButton("Ok", new
                        DialogInterface.OnClickListener() {
         public void onClick(DialogInterface dialog, int
                        whichButton)
{
                File oldfile = new File(currentDir,oldName.
                              getText().toString());
                File newfile = new File(currentDir,newName.
                              getText().toString());
                oldfile.renameTo(newfile);
```

```
                loadFileList();
    }
        });
        alert.setNegativeButton("Cancel", new
                                DialogInterface.OnClickListener() {
        public void onClick(DialogInterface dialog, int
                            whichButton) {
                dialog.cancel();
            }
        });
        alert.show();
    }
    private void loadFileList(){
        TextView tv = (TextView)findViewById(R.id.textView);
        tv.setText("");
        for(File obj : currentDir.listFiles())
        {
            if(obj.isDirectory())
tv.setText(Html.fromHtml(Html.toHtml(tv.getEditableText()).
toString()+"<br /> <b>"+obj.getName()+"</b>"));
else
tv.setText(Html.fromHtml(Html.toHtml(tv.getEditableText()).
toString()+"<br />"+obj.getName()));
    }
    private boolean isExternalStorageWritable() {
        String state = Environment.getExternalStorageState();
        if (Environment.MEDIA_MOUNTED.equals(state)) {
            return true;
        }
        return false;
    }
```

```
    public boolean isExternalStorageReadable() {
        String state = Environment.getExternalStorageState();
        if (Environment.MEDIA_MOUNTED.equals(state) ||
             Environment.MEDIA_MOUNTED_READ_ONLY.equals(state)) {
            return true;
        }
        return false;
    }

    private void showToast(String message){
    Toast.makeText(getApplicationContext(),message,Toast.
                                LENGTH_SHORT).show();
    }
}
```

7. Navigate to app → manifests and open the file AndroidManifest.xml.
8. Add write permissions to the application by modifying the code as shown in Code Snippet 24.

Code Snippet 24:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/
android"
    package="aptech.com.externalfs" >
    <uses-permission android:name="android.permission.WRITE_
                                    EXTERNAL_STORAGE" />
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <activity
            android:name=".MainActivity"
            android:label="@string/app_name" >
```

```
                    <intent-filter>
                    <action android:name="android.intent.action.
                                          MAIN" />


                <category
                android:name="android.intent.category.LAUNCHER"
                                                              />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

9. Click the Run menu and select Run 'app'.

   The application is deployed on the AVD and the output is as shown in figure 3.17.

Figure 3.17: ExternalFS Application

Click MAKE DIRECTORY and enter the name as shown in figure 3.18.

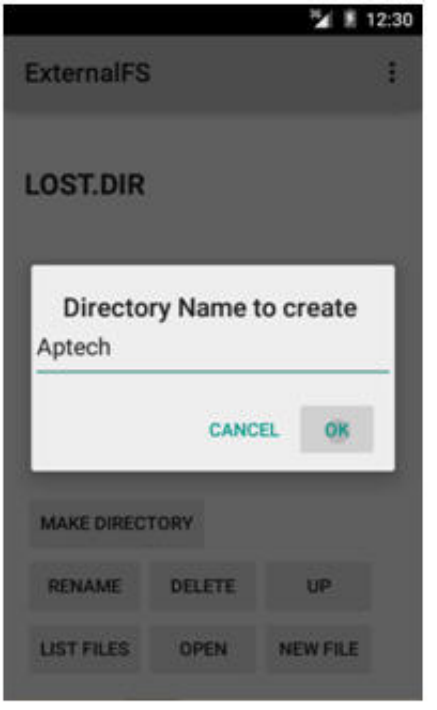


Figure 3.18: Create Directory Dialog

Click LIST FILES to view the files and directories on the external storage as shown in figure 3.19.

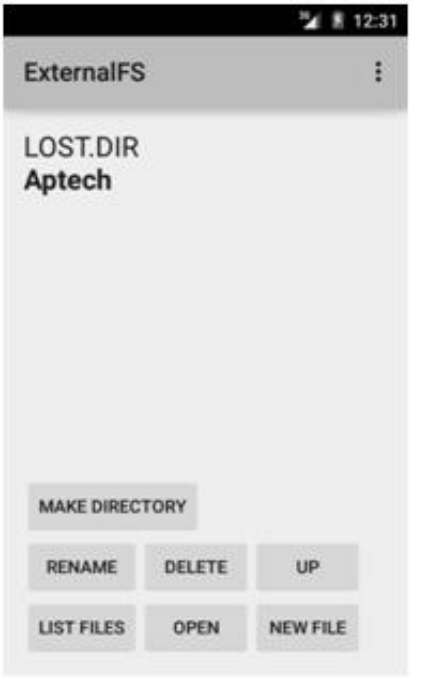


Figure 3.19: Files in the Current Directory Listed

Directories are shown in bold whereas files are shown in normal text.

Click OPEN and enter the newly created directory name as shown in figure 3.20.

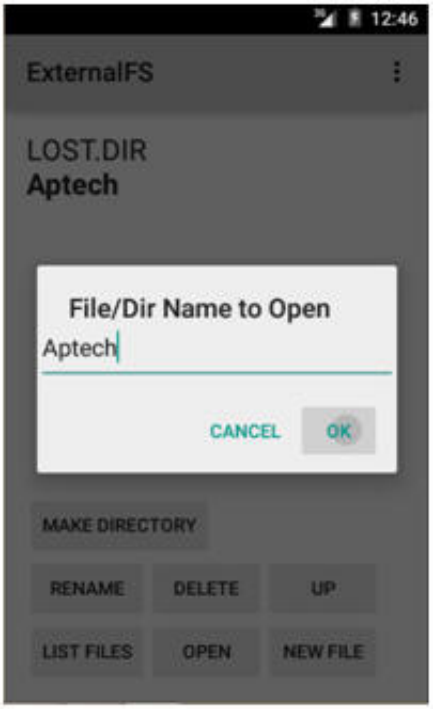


Figure 3.20: Open Dialog

Click OK. The application will open the directory if the name is a directory. If the name is a file, it will open the file for reading as shown in the previous example. The new file functionality is similar to that of the previous example. Click UP to navigate to the parent directory.

Click RENAME and enter the details as shown in figure 3.21 to rename the file/ directory.

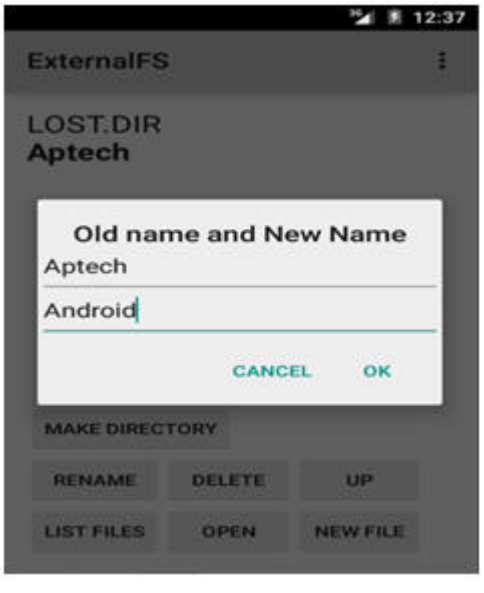


Figure 3.21: Rename Dialog

The file/directory is renamed as shown in figure 3.22.

Figure 3.22: File/Directory Renamed Successfully

Click DELETE, enter the file name and click OK to delete the file as shown in figure 3.23.

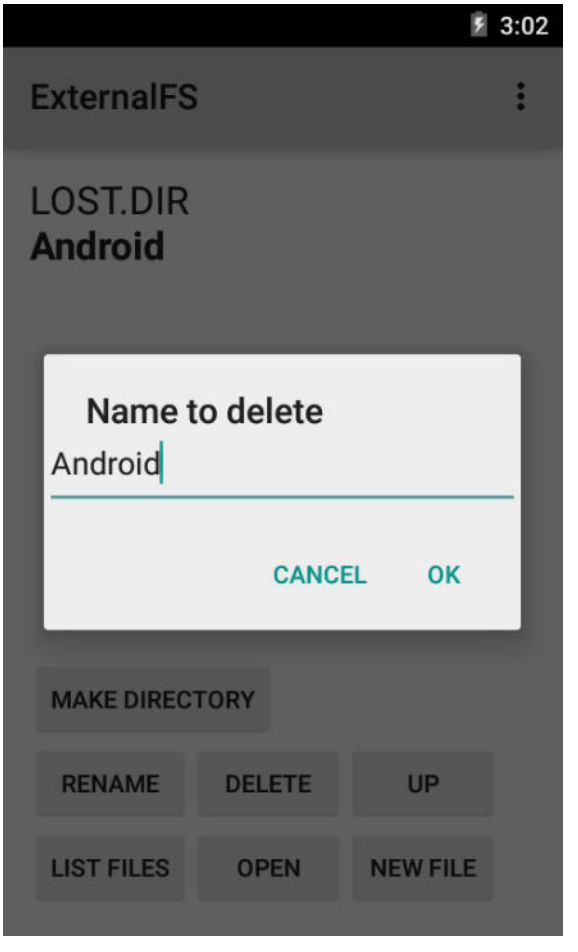


Figure 3.23: Delete Dialog

The file/directory is deleted as shown in figure 3.24.

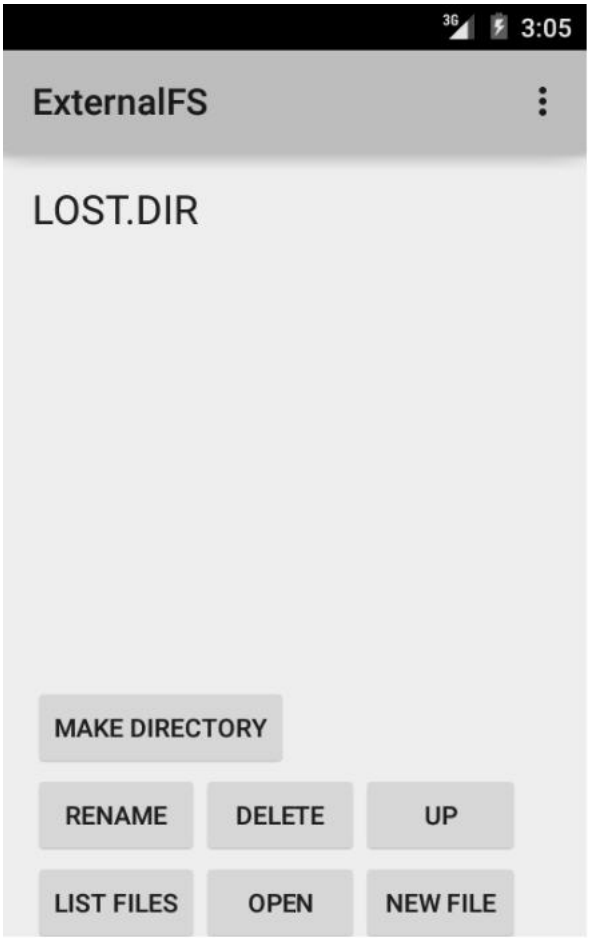


Figure 3.24: File/Directory Deleted Successfully

Note - Click LIST FILES if the updates are not visible.

3.4 Notifications

Android's notifications enable an app to inform the user about events. Events include calendar events and other alerts. Whenever a notification is added by an application, it first displays a notification icon. In order to view the notification, the user can open the Notification Drawer. The application has no control over both the notification area and the notification drawer. They are system controlled areas.

In order to display a notification, the Notification.Builder class needs to be used to prepare the notification. Every notification needs to contain the following information:

- **Small Icon** – setSmallIcon()
- **Title** – setContentTitle()
- **Detailed Text** – setContentText()

Every notification has a priority value which decides where it appears in the list of notifications. The priority of the notification can be set using setPriority().

In addition to the small icon displayed in the notification bar, each notification can also specify a large icon to be displayed in the notification drawer. This is done using the setLargeIcon() method.

It is possible to set up an artificial back stack to the loaded activity. This is useful to set up an artificial interaction. For example, as soon as a new message notification is displayed and the user clicks the notification, he is directed to the message. By clicking Back, it returns user to the Home page. Instead, it is preferable to redirect the user to the message history/ list. For this purpose the TaskStackBuilder can be used.

Starting from API level 16 (Android 4.1) it is possible to set extended layouts to notifications using the setStyle() method of Notification.Builder.

Each notification has a notification id managed by the application. This id can be used later on to update the notification or to remove the notification. Any new notification with the same id will replace the older notification. Calling the cancel() method on the notification id will remove the notification.

Notifications can also be used to show the progress of an operation. This is especially useful when performing a time consuming operation such as I/O related operation, downloads, or extraction.

Starting from API level 21 (Android 5.0), it is possible to display Heads Up notification. A heads up notification is a dialog box displayed directly to the user on top of any running applications. Heads up are displayed when one of the following criteria is met:

- The user's activity is in fullscreen mode.
- The notification has high priority and uses ringtones or vibrations.

The developer is not in direct control of displaying a notification as heads up. Setting the priority to PRIORITY_HIGH or PRIORITY_MAX has the maximum likelihood of displaying the notification as a heads up.

Lock Screen Notification has been standardized since API level 17 (Android 4.2) which allows the developer to display notification text on the lock screen. This can be used to notify messages and other similar tasks. Notifications can be made to be interactable. This feature can be used to substitute the lock screen widget support dropped in Android 5.1. This can be useful to implement features such as lock screen music controls. In order to display a lock screen Notification, the visibility needs to be set to VISIBILITY_PUBLIC or VISIBILITY_PRIVATE using the setVisibility() method. Setting it to VISIBILITY_SECRET will not display the notification on the lock screen.

Lock Screen controls can be added to the notification using the addAction() method as shown in Code Snippet 25.

Code Snippet 25:

```
.addAction(R.drawable.ic_button, "Press Button", buttonIntent)
```

Multiple actions can be added to provide additional controls.

In the following example, the notification features have been used to display different types of notifications:

1. Create a new project called Notifications in Android Studio.
2. Navigate to app → res. Right-click and select New → Image Asset. The Asset Studio dialog box is displayed.
3. Enter the details as shown in figure 3.25.

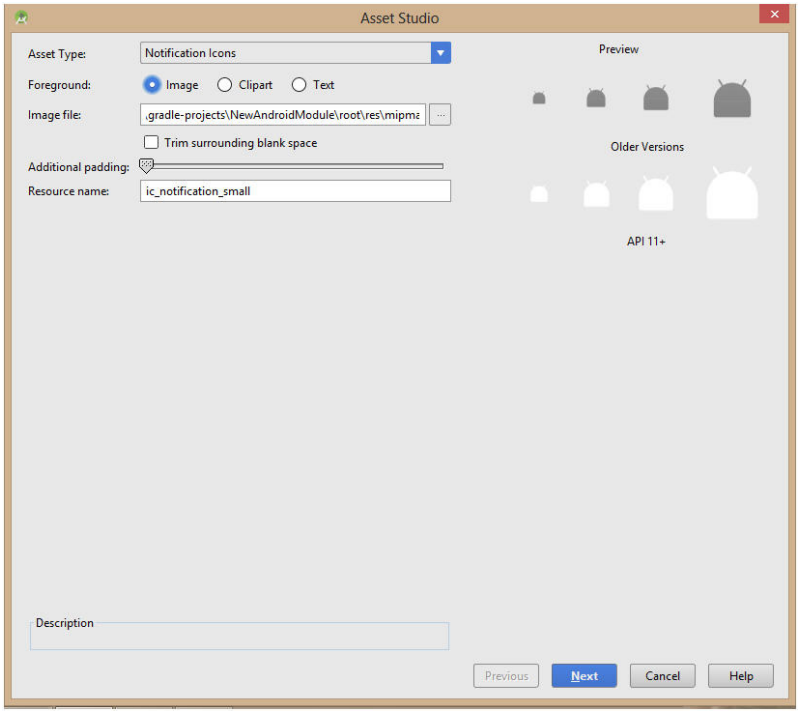


Figure 3.25: Notification Icon Dialog Box

4. Click Next.
5. Click Finish.
6. Repeat step 2.

7. Enter the details as shown in figure 3.26.

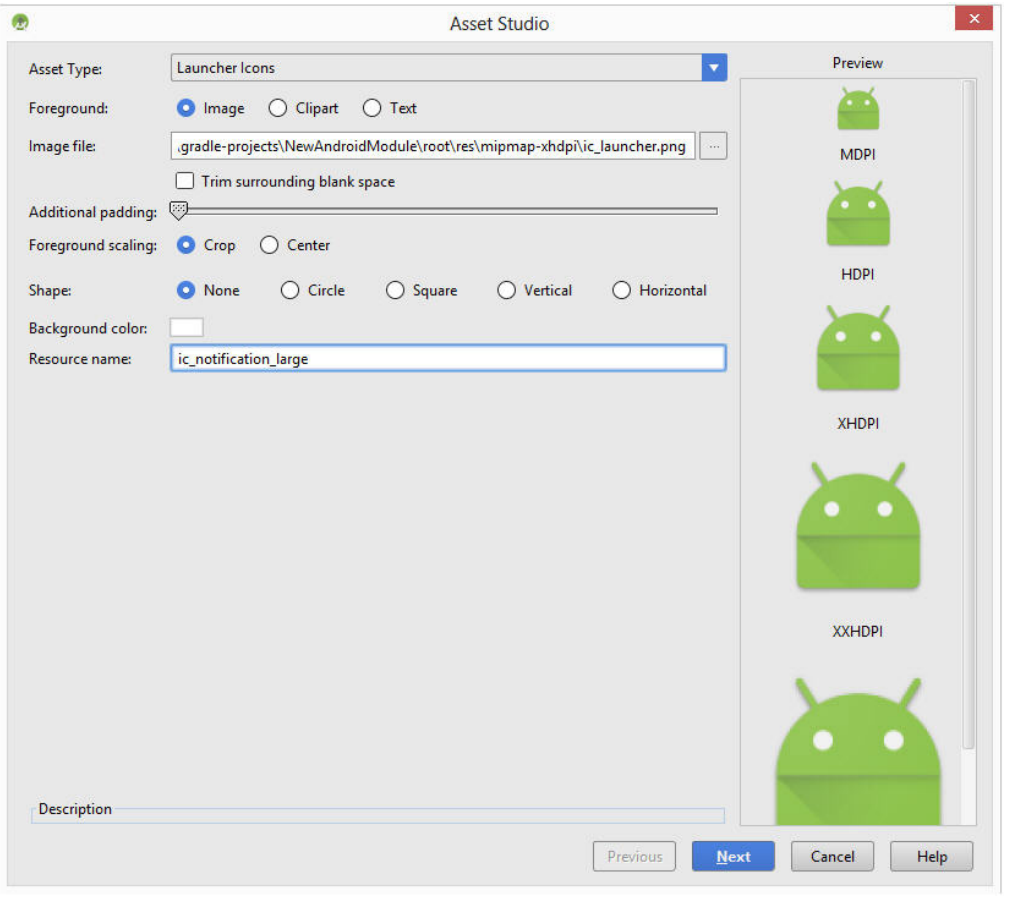


Figure 3.26: Large Notification Icon Dialog Box

8. Repeat steps 4 and 5.
9. Navigate to app → res → layout.
10. Open activity_main.xml for editing and modify the code as shown in Code Snippet 26.

Code Snippet 26:

```
<RelativeLayout

 xmlns:android="http://schemas.android.com/apk/res/android"

    xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"

    android:layout_height="match_parent" android:paddingLeft="@
dimen/activity_horizontal_margin"

    android:paddingRight="@dimen/activity_horizontal_margin"

    android:paddingTop="@dimen/activity_vertical_margin"

    android:paddingBottom="@dimen/activity_vertical_margin"
tools:context=".MainActivity">

    <Button

        android:layout_width="wrap_content"

        android:layout_height="wrap_content"

        android:text="Basic Notification"

        android:id="@+id/button"

        android:onClick="basic"

        android:layout_alignParentTop="true"

        android:layout_alignParentStart="true" />

   <Button

        android:layout_width="wrap_content"

        android:layout_height="wrap_content"

        android:text="Expanded Layout Notification"

        android:id="@+id/button3"

        android:onClick="layout"

        android:layout_alignParentStart="true"

        android:layout_below="@+id/button" />

   <Button

        android:layout_width="wrap_content"
```

```
        android:layout_height=”wrap_content”
        android:text=”Progress Notification”
        android:onClick=”progress”
        android:id=”@+id/button4”
        android:layout_below=”@+id/button3”
        android:layout_alignParentStart=”true” />
    <Button
        android:layout_width=”wrap_content”
        android:layout_height=”wrap_content”
        android:text=”Heads Up Notification”
        android:id=”@+id/button5”
        android:onClick=”headsup”
        android:layout_below=”@+id/button4”
        android:layout_alignParentStart=”true” />
    <Button
        android:layout_width=”wrap_content”
        android:layout_height=”wrap_content”
        android:text=”Lock Screen Notification”
        android:id=”@+id/button6”
        android:onClick=”lockscreen”
        android:layout_below=”@+id/button5”
        android:layout_alignParentStart=”true” />
    <Button
        android:layout_width=”wrap_content”
        android:layout_height=”wrap_content”
        android:text=”Clear Notifications”
        android:id=”@+id/button7”
        android:onClick=”clearall”
```

```
        android:layout_below="@+id/button6"
        android:layout_alignParentStart="true" />
</RelativeLayout>
```

11. Navigate to app → java → com.aptech.notifications.
12. Open MainActivity.java for editing and modify the code as shown in Code Snippet 27.

Code Snippet 27:

```
package aptech.com.notifications;
import android.app.Activity;
import android.app.Notification;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.app.TaskStackBuilder;
import android.content.Context;
import android.content.Intent;
import android.graphics.BitmapFactory;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.app.Notification.Builder;
public class MainActivity extends Activity {
    int id;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        id = 0;
    }
@Override
    public boolean onCreateOptionsMenu(Menu menu) {
```

```
        // Inflate the menu; this adds items to the action bar
if it is present.
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
        // Handle action bar item clicks here. The action bar will
        // automatically handle clicks on the Home/Up button, so long
        // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
    public void basic(View view) {
        Notification.Builder notification = new Notification.
                                            Builder(this)
                .setSmallIcon(R.drawable.ic_notification_small)
                .setContentTitle("Basic Notification")
                .setContentText("A simple notification!")
.setLargeIcon(BitmapFactory.decodeResource(getResources(),
R.mipmap.ic_notification_large));
        Intent returnIntent = new Intent(this, MainActivity.
                                         class);
        TaskStackBuilder stackBuilder = TaskStackBuilder.
                                        create(this);
        stackBuilder.addParentStack(MainActivity.class);
```

```
                stackBuilder.addNextIntent(returnIntent);

                PendingIntent returnPendingIntent = stackBuilder.
getPendingIntent(0,PendingIntent.FLAG_UPDATE_CURRENT);

        notification.setContentIntent(returnPendingIntent);

        NotificationManager mNotificationManager =
(NotificationManager) getSystemService(Context.NOTIFICATION_
SERVICE);

        mNotificationManager.notify(0, notification.build());

    }

    public void layout(View view) {

        Notification.Builder notification = new Notification.
                                            Builder(this)

                 .setSmallIcon(R.drawable.ic_notification_small)

                .setContentTitle("Expanded Layout Notification")

               .setContentText("This notification can be expanded")

                .setLargeIcon(BitmapFactory.
decodeResource(getResources(), R.mipmap.ic_notification_large));

        //Expanded Layout

        Notification.InboxStyle inboxStyle = new Notification.
                                             InboxStyle();

        inboxStyle.setBigContentTitle("Multi line Expanded
                                      Layout ");

        for( int i=0;i<5;i++)

        {

            inboxStyle.addLine(" ");

        }

        notification.setStyle(inboxStyle);

     Intent returnIntent = new Intent(this, MainActivity.class);

     TaskStackBuilder stackBuilder = TaskStackBuilder.
                                     create(this);

                stackBuilder.addParentStack(MainActivity.class);
```

```
                stackBuilder.addNextIntent(returnIntent);
        PendingIntent returnPendingIntent = stackBuilder.
getPendingIntent(0,PendingIntent.FLAG_UPDATE_CURRENT);
        notification.setContentIntent(returnPendingIntent);
        NotificationManager mNotificationManager =
(NotificationManager) getSystemService(Context.NOTIFICATION_
SERVICE);
        mNotificationManager.notify(1, notification.build());
    }
    public void progress(View view) {
        final Notification.Builder notification = new Notification.
Builder(this)
                .setSmallIcon(R.drawable.ic_notification_small)
                .setContentTitle("Progress Notification")
                .setContentText("A simple notification!")
                .setLargeIcon(BitmapFactory.
                        decodeResource(getResources(),
        R.mipmap.ic_notification_large));
        Intent returnIntent = new Intent(this, MainActivity.
                        class);
        TaskStackBuilder stackBuilder = TaskStackBuilder.
                                create(this);
        stackBuilder.addParentStack(MainActivity.class);
        stackBuilder.addNextIntent(returnIntent);
        PendingIntent returnPendingIntent = stackBuilder.
getPendingIntent(0,PendingIntent.FLAG_UPDATE_CURRENT);
notification.setContentIntent(returnPendingIntent);
        new Thread(
                new Runnable() {
                    NotificationManager mNotifyManager
=(NotificationManager)getSystemService(Context.NOTIFICATION_
SERVICE);
   @Override
```

```
                    public void run() {
                        int incr;
                        for (incr = 0; incr <= 100; incr+=5) {
                       notification.setProgress(100, incr, false);
                       mNotifyManager.notify(0, notification.
                                                  build());
                            try {
                                Thread.sleep(5*1000);
                            } catch (InterruptedException e) {
                                e.printStackTrace();
                            }
                        }
                  notification.setContentText("Download
complete").setProgress(0, 0, false);
                  mNotifyManager.notify(2, notification.build());
                    }
                }
        ).start();
    }
    public void headsup(View view) {
        Notification.Builder notification = new Notification.
                                            Builder(this)
          .setSmallIcon(R.drawable.ic_notification_small)
                        .setContentTitle("Heads Up Notification")
          .setContentText("A Heads Up Notification!")
          .setPriority(Notification.PRIORITY_MAX) // Set to max
to increase likelihood of Heads Up notification
          .setLargeIcon(BitmapFactory.
decodeResource(getResources(), R.mipmap.ic_notification_large));
        Intent returnIntent = new Intent(this, MainActivity.class);
        TaskStackBuilder stackBuilder = TaskStackBuilder.
                                        create(this);
```

```
        stackBuilder.addParentStack(MainActivity.class);

        stackBuilder.addNextIntent(returnIntent);

        PendingIntent returnPendingIntent =stackBuilder.
getPendingIntent(0,PendingIntent.FLAG_UPDATE_CURRENT);

        notification.setContentIntent(returnPendingIntent);

        NotificationManager mNotificationManager =
(NotificationManager) getSystemService(Context.NOTIFICATION_
SERVICE);

        mNotificationManager.notify(3, notification.build());

    }

    public void lockscreen(View view) {

        Notification.Builder notification =new Notification.
Builder(this)

                .setSmallIcon(R.drawable.ic_notification_small)

                .setVisibility(Notification.VISIBILITY_PUBLIC)

                .setContentTitle("Lock Screen Notification")

                .setContentText("Lock Screen information text")

.setLargeIcon(BitmapFactory.decodeResource(getResources(),
R.mipmap.ic_notification_large));

       Intent returnIntent = new Intent(this, MainActivity.
                             class);

       TaskStackBuilder stackBuilder = TaskStackBuilder.
                                       create(this);

        stackBuilder.addParentStack(MainActivity.class);

        stackBuilder.addNextIntent(returnIntent);

        PendingIntent returnPendingIntent =stackBuilder.
getPendingIntent(0,PendingIntent.FLAG_UPDATE_CURRENT);

        notification.setContentIntent(returnPendingIntent);

        NotificationManager mNotificationManager =
(NotificationManager) getSystemService(Context.NOTIFICATION_
SERVICE);

        mNotificationManager.notify(4, notification.build());
```

```
  }

    public void clearall(View view) {

    NotificationManager mNotificationManager = (NotificationManager)
getSystemService(Context.NOTIFICATION_SERVICE);

        mNotificationManager.cancelAll();

    }

}
```

13. Navigate to run menu and select Run 'app'.

The app is loaded onto the AVD and the output is shown in figure 3.27.

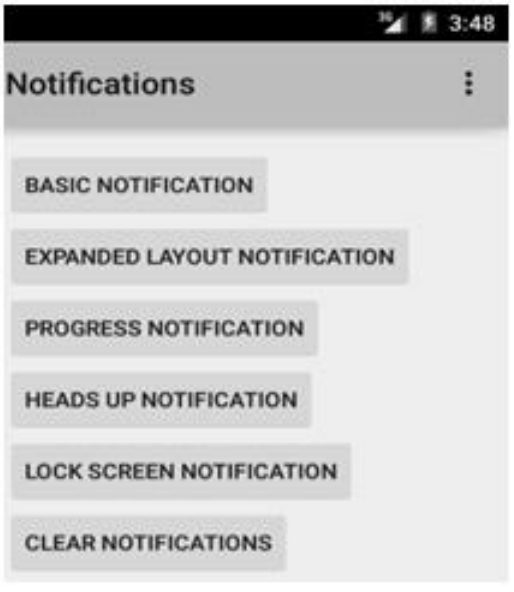


Figure 3.27: The Notifications Application

By clicking the BASIC NOTIFICATION button, a basic notification is displayed as shown in figure 3.28.

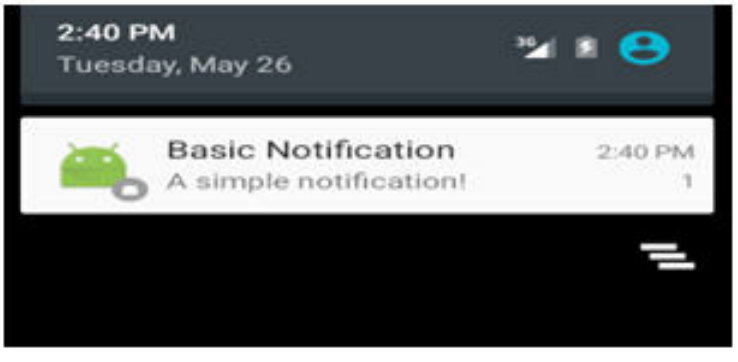


Figure 3.28: Basic Notification

By clicking the EXPANDED LAYOUT NOTIFICATION button, an expanded layout notification is displayed as shown in figure 3.29.

Figure 3.29: Expanded Layout Notification

By clicking the PROGRESS NOTIFICATION button, a notification is displayed with the progress updating every five seconds as shown in figure 3.30.

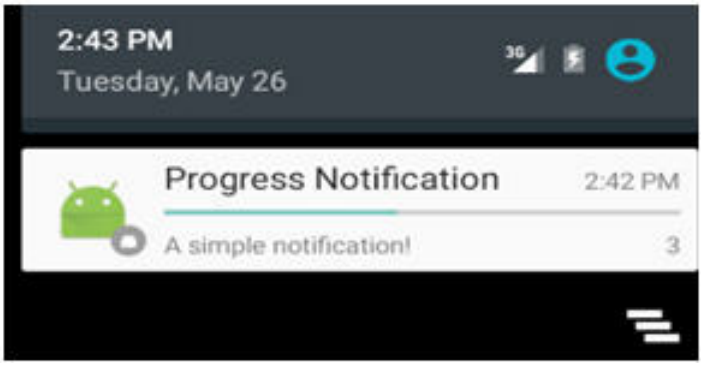


Figure 3.30: Progress Notification

Clicking the HEADS UP NOTIFICATION button may display a heads up display, if the criteria is met as explained earlier.

By clicking LOCK SCREEN NOTIFICATION, a lock screen notification is displayed as shown in figure 3.31.

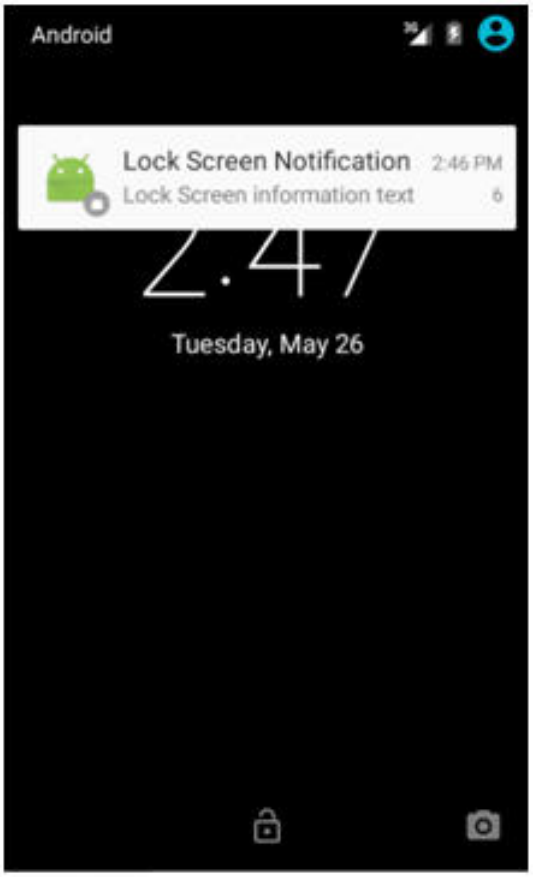


Figure 3.31: Lock Screen Notification

By clicking CLEAR NOTIFICATION, all the notifications created by the app are cleared as shown in figure 3.32.

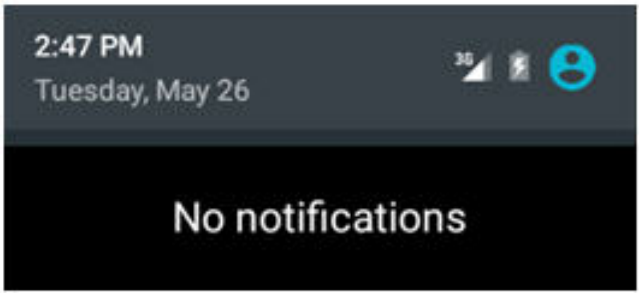


Figure 3.32: All Notifications Cleared

3.5 Android Security Model

At the center of the Android security framework is the Linux Kernel. The Linux Kernel offers a robust and well-tested security system. The main security features that the Kernel offers are as follows:

- Permissions tailored to user needs.
- Separating processes.
- Securing Inter-Process Communication (IPC).
- Eliminating parts of the Kernel that are not secure.
- Separating resources of one user from another.

The Android security model also offers several user security options and protection from third-party applications. It is important to understand permissions, the Application Sandbox, and user security options to comprehend the Android security model.

- **Permissions**

  Another important aspect of the Android Security framework is the concept of permissions. The basic rule of the security framework is that none of the applications have permissions to execute operations that might severely affect other applications. Also, no application has permissions to perform an operation that could affect the OS or any user information, such as e-mail or SMS.

  Permissions also encompass user permissions for application access, application certification, user identifications, access to files, and root permissions.

  When the application is installed, it states the permissions that are additionally required for functions that are not authorized by the default permissions. The user needs to provide consent to approve permissions for these capabilities.

  The developer has to sign a certificate with a private key that identifies the author of the application. The certification enables the system to approve or reject access for applications that request permissions at the signature level. Based on the certification, the system can also determine whether or not to approve an application requesting permission for being granted an identity similar to a different application.

  When a package is installed on a device, the OS delegates a unique user identification for the package. This ID is valid for the package until it is deleted from the device. When the package is installed on another device, it would have a different user ID. Different packages can only be executed in a single process using the shared User Id attribute in the Android Manifest file. This assigns both packages the same user ID and related permissions thereby considering the two different packages as a single application.

  The application's data would also be protected by the user ID and would be inaccessible by other applications. The developer can set the read or write flags when a new file is created to enable other applications to access the file. Although the file belongs to a specific application, setting read/write flags enable all other applications access to the file. File system permissions ensure that one user will not be able to access or change another user's data.

Root permissions set for devices ensure that neither an application, nor a user can alter the OS, another application or the Kernel. The Root user, on the other hand, can access all applications and their data. When a user changes root permissions, the security of the system is compromised. On the other hand, a developer can modify the root permissions on a device to install another OS.

Separating Application Processes and Resources

A unique aspect of the Linux Kernel is that it separates processes and resources for one application from another, while providing the ability for applications to share resources. This is called the Application Sandbox wherein unique user identification is designated to each application and several applications are executed with common user permissions using group IDs. As a rule, applications cannot communicate and have restricted access to the OS. The OS keeps track of actions that applications perform and restrict them from security breaches. When it comes to resource access, the additional capabilities that the application requires must be defined in the Android Manifest file.

User Security Options

Device management, secure password, data encryption, certification based system, and Virtual Private Network (VPN) to ensure secure network connectivity are the features that Android provides to ensure user security.

- **Device Management**: Administrators can perform functions like enforcing security policies using certain applications, such as e-mail and can delete data on devices that were lost or were stolen.
- **Secure Password**: While accessing a device, the Android OS will verify a password provided by the user.
- **Data Encryption**: The Android security framework enables all of the user's data to be encrypted by providing an encryption key. This enables the generation of a key from the user's password, which blocks unauthorized access to user data.
- **Certification Based System**: Android comes with an existing list of Certificate Authorities (CAs) for enabling Secure Sockets Layer (SSL) connections. The later Android versions (3.0 onwards) also enable users to add or disable CAs.
- **VPN**: Android comes with a VPN client that enables applications to access the Internet only through the VPN.
- **Protection Relating to Third-Party Applications**: Android has a system of informing users when their devices are communicating with third-party applications. The different kinds of permissions that the third-party application requests are displayed clearly to the user at the time of installation.

Android also informs the user about functions that might create a cost to the user. It does not provide low-level SIM card access to third-party applications. It also prevents access to sensitive devices, such as camera and microphone. Android protects information about user preferences, device usage, and so on. It offers users the ability to verify applications and provides options for managing digital rights.

3.5.1 Security Enhancements in Android Lollipop

In addition to the security features of the previous versions of Android, Lollipop comes with few additional features to provide a secure environment. Some of them are as follows:

- **Full Disk Encryption Enabled by Default**: All devices running with Vanilla (non-modified) Android Lollipop have Disk Encryption enabled by default. The behavior can be enabled or modified by accessing Settings → Security.
- **Improved Encryption Procedures**: Android Lollipop comes with improved security standards for encrypting drive data as well as key input translation to remove possibility for a key logger.
- **SELinux Access Control**: Android now implements SELinux enforcing mode for all domains. This adds Kernel level security and closes any possible vulnerabilities.
- **Updates to Cryptography of HTTPS and SSL**: Updates to Https and SSL ensure a safe browsing session for the user when performing data sensitive tasks such as bank transactions.
- **Position Independent Execution Removed**: Removal of the PIE feature closed a memory manipulation vulnerability.
- **Multi User Profiles**: Support for multiple users has been added to restrict access to certain features to users on shared devices.
- **Additional Locking/Unlocking Options**: Additional unlock options are added to support NFC devices, trusted devices, and proximity unlock when the user is near a trusted user.

Check Your Progress

1. When setting preferences using XML files, the XML files should contain a list of which of the following?

1.	Preference Objects	2.	Preference Managers
3.	PreferenceCheckBoxes	4.	Preference Subclasses

(A)	1, 3	(B)	2, 4
(C)	1, 4	(D)	1, 2

2. Which of the following statements are true?

1.	Internal Storage contents are removed when the application is removed.
2.	External Storage contents are removed when the application is removed.
3.	Cache Directory will be retained when the application is removed.
4.	Preferences are used to store assets.

(A)	4	(B)	2, 3, 4
(C)	2, 3	(D)	1

3. Android offers permissions for which of the following application operations?

(A)	Accessing External File System	(B)	Changing root directory
(C)	Changing user id of the application	(D)	VPN

4. Match the following components with the corresponding purpose.

	Component		Purpose
a.	Internal Storage	1.	To store persistent data
b.	Notifications	2.	To store data that can be removed along with the application
c.	Preferences	3.	To display an intimation about an event
d.	External Storage	4.	To store user settings

(A)	a-2, b-3, c-4, d-1	(B)	a-1, b-2, c-3, d-4
(C)	a-4, b-1, c-3, d-2	(D)	a-3, b-4, c-2, d-1

5. Which of the following security features is unique to Android Lollipop?

(A)	Drive Encryption by default	(B)	Heads Up notifications
(C)	Face unlock	(D)	Pattern unlock

Answers

1.	C
2.	D
3.	A
4.	A
5.	A

Summary

- Preferences are a subclass that helps users to customize their applications. A developer can specify preferences using an XML file or using code.
- Title, Intents, and sub-screens are the different ways that helps to group settings.
- The Android file system offers Internal and external storage. Internal Storage is removed along with the application. External Storage can persist.
- Android's notifications enable an app to inform the user about events.
- Starting from Android Lollipop, Notifications can be made to be intractable on lock screen and act as substitutes for lock screen widgets.
- The Android security model provides several security features, such as permissions, the Application Sandbox, several user security options, Full Disk Encryption, and protection from third-party applications.

Onlinevarsity

04 Android User Interface (UI)

Welcome to the Session, **Android User Interface (UI)**.

This session gives an overview of Android User Interface (UI). It describes the views, layouts, User Interface (UI) components, styles and themes, and the procedure for handling user events.

In this Session, you will learn to:

- Explain the process to create the UI for Android applications
- Describe the views, layouts, UI components, styles and themes
- Explain the procedure for handling user events

Session

04

Android User Interface (UI)

4.1 Introduction

The UI is a space that enables interaction between the user and the machine. Feedback from the machine helps the user to manage operating decisions. UI is made up of both hardware and software components.

UI enables the following actions:

- **Input:** User manipulates the controls.
- **Output:** Machine responds to the user's actions.

Having understood the basic concept of UI, it is easy to appreciate how crucial UI is for PCs, mobile phones, smartphones, and other electronic gadgets. Without UI, it is impossible to interact with the gadget and use it for any purpose. Touch screen, Graphical User Interface (GUI), and Text-based User Interface (TUI) are just a few examples of the different types of UIs that are available.

4.2 Android User Interface

User Interface is what the user will see and interact with to perform operations. Android comes with many friendly UI elements and layouts which help to build interactive applications.

4.2.1 Overview of UI

Android offers various tools for the developer to build the GUI for an application. It also enables the developer to build an interface for menus, dialogs, and notifications. The tools that Android offers for UI developers include pre-built components and controls for the UI. The developer must understand certain concepts to build the UI. They are as follows:

- Views
- Layouts
- UI Components

View objects are the primary UI units of an Android application. A View, being an UI object, forms the common platform of interaction between the user and the application. View is a built-in class used for building different UI components. Using the View class, the developer has to specify the screen area that the user needs to interact with. Views are used to represent elements, such as lists, buttons, check boxes, and so on. Android offers various built-in views. However, if these are not sufficient, the developer can create specific views for apps.

Another important concept that the developer needs to understand while building the UI for an app, is layouts. Layouts act as containers for views. The type of layout determines the location of the View object on the UI screen. Layouts consist of linear layout, relative layout, list view, and grid view.

UI components involve understanding of several concepts such as:

- Input Controls
- Input Events
- Menus
- Action Bar
- Settings
- Dialogs
- Additional Views
- Status Notifications
- Toasts
- Search
- Drag and Drop

4.2.2 Views

All UI components in Android are built using View and ViewGroup objects. Views enable the developer to create the UI. Each item in the Android UI belongs to the View class. Views are also known as widgets and help to provide interactivity using buttons, text fields, and so on. There are a number of existing built-in views that Android provides.

To build a unique UI for an app, the developer needs to understand the View object, as well as, the objects belonging to ViewGroup. A set of several views make a ViewGroup. In other words, a ViewGroup is a container that contains and organizes other child views. Using the subclasses of ViewGroup such as RelativeLayout, LinearLayout, and View objects such as Button, TextView, ListView, and so on, the developer can build layout of the UI. ViewGroup and View objects are arranged in a hierarchy for each component, as shown in figure 4.1.

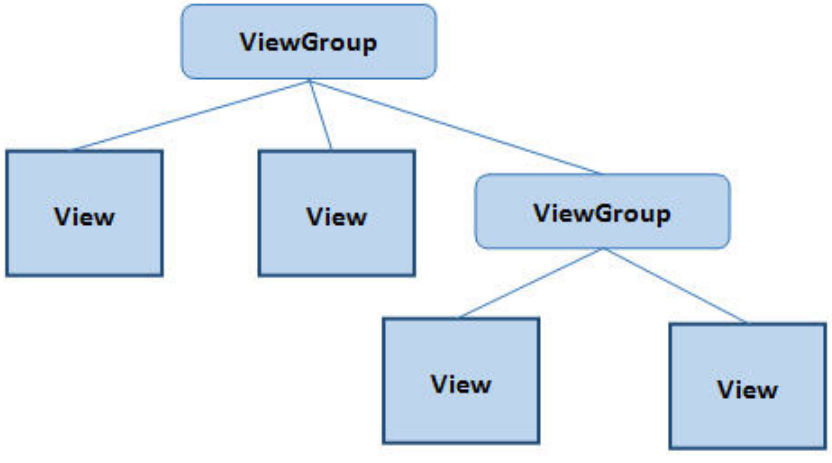


Figure 4.1: View Hierarchy

Each ViewGroup can have other ViewGroup and View within. These are called child Views. Upon creating a hierarchy, the developer can add criteria for interactivity. In other words, the developer can perform certain common operations. For instance, the developer can use set properties option to determine the text in a TextView. The properties related to this function are different for various View subclasses.

Other types of operations that a developer can perform on views are Set Focus, Setup Listeners, and Set Visibility.

- **Set Focus:** When this criterion has been added, it controls the moving focus for views depending on the inputs from the user.
- **Setup Listeners:** This provides the option of setting notifications for listeners for certain changes in the View. For instance, when there is a change in the focus, the listener can be notified.
- **Set Visibility:** When this criterion has been added, the user can hide or show Views.

Other concepts related to Views include:

- **Attributes**

  Every View and ViewGroup object possesses its own range of XML attributes. Some attributes are specific to a View object - for example the TextView supports the textSize attribute. However, these attributes are also inherited by any View objects that may extend this class. There are other attributes that are common to all View objects, because they are inherited from the root View class. One of these attributes is ID which is common to all. Following is the list of some common attributes present in an object of the View class:

  - ID
  - Setting the Padding and Size of a View
  - Setting View Location

  ID

  Each View object will have an integer ID associated with it, which uniquely identifies it as a resource within the tree. The attribute is used to provide reference to an application when it is compiled. The ID attribute will be assigned in the XML file as a string. This is an XML attribute common to all View objects and is defined by the View class.

  The syntax used for an ID inside an XML is as follows:

  Syntax:

  ```
  android: id="@+id/my Button"
  ```

  Where,

  '@' - symbol indicates that the XML parser should parse and expand the ID string and also recognize it as a resource.

  '+' - symbol identifies it as a new resource and adds it as a resource when required.

Note - When the developer provides a reference to an Android resource ID, the name of the Android package must replace the '+' symbol.

Setting the Padding and Size of a View

The developer can set the size and padding of a View. Setting the padding helps to measure the dimensions of the View. The padding can be provided for the left, right, top and bottom of the View area in pixels. The set Padding (int, int, int, int) method can be used to set the padding for a View. The developer can use the different getPaddingXXX() method to obtain the current setting. XXX stands for left, top, and so on. For instance, the query getPaddingLeft() displays the current setting for the left part of the View area.

The width and height of the View area can be set to determine the View size. The developer can set the width and height using measured width and measured height options in the layout. For instance, specifying measure (int, int) in the layout determines the width and height of all Views in the layout. Using layout (int, int, int, int) specifies the measurement for the child views.

Setting View Location

A View is in the shape of a rectangle. The View's location is specified using left and top co-ordinates and the dimensions of View are expressed as width and height. The unit of measurement is in pixels. A View's location can be obtained using getLeft() and getTop() method which returns the X and Y co-ordinates respectively. The location of the View is returned which is relative to the parent view.

Layout Parameters

The attribute layout_samplename defines the layout parameters for a View object and these parameters confirm to the ViewGroup in which the View resides. The nested class inside the ViewGroup class extends the ViewGroup.LayoutParams. This nested subclass contains property types for defining the size and position of child views. The ViewGroup class also includes many other parameters such as height, width, and so on and each view is required to define them. The width and height of the view is normally set using the built-in constants as it ensures that the application will display the UI properly in varying screen sizes. The two constants used are as follows:

- **wrap_content**: Indicates that the View should size itself according to the dimension required by the content.
- **match_parent**: Indicates that the View should be as large as the parent view group.

Some of the common types of Views in Android include:

- **Basic Views**: Views such as 'TextView', 'EditText', and 'Button' are most frequently used.

- **Picker Views**: Views that allow users to choose a desired format/design from a list. Examples are the 'TimePicker' and 'DatePicker' Views.
- **List Views**: Views that display a detailed list of items. This includes the 'ListView' and the 'Spinner' Views.
- **Display Views**: Views that display images, such as the 'Gallery' and 'ImageSwitcher' Views.
- **Menus**: Views that display additional and context-related menu items.

Figure 4.2 illustrates the different views in Android.

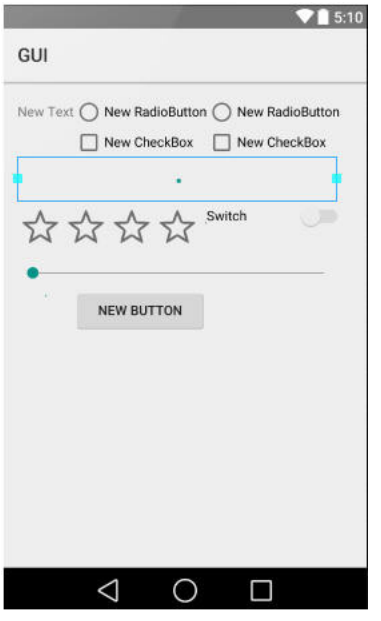


Figure 4.2: Different Views in Android

Other settings that the developer can work on include drawing, focus handling, event handling, settings for touch mode, and security settings.

- When drawing settings are provided, the View is able to draw according to the specified setting without invoking the onDraw() option.
- Focus handling refers to the way in which movement of Views responds to user inputs. The setFocusable() option can be used for focus handling. For the touch mode, Views determine the focus from the setFocusableInTouchMode() option.
- View settings can be customized in a way that focus is handled smoothly irrespective of whether the user is in the touch mode or not. Interactive options, such as buttons need to be provided for touch mode. When the user switches from the touch mode, the device will find a View and establish focus.
- When it comes to event handling, a View is designated an event, which it handles and makes notifications to the listeners if required. During event processing, the View's appearance and measurements can be changed.

Finally, a View can also be customized by overriding the standard options. Overrides can be set for View creation, layout, drawing, event processing, focus, and window attachment.

4.2.3 Overview of Layouts

A layout is an extension of ViewGroup class. It defines the visual structure of the UI. It mainly comprises interconnected child views. It helps to define the architecture of the UI present in an Activity. Therefore, it can be used for developing complex interfaces.

The developer can either write code for specifying a layout or create an XML file. Creating an XML file is a simpler way of specifying a layout. Every element in the XML file is either a View or a ViewGroup object. The Android framework gives the developer flexibility to use either or both methods to declare and control the UI layout.

Advantages of declaring UI in XML

The advantages of declaring the UI in XML are as follows:

- It enables the developer to separate the presentation of your application from the application logic that controls its behavior.
- The developer's UI descriptions are external to the application code, so that the developer can make changes without having to modify the source code and recompile.
- Declaring the layout in XML also helps in easy visualization of the structure of the UI.
- XML declarations are reflected in the GUI editor of the IDE.
- UI declared in XML can easily be reused in other projects.

  As a result, it is easier to debug problems.

General rules for writing layout in XML

Using Android's XML vocabulary, the developer can quickly design UI layouts and their screen elements. It is very similar to creating Web pages with the help of HTML that is, by using a series of nested elements. The layout is specified in activity_main.xml file present in the res/layout/ folder.

Here are some general rules for writing layout in XML:

1. Declaring UI elements in XML follows the same structure that one follows while naming classes and methods. In other words, it means that element and attribute name of UI elements matches the class name and method names respectively.
2. The direct relation between the class name and methods with that of UI element name and its attributes helps the developer to understand easily.

3. Vocabulary is same except for some instances where there are naming differences. For example, the EditText element has a text attribute that corresponds to EditText.setText().
4. Each layout file must contain exactly one root element, which must be a View or ViewGroup object.
5. The developer needs to define the root element and then add additional layout objects or widgets as child elements to build a 'View' hierarchy that defines the layout.
6. Once the developer declares the default layout in a XML file, it is necessary to save it with the .xml extension in Android projects res/layout/ directory to compile it properly.

Thus, to provide a layout in the XML file, the developer must name the XML element for a View with relation to the View class it signifies. For instance, the <EditText> element creates an <EditText> widget.

Code Snippet 1 displays a simple RelativeLayout which looks similar to figure 4.2.

Code Snippet 1:

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
tools:context=".MainActivity">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="New Text"
        android:id="@+id/textView"
        android:layout_alignParentTop="true"
        android:layout_alignParentStart="true" />
```

```
<RadioButton
    android:layout_width=”wrap_content”
    android:layout_height=”wrap_content”
    android:text=”New RadioButton”
    android:id=”@+id/radioButton”
    android:layout_alignBottom=”@+id/textView”
    android:layout_toEndOf=”@+id/textView” />
<RadioButton
    android:layout_width=”wrap_content”
    android:layout_height=”wrap_content”
    android:text=”New RadioButton”
    android:id=”@+id/radioButton2”
    adroid:layout_alignBottom=”@+id/radioButton”
    android:layout_toEndOf=”@+id/radioButton” />
<CheckBox
    android:layout_width=”wrap_content”
    android:layout_height=”wrap_content”
    android:text=”New CheckBox”
    android:id=”@+id/checkBox”
    android:layout_below=”@+id/radioButton”
    android:layout_toEndOf=”@+id/textView” />
<CheckBox
    android:layout_width=”wrap_content”
    android:layout_height=”wrap_content”
    android:text=”New CheckBox”
    android:id=”@+id/checkBox2”
    android:layout_below=”@+id/radioButton”
    android:layout_alignEnd=”@+id/radioButton2”
```

```
        android:layout_toEndOf="@+id/radioButton" />
    <ProgressBar
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/progressBar"
        android:layout_below="@+id/checkBox"
        android:layout_alignParentStart="true"
        android:layout_alignEnd="@+id/checkBox2"
        android:progress="50" />
    <RatingBar
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/ratingBar"
        android:layout_below="@+id/progressBar"
        android:layout_alignParentStart="true"
        android:layout_toStartOf="@+id/radioButton2" />
    <Switch
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="New Switch"
        android:id="@+id/switch1"
        android:layout_below="@+id/progressBar"
        android:layout_alignEnd="@+id/progressBar" />
    <SeekBar
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/seekBar"
        android:layout_below="@+id/ratingBar"
```

```
        android:layout_alignParentStart="true"
        android:layout_alignEnd="@+id/radioButton2"
        android:indeterminate="false" />
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="New Button"
        android:id="@+id/button"
        android:layout_below="@+id/seekBar"
        android:layout_alignEnd="@+id/ratingBar" />
</RelativeLayout>
```

- **Load the XML Resource**

As already discussed, when the developer has finished creating the XML layout, it must be saved in the res/layout/ directory with a '.xml' extension. The XML file in an application is compiled into a View resource. The developer must load the View resource in the application code. This is done in the onCreate() callback method of the Activity class. A reference to the View resource must be provided in the setContentView() call back feature as well. For instance, if the XML file is saved as activity_main.xml, it must be loaded in the application code for the Activity class.

Code Snippet 2 displays the XML file activity_main.xml being loaded in the application code 2.

Code Snippet 2:

```
@Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
```

- **Reference the Widget Programmatically**

The developer can also use the XML layout to add View object and provide references to them in the application source code.

To do this, the developer has to perform the following steps:

1. Define a View or widget in the layout file, activity_main.xml and assign a unique ID. Code Snippet 3 shows the declarations in the layout file of a Button widget.

Code Snippet 3:

```
<Button
        android:layout_width=”wrap_content”
        android:layout_height=”wrap_content”
        android:text=”New Button”
        android:id=”@+id/button”
        android:layout_below=”@+id/seekBar”
        android:layout_alignEnd=”@+id/ratingBar” />
```

Note - It is important to designate an ID for a relative layout to enable sibling views to determine their layout relative to other sibling views based on the ID.

2. Create an instance for the View object and obtain its reference programmatically from the layout file in the onCreate() method of the Activity class. Code Snippet 4 demonstrates how to capture the widget from the layout.

Code Snippet 4:

```
Button buttonName = (Button) findViewById(R.id.button);
```

Android supports different layouts or ViewGroup to construct the UI. The most common layouts include:

1. Linear Layout
2. Relative Layout
3. Table Layout
4. Absolute Layout
5. Frame Layout
6. List View
7. Grid View

Table 4.1 lists some of the common attributes of a ViewGroup.

Attributes	Description
layout_width	Defines the width of the ViewGroup
layout_height	Defines the height of the ViewGroup
layout_marginTop	Defines the space on the top of the ViewGroup
layout_marginBottom	Defines the space on the bottom of the ViewGroup
layout_marginLeft	Defines the space on the left of the ViewGroup
layout_marginRight	Defines the space on the right of the ViewGroup
layout_x	Defines the x-coordinate of the ViewGroup
layout_y	Defines the y-coordinate of the ViewGroup

Table 4.1: Common Attributes of ViewGroup

Getters and setters are available for these attributes.

4.3 Working with Layouts

While working with layouts, it is preferable that the developers work using XML from external resources. The XML layout will comprise a root node. This root node can have multiple nested layouts and views for designing the UI. The various layouts for an Android application are explained in detail.

☐ XML for UI Layouts

Android Studio comes with an in built designer which can be used for designing User Interfaces Graphically. The developer can use the design view or the XML view to design the User Interface.

1. **Design (Graphical) View**: This allows the user to preview how the controls will appear on various screens and orientation as shown in figure 4.3.

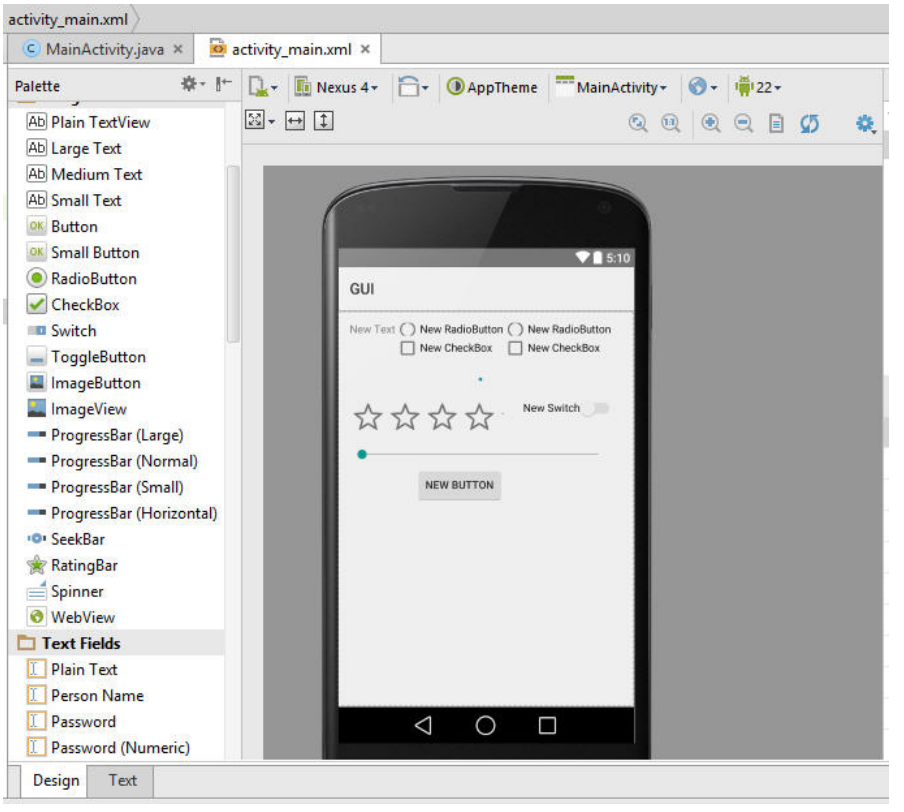


Figure 4.3: Layout Resource Designer in Android Studio

2. **Text (XML) View**: This shows the user the XML definition of the resource as shown in figure 4.4.

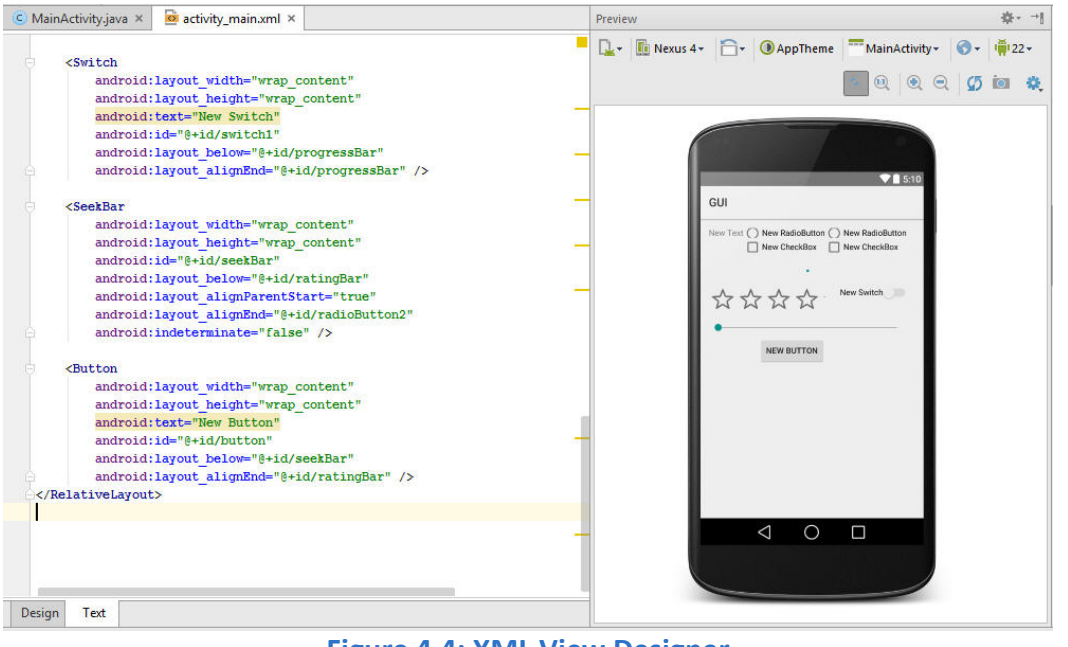


Figure 4.4: XML View Designer

In addition to the GUI Designer tools, Android Studio Designer provides the functionality to add layouts and add widgets to layouts as shown in figure 4.5.

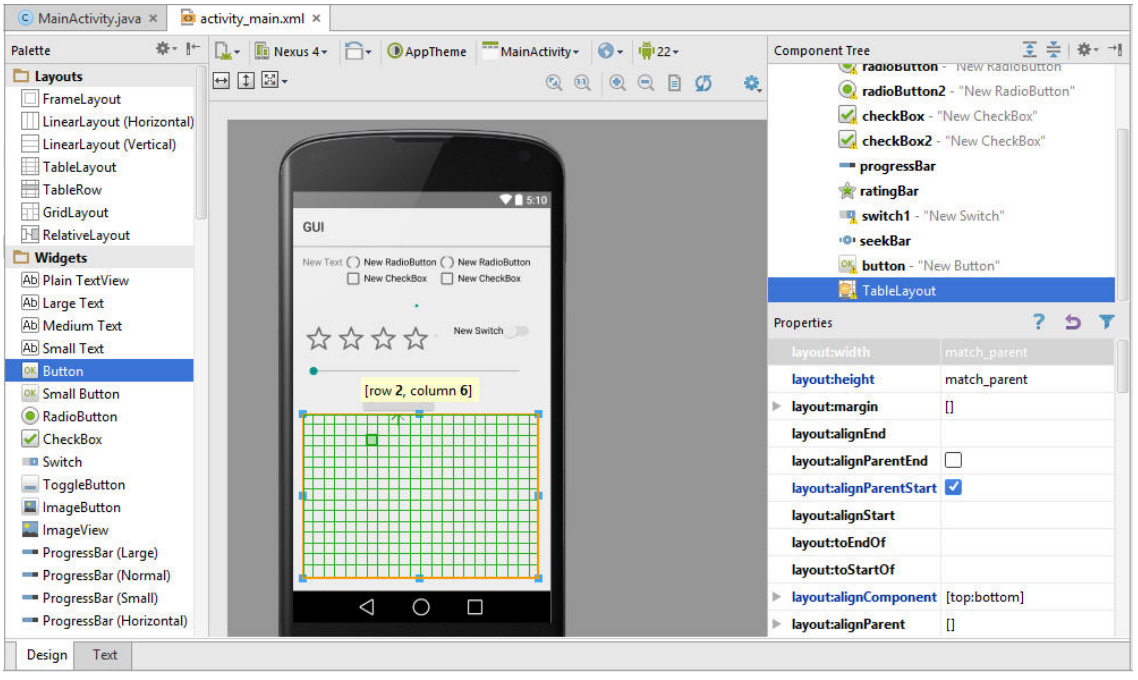


Figure 4.5: Additional Features of Android Studio Designer

Multi Device Designing is also possible and unique to Android studio. This feature allows the developer to test the layout on different devices of different screen sizes and screen resolutions.

While working with the designer in Android Studio:

- Drag and drop Views (widgets) to the layout.
- Double-click a View to bring up the option to edit the basic properties and the 'id' of the View.
- Use the properties panel on the right to edit the properties of the View or layout.
- Use the Text tab to edit XML definition directly.

4.3.1 Linear Layout

It is the simplest type of layout. Linear Layout aligns all its child nodes in a single direction, either vertically or horizontally, depending on the orientation attribute specified by the user. A scrollbar appears if the window length exceeds the length of the screen. The android:orientation attribute helps the developer to specify the layout direction.

This section explains how to create a Linear Layout with vertical orientations.

The steps to create a LinearLayout are as follows:

1. Start Android Studio.
2. Click File → New → Project or select the Start a New Android Studio Project in the welcome screen to display the Create New Project window.
3. Enter the Application Name and Company Domain as LinearLayout and aptech.com respectively. Click Next.
4. Select the Minimum SDK as API 21 and click Next.
5. Select activity type as Blank Activity and click Next.
6. Enter the Activity name as MainActivity and click Finish.
7. Navigate to res/layout.
8. Right–click layout and select New → Layout Resource File.
9. Specify file name as linear_layout and root element as LinearLayout, and click OK.
10. Modify the code in the linear_layout.xml file as shown in Code Snippet 5.

Code Snippet 5:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textEmailAddress"
        android:ems="10"
        android:id="@+id/userName" />
    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textPassword"
```

```
        android:ems="10"
        android:id="@+id/passWord" />
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Login"
        android:id="@+id/button" />
</LinearLayout>
```

11. Modify the code in MainActivity.java file to load the linear_layout.xml file as the layout view as shown in Code Snippet 6.

Code Snippet 6:

```
package aptech.com.linearlayout;
import android.app.Activity;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
public class MainActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.linear_layout);
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if it
is present.
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
```

```
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
  // Handle action bar item clicks here. The action bar will
// automatically handle clicks on the Home/Up button, so long
 // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
}
```

12. Click File → Save All to save all the changes.
13. Click Run → Run 'app'.

The emulator starts with the LinearLayout project. The output of the project will be as shown in figure 4.6.

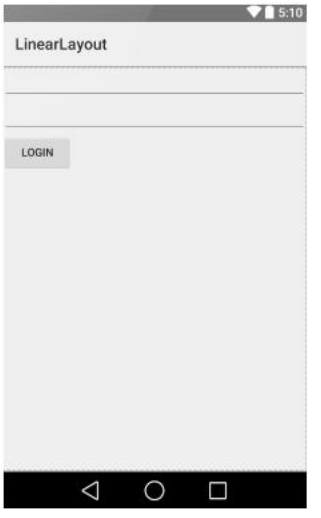


Figure 4.6: LinearLayout

In the Code Snippet, the attributes used are as follows:

- **android:layout_width="match_parent"**: It specifies the width of the layout. The width may either have the value set to wrap_content or match_parent which means that the view must be as large to enclose its content or matches the size of the parent view respectively.
- **android:layout_height="match_parent"**: It specifies the height of the layout.
- **android:orientation="vertical"**: It specifies the orientation of the layout. The two types of orientations are vertical and horizontal, which display layout either horizontally and vertically.

4.3.2 Relative Layout

In this type of layout, the developer can specify the location of child objects relative to each other or to the parent. A relative layout is very useful for designing a user interface because it can remove the need for nested view groups keeping the layout tree simple, thus, improving performance. If a developer has several nested linear layout groups, the groups could be replaced with a single relative layout instead.

The steps to create a Relative Layout are as follows:

1. Create a new project named RelativeLayout.
2. Navigate to res/layout, Right-click and select New → Layout Resource File.
3. Specify file name as relative_layout and root element as RelativeLayout and Click OK.
4. Modify the code in the relative_layout.xml file as shown in Code Snippet 7.

Code Snippet 7:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:id="@+id/relativeLayout">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/user_name"
```

```
        android:id="@+id/userName"
        android:layout_alignParentTop="true"
        android:layout_alignParentStart="true" />
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/pass_word"
        android:id="@+id/passWord"
        android:layout_below="@+id/editText"
        android:layout_toStartOf="@+id/editText" />
    <EditText
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:inputType="textEmailAddress"
        android:ems="10"
        android:id="@+id/editText"
        android:layout_alignParentTop="true"
        android:layout_toEndOf="@+id/userName" />
    <EditText
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:inputType="textPassword"
        android:ems="10"
        android:id="@+id/editText2"
        android:layout_below="@+id/editText"
        android:layout_toEndOf="@+id/userName" />
    <Button
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
        android:text="@string/login_text"
        android:id="@+id/login"
        android:layout_marginTop="32dp"
        android:layout_below="@+id/editText2"
        android:layout_alignStart="@+id/editText2" />
</RelativeLayout>
```

5. Modify the code in MainActivity.java file to load the relative_layout.xml file as the layout view as shown in Code Snippet 8.

Code Snippet 8:

```
package aptech.com.relativelayout;
import android.app.Activity;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
public class MainActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.realtive_layout);
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if
//it is present.
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
    @Override
```

```
    public boolean onOptionsItemSelected(MenuItem item) {
  // Handle action bar item clicks here. The action bar will
// automatically handle clicks on the Home/Up button, so long
   // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
//noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
}
```

6. Navigate to res → values and open strings.xml for editing.
7. Modify the code as shown in Code Snippet 9.

Code Snippet 9:

```
<resources>
    <string name="hello_world">Hello world!</string>
    <string name="action_settings">Settings</string>
    <string name="user_name">User Name</string>
    <string name="pass_word">Password</string>
</resources>
```

8. Click File → Save All to save all the changes.
9. Click Run → Run 'app'.

The Emulator starts with the RelativeLayout project. The output of the project will be as shown in figure 4.7.

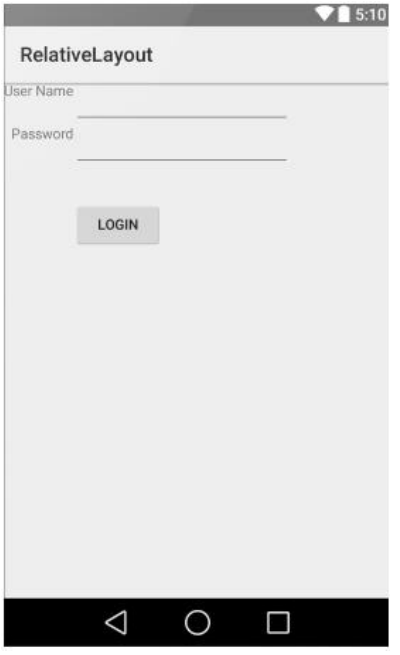


Figure 4.7: Relative Layout - Output

4.3.3 Table Layout

In this type of layout, the screen area is divided in a tabular format. The developer can specify the row and column position for each view inside the layout. It is one of the most commonly used layouts in UI design.

4.3.4 List View

ListView is a view group that displays a list of scrollable items. An Adapter that pulls content from a source, such as a query or an array, helps to insert the list items automatically. Each item result is converted into a View and added to the list by the Adapter.

The steps to create a ListView layout are as follows:

1. Create a new project name ListViewLayout.

2. Modify the code in activity_main.xml as shown in Code Snippet 10.

Code Snippet 10:

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
    tools:context=".ListActivity">
    <ListView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/listView"
        android:layout_alignParentTop="true"
        android:layout_alignParentStart="true" />
</RelativeLayout>
```

The code in the activity_main.xml creates an activity with a ListView. This ListView will be populated with contact details programmatically in MainActivity.java.

3. Modify MainActivity.java as shown in Code Snippet 11.

Code Snippet 11:

```
package aptech.com.listview;
import android.app.Activity;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.widget.ListView;
import android.widget.SimpleAdapter;
```

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
public class ListActivity extends Activity {
    private List<Map<String, Object>> contactsList;
    private ListView listView;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_list);
        contactsList = new ArrayList<Map<String, Object>>();
        listView=(ListView)findViewById(R.id.listView);
        addContacts();
        SimpleAdapter adpater = new
SimpleAdapter(this,contactsList,android.R.layout.simple_list_
item_2,new String[]{"name", "number"}, new int[] {android.R.id.
text1, android.R.id.text2});
        listView.setAdapter(adpater);
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if
it is present.
        getMenuInflater().inflate(R.menu.menu_list, menu);
        return true;
    }
    @Override
```

```
    public boolean onOptionsItemSelected(MenuItem item) {
 // Handle action bar item clicks here. The action bar will
 // automatically handle clicks on the Home/Up button, so long
// as you specify a parent activity in androidManifest.xml.
        int id = item.getItemId();
//noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
    private void addContacts()
    {
        Map<String, Object> contact = new HashMap<String,
Object>();
        contact.put("name","John");
        contact.put("number","+1 248 242 311");
        contactsList.add(contact);
        contact = new HashMap<String, Object>();
        contact.put("name","Peter");
        contact.put("number","+1 248 242 347");
        contactsList.add(contact);
        contact = new HashMap<String, Object>();
        contact.put("name","Mathew");
        contact.put("number","+1 248 287 417");
        contactsList.add(contact);
        contact = new HashMap<String, Object>();
        contact.put("name","Andrew");
```

```
        contact.put("number","+1 948 282 411");
        contactsList.add(contact);
        contact = new HashMap<String, Object>();
        contact.put("name","Marshall");
        contact.put("number","+1 348 882 441");
        contactsList.add(contact);
        contact = new HashMap<String, Object>();
        contact.put("name","Kate");
        contact.put("number","+1 548 281 397");
        contactsList.add(contact);
    }
}
```

ListView requires an adapter to setup the display data. For this purpose, the SimpleAdapter object has been instantiated. The constructor for SimpleAdapter accepts following five arguments: The context, a List<Map<String, Object>> object, a resource, a 'from' String list of column names, and a 'to' map to the column names.

The addContacts method adds contacts data to the contactsList object. Finally, the adapter is set to the ListView by using the setAdpater() method.

4. Click File → Save All to save all the changes.
5. Click Run → Run 'app'.

The Emulator starts with the ListView project. The output of the project will be as shown in figure 4.8.

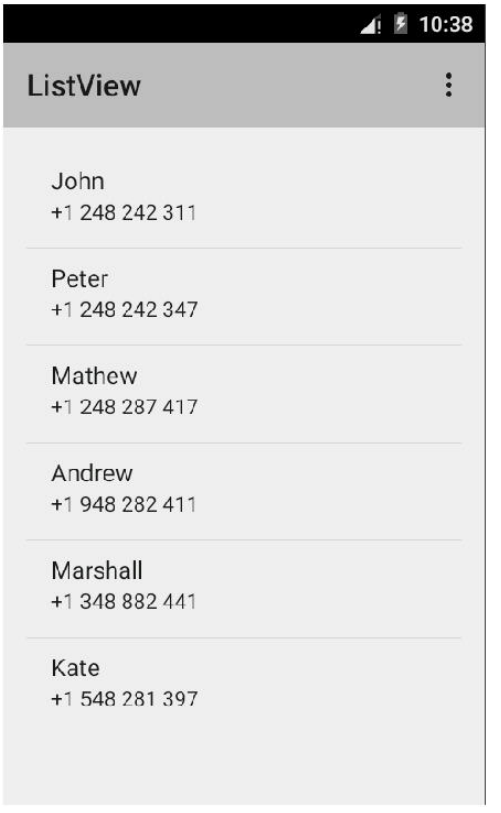


Figure 4.8: Project ListView - Output

4.3.5 Grid View

This layout displays a scrolling grid consisting of rows and columns. GridView is a ViewGroup that displays items in a scrollable grid. The grid items are automatically inserted to the layout using a ListAdapter.

The steps to create a GridView layout are as follows:

1. Create a new project named GridViewLayout.
2. Navigate to GridViewLayout → res → drawable in the Project pane.
3. Copy - paste .jpg or .png files that are to be displayed in the image grid. Save the image files into the project's res/drawable/directory.

   Note - Name of the image should be in lower case and there should be no spaces in the file name.

4. Navigate to GridViewLayout → res → layout → activity_main.xml in the Project pane.
5. Double-click the activity_main.xml file to open it.

6. Modify the code in activity_main.xml file as shown in Code Snippet 12.

Code Snippet 12:

```
<?xmlversion="1.0"encoding="utf-8"?>
<GridView
xmlns:android="http://schemas.android.com/apk/res/
android"
android:id="@+id/gridView1"
android:numColumns="auto_fit"
android:gravity="center"
android:columnWidth="100dp"
android:stretchMode="columnWidth"
android:layout_width="fill_parent"
android:layout_height="fill_parent">
</GridView>
```

7. To create animals.xml file right-click the res/layout/ folder.
8. Click New → Layout Resource File to display the New Layout Reference File dialog box.
9. Type 'animals' in the File name box.
10. Click OK.

    The animals.xml file is displayed and if it is not displayed then double-click and open the file from the project explorer pane.

11. Modify the code in the animals.xml file as shown in Code Snippet 13.

Code Snippet 13:

```
<?xmlversion="1.0"encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:padding="5dp" android:orientation="vertical">
<ImageView
```

```
android:id="@+id/grid_item_image"
android:layout_width="100dp"
android:layout_height="100dp"
android:layout_marginRight="20dp"
android:src="@drawable/ic_launcher">
</ImageView>
<TextView
android:id="@+id/grid_item_label"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="@+id/label"
android:layout_marginTop="5px"
android:textSize="18sp">
</TextView>
</LinearLayout>
```

12. Open the Activity class named MainActivity.java file present is the src folder.
13. Modify the code of the onCreate() method as shown in Code Snippet 14.

Code Snippet 14:

```
package com.example.gridviewlayout;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.view.View;
import android.widget.AdapterView;
import android.widget.GridView;
import android.widget.TextView;
import android.widget.Toast;
import android.widget.AdapterView.OnItemClickListener;
public class MainActivity extends Activity {
```

```
GridView gridView;
staticfinal String[] IMAGES = new String[] { “Image 1”,
“Image 2”, “Image 3”, “Image 4” };
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
gridView = (GridView) findViewById(R.id.gridView1);
gridView.setAdapter(new ImageAdapter(this, IMAGES));
gridView.setOnItemClickListener(new
OnItemClickListener() {
public void onItemClick(AdapterView<?> parent,
View vw,
int position, long id) {
Toast.makeText(getApplicationContext(),
“Click ListItem Number ”+ position,Toast.LENGTH_ LONG)
                                                .show();
}
});
}
@Override
public boolean onCreateOptionsMenu(Menu menu) {
// Inflate the menu; this adds items to the action bar if it
//is present.
getMenuInflater().inflate(R.menu.main, menu);
returntrue;
}
}
```

In the code, the activity_main.xml layout is set for the content view and the Grid is captured from the layout using findViewById(int)method.

The setAdapter() method then sets a custom adapter named ImageAdapter as the source for all items to be displayed in the grid.

To initiate an action when an item in the grid is clicked, the setOnItemClickListener() method passes a new anonymous instance of AdapterView.OnItemClickListener. This anonymous instance defines the onItemClick() callback method to display a Toast message. The message displays the index position (zero-based) of the selected item (in a real world scenario, the position could be used to get the full sized image for some other task).

14. To create a new class called ImageAdapter that extends the BaseAdapter class, right-click src → com.example.gridviewlayout folder to display the context menu.
15. Click New → Class to display the New Java Class dialog box.
16. Type ImageAdapter in the Name box.
17. Type the code as shown in Code Snippet 15.

Code Snippet 15:

```
package com.example.gridviewlayout;
import android.content.Context;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.BaseAdapter;
import android.widget.ImageView;
import android.widget.TextView;
imageView.setImageResource(R.drawable.lighthouse);
}
} else {
gridView = (View) convertView;
}
return gridView;
}
@Override
public int getCount() {
```

```
returnvalues.length;
}
@Override
public Object getItem(int position) {
returnnull;
}
@Override
public long getItemId(int position) {
return 0;
}
}
```

The code implements some required methods inherited from BaseAdapter class. The constructor and getCount()method are self-explanatory. The getView() method creates a new View object for each image added to the ImageAdapter class. In other words, it displays the image at the specified position in the data set. When this method is invoked, a View object is passed and a check is performed to see if the object is null. If it is null, an object of ImageView class is instantiated and configured with desired properties for the image presentation.

This class will be used to access the data. In this case, it will access the images. The getCount() method is overridden and returns a count of the total number of images present in the data set. The getItem() method returns the image present at the specified position in the data set. Finally, the setImageResource() method is used to initialize the Image object by passing image name.

18. Execute the application by selecting Run → Run 'app'.

The output of the application will be as shown in figure 4.9.

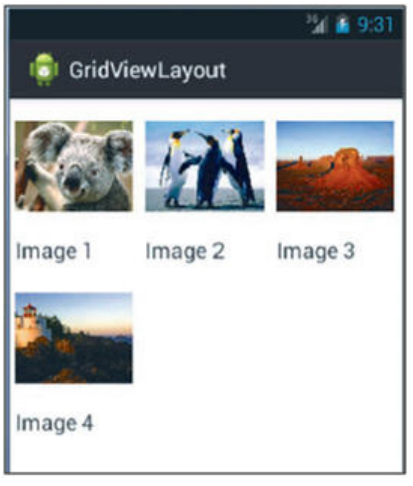


Figure 4.9: GridView Layout

Note - If any of the images is clicked, its position is displayed.

4.4 UI Components

In Android, UI components are interactive controls in an app's UI that enables the user to perform a wide range of actions. Through UI components, the user can interact with the application. Some of the basic UI components are Button, TextView, DatePicker, ProgressBar, and so on as shown in figure 4.10.

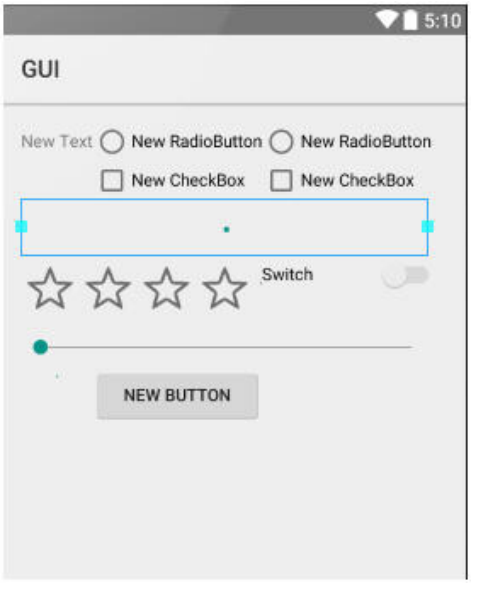


Figure 4.10: UI Input Control Components

4.4.1 Input Controls

As discussed, Android provides a number of input controls. The different input controls are listed in table 4.2.

Input Controls	Description	Code Snippet
Button	It consists of a text or icon or a combination of both, that communicates what action would occur if user clicks it.	`<Button android:layout_height="wrap_ content" android:text="@ string/ button_text" android:drawableLeft="@ drawable/button1_icon"/>`

Input Controls	Description	Code Snippet
Check box	It allows the user to select one or multiple options from available set. When the user selects a CheckBox, the CheckBox object receives an on-click event. To create each check box option, create a CheckBox in the layout. Each check box is managed separately, hence it is necessary to register an event listener for each.	<CheckBox xmlns:android="http:// schemas. android.com/apk/res/ android" android:id="@+id/ checkBox1" android:layout_ width="wrap_content" android:layout_ height="wrap_content" />
Radio button	It allows users to select one option from a set. When the user selects one of the radio buttons, the corresponding RadioButton object receives an onClick event.	<?xml version="1.0" encoding="utf-8"?> <RadioGroup xmlns:android= "http://schemas.android. com/ apk/res/android" android:layout_width="fill_ parent" android:layout_ height="wrap_content" android:orientation= "vertical"> <RadioButton android :id="@+id/ radio1" android:layout_ width= "wrap_content" android:layout_ height= "wrap_content" android:text="@ string/Male" android:onClick="onRadio Button Clicked" /> <RadioButton android: id="@+id/ radio2" android:layout_ width= "wrap_content" android: layout_ height= "wrap_ content" android:text= "@ string/ Female" android: onClick="onRadioButton Clicked" /> </RadioGroup>

Input Controls	Description	Code Snippet
Spinner	It enables the user to quickly select an option or value from a drop-down list. Spinners provide a quick way to select one value from a set. In the default state, a spinner shows its currently selected value. When the user touches the spinner, it displays a drop-down menu with all other available values, from which the user can select a new one.	<Spinner xmlns:android= ”http://schemas. android.com /apk/ res/ android” android:id=”@+id/ spinner1” android:layout_ width= ”wrap_content” android: layout_ height= ”wrap_content” />
Picker	It enables the user to select time or pick a date in required format like AM/PM or DD-MM-YY. It is recommended that the developer utilizes DialogFragment to host each time or date picker. The DialogFragment manages the dialog lifecycle and allows display of the pickers in different layout configurations, such as in a basic dialog on handsets or as an embedded part of the layout on large screens.	<DatePicker xmlns: android=”http:// schemas .android.com/ apk/ res/android” android:id= ”@+id/ datePicker1” android:layout_width= ”wrap_ content” android:layout_ height=”wrap_ content” />
Switch	It enables the user to turn ON or turn OFF a certain feature. It acts as a GUI representation of a Boolean value.	<Switch android:layout_ width=”wrap_content” android:layout_ height=”wrap_content” android:text=”New Switch” android:id=”@+id/ switch1”/>

Input Controls	Description	Code Snippet
SeekBar	It acts similar to a horizontal slider on a desktop. It allows the user to slide the bar to set it to an arbitrary value.	<SeekBar android:layout_ width=”wrap_content” android:layout_height =”wrap_content” android:id=”@+id/seekBar” android:layout_below=”@+id/ ratingBar” android:layout_ alignParentStart=”true”/>
Toggle Button	It acts as a single valued check box, which cannot be grouped together	<ToggleButton android:layout_width=”wrap_ content” android:layout_ height=”wrap_content” android:text=”New ToggleButton” android:id=”@+id/ toggleButton” android:layout_below=”@+id/ button”/>
Text Input	It acts as a line edit to accept text input form the user.	<EditText android:layout_width=”wrap_ content” android:layout_height =”wrap_content” android:id=”@+id/ editText”/>

Table 4.2: Input Controls

4.4.2 Display View

Android provides a number of display views to show output to the user. Some of them are listed in table 4.3.

Display View	Description	Code Snippet
Text View	It is used to display text to the user.	`<TextView android:layout_width ="wrap_content" android:layout_ height="wrap_content" android:text="New Text" android:id="@+id/ textView" android:layout_ centerVertical="true" android:layout_ alignParentStart="true" />`
Progress Bar	It is used to display the progress of a process to the user.	`<ProgressBar android:layout_width ="wrap_content" android:layout_height ="wrap_content" android:id="@+id/ progressBar2" />`
Image View	It is used to display an image to the user such as a logo.	`<ImageView android:layout_width ="wrap_content" android:layout_height ="wrap_content" android:id="@+id/ imageView" />`
Video View	It is used to show a video within the application.	`<VideoView android:layout_width ="wrap_content" android:layout_height ="wrap_content" android:id="@+id/ videoView" />`

Table 4.3: Display Widgets

4.4.3 Menus

Menu is an UI component which holds several items that provide navigation or settings or more functionality to an application. It is a common interface component seen in Android phones and appears when menu buttons on the device are clicked. The menu displays all available options. Android 4.0 onwards, the standard menu button on device has been replaced with the Action bar menu button as shown in figure 4.11.

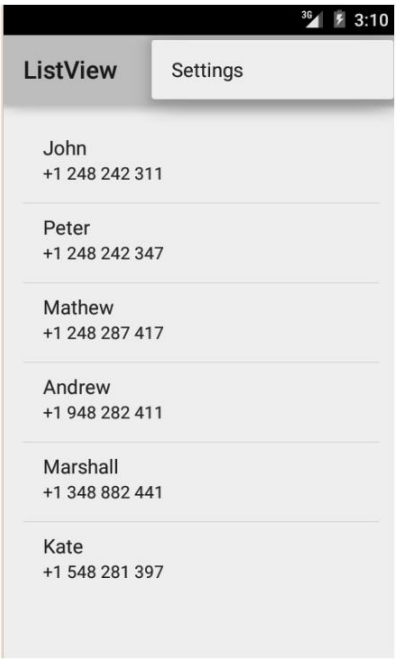


Figure 4.11: Action Menu in Android

4.4.4 Action Bar

Action bar is always present at the top of the Android screen, which is used to identify the application, user location, user action, and other user interface elements, through which user can navigate or perform an action. It provides user actions and navigation modes. Commonly used actions can directly be taken from options menu and placed in the action bar. Other options are available in 'overflow menu' button in the action bar as shown in figure 4.12.

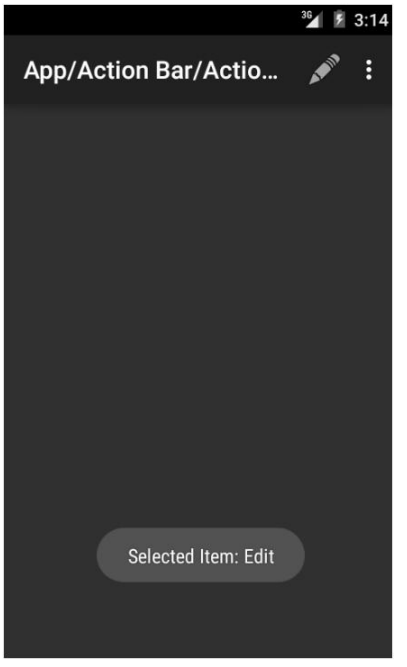


Figure 4.12: Action Bar

4.4.5 Dialogs

Dialogs are prompt or alert displayed to the user to take a decision or to input any information. The dialogs are also used to notify user when a task has been completed. It does not fill the entire screen and usually appears when a user has to take a particular action before proceeding.

The different types of Dialogs are AlertDialog, Toast, TimePicker, DatePicker, and so on.

4.4.6 Some Additional Views

Other than the Views discussed, Android contains some important Views such as:

- **TabHost** – Is used to maintain tabs in an application such as in default contacts application. Each tab contains child layout to navigate within.
- **WebView** – Is used to load URL. WebView is like a browser which will display the Web content within.
- **SearchView** – Is used to provide search capability with a search provider within the application.

4.4.7 Status Notifications

Notifications are used to notify or provide alerts to the user when a message or notification arrives. Notification contains icon, title, body, and the notification arrival time. The Notification Manager ensures that the status bar icons are updated regularly. A status bar notification displays an icon on the status bar along with a message. When the notification is chosen, an Intent is sent by Android to launch the Activity. The status bar notification can be initiated by an Activity or Service class.

A status bar notification is created when the background service wants to inform the user about an event that requires the user to respond. The user can view the details by opening the 'Notifications drawer' as shown in figure 4.13.

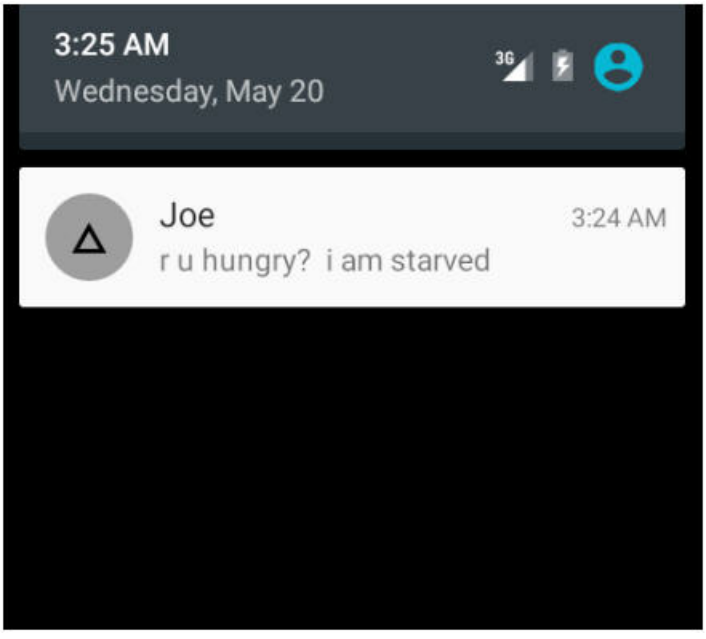


Figure 4.13: Notification in Notification Drawer

4.4.8 Toasts

These are simple messages that provide feedback about a user's action in a popup window. It is displayed in a small window and does not interfere with the user's ongoing activity.

Figure 4.14 displays a Toast message.

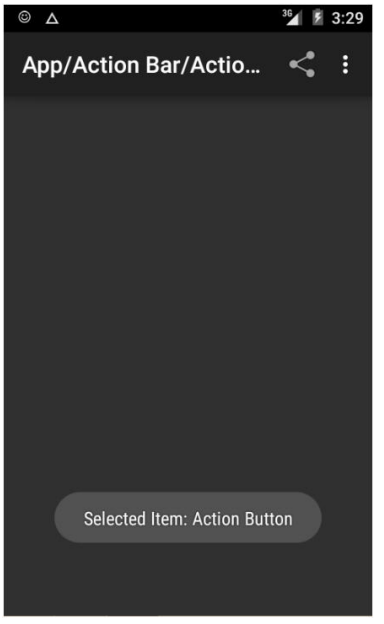


Figure 4.14: Toast Message

4.4.9 Search

It is an important feature in Android and enables user to search for an item in the gadget or the Internet by entering a keyword, as shown in figure 4.15.

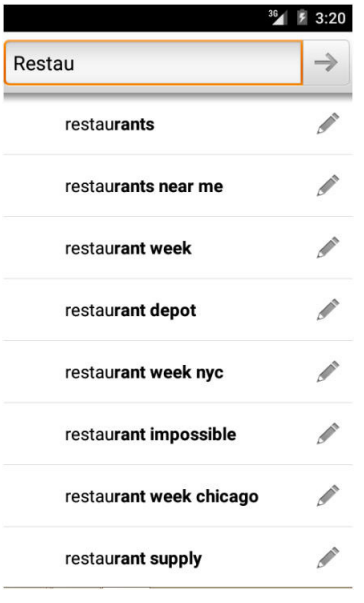


Figure 4.15: Search

4.4.10 Drag and Drop

This UI component allows user to move data from one View to another using a graphical drag and drop gesture. The framework consists of drag event class, drag listeners, helper methods, and classes.

4.4.11 Accessibility

Android has accessibility features to cater to the needs of users with special challenges like visual impairment or hearing difficulty. This includes 'text-to-speech' converter, audio prompting, gesture navigation, trackball, and directional pad navigation.

4.5 Styles and Themes

Presentation and formatting of an application can be improved using Styles and Themes. Android does possess default styles and themes. A style can be referred to as a collection of properties that specify the look and format for a View or window in UI. A style can specify properties such as height, padding, font color, font size, background color, and much more. A style is defined in an XML resource that is separate from the XML that specifies the layout. Styles in Android are similar to cascading stylesheets in Web design. In other words, they allow the user to separate the design from the content.

All of the attributes related to style can be removed from the XML layout file and incorporated into a style definition file. This can be named by any arbitrary name. This is then applied with the style attribute.

A theme is a style applied to an entire Activity or application, rather than an individual View. When a style is applied as a theme, every View in the Activity or application will apply each style property that it supports.

The user can create a set of styles and save them in an XML file in the res/values/ directory of the project. It should also be noted that the root node of the XML file must be <resources>.

4.5.1 Defining Styles and Theme

To create a style it is necessary to add a <style> element to the file with a name attribute that uniquely identifies the style. This attribute is not optional. To this <style> element, an <item> element is added to represent each property of that style. The name attribute of the item element is used for declaring the style property and a value to go with it. This attribute is not optional (this attribute is required). The value for the <item> element can be a keyword string, color represented in a hexadecimal format, a reference to another resource type or other value depending on the style property.

The example will explain how to apply styles in detail.

The steps to create a StyleExample project are as follows:

1. Create a new application called StyleProject in Android Studio.

2. Create a new xml file named newstyle inside the /res/value directory by right-clicking the res/value folder to display the context menu.
3. Select New → Values Resource File.
4. Modify the code as shown in Code Snippet 16.

Code Snippet 16:

```
<?xmlversion="1.0"encoding="UTF-8"?>
<resources>
<stylename="NewStyle"parent="@android:style/TextAppearance.
Medium">
<itemname="android:layout_width">fill_parent
</item>
<itemname="android:layout_height">wrap_content
</item>
<itemname="android:background">#551033
</item>
<itemname="android:textColor">#86C67C
</item>
<itemname="android:typeface">monospace
</item>
</style>
</resources>
```

5. Navigate and open res → layout → activity_main.xml file.
6. Add the <style> element in <TextView> as shown in Code Snippet 17.

Code Snippet 17:

```
<Relative Layout
xmlns:android="http://schemas.android.com/ apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
```

```
android:paddingBottom="@dimen/activity_vertical_margin"
android:paddingLeft="@dimen/activity_horizontal_margin"
android:paddingRight="@dimen/activity_horizontal_margin"
android:paddingTop="@dimen/activity_vertical_margin"
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
style="@style/NewStyle"
android:text="@string/hello_world"/>
</RelativeLayout>
```

Once you run the application, the output will be as shown in figure 4.16.

Figure 4.16: Adding Styles

4.5.2 Inheriting Styles

The <style> element has the parent attribute which allows the user to specify a style from the parent and inherit its properties. In other words, the user can utilize this to inherit properties from an existing style and then can define only the properties that are required to be modified.

It is possible to inherit the Android platform's default text appearance and then modify it according to developer's needs.

If the developer wishes to inherit from self-defined styles, it is not necessary to use the parent attribute. Instead, the developer can just prefix the name of the style to be inherited to the name of the new style, separated by a period.

For example, to create a new style that inherits the **NewStyle** style defined, but make the color red, the user can author the new style as shown in Code Snippet 18.

Code Snippet 18:

```
<stylename="NewStyle.Red">
<item name="android:textColor">#FF0000</item>
</style>
```

4.5.3 Style Properties

The best place to find properties that apply to a specific View is the corresponding class reference, which lists all the supported XML attributes. For example, all the attributes listed in the table of TextView XML attributes can be used in a style definition for a TextView element (or one of its subclasses).

Some style properties are not supported by any View element and can only be applied as a theme. These style properties apply to the entire window and not to any type of View. For example, style properties for a theme can hide the application title, hide the status bar or change the window's background. This kind of style properties do not belong to any View object. To find out theme-only style properties, the user can look at the 'R.attr' reference for attributes that begin with window.

4.5.4 Application of Styles and Themes to the UI

A style can be set in the following two ways:

1. Adding the 'style' attribute to a View element in the XML for the layout. This is added to an individual View.
2. Adding the android:theme attribute to the <activity> or <application> in the Android manifest file that sets the style to the entire application.

When the developer applies a style to a single View in the layout, the properties defined by the style are applied only to that View. If a style is applied to a ViewGroup, the child View elements will not inherit the style properties. It will be applied to only the element where the style and its properties are applied. However, the style can be applied as a theme by the developer so that it applies to all View elements.

To apply a style definition as a theme, the user must apply the style to an Activity or application in the Android manifest. Every View within the Activity or application will apply each property that it supports.

4.5.5 Application of a Theme for an Activity or Application

To set a theme for all the activities of an application, open the **AndroidManifest.xml** file and edit the <application> tag to include the android:theme attribute with the style name.

If the theme is to be applied to a single Activity in the application, then the android:theme attribute is required to be added to the <activity> tag instead. Just as Android provides other built-in resources, there are many pre-defined themes that can be used.

If you like a theme but want to tweak it, just add the theme as the parent of your custom theme.

The example will explain how to use Theme in an application.

The steps to create a ThemeExample project are as follows:

1. Create a New Project called ThemeExample in Android Studio.
2. Navigate to res → values → styles.xml file in the Projects pane.
3. Double-click styles.xml file.
4. Modify the code as shown in Code Snippet 19.

Code Snippet 19:

```
<resources>
<stylename="NewStyle">
<itemname="android:windowNoTitle">true</item>
<itemname="android:text">This is a theme.</item>
<itemname="android:textColor">#66FF66</item>
<itemname="android:textSize">30sp</item>
<itemname="android:windowBackground">@drawable/lighthouse</ item>
</style>
</resources>
```

5. Double-click AndroidManifest.xml file.

6. Modify the code under <application> tag in AndroidManifest.xml file to apply the android:theme attribute as shown in Code Snippet 20.

Code Snippet 20:

```
<?xmlversion="1.0"encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/
android"
package="com.example.themeexample"
android:versionCode="1"
android:versionName="1.0">
<application
android:allowBackup="true"
android:icon="@drawable/ic_launcher"
android:label="@string/app_name"
android:theme="@style/NewStyle" >
<activity
android:name="com.example.themeexample.MainActivity"
android:label="@string/app_name">
<intent-filter>
<actionandroid:name="android.intent.action.MAIN"/>
<categoryandroid:name="android.intent.category.LAUNCHER"/>
</intent-filter>
</activity>
</application>
</manifest>
```

The output of the project will be as shown in figure 4.17.

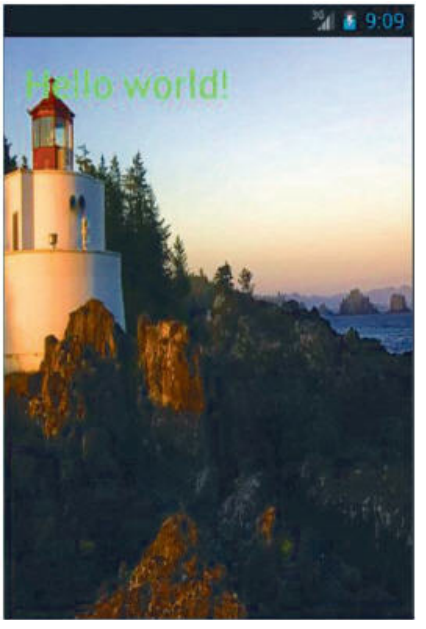


Figure 4.17: Themes in Application

- **Using Platform Styles and Themes:**

  The Android platform provides a large collection of styles and themes that can be used in applications. The user can find a reference of all available styles in the 'R.style' class. To use the styles listed there, replace all underscores in the style name with a period.

  Code Snippet 21 displays the use of Dialog theme in AndroidManifest.xml.

Code Snippet 21:

```
<activity android:theme="@android:style/Theme.Dialog">
```

It should be noted that the 'R.style' reference is not well documented and does not thoroughly describe the styles. So, viewing the actual source code for these styles and themes will give the user a better understanding of what style properties each one provides.

4.6 Handling User Events

Android provides a number of ways to intercept the events generated by a widget present in an Android application. In Android, an event refers to a response generated for an external action. An example of an event is a user interacting with the help of a Touchscreen. The Android framework maintains an Event Queue in which the events are arranged as they occur. The events are removed on First-In-First-Out (FIFO) basis.

When a user touches the screen, it is an input event. The event is passed to the View positioned on the screen along with information. The View requires an 'event listener' to handle the event. The Android View class comes equipped with a range of event listener interfaces. Each of these, in turn, contains an abstract declaration for a callback method.

The two actions that need to be performed by the developer to inform the user about the user input events are as follows:

- Defining an event listener and registering it with the view.
- Overriding an existing callback method for the view.

In order to handle a particular event, it is important that the Event Listener implements the corresponding callback method. For example, if a button should respond to a click event when a user touches it on the screen, then it must register the View with a click listener and implement the corresponding onClick callback procedure.

View class consists of a collection of nested interfaces with callback methods. These interfaces are known as event listeners and are generally named as 'On <something> Listener'. The callback methods has the following naming pattern, On <something>(). Callback methods are the methods adopted by the system to respond to an event.

4.6.1 Event Listener

It is an interface which exists in the View class. Each of the event listeners consist of a single callback method which are invoked when the view to which the listener is registered generates an event due to user interaction.

Table 4.4 lists the event listener interfaces and their associated callback methods.

Event Listener Interface	Callback Methods	Description
View.OnClickListener	onClick ()	Invoked when the user touches or focuses on the item that uses the navigation key or trackball or presses the Enter key.
View.OnLongClickListener	onLongClick()	Invoked when the user touches and holds the items or navigates and holds the trackball for a second and the callback method returns a boolean value of true or false.
View.OnFocusChange Listener	onFocusChange()	Invoked while the user navigates to or away from the item that uses the trackball or the navigation keys.

Event Listener Interface	Callback Methods	Description
View.OnKeyListener	onKey()	Invoked when the user is focused on the item and releases/presses a key on the device and the callback method returns a Boolean value of true if the event is handled and should stop else it returns false.
View.OnTouchListener	onTouch()	Invoked when the user performs any action such as touching, pressing or movement on the screen and the callback method returns a boolean value of true or false.
View.OnCreateContext MenuListener	onCreateContext Menu()	Invoked when there is a construction of a context menu.

Table 4.4: Event Listener Interfaces and their Associated Callback Methods

4.6.2 Handling User Events

When a user is building a custom component from View, it is possible to define several callback methods as default event handlers. Some of the common callbacks method used for event handling include:

- **`onKeyDown(int, KeyEvent)`** - Invoked when a key has been pressed and was not handled by the Views within an Activity.
- **`onKeyUp(int, KeyEvent)`** - Invoked when a key was released and was not handled by the Views within an Activity.
- **`onTrackballEvent(MotionEvent)`** - Invoked when a trackball motion event occurs.
- **`onTouchEvent(MotionEvent)`** - Invoked when a touch screen motion event occurs.
- **`onFocusChanged(boolean, int, Rect)`** - Invoked when the View gains or loses focus.

4.7 Working with Listeners

To understand the concept of listener interface an UI is created. Whenever a user interacts with a View object, an event will be generated. This event will be handled and a corresponding message will be displayed. In the next list, we will create an application utilizing all the concepts that were explained so far.

The steps to create a Contacts Application project are as follows:

1. Create a New Project in Android Studio named Contacts Application with a Blank Activity.
2. Navigate to app → Java. Right-click and select New → Package.
3. Create a new package with the name 'helpers'.
4. Right-click the package and select New → Class.
5. Enter the class name as Contact as shown in figure 4.18.

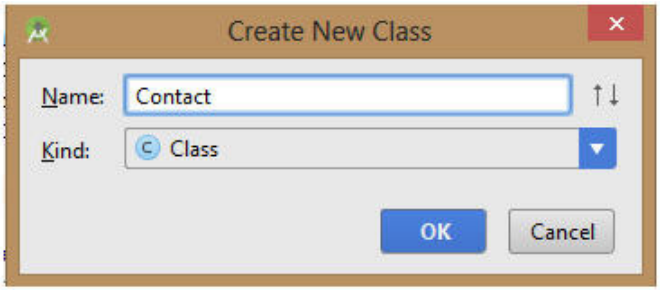


Figure 4.18: Creating New Class as Contact

6. Modify the code of Contact.java as shown in Code Snippet 22.

Code Snippet 22:

```
package aptech.com.helpers;
public class Contact {
    int id;
    private String name;
    private String nickName;
    private String number;
    private String address;
    private String notes;
public Contact(int id, String name,  String number, String
nickName, String address, String notes) {
        this.id = id;
        this.name = name;
        this.number = number;
        this.nickName = nickName;
        this.address = address;
        this.notes = notes;
    }
```

```
public int getId() {
    return id;
}
public void setId(int id) {
    this.id = id;
}
public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}
public String getNickName() {
    return nickName;
}
public void setNickName(String nickName) {
    this.nickName = nickName;
}
public String getNumber() {
    return number;
}
public void setNumber(String number) {
    this.number = number;
}
public String getAddress() {
    return address;
}
public void setAddress(String address) {
```

```
        this.address = address;
    }
    public String getNotes() {
        return notes;
    }
    public void setNotes(String notes) {
        this.notes = notes;
    }
    public String toString() {
        return getName();
    }
}
```

The Contact class acts as a container for the contact information.

7. Navigate to app → res. Right-click and select New → Image Asset.
8. Enter the details as shown in figure 4.19.

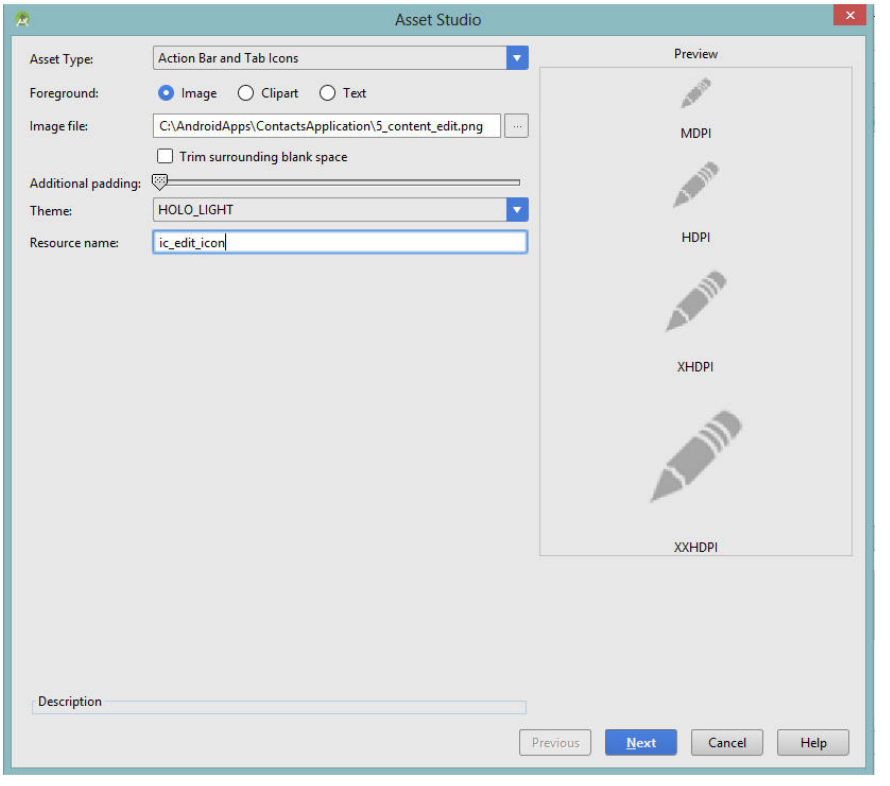


Figure 4.19: Asset Studio

9. Click Next and then, click Finish.
10. Repeat Step 7 and enter the details as shown in figure 4.20.

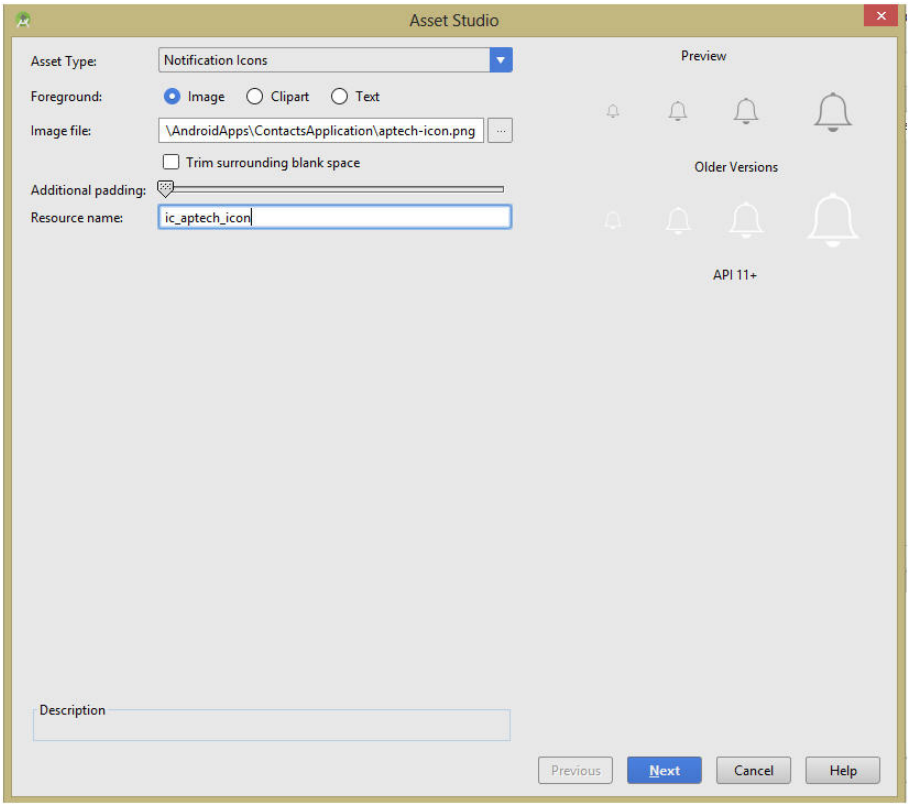


Figure 4.20: Asset Studio-Resource Name

11. Repeat Steps 9 and 10.
12. Copy the logo to the drawable folder as shown in figure 4.21.

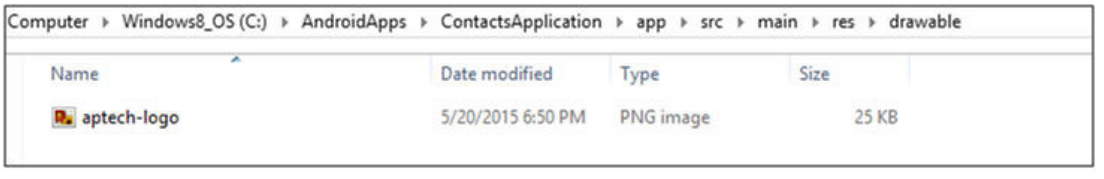


Figure 4.21: Logo Added in Resource Folder

13. Repeat Steps 9 and 10.
14. Navigate to res → values. Open strings.xml for editing.

15. Modify the code as shown in Code Snippet 23.

Code Snippet 23:

```
<resources>
    <string name="app_name">Contacts Application</string>
    <string name="hello_world">Hello world!</string>
    <string name="action_settings">Settings</string>
    <string name="new_contact">New Contact</string>
    <string name="contact_list">Contact List</string>
    <string name="title_activity_new_contact">New Contact</
     string>
    <string name="title_activity_edit_contact">View/Edit
    Contact</string>
    <string name="title_activity_contact_list">Contact List</
     string>
    <string name="action_edit">Edit Contact</string>
    <string name="home_page">Home Page</string>
</resources>
```

16. Create three new activities with the names ContactList, EditContact, and New Contact.
17. Navigate to /res/layout and open activity_main.xml for editing.
18. Modify the code as shown in Code Snippet 24.

Code Snippet 24:

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools" android:layout_
width="match_parent"
android:layout_height="match_parent"
android:paddingLeft="@dimen/activity_horizontal_margin"
android:paddingRight="@dimen/activity_horizontal_margin"
android:paddingTop="@dimen/activity_vertical_margin"
android:paddingBottom="@dimen/activity_vertical_margin"
tools:context=".MainActivity">
```

```
<ImageView
    android:layout_width=”match_parent”
    android:layout_height=”match_parent”
    android:id=”@+id/imageView”
    android:layout_alignParentTop=”true”
    android:layout_alignParentStart=”true”
    android:src=”@drawable/aptech_logo” />
    <TextView
        android:layout_width=”wrap_content”
        android:layout_height=”wrap_content”
        android:text=”Click on the Menu to Start”
        android:id=”@+id/textView6”
        android:layout_alignBottom=”@+id/imageView”
        android:layout_alignParentStart=”true”
        android:layout_alignEnd=”@+id/imageView”
        android:textSize=”30dp” />
</RelativeLayout>
```

The main activity consists of a RelativeLayout with a centered ImageView. The ImageView displays the resource aptech_logo. A TextView is set to display information about using the application.

19. Navigate to res → menu → menu_main.xml and open it for editing.
20. Modify the code as shown in Code Snippet 25.

Code Snippet 25:

```
<menu
xmlns:android=”http://schemas.android.com/apk/res/android”
    xmlns:tools=”http://schemas.android.com/tools”
tools:context=”.MainActivity”>
<item android:id=”@+id/action_new_contact”
        android:title=”@string/new_contact”
        android:orderInCategory=”100”
        android:showAsAction=”never”
```

```
        android:onClick="loadNewContactActivity"/>
    <item android:id="@+id/action_contact_list"
        android:title="@string/contact_list"
        android:orderInCategory="100"
        android:showAsAction="never"
        android:onClick="loadContactListActivity" />
    <item android:id="@+id/action_settings"
        android:title="@string/action_settings"
        android:orderInCategory="100"
        android:showAsAction="never" />
</menu>
```

Two additional Menu Items are added to load the New Contact and Contact List Activities. The respective onClick() listeners are set in XML using the android:onClick XML properties.

21. Navigate to app → java → aptech.com.contactsapplication and open MainActivity.java for editing.
22. Modify the code as shown in Code Snippet 26.

Code Snippet 26:

```
package aptech.com.contactsapplication;
import android.app.Activity;
import android.content.Intent;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import java.util.ArrayList;
import java.util.List;
import aptech.com.helpers.Contact;
public class MainActivity extends Activity {
    public static List<Contact> contactsList;
    @Override
```

```
        protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        contactsList = new ArrayList<Contact>();
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if
it is present.
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
  // Handle action bar item clicks here. The action bar will
// automatically handle clicks on the Home/Up button, so
// long as you specify a parent activity
 // in androidManifest.xml.
        int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
    public void loadNewContactActivity(MenuItem item) {
        Intent intent = new Intent(MainActivity.this, NewContact.
                        class);
```

```
        startActivity(intent);
    }
    public void loadContactListActivity(MenuItem item) {
        Intent intent = new Intent(MainActivity.this,
ContactList.class);
        startActivity(intent);
    }
}
```

The MainActivity.java file contains the code to load the Actionbar Menu items and to handle the onClick events for the menu items.

A static field public static List<Contact> contactsList; is used to store the current contact list.

Once the user selects the New Contact Option from the Menu, the New Contact Activity is loaded.

23. Navigate to /res/layout and open activity_new_contact.xml for editing.
24. Modify the code as shown in Code Snippet 27.

Code Snippet 27:

```
<TableLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
    tools:context="aptech.com.contactsapplication.NewContact">
    <TableRow
        android:layout_width="match_parent"
        android:layout_height="match_parent">
        <TextView
```

```
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Name”
            android:id=”@+id/textView”
            android:layout_column=”0” />
        <EditText
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:inputType=”textPersonName”
            android:text=”Name”
            android:ems=”10”
            android:id=”@+id/editText”
            android:layout_column=”1” />
    </TableRow>
    <TableRow
        android:layout_width=”match_parent”
        android:layout_height=”match_parent”>
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Phone Number”
            android:id=”@+id/textView2”
            android:layout_column=”0” />
    <EditText
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:inputType=”phone”
            android:ems=”10”
            android:id=”@+id/editText2”
```

```
            android:layout_column="1"
            android:text="+1 xxx xxx xxx" />
    </TableRow>
    <TableRow
        android:layout_width="match_parent"
        android:layout_height="match_parent">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Nick Name"
            android:id="@+id/textView3"
            android:layout_column="0" />
        <EditText
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:inputType="textPersonName"
            android:text="Nick Name"
            android:ems="10"
            android:id="@+id/editText3"
            android:layout_column="1" />
    </TableRow>
    <TableRow
        android:layout_width="match_parent"
        android:layout_height="match_parent">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Address"
```

```
            android:id="@+id/textView4"
            android:layout_column="0" />
        <EditText
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:inputType="textPostalAddress"
            android:ems="10"
            android:id="@+id/editText4"
            android:layout_column="1"
            android:text="H No, Street Address, City" />
    </TableRow>
    <TableRow
            android:layout_width="match_parent"
            android:layout_height="match_parent">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Notes"
            android:id="@+id/textView5"
            android:layout_column="0" />
        <EditText
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:inputType="textMultiLine"
            android:ems="10"
            android:id="@+id/editText5"
            android:layout_column="1" />
    </TableRow>
<TableRow
```

```
            android:layout_width="match_parent"
            android:layout_height="match_parent">
        <Button
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Add Contact"
            android:id="@+id/button"
            android:layout_column="1" />
    </TableRow>
</TableLayout>
```

The main activity consists of a TableLayout Text entry fields for the contact details.

25. Navigate to res → menu → menu_new_contact.xml and open it for editing.
26. Modify the code as shown in Code Snippet 28.

Code Snippet 28:

```
<menu
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    tools:context="aptech.com.contactsapplication.NewContact">
    <item android:id="@+id/action_home_page" android:title="@
string/home_page"
        android:orderInCategory="100"
        android:showAsAction="never"
        android:onClick="loadHomePage"/>
    <item android:id="@+id/action_contact_list"
        android:title="@string/contact_list"
        android:orderInCategory="100"
        android:showAsAction="never"
        android:onClick="loadContactListActivity" />
    <item android:id="@+id/action_settings"
        android:title="@string/action_settings"
```

```
        android:orderInCategory="100"
android:showAsAction="never" />

</menu>
```

Two additional Menu Items are added to load the Home Page and Contact List Activities. The respective onClick() listeners are set in XML using the android:onClick XML properties.

27. Navigate to app → java → aptech.com.contactsapplication and open NewContact.java for editing.

28. Modify the code as shown in Code Snippet 29.

Code Snippet 29:

```
package aptech.com.contactsapplication;

import android.app.Activity;

import android.app.Notification;

import android.app.NotificationManager;

import android.app.PendingIntent;

import android.app.TaskStackBuilder;

import android.content.Context;

import android.content.Intent;

import android.os.Bundle;

import android.view.Menu;

import android.view.MenuItem;

import android.view.View;

import android.widget.Button;

import android.widget.EditText;

import android.app.Notification.Builder;

import aptech.com.helpers.Contact;

public class NewContact extends Activity {

    private View.OnClickListener addButtonListener = new View.
OnClickListener() {

        public void onClick(View v) {
```

```
  int id = MainActivity.contactsList.size();
  String name = ((EditText)findViewById(R.id.editText)).
getText().toString();
  String number = ((EditText)findViewById(R.id.editText2)).
getText().toString();
  String nickName = ((EditText)findViewById(R.id.editText3)).
getText().toString();
  String address = ((EditText)findViewById(R.id.editText4)).
getText().toString();
  String notes = ((EditText)findViewById(R.id.editText5)).
getText().toString();
  Contact contact = new
Contact(id,name,number,nickName,address,notes);
  MainActivity.contactsList.add(contact);
  showNotification(contact.getName());
 Intent intent = new Intent(NewContact.this, ContactList.
class);
 startActivity(intent);
        }
    };
  private void showNotification(String name) {
  Notification.Builder nBuilder =
                new Notification.Builder(this)
.setSmallIcon(R.drawable.ic_aptech_icon)
.setContentTitle("New Contact Added")
.setContentText("Contact created "+name);
Intent resultIntent = new Intent(this, ContactList.class);
TaskStackBuilder stackBuilder = TaskStackBuilder.create(this);
        stackBuilder.addParentStack(ContactList.class);
        stackBuilder.addNextIntent(resultIntent);
        PendingIntent resultPendingIntent =
```

```
                stackBuilder.getPendingIntent(
                0, PendingIntent.FLAG_UPDATE_CURRENT);
        nBuilder.setContentIntent(resultPendingIntent);
        NotificationManager mNotificationManager =
                                    (NotificationManager)
        getSystemService(Context.NOTIFICATION_SERVICE);
        mNotificationManager.notify(0, nBuilder.build());
    }
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_new_contact);
        Button button = (Button)findViewById(R.id.button);
        button.setOnClickListener(addButtonListener);
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
  // Inflate the menu; this adds items to the action bar if
 //it is present.
  getMenuInflater().inflate(R.menu.menu_new_contact, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
// Handle action bar item clicks here. The action bar will
// automatically handle clicks on the Home/Up button, so
// long as you specify a parent activity in
//androidManifest.xml.
         int id = item.getItemId();
        //noinspection SimplifiableIfStatement
```

```
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
public void loadHomePage(MenuItem item) {
   Intent intent = new Intent(NewContact.this, MainActivity.
class);
        startActivity(intent);
    }
    public void loadContactListActivity(MenuItem item) {
        Intent intent = new Intent(NewContact.this, ContactList.
class);
        startActivity(intent);
    }
}
```

The NewContact.java file contains the code to load the Actionbar Menu items and to handle the onClick events for the menu items.

Once the user clicks the Add Contact button, the text entered in the respective fields is retrieved and a new contact object is created and added to the list. A notification is displayed to the user in the Notification bar signifying the addition of the contact.

The ListContacts Activty is loaded to show the added contact.

29. Navigate to /res/layout and open activity_contact_list.xml for editing.
30. Modify the code as shown in Code Snippet 30.

Code Snippet 30:

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
tools:context=".ListActivity">
  <ListView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/listView"
        android:layout_alignParentTop="true"
        android:layout_alignParentStart="true"
        android:choiceMode="singleChoice" />
</RelativeLayout>
```

The layout is similar to the example described earlier for ListView.

31. Navigate to res → menu → menu_ contact_list.xml and open it for editing.
32. Modify the code as shown in Code Snippet 31.

Code Snippet 31:

```
<menu
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    tools:context="aptech.com.contactsapplication.ContactList">
    <item android:id="@+id/action_home_page"
        android:title="@string/home_page"
        android:orderInCategory="100"
        android:showAsAction="never"
        android:onClick="loadHomePage"/>
    <item android:id="@+id/action_new_contact" android:title="@
string/new_contact"
```

```
        android:orderInCategory="100"
        android:showAsAction="never"
        android:onClick="loadNewContactActivity"/>
    <item android:id="@+id/action_settings"
        android:title="@string/action_settings"
        android:orderInCategory="100"
        android:showAsAction="never" />
</menu>
```

Two additional Menu Items are added to load the Home Page and New Contact Activities. The respective onClick() listeners are set in XML using the android:onClick XML properties.

33. Navigate to app → java → aptech.com.contactsapplication and open Contact List.java for editing.
34. Modify the code as shown in Code Snippet 32.

Code Snippet 32:

```
package aptech.com.contactsapplication;
import android.app.Activity;
import android.content.Intent;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.AdapterView;
import android.widget.ArrayAdapter;
import android.widget.EditText;
import android.widget.ListView;
import android.widget.AdapterView.OnItemClickListener;
import aptech.com.helpers.Contact;
public class ContactList extends Activity {
    private ListView listView;
```

```
    private OnItemClickListener listViewListener = new
OnItemClickListener() {
        public void onItemClick(AdapterView<?> parent, View view,
int position, long id) {
            Contact  contact = (Contact)
            listView.getItemAtPosition(position);
            Intent intent = new Intent(ContactList.this,
                            EditContact.class);
           intent.putExtra("id",contact.getId() );
            startActivity(intent);
        }
    };
@Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_contact_list);
        listView=(ListView)findViewById(R.id.listView);
        ArrayAdapter<Contact> adapter = new
ArrayAdapter<Contact>(this,android.R.layout.simple_list_item_1,
android.R.id.text1, MainActivity.contactsList);
        listView.setAdapter(adapter);
        listView.setOnItemClickListener(listViewListener);
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if
it is present.
        getMenuInflater().inflate(R.menu.menu_contact_list, menu);
        return true;
    }
    @Override
```

```
    public boolean onOptionsItemSelected(MenuItem item) {
  // Handle action bar item clicks here. The action bar will
// automatically handle clicks on the Home/Up button, so
//long as you specify a parent activity in //androidManifest.
xml.
         int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
       return super.onOptionsItemSelected(item);
    }
public void loadHomePage(MenuItem item) {
        Intent intent = new Intent(ContactList.this,
MainActivity.class);
        startActivity(intent);
    }
    public void loadNewContactActivity(MenuItem item) {
        Intent intent = new Intent(ContactList.this,
                        NewContact.class);
        startActivity(intent);
    }
}
```

The Added contacts are displayed to the user using a ListView. Once one of the contacts has been clicked by the user, it is opened for View/Edit by loading the EditContact Activity. The id of the contact is passed on to the EditContact activity by using Intent Extras.

The onClick Action Listener is set by creating the OnItemClickListener object and overriding the onItemClick() method.

35. Navigate to /res/layout and open activity_edit_contact.xml for editing.
36. Modify the code as shown in Code Snippet 33.

Code Snippet 33:

```
<TableLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
    tools:context="aptech.com.contactsapplication.NewContact">
     <TableRow
         android:layout_width="match_parent"
        android:layout_height="match_parent">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Name"
            android:id="@+id/textView"
            android:layout_column="0" />
        <EditText
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:inputType="textPersonName"
            android:text="Name"
            android:ems="10"
            android:id="@+id/editText"
            android:layout_column="1" />
```

```
    </TableRow>
    <TableRow
        android:layout_width=”match_parent”
        android:layout_height=”match_parent”>
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Phone Number”
            android:id=”@+id/textView2”
            android:layout_column=”0” />
        <EditText
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:inputType=”phone”
            android:ems=”10”
            android:id=”@+id/editText2”
            android:layout_column=”1”
            android:text=”+1 xxx xxx xxx” />
    </TableRow>
    <TableRow
        android:layout_width=”match_parent”
        android:layout_height=”match_parent”>
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Nick Name”
            android:id=”@+id/textView3”
            android:layout_column=”0” />
```

```
        <EditText
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:inputType=”textPersonName”
            android:text=”Nick Name”
            android:ems=”10”
            android:id=”@+id/editText3”
            android:layout_column=”1” />
    </TableRow>
    <TableRow
        android:layout_width=”match_parent”
        android:layout_height=”match_parent”>
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Address”
            android:id=”@+id/textView4”
            android:layout_column=”0” />
        <EditText
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:inputType=”textPostalAddress”
            android:ems=”10”
            android:id=”@+id/editText4”
            android:layout_column=”1”
            android:text=”H No, Street Address, City” />
    </TableRow>
    <TableRow
```

```
        android:layout_width="match_parent"
        android:layout_height="match_parent">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Notes"
            android:id="@+id/textView5"
            android:layout_column="0" />
        <EditText
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:inputType="textMultiLine"
            android:ems="10"
            android:id="@+id/editText5"
            android:layout_column="1" />
    </TableRow>
    <TableRow
        android:layout_width="match_parent"
        android:layout_height="match_parent">
        <Button
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Save Contact"
            android:id="@+id/button"
            android:onClick="saveContact"
            android:layout_column="1" />
    </TableRow>
</TableLayout>
```

The layout is similar to the layout used in the NewContact Activity.

37. Navigate to res → menu → menu_ edit_contact.xml and open it for editing.

38. Modify the code as shown in Code Snippet 34.

Code Snippet 34:

```
<menu
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
tools:context="aptech.com.contactsapplication.EditContact">
    <item android:id="@+id/action_edit"
        android:icon="@drawable/ic_edit_icon"
        android:title="@string/action_edit"
        android:onClick="makeContactEditable"
        android:showAsAction="ifRoom" />
    <item android:id="@+id/action_new_contact"
        android:title="@string/new_contact"
        android:orderInCategory="100"
        android:showAsAction="never"
        android:onClick="loadNewContactActivity"/>
    <item android:id="@+id/action_contact_list"
        android:title="@string/contact_list"
        android:orderInCategory="100"
        android:showAsAction="never"
        android:onClick="loadContactListActivity" />
    <item android:id="@+id/action_settings"
        android:title="@string/action_settings"
        android:orderInCategory="100"
        android:showAsAction="never" />
</menu>
```

Two additional Menu Items are added to load the Contact List and New Contact Activities. The respective onClick() listeners are set in XML using the android:onClick XML properties.

39. Navigate to app → java → aptech.com.contactsapplication and open EditContact.java for editing.

40. Modify the code as shown in Code Snippet 35.

Code Snippet 35:

```
package aptech.com.contactsapplication;
import android.app.Activity;
import android.content.Intent;
import android.graphics.Color;
import android.os.Bundle;
import android.text.InputType;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;
import aptech.com.helpers.Contact;
public class EditContact extends Activity {
    int id;
    Contact contact;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_edit_contact);
        id = getIntent().getIntExtra("id", 0);
        contact = MainActivity.contactsList.get(id);
        initContactDetails();
    }
    private void initContactDetails() {
        EditText name = (EditText)findViewById(R.id.editText);
```

```
        EditText number = (EditText)findViewById(R.id.editText2);
        EditText nickName = (EditText)findViewById(R.
                            id.editText3);
        EditText address = (EditText)findViewById(R.id.editText4);
        EditText notes = (EditText)findViewById(R.id.editText5);
        contact = MainActivity.contactsList.get(id);
        name.setText(contact.getName());
        number.setText(contact.getNumber());
        nickName.setText(contact.getNickName());
        address.setText(contact.getAddress());
        notes.setText(contact.getNotes());
        disableEditText(name);
        disableEditText(number);
        disableEditText(nickName);
        disableEditText(address);
        disableEditText(notes);
        Button saveButton = (Button)findViewById(R.id.button);
        saveButton.setVisibility(View.GONE);
    }
    private void disableEditText(EditText editText) {
        editText.setFocusable(false);
    }
    private void enableEditText(EditText editText) {
        editText.setFocusable(true);
    }
   @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if
it is present.
```

```
        getMenuInflater().inflate(R.menu.menu_edit_contact, menu);
         return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
 // Handle action bar item clicks here. The action bar will
// automatically handle clicks on the Home/Up button, so
//long as you specify a parent activity in
//androidManifest.xml.
         int id = item.getItemId();
         //noinspection SimplifiableIfStatement
         if (id == R.id.action_settings) {
             return true;
         }
         return super.onOptionsItemSelected(item);
    }
  public void makeContactEditable(MenuItem item) {
    EditText name = (EditText)findViewById(R.id.editText);
   EditText number = (EditText)findViewById(R.id.editText2);
 EditText nickName = (EditText)findViewById(R.id.editText3);
 EditText address = (EditText)findViewById(R.id.editText4);
 EditText notes = (EditText)findViewById(R.id.editText5);
 enableEditText(name);
         enableEditText(number);
         enableEditText(nickName);
         enableEditText(address);
         enableEditText(notes);
    Button saveButton = (Button)findViewById(R.id.button);
```

```
    saveButton.setVisibility(View.VISIBLE);
    }
    public void saveContact(View view) {
 String name = ((EditText)findViewById(R.id.editText)).getText().
toString();
 String number = ((EditText)findViewById(R.id.editText2)).
getText().toString();
 String nickName = ((EditText)findViewById(R.id.editText3)).
getText().toString();
 String address = ((EditText)findViewById(R.id.editText4)).
getText().toString();
 String notes = ((EditText)findViewById(R.id.editText5)).
getText().toString();
    Contact contact = new
Contact(id,name,number,nickName,address,notes);
    MainActivity.contactsList.set(id, contact);
    initContactDetails();
 Toast.makeText(getApplicationContext(), "Contact Details Saved",
     Toast.LENGTH_LONG).show();
    }
    public void loadNewContactActivity(MenuItem item) {
        Intent intent = new Intent(EditContact.this, NewContact.
class);
        startActivity(intent);
    }
    public void loadContactListActivity(MenuItem item) {
        Intent intent = new Intent(EditContact.this, ContactList.
class);
        startActivity(intent);
    }
}
```

The same activity is used to both display the contact details and to allow editing. This is done by disabling the edit ability of the EditText View. The method disableEditText() is used for this purpose. The Save Contact button is also set to be invisible in display mode. The setVisibility() is used for this purpose. Initially, all the fields are set to un-editable i.e. display mode using the initContactDetails() method. This method is also responsible for loading the Contact data into the respective fields.

In order to identify the Contact being edited, the id value is retrieved from the Intent Extras using the getIntent().getIntExtra() method. This value will be used throughout the code to retrieve the Contact details form the contactList.

An ActionBar Button has been added to allow editing the Contact. Once the action bar has been clicked, the makeContactEditable() method gets invoked. This method sets the EditText views back to editable. The method enableEditText() is used for this purpose. The Save Contact Button is set to visible again. The user can now edit the contact details.

Finally, once the Save Contact button is clicked, the saveContact() method is invoked. The method creates a new Contact object with the updated details and replaces the old object with the new one. The page is then refreshed to reflect the new changes by calling the initContactDetails() method again.

A Toast is displayed to the user using the static method Toast.makeText(). The message confirms that the contact information has been saved.

The final structure of the project is as shown in figure 4.22.

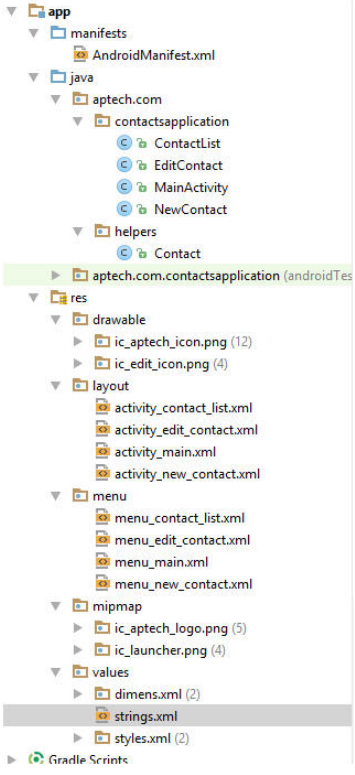


Figure 4.22: Final Structure of the Project

The output will be as shown in figure 4.23.

Figure 4.23: After Executing the Application

Open the Menu and select New Contact as shown in figure 4.24.

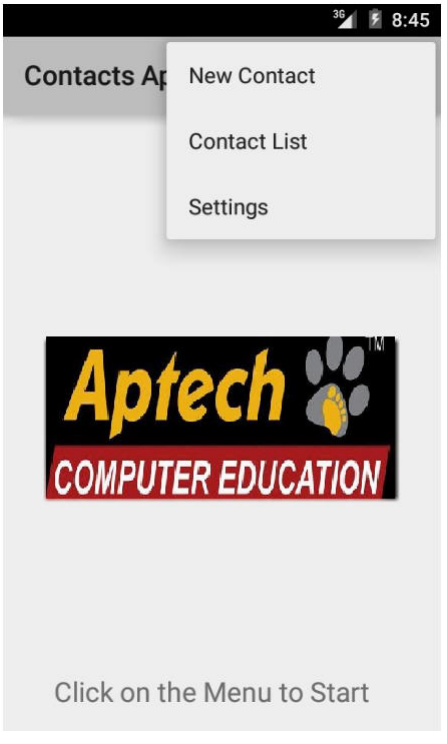


Figure 4.24: Selecting New Contact

The New Contact Screen is displayed. Enter the details as shown in figure 4.25.

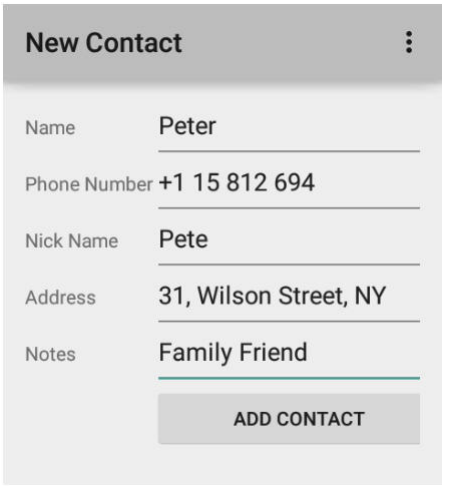


Figure 4.25: New Contact Screen

Repeat these steps to add multiple contacts as shown in figure 4.26.

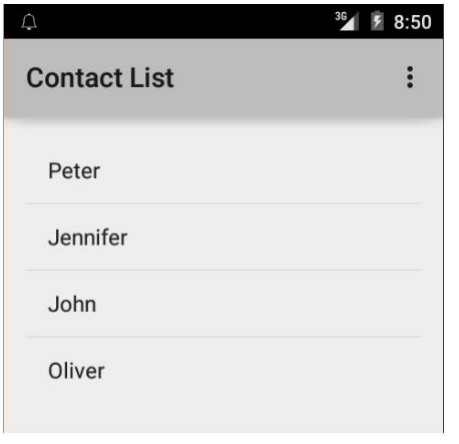


Figure 4.26: Adding Multiple Contacts

A notification is displayed when a contact is added. This can be viewed within the notification drawer as shown in figure 4.27.

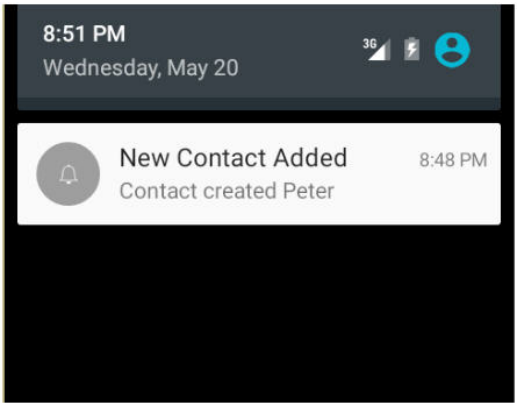


Figure 4.27: Notification Drawer

The user can return to the application by clicking the Notification. The application opens again with the Contact List screen.

Click any of the Contacts to view contact details as shown in figure 4.28.

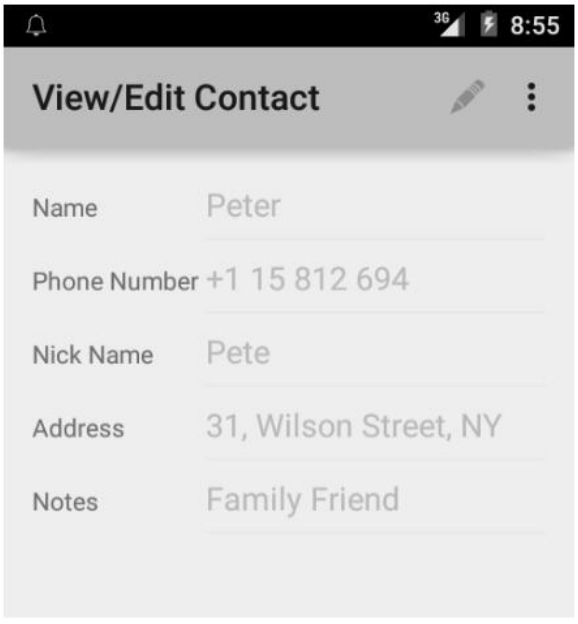


Figure 4.28: View or Edit Contact Screen

Click the Edit Action button to enable editing the contact. Edit the details as shown in figure 4.29.

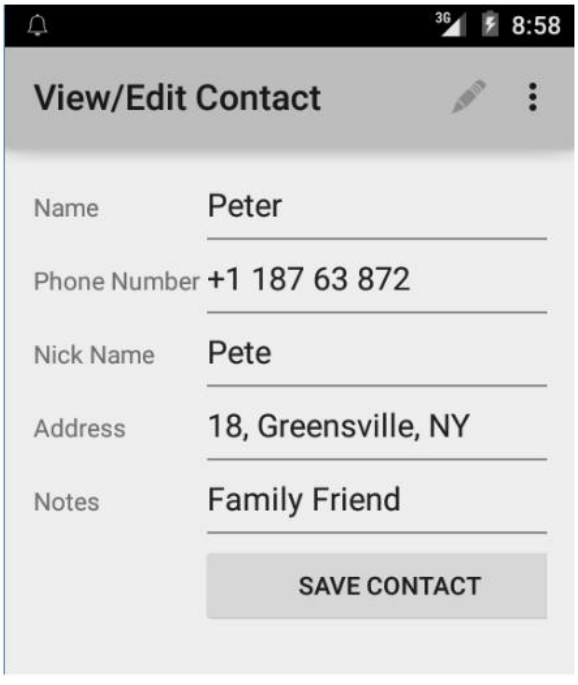


Figure 4.29: Editing Details

Click Save Contact. The contact details are saved. A toast is displayed to notify this information as shown in figure 4.30.

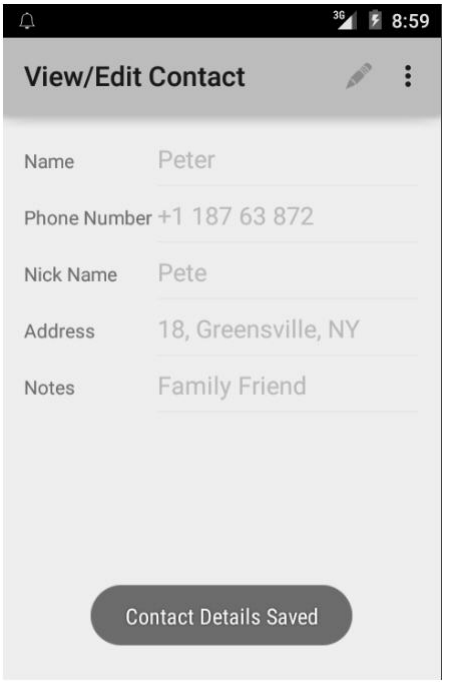


Figure 4.30: Saving Contact Details

The user may return to the New Contact screen or the Contact List using the menu as shown in figure 4.31.

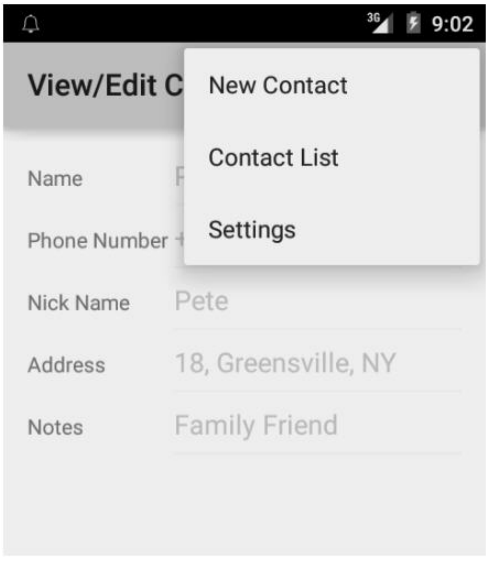


Figure 4.31: View/Edit Contact-Menu Screen

The application functions as an Address Book to store contact information. The contact data is stored in memory. As a result, the data is lost if the application is manually killed and restarted or if the device is rebooted. In order to avoid this, Databases need to be used to store the contact information.

Check Your Progress

1. Each layout file in Android can contain a maximum of how many root elements?

(A)	1	(B)	2
(C)	3	(D)	4

2. Which of the following supports setting a layout?

1.	ListView	2.	Notification
3.	Activities	4.	Views

(A)	1, 4	(B)	1, 2, 4
(C)	2, 4	(D)	1, 2, 3

3. Which of the following statement(s) are true?

1.	EditText can be used to receive text input.
2.	TextView's text can only be set programmatically.
3.	ProgressBar is an input widget.
4.	Switches can have up to three values.

(A)	1, 3, 4	(B)	4
(C)	1	(D)	2, 3

4. Which UI component displays popup window and allows user to take decision or enter information?

(A)	Toast	(B)	Dialog
(C)	Notification	(D)	Menu

5. Match the following purposes with their corresponding View/Layout.

Purpose		View/Layout	
a.	Used to display images	1.	ListView
b.	Used to display a checklist	2.	ImageView
c.	Used to display rows and columns	3.	RelativeLayout
d.	The default layout produced when creating a layout	4.	TableLayout

(A)	a-4, b-1, c-3, d-2	(B)	a-3, b-1, c-2, d-4
(C)	a-2, b-1, c-4, d-3	(D)	a-1, b-4, c-3, d-2

Answers

1.	A
2.	D
3.	C
4.	B
5.	C

Summary

- All UI components in Android are built using View and ViewGroup objects. View draws something on the screen that the user can interact with.
- Layout defines the visual structure of UI as in the case of an activity or an app widget. There are different types of layouts such as Linear layout, Relative layout, and Grid layout to name a few.
- In Android, UI components are interactive controls in an app's UI that enables the user to perform a wide range of actions. Some common GUI components are buttons, picker, and so on.
- A style is a collection of properties that specify the look and format for a View or window.
- A theme is a style applied to an entire Activity or application, rather than an individual View.
- In order to handle a particular event, it is necessary that the Event Listener implements the corresponding callback or Event Handler in response.

Onlinevarsity

GO DIGITAL

05

More UI Elements

Welcome to the Session, **More UI Elements.**

This session discusses in detail the components of Android User Interface (UI). It describes the advanced (UI) components, Adapters, dialogs, widgets, and the Material Design philosophy.

In this Session, you will learn to:

- Explain the use of adapters
- Identify the different types of adapters
- Describe the advanced and complex UI Components
- Explain the use of custom dialogs
- Explain and create Custom Widgets
- Explain and use Material Design Philosophy

Session

05

More UI Elements

5.1 Adapters

An Adapter is an object that acts as a bridge between the UI Components such as List view, Grid view, and so on and the underlying data source. The underlying data source can be an Array, database, and so on that fills data to the UI component. Adapter is responsible for providing view for every element of the data source.

Adapters can be of different types. They differ based on their data source and manipulation complexity that they present to the user. The different types of adapters present in Android are as follows:

- BaseAdapter
- SimpleAdapter
- ArrayAdapter
- SimpleCursorAdapter
- CursorAdapter
- ResourceCursorAdapter
- SpinnerAdapter
- SimpleCursorTreeAdapter
- CursorTreeAdapter
- HeaderViewListAdapter
- WrapperListAdapter

Following is a brief description of these adapters:

- **BaseAdapter**

  BaseAdapter is a common implementation for the adapter which can be used for both ListView and Spinner. It is an abstract base class for the adapter interface. The developer can implement his/her own adapters using BaseAdapter.

- **SimpleAdapter**

  A SimpleAdapter helps to map static data to the views defined in the XML. The data can be mapped to the view by using ArrayList of Maps and each element in the array will be represented as a separate row in the view.

- **ArrayAdapter**

  ArrayAdapter is backed by an array of objects to load the data to the UI view. In other words, it is used to bind an array of data to the view. It overrides the getView() method to inflate, populate, and return a custom view for the provided array data.

- **SimpleCursorAdapter**

  An adapter which maps columns from a cursor to TextViews or ImageViews defined in an XML file. The developer can specify which columns to be displayed, in which views the developer wants to display the columns, and the XML file that defines the appearance of these views.

- **CursorAdapter**

  This Adapter that is used for exposing the data from a Cursor to a ListView object using the column named _id.

- **ResourceCursorAdapter**

  ResourceCursorAdapter is very similar to the CursorAdapter, which doesn't have a new View method and is used for simple views. It is used to create views defined in an XML file. This Adapter is deprecated in API Level 11.

- **SpinnerAdapter**

  SpinnerAdapter acts as a bridge between the spinner component and the data source.

  SpinnerAdapter allows displaying of data in the following two ways:

  1. Displays data in the spinner view.
  2. Displays data in the drop-down list when the spinner is pressed.

- **SimpleCursorTreeAdapter**

  This Adapter can be used to map column data from the Cursor to the TextView or ImageView as defined in the XML file. The developer can specify the required columns and separate child views to display the specified content. Binding is done using the setViewValue() method of the SimpleCursorTreeAdapter, ViewBinder. The method returns a boolean value which can be used to determine whether binding has been successful.

- **CursorTreeAdapter**

  This Adapter can be used to display data in an expandable list view by using the data from the cursor. In this, the top-level Cursor exposes the group and the getChildrenCursor(Cursor) method returns cursors which exposes the child elements within the particular group.

  Note - The cursor must include a column named 'id', otherwise this class will not work.

- **HeaderViewListAdapter**

  HeaderViewListAdapter can be used to implement a ListView having a header at the top.

- **WrapperListAdapter**

  WrapperListAdapter can be used to wrap another ListAdapter. The method, getWrappedAdapter() is invoked for retrieving the wrapped adapter.

5.1.1 Working with BaseAdapter

This section explains the development of an application that displays a list view. The user can make multiple selections by using an object of the BaseAdapter class.

To develop the MultiSelectionListView application, perform the following steps:

1. Create a new project named MultiSelectionListView in Android Studio.
2. Navigate to res → layout folder.
3. Right-click and select New → Layout Resource File to create an XML file named linearlayout.
4. Modify the code in linearlayout.xml file as shown in Code Snippet 1.

Code Snippet 1:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/
res/android"
 android:layout_width="match_parent"
 android:layout_height="match_parent"
 android:orientation="horizontal"
 android:padding="10dp" >
 <CheckBox
  android:id="@+id/chkEnable"
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:layout_gravity="center_vertical"
  android:text="" />
<TextView
  android:id="@+id/tvTitle"
  android:layout_width="fill_parent"
  android:layout_height="wrap_content"
  android:layout_gravity="center_vertical"
  android:layout_marginLeft="10dp"
  android:textSize="16sp" />
</LinearLayout>
```

5. Navigate to res → layout folder.
6. Modify the code in activity_main.xml file as shown in Code Snippet 2.

Code Snippet 2:

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
 xmlns:tools="http://schemas.android.com/tools"
 android:layout_width="match_parent"
 android:layout_height="match_parent"
 android:paddingBottom="@dimen/activity_vertical_margin"
 android:paddingLeft="@dimen/activity_horizontal_margin"
 android:paddingRight="@dimen/activity_horizontal_margin"
 android:paddingTop="@dimen/activity_vertical_margin"
 tools:context=".MainActivity" >
<ListView
  android:id="@android:id/list"
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:layout_above="@+id/btnShowCheckedItems"
  android:layout_centerHorizontal="true"
  android:layout_centerVertical="true" />
 <Button
  android:id="@+id/btnShowCheckedItems"
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:layout_alignParentBottom="true"
  android:layout_centerHorizontal="true"
  android:text="View Selected" />
 </RelativeLayout>
```

7. Navigate to src → com.example.multiselectionlistview folder.
8. Right-click and select New → Java Class.
9. Create a new class named Product that will return the product details.
10. Modify the code in the Product class as shown in Code Snippet 3.

Code Snippet 3:

```
package com.example.multiselectionlistview;
public class Product {
String name;
 public Product(String name) {
  // TODO Auto-generated constructor stub
  this.name = name;
 }
 public String getName() {
  return name;
  }

 public void setName(String name) {
  this.name = name;
 }
@Override
 public String toString() {
  // TODO Auto-generated method stub
  return this.name;
  }
}
```

11. Navigate to src → com.example.multiselectionlistview folder.
12. Right-click and select New → Java Class.

13. Create a new adapter class named MultiSelectionAdapter which helps the user to select multiple products from a product list displayed as a check box item.
14. Modify the code in MultiSelectionAdapter as shown in Code Snippet 4.

Code Snippet 4:

```
package com.example.multiselectionlistview;
import java.util.ArrayList;
import android.content.Context;
import android.util.SparseBooleanArray;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.BaseAdapter;
import android.widget.CheckBox;
import android.widget.CompoundButton;
import android.widget.CompoundButton.OnCheckedChangeListener;
import android.widget.TextView;
public class MultiSelectionAdapter<T> extends BaseAdapter {
 Context mContext;
 LayoutInflater mInflater;
ArrayList<T> mList;
 SparseBooleanArray mSparseBooleanArray;
}
public MultiSelectionAdapter(Context context,ArrayList<T> list) {
//TODO Auto-generated constructor stub
  this.mContext = context;
  mInflater = LayoutInflater.from(mContext);
```

```
  mSparseBooleanArray = new SparseBooleanArray();
  mList = new ArrayList<T>();
  this.mList = list;
 }
 public ArrayList<T> getCheckedItems() {
  ArrayList<T> mTempArry = new ArrayList<T>();
  for (int i = 0; i < mList.size(); i++) {
   if (mSparseBooleanArray.get(i)) {
    mTempArry.add(mList.get(i));
  }
 }
 return mTempArry;
 }
 @Override
 public int getCount() {
  // TODO Auto-generated method stub
  return mList.size();
 }
 @Override
public Object getItem(int position) {
  // TODO Auto-generated method stub
return mList.get(position);
 }
 @Override
 public long getItemId(int position) {
  // TODO Auto-generated method stub
  return position;
 }
@Override
```

```
public View getView(int position, View convertView,
ViewGroup parent) {
  // TODO Auto-generated method stub
  if (convertView == null) {
    convertView = mInflater.inflate(R.layout.linearlayout, null);
  }
      TextView tvTitle = (TextView) convertView.findViewById(R.
                                                    id.tvTitle);
  tvTitle.setText(mList.get(position).toString());
   CheckBox mCheckBox = (CheckBox) convertView
    .findViewById(R.id.chkEnable);
  mCheckBox.setTag(position);
   mCheckBox.setChecked(mSparseBooleanArray.
get(position));
   mCheckBox.setOnCheckedChangeListener(new
OnCheckedChangeListener() {
  @Override
   public void onCheckedChanged(CompoundButton
buttonView,
     boolean isChecked) {
  // TODO Auto-generated method stub
 mSparseBooleanArray.put((Integer)buttonView.getTag(),isChecked);
  }
 });
 return convertView;
 }
}
```

15. Navigate to src → com.example.multiselectionlistview folder.
16. Modify the code in MainActivity file as shown in Code Snippet 5.

Code Snippet 5:

```
package com.example.multiselectionlistview;
import java.util.ArrayList;
import android.app.Activity;
import android.os.Bundle;
import android.util.Log;
import android.view.Menu;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.ListView;
import android.widget.Toast;
public class MainActivity extends Activity implements
OnClickListener {
ListView mListView;
Button btnShowCheckedItems;
ArrayList<Product> mProducts;
MultiSelectionAdapter<Product> mAdapter;
@Override
protected void onCreate(Bundle savedInstanceState) {
   super.onCreate(savedInstanceState);
   setContentView(R.layout.activity_main);
    bindComponents();
    init();
    addListeners();
}
private void bindComponents() {
   // TODO Auto-generated method stub
```

```
mListView = (ListView) findViewById(android.R.id.list);
     btnShowCheckedItems = (Button) findViewById(R.
                                        id.btnShowCheckedItems);
}
private void init() {
   // TODO Auto-generated method stub
   mProducts = new ArrayList<Product>();
   mProducts.add(new Product("Samsung"));
   mProducts.add(new Product("iPhone"));
  mProducts.add(new Product("HTC"));
  mProducts.add(new Product("LG"));
  mProducts.add(new Product("Micromax"));
  mProducts.add(new Product("Sony"));
  mProducts.add(new Product("Nexus"));
  mProducts.add(new Product("Acer"));
  mProducts.add(new Product("Videocon"));
  mProducts.add(new Product("Karbonn"));
  mProducts.add(new Product("Nokia"));
  mProducts.add(new Product("Motorola"));
  mAdapter = new MultiSelectionAdapter<Product>(this, mProducts);
  mListView.setAdapter(mAdapter);
}
private void addListeners() {
   // TODO Auto-generated method stub
   btnShowCheckedItems.setOnClickListener(this);
}
@Override
public void onClick(View v) {
   // TODO Auto-generated method stub
```

```
if (mAdapter != null) {
        ArrayList<Product> mArrayProducts = mAdapter.
                                              getCheckedItems();
      Log.d(MainActivity.class.getSimpleName(), "Selected Items:
                                    " + mArrayProducts.toString());
      Toast.makeText(getApplicationContext(),
     "Selected Items: " + mArrayProducts.toString(),
      Toast.LENGTH_LONG).show();
     }
}
@Override
public boolean onCreateOptionsMenu(Menu menu) {
   // Inflate the menu; this adds items to the action bar if it is
present.
  getMenuInflater().inflate(R.menu.main, menu);
  return true;
}
}
```

Figure 5.1 displays the output when the application is executed.

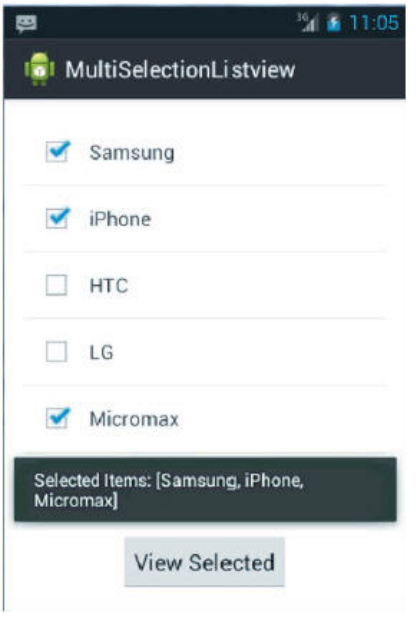


Figure 5.1: Using BaseAdapter Class

5.2 Advanced UI Components

UI components act as an interface between the user and the application to input data and to display the expected result. Basic UI components such as buttons, text-fields, dialogs, and so on have been already discussed in earlier sessions. In this session, some of the complex and advanced Android UI components that are discussed are as follows:

- Listview
- ScrollView
- TabBar
- WebView
- ViewFlipper
- VideoView

5.2.1 ListView

ListView is used for displaying a list of scrollable data by using an object of the Adapter class, which acts as a bridge between view and the data source. The Adapter is used for inserting data into the list that is retrieved from an array or a database using a query.

In this example, the MainActivity will be created by extending the ListActivity class. The ListActivity class has a ListView view as a part of the activity layout. Developers can use this ListActivity directly if the activity is fully reliant on a list structure.

Following application named ListFruit will display image and text in each row using a ListView widget. To develop the application, perform the following steps:

1. Create a project named ListFruit in Android Studio.
2. Navigate to res → layout folder.
3. Modify the code in main_activity.xml file as shown in Code Snippet 6.

Code Snippet 6:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/ res/
android"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:padding="5dp" >
<ImageView
     android:id="@+id/logo"
```

```
      android:layout_width="50px"
      android:layout_height="50px"
      android:layout_marginLeft="5px"
      android:layout_marginRight="20px"
      android:layout_marginTop="5px"
      android:src="@drawable/apple_new" >
       </ImageView>
<TextView
      android:id="@+id/label"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="@+id/label"
      android:textSize="30px" >
</TextView>
</LinearLayout>
```

The layout file contains an ImageView and a TextView widget to display the image of the fruit and its name.

4. Navigate to src → com.example.listfruit folder.
5. Right-click and select New → Java Class.
6. Create a new class named FruitArrayAdapter. The class will extend from the Array-Adapter class to read data from the data source.
7. Modify the code in the FruitArrayAdapter class as shown in Code Snippet 7.

Code Snippet 7:

```
package com.example.listfruit;
import android.content.Context;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ArrayAdapter;
import android.widget.ImageView;
```

```
import android.widget.TextView;
public class FruitArrayAdapter extends ArrayAdapter<String> {
private final Context context;
private final String[] values;
public FruitArrayAdapter(Context context, String[] values) {
    super(context, R.layout.activity_main, values);
    this.context = context;
    this.values = values;
}
@Override
public View getView(int position, View convertView, ViewGroup
                                              parent) {
LayoutInflater inflater = (LayoutInflater) context
             .getSystemService(Context.LAYOUT_INFLATER_ SERVICE);
View rowView = inflater.inflate(R.layout.activity_main, parent,
                                                    false);
TextView textView = (TextView) rowView.findViewById (R.id.label);
ImageView imageView = (ImageView) rowView. findViewById(R.
                                                      id.logo);
textView.setText(values[position]);
// Change icon based on name
String s = values[position];
System.out.println(s);
if (s.equals("Apple")) {
   imageView.setImageResource(R.drawable.apple_ new);
} else if (s.equals("Banana")) {
   imageView.setImageResource(R.drawable.banana_ new);
} else if (s.equals("Strawberry")) {
   imageView.setImageResource(R.drawable. strawberry_new);
} else {
   imageView.setImageResource(R.drawable. watermelon_new);
```

```
}
return rowView;
}
}
```

8. Navigate to src → com.example.listfruit folder.
9. Modify the code in MainActivity file as shown in Code Snippet 8. The Adapter class used in this code will be used to load the data.

Code Snippet 8:

```
package com.example.listfruit;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.app.ListActivity;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import android.widget.Toast;
import android.view.View;
public class MainActivity extends ListActivity {
static final String[] FRUITS = new String[] { "Apple",
                          "Banana", "Strawberry", "Watermelon" };
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
          // setContentView(R.layout.activity_main);
    setListAdapter(new FruitArrayAdapter(this, FRUITS));
}
@Override
public boolean onCreateOptionsMenu(Menu menu) {
   // Inflate the menu; this adds items to the action bar if it is
present.
```

```
    getMenuInflater().inflate(R.menu.main, menu);

    return true;

}

@Override

protected void onListItemClick(ListView l, View v, int position,
                               long id) {

        // get selected items

             String selectedValue = (String) getListAdapter().
                                    getItem(position);

        Toast.makeText(this, selectedValue, Toast.LENGTH_ SHORT).
                                            show();

}

}
```

Figure 5.2 displays the output after execution of the application.

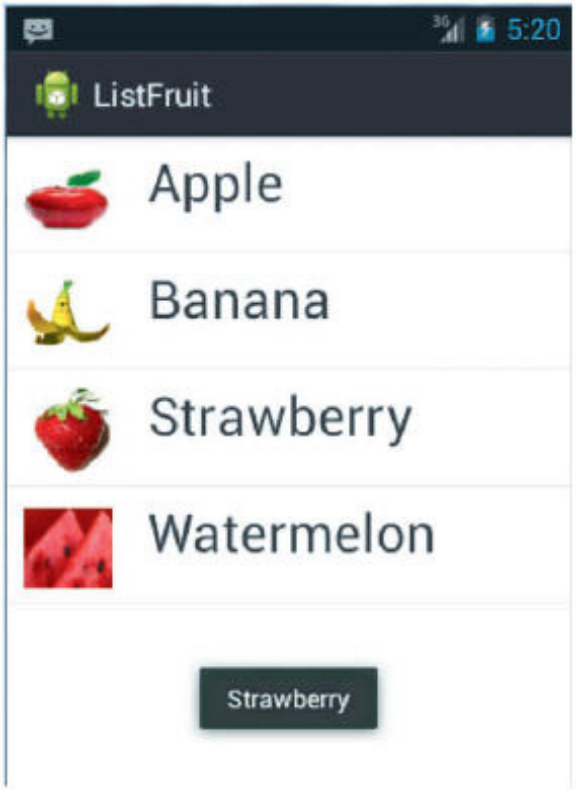


Figure 5.2: ListFruit - Output

5.2.2 ScrollView

ScrollView is a container that holds child views and has a property of scrolling when the child items size in the container exceeds the screen size. A ScrollView is a FrameLayout, meaning that the developer should place one child in it containing the entire contents to scroll. ScrollView should not be used with ListView, as ListView itself takes care of its own scroll.

Following application named ScrollViewExample demonstrates the use of ScrollView.

1. Create a project named ScrollViewExample in Android Studio.
2. Navigate to res → layout folder.
3. Modify the code in activity_main.xml file as shown in Code Snippet 9.

Code Snippet 9:

```
<RelativeLayout xmlns:android="http://schemas.android.com/ apk/
res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:paddingBottom="@dimen/activity_vertical_margin"
android:paddingLeft="@dimen/activity_horizontal_margin"
android:paddingRight="@dimen/activity_horizontal_ margin"
android:paddingTop="@dimen/activity_vertical_margin"
tools:context=".MainActivity" >
<ScrollView
android:layout_width="fill_parent"
android:layout_height="fill_parent" >
<LinearLayout
android:layout_width="fill_parent"
android:layout_height="fill_parent"
android:orientation="vertical" >
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="Red" />
```

```
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="Green" />
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="Yellow" />
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="Pink" />
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="Violet" />
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="Grey" />
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="Blue" />
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="White" />
```

```
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="Black" />
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="Brown" />
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="Red" />
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="Green" />
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="Yellow" />
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="Pink" />
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="Violet" />
```

```
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="Grey" />
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="Blue" />
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="White" />
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="Black" />
<TextView
android:layout_width="fill_parent"
android:layout_height="50dp"
android:text="Brown" />
</LinearLayout>
</ScrollView>
</RelativeLayout>
```

4. Navigate to src → com.example.scrollviewexample folder.
5. Modify the code in MainActivity file as shown in Code Snippet 10.

Code Snippet 10:

```
package com.example.scrollviewexample;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
public class MainActivity extends Activity {
@Override
protected void onCreate(Bundle savedInstanceState) {
     super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
}
}
```

Figure 5.3 displays the output after executing the application.

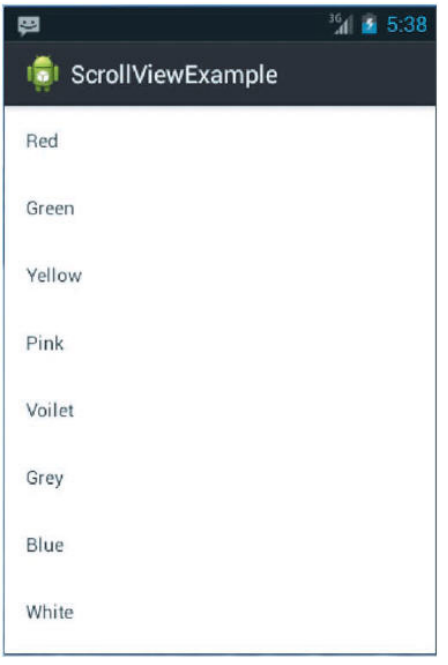


Figure 5.3: Scroll View Display

Note - The content scrolls when the user keeps the mouse button pressed and moves the cursor up and down.

5.2.3 TabBar

TabBar is used to display data in Tab format as is present in Android's contacts view. Tabs help to explore and switch between different views and functionalities that are present in the application. It also helps the developer to develop application for users where browsing for categorized data sets are required. It can be scrollable, fixed, or stacked tabs.

TabHost is a container for a tabbed window view. This object holds two children, a set of tab labels that the user clicks to select a specific tab and a FrameLayout object that displays the contents of that page. Following application named TabBarExample explains the use of TabView.

To develop the application, perform the following steps:

1. Create a project named TabBarExample in Android Studio.
2. Navigate to res → layout folder.
3. Modify the code in main_activity.xml file as shown in Code Snippet 11. The code shows the use of TabWidget.

Code Snippet 11:

```
<?xml version="1.0" encoding="utf-8"?>
<TabHost
xmlns:android="http://schemas.android.com/apk/res/ android"
android:id="@android:id/tabhost"
android:layout_width="match_parent"
android:layout_height="match_parent" >
<LinearLayout
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical" >
<TabWidget
     android:id="@android:id/tabs"
     android:layout_width="match_parent"
     android:layout_height="wrap_content" >
</TabWidget>
<FrameLayout
     android:id="@android:id/tabcontent"
```

```
      android:layout_width="match_parent"
      android:layout_height="match_parent" >
<LinearLayout
      android:id="@+id/songs_tab"
      android:layout_width="wrap_content"
      android:layout_height="match_parent" >
<TextView
    android:id="@+id/battery_layout"
    android:layout_width="wrap_content"
    android:layout_height="match_parent"
    android:text="Songs"
    android:textAlignment="center" />
</LinearLayout>
<LinearLayout
      android:id="@+id/movies_tab"
      android:layout_width="wrap_content"
      android:layout_height="match_parent" >
<TextView
      android:id="@+id/network_layout"
      android:layout_width="wrap_content"
      android:layout_height="match_parent"
      android:text="Movies"
      android:textAlignment="center" />
</LinearLayout>
<LinearLayout
      android:id="@+id/album_tab"
      android:layout_width="wrap_content"
      android:layout_height="match_parent" >
<TextView
      android:id="@+id/device_layout"
```

```
    android:layout_width="wrap_content"
    android:layout_height="match_parent"
    android:text="Albums"
    android:textAlignment="center" />
</LinearLayout>
</FrameLayout>
</LinearLayout>
</TabHost>
```

4. Navigate to src → com.example.tabbarexample folder.
5. Modify the code in MainActivity file as shown in Code Snippet 12. The code shows the declaration and implementation of TabHost and TabSpec.

Code Snippet 12:

```
package com.example.tabbarexample;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.app.TabActivity;
import android.view.Menu;
import android.widget.TabHost;
import android.widget.TabHost.TabSpec;
public class MainActivity extends TabActivity {
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
// Access Tab Host from Main Layout
TabHost tab_host = (TabHost) findViewById(android.R.id. tabhost);
// Set Tab Specification for Songs Tab
TabSpec songs_tab_spec = tab_host.newTabSpec("songs_ tab");
```

```
songs_tab_spec.setContent(R.id.songs_tab);
songs_tab_spec.setIndicator("Songs");
tab_host.addTab(songs_tab_spec);
// Set Tab Specification for Movies Tab
TabSpec movies_tab_spec = tab_host. newTabSpec("movies_tab");
movies_tab_spec.setContent(R.id.movies_tab);
movies_tab_spec.setIndicator("Movies");
tab_host.addTab(movies_tab_spec);
// Set Tab Specification for Album Tab
TabSpec album_tab_spec = tab_host.newTabSpec("album_ tab");
album_tab_spec.setContent(R.id.album_tab);
album_tab_spec.setIndicator("Albums");
tab_host.addTab(album_tab_spec);
}
@Override
public boolean onCreateOptionsMenu(Menu menu) {
// Inflate the menu; this adds items to the action bar if it is //
present.
getMenuInflater().inflate(R.menu.main, menu);
return true;
}
}
```

Figure 5.4 displays the output of the application.

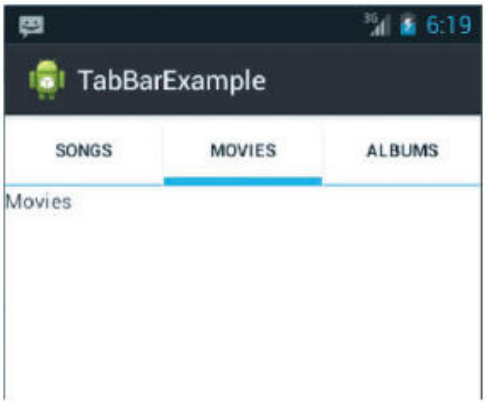


Figure 5.4: TabBarExample – Output

5.2.4 WebView

WebView is used to display Web pages as a part of the activity layout. It does not include any feature of Web browser. WebView loads Web pages and renders raw HTML data.

Following application named WebViewExample demonstrates the use of WebView. To develop the application, perform the following steps:

1. Create a project named WebViewExample in Android Studio.
2. Navigate to res → layout folder.
3. Modify the code in activity_main.xml file as shown in Code Snippet 13. The code uses the WebView to load a Web page.

Code Snippet 13:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/ res/
android"
android:layout_width="fill_parent"
android:layout_height="fill_parent"
android:orientation="vertical" >
<WebView
android:id="@+id/webview01"
android:layout_width="fill_parent"
android:layout_height="wrap_content"
android:layout_weight="1" >
</WebView>
</LinearLayout>
```

4. Navigate to src → com.example.webviewexample folder.
5. Modify the code in MainActivity file as shown in Code Snippet 14. The WebView object is used to load a Web page in the Java file.

Code Snippet 14:

```
package com.example.webviewexample;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.graphics.Bitmap;
import android.view.KeyEvent;
import android.webkit.WebView;
import android.webkit.WebViewClient;
public class MainActivity extends Activity {
WebView web;
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    web = (WebView) findViewById(R.id.webview01);
    web.setWebViewClient(new myWebClient());
    web.getSettings().setJavaScriptEnabled(true);
    web.loadUrl("http://www.android.com/");
}
public class myWebClient extends WebViewClient {
    @Override
    public void onPageStarted(WebView view, String url, Bitmap
                                favicon) {
```

```
            // TODO Auto-generated method stub
             super.onPageStarted(view, url, favicon);
        }
        @Override
        public boolean shouldOverrideUrlLoading(WebView view,
                                                String url) {
             // TODO Auto-generated method stub
           view.loadUrl(url);
          return true;
         }
    }
  // To handle "Back" key press event for WebView to go back to
//previous screen.
@Override
public boolean onKeyDown(int keyCode, KeyEvent event) {
      if ((keyCode == KeyEvent.KEYCODE_BACK) && web. canGoBack())
{
 web.goBack();
     return true;
      }
return super.onKeyDown(keyCode, event);
}
}
```

6. Add the following permission in AndroidManifest.xml file as shown in Code Snippet 15.

Code Snippet 15:

```
...
<uses-permission android:name="android.permission.INTERNET" />
</manifest>
```

Figure 5.5 displays the output of the application.

Figure 5.5: WebViewExample – Output

5.2.5 ViewFlipper

The ViewFlipper can be used to animate between two or more added views. Here, only one child-view can be visible at a time. User can navigate between child views. In ViewFlipper, flip-in and flip-out animations are used while navigating between child views. The developer can also set automatic flipping between each child views at regular interval.

Following application named ViewFlipperExample demonstrates the use of ViewFlipper. To develop the application, perform the following steps:

1. Create a project named ViewFlipperExample in Android Studio.
2. Navigate to res folder.
3. Create a new folder under it named anim.
4. Create a new Android XML file named flipin.

5. Modify the code in flipin.xml file as shown in Code Snippet 16.

Code Snippet 16:

```
<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/ android"
android:interpolator="@android:anim/decelerate_ interpolator" >
<translate
android:duration="500"
android:fromXDelta="-100%"
android:toXDelta="0%" />
</set>
```

6. Create another new Android XML file named flipout under res → xml.
7. Modify the code in flipout.xml file as shown in Code Snippet 17.

Code Snippet 17:

```
<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/ android"
android:interpolator="@android:anim/decelerate_ interpolator" >
<translate
android:duration="500"
android:fromXDelta="0%"
android:toXDelta="100%" /> </set>
```

8. Navigate to res → layout folder.
9. Modify the code in activity_main.xml file as shown in Code Snippet 18. The code uses ViewFlipper widget.

Code Snippet 18:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/ res/
android"
android:layout_width="fill_parent"
android:layout_height="fill_parent"
```

```
android:orientation="vertical" >
<TextView
android:layout_width="fill_parent"
android:layout_height="wrap_content"
android:text="View Flipper" />
<ViewFlipper
android:id="@+id/viewflipper"
android:layout_width="fill_parent"
android:layout_height="fill_parent" >
<LinearLayout
android:layout_width="fill_parent"
android:layout_height="fill_parent"
android:orientation="vertical" >
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="First Screen" />
<Button
android:id="@+id/button1"
android:layout_width="fill_parent"
android:layout_height="fill_parent"
android:text="Flip to Next page" />
</LinearLayout>
<LinearLayout
android:layout_width="fill_parent"
android:layout_height="fill_parent"
android:orientation="vertical" >
<TextView
```

```
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Second Screen" />
<Button
android:id="@+id/button2"
android:layout_width="fill_parent"
android:layout_height="fill_parent"
android:text="Flipback" />
</LinearLayout>
</ViewFlipper>
</LinearLayout>
```

10. Navigate to src → com.example.viewflipperexample folder.
11. Modify the code in MainActivity file as shown in Code Snippet 19. The ViewFlipper is implemented in Java file.

Code Snippet 19:

```
package com.example.viewflipperexample;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.view.View;
import android.view.animation.Animation;
import android.view.animation.AnimationUtils;
import android.widget.Button;
import android.widget.ViewFlipper;
public class MainActivity extends Activity {
@Override
protected void onCreate(Bundle savedInstanceState) {
```

```
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_main);
      final ViewFlipper MyViewFlipper = (ViewFlipper)
                                        findViewById(R.id.viewflipper);
      Button button1 = (Button) findViewById(R.id.button1);
      Button button2 = (Button) findViewById(R.id.button2);
      Animation animationFlipIn = AnimationUtils.
                                loadAnimation(this, R.anim.flipin);
      Animation animationFlipOut = AnimationUtils.
                                loadAnimation(this, R.anim.flipout);
      MyViewFlipper.setInAnimation(animationFlipIn);
      MyViewFlipper.setOutAnimation(animationFlipOut);
     button1.setOnClickListener(new Button. OnClickListener() {
            @Override
            public void onClick(View arg0) {
                  // TODO Auto-generated method stub
                  MyViewFlipper.showNext();
              }
       });
     button2.setOnClickListener(new Button. OnClickListener() {
      @Override
      public void onClick(View arg0) {
            // TODO Auto-generated method stub
             MyViewFlipper.showPrevious();
      }
     });
}
@Override
public boolean onCreateOptionsMenu(Menu menu) {
```

```
        // Inflate the menu; this adds items to the action bar if it
//is present.
        getMenuInflater().inflate(R.menu.main, menu);
        return true;
}
}
```

Figure 5.6 displays the output of the application.

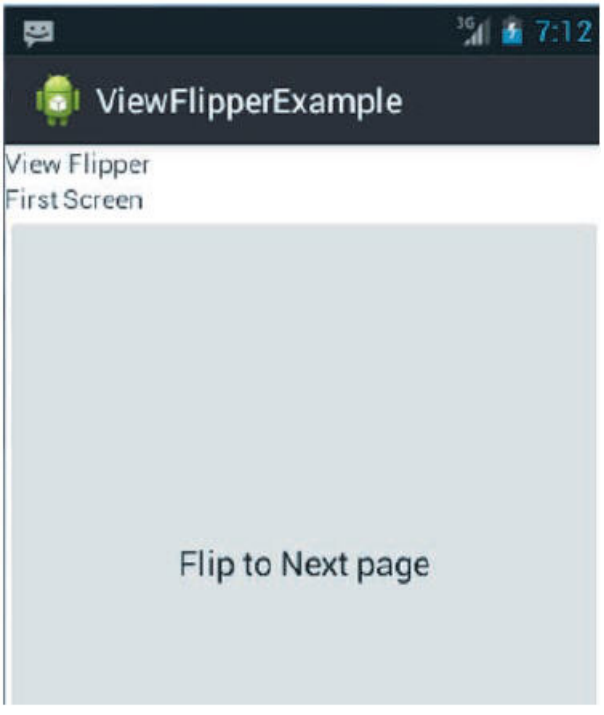


Figure 5.6: ViewFlipper – Output

Figure 5.7 displays the output when the user clicks in the shaded area.

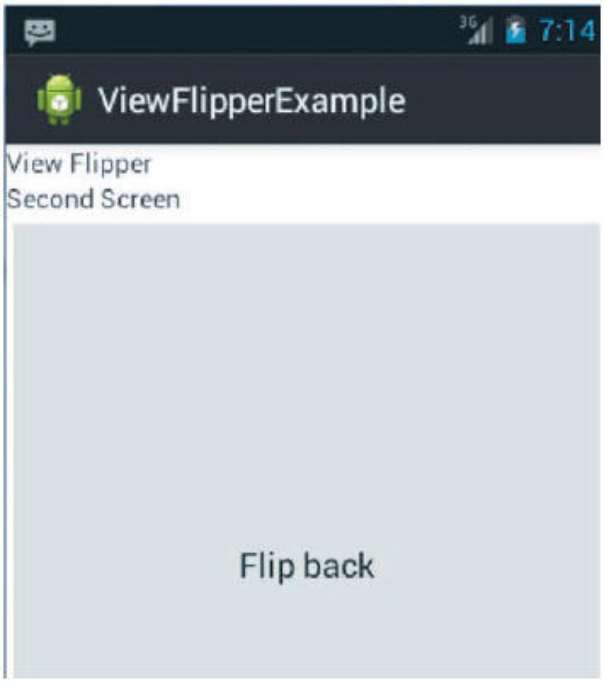


Figure 5.7: Flip Card

5.2.6 VideoView

VideoView is used to play a video file. Android 4.2 can support large extent of video formats including .aac format while lower versions cannot play all the video files of different formats.

5.3 Dialogs

A dialog is displayed in front of an activity as a small window. Dialogs help users in selecting and confirming actions, alerting, or displaying warning messages. When dialogs are active, the activity loses the focus. It is used for interrupting the user activity and forces the user to perform a task related to the current activity. A dialog can be created by extending the Dialog class. A dialog should not be instantiated directly. It should be done by using any one of the following subclasses:

1. **DatePicker Dialog** – This helps the user to select a date from the Date Picker View.
2. **TimePicker Dialog** – This helps the user to select a time from the Time Picker View.
3. **Progress Dialog** – This displays a progress wheel or progress bar to the user.
4. **Alert Dialog** – It can manage zero, one, two, or three buttons. It also manages a specified set of items that include check boxes or radio buttons.
5. **Toasts** – This displays information for a limited amount of time. They can be used like on screen notifications to show hints or status information.
6. **Custom Dialog** – A dialog with a custom layout as per user requirement.

5.3.1 DatePicker and TimePicker Dialogs

The steps to create the DateTime project are as follows:

1. Create a project named DateTime in Android Studio.
2. Navigate to DateTime → res → layout → activity_main.xml in the Package Explorer pane.
3. Double-click the activity_main.xml file.
4. Modify the code in activity_main.xml file as shown in Code Snippet 20.

Code Snippet 20:

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:tools="http://schemas.android.com/tools"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:onClick="selectDate" >
  <EditText
    android:id="@+id/editText1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignParentLeft="true"
    android:layout_alignParentTop="true"
    android:layout_marginLeft="20dp"
    android:layout_marginTop="56dp"
    android:ems="10"
    android:inputType="date" >
   <requestFocus />
  </EditText>
  <ImageButton
    android:id="@+id/imageButton1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
```

```
    android:layout_alignBottom="@+id/editText1"
    android:layout_toRightOf="@+id/editText1"
    android:cropToPadding="true"
    android:onClick="selectDate"
    android:src="@drawable/ic_datepicker" />
  <Button
    android:id="@+id/button1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignRight="@+id/editText1"
    android:layout_centerVertical="true"
    android:layout_marginRight="50dp"
    android:text="Next" />
  <TextView
    android:id="@+id/textView1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignParentTop="true"
    android:layout_alignRight="@+id/button1"
    android:layout_marginRight="32dp"
    android:layout_marginTop="20dp"
    android:text="Date Picker Example" />
</RelativeLayout>
```

5. Create an Android xml file named activitymain.xml in res/layout/ folder.
6. Modify the code as shown in Code Snippet 21.

Code Snippet 21:

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools"
   android:layout_width="match_parent"
```

```
  android:layout_height="match_parent" >
 <TextView
  android:id="@+id/text_picked_time"
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:layout_centerHorizontal="true"
  android:layout_centerVertical="true" />
 <Button
  android:id="@+id/button_pick_time"
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:layout_below="@id/text_picked_time"
  android:layout_centerHorizontal="true"
  android:layout_centerVertical="true"
  android:text="Click for Time" />
 <TextView
  android:id="@+id/textView1"
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:layout_alignParentTop="true"
  android:layout_alignRight="@+id/text_picked_time"
  android:layout_marginTop="72dp"
  android:text="Time Picker Example" />
</RelativeLayout>
```

7. Navigate to src → com.example.datetime package.
8. Create a Java class named TimePickerFragment.java file.

9. Modify the code as given in Code Snippet 22.

Code Snippet 22:

```
package com.example.datetime;
import android.app.Activity;
import android.app.Dialog;
import android.app.TimePickerDialog;
import android.os.Bundle;
import android.support.v4.app.DialogFragment;
import android.text.format.DateFormat;
import android.widget.TimePicker;
import java.util.Calendar;
public class TimePickerFragment extends DialogFragment implements
TimePickerDialog.OnTimeSetListener{
  private TimePickedListener mListener;
  @Override
  public Dialog onCreateDialog(Bundle savedInstanceState)
  {
   // use the current time as the default values for the picker
   final Calendar c = Calendar.getInstance();
   int hour = c.get(Calendar.HOUR_OF_DAY);
   int minute = c.get(Calendar.MINUTE);
  // create a new instance of TimePickerDialog and return it
   return new TimePickerDialog(getActivity(), this, hour, minute,
DateFormat.is24HourFormat(getActivity()));
 }
   @Override
   public void onAttach(Activity activity)
 {
//when the fragment is initially shown (i.e. attached to
//the activity), cast the activity to the callback interface
//type
```

```
  super.onAttach(activity);
     try{
      mListener = (TimePickedListener) activity;
     }
      catch (ClassCastException e){
       throw new ClassCastException(activity.
toString() + " must implement " +
TimePickedListener.class.getName());
     }
    }
    public void onTimeSet(TimePicker view, int hourOfDay, int
minute) {
     // when the time is selected, send it to the activity via
//its callback interface method
    Calendar c = Calendar.getInstance();
    c.set(Calendar.HOUR_OF_DAY, hourOfDay);
    c.set(Calendar.MINUTE, minute);
    mListener.onTimePicked(c);
   }
   public static interface TimePickedListener{
    public void onTimePicked(Calendar time);
   }
 }
```

10. Create a Java class named Timepicker.java file, under src/com.example.datetime package.

11. Modify the code as given in Code Snippet 23.

Code Snippet 23:

```
package com.example.datetime;
import android.os.Bundle;
import android.support.v4.app.DialogFragment;
import android.support.v4.app.FragmentActivity;
import android.text.format.DateFormat;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.TextView;
import java.util.Calendar;
import com.example.datetime.TimePickerFragment.
                              TimePickedListener;
public class Timepicker extends FragmentActivity implements
TimePickedListener {
 private TextView mPickedTimeText;
 private Button mPickTimeButton;
 @Override
 public void onCreate(Bundle savedInstanceState) {
  super.onCreate(savedInstanceState);
  setContentView(R.layout.activitymaino);
   mPickedTimeText = (TextView) findViewById(R.id.text_
                                          picked_time);
   mPickTimeButton = (Button) findViewById(R.id.button_pick_
                                        time);
   mPickTimeButton.setOnClickListener(new OnClickListener() {
   public void onClick(View v) {
// show the time picker dialog
   DialogFragment newFragment =  new TimePickerFragment();
```

```
    newFragment.show(getSupportFragmentManager(),
    "timePicker");
    }
   });
 }
 public void onTimePicked(Calendar time) {
  // display the selected time in the TextView
  mPickedTimeText.setText(DateFormat.format("h:mm a", time));
 }
}
```

12. Navigate to src → com.example.datetime → MainActivity.java file.
13. Modify the code as given in Code Snippet 24.

Code Snippet 24:

```
package com.example.datetime;
import java.util.Calendar;
import android.annotation.SuppressLint;
import android.app.DatePickerDialog;
import android.app.Dialog;
import android.content.Intent;
import android.os.Bundle;
import android.view.Menu;
import android.view.View;
import android.view.View.OnClickListener;
import android.support.v4.app.FragmentActivity;
import android.support.v4.app.DialogFragment;
import com.example.datetime.R;
import android.widget.Button;
import android.widget.DatePicker;
import android.widget.EditText;
```

```
@SuppressLint("ValidFragment")
public class MainActivity extends FragmentActivity {
 EditText mEdit;
 @Override
 protected void onCreate(Bundle savedInstanceState) {
  super.onCreate(savedInstanceState);
  setContentView(R.layout.activity_main);
  Button next= (Button) findViewById(R.id.button1);
  next.setOnClickListener(new OnClickListener() {
     public void onClick(View v) {
     Intent o=new
Intent(getApplicationContext(),Timepicker.class);
    startActivity(o);
   }
  });
 }
 public void selectDate(View view) {
   DialogFragment newFragment = new SelectDateFragment();
    newFragment.show(getSupportFragmentManager(), "DatePicker");
 }
 public void populateSetDate(int year, int month, int day) {
 mEdit = (EditText)findViewById(R.id.editText1);
 mEdit.setText(month+"/"+day+"/"+year);
 }
public class SelectDateFragment extends DialogFragment
implements DatePickerDialog.OnDateSetListener {
 @Override
```

```
public Dialog onCreateDialog(Bundle savedInstanceState) {
   final Calendar calendar = Calendar.getInstance();
   int yy = calendar.get(Calendar.YEAR);
   int mm = calendar.get(Calendar.MONTH);
   int dd = calendar.get(Calendar.DAY_OF_MONTH);
   return new DatePickerDialog(getActivity(), this, yy, mm, dd);
 }
 public void onDateSet(DatePicker view, int yy, int mm, int dd) {
  populateSetDate(yy, mm+1, dd);
 }
  }
 @Override
 public boolean onCreateOptionsMenu(Menu menu) {
  // Inflate the menu; this adds items to the action bar if it is
present.
  getMenuInflater().inflate(R.menu.main, menu);
  return true;
 }
}
```

14. Navigate to AndroidManifest.xml file.
15. Add the line of code, <activity android name=".Timepicker"></activity>. Now, the AndroidManifest.xml file will look like the code present in Code Snippet 25.

Note - The image name should be in lowercase without spaces.

Code Snippet 25:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
  xmlns:android="http://schemas.android.com/apk/res/android"
  package="com.example.datetime"
  android:versionCode="1"
  android:versionName="1.0" >
  <uses-sdk
   android:minSdkVersion="8"
   android:targetSdkVersion="17" />
  <application
   android:allowBackup="true"
   android:icon="@drawable/ic_launcher"
   android:label="@string/app_name"
   android:theme="@style/AppTheme" >
  <activity
   android:name="com.example.datetime.MainActivity"
   android:label="@string/app_name" >
   <intent-filter>
    <action android:name="android.intent.action.MAIN" />
     <category android:name="android.intent.category.LAUNCHER" />
   </intent-filter>
   </activity>
   <activity android:name=".Timepicker"></activity>
  </application>
</manifest>
```

16. Navigate to res → drawable-hdpi folder.
17. Paste the image ic_datepicker.png.
18. Navigate to res → values → strings.xml.

19. Modify the code as given in Code Snippet 26.

Code Snippet 26:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <string name="app_name">DateTime</string>
  <string name="action_settings">Settings</string>
  <string name="hello_world">Hello world!</string>
  <string name="menu_settings">Settings</string>
  <string name="title_activity_main">MainActivity</string>
  <string name="select">Select</string>
  <string name="selectdate">Select Date</string>
</resources>
```

Once, the application is executed the output will be as shown in figure 5.8.

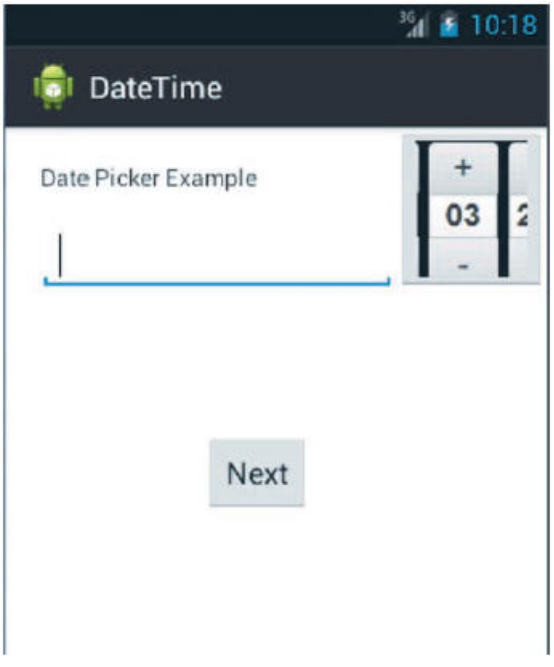


Figure 5.8: Date Picker

Once the date icon is clicked, the output will be as shown in figure 5.9.

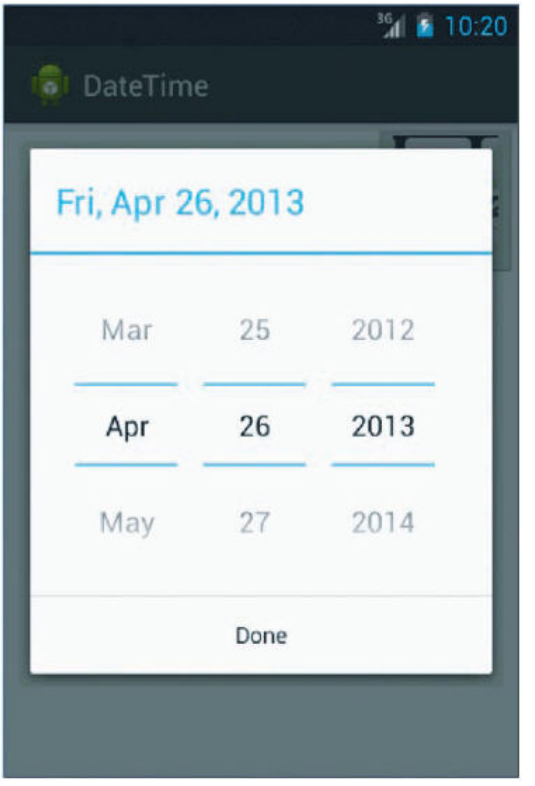


Figure 5.9: Pick the Date

Click Done and the output will be as shown in figure 5.10.

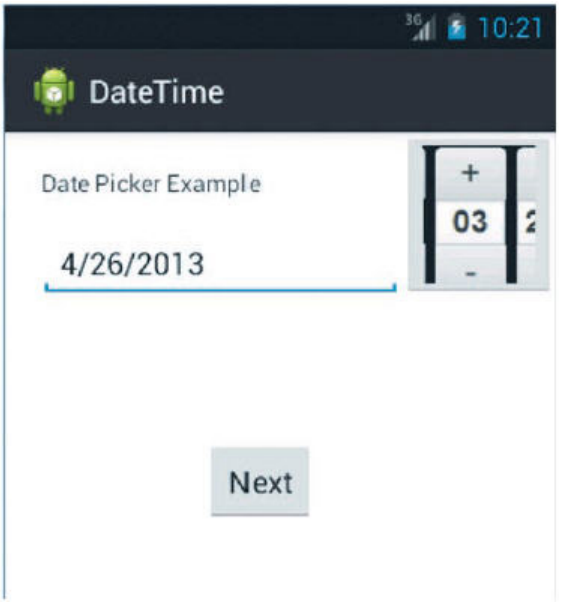


Figure 5.10: Clicking Done

Click Next and the output will be as shown in figure 5.11.

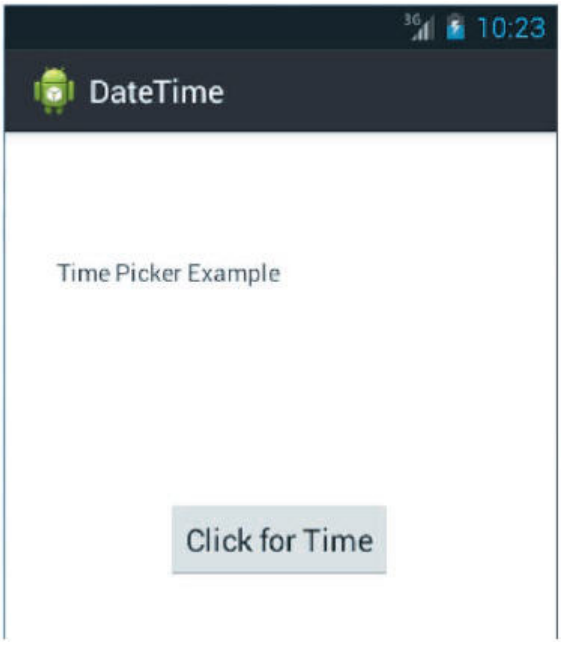


Figure 5.11: Clicking Next

Click the button 'Click for Time' and the output will be as shown in figure 5.12.

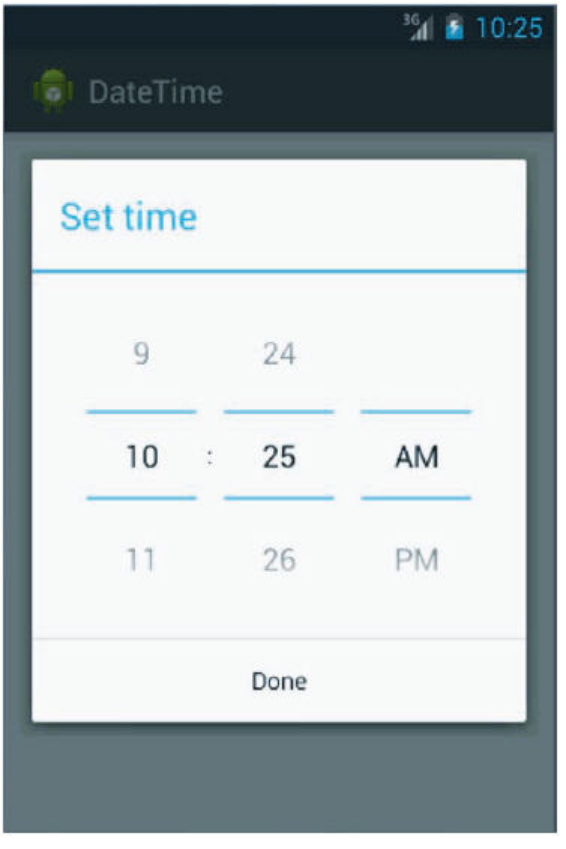


Figure 5.12: Set the Time

5.3.2 Progress Dialog Notification

It is a dialog with a progress wheel or bar. It is specifically useful for cases where the application fetches some data from the Internet and when done, displays it on the device screen. Since it takes 6-7 seconds to do that, meanwhile, you can use the progress dialog to show the status of download.

This section explains the procedure to create a ProgressDialog application to illustrate the use of a Progress dialog.

The steps to create a ProgressDialog application are as follows:

1. Create a project named ProgressDialog in Android Studio.
2. Navigate to res → layout → activity_main.xml.
3. Modify the code of activity_main.xml file as shown in Code Snippet 27.

Code Snippet 27:

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:paddingBottom="@dimen/activity_vertical_margin"
android:paddingLeft="@dimen/activity_horizontal_margin"
android:paddingRight="@dimen/activity_horizontal_margin"
android:paddingTop="@dimen/activity_vertical_margin"
tools:context=".MainActivity">
<Button
android:id="@+id/buttonprogressdialog"
android:layout_width="fill_parent"
android:layout_height="wrap_content"
android:layout_alignParentTop="true"
android:layout_centerHorizontal="true"
android:layout_marginTop="31dp"
android:layout_marginLeft="50dp"
android:layout_marginRight="50dp"
android:text="Progress Dialog"/>
</RelativeLayout>
```

4. Navigate to src → com.example.progressdialog → MainActivity.java file.
5. Modify the code of MainActivity.java file as shown in Code Snippet 28.

Code Snippet 28:

```
package com.example.progressdialog;
import android.os.Bundle;
import android.os.Handler;
import android.os.Message;
import android.app.Activity;
import android.view.Menu;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.view.View;
import android.app.ProgressDialog;
public class MainActivity extends Activity {
 Button pdring_btn, pdbar_btn;
 ProgressDialog myPd_bar;
 @Override
 protected void onCreate(Bundle savedInstanceState) {
  super.onCreate(savedInstanceState);
  setContentView(R.layout.activity_main);
  pdring_btn = (Button) findViewById(R.id.buttonprogressdialog);
  pdring_btn.setOnClickListener(new OnClickListener() {
   public void onClick(View v) {
    // TODO Auto-generated method stub
    final ProgressDialog myPd_ring = ProgressDialog.show(
                                MainActivity.this, "Please wait",
      "Loading please wait..", true);
       myPd_ring.setCancelable(true);
       new Thread(new Runnable() {
        @Override
public void run() {
```

```
// TODO Auto-generated method stub
      try {
       Thread.sleep(5000);
      } catch (Exception e) {
      }
      myPd_ring.dismiss();
     }
    }).start();
   }
  });
  pdbar_btn = (Button) findViewById(R.id.buttonprogressdialog);
  pdbar_btn.setOnClickListener(new OnClickListener() {
   @Override
   public void onClick(View v) {
    // TODO Auto-generated method stub
    myPd_bar = new ProgressDialog(MainActivity.this);
    myPd_bar.setMessage("Loading....");
    myPd_bar.setTitle("Please Wait..");
    myPd_bar.setProgressStyle(myPd_bar.STYLE_HORIZONTAL);
    myPd_bar.setProgress(0);
    myPd_bar.setMax(30);
    myPd_bar.show();
    new Thread(new Runnable() {
     @Override
public void run() {
      // TODO Auto-generated method stub
      try {
      while (myPd_bar.getProgress() <=myPd_bar.getMax()) {
        Thread.sleep(1000);
        handle.sendMessage(handle.obtainMessage());
```

```
if (myPd_bar.getProgress() == myPd_bar.getMax()) {
          myPd_bar.dismiss();
         }
        }
       } catch (Exception e) {
       }
      }
     }).start();
    }
    Handler handle = newHandler() {
     @Override
     public void handleMessage(Message msg) {
      // TODO Auto-generated method stub
      super.handleMessage(msg);
      myPd_bar.incrementProgressBy(5);
     }
    };
   });
 }
 @Override
 public boolean onCreateOptionsMenu(Menu menu) {
  // Inflate the menu; this adds items to the action bar if it is
//present.
  getMenuInflater().inflate(R.menu.main, menu);
  return true; }
}
```

Figure 5.13 displays the output.

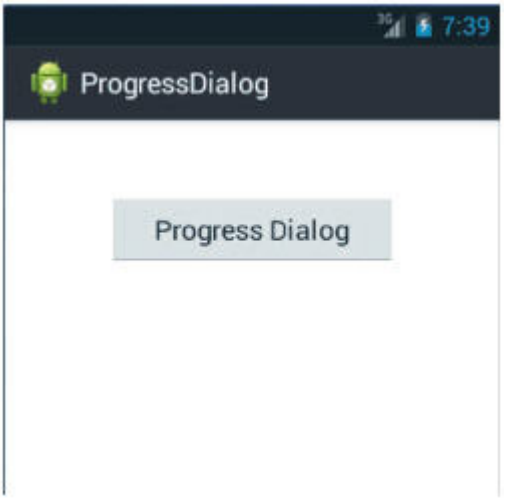


Figure 5.13: Application Displaying the Progress Dialog Button

Clicking the button will display the output as shown in figure 5.14.

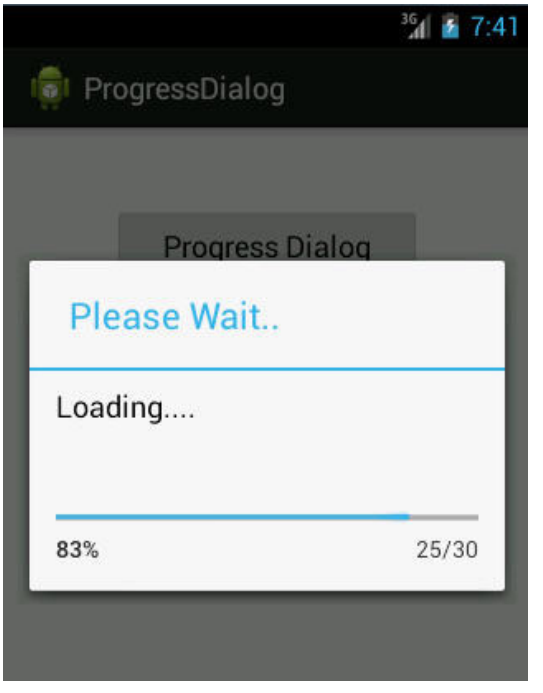


Figure 5.14: Progress Bar

5.3.3 Alert Dialog

Similar to dialogs, alerts also require an action to be performed by the user. Depending on the severity of the alert, the display differs. Most alerts do not require a title. The content can be displayed in a brief sentence that is a question or a statement. The action buttons need to appear below the statement. In rare cases, a title bar is required for an alert. The developer needs to provide these only when the alert is displaying important information, such as, data loss, loss of connectivity, and so on. The alert should again display a short statement or question and the user should be able to quickly make the next decision using the action buttons. Alert Dialogs can also be used to get text input from the user as shown in figure 5.15.

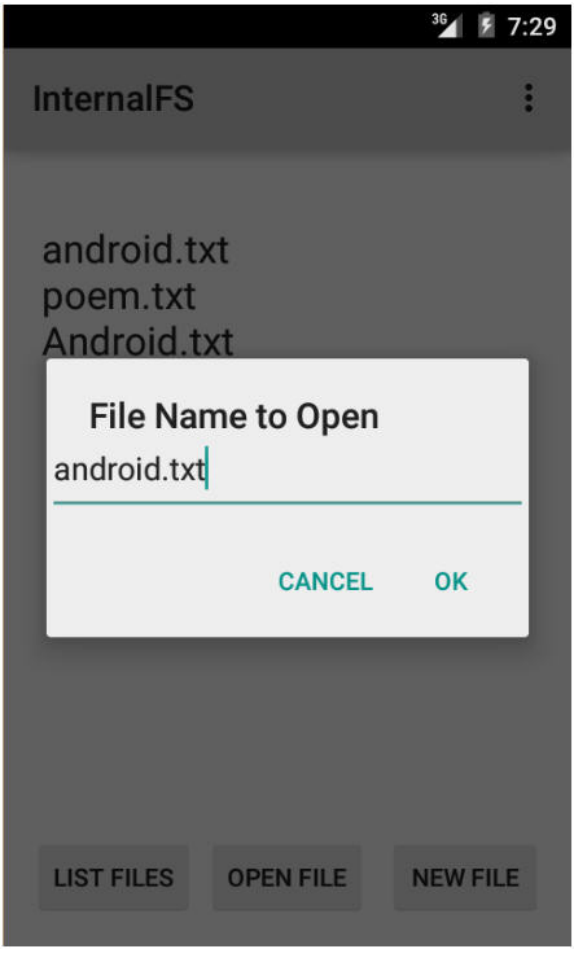


Figure 5.15: Dismissive and Affirmative Action

The dialog display includes a title region, content area, and action buttons. The title region displays the dialog content. For instance, it can make a request for an action from the user. The content area differs according to the type of dialog displayed. For instance, it could display a check box or an alert that requires the user to decide the next action. Action buttons typically require the user to perform an action, for instance, select Cancel or OK. There can also be action buttons that require specific actions.

The steps to create an application named AlertDialog are as follows:

1. Create a project named AlertDialog in Android Studio.
2. Navigate to res → layout → activity_main.xml.
3. Modify the code of activity_main.xml file as shown in Code Snippet 29.

Code Snippet 29:

```
<RelativeLayoutxmlns:android="http://schemas.android.com/apk/
res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:paddingBottom="@dimen/activity_vertical_margin"
android:paddingLeft="@dimen/activity_horizontal_margin"
android:paddingRight="@dimen/activity_horizontal_margin"
android:paddingTop="@dimen/activity_vertical_margin"
tools:context=".MainActivity">
<Button
android:id="@+id/buttonAlert "
android:layout_width="fill_parent"
android:layout_height="wrap_content"
android:layout_alignParentTop="true"
android:layout_centerHorizontal="true"
android:layout_marginLeft="50dp"
android:layout_marginRight="50dp"
android:layout_marginTop="31dp"
android:text="Alert Dialog"/>
</RelativeLayout>
```

4. Navigate to src → com.example.alertdialog → Main Activity.xml file.
5. Modify the code of MainActivity.xml file as shown in Code Snippet 30.

Code Snippet 30:

```
package com.example.alertdialog;
import android.os.Bundle;
import android.app.Activity;
import android.app.AlertDialog;
import android.content.Context;
import android.content.DialogInterface;
import android.view.Menu;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.Toast;
publicclass MainActivity extends Activity {
 final Context context = this;
 private Button button;
 @Override
 protected void onCreate(Bundle savedInstanceState) {
  super.onCreate(savedInstanceState);
  setContentView(R.layout.activity_main);
  button = (Button) findViewById(R.id.buttonAlert);
  // add button listener
  button.setOnClickListener(new OnClickListener() {
   @Override
   public void onClick(View arg0) {
    AlertDialog.Builder alertDialogBuilder = new
                              AlertDialog.Builder(context);
// set title
   alertDialogBuilder.setTitle("Confirmation..");
// set dialog message
   alertDialogBuilder.setMessage("Stay in this activity!")
```

```
     .setCancelable(false).setPositiveButton("Yes",
                            new DialogInterface.OnClickListener() {
         public void onClick(DialogInterface dialog,int id) {
          dialog.cancel();
         }
        })
      .setNegativeButton("No",
new DialogInterface.OnClickListener() {
         public void onClick(DialogInterface dialog,int id) {
          Toast.makeText(MainActivity.this, "Good Bye..", Toast.
                                              LENGTH_LONG).show();
 MainActivity.this.finish();
         }
        });
    // create alert dialog
    AlertDialog alertDialog = alertDialogBuilder.create();
    // show it
    alertDialog.show();
   }
  });
 }
 @Override
 public boolean onCreateOptionsMenu(Menu menu) {
  // Inflate the menu; this adds items to the action bar if it is
//present.
  getMenuInflater().inflate(R.menu.main, menu);
  return true;
 }
}
```

Figure 5.16 displays the output of the application.

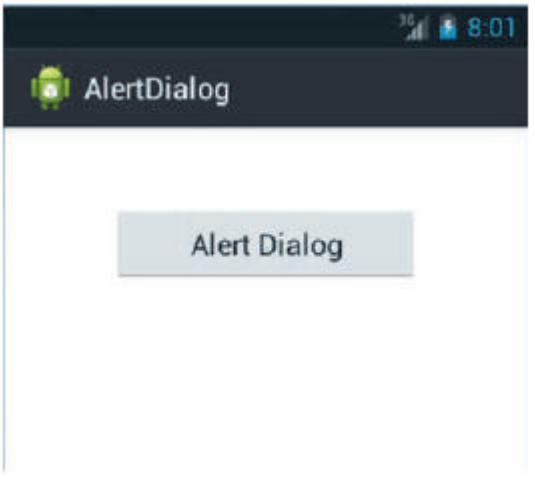


Figure 5.16: AlertDialog Application

Clicking Alert Dialog will display the output as shown in figure 5.17.

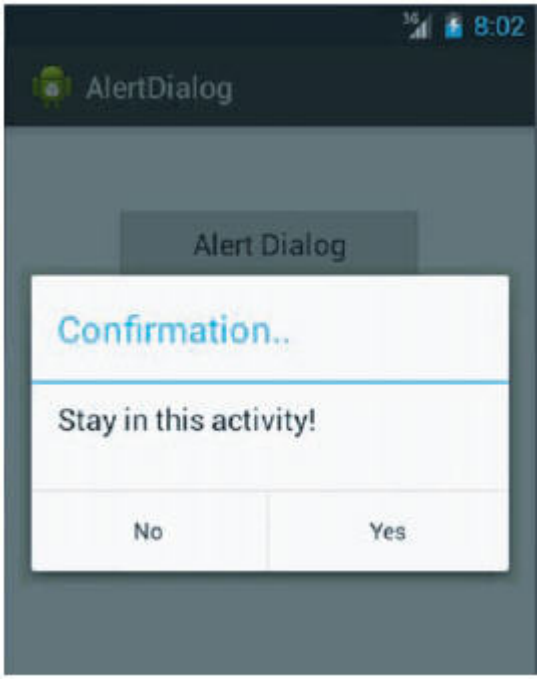


Figure 5.17: Alert Dialog

5.3.4 Popup Dialogs

Popups are smaller versions of dialogs. They need to be designed in such a way that the user only needs to make a selection to move forward or click outside the popup to exit it. They do not need to display action buttons or statements.

The steps to create a PopupNotification application are as follows:

1. Create a project named PopupNotification.
2. Navigate to res → layout → activity_main.xml file.
3. Modify the code of activity_main.xml file as shown in Code Snippet 31.

Code Snippet 31:

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:paddingBottom="@dimen/activity_vertical_margin"
android:paddingLeft="@dimen/activity_horizontal_margin"
android:paddingRight="@dimen/activity_horizontal_margin"
android:paddingTop="@dimen/activity_vertical_margin"
tools:context=".MainActivity">
<Button
android:id="@+id/popup"
android:layout_width="fill_parent"
android:layout_height="wrap_content"
android:layout_alignParentTop="true"
android:layout_centerHorizontal="true"
android:layout_marginTop="31dp"
android:layout_marginLeft="50dp"
android:layout_marginRight="50dp"
android:text="Popup Notification"/>
</RelativeLayout>
```

4. Navigate to src → com.example.popupnotification → MainActivity.java file.
5. Modify the code as shown in Code Snippet 32.

Code Snippet 32:

```
package com.example.popupnotification;
import android.os.Bundle;
import android.app.Activity;
import android.app.AlertDialog;
import android.view.Menu;
import android.view.View;
import android.widget.Button;
import android.view.View.OnClickListener;
import android.content.Context;
import android.content.DialogInterface;
public class MainActivity extends Activity {
 Button back;
 final Context context = this;
 @Override
 protected void onCreate(Bundle savedInstanceState) {
  super.onCreate(savedInstanceState);
  setContentView(R.layout.activity_main);
  back = (Button) findViewById(R.id.popup);
  back.setOnClickListener(new OnClickListener() {
@Override
   public void onClick(View v) {
    // TODO Auto-generated method stub
    AlertDialog.Builder alert_dialog = new AlertDialog.Builder(
                                        context);
    alert_dialog.setTitle("Confirmation...");
    alert_dialog.setNegativeButton("Cancel",
    new DialogInterface.OnClickListener() {
```

```
  @Override
       public void onClick(DialogInterface dialog,int which) {
       }
      });
    alert_dialog.setPositiveButton("Leave current",
                          new DialogInterface.OnClickListener() {
       @Override
       public void onClick(DialogInterface dialog, int which) {
        finish();
       }
      });
    alert_dialog.show();
   }
  });
 }
@Override
 public boolean onCreateOptionsMenu(Menu menu) {
  // Inflate the menu; this adds items to the action bar if it is
present.
getMenuInflater().inflate(R.menu.main, menu);
  returntrue;
 }
}
```

6. Click Run → Run.

The output of the application is as shown in figure 5.18.

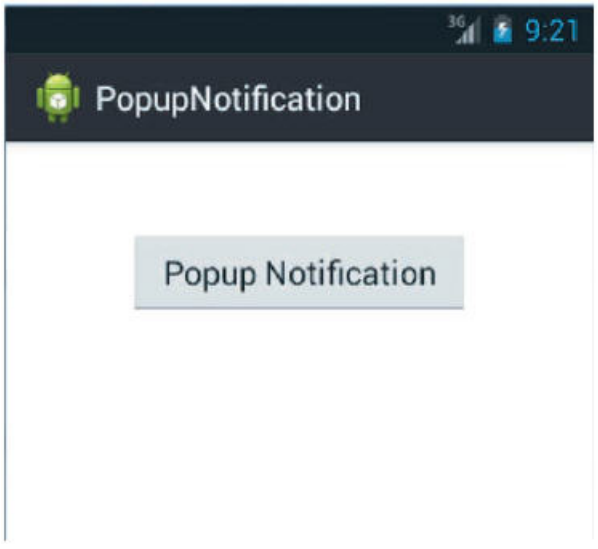


Figure 5.18: PopupNotification Application

On clicking Popup Notification the output is as shown in figure 5.19.

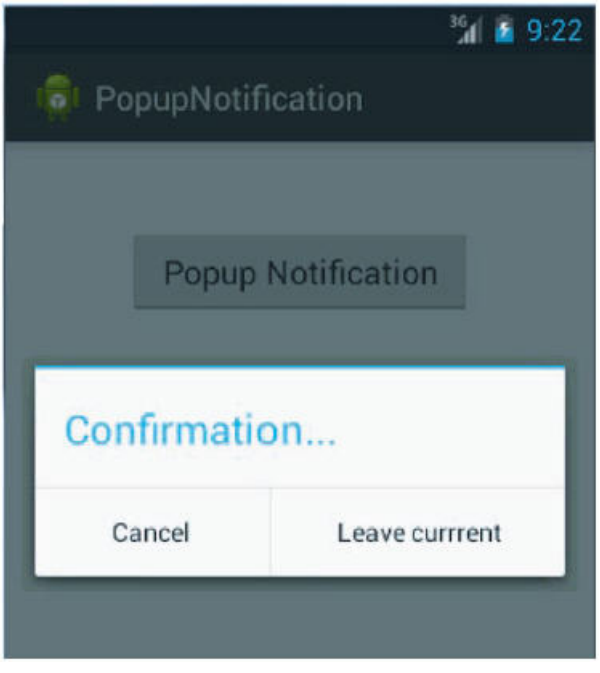


Figure 5.19: Popup Notification

5.3.5 Toasts

A toast is a popup that displays feedback for an activity. For instance, if you exit an SMS that is half complete, a toast displays the message, “Draft Saved” to inform the user that the message can be edited at a later time. Toasts are timed and exit automatically after the timeout period. The message that pops up on the window surface is referred to as the toast notification. It does not alter user’s current activity. It is visible for a few seconds and automatically disappears. It can be created from a Service or an Activity class. A Toast notification is created by declaring an object of the Toast class.

The static method, makeText(), is used to create a Toast object. The method accepts four arguments.

Syntax for makeText() method is as follows:

Syntax:

```
public static Toast makeText(Context c, CharSequence str, int duration)
```

Where,

c: represents the context to be used and can be application of Activity object.

str: represents the message to be displayed.

duration: represents the length of time it will be visible and can either be LENGTH_ SHORT or LENGTH_LONG.

- The show() method is used to display the Toast object.
- The setGravity() method is used to position the message in screen. By default it appears at bottom centered.

The steps to create an application named ToastApplication are as follows:

1. Create a project named ToastApplication in Android Studio.
2. Navigate to res → layout → activity_main.xml file.
3. Modify the code in activity_main.xml file as shown in Code Snippet 33.

Code Snippet 33:

```
<RelativeLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
```

```
  android:paddingBottom="@dimen/activity_vertical_margin"
  android:paddingLeft="@dimen/activity_horizontal_margin"
  android:paddingRight="@dimen/activity_horizontal_margin"
  android:paddingTop="@dimen/activity_vertical_margin"
                        tools:context=".MainActivity" >
 <Button
  android:id="@+id/button1"
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:layout_alignParentTop="true"
  android:layout_centerHorizontal="true"
  android:layout_marginTop="151dp"
  android:text="Click for Toast Notification" />
 </RelativeLayout>
```

4. Navigate to src → com.example.toastapplication → MainActivity.java file.
5. Modify the code of MainActivity.java file as shown in Code Snippet 34.

Code Snippet 34:

```
package com.example.toastapplication;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.Toast;
public class MainActivity extends Activity {
 @Override
 protected void onCreate(Bundle savedInstanceState) {
  super.onCreate(savedInstanceState);
  setContentView(R.layout.activity_main);
  Button b = (Button) findViewById(R.id.button1);
```

```
b.setOnClickListener(new OnClickListener() {
    @Override
    public void onClick(View v) {
     Toast.makeText(getApplicationContext(),
       "This is a Toast Notification", Toast.LENGTH_LONG).show();
    }
  });
}
 @Override
 public boolean onCreateOptionsMenu(Menu menu) {
   // Inflate the menu; this adds items to the action bar if it is
present.
  getMenuInflater().inflate(R.menu.main, menu);
  return true;
 }
}
```

On executing the application, the output will be as shown in figure 5.20.

Figure 5.20: Toast Application

5.3.6 Custom Dialogs

Following application named CustomDialogExample demonstrates the creation of a custom dialog containing an EditText view and a Toast. To develop the application, perform the following steps:

1. Create a project named CustomDialogExample in Android Studio.
2. Navigate to res → layout folder.
3. Modify the code in the activity_main.xml file as shown in Code Snippet 35.

Code Snippet 35:

```
<RelativeLayout xmlns:android="http://schemas.android.com/ apk/
res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:paddingBottom="@dimen/activity_vertical_margin"
android:paddingLeft="@dimen/activity_horizontal_margin"
android:paddingRight="@dimen/activity_horizontal_ margin"
android:paddingTop="@dimen/activity_vertical_margin"
                        tools:context=".MainActivity" >
<Button
android:id="@+id/button1"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_centerHorizontal="true"
android:layout_centerVertical="true"
android:text=" Enter Data " />
</RelativeLayout>
```

4. Navigate to src → com.example.customdialogexample folder.

5. Modify the code in MainActivity file as shown in Code Snippet 36.

Code Snippet 36:

```
package com.example.customdialogexample;
import android.os.Bundle;
import android.app.Activity;
import android.app.AlertDialog;
import android.content.DialogInterface;
import android.content.DialogInterface.OnClickListener;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;
public class MainActivity extends Activity {
EditText userData;
Button signIn;
@Override
protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        signIn = (Button) findViewById(R.id.button1);
        signIn.setOnClickListener(newView.OnClickListener() {
            @Override
            public void onClick(View v) {
                  // TODO Auto-generated method stub
                  alert();
            }
});
}
public void alert() {
AlertDialog.Builder alert = new AlertDialog.
```

```
Builder(MainActivity.this);
userData = new EditText(MainActivity.this);
alert.setTitle("Enter Data..");
alert.setView(userData);
alert.setPositiveButton("Enter Data", new OnClickListener() {
      @Override
      public void onClick(DialogInterface dialog, int which) {
            // TODO Auto-generated method stub
            if (!userData.getText().toString(). equals("")) {
                  Toast.makeText(getApplicationContext(),
                  "Data Entered is : " + userData.getText().
toString(), Toast.LENGTH_LONG).show();
            }
            else
            {
            Toast.makeText(getApplicationContext(),
                   "Field should not be empty",
                    Toast.LENGTH_LONG).show();
            }
      }
});
alert.setNegativeButton("Cancel", new OnClickListener() {
      @Override
      public void onClick(DialogInterface dialog, int which) {
            // TODO Auto-generated method stub
      }
 });
alert.show();
}
}
```

Figure 5.21 displays the output of the application when the user clicks Enter Text.

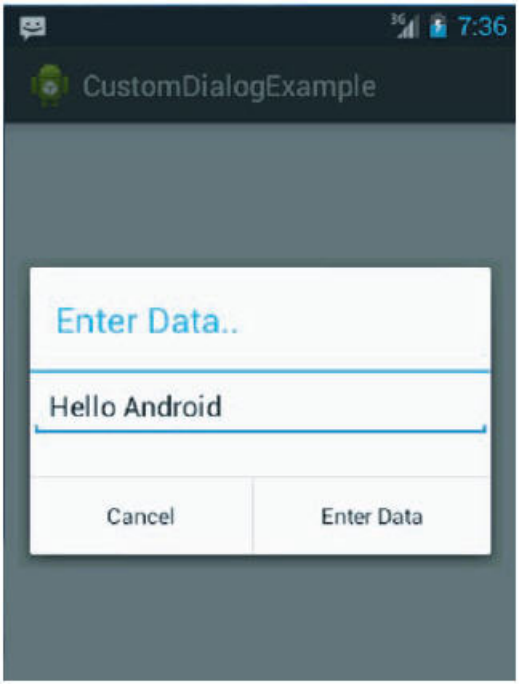


Figure 5.21: Custom Dialog - Output

Figure 5.22 displays the output when the user clicks Enter Data in the dialog.

Figure 5.22: Data Saved

5.4 Application Widgets

App Widgets are miniature application views that can be embedded in other applications (such as the Home screen) and receive periodic updates. Application widgets are useful to provide control to the application without using the entire screen. Android comes with several pre-installed widgets such as the Analog clock, Settings, and so on.

Widgets can be displayed on the home screen on any of the screens selected by the user. The developer has no control over where the widget is displayed or the position of the widget. Lock Screen widget support was introduced in API level 17 (Android 4.2) and dropped in API level 22 (Android 5.1). The procedure to create lock screen widget will be explained for legacy purposes.

For the application to register a widget, the widget needs to be specified in the AndroidManifest.xml file. The code to specify an app widget is demonstrated in Code Snippet 37.

Code Snippet 37:

```
<appwidget-provider
xmlns:android="http://schemas.android.com/apk/res/android"
   ...
   android:widgetCategory="keyguard|home_screen">
</appwidget-provider>
```

The widget category is used to specify the type of the widget. Keyguard specifies a lock screen widget and home_screen specifies a home screen widget. A widget can support either or both of the categories. It is also possible to set a different layout based on the screen the widget is displayed.

In order to implement a widget an AppWidgetProvider object is needed. The abstract class needs to be extended and the onUpdate() methods needs to be implemented. Other methods such as onRecieve(), onDisabled(), onEnabled(), and onDeleted() can be used if required by the developer.

Each app widget requires AppWidgetProviderInfo metadata provided in an xml file. For this purpose an XML file needs to be created. The created XML file is mentioned in the manifest file as shown in the Code Snippet 38.

Code Snippet 38:

```
<meta-data android:name="android.appwidget.provider"
    android:resource="@xml/quote_appwidget_info" />
```

The metadata contains information on the dimension and the look of the widget. The contents of a widget are displayed in Code Snippet 39.

Code Snippet 39:

```
<appwidget-provider
xmlns:android="http://schemas.android.com/apk/res/android"
    android:minWidth="40dp"
    android:minHeight="40dp"
    android:updatePeriodMillis="86400000"
    android:previewImage="@drawable/preview"
    android:initialLayout="@layout/quote_appwidget"
    android:resizeMode="horizontal|vertical"
    android:widgetCategory="home_screen|keyguard"
    android:initialKeyguardLayout="@layout/quote_keygaurd">
</appwidget-provider>
```

The type of the screen can be determined beforehand to decide the layout to be used for the widget. The code to achieve this is as shown in Code Snippet 40.

Code Snippet 40:

```
Bundle myOptions = appWidgetManager.getAppWidgetOptions
(appWidgetId);
int category = myOptions.getInt(AppWidgetManager.OPTION_
APPWIDGET_HOST_CATEGORY, -1);
boolean isKeyguard = (category == AppWidgetProviderInfo.WIDGET_
CATEGORY_KEYGUARD);
int widgetLayout = isKeyguard ? R.layout.quote_keygaurd :
R.layout.quote_appwidget;
```

Consider an application of quotes with options to show and refresh the quote on the Home screen and the lock screens. The steps are as follows:

1. Create a project called Widgets in Android Studio.
2. Navigate to res → layout and open activity_main.xml for editing.

3. Modify the code as shown in Code Snippet 41.

Code Snippet 41:

```
<RelativeLayout
    xmlns:android=”http://schemas.android.com/apk/res/android”
    xmlns:tools=”http://schemas.android.com/tools”
    android:layout_width=”match_parent”
    android:layout_height=”match_parent”
    android:paddingLeft=”@dimen/activity_horizontal_margin”
    android:paddingRight=”@dimen/activity_horizontal_margin”
    android:paddingTop=”@dimen/activity_vertical_margin”
    android:paddingBottom=”@dimen/activity_vertical_margin”
                          tools:context=”.MainActivity”>
    <TextView android:text=”@string/hello_world”
        android:layout_width=”wrap_content”
        android:layout_height=”wrap_content”
        android:id=”@+id/quoteView” />
    <Button
        android:layout_width=”wrap_content”
        android:layout_height=”wrap_content”
        android:text=”New Quote”
        android:id=”@+id/refresh”
        android:onClick=”refreshQuote”
        android:layout_below=”@+id/quoteView”
        android:layout_alignParentStart=”true” />
</RelativeLayout>
```

4. Create a new class called QuoteGenerator in the com.aptech.widgets package.

   This class contains a list of quotes and returns one at random when requested using the getQuote() method.

5. Modify the code as shown in Code Snippet 42.

Code Snippet 42:

```
package aptech.com.widgets;
import java.util.ArrayList;
import java.util.List;
import java.util.Random;
public class QuoteGenerator {
    private List<String> quotes;
    QuoteGenerator(){
        quotes = new ArrayList<String>();
        quotes.add("Success is a Sum of small efforts day in and
                                                    day out");
        quotes.add("The old believe everything");
        quotes.add("Slow and steady wins the race");
        quotes.add("Health is wealth");
        quotes.add("A friend in need is a friend indeed");
        quotes.add("Every cause produces more than one effect");
        quotes.add("Practice makes a man perfect");
        quotes.add("Every story I create, creates a part of me");
        quotes.add("One touch of nature makes the world kin");
        quotes.add("Don't forget to love yourself");
        quotes.add("Knowledge is divine");
    }
    public String getQuote(){
        Random rand = new Random();
        int randomNum = rand.nextInt(11);
        return quotes.get(randomNum);
    }
}
```

6. Navigate to app → manifests and open AndroidManifes.xml.

7. Modify the code as shown in Code Snippet 43.

Code Snippet 43:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
xmlns:android="http://schemas.android.com/apk/res/android"
    package="aptech.com.widgets" >
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <receiver android:name="QuoteAppWidgetProvider" >
            <intent-filter>
                <action android:name="android.appwidget.action.
                                      APPWIDGET_UPDATE" />
                <action android:name="android.appwidget.action.
                                      REFRESH" />
            </intent-filter>
            <meta-data android:name="android.appwidget.provider"
              android:resource="@xml/quote_appwidget_info" />
        </receiver>
        <activity
            android:name=".MainActivity"
            android:label="@string/app_name" >
            <intent-filter>
             <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.
                                        LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

8. Create a new class called QuoteAppWidgetProvider.
9. Modify the code as shown in Code Snippet 44.

Code Snippet 44:

```
package aptech.com.widgets;

import android.app.PendingIntent;

import android.appwidget.AppWidgetManager;

import android.appwidget.AppWidgetProvider;

import android.appwidget.AppWidgetProviderInfo;

import android.content.ComponentName;

import android.content.Context;

import android.content.Intent;

import android.net.Uri;

import android.os.Bundle;

import android.widget.RemoteViews;

public class QuoteAppWidgetProvider extends AppWidgetProvider {

    public void onUpdate(Context context, AppWidgetManager
                         appWidgetManager, int[] appWidgetIds) {

        final int N = appWidgetIds.length;

        for (int i=0; i<N; i++) {

            int appWidgetId = appWidgetIds[i];

            Bundle myOptions = appWidgetManager
                               getAppWidgetOptions (appWidgetId);

            int category = myOptions.getInt(AppWidgetManager.
                           OPTION_APPWIDGET_HOST_CATEGORY, -1);

            boolean isKeyguard = (category ==
                 AppWidgetProviderInfo.WIDGET_CATEGORY_KEYGUARD);

            int widgetLayout = isKeyguard ? R.layout.quote_
                               keygaurd : R.layout.quote_appwidget;

            Intent intent = new Intent("android.appwidget.action.
                                       REFRESH");

            PendingIntent pendingIntent = PendingIntent.
getBroadcast(context, 0, intent, PendingIntent.FLAG_UPDATE_
CURRENT);
```

```
Intent intent2 = new Intent(context, MainActivity.class);

            intent2.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK);

            PendingIntent pendingIntent2 = PendingIntent.
getActivity(context, 0, intent2, PendingIntent.FLAG_UPDATE_
CURRENT);

            RemoteViews views = new RemoteViews(context.
                                getPackageName(), widgetLayout);

            views.setOnClickPendingIntent(R.id.button,
                                          pendingIntent );

            views.setOnClickPendingIntent(R.id.textView,
                                         pendingIntent2);

            appWidgetManager.updateAppWidget(appWidgetId, views);

        }

    }

    @Override

    public void onReceive(Context context, Intent intent) {

        super.onReceive(context, intent);

        if (intent.getAction().equals("android.appwidget.action.
REFRESH")) {

            AppWidgetManager appWidgetManager = AppWidgetManager.
                                            getInstance(context);

            RemoteViews views = new RemoteViews(context.
getPackageName(), R.layout.quote_appwidget);

            ComponentName watchWidget =new
ComponentName(context, QuoteAppWidgetProvider.class);

            QuoteGenerator gen = new QuoteGenerator();

            views.setTextViewText(R.id.textView, gen.getQuote());


            appWidgetManager.updateAppWidget(watchWidget, views);

        }

    }

}
```

10. Create two new layout resources called quote_appwidget.xml and quote_keygaurd.xml for the home screen and lock screen respectively.

11. Modify the code for both the files with the code as shown in Code Snippet 45.

Code Snippet 45:

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="@dimen/appwidget_min_width"
    android:layout_height="@dimen/appwidget_min_height"
    android:padding="@dimen/widget_margin">
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:background="@android:color/holo_blue_dark">
        android:orientation="horizontal">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="37dp"
            android:textAppearance="?android:attr/
                                    textAppearanceMedium"
            android:text="Quotes"
            android:clickable="true"
            android:id="@+id/textView"
            android:layout_weight="1"
            android:textColor="@android:color/background_light"
            android:padding="@dimen/widget_margin" />
        <Button
            style="?android:attr/buttonStyleSmall"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Refresh"
```

```
                        android:id="@+id/button"
            android:textColor="@android:color/holo_blue_dark"
            android:padding="@dimen/widget_margin" />
    </LinearLayout>
</FrameLayout>
```

12. Create a new image asset for the preview image as shown in figure 5.23.

Figure 5.23: Widget Preview

13. Create a new xml file under res → xml called quote_appwidget_info.xml.
14. Modify the code as shown in Code Snippet 46.

Code Snippet 46:

```
<appwidget-provider
xmlns:android="http://schemas.android.com/apk/res/android"
    android:minWidth="40dp"
    android:minHeight="40dp"
    android:updatePeriodMillis="86400000"
    android:previewImage="@drawable/preview"
    android:initialLayout="@layout/quote_appwidget"
    android:resizeMode="horizontal|vertical"
    android:widgetCategory="home_screen|keyguard"
    android:initialKeyguardLayout="@layout/quote_keygaurd">
</appwidget-provider>
```

15. Navigate to res → values. Open dimens.xml for editing.

16. Modify the code as shown in Code Snippet 47.

Code Snippet 47:

```
<resources>

    <!-- Default screen margins, per the Android Design
guidelines. -->

    <dimen name="activity_horizontal_margin">16dp</dimen>

    <dimen name="activity_vertical_margin">16dp</dimen>

    <dimen name="widget_margin">8dp</dimen>

    <dimen name="appwidget_margin">0dp</dimen>

    <dimen name="appwidget_min_width">294dp</dimen>

    <dimen name="appwidget_min_height">72dp</dimen>

</resources>
```

17. Go to run menu and select Run 'app'.

    The application is deployed on the AVD. The output is as shown in figure 5.24.

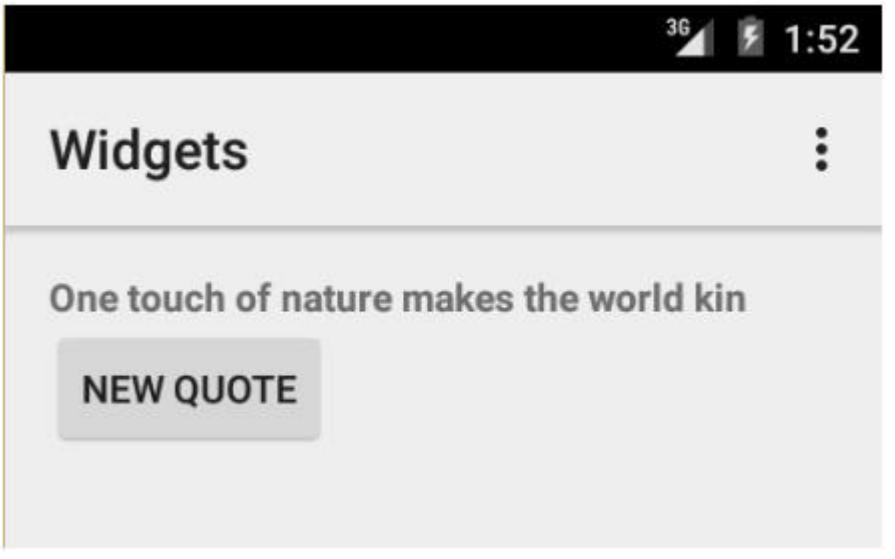


Figure 5.24: Quote Application

The user can get a new quote by clicking the New Quote button as shown in figure 5.25.

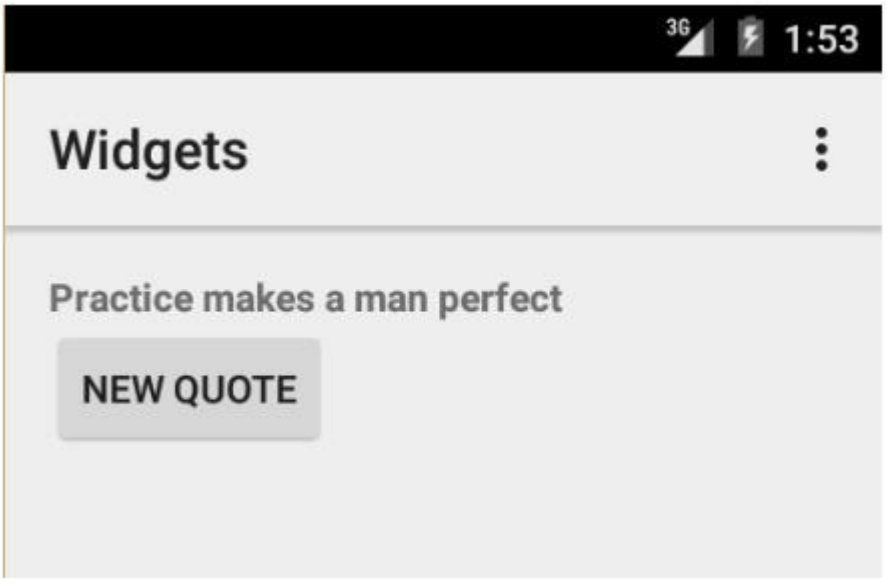


Figure 5.25: New Quote

The user can exit the application and return to the app drawer to add the new widget as shown in figure 5.26.

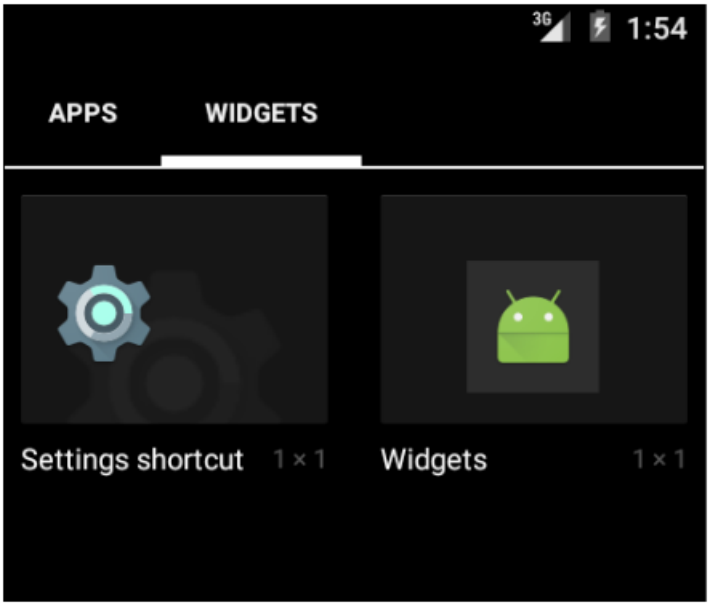


Figure 5.26: Application Launcher Widgets

The widget can be added to the home screen as shown in figure 5.27.

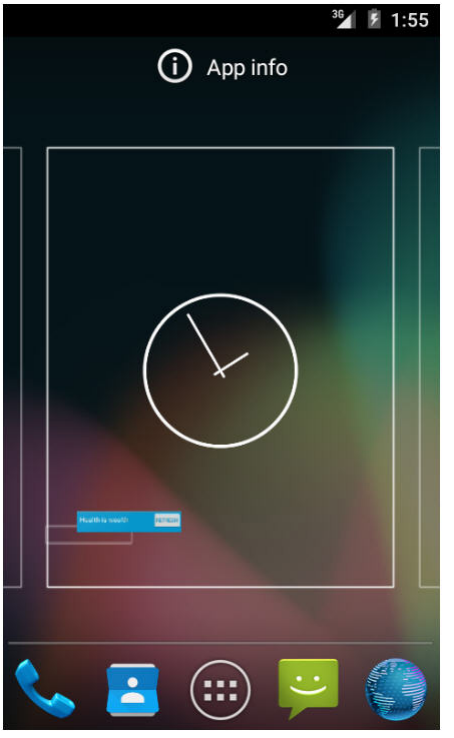


Figure 5.27: Adding the Widget to the Home Screen

The widget is displayed on the home screen as shown in figure 5.28.

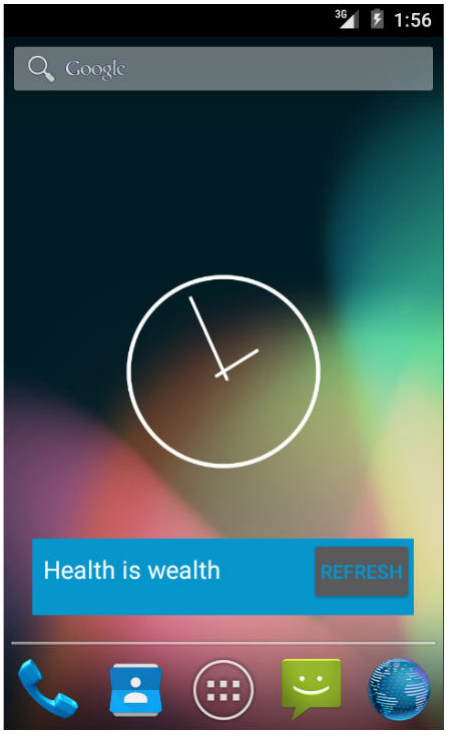


Figure 5.28: Widget on the Home Screen

By clicking the Refresh button a new quote is displayed as shown in figure 5.29.

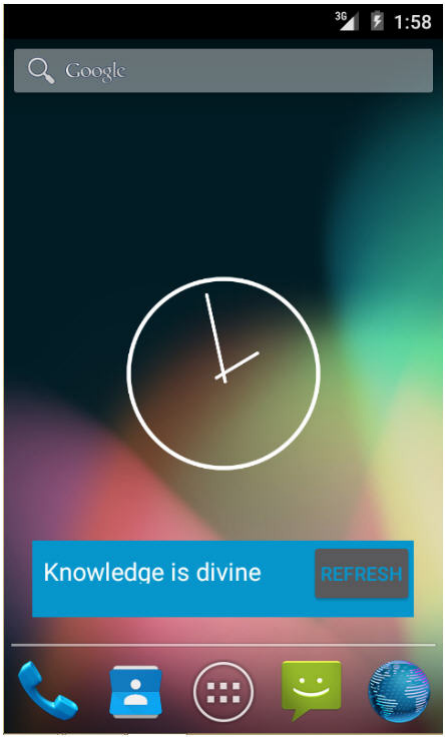


Figure 5.29: Quote Refreshed

By clicking the Quote itself, the application will be loaded as shown in figure 5.30.

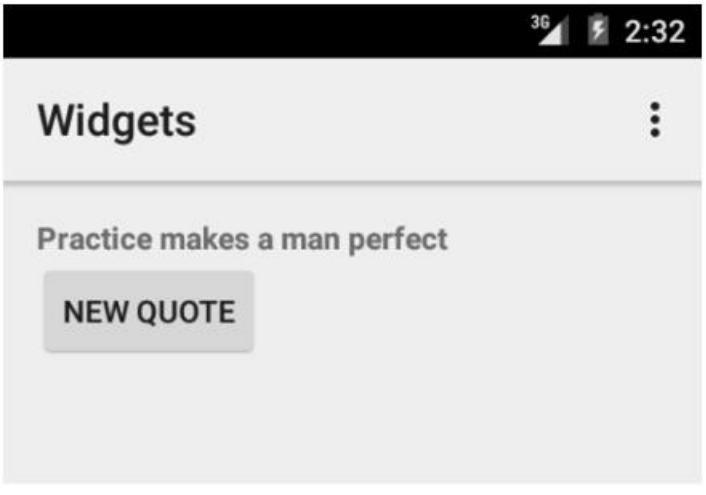


Figure 5.30: Loading the Application Activity from the Widget

5.5 Material Design Philosophy

With the release of Android Lollipop, Google has introduced a new UI design philosophy called **Material Design**. The material design specifies a set of design guidelines to maintain a look, consistent with the base operating system. Although it is not mandatory to follow the Material Design guidelines, Google strongly recommends doing so, at least at a base level. An app with and without Material Design is shown in figure 5.31.

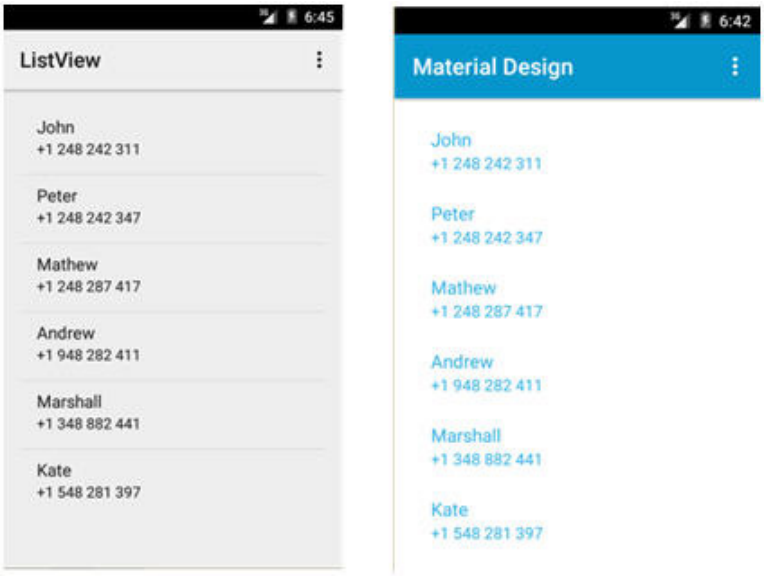


Figure 5.31: Application With and Without Material Design

In order to implement the material design, the developer can implement certain specifications described here.

5.5.1 Material Theme

The simplest guideline towards a material look and feel is utilizing the system themes provided by Google. API level 20 (Android 5.0) comes with Theme.Material (Dark Version) and Theme. Material.Light (Light Version) themes. The developer can set the theme by navigating to res → values → styles.xml and modifying the code as shown in Code Snippet 48.

Code Snippet 48:

```
<resources>
    <!-- Base application theme. -->
    <style name="AppTheme" parent="android:Theme.Material">
        …….
</style>
</resources>
```

Next, edit the AndroidManifest.xml to set the theme to the derived theme as shown in Code Snippet 49.

Code Snippet 49:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
xmlns:android="http://schemas.android.com/apk/res/android"
    package="aptech.com.materialdesign" >
    <application
        ….
        android:theme="@style/AppTheme" >
```

Note - It is possible to directly set the material theme in AndroidManifest.xml. Although, this process is not recommended.

The examples of material dark and material light theme are shown in figure 5.32.

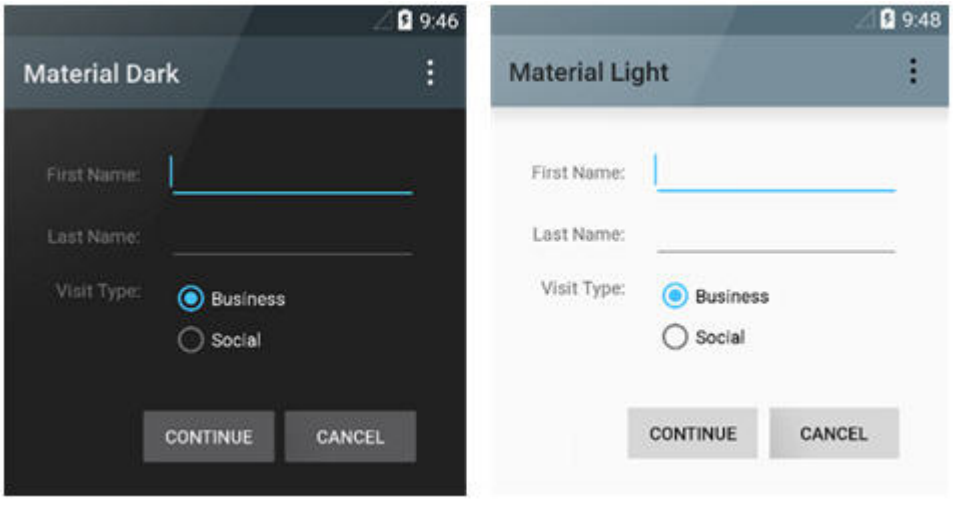


Figure 5.32: Material Light and Material Dark Themes

5.5.2 Transparent Activity Bars and Full Screen

Android Lollipop brings the ability to use transparent Activity bars and to use the entire screen without the notification and the navigation bar. The material design philosophy recommends utilizing this feature when the activity content is static and prominently multimedia information. A browser with a transparent activity bar is shown in figure 5.33.

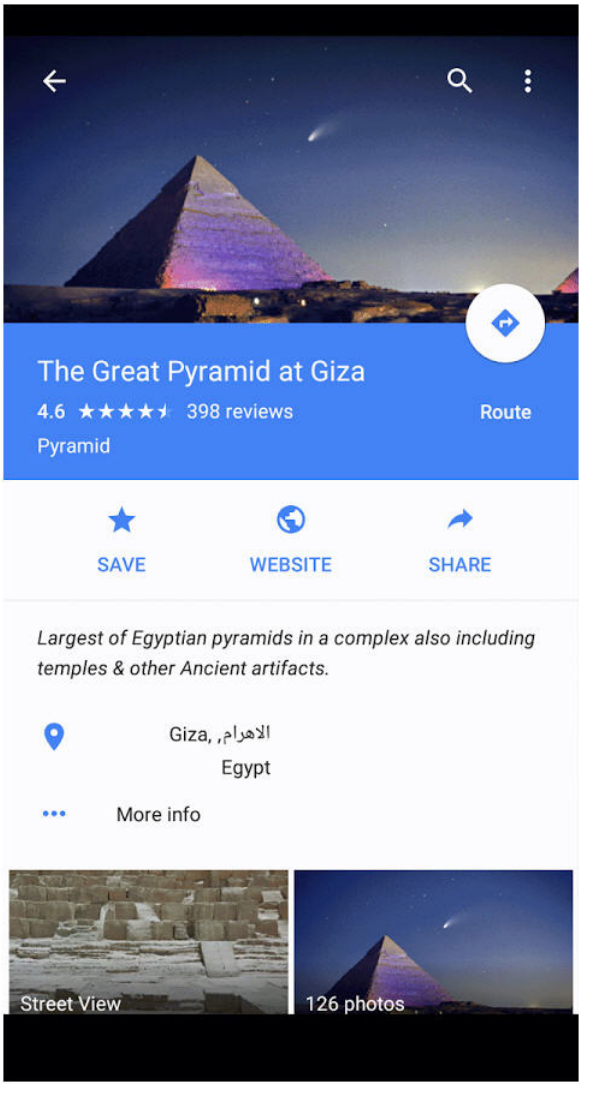


Figure 5.33: Application with Transparent Activity Bar and No Notification Bar

An action bar can be set to transparent by styling it to color/transparent as shown in Code Snippet 50.

Code Snippet 50:

```
<item name="colorPrimary">@android:color/transparent</item>
```

5.5.3 Lists and Cards

Google recommends the developers to use the RecyclerView instead of the standard ListView. The RecyclerView comes with a more flexible layout and fully integrates with the Material Deisgn philosophy. The API to use a RecyclerView is similar to that of a ListView.

The official Gmail app utilizes the ReyclerView. An example of RecyclerView is as shown in figure 5.34.

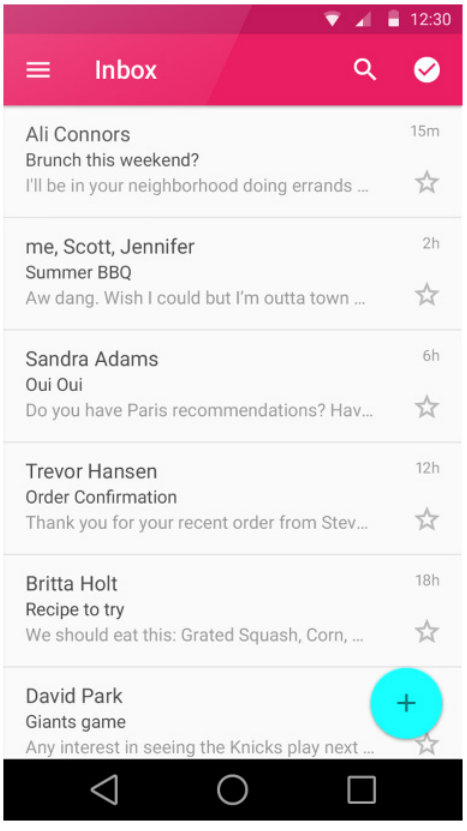


Figure 5.34: RecyclerView

CardView is an essential layout component of the material design philosophy. The API and functionality is similar to a FrameLayout. The screen is divided into sections called 'cards'. The CardView, similar to the RecyclerView, provides a more flexible layout and fully integrates with the Material design philosophy. An example of CardView is shown in figure 5.35.

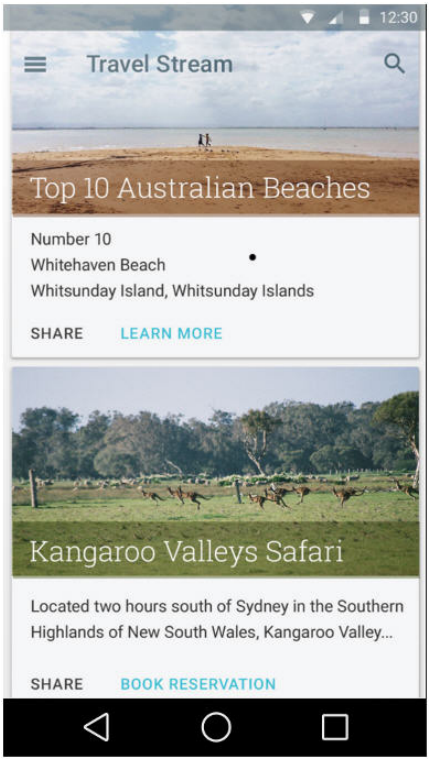


Figure 5.35: CardView

5.5.4 Uniform Layout

Google recommends designing the application layout as multiples of a unit measurement. Dividers of uniform size are recommended whenever a side panel or a navigational menu needs to be displayed. This is contradictory to the philosophy of the metro theme of windows, where each 'tile' is deliberately made of different dimensions. The philosophy is illustrated in figure 5.36.

Figure 5.36: Uniform Layout Philosphy

5.5.5 Shadows

Shadows and elevations are recommended to be used wherever applicable. The android:elevation property can be used to set the elevation value of a view. The property makes the view to appear elevated or hovering above the rest of the layout. The style code to set the elevation and shadow value is shown in Code Snippet 51.

Code Snippet 51:

```
<TextView
    android:id="@+id/textView"
    android:elevation="2dp" >
```

An example of an elevated view is shown in figure 5.37.

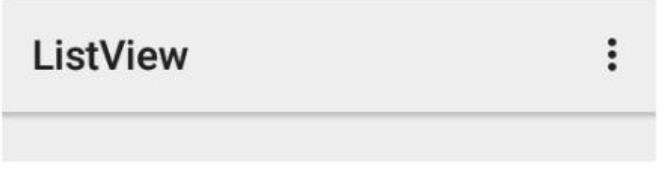


Figure 5.37: Shadows and Elevation

5.5.6 Animations

The material design specification recommends using animations to transition between activities appear more natural. Usage of expand animations is recommended for RecyclerView, CardView, or any similar formats. A fade out animation can be used if a fresh unrelated activity is being loaded. Touch feedback is recommended on clickable objects. Circular reveals are recommended to be used whenever the Activity being displayed is changing the screen orientation.

5.5.7 Color Scheme

Material design utilizes Bright color schemes throughout the application. The developer is strongly recommended to style the app as per the brand requirement. The ability to color the Action and the notification bar allows for full customization of look and feel. The developer can use the holo car pallete using '@android:color/holo_'. The entire color palette specification for can be downloaded from:

http://material-design.storage.googleapis.com/publish/material_v_3/material_ext_publish/0B0J8hsRkk91LSGx6b0w3WWpMQ1k/color_swatches.zip

The color palette can be used to determine the right theme to go along with the application interface. It is strongly recommended to implement a theme which inherits from the base material themes @android:style/Theme.Material or @android:style/Theme.Material. This is to ensure that there is a fall back material color scheme for elements whose color schemes are not customized in the theme. An example material design theme is shown in Code Snippet 52.

Code Snippet 52:

```
<resources>
    <!-- Base application theme. -->
    <style name="AppTheme" parent="android:Theme.Material">

        <item name="android:colorPrimary">@android:color/holo_
                                          green_light </item>
        <item name="android:colorPrimaryDark">@android:color/
                                          holo_green_dark </item>
        <item name="android:statusBarColor">@android:color/holo_
                                          green_dark </item>
        <item name="android:colorAccent">@android:color/holo_
                                          blue_light</item>
        <item name="android:textColorPrimary">@android:color/
                                          white</item>
        <item name="android:windowBackground">@android:color/
                                          white</item>
        <item name="android:navigationBarColor">@android:color/
                                          holo_green_dark</item>
    </style>
</resources>
```

5.5.8 Maintaining Compatibility

Material Design philosophy was introduced in Android 5.0 lollipop. Earlier versions of Android do not fully support certain feature essential to the material design such as RecyclerView and CardView widgets. In order to deal with the issues with Material widgets, the developer can use the v7 App Compat Support library. This ensures that the code is still functional in older version of Android. Alternative styles can be designed for older versions by separating the styles.xml for older versions and styles-v21.xml for devices supporting API level 21 and higher. The same philosophy can be used for layouts and values with layouts-v21 and values-v21.

The color scheme and the layout schemes are compatible with any version of Android. The Material theme may not be available to inherit from in this scenario. As a workaround, the developer can rewrite the entire theme, to ensure the consistency of the look of the application.

Check Your Progress

1. Which of the following adapter(s) are used for exposing data from a Cursor to a ListView object?

1.	CursorAdapter	2.	ResourceCursorAdapter
3.	SimpleCursorAdapter	4.	ArrayAdapter

(A)	1, 4	(B)	2, 4
(C)	1, 3	(D)	3, 4

2. Which of the following statement(s) stating the characteristics for an Adapter are true?

1.	It is an extendable class.
2.	It acts as a bridge between View and data source.
3.	It makes the view visible.
4.	It helps in displaying images.

(A)	1, 3, 4	(B)	1, 2
(C)	2, 4	(D)	2, 3

3. Which of the following is a timed dialog?

(A)	Alert Dialog	(B)	Toast
(C)	Progress Dialog	(D)	Date Picker Dialog

4. Lock Screen widget is introduced in which version of Android?

(A)	4.1	(B)	4.2
(C)	5.0	(D)	5.1

5. Match the following purposes with their corresponding View/Dialog.

	Purpose		View/Dialog
a.	Displays information for a limited amount of time	1.	CardView
b.	Shows the progress of an operation	2.	RecyclerView
c.	Material replacement for ListView	3.	Toasts
d.	Material replacement for FrameLayout	4.	ProgressDialog

(A)	a-3, b-4, c-2, d-1	(B)	a-3, b-1, c-2, d-4
(C)	a-1, b-4, c-3, d-2	(D)	a-1, b-3, c-4, d-2

Answers

1.	C
2.	B
3.	B
4.	B
5.	A

Summary

- Adapter is an object that acts as a bridge between the UI Components such as List view, Grid view, and so on and the underlying data sources.
- User Interface components act as an interface between the user and the application to input data and display the expected result.
- A dialog is displayed in front of an activity as a small window and can be a DatePickerDialog, TimePickerDialog, ProgressDialog, or AlertDialogs.
- Popups are smaller versions of dialogs which are designed in such a way so that users can make a selection to move forward or click outside the popup.
- Toast is a popup that displays feedback for an activity.
- Custom Widgets can be created by extending the AppWidgetProvider class. They can be placed on the home screen or the lock screen.
- Lock screen widget support was dropped in API level 22.
- Material Design is Google's new optional design philosophy for creating application UI.

BE AHEAD OF EVERYONE ELSE

READ ARTICLES

Onlinevarsity

06

Media Handling

Welcome to the Session, **Media Handling.**

This session introduces the different ways of handling media in Android. It also discusses the multimedia capabilities of mobile devices and explains the process of creating simple and basic applications rich in multimedia. It explains features in your mobile devices which most of the users may not be aware of.

In this Session, you will learn to:

- Explain the use of graphics in Android
- Explain animation
- Explain OpenGL and OpenGL rendering
- Explain playing of audio and video with Media Player
- Explain the process of capturing image and video using camera
- Explain the process of creating a live wallpaper

Session

06

Media Handling

6.1 Introduction

Multimedia capabilities of mobile devices play a significant role for consumers. Android's open source technology provides support for multimedia capabilities. It ensures that it offers multimedia API for playing and recording of wide range of image, audio and video formats. Camera API also exposes the capabilities to build applications with comprehensive multimedia functionality.

6.2 Graphics

In media handling, Graphics plays a very important role. Android provides basic level of graphics tools such as canvases, colors filters, points, and rectangles that allows the developer to handle drawing on the screen directly. Android provides different techniques for performing varying graphical tasks. The technique for graphic programming varies depending on graphical demand such as between a 2D and 3D, static and dynamic, and so on. However, all of them require the knowledge of the underlying graphics libraries.

The session will look at how to create the simplest form of 2D graphics. However, before doing that, a quick look is required on a few important concepts of the 2D graphics such as Views, Canvas, Paint, and onDraw() method. They are as follows:

- Views are known to be the visible elements in an Android UI.
- Each View is associated with the Canvas.
- When the View is displayed, its onDraw() method is automatically invoked by Android.
- The developer can be creating their own View with the onDraw() method to display basic objects using the Canvas.
- Canvas has methods for drawing Arcs, Bitmaps, Circles, Lines, Ovals, Paths, Rectangles, and so on.
- Paint has the method for setting the alpha, color, shade, stroke, and so on.

FunWithDrawing is a simple 2D graphic application. To develop this application, perform the following steps:

1. Create a new project named FunWithDrawing with the activity class name as FunWithDrawingActivity in Android Studio.
2. Navigate to src → com.example.funwithdrawing folder.
3. Create a custom View class named DrawableView by right-clicking the project's package folder to display the context menu and selecting New → Class. The class should extend from android.view.View and also implement the necessary constructors.

4. Modify the code in DrawableView class as shown in Code Snippet 1.

Code Snippet 1:

```
package com.example.funwithdrawing;
import android.content.Context;
import android.graphics.Canvas;
import android.graphics.Color;
import android.graphics.Paint;
import android.util.AttributeSet;
import android.view.MotionEvent;
import android.view.View;
public class DrawableView extends View {
public DrawableView(Context context) {
      super(context);
      // TODO Auto-generated constructor stub
}
public DrawableView(Context c, AttributeSet a) {
      super(c, a);
}
protected void onDraw(Canvas canvas) {
      // this is the "paintbrush"
      Paint = new Paint();
      // set the color randomly
      int whichColor = (int) (Math.random() * 4);
      if (whichColor == 0)
      paint.setColor(Color.RED);
      else if (whichColor == 1)
      paint.setColor(Color.GREEN);
      else if (whichColor == 2)
      paint.setColor(Color.BLUE);
```

```
        else
        paint.setColor(Color.YELLOW);
        // draw Rectangle with corners at (40, 20) and (90, 80)
        canvas.drawRect(40, 20, 90, 80, paint);
}
public boolean onTouchEvent(MotionEvent event) {
        // if it's an up ("release") action
        if (event.getAction() == MotionEvent.ACTION_UP) {
                // get the coordinates
                float x = event.getX();
                float y = event.getY();
                // see if they clicked the box
                if (x >= 40 && x <= 90 && y >= 20 && y <= 80) {
        // redraw the View... this calls onDraw again!
                        invalidate();
                }
        }
        // indicates that the event was handled
        return true;
} // end of onTouchEvent
}
```

In the code, the onDraw() method is used for drawing primitive shapes such as rectangle, oval, arc by invoking the drawRect(), drawOval(), and drawArc() method respectively. The method also declares an instance of Paint class for setting the paintbrush color using the setColor() method of the class. Thus, the Paint class specifies how to draw.

The onTouchEvent() method is invoked when the event takes place and inturn invokes the invalidate() method which is used for redrawing the screen.

5. Navigate to res → layout folder.

6. Modify the code in activity_fun_with_drawing.xml file as shown in Code Snippet 2.

Code Snippet 2:

```
<?xml version="1.0" encoding="utf-8"?>
<com.example.funwithdrawing.DrawableView
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent" />
```

7. Navigate to src → com.example.funwithdrawing folder.
8. Modify the code in FunWithDrawingActivity as shown in Code Snippet 3.

Code Snippet 3:

```
package com.example.funwithdrawing;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
public class FunWithDrawingActivity extends Activity {
@Override
protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_fun_with_drawing);
 }
@Override
public boolean onCreateOptionsMenu(Menu menu) {
// Inflate the menu; this adds items to the action bar if it //is
present.
      getMenuInflater().inflate(R.menu.fun_with_drawing, menu);
return true;
}
}
```

Figure 6.1 displays the output of the application.

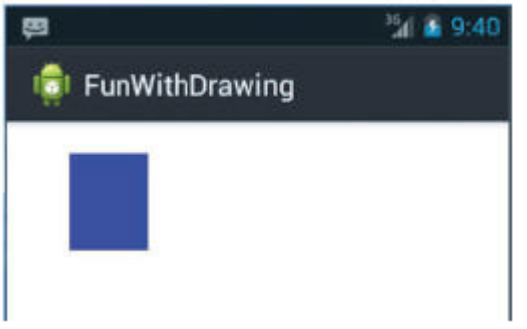


Figure 6.1: FunWithDrawing - Output

6.3 Animation

Animation is a very interesting concept which every learner is keen to know and implement. Therefore, developing animated application plays a very vital role when it comes to apps development in Android. Android provides a set of API for applying animation to UI controls and for drawing 2D and 3D graphics. Android 3.0 introduces the Properties Animation API which allows changing properties of an object over a specific time period.

Android framework provides two animation types namely, property animation and view animation. Property animation has been introduced in Android 3.0 version. Both animation systems are feasible, but the property animation system is the preferred method to use. This is because property animation is more flexible and offers more features. In addition to these two systems, the developer can use the traditional Drawable animation. The drawable animation allows the developer to load drawable resources and display them one frame after another. It is created with a series of images which are displayed in a sequence and played in an order.

The API allows you to define arbitrary object properties such as a start and end value and also apply a time-based change to this attribute. This API can also be applied on any Java object and not only on Views.

The super class of the animation API is the Animator class. The object of Animator class is used to modify the attributes of an object.

You can also add an AnimatorListener class to your Animator class. This listener is invoked in the different phases of the animation. You can use this listener to perform actions before or after a certain animation, for example, after adding or removing a View from a View Group.

6.3.1 Android Resources

Android resource defines two types of animations. They are as follows:

- **Property Animation**: Creates an animation by modifying an object's property values over a set period of time with an Animator. Introduced in Android 3.0 (API level 11), the property animation system lets you animate properties of any object, including ones that are not rendered to the screen. The system is extensible and lets you animate properties of custom types as well. To create a property animation, the following points are to be considered:
  - **Definition**: An animation defined in XML file that modifies properties of the target object, such as background colors or alpha value, over a set amount of time.
  - **File location**: File is located in res/animator folder. The file name will be used as a resource ID.
  - **Compiled Resource Data Type**: Resource pointer to a Value Animator, Object Animator, or an Object Set.
  - **Resource Reference**: Resource will be referenced in Java file as: R.animator. filename, in XML file as: @[package]animator/filename.
- **View Animation**: It is the older system and can be used with Views. It is easy to setup and offers enough capabilities to meet the varied needs of an application. There are two types of the animation that you can do with the view animation framework. They are as follows:
  1. **Tween Animation**: Creates an animation by performing a series of transformations on the contents of a View object. An animation defined in an XML file helps to perform transitions such as rotating, fading, moving and stretching on a View object. The single root element in the XML file can be either a single <alpha>, <scale>, <translate>, <rotate>, interpolator element, or <set> element that holds groups of these elements. The points to be considered for tween animations are as follows:
     - **File Location**: File is located in res/anim/folder. The filename will be used as a resource ID.
     - **Compiled Resource Data Type**: Resource pointer to an Animation.
     - **Resource Reference**: Resource will be referenced in Java file as: R.anim.filename and in XML file as: @[package]anim/filename.
  2. **Frame Animation**: Creates animation by showing a sequence of images in an order with a Drawable object. The points to be considered for frame animations are as follows:
     - **File Location**: File is located in res/drawable/ folder. The filename will be used as a resource ID.

- **Compiled Resource Data Type**: Resource pointer to an AnimationDrawable.
- **Resource Reference**: Resource will be referenced in Java file as: R.drawable. filename and in XML file as: @[package]drawable/filename.
- **Drawable Animation**: This involves displaying Drawable resources one after the other, like a roll of film. This method of animation is useful if you want to animate things that are easier to represent with Drawable resources, such as a progression of bitmaps.

Following application named AnimationExample is a simple animation application. To create this application, perform the following steps:

1. Create a new project named AnimationExample with the activity class name as FirstActivity.
2. Navigate to res → layout folder.
3. Modify the code in activity_first.xml file as shown in Code Snippet 4.

Code Snippet 4:

```
<RelativeLayout xmlns:android="http://schemas.android.com/
apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:paddingBottom="@dimen/activity_vertical_margin"
android:paddingLeft="@dimen/activity_horizontal_margin"
android:paddingRight="@dimen/activity_horizontal_margin"
android:paddingTop="@dimen/activity_vertical_margin"
tools:context=".FirstActivity" >
<Button
      android:id="@+id/TheFirstButton"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_alignParentLeft="true"
      android:layout_alignParentTop="true"
```

```
      android:layout_marginTop="47dp"
      android:text="Animation1" />
</RelativeLayout>
```

4. Navigate to src → com.example.animationexample folder.
5. Modify the code in FirstActivity file as shown in Code Snippet 5.

Code Snippet 5:

```
package com.example.animationexample;
import android.os.Bundle;
import android.app.Activity;
import android.content.Intent;
import android.view.Menu;
import android.view.View;
import android.widget.Button;
import android.view.View.OnClickListener;
public class FirstActivity extends Activity {
Button firstbutton;
@Override
protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_first);
      firstbutton = (Button) findViewById(R.id.TheFirstButton);
      firstbutton.setOnClickListener(new OnClickListener() {
      public void onClick(View v) {
            Intent intent = new Intent(getApplicationContext(),
            SecondActivity.class);
            startActivity(intent);
            overridePendingTransition(R.anim.anim_in, R.ani m.hold);
            }
      });
```

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
      // Inflate the menu; this adds items to the action bar if
it is present.
      getMenuInflater().inflate(R.menu.first, menu);
      return true;
}
}
```

6. Navigate to res → layout folder.
7. Create a New Android XML file named second.xml.
8. Modify the code in second.xml file as shown in Code Snippet 6.

Code Snippet 6:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/ apk/
res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical" >
<Button
      android:id="@+id/TheSecondButton"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_alignParentRight="true"
      android:layout_alignParentTop="true"
      android:layout_marginRight="45dp"
      android:layout_marginTop="97dp"
      android:text="Back" /></RelativeLayout>
```

9. Navigate to res folder.
10. Right-click and select New → Directory to create a new directory named anim.
11. Create a New Android XML file under res/anim named anim_in.xml.

12. Modify the code in anim_in.xml file as shown in Code Snippet 7.

Code Snippet 7:

```
<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/ android" >
<translate
      android:duration="500"
      android:fromXDelta="100%p"
      android:fromYDelta="0%p"
      android:toXDelta="0%p"
      android:toYDelta="0" /></set>
```

13. Create another New Android XML file under res/anim folder named hold.xml.
14. Modify the code in hold.xml file as shown in Code Snippet 8.

Code Snippet 8:

```
<?xml version="1.0" encoding="utf-8"?>
<set>
<translate
      xmlns:android="http://schemas.android.com/apk/res/ android"
      android:duration="500"
      android:fromXDelta="0"
      android:interpolator="@android:anim/accelerate_ interpolator"
      android:toXDelta="0" />
</set>
```

15. Navigate to src → com.example.animationexample folder.
16. Create a new class named SecondActivity.

17. Modify the code in SecondActivity file as shown in Code Snippet 9.

Code Snippet 9:

```
package com.example.animationexample;
import android.app.Activity;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.*;
public class SecondActivity extends Activity {
Button secondbutton;
@Override
protected void onCreate(Bundle savedInstanceState) {
      // TODO Auto-generated method stub
      super.onCreate(savedInstanceState);
            setContentView(R.layout.second);
      secondbutton = (Button) findViewById(R.
                  id.TheSecondButton);
      secondbutton.setOnClickListener(newOnClickListener(){
            public void onClick(View v) {
                  // TODO Auto-generated method stub
                  Intent intent = new
Intent(getApplicationContext(),
                  FirstActivity.class);
                  startActivity(intent);
                  overridePendingTransition(R.anim.anim_in,
                  R.anim. hold);
            }
      });
}
}
```

18. Add the <activity> tag in AndroidManifest.xml file as shown in Code Snippet 10 before the closing </application> tag.

Code Snippet 10:

```
…
<activity android:name=".SecondActivity"></activity>
…
```

Figure 6.2 displays the output after execution of the application.

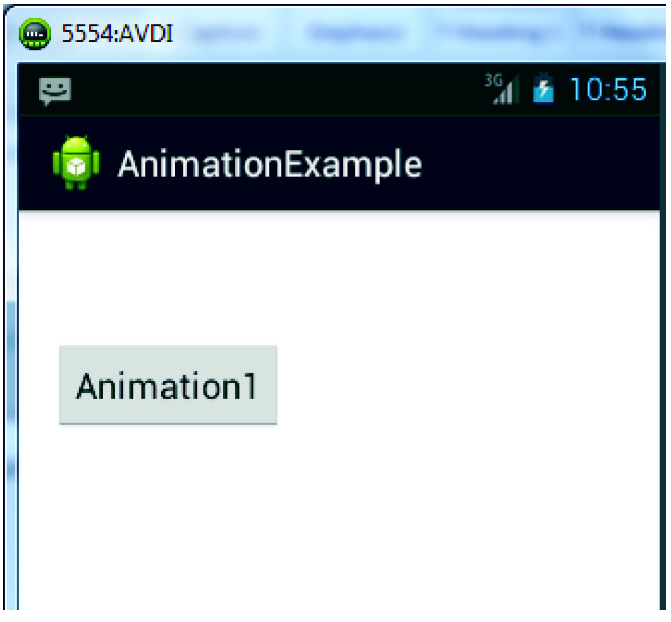


Figure 6.2: Animation Example

On clicking the button the title animates.

6.4 OpenGL ES

OpenGL is a cross platform 2D and 3D graphics rendering library. Android supports the OpenGL ES standard (Open GL for Embedded Systems). The OpenGL ES library is used to develop multimedia applications and games for the Android platform. This section will explain the process to create 2D Open GL objects and how to render them onto the screen using a GLSurfaceView.

6.4.1 History and Support

Table 6.1 shows the support for OpenGL ES version by different versions of Android.

OpenGL Version	Android Version
OpenGL ES 1.0	Android 1.0
OpenGL ES 2.0	Android 2.2
OpenGL ES 3.0	Android 4.3
OpenGL ES	Android 5.0

Table 6.1: OpenGL Support in Android

6.4.2 OpenGL Co-ordinate System

The basics of drawing on an OpenGL Surface are similar to those explained earlier. OpenGL assumes the center of the co-ordinate system to be at the center of the screen. The co-ordinates themselves are mentioned in fractions varying from 0 to 1. The extreme end of the screen is specified by the value of 1f and the center by 0f. A point half way in between is specified by 0.5f.

However, OpenGL does not account for varying screen sizes and varying screen aspect ratio. It always assumes the canvas to be a square and as a result, on rectangular displays, the canvas is 'stretched', as shown in figure 6.3.

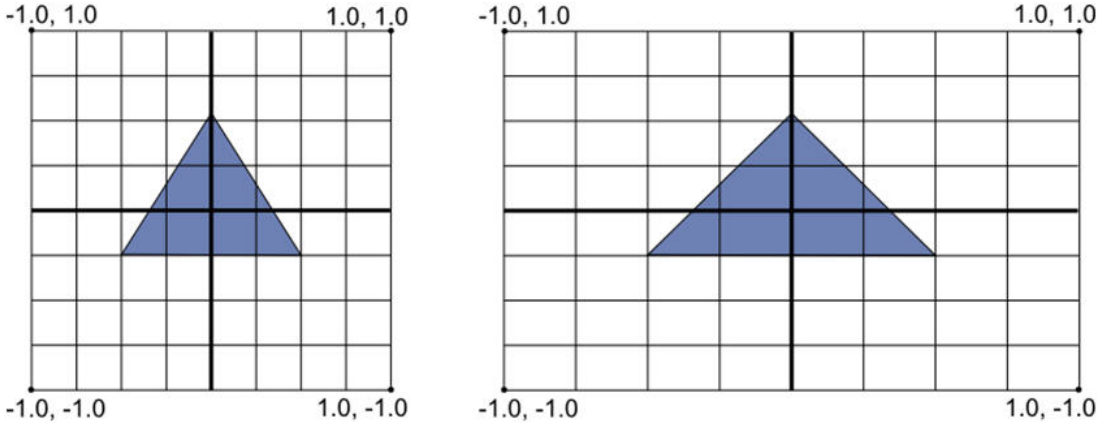


Figure 6.3: OpenGL Co-ordinate System

In order to counter this behavior, the concepts of projection views and camera views are used.

6.4.3 Views

Views allow the object to be 'perceived' in a different way from what they are represented in co-ordinates. The projection and camera views are described here.

- **Projection View**

  A Projection View allows the object to be 'projected' onto the screen according to the size and the aspect ratio of the screen. It is responsible for transforming the co-ordinates from a square system to the rectangular varying screen sizes. Programmatically, we use a projection matrix which contains the information to map the co-ordinates appropriately.

- **Camera View**

  The camera view is where the object appears to be perceived from. Every OpenGL application has a camera point which is used to observe the objects. In order to move the object on the screen, we can move the camera away from the object to make it appear as if the object has moved. This concept is central to transformation. Transformations are of two types:

  - **Translation**: Moving an object from one point to another.
  - **Rotation**: Rotating the object at a specified angle.

  Programmatically, we use a camera transformation matrix for this purpose.

6.4.4 Shaders and Shader Code

A shader is a user defined code that runs at some stage in the rendering process. In OpenGL ES, the shader code is specified using a string literal as shown in Code Snippet 11.

Code Snippet 11:

```
private final String vertexShaderCode =
        "uniform mat4 uMVPMatrix;" +
                "attribute vec4 vPosition;" +
                "void main() {" +
                "  gl_Position = uMVPMatrix * vPosition;" +
                "}";
```

6.4.5 Vertices and Draw Order

In OpenGL, vertices are not drawn simultaneously. The programmer is responsible for specifying the order in which the vertices are drawn. For example, in order to draw a square it is required to draw two triangles across the diagonal. That is, if the square has four vertices namely, A, B, C, and D, then the draw order would be specified as:

A → B → C → A → C → D

Programmatically it is done using a draw order matrix as shown in Code Snippet 12.

Code Snippet 12:

```
private final short drawOrder[] = { 0, 1, 2, 0, 2, 3 };
```

6.4.6 OpenGL Colors

Colors in OpenGL are represented as a matrix of four units: Red value, Green value, Blue value, and Alpha value. The Alpha value specifies the transparency of the color. All the values are specified in a range of 0f to 1f.

- **Converting Colors**:

  For the following example, the color, holo_red_light will be used. The process of converting the color is described here.

  The Hex value of the color is FF4444

  The R value is 255

  The OpenGL R value is 255/255 = 1f

  The G value is 68

  The OpenGL G value is 68/255 = 0.27f

  The B value is 68

  The Open B value is 68/255 = 0.27f

  The Alpha value will be used as no transparency

  Therefore, Alpha value = 255/255 = 1f

  The OpenGL color matrix would be: (1.0f, 0.27f, 0.27f, 1.0f)

6.4.7 Steps to Render an OpenGL Object

- **Creating a View**

  In order to render the OpenGL objects, a view needs to be created. For this purpose, extend the class GLSurfaceView from the package android.opengl.GLSurfaceView.

The view itself does not render the objects. The purpose of a view is only to display the rendered objects. In order to render them, a renderer is required.

- **Creating a Renderer**

In order to implement a renderer, extend the class GLSurfaceview.Renderer.

Following methods need to be implemented:

- onSurfaceCreated()
- onSurfaceChanged()
- onDrawFrame()

The onSurfaceCreated() method is called only once after the surface is ready. The onSurfaceChanged() method is called whenever the orientation of the device changes and the onDrawFrame() method is called once per frame. This is where the object render code is placed.

- **Creating Objects**

Objects can be rendered directly on the onDrawFrame() method. However, this process is not recommended as it makes the Renderer less flexible and modification of code can become extremely cumbersome. Hence, we create OpenGL objects. A valid OpenGL object consists of three things:

- Shader code
- Draw order
- Draw method

- **Tying the Code Together**

Following are the steps to display an OpenGL object on screen:

1. Create a new Activity with an implementation of GLSurfaceView.
2. Create a new Renderer for the GLSurfaceView.
3. Set the renderer to the view by using the method setRenderer().
4. Create a new OpenGL object in the renderer.
5. In the onDrawFrame() method, call the draw method on the objects to be displayed.

6.4.8 An OpenGL Example

In this section, the process of rendering and OpenGL object on screen will be explained.

1. Create a new Project called OpenGLRendering in Android Studio with a blank activity.
2. Navigate to app → java → aptech.com.openglrendering and open MainActivity.java for editing.

3. Modify the code as shown in Code Snippet 13.

Code Snippet 13

```
package aptech.com.openglrendering;
import android.app.Activity;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
public class MainActivity extends Activity {
    @Override
protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        CustomGLSurfaceView view = new CustomGLSurfaceView(this);
        setContentView(view);
    }
   @Override
    public boolean onCreateOptionsMenu(Menu menu) {
   // Inflate the menu; this adds items to the action bar if it
//is present.
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
    // Handle action bar item clicks here. The action bar will
    // automatically handle clicks on the Home/Up button, so long
    // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        //noinspection SimplifiableIfStatements
```

```
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
}
```

4. Create a new class called CustomGLSurfaceView and modify the code as shown in Code Snippet 14.

Code Snippet 14:

```
package aptech.com.openglrendering;
import android.content.Context;
import android.opengl.GLSurfaceView;
public class CustomGLSurfaceView extends GLSurfaceView {
        private final CustomGLRender mRenderer;
        public CustomGLSurfaceView(Context context){
            super(context);
            setEGLContextClientVersion(2);
           mRenderer = new CustomGLRender();
            setRenderer(mRenderer);
        }
}
```

5. Create a new class called CustomGLRender and modify the code as shown in Code Snippet 15.

Code Snippet 15:

```
package aptech.com.openglrendering;
import android.opengl.GLES20;
import android.opengl.GLSurfaceView;
import android.opengl.Matrix;
```

```
import javax.microedition.khronos.egl.EGLConfig;
import javax.microedition.khronos.opengles.GL10;
public class CustomGLRender implements GLSurfaceView.Renderer  {
    private Square square;
    private final float[] MVPMatrix = new float[16];
    private final float[] projectionMatrix = new float[16];
    private final float[] viewMatrix = new float[16];
    public CustomGLRender(){
        square = new Square();
    }
    @Override
    public void onSurfaceCreated(GL10 gl, EGLConfig config) {
        GLES20.glClearColor(1.0f, 0.27f, 0.27f, 1.0f);
        square = new Square();
    }
   @Override
   public void onSurfaceChanged(GL10 gl, int width, int height) {
        GLES20.glViewport(0, 0, width, height);
        float ratio = (float) width / height;
        Matrix.frustumM(projectionMatrix, 0, -ratio, ratio, -1,
        1, 3, 7);
    }
    @Override
    public void onDrawFrame(GL10 gl) {
     GLES20.glClear(GLES20.GL_COLOR_BUFFER_BIT | GLES20.GL_
     DEPTH_BUFFER_BIT);
     Matrix.setLookAtM(viewMatrix, 0, 0, 0, -3, 0f, 0f, 0f, 0f,
     1.0f, 0.0f);
```

```
     Matrix.multiplyMM(MVPMatrix, 0, projectionMatrix, 0,
     viewMatrix, 0);
    square.draw(MVPMatrix);
 }
    public static int loadShader(int type, String shaderCode){
        int shader = GLES20.glCreateShader(type);
        GLES20.glShaderSource(shader, shaderCode);
        GLES20.glCompileShader(shader);
        return shader;
    }
}
```

6. Create a new class called square and modify the code as shown in Code Snippet 16.

Code Snippet 16:

```
package aptech.com.openglrendering;
import java.nio.ByteBuffer;
import java.nio.ByteOrder;
import java.nio.FloatBuffer;
import java.nio.ShortBuffer;
import android.opengl.GLES20;
public class Square {
    private final String vertexShaderCode =
            "uniform mat4 uMVPMatrix;" +
                    "attribute vec4 vPosition;" +
                    "void main() {" +
                    "  gl_Position = uMVPMatrix * vPosition;" +
                    "}";
    private final String fragmentShaderCode =
            "precision mediump float;" +
```

```
                    "uniform vec4 vColor;" +
                    "void main() {" +
                    "  gl_FragColor = vColor;" +
                    "}";
    private final FloatBuffer vertexBuffer;
    private final ShortBuffer drawListBuffer;
    private final int program;
    private int positionHandle;
    private int colorHandle;
    private int MVPMatrixHandle;
    static final int COORDS_PER_VERTEX = 3;
    static float vertices[] = {
            -0.5f,  0.5f, 0.0f,
            -0.5f, -0.5f, 0.0f,
            0.5f, -0.5f, 0.0f,
            0.5f,  0.5f, 0.0f };
    private final short drawOrder[] = { 0, 1, 2, 0, 2, 3 };
    private final int vertexStride = COORDS_PER_VERTEX * 4;
    float color[] = { 1.0f, 1.0f, 0.27f, 1.0f };
    public Square() {
ByteBuffer bb = ByteBuffer.allocateDirect(vertices.length * 4);
        bb.order(ByteOrder.nativeOrder());
        vertexBuffer = bb.asFloatBuffer();
        vertexBuffer.put(vertices);
        vertexBuffer.position(0);
ByteBuffer dlb = ByteBuffer.allocateDirect(drawOrder.length * 2);
        dlb.order(ByteOrder.nativeOrder());
        drawListBuffer = dlb.asShortBuffer();
```

```
        drawListBuffer.put(drawOrder);
        drawListBuffer.position(0);
        int vertexShader = CustomGLRender.loadShader(GLES20.GL_
                           VERTEX_SHADER,vertexShaderCode);
        int fragmentShader = CustomGLRender.loadShader(GLES20.
              GL_FRAGMENT_SHADER,fragmentShaderCode)
        program = GLES20.glCreateProgram();
        GLES20.glAttachShader(program, vertexShader);
        GLES20.glAttachShader(program, fragmentShader);
        GLES20.glLinkProgram(program);
    }
public void draw(float[] mvpMatrix) {
        GLES20.glUseProgram(program);
        positionHandle = GLES20.glGetAttribLocation(program,
                         "vPosition");
        GLES20.glEnableVertexAttribArray(positionHandle);
        GLES20.glVertexAttribPointer(positionHandle, COORDS_PER_
VERTEX,GLES20.GL_FLOAT, false,vertexStride, vertexBuffer);
        colorHandle = GLES20.glGetUniformLocation(program, "vColor");
        GLES20.glUniform4fv(colorHandle, 1, color, 0);
        MVPMatrixHandle = GLES20.glGetUniformLocation(program,
                          "uMVPMatrix");
        GLES20.glUniformMatrix4fv(MVPMatrixHandle, 1, false,
        mvpMatrix, 0);
        GLES20.glDrawElements(GLES20.GL_TRIANGLES,
     drawOrder.length,GLES20.GL_UNSIGNED_SHORT, drawListBuffer);
        GLES20.glDisableVertexAttribArray(positionHandle);
    }
}
```

7. Navigate to res → values → styles.xml and modify the code as shown in Code Snippet 17.

Code Snippet 17:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="AppTheme" parent="android:Theme.Material">
        <item name="android:colorPrimary">@android:color/holo_
blue_light</item>
        <item name="android:colorPrimaryDark">@android:color/
holo_blue_dark </item>
        <item name="android:colorAccent">@android:color/holo_
red_light</item>
        <item name="android:textColorPrimary">@android:color/
white</item>
    </style>
</resources>
```

8. Navigate to Manifests and open AndroidManifest.xml for editing.

9. Modify the code as shown in Code Snippet 18.

Code Snippet 18:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/
android"
    package="aptech.com.openglrendering" >
    <uses-feature android:glEsVersion="0x00020000"
    android:required="true" />
    <application
      android:allowBackup="true"
      android:icon="@mipmap/ic_launcher"
      android:label="@string/app_name"
      android:theme="@style/AppTheme" >
```

```
      <activity
        android:name=".MainActivity"
        android:label="@string/app_name" >
        <intent-filter>
        <action android:name="android.intent.action.  MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

The uses-feature tag ensures that the application is only installed on devices that support OpenGL ES version 2.0 or higher.

10. Navigate to Run menu and select Run 'app'.

The application is deployed on the AVD and the output is displayed as shown in figure 6.4.

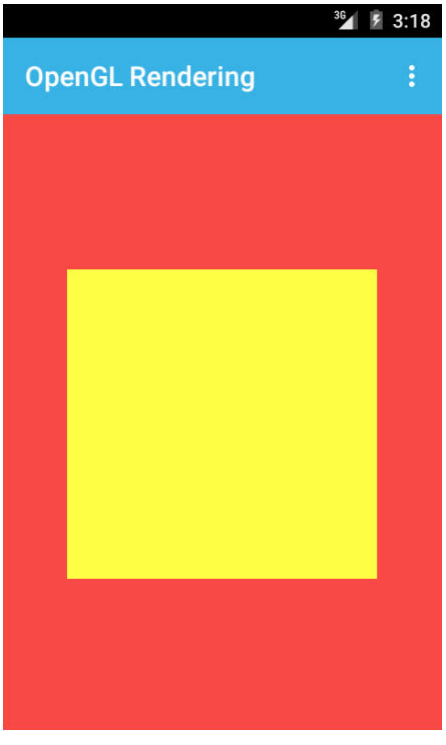


Figure 6.4: OpenGL Rendering Application

6.5 Audio/Video

Android uses OpenCore as its core component of media framework. OpenCore supports different file formats such as MP3, AAC, AAC+, 3GPP, MPEG-4, JPEG. Following classes are used to play audio and video in Android framework:

- **MediaPlayer**: This is responsible for playing audio and video and is the primary class.
- **AudioManager**: This is responsible for managing audio source and audio output on the device.

6.5.1 Media Player Basics

The Android multimedia framework includes support for playing variety of common media types, so that one can easily integrate audio, video, and images into your applications. One can play audio or video from media files stored in your application's resources (raw resources), from standalone files in the filesystem, or from a data stream arriving over a network connection. All of these use MediaPlayer APIs. MediaPlayer class is the most important component of media framework. Various sources from which media can be played are as follows:

- Resource folder
- File System path
- URL

The different states of a media player are described as follows:

- Creation and initialization of the media player with the media to play using the method new() and setDataSource().
- Preparation of the media player for playback using the method prepare().
- Start of the playback using the method start().
- Pause or Stop of the playback using the pause() or stop() method.
- Termination of the playback using stop() method.

In the AndroidManifest.xml file the appropriate permission should be provided based on use of related features. The permission that can be granted is as follows:

- **Internet Permission:** This permission is required when the media player is streaming content from network. The application should have the following permission as shown in Code Snippet 19.

Code Snippet 19:

```
<uses-permission android:name="android.permission.INTERNET" />
```

- **Wake Lock Permission:** This permission is provided when the developer does not want the screen to dim or the processor to sleep. The application should have the following permission as shown in Code Snippet 20.

Code Snippet 20:

```
<uses-permission android:name="android.permission.WAKE_LOCK" />
```

6.5.2 Playing an Audio Clip

Following application named MediaDemo plays audio files. To develop the application, perform the following steps:

1. Create a new project named MediaDemo.
2. Navigate to res folder.
3. Create a new folder under res folder named 'raw'.
4. Copy all the audio files under res/raw folder.
5. Navigate to the res → layout folder.
6. Modify the code in activity_main.xml file as shown in Code Snippet 21.

Code Snippet 21:

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/ res/
android"
android:layout_width="fill_parent"
android:layout_height="fill_parent"
android:orientation="vertical" >
<Button
      android:id="@+id/startPlayerBtn"
      android:layout_width="fill_parent"
      android:layout_height="wrap_content"
      android:onClick="doClick"
      android:text="Start Player" />
<Button
      android:id="@+id/pausePlayerBtn"
```

```
      android:layout_width="fill_parent"
      android:layout_height="wrap_content"
      android:onClick="doClick"
      android:text="Pause Player" />
<Button
      android:id="@+id/restartPlayerBtn"
      android:layout_width="fill_parent"
      android:layout_height="wrap_content"
      android:onClick="doClick"
      android:text="Restart Player" />
<Button
      android:id="@+id/stopPlayerBtn"
      android:layout_width="fill_parent"
      android:layout_height="wrap_content"
      android:onClick="doClick"
      android:text="Stop Player" /></LinearLayout>
```

7. Navigate to src → com.example.mediademo folder.
8. Modify the source code in MainActivity file as shown in Code Snippet 22.

Code Snippet 22:

```
package com.example.mediademo;
import android.media.AudioManager;
import android.media.MediaPlayer;
import android.os.Bundle;
import android.os.PowerManager;
import android.app.Activity;
import android.content.res.AssetFileDescriptor;
import android.util.Log;
import android.view.Menu;
```

```
import android.view.View;
import android.media.MediaPlayer.OnPreparedListener;
public class MainActivity extends Activity implements
OnPreparedListener {
private static final String TAG = null;
private MediaPlayer;
private int playbackPosition = 0;
@Override
protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_main);
}
public void doClick(View view) {
      switch (view.getId()) {
            case R.id.startPlayerBtn:
            try {
                  // Only have one of these play methods uncommented
                  // playAudio(AUDIO_PATH);
                  playLocalAudio();
                  // playLocalAudio_UsingDescriptor();
            } catch (Exception e) {
                  e.printStackTrace();
            }
            break;
            case R.id.pausePlayerBtn:
            if(mediaPlayer!= null&& mediaPlayer.isPlaying()){
                  playbackPosition = mediaPlayer.
                                    getCurrentPosition();
                  mediaPlayer.pause();
```

```
            }
            break;
            case R.id.restartPlayerBtn:
            if(mediaPlayer!= null&&!mediaPlayer.isPlaying()){
                  mediaPlayer.seekTo(playbackPosition);
                  mediaPlayer.start();
            }
            break;
            case R.id.stopPlayerBtn:
            if (mediaPlayer != null) {
                  mediaPlayer.stop();
                  playbackPosition = 0;
             }
            break;
      }
}
private void playAudio(String url) throws Exception {
      killMediaPlayer();
      mediaPlayer = new MediaPlayer();
      mediaPlayer.setAudioStreamType(AudioManager.STREAM_ MUSIC);
      mediaPlayer.setDataSource(url);
      mediaPlayer.setOnPreparedListener(this);
      mediaPlayer.setWakeMode(this, PowerManager.PARTIAL_ WAKE_LOCK);
      mediaPlayer.prepareAsync();
}
private void playLocalAudio() throws Exception {
      // add the name of the song to be played in the raw folder
```

```
      mediaPlayer = MediaPlayer.create(this, R.raw.geetgatahoonmein);
     mediaPlayer.setAudioStreamType(AudioManager.STREAM_ MUSIC);
      mediaPlayer.start();
}
private void playLocalAudio_UsingDescriptor() throws Exception {
      openRawResourceFd(
      R.raw.flyaway);
      if (fileDesc != null) {
            mediaPlayer = new MediaPlayer();
            mediaPlayer.setAudioStreamType(AudioManager. STREAM_MUSIC);
            mediaPlayer.setDataSource(fileDesc.getFileDescriptor(),
fileDesc.getStartOffset(), fileDesc. getLength());
            fileDesc.close();
            mediaPlayer.prepare();
            mediaPlayer.start();
      }
}
@Override
protected void onDestroy() {
      super.onDestroy();
      killMediaPlayer();
}
private void killMediaPlayer() {
      if (mediaPlayer != null) {
      try {
            mediaPlayer.release();
```

```
        } catch (Exception e) {
              e.printStackTrace();
        }
        }
}
public void onPrepared(MediaPlayer mp) {
        Log.d(TAG, "MediaPlayer is prepared");
        mp.start();
}
@Override
public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if
it is present.
        getMenuInflater().inflate(R.menu.main, menu);
        return true;
}
}
```

9. Modify the code in AndroidManifest.xml file as shown in Code Snippet 23 for setting the permission.

Code Snippet 23:

```
<uses-permission android:name="android.permission.WAKE_LOCK"></
uses-permission>
```

Figure 6.5 displays the output when the code is executed in the emulator.

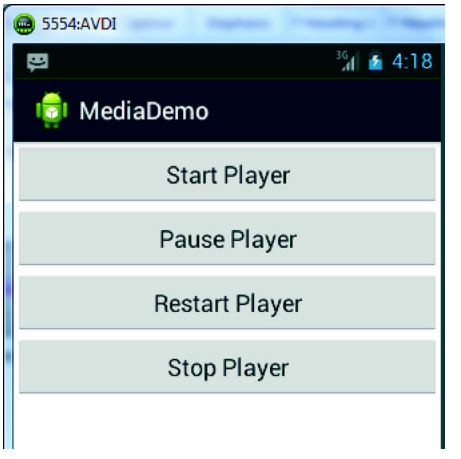


Figure 6.5: MediaDemo – Output

6.5.3 Playing a Video

To play a video, an application named VideoDemo is created. The steps to create the demo are as follows:

1. Create a project named ViewDemoApp.
2. Navigate to res → layout folder.
3. Modify the code in activity_main.xml file as shown in Code Snippet 24.

Code Snippet 24:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/ res/
android"
android:layout_width="fill_parent"
android:layout_height="fill_parent"
android:orientation="vertical" >
<LinearLayout
      android:layout_width="fill_parent"
      android:layout_height="wrap_content"
      android:orientation="vertical" >
<VideoView
            android:id="@+id/myvideoview"
            android:layout_width="fill_parent"
            android:layout_height="fill_parent" />
</LinearLayout>
</LinearLayout>
```

4. Navigate to src → com.example.viewdemoapp folder.

5. Modify the code in MainActivity file as shown in Code Snippet 25.

Code Snippet 25:

```
package com.example.viewdemoapp;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.widget.MediaController;
import android.widget.VideoView;
public class MainActivity extends Activity {
stringSrcPath= "/sdcard/thebestlineincartoonhistory.mp4";
@Override
protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
            setContentView(R.layout.activity_main);
            VideoView myVideoView = (VideoView) findViewById(R.
                                    id.myvideoview);
      myVideoView.setVideoPath(SrcPath);
      myVideoView.setMediaController(new MediaController(this));
      myVideoView.requestFocus();
      myVideoView.start();
}
@Override
public boolean onCreateOptionsMenu(Menu menu) {
      // Inflate the menu; this adds items to the action bar if it
is present.
getMenuInflater().inflate(R.menu.main, menu);
      return true;
}
}
```

Before executing the application one needs to place the video clip in the sdcard. This can be done by performing the following steps:

- Start DDMS by navigating to Tools → Android and select Android Device Monitor.
- Once in DDMS perspective, navigate to the File Explorer tab as shown in figure 6.6.
- Place the video clip in the sdcard by navigating to the top right corner in the DDMS perspective and then, click the icon 'Push a file onto device'.
- Navigate to the desired location of the video clip and click Open to place the video clip in the sdcard.

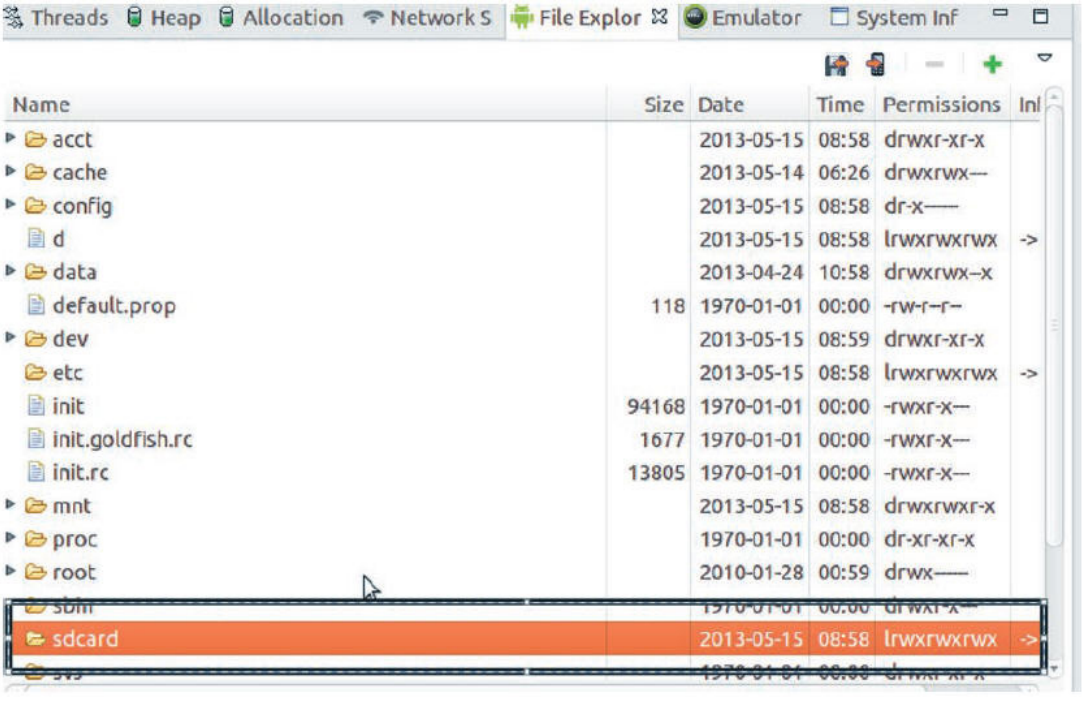


Figure 6.6: DDMS Perspective

Figure 6.7 displays the output once the application is executed.

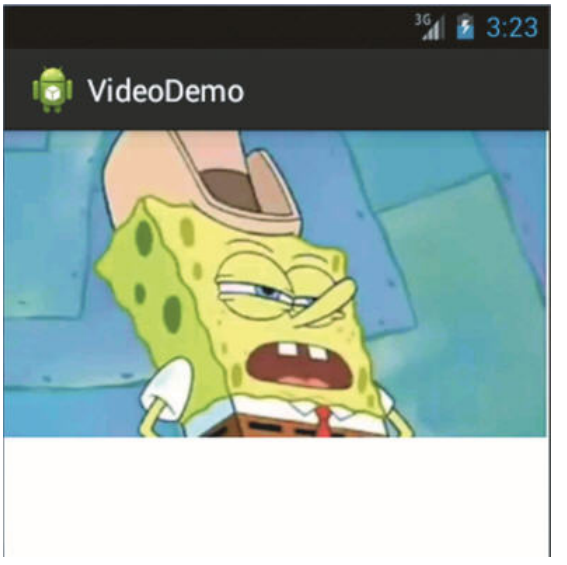


Figure 6.7: VideoDemo App

6.5.4 MediaStore API

The earlier example demonstrates the basics of opening a single file, whose path is hardcoded into the application. However, in real-time applications, it is important to discover and list the media available on the device. Android provides the MediaScanner Service which searches and maintains a record of all the media available on the device. This section describes the use of Media Store API to create a full-fledged Audio Player. The application can be extended to support Video as well.

MediaStore is a content provider and uses the same API. In order to generate a query, the developer first needs a reference to a content resolver.

The information to retrieve is set using a projection string and the conditions are set using a selection string. The query is then executed on the content resolver using the query() method. A cursor to the information is returned. The code to query all the video files is shown in Code Snippet 26.

Code Snippet 26:

```
Cursor videoCursor = getContentResolver().query(

MediaStore.Video.Media.EXTERNAL_CONTENT_URI,projection,
selection, null, null);
```

Following are the steps to create a media player using the MediaStore API:

1. Create a new project called AudioVideoPlayback in Android Studio.

2. Navigate to res → layout and open main_activity.xml for editing.
3. Modify the code as shown in Code Snippet 27.

Code Snippet 27:

```
<<RelativeLayout

xmlns:android=”http://schemas.android.com/apk/res/android”

    xmlns:tools=”http://schemas.android.com/tools” android:layout_
width=”match_parent”

    android:layout_height=”match_parent”

android:paddingLeft=”@dimen/activity_horizontal_margin”

    android:paddingRight=”@dimen/activity_horizontal_margin”

    android:paddingTop=”@dimen/activity_vertical_margin”

    android:paddingBottom=”@dimen/activity_vertical_margin”
tools:context=”.MainActivity”>

    <ListView

        android:layout_width=”wrap_content”

        android:layout_height=”wrap_content”

        android:id=”@+id/listView”

        android:layout_alignParentTop=”true”

        android:layout_alignParentStart=”true” />

</RelativeLayout>
```

4. Right-click layout and select New → Layout resource file.
5. Create a new resource called media_list_item.xml and modify the code as shown in Code Snippet 28.

Code Snippet 28:

```
<?xml version=”1.0” encoding=”utf-8”?>
<LinearLayout
xmlns:android=”http://schemas.android.com/apk/res/android”
    android:layout_width=”match_parent” android:layout_
height=”match_parent”
    android:orientation=”horizontal”>
    <ImageView
        android:layout_width=”wrap_content”
        android:layout_height=”match_parent”
        android:id=”@+id/imageView”
        android:layout_weight=”1”
        android:src=”@mipmap/ic_music_icon”
        android:nestedScrollingEnabled=”false” />
    <LinearLayout
        android:orientation=”vertical”
        android:layout_width=”wrap_content”
        android:layout_height=”match_parent”
        android:layout_weight=”2”>
<TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Song Name”
            android:id=”@+id/songNameView”
            android:textColor=”@android:color/holo_red_light”
            android:textStyle=”bold”
            android:textSize=”@dimen/song_name_size” />
```

```
<TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Artist”
            android:id=”@+id/artistView”
            android:textSize=”15dp” />
    </LinearLayout>
    <LinearLayout
        android:orientation=”vertical”
        android:layout_width=”wrap_content”
        android:layout_height=”match_parent”
        android:layout_weight=”1”>
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Length”
            android:id=”@+id/lengthView”
            android:textAlignment=”viewEnd”
            android:textColor=”@android:color/holo_red_light”
            android:textSize=”15dp”
            android:paddingTop=”5dp”
            android:paddingLeft=”10dp” />
    </LinearLayout>
</LinearLayout>
```

This layout will serve as the item layout for the media list.

6. Right-click the app and Navigate to New → Activity and select Blank Activity.
7. Create an activity named AudioPlayerActivity.
8. Navigate to res → layout and open activity_audio_player.xml for editing.

9. Modify the contents as shown in Code Snippet 29.

Code Snippet 29:

```
<LinearLayout
xmlns:android=”http://schemas.android.com/apk/res/android”
    xmlns:tools=”http://schemas.android.com/tools”
android:layout_width=”match_parent”
android:layout_height=”match_parent” android:paddingLeft=”@
dimen/activity_horizontal_margin”
    android:paddingRight=”@dimen/activity_horizontal_margin”
    android:paddingTop=”@dimen/activity_vertical_margin”
    android:paddingBottom=”@dimen/activity_vertical_margin”
    tools:context=”aptech.com.audiovideoplayback.
                AudioPlayerActivity”
    android:orientation=”vertical”>
    <ImageView
        android:layout_width=”match_parent”
        android:layout_height=”wrap_content”
        android:id=”@+id/imageView2”
        android:layout_weight=”3”
        android:padding=”5dp”
        android:src=”@drawable/music”
        android:textAlignment=”center”
        android:contentDescription=”@string/albumart_label” />
        <SeekBar
        android:layout_width=”match_parent”
        android:layout_height=”wrap_content”
        android:id=”@+id/seekBar”
        android:layout_weight=”0” />
        <LinearLayout
        android:orientation=”horizontal”
```

```
android:layout_width=”match_parent”
android:layout_height=”wrap_content”
>
<Button
    style=”?android:attr/buttonStyleSmall”
    android:layout_width=”wrap_content”
    android:layout_height=”wrap_content”
    android:text=”Rewind”
    android:id=”@+id/button”
    android:onClick=”onRewindClicked”
    android:layout_weight=”1”
    android:background=”@android:color/holo_blue_light”
    android:textColor=”@android:color/white” />
<Button
    style=”?android:attr/buttonStyleSmall”
    android:layout_width=”wrap_content”
    android:layout_height=”wrap_content”
    android:text=”PLAY”
    android:id=”@+id/button2”
    android:onClick=”onPlayClicked”
    android:layout_weight=”2”
    android:textStyle=”bold”
    android:background=”@android:color/holo_blue_dark”
    android:textColor=”@android:color/white” />
<Button
    style=”?android:attr/buttonStyleSmall”
    android:layout_width=”wrap_content”
    android:layout_height=”wrap_content”
    android:text=”Forward”
```

```
            android:id="@+id/button3"
            android:onClick="onForwardClicked"
            android:layout_weight="1"
            android:background="@android:color/holo_blue_light"
            android:textColor="@android:color/white" />
    </LinearLayout>
</LinearLayout>
```

10. Navigate to java → aptech.com.audiovideoplayback and open MainAcitivty.java for editing. Modify the code as shown in Code Snippet 30.

Code Snippet 30:

```
package aptech.com.audiovideoplayback;
import android.app.Activity;
import android.content.Intent;
import android.database.Cursor;
import android.os.Bundle;
import android.os.Environment;
import android.provider.MediaStore;
import android.util.Log;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.AdapterView;
import android.widget.ListView;
import android.widget.TextView;
import android.widget.Toast;
import java.io.File;
import java.util.ArrayList;
import java.util.List;
public class MainActivity extends Activity {
```

```
    private AdapterView.OnItemClickListener listViewListener = new
AdapterView.OnItemClickListener() {
        public void onItemClick(AdapterView<?> parent, View view,
int position,long id) {
            Cursor cursor = ((MediaCursorAdaptor)parent.
                            getAdapter()).getCursor();
            cursor.moveToPosition(position);
            String mediaId = cursor.getString(0);
            String albumId = cursor.getString(6);
            String songName = cursor.getString(2);
            Intent i = new
Intent(MainActivity.this,AudioPlayerActivity.class);
            i.putExtra("id",mediaId);
            i.putExtra("albumId",albumId);
            i.putExtra("songName",songName);
            i.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
            startActivity(i);
        }
    };
@Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        setTitle("Media Player");
        getMediaList();
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
```

```
        // Inflate the menu; this adds items to the action bar if
it is present.
       getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
    // Handle action bar item clicks here. The action bar will
   // automatically handle clicks on the Home/Up button, so long
  // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
        return true;
         }
       return super.onOptionsItemSelected(item);
}
    private void showToast(String message)
    {
    Toast.makeText(getApplicationContext(), message, Toast.
    LENGTH_SHORT).show();
    }
    private void getMediaList() {
    String selection = MediaStore.Audio.Media.IS_MUSIC + " != 0";
        String[] projection = {
                MediaStore.Audio.Media._ID,
                MediaStore.Audio.Media.ARTIST,
                MediaStore.Audio.Media.TITLE,
                MediaStore.Audio.Media.DATA,
```

```
                MediaStore.Audio.Media.DISPLAY_NAME,
                MediaStore.Audio.Media.DURATION,
                MediaStore.Audio.Media.ALBUM_ID
        };
        Cursor cursor = getContentResolver().query(
                MediaStore.Audio.Media.EXTERNAL_CONTENT_URI,
                projection,
                selection,
                null,
                null);
        MediaCursorAdaptor adapter = new
MediaCursorAdaptor(this,cursor);
        ListView listView = (ListView)findViewById(R.id.listView);
        listView.setAdapter(adapter);;
        listView.setOnItemClickListener(listViewListener);
    }
}
```

11. In order to populate the ListView, create a new class called MediaCursorAdapter.
12. Modify the code as shown in Code Snippet 31.

Code Snippet 31:

```
package aptech.com.audiovideoplayback;
import android.content.Context;
import android.database.Cursor;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.CursorAdapter;
import android.widget.TextView;
public class MediaCursorAdaptor extends CursorAdapter {
    public MediaCursorAdaptor(Context context, Cursor cursor) {
```

```
super(context, cursor, 0);
    }
    @Override
    public View newView(Context context, Cursor cursor,
ViewGroup parent) {
        return LayoutInflater.from(context).inflate(R.layout.
media_list_item, parent, false);
    }
    // The bindView method is used to bind all data to a given
view
    // such as setting the text on a TextView.
    @Override
    public void bindView(View view, Context context, Cursor
cursor) {
        TextView songNameView = (TextView) view.findViewById(R.
id.songNameView);
        TextView artistView = (TextView) view.findViewById(R.
id.artistView);
        TextView lengthView = (TextView) view.findViewById(R.
id.lengthView);
        String songName = cursor.getString(2);
        String artist = cursor.getString(1);
        int time = cursor.getInt(5);
        int seconds = (int) (time / 1000) % 60 ;
        int minutes = (int) ((time / (1000*60)) % 60);
        int hours   = (int) ((time / (1000*60*60)) % 24);
        String length="";
        if(hours>0)
            length+=Integer.toString(hours)+":";
        if(minutes>0)
```

```
        length+=Integer.toString(minutes)+":";
        length+=Integer.toString(seconds);
        songNameView.setText(songName);
        artistView.setText(artist);
        lengthView.setText(length);
    }
}
```

13. Open AudioPlayerActivity.java for editing and modify the code as shown in Code Snippet 32.

Code Snippet 32:

```
package aptech.com.audiovideoplayback;
import android.app.Activity;
import android.database.Cursor;
import android.media.AudioManager;
import android.media.MediaPlayer;
import android.net.Uri;
import android.os.Bundle;
import android.os.PowerManager;
import android.provider.MediaStore;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.Button;
import android.widget.ImageView;
import android.widget.ProgressBar;
import android.widget.SeekBar;
import java.io.IOException;
public class AudioPlayerActivity extends Activity {
    String mediaId;
    Uri mediaURI;
```

```
    String albumId;
    String songName;
    MediaPlayer mediaPlayer;
    ProgressUpdater pUpdater;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_audio_player);
        mediaId =  getIntent().getExtras().getString("id");
        albumId =  getIntent().getExtras().getString("albumId");
        songName =  getIntent().getExtras().
getString("songName");
        mediaPlayer = new MediaPlayer();
        pUpdater = new ProgressUpdater();
        setTitle(songName);
        mediaURI= Uri.withAppendedPath(MediaStore.Audio.Media.
EXTERNAL_CONTENT_URI, mediaId);
        //Set the album art image
        ImageView songArt = (ImageView)findViewById(R.
id.imageView2);
        Cursor cursorAlbum = getContentResolver().
query(MediaStore.Audio.Albums.EXTERNAL_CONTENT_URI,new String[]
{MediaStore.Audio.Albums._ID, MediaStore.Audio.Albums.ALBUM_
ART},MediaStore.Audio.Albums._ID+ "=" + albumId, null, null);
        if(cursorAlbum != null  && cursorAlbum.moveToFirst()){
String uri = cursorAlbum.getString(cursorAlbum.
getColumnIndex("album_art"));
            cursorAlbum.close();
            if(uri != null )
                songArt.setImageURI(Uri.parse(uri));
        }
```

```
SeekBar sBar = (SeekBar)findViewById(R.id.seekBar);
        sBar.setOnSeekBarChangeListener(new SeekBar.
OnSeekBarChangeListener() {
            @Override
            public void onStopTrackingTouch(SeekBar seekBar) {}
            @Override
            public void onStartTrackingTouch(SeekBar seekBar) {}
            @Override
            public void onProgressChanged(SeekBar seekBar, int
progress,boolean fromUser) {
                if(fromUser)
                { mediaPlayer.seekTo((progress*mediaPlayer.
getDuration())/100);}
            }
        });
mediaPlayer.setAudioStreamType(AudioManager.STREAM_MUSIC);
        try {
                mediaPlayer.setDataSource(getApplicationContext(),
mediaURI);
                mediaPlayer.setWakeMode(this, PowerManager.PARTIAL_
WAKE_LOCK);
                mediaPlayer.prepare();
        } catch (IOException e) {
                e.printStackTrace();
        }
    }
    @Override
    protected void onStop() {
        super.onStop();
    }
```

```
@Override
    protected void onDestroy() {
        mediaPlayer.stop();
        pUpdater.stop();
        mediaPlayer.release();
        super.onDestroy();
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if
it is present.
      getMenuInflater().inflate(R.menu.menu_audio_player, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
    // Handle action bar item clicks here. The action bar will
   // automatically handle clicks on the Home/Up button, so long
   // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
    private class ProgressUpdater implements Runnable {
        private Boolean stop = new Boolean(false);
        public void stop() {
```

```
stop=true;
        }
        @Override
        public void run() {
            while (!stop) {
                SeekBar sBar = (SeekBar) findViewById(R.id.seekBar);
          sBar.setProgress((mediaPlayer.getCurrentPosition()*100)/
mediaPlayer.getDuration());
                try {
                    Thread.sleep(500);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        }
    }
    private void startProgressUpdater(){
             mediaPlayer.setOnCompletionListener(new  MediaPlayer.
OnCompletionListener() {
            @Override
            public void onCompletion(MediaPlayer mp) {
                pUpdater.stop();
                SeekBar sBar = (SeekBar) findViewById(R.id.seekBar);
                sBar.setProgress(mp.getCurrentPosition());
            }
        });
        new Thread(pUpdater).start();
    }
    public void onPlayClicked(View view) {
```

```
Button playButton = (Button) findViewById(R.id.button2);
        if(!mediaPlayer.isPlaying()) {
            mediaPlayer.start();
            startProgressUpdater();
            playButton.setText("Pause");
        }
        else {
            playButton.setText("Play");
            mediaPlayer.pause();
        }
    }
    public void onRewindClicked(View view) {
       mediaPlayer.seekTo(mediaPlayer.getCurrentPosition()-5000);
    }
    public void onForwardClicked(View view) {
       mediaPlayer.seekTo(mediaPlayer.getCurrentPosition()+5000);
    }
}
```

14. Add an image named music.jpg to represent an audio file as shown in figure 6.8.

Figure 6.8: Music Icon

15. Navigate to res → values.

16. Modify the styles.xml as shown in Code Snippet 33.

Code Snippet 33:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="AppTheme" parent="android:Theme.Material.Light">
       <item name="android:colorPrimary">@android:color/holo_red_light </item>
        <item name="android:colorPrimaryDark">@android:color/holo_red_dark </item>
             <item name="android:actionBarStyle">@style/AppTheme.ActionBarStyle</item>
    </style>
<style name="AppTheme.ActionBarStyle" parent="@android:style/Widget.Material.ActionBar">
          <item name="android:background">@android:color/holo_red_light</item>
     <item name="android:titleTextStyle">@style/AppTheme.ActionBar.TitleTextStyle</item>
    </style>
       <style name="AppTheme.ActionBar.TitleTextStyle" parent="@android:style/TextAppearance.Material.WindowTitle">
        <item name="android:textColor">@android:color/white</item>
    </style>
    <style name="PlayTheme" parent="android:Theme.Material.Light">
            <item name="android:colorPrimary">@android:color/holo_blue_light </item>
        <item name="android:colorPrimaryDark">@android:color/holo_blue_dark </item>
<item name="android:actionBarStyle">@style/PlayTheme.ActionBarStyle</item>
    </style>
   <style name="PlayTheme.ActionBarStyle" parent="@android:style/Widget.Material.Light.ActionBar">
         <item name="android:background">@android:color/holo_blue_light</item>
```

```
                    <item name="android:titleTextStyle">@style/
PlayTheme.ActionBar.TitleTextStyle</item>
    </style>
     <style  name="PlayTheme.ActionBar.TitleTextStyle"  parent="@
android:style/TextAppearance.Material.WindowTitle">
       <item name="android:textColor">@android:color/white</item>
    </style>
</resources>
```

17. Modify strings.xml as shown in Code Snippet 34.

Code Snippet 34:

```
<resources>
    <string name="app_name">Media Player</string>
    <string name="action_settings">Settings</string>
      <string  name="title_activity_audio_player">Audio  Player</
string>
    <string name="albumart_label">Album Art</string>
</resources>
```

18. Navigate to manifests, open AndroidManifest.xml, and modify the code as shown in Code Snippet 35.

Code Snippet 35:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
xmlns:android="http://schemas.android.com/apk/res/android"
    package="aptech.com.audiovideoplayback" >
        <uses-permission  android:name="android.permission.WRITE_
EXTERNAL_STORAGE" />
  <uses-permission android:name="android.permission.WAKE_LOCK" />
   <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name
```

```
        android:theme="@style/AppTheme" >
        <activity
            android:name=".MainActivity"
            android:label="@string/app_name"
            android:theme="@style/AppTheme">
        <intent-filter>
           <action
               android:name="android.intent.action.MAIN" />
                 <category  android:name="android.intent.category.
                                        LAUNCHER" />
            </intent-filter>
        </activity>
        <activity
            android:name=".AudioPlayerActivity"
            android:label="@string/title_activity_audio_player"
            android:launchMode="singleTask"
            android:taskAffinity=""
            android:theme="@style/PlayTheme"
            >
        </activity>
    </application>
</manifest>
```

The uses-permission tag fetchs the permission to read and write from external storage and to keep the device awake.

19. Start DDMS by navigating to Tools → Android and select Android Device Monitor.
20. Once in DDMS perspective navigate to the File Explorer tab.
21. Click 'Push a file onto the Device' icon and select a file to copy an audio clip to the sdcard/Music folder.
22. Close DDMS.
23. Navigate to run menu and select Run 'app'.

The application is deployed on the AVD and the output is displayed as shown in figure 6.9.

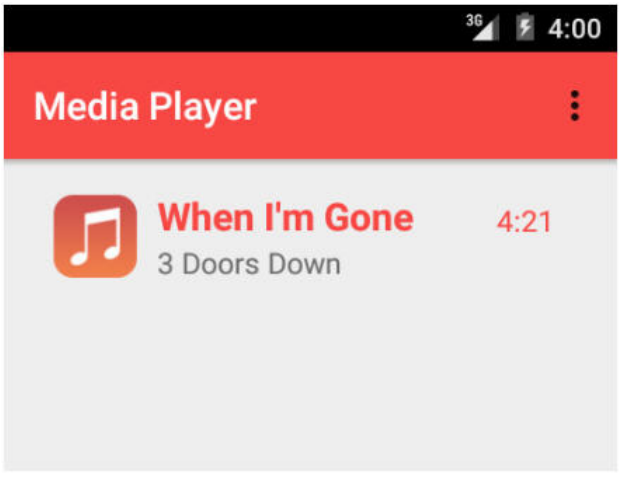


Figure 6.9: Media Player Application

Currently, only one file is added. If multiple files are added, they will be listed on the screen. The song name is listed, followed by the artist's name listed under it, and the length of the song to the right. All of this information is extracted from the file using the MediaStore API. The user can choose a song to play. The audio player is displayed as shown in figure 6.10.

Figure 6.10: Audio Player Screen

The album art of the song is automatically extracted if available and displayed. The user can start playing the song, by clicking the PLAY button as shown in figure 6.11.

Figure 6.11: Audio Playing

The progress of the song is displayed on the seekbar.

The REWIND and FORWARD buttons move the song behind or forward by five seconds. In addition to the buttons, the user can directly navigate to a time period by using the seekbar as shown in figure 6.12.

Figure 6.12: Seek Functionality

By pressing the home button of the device, the audio will continue to play in the background. A control widget can be provided to control the playback in the background.

6.6 Camera

The Android framework provides support for various cameras and camera features available on devices, allowing you to capture pictures and videos in your application. The Android framework supports capturing images and video through the Camera API or camera Intent.

6.6.1 List of Relevant Classes

The relevant classes are as follows:

1. **Camera**

   The characteristics of this class are as follows:

   - This is the main class and is responsible for controlling the device camera.
   - The Camera class is used to set image capture settings, start/stop preview, snap pictures, and retrieve frames for encoding the video.
   - It is present in android.hardware.camera package.
   - It is derived from Object class.

   The permission must be granted in AndroidManifest.xml file as shown in Code Snippet 36.

Code Snippet 36:

```
<uses-permission android:name="android.permission.CAMERA" />
<uses-feature android:name="android.hardware.camera" />
<uses-feature android:name="android.hardware.camera.autofocus" />
```

2. **SurfaceView**

   The characteristics of this class are as follows:

   - This class is used to present a live camera preview to the user.
   - It is present in android.view.SurfaceView package.
   - It is derived from View class.

3. **MediaRecorder**

   The characteristics of this class are as follows:

   - This class is used to record video from the camera.
   - It is present in android.media.MediaRecorder package.

6.6.2 Working with Camera

Following are the steps to create a Camera application:

1. Create a project named ImageCaptureDemo in Android Studio.
2. Navigate to res → layout folder.
3. Modify the code in activity_main.xml file as shown in Code Snippet 37.

Code Snippet 37:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/ res/
android"
android:layout_width="fill_parent"
android:layout_height="fill_parent"
android:orientation="vertical" >
<TextView
      android:layout_width="fill_parent"
      android:layout_height="wrap_content"
      android:layout_margin="3dp"
      android:text="Intro"
      android:textStyle="bold" />
<Button
      android:id="@+id/capture_btn"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Capture" />
<ImageView
```

```
      android:id="@+id/picture"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_margin="5dp"
      android:background="@drawable/pic_border"
      android:contentDescription="Picture" />
<LinearLayout
      android:id="@+id/callme"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content" >
</LinearLayout>
</LinearLayout>
```

4. Create another Android XML File under res → layout folder named activity_shoot_and_crop.xml.
5. Modify the code in the file as shown in Code Snippet 38.

Code Snippet 38:

```
<RelativeLayout xmlns:android="http://schemas.android.com/ apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".ShootAndCropActivity" >
<TextView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_centerHorizontal="true"
      android:layout_centerVertical="true"
      android:text="@string/hello_world" />
</RelativeLayout>
```

6. Navigate to src → com.example.imagecapturedemo folder.

7. Modify the code in MainActivity file as shown in Code Snippet 39.

Code Snippet 39:

```
package com.example.imagecapturedemo;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.view.View.OnClickListener;
import android.content.ActivityNotFoundException;
import android.content.Intent;
import android.graphics.Bitmap;
import android.net.Uri;
import android.provider.MediaStore;
import android.view.View;
import android.widget.Button;
import android.widget.ImageView;
import android.widget.LinearLayout;
import android.widget.Toast;
public class MainActivity extends Activity implements OnClickListener
{
Button next;
//keep track of camera capture intent
final int CAMERA_CAPTURE = 1;
//keep track of cropping intent
final int PIC_CROP = 2;
//captured picture uri
private Uri picUri;
@Override
protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
```

```
      setContentView(R.layout.activity_main);
//retrieve a reference to the UI button
Button captureBtn = (Button)findViewById(R.id.capture_ btn);
      //handle button clicks
      captureBtn.setOnClickListener(this);
}
/**
* Click method to handle user pressing button to launch camera
*/
public void onClick(View v) {
      if (v.getId() == R.id.capture_btn) {
            try {
 //use standard intent to capture an image
Intent captureIntent = new Intent(MediaStore. ACTION_IMAGE_
CAPTURE);
//we will handle the returned data in onActivityResult
 startActivityForResult(captureIntent, CAMERA_CAPTURE);
 }
catch(ActivityNotFoundException anfe){
//display an error message
String errorMessage = "Whoops - your device doesn't support
capturing images!";
Toast toast = Toast.makeText(this, errorMessage, Toast.LENGTH_
SHORT);
            toast.show();
      }
}
}
/**
```

```
* Handle user returning from both capturing and cropping the image
*/
protected void onActivityResult(int requestCode, int resultCode,
Intent data) {
if (resultCode == RESULT_OK) {
      //user is returning from capturing an image using the camera
            if(requestCode == CAMERA_CAPTURE){
            //get the Uri for the captured image
                  picUri = data.getData();
            //carry out the crop operation
            performCrop();
            Button next =new Button(this);
            next.setText("Next");
            LinearLayout ll = (LinearLayout)findViewById(R.id.callme);
// It could not be a LinearLayout, of course ll.addView(next);
            next.setOnClickListener(new OnClickListener() {
                  @Override
                  public void onClick(View v) {
                        // TODO Auto-generated method stub
                        Intent a=new Intent(getApplicationC
ontext(),Sound.class);
                        startActivity(a);
                  }
            });
      }
      //user is returning from cropping the image
            else if(requestCode == PIC_CROP){
```

```
                    //get the returned data
Bundle extras = data.getExtras();
            //get the cropped bitmap
            Bitmap thePic = extras.getParcelable("data");
            //retrieve a reference to the ImageView
            ImageView picView = (ImageView)findViewById(R.
id.picture);
            //display the returned cropped image
            picView.setImageBitmap(thePic);
            }
      }
}
/**
* Helper method to carry out crop operation
*/
private void performCrop(){
      //take care of exceptions
      try {
             //call the standard crop action intent (the user
device may not support it)
            Intent cropIntent = new Intent("com.android. camera.
action.CROP");
            //indicate image type and Uri
            cropIntent.setDataAndType(picUri, "image/*");
            //set crop properties
            cropIntent.putExtra("crop", "true");
            //indicate aspect of desired crop
            cropIntent.putExtra("aspectX", 1);
            cropIntent.putExtra("aspectY", 1);
```

```
            //indicate output X and Y
                  cropIntent.putExtra("outputX", 256);
            cropIntent.putExtra("outputY", 256);
            //retrieve data on return
            cropIntent.putExtra("return-data", true);
            //start the activity - we handle returning in
onActivityResult
            startActivityForResult(cropIntent, PIC_CROP);
      }
      //respond to users whose devices do not support the crop
action
      catch(ActivityNotFoundException anfe){
            //display an error message
             String errorMessage = "Whoops - your device doesn't
support the crop action!";
             Toast toast = Toast.makeText(this, errorMessage,
Toast.LENGTH_SHORT);
            toast.show();
      }
}
@Override
public boolean onCreateOptionsMenu(Menu menu) {
      // Inflate the menu; this adds items to the action bar if it
is present.
      getMenuInflater().inflate(R.menu.main, menu);
      return true;
}
}
```

8. Create a new class named Sound under src → com.example.imagecapturedemo folder.

9. Modify the code in Sound class as shown in Code Snippet 40.

Code Snippet 40:

```
package com.example.imagecapturedemo;
import android.app.Activity;
import android.media.MediaRecorder;
import android.os.Bundle;
public class Sound extends Activity{
@Override
protected void onCreate(Bundle savedInstanceState) {
      // TODO Auto-generated method stub
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_shoot_and_crop);
      MediaRecorder recorder;
}
}
```

10. Add a few permissions and the activity class in the AndroidManifest.xml file as shown in Code Snippet 41.

Code Snippet 41:

```
<activity android:name=".Sound" ></activity></application>
<uses-permission android:name="android.permission.CAMERA" >
</uses-permission>
<uses-permission android:name="android.permission.RECORD_ AUDIO"
/>
```

Figure 6.13 displays the output once the application is executed.

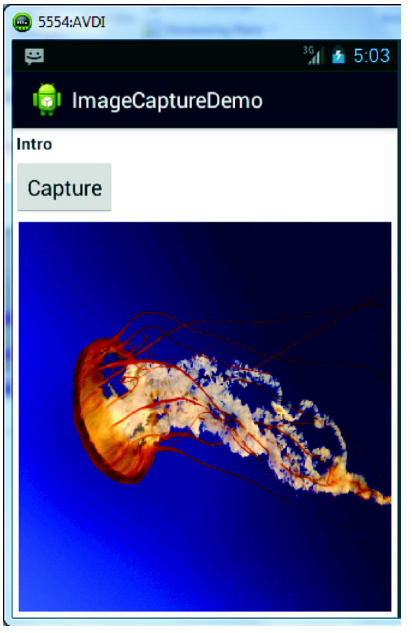


Figure 6.13: ImageCaptureDemo

Note - The capturing of image will take place in a real-time Android device.

6.6.3 Camera2 API

With Lollipop, Google has introduced a new API to access camera hardware. The old API is still functional, but the newer API is aimed to provide access to more powerful features which are now common in modern devices. These features include multiple cameras, Color Correction, Auto focus, and RAW Image capture.

The underlying ideology of the API has also undergone a major change. In the earlier iterations, Camera objects were used to represent a physical camera and all the functionalities were incorporated into this camera object. Camera2 API uses a CameraManager to interact with the physical hardware. The Camera objects are replaced with CameraDevice objects, but the functionality of the object is reduced to just representing the camera device.

The classes for the camera2 API are provided in the android.hardware.camera2 package.

- **Capture Process**:

  The process to capture or preview the camera feed is described as follows:

  1. Get a reference to the camera manager instance using the getSystemService(Context. CAMERA_SERVICE) method.

2. Use the getCameraIdList() method to retrieve an id list of connected camera devices of the camera manager.
3. Retrieve the camera characteristics using getCameraCharacteristics() method of the camera manager.
4. Save the supported capture sizes and initialize the preview resolution.
5. Open the camera using the openCamera() method of the camera manager.
6. Get a reference to the CameraDevice object from the onOpened() callback once the camera is opened.
7. Create a new MediaRecorder object.
8. Initialize the media recorder object with the appropriate settings.
9. Retrieve the MediaRecorder surface using the getSurface() method.
10. (Optional) Get the surface of preview target so that the user can know what is being captured.
11. Create a capture session request by passing the list of surfaces containing the surfaces of the media recorder and the preview target. This is done using the createCaptureSession() method of the camera device. The camera data will be sent to these surfaces.
12. Start recording video when needed using the MediaRecorder's start() method.
13. The recording can be stopped using the stop() method.
14. (Optional) A background thread and handler can be used to not block the UI while waiting for IO and the camera to open, and so on.

Following example explains the process of capturing Audio and Video using the new Camera2 API.

Note - It is strongly recommended to use a custom emulator or a real device running Android Lollipop or newer to run this example. The Android version is very important as older versions do not support the camera2 API.

For the purpose of this example, a custom emulator (Andy) is used with an emulated camera.

1. Create a new project called Camera in Android Studio.
2. Navigate to res → layout and open main_activity.xml for editing.

3. Modify the code as shown in Code Snippet 42.

Code Snippet 42:

```
<LinearLayout
    xmlns:android=”http://schemas.android.com/apk/res/android”
    xmlns:tools=”http://schemas.android.com/tools” android:layout_
width=”match_parent”
    android:layout_height=”match_parent” android:paddingLeft=”@
dimen/activity_horizontal_margin”
    android:paddingRight=”@dimen/activity_horizontal_margin”
    android:paddingTop=”@dimen/activity_vertical_margin”
    android:paddingBottom=”@dimen/activity_vertical_margin”
tools:context=”.MainActivity”
    android:orientation=”vertical”>
    <SurfaceView
        android:layout_width=”fill_parent”
        android:layout_height=”wrap_content”
        android:id=”@+id/surfaceView”
        android:layout_weight=”3” />
    <LinearLayout
        android:orientation=”horizontal”
        android:layout_width=”match_parent”
        android:layout_height=”wrap_content”
        android:layout_weight=”0”>
        <Button
            style=”?android:attr/buttonStyleSmall”
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Capture”
            android:id=”@+id/button”
            android:onClick=”captureClicked”
            android:layout_weight=”1”
```

```
            android:textColor="@android:color/holo_red_dark" />
    </LinearLayout>
</LinearLayout>
```

4. Navigate to java → aptech.com.camera and open MainActivity.java for editing.
5. Modify the code as shown in Code Snippet 43.

Code Snippet 43:

```
package aptech.com.camera;
import android.app.Activity;
import android.content.Context;
import android.graphics.SurfaceTexture;
import android.hardware.camera2.params.StreamConfigurationMap;
import android.media.MediaRecorder;
import android.os.Bundle;
import android.os.Handler;
import android.os.HandlerThread;
import android.util.Size;
import android.view.Menu;
import android.view.MenuItem;
import android.view.Surface;
import android.view.SurfaceView;
import android.view.View;
import android.widget.Button;
import android.hardware.camera2.*;
import android.widget.Toast;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
public class MainActivity extends Activity {
```

```
    private String cameraId;
    private CameraCaptureSession previewSession;
    private CameraDevice cameraDevice;
    private Size videoSize;
    private MediaRecorder mediaRecorder;
    private CaptureRequest.Builder previewBuilder;
    private SurfaceView surfaceView;
    private boolean isRecording =false;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        surfaceView = (SurfaceView) findViewById(R.id.surfaceView);
        openCamera();
    }
    private void openCamera(){ incorrect code starts from here
        CameraManager manager = (CameraManager)
getSystemService(Context.CAMERA_SERVICE);
        try {
            cameraId = manager.getCameraIdList()[0];
            Toast.makeText(this,"Opened Camera with id "+
cameraId, Toast.LENGTH_SHORT).show();
            CameraCharacteristics characteristics = manager.
getCameraCharacteristics(cameraId);
            StreamConfigurationMap map = characteristics.
get(CameraCharacteristics.SCALER_STREAM_CONFIGURATION_MAP);
           videoSize =  map.getOutputSizes(SurfaceTexture.class)
[0];
           mediaRecorder = new MediaRecorder();
           manager.openCamera(cameraId, mStateCallback, null);
```

```
        } catch (CameraAccessException e) {
            Toast.makeText(this, "Cannot access the camera.",
Toast.LENGTH_SHORT).show();
        } catch (NullPointerException e) {
        }
    }
    private CameraDevice.StateCallback mStateCallback = new
CameraDevice.StateCallback() {
        @Override
        public void onOpened(CameraDevice cameraDevice) {
            MainActivity.this.cameraDevice = cameraDevice;
            startPreview();
        }
        @Override
        public void onDisconnected(CameraDevice cameraDevice) {
            cameraDevice.close();
            MainActivity.this.cameraDevice = null;
        }
        @Override
      public void onError(CameraDevice cameraDevice, int error){
            cameraDevice.close();
            MainActivity.this.cameraDevice = null;
        }
    };
   private void initMediaRecorder(){
        mediaRecorder.setAudioSource(MediaRecorder.AudioSource.MIC);
mediaRecorder.setVideoSource(MediaRecorder.VideoSource.SURFACE);
mediaRecorder.setOutputFormat(MediaRecorder.OutputFormat.
MPEG_4);
```

```
        mediaRecorder.setVideoEncodingBitRate(10000000);
        mediaRecorder.setVideoFrameRate(30);
        mediaRecorder.setOutputFile(getExternalFilesDir(null).
getAbsolutePath() + "/media/abc.mp4");
        mediaRecorder.setVideoSize(videoSize.getWidth(),
videoSize.getHeight());
        mediaRecorder.setVideoEncoder(MediaRecorder.VideoEncoder.
H264);
        mediaRecorder.setAudioEncoder(MediaRecorder.AudioEncoder.
AAC);
        try {
            mediaRecorder.prepare();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
    private void startPreview(){
        initMediaRecorder();
        Surface surface = surfaceView.getHolder().getSurface();
        try {
            previewBuilder = cameraDevice.
createCaptureRequest(CameraDevice.TEMPLATE_RECORD);
            List<Surface> surfaces = new ArrayList<Surface>();
           surfaces.add(surface);
            previewBuilder.addTarget(surface);
           Surface recorderSurface = mediaRecorder.getSurface();
            surfaces.add(recorderSurface);
            previewBuilder.addTarget(recorderSurface);
           cameraDevice.createCaptureSession(surfaces,
mPreviewStateCallback, null);
```

```
        } catch (CameraAccessException e) {
            e.printStackTrace();
        }
    }
    private CameraCaptureSession.StateCallback
mPreviewStateCallback = new CameraCaptureSession.StateCallback()
{
        @Override
        public void onConfigured(CameraCaptureSession session) {
            previewSession = session;
            previewBuilder.set(CaptureRequest.CONTROL_MODE,
CameraMetadata.CONTROL_MODE_AUTO);
            HandlerThread backgroundThread = new
HandlerThread("CameraPreview");
            backgroundThread.start();
            Handler backgroundHandler = new
Handler(backgroundThread.getLooper());
            try {          previewSession.
setRepeatingRequest(previewBuilder.build(), null,
backgroundHandler);
            } catch (CameraAccessException e) {
                e.printStackTrace();
            }
        }
       @Override
        public void onConfigureFailed(CameraCaptureSession
session) {
        }
    };
   @Override
    public boolean onCreateOptionsMenu(Menu menu) {
```

```
        // Inflate the menu; this adds items to the action bar if
it is present.
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
    // Handle action bar item clicks here. The action bar will
    // automatically handle clicks on the Home/Up button, so long
    // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
   public void captureClicked(View view) {
        Button captureButton = (Button)findViewById(R.id.button);
        if(isRecording) {
            mediaRecorder.start();
            captureButton.setText("Stop Capture");
            isRecording=true;
        }
       else {
            mediaRecorder.stop();
            mediaRecorder.reset();
```

```
            captureButton.setText("Capture");
            isRecording=false;
        }
    }
}
```

6. Navigate to manifests and open AndroidManifest.xml for editing.
7. Modify the code as shown in Code Snippet 44.

Code Snippet 44:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/
android"
    package="aptech.com.camera" >
    <uses-feature android:name="com.android.hardware.camera2.
full"/>
    <uses-permission android:name="android.permission.RECORD_
AUDIO" />
    <uses-permission android:name="android.permission.CAMERA" />
    <uses-permission android:name="android.permission.WRITE_
EXTERNAL_STORAGE" />
   <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <activity
           android:name=".MainActivity"
            android:label="@string/app_name" >
            <intent-filter>
           <action android:name="android.intent.action.MAIN" />
```

```
    <category android:name="android.intent.category.LAUNCHER" />
         </intent-filter>
        </activity>
    </application>
</manifest>
```

The uses-feature tag prevents app from being installed on devices that do not support camera2 API.

8. Navigate to the run menu and select Run 'app'.

   The application is deployed on the AVD and the output is shown in figure 6.14.

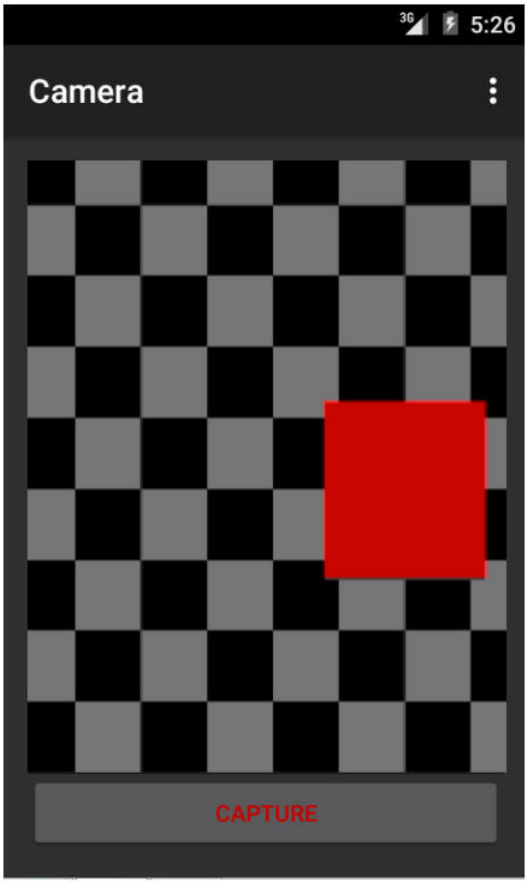


Figure 6.14: Camera Application Output

By clicking CAPTURE, the video begins to be recorded as shown in figure 6.15.

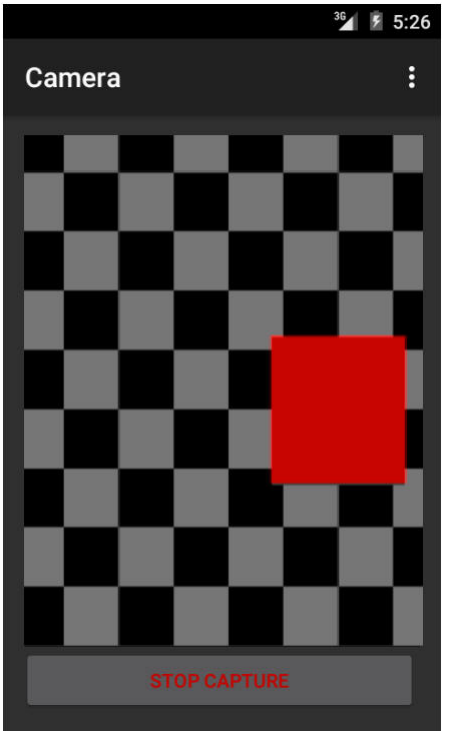


Figure 6.15: Video being Recorded

To stop the recording, the Stop Capture button can be clicked. The recorded file can be accessed from DDMS as shown in figure 6.16.

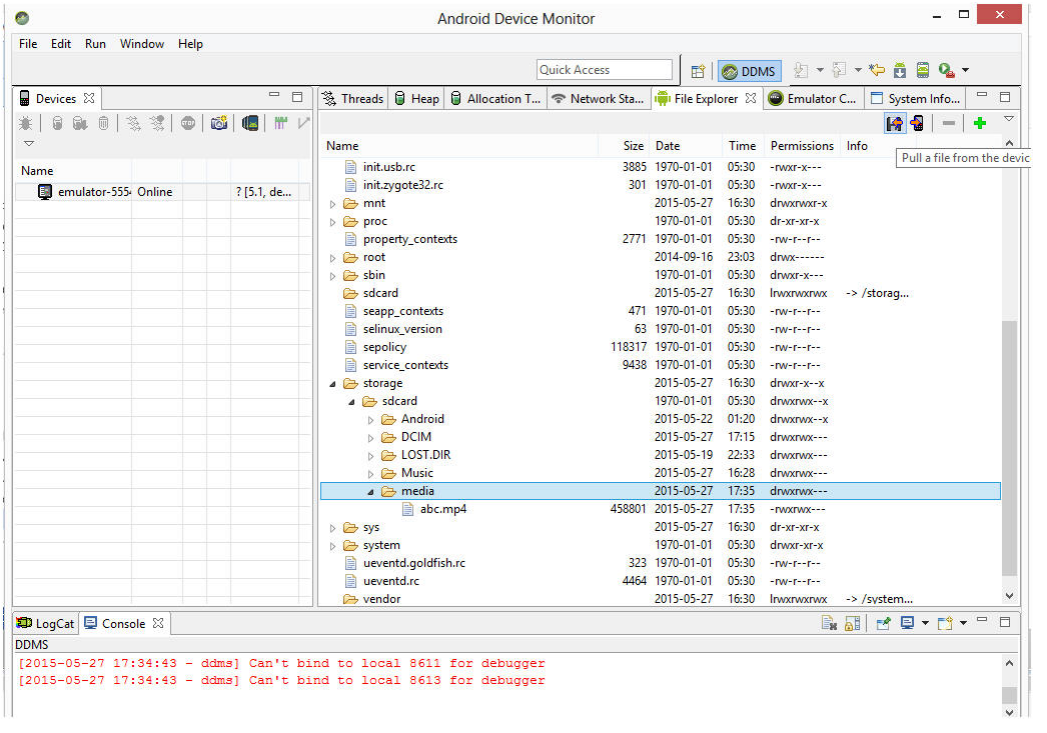


Figure 6.16: Recorded File in DDMS

The file can be pulled from the device to view on the PC as shown in figure 6.17.

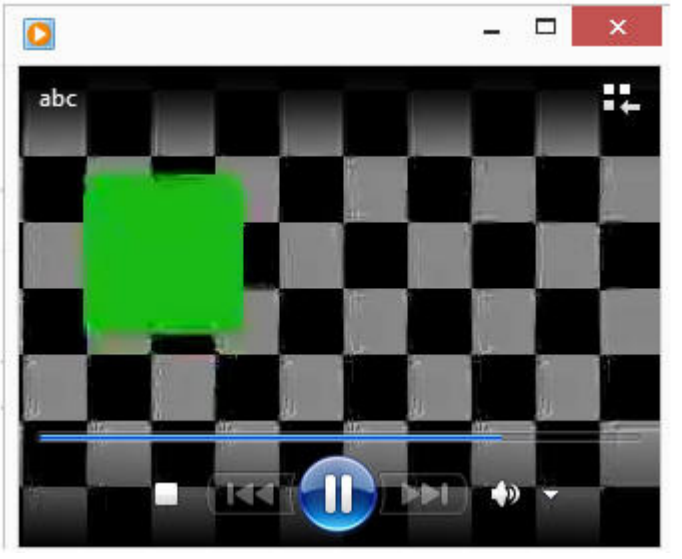


Figure 6.17: Recorded Video Playing on the Desktop

Note - If the AVD is used to test the application, certain AVD system images do not support emulated cameras properly with Camera2 API. In this case, the developer can either test the application on a real device or a by adding a read camera to the AVD from the AVD Manager.

6.7 Live Wallpapers

Live wallpapers are animated and sometimes interactive wallpapers. This section describes the process to create live wallpaper with Wallpaper setting and onTouch interaction.

6.7.1 Steps to Create Live Wallpaper

Following are the steps to create a live wallpaper:

- **Manifest and Permission**

  The manifest file needs to contain a service extending the Wallpaper service with the permission android.permission.BIND_WALLPAPER.

An example configuration is shown in Code Snippet 45.

Code Snippet 45:

```
<service
    android:name=".AnimatedWallpaperService"
    android:enabled="true"
    android:label="Live Wallpaper"
    android:permission="android.permission.BIND_WALLPAPER" >
    <intent-filter>
        <action android:name="android.service.wallpaper.
WallpaperService" />
    </intent-filter>
    <meta-data
        android:name="android.service.wallpaper"
        android:resource="@xml/wallpaper" >
    </meta-data>
</service>
```

- **Settings Activity**

  A settings activity needs to be specified which saves the preferences and settings to configure the live wallpaper.

- **Metadata Description**

  An xml file needs to be specified which holds the details of the live wallpaper such as the name of the wallpaper, a thumbnail, description, and so on.

  An example metadata file is shown in Code Snippet 46.

Code Snippet 46:

```
<?xml version="1.0" encoding="utf-8"?>
<wallpaper
xmlns:android="http://schemas.android.com/apk/res/android"
    android:label="Live Wallpaper"
    android:thumbnail="@drawable/thumb"
    android:description="@string/app_name"
```

```
android:settingsActivity="aptech.com.livewallpaper.SettingsActivity">
</wallpaper>
```

- **Wallpaper Service**

  A class extending the WallpaperService class needs to exist in the project. The onCreateEngine() method needs to be overridden to return an engine object.

- **Wallpaper Engine**

  The rendering of the wallpaper is handled by the Wallpaper engine. In order to create an engine, a new inner class extending the class WallpaperService.Engine needs to be created. Following methods need to be implemented:

  - onCreate
  - onDestroy
  - onTouchEvent
  - onVisibilityChanged

  The developer can write the code to render the animation in the onCreate(SurfaceHolder obj) method. The render method can be any method as per the preference of the developer.

  If the graphics methods or animations described earlier are used, the canvas to draw can be retrieved by using the lockCanvas() method on the Surface Holder object. If OpenGL is being used to render the animation, then the surface can be retrieved using the getSurface() method of the surface holder object. Any one of these methods can be used to try live wallpaper.

  Since, both these methods have been explained earlier, in the following example, an animated GIF will be used to create a Live Wallpaper. We will utilize the Movie class to draw the frames onto the screen.

6.7.2 Live Wallpaper Example

Following are the steps to create a Live Wallpaper:

1. Create a project called LiveWallpaper in Android Studio with No Activity.
2. Navigate to manifests and open AndroidManifest.xml for editing.

3. Modify the code as shown in Code Snippet 47.

Code Snippet 47:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/
android"
    package="aptech.com.livewallpaper" >
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <service
            android:name=".AnimatedWallpaperService"
            android:enabled="true"
            android:label="Live Wallpaper"
      android:permission="android.permission.BIND_WALLPAPER" >
            <intent-filter>
                <action
android:name="android.service.wallpaper.WallpaperService" />
            </intent-filter>
           <meta-data
                android:name="android.service.wallpaper"
                android:resource="@xml/wallpaper" >
            </meta-data>
        </service>
        <activity
            android:name=".SettingsActivity"
            android:label="@string/title_activity_settings" >
            <intent-filter>
```

```
            <action android:name="android.intent.action.MAIN" />
          </intent-filter>
        </activity>
    </application>
</manifest>
```

Notice that there is no activity with the launcher intent. As a result, the wallpaper will not show up in the app launcher. This is the desired behavior.

4. Create a new directory under res called 'xml'.
5. Create a new xml file named wallpaper.xml and modify the code as shown in Code Snippet 48.

Code Snippet 48:

```
<?xml version="1.0" encoding="utf-8"?>
<wallpaper
xmlns:android="http://schemas.android.com/apk/res/android"
    android:label="Live Wallpaper"
    android:thumbnail="@drawable/thumb"
    android:description="@string/app_name"
android:settingsActivity="aptech.com.livewallpaper.
SettingsActivity">
</wallpaper>
```

6. Right-click the app folder and navigate to New → Folder, and select Assets Folder.
7. Place three animated gifs with numeric names as shown in figure 6.18.

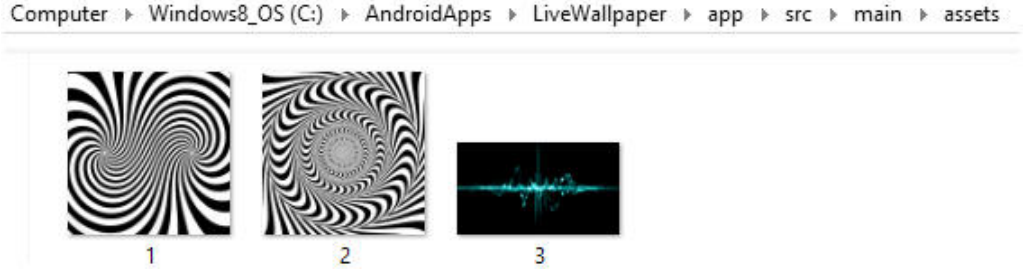


Figure 6.18: gif Images in the Assets Folder

The developer can choose any animated gif.

8. Create a new class called AnimatedWallpaperService in the aptech.com.livewallpaperpackage.

9. Modify the code as shown in Code Snippet 49.

Code Snippet 49:

```
package aptech.com.livewallpaper;
import android.content.SharedPreferences;
import android.graphics.Canvas;
import android.graphics.Movie;
import android.os.Handler;
import android.preference.PreferenceManager;
import android.service.wallpaper.WallpaperService;
import android.view.MotionEvent;
import android.view.SurfaceHolder;
import android.widget.Toast;
import java.io.File;
import java.io.IOException;
import java.io.InputStream;
public class AnimatedWallpaperService extends WallpaperService {
   private class WallpaperEngine extends WallpaperService.Engine {
         int fps=60;
         private final int frameDuration;
        private SurfaceHolder holder;
         private Movie movie;
         private boolean visible;
         private Handler handler;
         int id=0;
         int idMax=3;
         boolean cycle;
```

```
        public WallpaperEngine() {
            SharedPreferences prefs = PreferenceManager.
getDefaultSharedPreferences(AnimatedWallpaperService.this);
            cycle = prefs.getBoolean("cycle", true);
            frameDuration= (1000/fps);
            InputStream animation = null;
            try {
                animation = getAssets().open("1.gif");
            } catch (IOException e) {
                e.printStackTrace();
            }
            movie = Movie.decodeStream(animation);
            handler = new Handler();
        }
       @Override
        public void onCreate(SurfaceHolder surfaceHolder) {
            super.onCreate(surfaceHolder);
            this.holder = surfaceHolder;
        }
       private Runnable drawThread = new Runnable() {
            public void run() {
                draw();
           }
        };
        private void draw() {
            if (visible) {
                Canvas canvas = holder.lockCanvas();
                canvas.save();
                canvas.scale(2f, 2f);
```

```
                movie.draw(canvas, 0, 0);
                canvas.restore();
                holder.unlockCanvasAndPost(canvas);
                movie.setTime((int) (System.currentTimeMillis()
% movie.duration()));
                handler.removeCallbacks(drawThread);
                handler.postDelayed(drawThread, frameDuration);
            }
        }
        @Override
        public void onTouchEvent(MotionEvent event) {
            InputStream animation = null;
            if(id==idMax)
                id=0;
            else
                id++;
            String fileName = Integer.toString(id)+".gif";
            try {
                animation = getAssets().open(fileName);
            } catch (IOException e) {
                e.printStackTrace();
            }
            movie = Movie.decodeStream(animation);
        }
        @Override
        public void onDestroy() {
            super.onDestroy();
            handler.removeCallbacks(drawThread);
```

```
        }
        @Override
        public void onVisibilityChanged(boolean visible) {
            this.visible = visible;
            if (visible) {
                handler.post(drawThread);
            } else {
                handler.removeCallbacks(drawThread);
            }
        }
    }
    @Override
    public Engine onCreateEngine() {
        return new WallpaperEngine();
    }
}
```

10. Right-click app and select New → Activity and select Settings Activity.
11. Name the activity as SettingActivity.
12. Modify the code as shown in Code Snippet 50.

Code Snippet 50:

```
package aptech.com.livewallpaper;
import android.annotation.TargetApi;
import android.content.Context;
import android.content.res.Configuration;
import android.media.Ringtone;
import android.media.RingtoneManager;
import android.net.Uri;
import android.os.Build;
import android.os.Bundle;
import android.preference.ListPreference;
```

```
import android.preference.Preference;
import android.preference.PreferenceActivity;
import android.preference.PreferenceCategory;
import android.preference.PreferenceFragment;
import android.preference.PreferenceManager;
import android.preference.RingtonePreference;
import android.text.TextUtils;
import java.util.List;
public class SettingsActivity extends PreferenceActivity {
/*
// You can return to the Wallpaper manager using the
following code
    Intent intent = new Intent(WallpaperManager.ACTION_
CHANGE_LIVE_WALLPAPER);
    intent.putExtra(WallpaperManager.EXTRA_LIVE_WALLPAPER_
COMPONENT,new ComponentName(this, AnimatedWallpaperService.
class));
    startActivity(intent);
 */
    @Override
    protected void onPostCreate(Bundle savedInstanceState) {
        super.onPostCreate(savedInstanceState);
        setupSimplePreferencesScreen();
    }
    private void setupSimplePreferencesScreen() {
        addPreferencesFromResource(R.xml.pref_general);
    }
}
```

13. Navigate to res →xml and open pref_general.xml for editing.

14. Modify the code as shown in Code Snippet 51.

Code Snippet 51:

```
<PreferenceScreen
xmlns:android=”http://schemas.android.com/apk/res/android”>
    <CheckBoxPreference
        android:key=”cycle”
        android:title=”Enable Animation Cycle”
       android:summary=”Changes the wallpaper animation when the
screen is touched”
        android:defaultValue=”true” />
</PreferenceScreen>
```

15. Create a preview image and copy it to the res → drawable directory as shown in figure 6.19.

Figure 6.19: Preview Image for the Wallpaper

16. Navigate to Run menu and select Run ‘app’.

The Live wallpaper is installed on the AVD. In order to change the wallpaper, click and hold on the main screen as shown in figure 6.20.

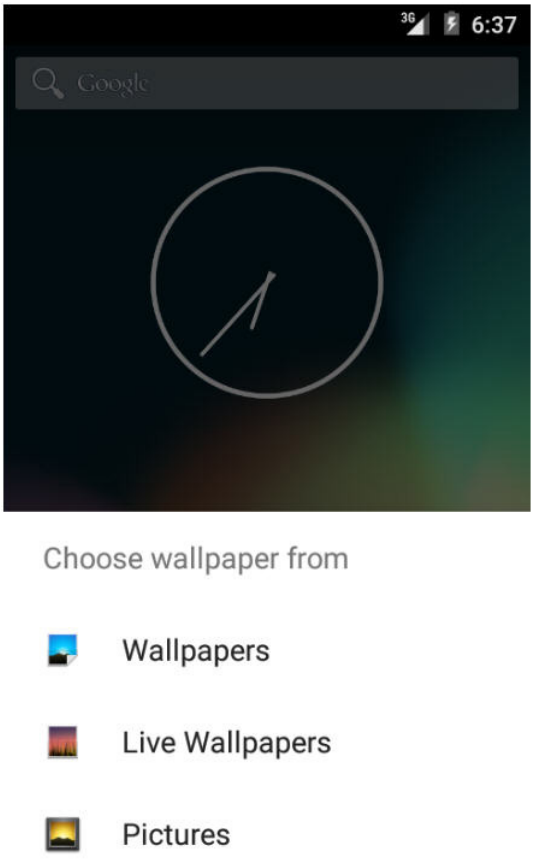


Figure 6.20: Launching the Wallpaper Manager

Select Live Wallpapers to list the wallpaper installed. The created wallpaper is listed here as shown in figure 6.21.

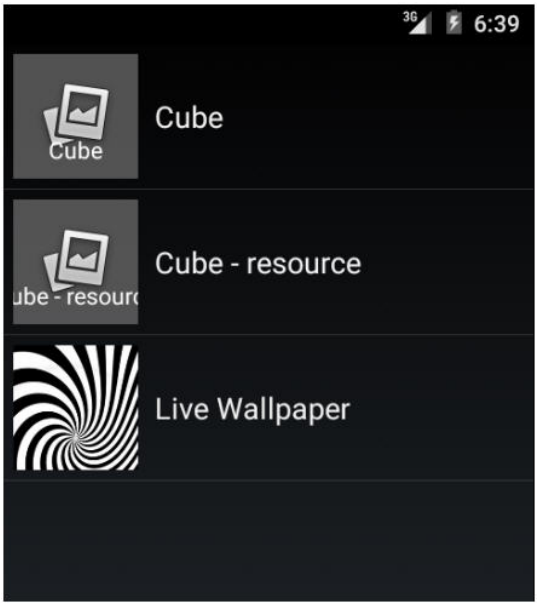


Figure 6.21: Live Wallpaper

Click Live Wallpaper to show the options Settings and Set wallapaper as shown in figure 6.22.

Figure 6.22: Wallpaper Selected

Click Settings to display the wallpaper settings as shown in figure 6.23.

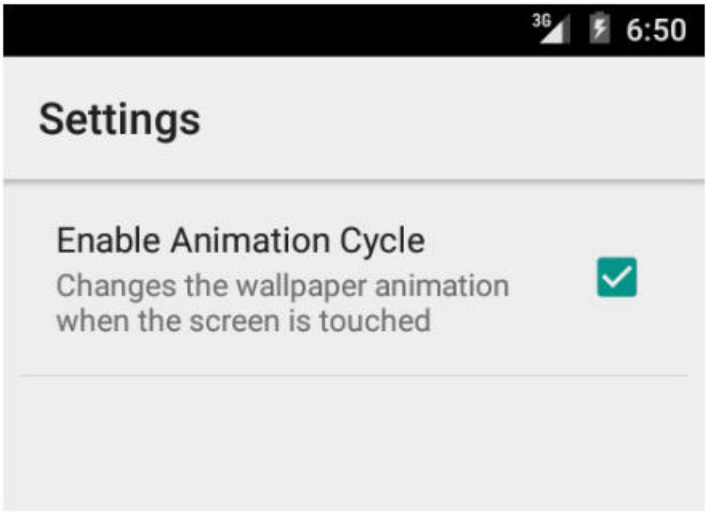


Figure 6.23: Wallpaper Settings

The cycle option can be enabled or disabled as per the user preference. If enabled, a different animation is displayed as shown in figure 6.24.

Click Back and select Set Wallpaper to set the wallpaper.

Figure 6.24: Wallpaper Set

If the Cycle setting is enabled, the animation changes whenever the user touches the screen as shown in figure 6.25.

Figure 6.25: Wallpaper Animation Cycle

Check Your Progress

1. Which types of media are handled by Android?

1.	Audio	2.	Video
3.	Stream	4.	RAW

(A)	1, 4	(B)	2, 4
(C)	1, 2	(D)	3, 4

2. Which of the following statements are true?

1.	OpenGL ES 3.0 was introduced in Android Lollipop.
2.	Animations were introduced in Android JellyBean.
3.	MediaStore is a content provider.
4.	MediaRecorder is used to record audio/video.

(A)	3, 4	(B)	1, 2
(C)	2, 4	(D)	2, 3

3. Which permission needs to be granted in AndroidManifest.xml file while listening to audio/video?

(A)	android.permission.WAKE_LOCK	(B)	android.permission.INTERNET
(C)	android.permission.SLEEP	(D)	android.permission.READ_DATA

4. OpenGL objects are displayed using which GL?

(A)	Renderer	(B)	SurfaceView
(C)	Surface	(D)	Application

5. Match the following descriptions with the correspoinding class.

	Description		Class
a.	Object used for drawing.	1.	Strings
b.	Object used to specify Shader code.	2.	Camera
c.	The class that has been introduced in Camera2 API.	3.	Canvas
d.	The class used to represent a physical camera in camera API.	4.	CameraDevice

(A)	a-2, b-3, c-4, d-1	(B)	a-2, b-4, c-1, d-3
(C)	a-3, b-1, c-4, d-2	(D)	a-1, b-3, c-4, d-2

Answers

1.	C
2.	A
3.	A
4.	B
5.	C

Summary

- Android provides basic level of graphics tools such as canvases, colors filters, points, and rectangles that allows the developer to handle drawing on the screen directly.
- Android framework provides two animation types namely Property Animation and View Animation.
- Property Animation creates an animation by modifying an object's property values over a set period of time with an Animator. View Animation creates an animation by performing a series of transformations on the contents of a View object.
- OpenGL is the 2D/3D rendering library supported by Android. It can used to develop rich multimedia applications and games.
- Android multimedia framework includes support for playing variety of common media types, so that one can easily integrate audio, video, and images into your applications. The MediaStore API allows searching for media content.
- Android framework supports capturing images and video through the Camera API or camera Intent.
- Lollipop introduced the new Camera2 API which allows low level access to camera functionality with more features.

Onlinevarsity

07 Data Handling and Content Providers

Welcome to the Session, **Data Handling and Content Providers.**

This session introduces various ways of handling data in Android. The session proceeds with an explanation of various ways of storing data using different storage systems.

In this Session, you will learn to:

- Explain the process of saving and loading preferences.
- Explain persistence of data to file
- Explain external storage
- Explain internal storage
- Explain SQLite Database
- Explain the process of storing and retrieving data from the database
- Explain Content Providers
- Make and use content providers
- Explain Resources and assets

Session

07

Data Handling and Content Providers

7.1 Introduction

Android provides different ways of saving persistent application data. The solution of storage that the developer chooses depends on the specific needs, such as whether the data should be private to the application or accessible to other applications (and the user) and how much storage space does the data require. Different data storage options that are provided by Android are as follows:

- **Shared Preference**: Stores all the private primitive data as key-value pairs. This data will be accessed from within the application.
- **Internal Storage**: Stores the private data on the device memory. If the application is uninstalled then the data will be lost.
- **External Storage**: Stores the public data on the shared external storage. If the application is uninstalled then the data will not be lost and will be present in the storage.
- **SQLite Database**: Stores the structured data in a private database.

7.2 Persisting Data to Shared Preferences File

Android provides SharedPreferences object to help the developer to save simple application data. Using the SharedPreferences object, the developer can save and retrieve the desired data through the use of key/value pairs. The developer needs to specify a key for the data that is required to be saved and then both the key and its value will be saved automatically to an XML file. It enables the developer to save primitive data types such as boolean, float, long, string, and so on.

The SharedPreferences class is present in the android.content package and the developer needs to import the class to work with an object of the class.

To use a SharedPreferences object, one needs to create a variable of type SharedPreferences as shown in Code Snippet 1.

Code Snippet 1:

```
public class PreferencesActivity extends Activity {

SharedPreferences prefs;

String prefName = "MyPref";

…

}
```

In the Code Snippet, a string variable is also created to store the name of the preference file that will be used later by the SharedPreferences object.

Following methods can be used to obtain an instance of the SharedPreference class:

- **getSharedPreferences()** - Allows the developer to obtain the reference of multiple preference files.
- **getPreference()** - Allows the developer to obtain the reference of one preference file for the Activity class.

Thus, before using the SharedPreferences object, use the getSharedPreferences() method to obtain an instance of the SharedPreferences object, passing it the name of the preference file, as well as the operating mode that is MODE_PRIVATE as shown in Code Snippet 2.

Code Snippet 2:

```
//---get the SharedPreferences object---
prefs = getSharedPreferences(prefName, MODE_PRIVATE);
```

Once, an object of the SharedPreferences class is created, the developer needs to save some values into the SharedPreferences object. For this, the developer needs to create an instance of the SharedPreferences.Editor class so that the values can be added in batches. This is done by invoking the edit() method on the object of the SharedPreferences class as shown in Code Snippet 3.

Code Snippet 3:

```
SharedPreferences.Editor editor = prefs.edit();
```

Next, the developer can add key/value pairs to the Editor object by invoking its various methods such as putBoolean(), putString(), putLong(), putInt(), or putFloat() (which method to use depends on the type of data that is being saved) as shown in Code Snippet 4.

Code Snippet 4:

```
...
//---save some values using the SharedPreferences object---
editor.putFloat("temperature", 85);
editor.putBoolean("authenticated", true);
editor.putString("username", "Wei-Meng Lee");
...
```

Finally, the commit() method of the Editor class is invoked to commit the changes back to the SharedPreferences object so that the values can be saved to persistent storage. Code Snippet 5 displays the use of commit() method.

Code Snippet 5:

```
…
//---saves the values---
editor.commit();
…
```

Retrieving data from the SharedPreferences object is similar. First, one needs to obtain an instance of the SharedPreferences object and then, use the various methods (depending on the type of data to be retrieved) to retrieve the values based on the keys as shown in Code Snippet 6.

Code Snippet 6:

```
…
prefs = getSharedPreferences(prefName, MODE_PRIVATE);
float temperature = prefs.getFloat("temperature", 50);
boolean authenticated = prefs.getBoolean("authenticated",
false);
String username = prefs.getString("username", "");
…
```

The file will be saved under the directory data/data/<application_package-name>/shared preference folder. This can be viewed using DDMS perspective.

7.3 External Storage

External storage is another type of storage system in Android. The word external signifies that it is outside and all Android-compatible devices support a shared 'external storage' which can be used to save files. This can be a removable storage media (SD card). Files which are saved to the external storage are readable by everyone and can be modified by the user. The modification can only be performed when the developer enables the USB mass storage to transfer files on a computer.

Various advantages of external storage are as follows:

- External storage will be extending the internal storage space of the Android device so that more apps can be stored.
- External Storage stores all the data in the SD card of a particular Android device and the file which is stored in the SD card can be retrieved and the data which is stored in that particular SD card can be used in some other device.

Various disadvantages of external storage are as follows:

- If the developer has put a few set of apps on SD card of the Android device, then on removal of the SD card, the apps are no longer available for use until and unless the developer replaces back the SD card.
- It will take long time for your device to boot-up or shut down.
- The apps placed on the SD card require frequent update to support the functionality.

7.4 Internal Storage

As discussed in internal storage, the developer saves the files directly on to the device's internal storage. Also, by default the files, which are saved to internal storage are private to that particular application. Any other application cannot access them (nor can the user). Once the user uninstalls the application, the files are removed.

Using an internal file to persist data is simple as the developer can save and retrieve the data using the Android.content.Context's openFileOutput() and openFileInput() method, which will return an object of java.io.FileOutputStream and a java.io.FileInputStream class respectively. Using the object of the FileOutputStream and FileInputStream class, Java file I/O operations can be performed on the data. The topic is covered in detail in section 3.3.2.

To create and write to a private file present in the internal storage, perform the following tasks:

1. Invoke openFileOutput() method with the file name and the operating mode. This returns an object of FileOutputStream class.
2. Write to the file by invoking the write() method.
3. Close the stream invoking the close() method.

   Code Snippet 7 demonstrates the use of FileOutputStream class.

Code Snippet 7:

```
…
String FILENAME = "android_file_save";// saving into file
String string = "Hello Internal Storage";
FileOutputStream fos = context.openFileOutput(FILENAME, Context.
MODE_PRIVATE);
// write I/O using write() and close()
fos.write(string.getBytes());
fos.close();
…
```

In the Code Snippet, the line Context.MODE_PRIVATE helps in creating a file or replacing the file of the same name and also, makes it private to the application. It means that our internal files are only visible to us. There are other modes, such as MODE_APPEND, MODE_WORLD_READABLE, and MODE_WORLD_WRITABLE which are self-explanatory. MODE_APPEND is used when data is required to be added at the end of the file, otherwise it will overwrite the file every time something is written to it.

To read a file from internal storage, perform the following tasks:

1. Invoke the openFileInput() method and pass the name of the file to read as its argument. This returns an object of FileInputStream() class.
2. Read bytes from the file by invoking the read() method.
3. Then, close the stream by invoking the close() method.

The absolute path of saved file can be retrieved by invoking the getFilesDir() method of the Context class. The fileList() method will return an array of files saved by the application.

These internal files are deleted once the application itself is removed by the user. In case the developer would like to delete an internal file, in the course of application workflow then the deleteFile() method of the Context class can be used.

As it can be understood that the internal file storage API is provided by the Context class, which basically provides access to our application environment.

Various advantages of internal storage are as follows:

- In this type of storage, the developer will be storing the apps or the data directly on to your device.
- All the data and apps saved in the device will be present in the device only.

Various disadvantages of internal storage are as follows:

- When the device is very low on internal storage space, then Android might delete the cache files to recover space.
- Once the internal memory is full then the device will send an alert informing about memory full and incoming SMS will be rejected.

7.5 SQLite Database

Android uses SQLite database to store relational data. SQLite is a program provided by Android to execute services, such as creating tables and databases, amending rows, executing queries, and so on. The SQLite database created for an Android application is private and cannot be accessed by other applications. For example, if you want to store the results of the students in your school, it is much more efficient to use a database to represent them because you can use the database for querying and retrieve the results according to the specific requirement. SQLite as a database is reliable and is a part of the Android stack.

To develop a database application, the following points are to be noted:

- To create a database for your Android App, the developer needs to use the Android API package such as android.database.sqlite.
- The developer needs to create a class which is a subclass of android.database. sqlite. SQLiteOpenHelper class. This is because this class will enable the developer to create, open, close, and upgrade the database.
- The developer needs to import android.database.sqliteSQLiteDataBase class. Using this class instance and its properties the developer can execute the SQL statements to perform the CRUD operations such as create(insert), retrieve, update, and delete.

7.5.1 Storing and Retrieving Data

Storage and retrieval of data is one of the most important tasks that are performed by a database operator. It is the most important characteristics when it comes to database perspective. SQLite queries can be executed by using the query() method present in the SQLite database. The query() method will return a Cursor object that points to all the retrieved rows. In other words, it helps by caching the result of query. It also provides functions with the help of which one can navigate to the desired row and obtain the data from the specified row and columns.

7.5.2 Adding, Modifying, Searching, and Removing SQLite Database

For Android, SQLite is 'baked into' the Android runtime, so every Android database based application can create tables in SQLite databases. The databases created in SQLite will be accessible to the classes present in the application and not to the classes present outside the application.

Android provides its own API to deal with database connectivity. This API is provided in android.database and android.database.sqlite packages. In this package the most important classes are as follows:

1. SQLiteOpenHelper
2. SQLiteDatabase
3. Cursor
4. QLException

- **SQLiteOpenHelper**:

  The main functionality of the class is to open the database if it exists, create it if it does not, and upgrade the version as required. It provides a constructor to construct a helper class by subclassing this class and overriding the methods named onCreate() an onUpgrade().

The syntax for the constructor is as follows:

Syntax:

```
SQLiteOpenHelper (Context context, String name, SQLiteDatabase.
CursorFactory factory, int version)
```

where,

context – represent the context to create or open the database.

name – represents the name of the database.

factory – represents the factory class used for creating the cursor object.

version – represent the number of the database.

Some of the commonly used methods of this class are as follows:

- **`onCreate (SQLiteDatabase db)`**

  The method is invoked when the database is created for the first time. This is where the table is created and populated. The database name is passed as an argument.

- **`onUpgrade (SQLiteDatabase db, int oldVersion, int newVersion)`**

  The method is invoked when the database needs to be upgraded. The implementation should use this method to drop tables, add tables, or perform any other operations such as the need to upgrade to the new schema version.

- **SQLiteDatabase**:

The class has methods to create, delete, execute SQL commands, and perform other common database management tasks. Some of the commonly used methods of this class are as follows:

- **`execSQL (String sql):`**

  This method is used to execute a single SQL statement that is not a SELECT statement or any other SQL statement that returns data. It is used generally for creating tables. The sql statement to be executed is passed as an argument.

- **`long insert (String table, String nullColumnHack, ContentValues values):`**

  It is a convenience method used for inserting a row into the database. The method returns the row ID of the newly inserted row or -1 if an error takes place. The method accepts the table name and the initial column values for the row.

- **`int update (String table, ContentValues values, String whereClause, String[] whereArgs):`**

  It is a convenience method used for updating rows in the database. The method returns the number of rows affected. It accepts the table name, new column values, and an optional where clause.

- **`int delete (String table, String whereClause, String[] whereArgs):`**

  It is a convenience method used for deleting rows in the database. The method returns the number of rows affected if a where clause is passed, otherwise it returns 0.

- **`Cursor query (String table, String[] columns, String selection, String[] selectionArgs, String groupBy, String having, String orderBy):`**

  The method queries the given table and returns a Cursor over the result set.

☐ **Cursor**:

This interface provides random read-write access methods to the result set returned by a database query. Table 7.1 lists some of the methods present in Cursor class.

Methods	Description
public int getColumnIndex(String colName)	Returns the index of the indicated column as the result.
public String getColumnName(int colIndex)	Returns the name of the specified column as the result.
public abstract String[] getColumnNames()	Returns the column names as a string array.
public int getColumnIndexOrThrow (String colName)	Returns the index of a column. The concept of throwing an exception arises when there is no existence of the column with the indicated name.
public abstract int getCount()	Returns the count of number of rows.
public final int getPosition()	Returns the present cursor position in the result set.
public final boolean moveToPosition (int pos)	Moves the cursor towards the desired row in the result set and returns true, if the required record exists.
public final boolean moveToFirst()	Moves the cursor to the first row in the queried result set and returns false, if there are no records in the result set.
public final boolean moveToNext()	Moves the cursor to the subsequent rows in the result set and returns false, if the cursor is after the last record.
public final boolean moveToPrevious()	Moves the cursor to the previous rows in the result set and returns false if the cursor is before the first record.

Table 7.1: Methods of Cursor

Besides the methods mentioned, the Cursor interface has getXXX() style of methods to retrieve XXX type of values. Some of the commonly used methods of this type are as follows:

- getString(int columnindex)
- getLong(int columnindex)
- getInt(int columnindex)
- getFloat(int columnindex)
- getDouble(int columnindex)

7.5.3 Working with Databases

As we know content provider manages access to a central repository of data. This section describes how to create an SQLite database, store the book details, and retrieve it from the database. The steps to create the application are as follows:

1. Create a project named MyAndroidDatabaseApp in Android Studio.
2. Navigate to src → com.example.myandroiddatabaseapp folder.
3. Create a new class named DBController.
4. Modify the code in DBController file as shown in Code Snippet 8. This class takes care of all the CRUD (Create, Retrieve, Update, Delete) operations.

Code Snippet 8:

```
package com.example.myandroiddatabaseapp;
import java.util.ArrayList;
import java.util.HashMap;
import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;
import android.util.Log;
public class DBController extends SQLiteOpenHelper {
private static final String LOGCAT = null;
public DBController(Context applicationcontext) {
      super(applicationcontext, "androidsqlite.db", null, 1);
```

```
    Log.d(LOGCAT, "Created");
}
@Override
public void onCreate(SQLiteDatabase database) {
      String query;
     query = "CREATE TABLE books ( bookId INTEGER PRIMARY KEY,
bookName TEXT)";
      database.execSQL(query);
      Log.d(LOGCAT, " books Created");
}
@Override
public void onUpgrade(SQLiteDatabase database, int version_old,
int current_version) {
      String query;
      query = "DROP TABLE IF EXISTS books";
      database.execSQL(query);
      onCreate(database);
}
public void insertBook(HashMap<String, String> queryValues) {
      SQLiteDatabase database = this.getWritableDatabase();
      ContentValues values = new ContentValues();
      values.put("bookName", queryValues.get("bookName"));
      database.insert("books", null, values);
      database.close();
}
public int updateBook(HashMap<String, String> queryValues) {
      SQLiteDatabase database = this.getWritableDatabase();
      ContentValues values = new ContentValues();
      values.put("bookName", queryValues.get("bookName"));
      returndatabase.update("books", values, "bookId" + "=?",
new String[] { queryValues.get("bookId") });
```

```
// String updateQuery =
// "Update words set txtWord='"+word+"' where txtWord='"+
oldWord +"'";
// Log.d(LOGCAT,updateQuery);
// database.rawQuery(updateQuery, null);
// return database.update("words", values, "txtWord = ?", new
String[]
// { word });
}
public void deleteBook(String id) {
Log.d(LOGCAT, "delete");
SQLiteDatabase database = this.getWritableDatabase();
String deleteQuery = "DELETE FROM books where bookId= '" + id +
"'";
Log.d("query", deleteQuery);
database.execSQL(deleteQuery);
}
public ArrayList<HashMap<String, String>> getAllBooks() {
ArrayList<HashMap<String, String>> wordList;
wordList = new ArrayList<HashMap<String, String>>();
String selectQuery = "SELECT * FROM books";
SQLiteDatabase database = this.getWritableDatabase();
Cursor cursor = database.rawQuery(selectQuery, null);
if (cursor.moveToFirst()) {
      do {
            HashMap<String, String> map = new HashMap<String,
String>();
            map.put("bookId", cursor.getString(0));
            map.put("bookName", cursor.getString(1));
wordList.add(map);
```

```
} while (cursor.moveToNext());
}
// return contact list
return wordList;
}
public HashMap<String, String> getBookInfo(String id) {
HashMap<String, String> wordList = new HashMap<String, String>();
SQLiteDatabase database = this.getReadableDatabase();
String selectQuery = "SELECT * FROM books where bookId='" + id +
"'";
Cursor cursor = database.rawQuery(selectQuery, null);
if (cursor.moveToFirst()) {
      do {
            // HashMap<String, String> map = new HashMap<String,
String>();
            wordList.put("bookName", cursor. getString(1));
            // wordList.add(map);
      } while (cursor.moveToNext());
}
return wordList;
}
}
```

5. Navigate to res → layout folder.
6. Modify the code in activity_main.xml file as shown in Code Snippet 9.

Code Snippet 9:

```
<RelativeLayout xmlns:android="http://schemas.android.com/ apk/
res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:paddingBottom="@dimen/activity_vertical_margin"
```

```
android:paddingLeft="@dimen/activity_horizontal_margin"
android:paddingRight="@dimen/activity_horizontal_ margin"
android:paddingTop="@dimen/activity_vertical_margin"
tools:context=".MainActivity" >
<RelativeLayout
      android:id="@+id/relativeLayout1"
      android:layout_width="fill_parent"
      android:layout_height="40dp"
      android:background="#000000"
      android:orientation="vertical" >
<TextView
      android:id="@+id/textView1"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_marginLeft="5dp"
      android:text="Books"
      android:textAppearance="?android:attr/ textAppearanceLarge"
      android:textColor="#FFFFFF" />
<Button
      android:id="@+id/button1"
      android:layout_width="41dp"
      android:layout_height="40dp"
      android:layout_alignParentRight="true"
      android:layout_alignParentTop="true"
      android:background="#454545"
      android:onClick="showAddForm"
      android:text="+"
      android:textColor="#FFFFFF"
```

```
      android:textSize="30sp" />
</RelativeLayout>
<RelativeLayout
      android:id="@+id/relativeLayout1"
      android:layout_width="fill_parent"
      android:layout_height="match_parent"
      android:layout_alignParentLeft="true"
      android:layout_below="@+id/relativeLayout1"
      android:orientation="vertical"
      android:layout_marginTop="40dp">
<ListView
      android:id="@android:id/list"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:layout_alignParentLeft="true">
</ListView>
</RelativeLayout>
</RelativeLayout>
```

7. Navigate to src → com.example.myandroiddatabaseapp.
8. Modify the code in MainActivity file as shown in Code Snippet 10.

Code Snippet 10:

```
package com.example.myandroiddatabaseapp;
import java.util.ArrayList;
import java.util.HashMap;
import android.os.Bundle;
import android.app.ListActivity;
import android.content.Intent;
import android.view.View;
```

```
import android.widget.AdapterView;
import android.widget.ListAdapter;
import android.widget.ListView;
import android.widget.SimpleAdapter;
import android.widget.TextView;
import android.widget.AdapterView.OnItemClickListener;
public class MainActivity extends ListActivity {
Intent intent;
TextView bookId;
DBController controller = new DBController(this);
@Override
protected void onCreate(Bundle savedInstanceState) {
            super.onCreate(savedInstanceState);
            setContentView(R.layout.activity_main);
      ArrayList<HashMap<String, String>> bookList = controller.
getAllBooks();
      if(bookList.size()!=0) {
            ListView lv = getListView();
lv.setOnItemClickListener(new OnItemClickListener() {
             @Override
             public void onItemClick(AdapterView<?> parent, View
view,int position, long id) {
      bookId = (TextView) view.findViewById(R. id. bookId);
      String valBookId = bookId.getText(). toString();
Intent objIndent = new Intent(getApplicationContext(),EditBook.
class);
                  objIndent.putExtra("bookId", valBookId);
                  startActivity(objIndent);
                  }
```

```
            });

      ListAdapter adapter = new SimpleAdapter ( MainActivity.
this, bookList, R.layout.view_ book_entry, new String[] {
"bookId","bookName"}, new int[] {R.id. bookId, R.id. bookName});

      setListAdapter(adapter);

      }

}

public void showAddForm(View view) {

Intent objIntent = new Intent(getApplicationContext(), NewBook.
class);

startActivity(objIntent);

}

}
```

9. Navigate to res → layout folder.
10. Create a new Android layout file named view_book_entry.xml.
11. Modify the code in view_book_entry.xml file as shown in Code Snippet 11.

Code Snippet 11:

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout

xmlns:android="http://schemas.android.com/apk/ res/android"

android:layout_width="fill_parent"

android:layout_height="wrap_content"

android:orientation="vertical" >

<TextView

      android:id="@+id/bookId"

      android:layout_width="fill_parent"

      android:layout_height="wrap_content"

      android:visibility="gone" />

<TextView
```

```
      android:id="@+id/bookName"
      android:layout_width="fill_parent"
      android:layout_height="wrap_content"
      android:layout_marginBottom="5dp"
      android:layout_marginTop="5dp"
      android:paddingLeft="6dip"
      android:paddingTop="6dip"
      android:textColor="#A4C739"
      android:textSize="17sp"
      android:textStyle="bold" />
</LinearLayout>
```

12. Navigate to res → layout folder.
13. Create a new Android layout file named add_new_book.xml.
14. Modify the code in add_new_book.xml file as shown in Code Snippet 12.

Code Snippet 12:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/ res/android"
     android:layout_width="match_parent"
     android:layout_height="match_parent"
     android:background="#CCCCCC"
     android:orientation="vertical"
     android:paddingTop="1dp" >
<TextView
      android:id="@+id/textView1"
      android:layout_width="match_parent"
      android:layout_height="wrap_content
```

```
      android:background="#000000"
      android:padding="5dp"
      android:text="Add Book"
      android:textAppearance="?android:attr/ textAppearanceLarge"
      android:textColor="#FFFFFF" />
<RelativeLayout
      android:layout_width="match_parent"
      android:layout_height="match_parent"
      android:background="#FFFFFF"
      android:orientation="vertical"
      android:padding="10dp" >
<TextView
      android:id="@+id/textView1"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_alignParentLeft="true"
      android:layout_alignParentTop="true"
      android:layout_marginLeft="24dp"
      android:layout_marginTop="30dp"
      android:text="Book" />
<EditText
      android:id="@+id/bookName"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_alignLeft="@+id/textView1"
      android:layout_below="@+id/textView1"
      android:ems="10" >
      <requestFocus />
```

```
</EditText>
<Button
      android:id="@+id/btnadd"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_alignLeft="@+id/bookName"
      android:layout_below="@+id/bookName"
      android:layout_marginTop="32dp"
      android:onClick="addNewBook"
      android:text="Save" />
</RelativeLayout>
</LinearLayout>
```

15. Navigate to src → com.example.myandroiddatabaseapp.
16. Create a new class named NewBook.
17. Modify the code in NewBook file as shown in Code Snippet 13.

Code Snippet 13:

```
package com.example.myandroiddatabaseapp;
import java.util.HashMap;
import com.example.myandroiddatabaseapp.DBController;
import android.app.Activity;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.EditText;
public class NewBook extends Activity {
EditText bookName;
DBController controller = new DBController(this);
@Override
```

```
public void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.add_new_book);
      bookName = (EditText) findViewById(R.id.bookName);
}
public void addNewBook(View view) {
      HashMap<String, String> queryValues = new HashMap<String,
String>();
      queryValues.put("bookName", bookName.getText(). toString());
      controller.insertBook(queryValues);
      this.callHomeActivity(view);
}
public void callHomeActivity(View view) {
      Intent objIntent = new Intent(getApplicationContext(),
      MainActivity.class);
      startActivity(objIntent);
}
}
```

18. Navigate to res → layout folder.
19. Create a new Android layout file named activity_edit_book.xml.
20. Modify the code in activity_edit_book.xml file as shown in Code Snippet 14.

Code Snippet 14:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/ res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:background="#CCCCCC"
android:orientation="vertical"
```

```
      android:paddingTop="1dp" >
<TextView
      android:id="@+id/textView1"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:background="#000000"
      android:padding="5dp"
      android:text="Edit Book"
      android:textAppearance="?android:attr/ textAppearanceLarge"
            android:textColor="#FFFFFF" />
<RelativeLayout
      android:layout_width="match_parent"
      android:layout_height="match_parent"
      android:background="#FFFFFF"
      android:orientation="vertical"
      android:padding="10dp" >
<TextView
      android:id="@+id/textView2"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_alignParentLeft="true"
      android:layout_alignParentTop="true"
      android:layout_marginLeft="24dp"
      android:layout_marginTop="30dp"
      android:text="Task" />
<EditText
      android:id="@+id/bookName"
      android:layout_width="wrap_content"
```

```
      android:layout_height="wrap_content"
      android:layout_alignLeft="@+id/textView2"
      android:layout_below="@+id/textView2"
      android:ems="10" >
<requestFocus />
</EditText>
<Button
      android:id="@+id/button1"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_alignLeft="@+id/bookName"
      android:layout_below="@+id/bookName"
      android:layout_marginTop="19dp"
      android:onClick="editBook"
      android:text="Edit" />
<Button
      android:id="@+id/button2"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_alignBaseline="@+id/button1"
      android:layout_alignBottom="@+id/button1"
      android:layout_centerHorizontal="true"
      android:onClick="removeBook"
      android:text="Delete" />
</RelativeLayout>
</LinearLayout>
```

21. Navigate to src → com.example.myandroiddatabaseapp.
22. Create a new class named EditBook.

23. Modify the code in EditBook file as shown in Code Snippet 15.

Code Snippet 15:

```
package com.example.myandroiddatabaseapp;

import java.util.HashMap;

import android.app.Activity;

import android.content.Intent;

import android.os.Bundle;

import android.util.Log;

import android.view.View;

import android.widget.EditText;

public class EditBook extends Activity {
     EditText bookName;

     DBController controller = new DBController(this);

@Override

public void onCreate(Bundle savedInstanceState) {

      super.onCreate(savedInstanceState);

      setContentView(R.layout.activity_edit_book);

      bookName = (EditText) findViewById(R.id.bookName);

      Intent objIntent = getIntent();

      String bookId = objIntent.getStringExtra("bookId");

      Log.d("Reading: ", "Reading all contacts..");

   HashMap<String, String> bookList = controller.
                                        getBookInfo(bookId);

      Log.d("bookName", bookList.get("bookName"));

      if (bookList.size() != 0) {

            bookName.setText(bookList.get("bookName"));

      }

}

public void editBook(View view) {
```

```
      HashMap<String, String> queryValues = new HashMap<String,
                                              String>();
      bookName = (EditText) findViewById(R.id.bookName);
      Intent objIntent = getIntent();
      String bookId = objIntent.getStringExtra("bookId");
      queryValues.put("bookId", bookId);
     queryValues.put("bookName", bookName.getText().toString());
      controller.updateBook(queryValues);
      this.callHomeActivity(view);
}
public void removeBook(View view) {
      Intent objIntent = getIntent();
      String bookId = objIntent.getStringExtra("bookId");
      controller.deleteBook(bookId);
      this.callHomeActivity(view);
}
public void callHomeActivity(View view) {
      Intent objIntent = new Intent(getApplicationContext(),
      MainActivity.class);
      startActivity(objIntent);
}
}
```

24. Modify the code in AndroidManifest.xml file to add the code before the </ application> tag as shown in Code Snippet 16.

Code Snippet 16:

```
<activity android:name=".NewBook" >
</activity>
<activity android:name=".EditBook" >
</activity>
```

Figure 7.1 displays the output when the application is executed.

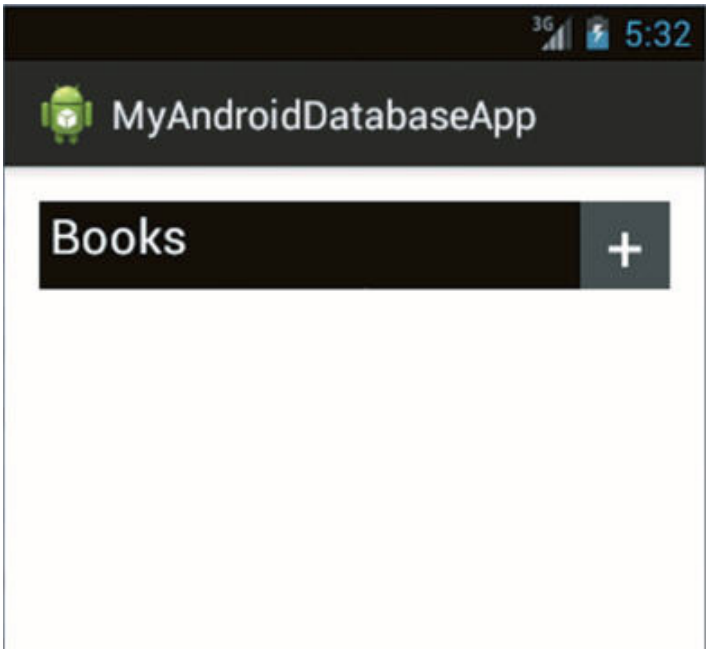


Figure 7.1: Book Database - Output

Figure 7.2 displays the output when the plus symbol on the upper right corner is clicked.

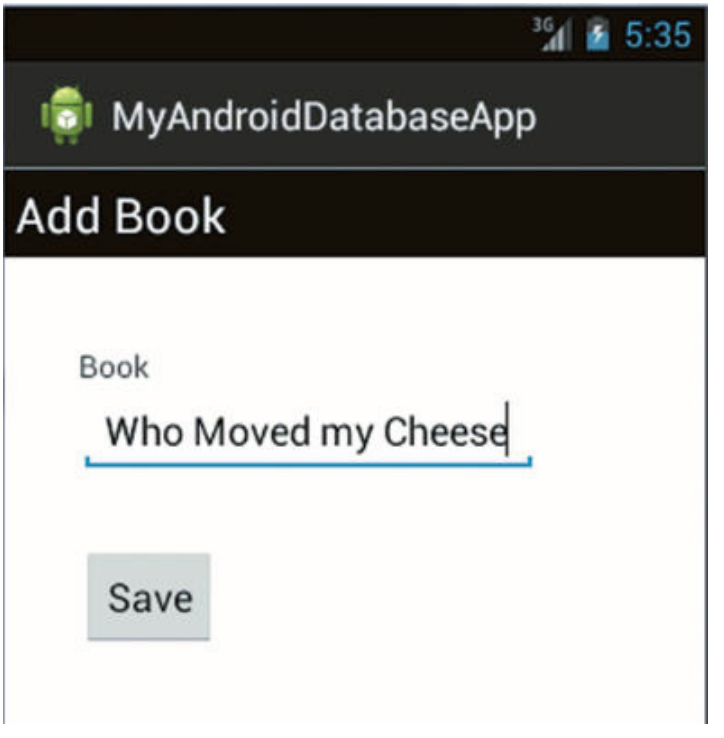


Figure 7.2: Adding Book to the Book Database

Figure 7.3 displays the output when the user clicks Save and the book name gets added to the Book database.

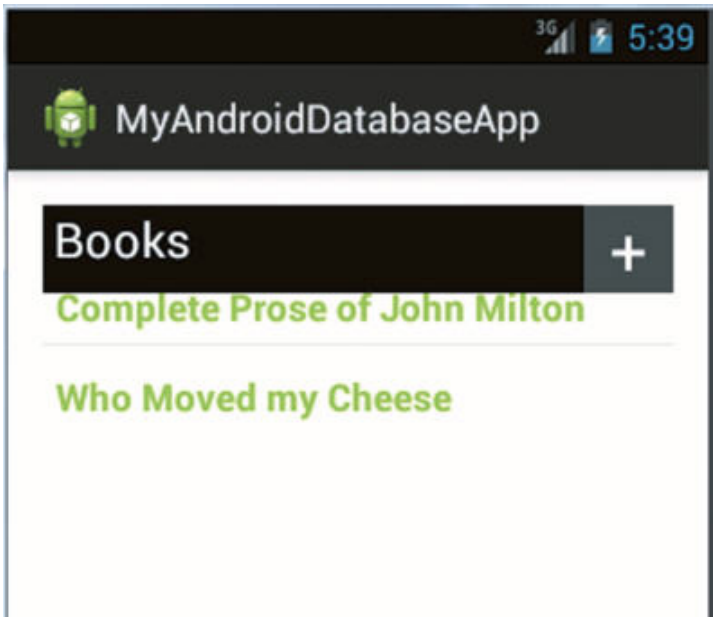


Figure 7.3: Book Added to the Database

Figure 7.4 displays the output when the book is selected to display the Edit Book pane.

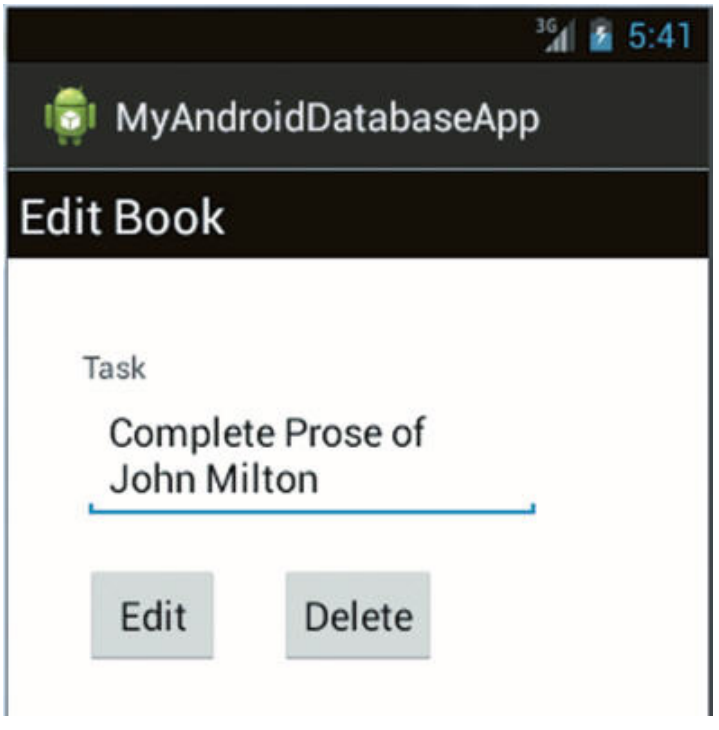


Figure 7.4: Edit Book

Figure 7.5 displays the output after the book name has been edited in the Edit Book pane.

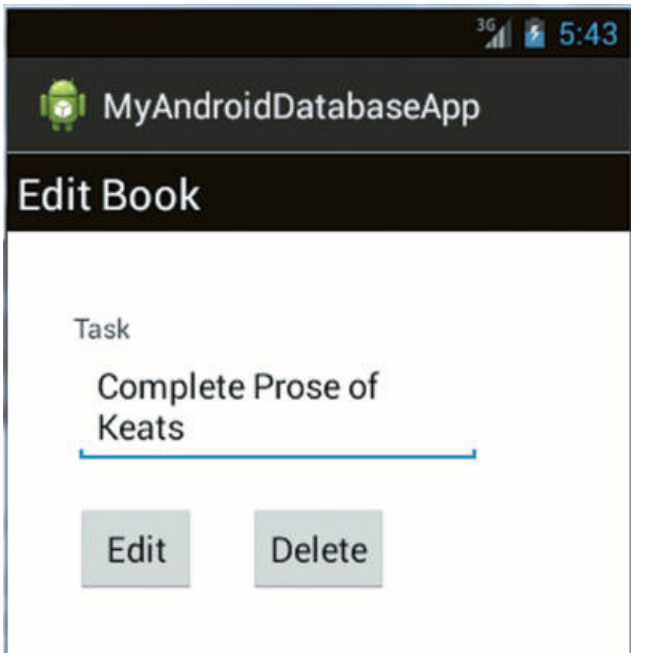


Figure 7.5: Edited Book Name

Figure 7.6 displays the output when the user has clicked Edit.

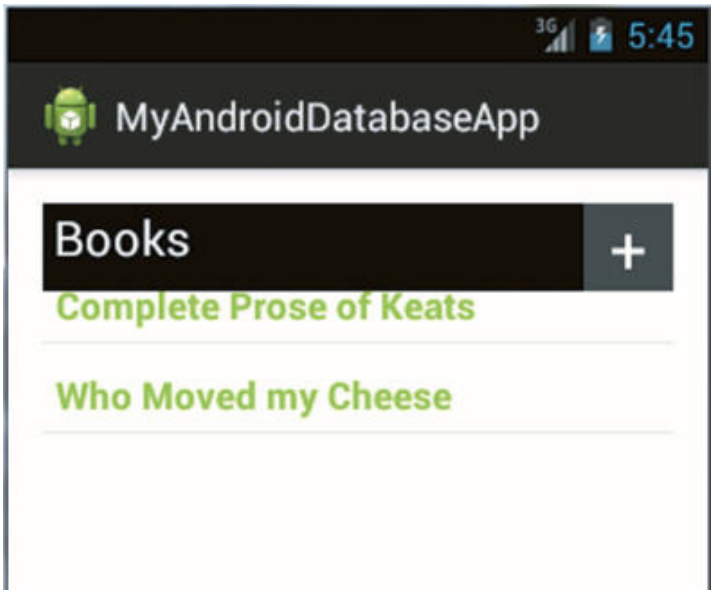


Figure 7.6: Edited BookList

7.5.4 Revisiting the Contacts Application

With the knowledge of databases, one can create and extend the Contacts application to support database storage for the contacts instead of memory. This will ensure the contacts are retained even after the application is closed.

1. Create a Contacts application project in Android Studio.
2. Navigate to app → java → aptech.com.contactsapplication.helpers and create a new class called DataBaseManager.
3. Modify the code as shown in Code Snippet 17.

Code Snippet 17:

```
package aptech.com.contactsapplication.helpers;
import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;
import java.util.ArrayList;
import java.util.List;
public class DatabaseManager {
    Context context;
public DatabaseManager(Context context)    {
    this.context=context;
    }
    private class DBHelper extends SQLiteOpenHelper {
    public static final int DATABASE_VERSION = 1;
        public static final String DATABASE_NAME = "AptechContacts.db";
       public DBHelper(Context context) {
            super(context, DATABASE_NAME, null, DATABASE_
                                                VERSION);
       }
```

```
        @Override
        public void onCreate(SQLiteDatabase db) {
            db.execSQL("CREATE TABLE contacts " +
                    "(" +
                    "id INT AUTOINCREMENT PRIMARY KEY" +
                    "name TEXT " +
                    "phone TEXT" +
                    "nick TEXT" +
                    "address TEXT" +
                    "notes TEXT" +
                    ");");
        }
        @Override
        public void onUpgrade(SQLiteDatabase db, int oldVersion,
                                              int newVersion) {
            db.execSQL("DROP TABLE IF EXISTS contacts");
            onCreate(db);
        }
    }
DBHelper dbHelper = new DBHelper(context);
 public void addContact(Contact contact){
        SQLiteDatabase db = dbHelper.getWritableDatabase();
        ContentValues values = new ContentValues();
        values.put("name", contact.getName());
        values.put("phone",contact.getNumber());
        values.put("nick",contact.getNickName());
        values.put("address",contact.getAddress());
        values.put("notes", contact.getNotes());
```

```
        db.insert("contacts",null,values);
    }
    public Contact getContact(int id){
        SQLiteDatabase db = dbHelper.getReadableDatabase();
        Cursor cursor = db.rawQuery("SELECT * FROM contacts
                                    WHERE id=" + id + "", null);
        cursor.moveToFirst();
        String name = cursor.getString(1);
        String number = cursor.getString(2);
        String nickName = cursor.getString(3);
        String address = cursor.getString(4);
        String notes = cursor.getString(5);
        Contact contact = new  Contact(id,name,number,
                               nickName,address,notes);
        return  contact;
    }
    public List<Contact> getAllContacts(){
        List<Contact> contactList = new ArrayList<Contact>();
        SQLiteDatabase db = dbHelper.getReadableDatabase();
        Cursor cursor = db.rawQuery("SELECT * FROM contacts",
                                                  null);
       while(cursor.moveToNext())
        {
            int id = cursor.getInt(0);
            String name = cursor.getString(1);
            String number = cursor.getString(2);
            String nickName = cursor.getString(3);
            String address = cursor.getString(4);
```

```
            String notes = cursor.getString(5);
            Contact contact = new Contact(id, name, number,
                                    nickName, address, notes);
            contactList.add(contact);
        }
        return  contactList;
    }
    public void saveContact(int id, Contact contact)
    {
        String selectionCriteria = "id = "+Integer.toString(id);
        SQLiteDatabase db = dbHelper.getWritableDatabase();
        ContentValues values = new ContentValues();
        values.put("name",contact.getName());
        values.put("phone",contact.getNumber());
        values.put("nick",contact.getNickName());
        values.put("address",contact.getAddress());
        values.put("notes", contact.getNotes());
        db.update("contacts",values,selectionCriteria,null);
    }
}
```

The DatabaseManager class is responsible for interacting with the database. It exposes a simple interface for use with the rest of the application.

4. Navigate to aptech.com.contactsapplication and open MainActivity.java for editing.
5. Modify the code as shown in Code Snippet 18.

Code Snippet 18:

```
package aptech.com.contactsapplication;
import android.app.Activity;
import android.content.Intent;
```

```
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
public class MainActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if
it is present.
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
    // Handle action bar item clicks here. The action bar will
    // automatically handle clicks on the Home/Up button, so long
  // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
       //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
```

```
    public void loadNewContactActivity(MenuItem item) {
      Intent intent = new Intent(MainActivity.this,
                                                NewContactclass);
        startActivity(intent);
    }
    public void loadContactListActivity(MenuItem item) {
        Intent intent = new Intent(MainActivity.this,
                                              ContactListclass);
        startActivity(intent);
    }
}
```

6. Create a new Activity called ContactList and open ContactList.java for editing.
7. Modify the code as shown in Code Snippet 19.

Code Snippet 19:

```
package aptech.com.contactsapplication;
import android.app.Activity;
import android.content.Intent;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.AdapterView;
import android.widget.AdapterView.OnItemClickListener;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import helpers.Contact;
import helpers.DatabaseManager;
public class ContactList extends Activity {
    DatabaseManager manager;
```

```
    private ListView listView;
    private OnItemClickListener listViewListener = new
                                        OnItemClickListener() {
        public void onItemClick(AdapterView<?> parent, View
                                view, int position, long id) {
        Contact  contact = (Contact) listView.
                                getItemAtPosition(position);
      Intent intent = new Intent(ContactList.this, EditContact.
                                                    class);
            intent.putExtra("id",contact.getId() );
            startActivity(intent);
        }
    };
   @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_contact_list);
        manager = new DatabaseManager(this);
        listView=(ListView)findViewById(R.id.listView);
        ArrayAdapter<Contact> adapter = new
ArrayAdapter<Contact>(this,android.R.layout.simple_list_item_1,
android.R.id.text1, manager.getAllContacts());
       listView.setAdapter(adapter);
       listView.setOnItemClickListener(listViewListener);
    }
   @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if
it is present.
        getMenuInflater().inflate(R.menu.menu_contact_list, menu);
```

```
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
   // Handle action bar item clicks here. The action bar will
  // automatically handle clicks on the Home/Up button, so long
   // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
    public void loadHomePage(MenuItem item) {
        Intent intent = new Intent(ContactList.this,
MainActivity.class);
        startActivity(intent);
    }
   public void loadNewContactActivity(MenuItem item) {
        Intent intent = new Intent(ContactList.this, NewContact.
class);
        startActivity(intent);
   }
}
```

ContactList now uses an instance of DataBaseManager to retrieve the list of contacts stored in the database.

8. Create a new Activity called EditContact and open EditContact.java for editing.
9. Modify the code as shown in Code Snippet 20.

Code Snippet 20:

```
package aptech.com.contactsapplication;
import android.app.Activity;
import android.content.Intent;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;
import helpers.Contact;
import helpers.DatabaseManager;
public class EditContact extends Activity {
   int id;
    Contact contact;
    DatabaseManager manager;
   @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_edit_contact);
        id = getIntent().getIntExtra("id", 0);
        manager = new DatabaseManager(this);
        contact = manager.getContact(id);
        initContactDetails();
    }
    private void initContactDetails() {
```

```
        EditText name = (EditText)findViewById(R.id.editText);
        EditText number = (EditText)findViewById(R.id.editText2);
        EditText nickName = (EditText)findViewById(R.id.editText3);
        EditText address = (EditText)findViewById(R.id.editText4);
        EditText notes = (EditText)findViewById(R.id.editText5);
        contact = manager.getContact(id);
        name.setText(contact.getName());
        number.setText(contact.getNumber());
        nickName.setText(contact.getNickName());
        address.setText(contact.getAddress());
        notes.setText(contact.getNotes());
        disableEditText(name);
        disableEditText(number);
        disableEditText(nickName);
        disableEditText(address);
        disableEditText(notes);
        Button saveButton = (Button)findViewById(R.id.button);
        saveButton.setVisibility(View.GONE);
    }
   private void disableEditText(EditText editText) {
        editText.setFocusable(false);
        editText.setEnabled(false);
        editText.setCursorVisible(false);
    }
    private void enableEditText(EditText editText) {
        editText.setFocusable(true);
        editText.setEnabled(true);
        editText.setCursorVisible(true);
    }
```

```
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
  // Inflate the menu; this adds items to the action bar if
  //it is present.
  getMenuInflater().inflate(R.menu.menu_edit_contact, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
// Handle action bar item clicks here. The action bar will
// automatically handle clicks on the Home/Up button, so long
// as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
   public void makeContactEditable(MenuItem item) {
       EditText name = (EditText)findViewById(R.id.editText);
       EditText number = (EditText)findViewById(R.id.editText2);
       EditText nickName = (EditText)findViewById(R.
                                          id.editText3);
       EditText address = (EditText)findViewById(R.
                                         id.editText4);
       EditText notes = (EditText)findViewById(R.id.editText5);
       enableEditText(name);
       enableEditText(number);
       enableEditText(nickName);
```

```
        enableEditText(address);
        enableEditText(notes);
        Button saveButton = (Button)findViewById(R.id.button);
        saveButton.setVisibility(View.VISIBLE);
    }
    public void saveContact(View view) {
      String name = ((EditText)findViewById(R.id.editText)).
                                          getText().toString();
      String number = ((EditText)findViewById(R.id.editText2)).
                                          getText().toString();
      String nickName = ((EditText)findViewById(R.id.editText3)).
                                          getText().toString();
      String address = ((EditText)findViewById(R.id.editText4)).
                                          getText().toString();
      String notes = ((EditText)findViewById(R.id.editText5)).
                                          getText().toString();
      Contact contact = new Contact(id,name,number,
                                    nickName,address,notes);
      manager.saveContact(id,contact);
      initContactDetails();
      Toast.makeText(getApplicationContext(), "Contact Details
                                               Saved",
               Toast.LENGTH_LONG).show();
    }
    public void loadNewContactActivity(MenuItem item) {
        Intent intent = new Intent(EditContact.this, NewContact.
                                                    class);
        startActivity(intent);
    }
    public void loadContactListActivity(MenuItem item) {
        Intent intent = new Intent(EditContact.this,
                                    ContactList. class);
```

```
        startActivity(intent);
    }
}
```

EditContact now uses an instance of DatabaseManager to store the changes in the database.

10. Create a new activity called NewContact and open NewContact.java for editing.
11. Modify the code as shown in Code Snippet 21.

Code Snippet 21:

```
package aptech.com.contactsapplication;
import android.app.Activity;
import android.app.Notification.Builder;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.app.TaskStackBuilder;
import android.content.Context;
import android.content.Intent;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import helpers.Contact;
import helpers.DatabaseManager;
public class NewContact extends Activity {
    DatabaseManager manager;
    private View.OnClickListener addButtonListener = new View.
OnClickListener() {
```

```
        public void onClick(View v) {
           String name = ((EditText)findViewById(R.id.editText)).
                                    getText().toString();
           String number = ((EditText)findViewById(R.
                            id.editText2)).getText().toString();
           String nickName = ((EditText)findViewById(R.
                            id.editText3)).getText().toString();
            String address = ((EditText)findViewById(R.
                            id.editText4)).getText().toString();
            String notes = ((EditText)findViewById(R.
                            id.editText5)).getText().toString();
            Contact contact = new Contact(0,name,number,
                                      nickName,address,notes);
            manager.addContact(contact);
            showNotification(contact.getName());
           Intent intent = new Intent(NewContact.this,
                                    ContactList.class);
            startActivity(intent);
        }
    };
    private void showNotification(String name) {
        Builder nBuilder =
                new Builder(this)
                        .setSmallIcon(R.drawable.ic_aptech_icon)
                        .setContentTitle("New Contact Added")
                       .setContentText("Contact created "+name);
      Intent resultIntent = new Intent(this, ContactList.class);
      TaskStackBuilder stackBuilder = TaskStackBuilder.
                                        create(this);
```

```
        stackBuilder.addParentStack(ContactList.class);
        stackBuilder.addNextIntent(resultIntent);
        PendingIntent resultPendingIntent =
                stackBuilder.getPendingIntent(0,PendingIntent
                                              .FLAG_UPDATE_CURRENT);
        nBuilder.setContentIntent(resultPendingIntent);
        NotificationManager mNotificationManager =
        (NotificationManager) getSystemService(Context.
                                    NOTIFICATION_SERVICE);
        mNotificationManager.notify(0, nBuilder.build());
    }
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_new_contact);
        manager = new DatabaseManager(this);
        Button button = (Button)findViewById(R.id.button);
        button.setOnClickListener(addButtonListener);
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if
it is present.
        getMenuInflater().inflate(R.menu.menu_new_contact, menu);
        return true;
    }
```

```
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
// Handle action bar item clicks here. The action bar will
// automatically handle clicks on the Home/Up button, so long
// as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
    public void loadHomePage(MenuItem item) {
        Intent intent = new Intent(NewContact.this,
                                   MainActivity.class);
        startActivity(intent);
    }
    public void loadContactListActivity(MenuItem item) {
        Intent intent = new Intent(NewContact.this, ContactList.
                                                   class);
        startActivity(intent);
    }
}
```

The code now uses the DatabaseManager instance to store the contacts in the database.

12. Navigate to Run menu an select Run 'app'.

The application is deployed on the AVD and the output is as shown in figure 7.7.

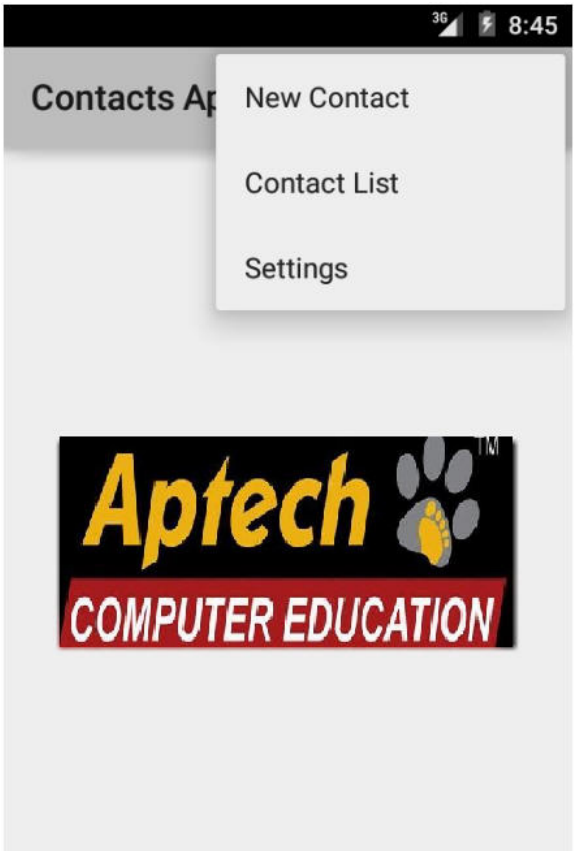


Figure 7.7: Contacts Application with Database Storage

The application functionality is the same as described earlier, but the underlying storage mechanism is changed to be more practical and usable in real time.

7.6 Content Providers

Content Providers provide access to data using a unified API. The primary purpose of a content provider is to act as a common gateway to applications for accessing the data. The content provider is also responsible for access management to the data. Irrespective of the method the data is stored, data is almost always presented to the application in a table format. A cursor is returned to navigate the data.

7.6.1 Content URI

Each content provider has a unique Content URI for the data associated with it. This URI can be used by the applications to query data from the content provider. An example content URI is shown in Code Snippet 22.

Code Snippet 22:

```
Content: //Domain/subdomain
```

The Uri.Builder class can be used to construct well-formed URIs within the application.

7.6.2 Searching and Accessing Content from a Content Provider

Content Providers use the query system to retrieve data. A reference to the Content Resolver instance is retrieved. The query() methods are used for this purpose.

- The first argument to the content provider is a URI to the content.
- The second argument is the projection specification. It contains the list of the columns to retrieve from the table format.
- The third argument is the selection statement. It specifies the selection criteria of the data. It is similar to the 'WHERE' clause in SQL or the 'IF' condition in Java.
- The fourth optional argument is the arguments for the selection statement. This is used if placeholders are used in the selection statement. It can be null otherwise.
- The final optional argument is sort order specification. It decides how the retrieved content is sorted before being presented to the application.

The method returns a Cursor object to the data table. The cursor can be used to navigate the rows and columns of the data table.

The usage of content providers is shown in Code Snippet 23.

Code Snippet 23:

```
String selection = MediaStore.Audio.Media.IS_MUSIC + " != 0";
String[] projection = {
        MediaStore.Audio.Media._ID,
        MediaStore.Audio.Media.ARTIST,
        MediaStore.Audio.Media.TITLE,
        MediaStore.Audio.Media.DATA,
        MediaStore.Audio.Media.DISPLAY_NAME,
        MediaStore.Audio.Media.DURATION,
       MediaStore.Audio.Media.ALBUM_ID
};
```

```
Cursor videoCursor = getContentResolver().query(MediaStore.Video.
Media.EXTERNAL_CONTENT_URI,projection,selection, null, null);
```

7.6.3 Adding, Modifying, and Removing Content

Inserting Content

To insert data, the insert() method of the content resolver can be used. For setting the insert values a ContentValues object needs to be created. The data that needs to be added is added as a Key-Value pair. The code is shown in Code Snippet 24.

Code Snippet 24:

```
ContentValues insertValues = new ContentValues();

insertValues.put(CUSTOM.Domain.Column_name,"column value");
```

The content values are then passed to the insert method as shown in Code Snippet 25.

Code Snippet 25:

```
…

Uri contentURI = uriBuilder.build();

getContentResolver().insert(uri,insertValues);

…
```

Updating Content

To update data, update() method of the content resolver can be used. For updating data a ContentValues object is created with the new values of the data. The object is then passed to the update method along with selection criteria of the entries that need to be updated and an optional placeholder's values if required. The code to achieve this is demonstrated in Code Snippet 26.

Code Snippet 26:

```
…

ContentValues updatedValues = new ContentValues();

updatedValues.put(CUSTOM.Domain.Column_name,"column value");

….
```

```
String selectionCriteria = CUSTOM.Domain.ID + " = 3 AND
"+CUSTOM.Domain.NAME+" = ?";

String[] placeHolderValues ={"customer 3"};

getContentResolver().update(uri,updatedValues,
                            selectionCriteria,placeHolderValues);
```

- **Deleting Content**

  To delete data, delete() method of the content resolver can be used. The method accepts a URI along with the selection criteria of the entries to be deleted and the placeholder's values if required. The code to achieve this is as shown in Code Snippet 27.

Code Snippet 27:

```
String selectionCriteria = CUSTOM.Domain.ID + " = 4 ";

getContentResolver().delete(uri,selectionCriteria,null);

// No placeholder values. Hence last argument is null
```

7.6.4 Using Native Android Content Providers

Android provides several content providers to access from and add content to. The native content providers of Android can be utilized by importing the package android.provider. The set of native content providers are as follows:

- Calendar
- Contact
- Media Store
- Openable Columns
- Sync States
- Telephony

The API to access any content provider is similar. However, certain content providers need the manifest to explicitly specify the permission to access them. The column names and sample Code Snippets to handle content from the Calendar, Contacts, and content provider are shown here.

- **Calendar**

  The calendar application provides the capabilities of event planning and reminders in Android. The calendar data is stored in the calendar repository accessed by the calendar provider.

The column names for the Events table of the Calendar content provider are listed in table 7.2.

Column Name	Description
NAME	The name of the calendar event to identify it programmatically.
CALENDAR_DISPLAY_NAME	The name of the calendar event that is displayed to the user.
VISIBLE	An integer which toggles the visibility of the event to the user. 0 indicates that the event is added, but it should not be displayed to the user (yet). 1 indicated that it should be visible.
SYNC_EVENTS	An integer which toggles the storage option from local (on Device) event to online Google calendar of the user. 1 indicates enabled whereas, 0 indicates disabled.

Table 7.2: Columns of the Event Table

The permissions for accessing calendar content, which are to be placed in the manifest file are shown in Code Snippet 28.

Code Snippet 28:

```
<uses-permission
  android:name="android.permission.READ_CALENDAR" />
<uses-permission
   android:name="android.permission.WRITE_CALENDAR" />
```

Example code for retrieving a calendar event is shown in Code Snippet 29.

Code Snippet 29:

```
Cursor cur = null;
String selection = "((" + Calendars.ACCOUNT_NAME+ "= John" AND
Calendars.OWNER_ACCOUNT + " = john@Google.com))";
cur = getContentResolver().query(Calendars.CONTENT_URI, EVENT_
PROJECTION, selection, selectionArgs, null);
```

The selection string is used to set the selection criteria for the query results. The query is then executed to retrieve a reference to an events cursor.

For Calendar API, only retrieve and update is supported. Insert should not be done directly and a Sync Adapter needs to be used. There are several other tables of the calendar provider which provide more information which is available in the API documentation.

- **Contacts**

The contacts provider acts as a central repository to all forms of contacts and relationships stored on the device. This is not just limited to phone contacts, but also social media contacts and visiting card information.

The column names for the Raw Contacts table of the Contacts content provider are listed in table 7.3.

Column Name	Description
ACCOUNT_NAME	A name for the contact.
ACCOUNT_TYPE	The type and source of the contact information. Social media, local, and so on.
DELETED	The values help syncing the data with online provider. For example, if a contact is deleted locally, it still needs to be also deleted on the Sync Server. Hence, the contact is stored until the contact is deleted on the server as well. The DELETED flag is set to 1 until this occurs.

Table 7.3: Columns of the Raw Contacts Table

The column names for the Profile Contacts table of the Contacts content provider are listed in table 7.4.

Column Name	Description
Profile._ID	A unique id identifies the contact.
Profile.DISPLAY_NAME_PRIMARY	The name displayed to the user.
Profile.LOOKUP_KEY	A lookup key to provide search functionality.
Profile.PHOTO_THUMBNAIL_URI	Content URI for the thumbnail of the contact.

Table 7.4: Columns of the Profile Contacts Table

The permissions for accessing calendar content, which is to be placed in the manifest file is shown in Code Snippet 30.

Code Snippet 30:

```
<uses-permission android:name="android.permission.READ_CONTACTS">

<uses-permission android:name="android.permission.WRITE_CONTACTS">
```

Example code for retrieving a calendar event is shown in Code Snippet 31.

Code Snippet 31:

```
Cursor cur = null;
String[] columns = {
        Profile._ID,
        Profile.DISPLAY_NAME_PRIMARY,
        Profile.LOOKUP_KEY,
        Profile.PHOTO_THUMBNAIL_URI
    };
cur = getContentResolver().query(Profile.CONTENT_URI, columns ,
null, null, null);
```

7.6.5 Creating Content Providers

It is possible to create Content providers to custom data. This can be done by extending the ContentProvider class. Before creating a Content Provider, the developer needs to understand when to create a content provider.

A content provider can be created when:

1. You want to provide complex data or files to other applications.
2. To provide search suggestions from the application to the search feature of Android.

A content provider should not be created when:

1. Providing access to databases.
2. To act as a mode of communication between applications.

- **Data Storage**:

  In order to provide content, the content provider needs to be able to store the data using one of the data storage methods discussed earlier. The most commonly used storage options are:

  - **Files** – This option involves using files stored on the internal or the external storage. Internal storage is preferable as External storage is unreliable.

- **SQLite Databases** – The data can be stored on SQLite databases if the data is structured data. The tables of the database need not match the tables exposed by the content provider. If they do not match, the developer needs to restructure the data retrieved from the database tables.

☐ **Content URI**:

Every content provider needs to provide a URI that identifies the data provided by the content provider. A typical content URI has three parts:

- Authority
- Path Structure
- Content Id

The authority uniquely identifies the provider itself. The most commonly used authority is the package name followed by the provider. For example a URI authority for the contacts application would be com.aptech.contactsapplication.provider.

The path structure defines the type of data that is being accessed from the content provider. This can be decided based on the nature of the content. For example, a URI path for the contacts application would be:

content://com.aptech.contactsapplication.provider/details/

The content id provides an optional unique identifier to a single element of the data. For example, to retrieve a single row containing the contact details of a single contact with the contact id 3, the URI would be:

content://com.aptech.contactsapplication.provider/details/3.

☐ **URI Patterns**:

URI patterns provide a much simpler method for creating content queries. It is the responsibility of the developer to implement the functionality for URI patterns. For example, all the contacts can be retrieved by the URI as shown in Code Snippet 32.

Code Snippet 32:

```
content://com.aptech.contactsapplication.provider/details/*
```

Only the names of all the contacts can be retrieved by using the URI as shown in Code Snippet 33.

Code Snippet 33:

```
content://com.aptech.contactsapplication.provider/details/names/*
```

The id number of a specific contact can also be specified to retrieve a single row as shown in Code Snippet 34.

Code Snippet 34:

```
content://com.aptech.contactsapplication.provider/details/4
```

In order to implement this behavior, the developer can use the UriMatcher class along with a unique id for each Uri pattern as shown in Code Snippet 35.

Code Snippet 35:

```
static final UriMatcher uriMatcher;
    static{
        uriMatcher = new UriMatcher(UriMatcher.NO_MATCH);
        uriMatcher.addURI(providerName, "contacts", 1);
        uriMatcher.addURI(providerName, "contacts/*", 2);
        uriMatcher.addURI(providerName, "contacts/#", 3);
    }
```

- **Implementing the Content Provider**:

In order to implement a Content Provider, a concrete class extending the ContentProvider class needs to be created. Following methods need to be overridden and implemented.

- onCreate()
- query()
- update()
- delete()
- insert()
- getType()

A content provider which provides access to a database table is demonstrated in Code Snippet 36.

Code Snippet 36:

```
package aptech.com.contentproviders;
import android.content.ContentProvider;
import android.content.ContentUris;
import android.content.ContentValues;
import android.content.Context;
import android.content.UriMatcher;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;
import android.database.sqlite.SQLiteQueryBuilder;
import android.net.Uri;
import android.text.TextUtils;
import java.util.HashMap;
public class ContactContentProvider extends ContentProvider {
    static final String providerName = "aptech.com.
                                        contenprovider.provider";
    static final String uri = "content://" + providerName + "/
                                         contacts";
    static final Uri URI = Uri.parse(uri);
    private static HashMap<String, String> projectMap;
    static final String _ID = "_id";
    static final String NAME = "name";
    static final UriMatcher uriMatcher;
    static{
        uriMatcher = new UriMatcher(UriMatcher.NO_MATCH);
        uriMatcher.addURI(providerName, "contacts", 1);
        uriMatcher.addURI(providerName, "contacts/#", 2);
    }
```

```
 private SQLiteDatabase db;
 static final String DATABASE_NAME = "contacts";
 static final String TABLE_NAME = "details";
 static final int DATABASE_VERSION = 1;
 static final String CREATE_DB_TABLE =
         " CREATE TABLE " + TABLE_NAME +
                 " (_id INTEGER PRIMARY KEY AUTOINCREMENT, " +
                  " name TEXT NOT NULL );";
private static class DatabaseManager extends
                                      SQLiteOpenHelper{
     DatabaseManager(Context context){
       super(context, DATABASE_NAME, null, DATABASE_VERSION);
     }
     @Override
     public void onCreate(SQLiteDatabase db) {
         db.execSQL(CREATE_DB_TABLE);
     }
     @Override
     public void onUpgrade(SQLiteDatabase db, int oldVersion,
                           int newVersion) {
         db.execSQL("DROP TABLE IF EXISTS " +  TABLE_NAME);
         onCreate(db);
     }
 }
 @Override
 public boolean onCreate() {
     Context context = getContext();
     DatabaseManager databaseManager = new
                                    DatabaseManager(context);
```

```
        db = databaseManager.getWritableDatabase();
        return (db == null)? false:true;
    }
    @Override
    public Uri insert(Uri uri, ContentValues values) {
    long contactId = db.insert(TABLE_NAME, "", values);
        Uri _uri = ContentUris.withAppendedId(URI, contactId);
        if (contactId > 0)        {
       getContext().getContentResolver().notifyChange(_uri,
                                                    null);
        }
        return _uri;
    }
    @Override
    public Cursor query(Uri uri, String[] projection, String
selection,String[] selectionArgs, String sortOrder) {
        SQLiteQueryBuilder queryBuilder = new
                                          SQLiteQueryBuilder();
   //If the tablename being exposed does not match the database
   //table name, the developer may write a switch case statement
here
   //to resolve it to the appropriate table name
   //If the column names exposed donot match, Write the
resolving    //code here
    //projectMap.put("queryColumnName","dbColumnName");
        queryBuilder.setTables(TABLE_NAME);
        switch (uriMatcher.match(uri)) {
            case 1:
                queryBuilder.setProjectionMap(projectMap);
                break;
```

```
            case 2:
                queryBuilder.appendWhere(_ID + "=" + uri.
                                    getPathSegments().get(1));
                break;
        }
        if (sortOrder == null || sortOrder == ""){
                // sorting code
        }
       Cursor cursor = queryBuilder.query(db, projection,
                selection, selectionArgs,null, null, sortOrder);
        cursor.setNotificationUri(getContext().
                                getContentResolver(), uri);
       return cursor;
    }
   @Override
    public int delete(Uri uri, String selection, String[]
selectionArgs) {
       int count = 0;
        switch (uriMatcher.match(uri)){
            case 1:
                count = db.delete(TABLE_NAME, selection,
                                    selectionArgs);
                break;
            case 2:
                String id = uri.getPathSegments().get(1);
                count = db.delete( TABLE_NAME, _ID +  " = " + id +
                        (!TextUtils.isEmpty(selection) ? " AND ("
+ selection + ')' : ""), selectionArgs);
                break;
        }
```

```
        getContext().getContentResolver().notifyChange(uri,
                                                        null);
        return count;
    }
    @Override
    public int update(Uri uri, ContentValues values, String
                          selection,String[] selectionArgs) {
        int count = 0;
        switch (uriMatcher.match(uri)){
            case 1:
                count = db.update(TABLE_NAME, values,
                        selection, selectionArgs);
                break;
           case 2:
                count = db.update(TABLE_NAME, values, _ID +
                      " = " + uri.getPathSegments().get(1) +
                     (!TextUtils.isEmpty(selection) ? " AND
                     (" + selection + ')' : ""), selectionArgs);
                break;
        }
        getContext().getContentResolver().notifyChange(uri, null);
        return count;
    }
    @Override
    public String getType(Uri uri) {
       return
       "vnd.android.cursor.item/vnd.aptech.com.
                                    contentprovider.provider";
    }
}
```

The content provider can be accessed from an application using Code Snippet 37.

Code Snippet 37:

```
Uri contentUri = Uri.parse("content://aptech.com.
contentprovider.provider/contacts);

Cursor cursor = getContentResolver().
                query(contentUri,null,null,null,null);
```

The code retrieves all the entries from the content provider in the cursor.

- **Manifest Changes**:

  The content provider needs to be registered by using the <provider> tags in the manifest file. An example for the provider tag is as shown in Code Snippet 38.

Code Snippet 38:

```
<provider

android:name="com.aptech.contactsapplication.CONTENT_PROVIDER_
                                                CLASS_NAME"

android:authorities="com.aptech.contactsapplication.provider" />
```

7.6.6 Content Provider MIME Types

MIME is the abbreviation for Multipurpose Internet Mail Extensions. MIME was added as an extension to the e-mail protocol to support custom transfer of custom type of data, such as Images, Video and Application, and so on, which were not initially supported.

Android supports the ability to return MIME media types or MIME message content from content providers. This allows for handling data which cannot be handled using the table format.

Android also provides the ability to define custom MIME types. This is useful when creating Custom Content Handlers or supporting vendor specific implementations. An example of this is the TimeScape's content provider in Sony's older generation of Xperia phones.

7.7 Resources and Assets

Earlier, the overview of resources and assets was given. In this section, the underlying mechanisms used by Android and advanced topics will be described.

7.7.1 Resources

The basics of resources are already covered in the earlier sessions. Resources are used to separate the functional code of the application from the static content.

- **Resource Storage**

  Resources are not stored on the Android File System. They are packaged as part of the application itself. This is the reason resources cannot be accessed with the standard File class. Resources are local to the application and cannot be accessed outside the application. The developer needs to keep in mind that resources are accessible if the application is decompiled using APK Studio or a similar application. Any sensitive or copyrighted content is advised to be encrypted.

- **Locating Resources**

  A resource can be referenced using the resource identifier. The standard format for a resource identifier is as shown in Code Snippet 39.

Code Snippet 39:

```
[<package_name>.]R.<resource_type>.<resource_name>
```

The package name can be omitted in most cases. Whenever a resource is referred in an application, Android follows the following steps to locate the resource:

1. Locate the package that is being referred. If it is not mentioned, the application name is used such as com.aptech.app_name to locate the apk holding the resource.
2. Identify the resource type being referred to.
3. List the directories capable of holding the resource type. For example, drawable can be held in drawable, drawable-hdpi, drawable-mdpi, and so on.
4. Eliminate the types not supported by the device. For example, depending on the resolution, the drawable-mdpi could be eliminated.
5. Repeat steps 2-4 until all the qualifiers are eliminated, such as language, key support, and so on until a maximum of one directory remains.
6. Search the directory for a file name matching the resource name without the extension.
7. Return an integer identifier to the resource.

- **Handling Runtime Configuration Changes**

  Certain device types have a changing configuration during run time. A prime example for this would be the media centre devices running Android connected to televisions. Depending on the screen they are connected to, the resolution of the display can change on the fly.

The developer may have intended the application to use a different set of resources for this changed configuration, such as different icon, different layout, and so on. To deal with the issue, the developer can either handle the changes manually in code or let Android auto configure the application based on the current configuration.

To manually handle a configuration change, the type of changes that are to be handled manually needs to be specified in the Manifest file as shown in Code Snippet 40.

Code Snippet 40:

```
<activity android:name=".MainActivity"
   android:configChanges="orientation|keyboardHidden"
   android:label="@string/app_name">
```

Whenever an orientation change occurs or the on screen keyboard is hidden or shown, the onConfigurationChanged() method of the activity is called. The developer needs to override the methods and implement the changes here. An example is as demonstrated in Code Snippet 41.

Code Snippet 41:

```
@Override
public void onConfigurationChanged(Configuration newConfig) {
    super.onConfigurationChanged(newConfig);
if (newConfig.orientation == Configuration.ORIENTATION_LANDSCAPE){
        setContentView(R.layout.horizontal_layout);
    }
else{
       setContentView(R.layout.vertical_layout);
  }
}
```

- **Retrieving Raw Data**

It is possible to read the raw data from a resource. This procedure is usually not recommended. Assets are a better alternative for this purpose. If resources are being used, then the developer is recommended to create a new directory called 'raw' in the /res directory to hold the raw resource.

The code to retrieve raw data from a resource with the file name picture.jpg stored in the raw directory is demonstrated in Code Snippet 42.

Code Snippet 42:

```
InputStream iS = getResources().openRawResource("picture")
```

7.7.2 Assets

Assets are functionally similar to resources, but serve a different purpose. Data stored in resources needs to be structured data and it needs to belong to a supported data type. Assets, on the other hand can hold any type of data. As a result, assets are not given a resource id. They are directly identified by their file names.

Assets are stored in their own directory inside the package apk. No changes are made and the files in the asset folder are stored as is. The AssetManager class is used to access the assets. The API is similar to the File class API. The code to open an asset called image.png is demonstrated in Code Snippet 43.

Code Snippet 43:

```
InputStream iS = context.getAssets().open("image.png");
```

In real time, assets are used to store textures, models, blob data, and other pre-compiled data in games and other similar applications. If the size exceeds beyond 100 MB, it is preferable to use external storage to store these assets.

Check Your Progress

1. Which information is needed to update data in content providers?

1.	URI	2.	File name
3.	Content Values	4.	Selection criteria

(A)	1, 4	(B)	1, 2, 4
(C)	1, 3, 4	(D)	3, 4

2. Which of the following statements are true?

1.	Cursors represent a single row of data.
2.	Content Providers can restrict access to data.
3.	SQLite databases have security mechanisms.
4.	External Storage is unreliable.

(A)	1, 3, 4	(B)	1, 2
(C)	1, 2, 4	(D)	2, 3

3. Which of the following class is used for executing SQL commands?

(A)	Cursor	(B)	SQLiteOpenHelper
(C)	SQLiteDatabase	(D)	SQLException

4. To create a custom content provider, the _____ class needs to be extended.

(A)	ContentResolver	(B)	ContentManager
(C)	Content	(D)	ContentProvider

5. Match the following data types with the corresponding storage location.

	Data Type		Storage Location
a.	Raw files	1.	Databases
b.	Structured tabular data	2.	Preferences
c.	Media Files	3.	Assets
d.	Options or settings of an application	4.	External Storage

(A)	a-2, b-3, c-4, d-1	(B)	a-3, b-1, c-4, d-2
(C)	a-3, b-1, c-2, d-4	(D)	a-1, b-3, c-4, d-2

Answers

1.	C
2.	C
3.	C
4.	D
5.	B

Summary

- The different types of storages available in Android are Shared Preferences, Internal Storage, External Storage, and SQLite Database.
- Android uses SQLite database to store relational data. SQLite queries can be executed by using the query() method present in the SQLiteDatabase. The query() method will return a Cursor object that points to all the retrieved rows.
- Content Providers in Android act as resources to information.
- Content Providers can be accessed using unique URIs.
- Custom Content Providers can be created by extending the ContentProvider class.
- Resources and assets are used to store static content for the application.
- Assets do not have a resource id generated and they are packaged as-is.

Onlinevarsity

LEARN@YOUR COMFORT

08 Services, Broadcast Receivers, and Intent Filters

Welcome to the Session, **Services, Broadcast Receivers, and Intent Filters.**

This session explains certain Android components that can be used in the Android application. It describes Android services and how they are implemented in the application. It also describes how systems or application events are registered by an Android component called broadcast receiver. Further, it also explains how intents allow the application to request functionality from other components of the Android systems.

In this Session, you will learn to:

- Explain services
- Explain service lifecycle
- Describe broadcast receiver and its working
- Explain filters
- Explain intent matching and its rules
- Explain filters in manifest file and broadcast receivers

Session

08

Services, Broadcast Receivers, and Intent Filters

8.1 Introduction

To develop an Android application, the four main components that play an important role are activities, services, content providers, and broadcast receivers. This session discusses services and broadcast receivers in detail. Services are executed in the background to perform long-running operations whereas broadcast receivers respond to announcements. The session also explains Intents which are used for activating components.

8.2 Services

Android application consists of different components which are the building blocks for an application. A system can enter the application through different components which acts as a point of entry. Though all components cannot act as a point of entry to the application, but each one has its own entity and plays a specific role. Service is one of the components.

8.2.1 Overview of Services

A service, as discussed earlier, is an Android component that can perform background tasks without providing a user interface. It is the base class for all services, which when extended, need to be created in a new thread. As the service uses the main thread by default, it slows down the performance of any running application. Service class creates application components that are specifically suited to handle functions that should run at the background. Any other Android component can start a service and even if the user switches to another application, a service will continue to run in the background. In other words, service is ideal for those cases where an UI need not be shown to the user. For example, playing music in the background while working with an application in the foreground.

Additionally, a component can communicate with a service by binding itself to it.

Service has higher priority when compared with an inactive activity. Thus, when there is a need of resources, service components are less likely to be killed. If a service has been terminated, it can be configured to restart when the resources are available. Using of service ensures that the application will run and respond even when they are not actively used.

8.2.2 Implementing Services

This section will explain how to create each type of service and use it from other application components.

Services are declared in the application's manifest file. To declare a service in the AndroidManifest.xml file, add a <service> element as a child of the <application> element as shown in Code Snippet 1.

Code Snippet 1:

```
<manifest ... >
...
<application ... >
<service android:name=".SampleService"/>
...
</application>
</manifest>
```

To define properties, attributes such as permissions to start the service and the process in which the service should run, can be included in the <service> element. The android:name attribute specifies the class name of the service and is the only required attribute. Once the application is published, the name should not be changed, as it might break some functionality where explicit Intents are used to reference the service.

8.2.3 Creating a Service

A component starts a service by invoking the startService() method. It results in a call to the Service 'class' onStartCommand() method. The lifecycle of this started service is not dependent on the component it starts and can run in the background indefinitely. The service either stops itself by invoking the stopSelf() method or is stopped by another component when stopService() method is invoked.

While starting a service, a component can also pass an Intent that specifies the service and include any data for the service to use. The Intent is received by the Service class in the onStartCommand() method. For example, some data needs to be uploaded on an online database. The service can be started by an activity and provide the data to be saved by passing an Intent to startService() method. The service will then receive the intent in the method onStartCommand(), connect to the Web, and perform the database transfer. When the transfer is done, the service stops itself and is destroyed.

Services are launched on the main application thread, meaning that any processing done in the onStartCommand() method will happen on the main GUI thread.

To create a started service, following are the two classes from which your service class can extend from:

- **IntentService**: IntentService is a subclass of Service class that handles all start requests one at a time. It uses a worker thread and is the best option to avoid handling of multiple requests simultaneously. The implementation of onHandleIntent() method will result in receiving the intent for each start request, enabling the user to perform the background work.

- **Service**: Service is the base class for all services, which when extended, needs to be created in a new thread.

Following section will describe how to implement the service by using either IntentService or Service class.

- **Creating a Service Extending the IntentService Class**: IntentService class is implemented to avoid multi-threading, that is handling of multiple requests simultaneously. It handles asynchronous requests on demand. It is started as a normal service, performs the tasks within a worker thread, and termi¬nates when the task is performed.

  The functions of IntentService class are as follows:

  - Creates a default worker thread and separates all intents given to onStartCommand() from the main thread.
  - Avoids multi-threading by creating a work queue where it passes one Intent at a time to the onHandleIntent() method.
  - Stops the service once all requests have been completed.
  - Provides default onBind() method to return null.
  - Provides default onStartCommand() method to send the Intent to the work queue and then to the method onHandleIntent().

  The section explains in detail the implementation of the IntentService class.

  The steps to create a IntentService project are as follows:

  1. Create a project named IntentServiceExample in Android Studio.
  2. Navigate to src → com.example.intentserviceexample folder.
  3. Create a new class called MyIntentService.
  4. Modify the code in the newly created class as shown in Code Snippet 2.

Code Snippet 2:

```
package com.example.intentserviceexample;
import android.app.IntentService;
import android.content.Intent;
import android.widget.Toast;
public class MyIntentService extends IntentService {
    public MyIntentService() {
        super("MyIntentService");
```

```
        // TODO Auto-generated constructor stub
    }
    @Override
    protected void onHandleIntent(Intent Intent) {
        Toast.makeText(this, "My IntentService started", Toast.
LENGTH_LONG) .show();
 }
    @Override
    public void onCreate() {
        super.onCreate();
        Toast.makeText(this, "Creating a Service ...", Toast.
LENGTH_LONG).show();
    }
}
```

Note - The class must override onHandleIntent() method.

5. Navigate to src → com.example.intentserviceexample → MainActivity.java file.
6. Modify the code as shown in Code Snippet 3.

Code Snippet 3:

```
package com.example.intentserviceexample;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.content.Intent;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
public class MainActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.activity_main);
        Button start = (Button) findViewById(R.id.button);
        start.setOnClickListener(new OnClickListener() {
            public void onClick(View V) {
                Intent = new Intent(MainActivity. this,
                        MyIntentService.class);
                startService(Intent);
            }
        });
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
      // Inflatethemenu;thisaddsitemstotheactionbarifitis present.
        getMenuInflater().inflate(R.menu.menu_main, menu);
    return true;
    }
}
```

7. Navigate to res → values → strings.xml file to change the string appearing as a text.
8. Modify the code in the strings.xml file as shown in Code Snippet 4.

Code Snippet 4:

```
<resources>
    <string name="app_name">Service Class</string>
    <string name="hello_world">Service Example!</string>
    <string name="action_settings">Settings</string>
</resources>
```

9. Navigate to res → layout → activity_main.xml file.

10. Modify the code in the activity_main.xml file as shown in Code Snippet 5.

Code Snippet 5:

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/
res/android"
    xmlns:tools="http://schemas.android.com/tools" android:layout_
width="match_parent"
    android:layout_height="match_parent" android:paddingLeft="@
dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
tools:context=".MainActivity">
<TextView android:text="@string/hello_world" android:layout_
width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/textView1" />
<Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Start Service"
        android:id="@+id/button"
        android:layout_below="@+id/textView1"
        android:layout_alignParentStart="true"
        android:layout_marginTop="50dp" />
</RelativeLayout>
```

11. Navigate to manifests and open the AndroidManifest.xml file to add the <Service> element.

12. Modify the code in AndroidManifest.xml file as shown in Code Snippet 6.

Code Snippet 6:

```
<?xml version=”1.0” encoding=”utf-8”?>
<manifest
xmlns:android=”http://schemas.android.com/apk/res/android”
    package=” com.example.intentserviceexample “ >
 <application
        android:allowBackup=”true”
        android:icon=”@mipmap/ic_launcher”
        android:label=”@string/app_name”
        android:theme=”@style/AppTheme” >
        <activity
            android:name=”.MainActivity”
            android:label=”@string/app_name” >
            <Intent-filter>
<action android:name=”android.Intent.action.MAIN” />
<category android:name=”android.Intent.category.LAUNCHER” />
            </Intent-filter>
        </activity>
        <Service android:enabled=”true”
        android:name=”.MyIntentService”>
    </Service>
</application>
</manifest>
```

Note - The <uses-sdk> element is not required. Android Studio will generate another manifest file and include the sdk versions (along with some other settings) in it. As of 1.1, the sdk requirements need not be specified in the manifest by the developer. They will be included as per the option selected when creating the project.

On execution of the application and clicking the START SERVICE, the output will be as shown in figure 8.1.

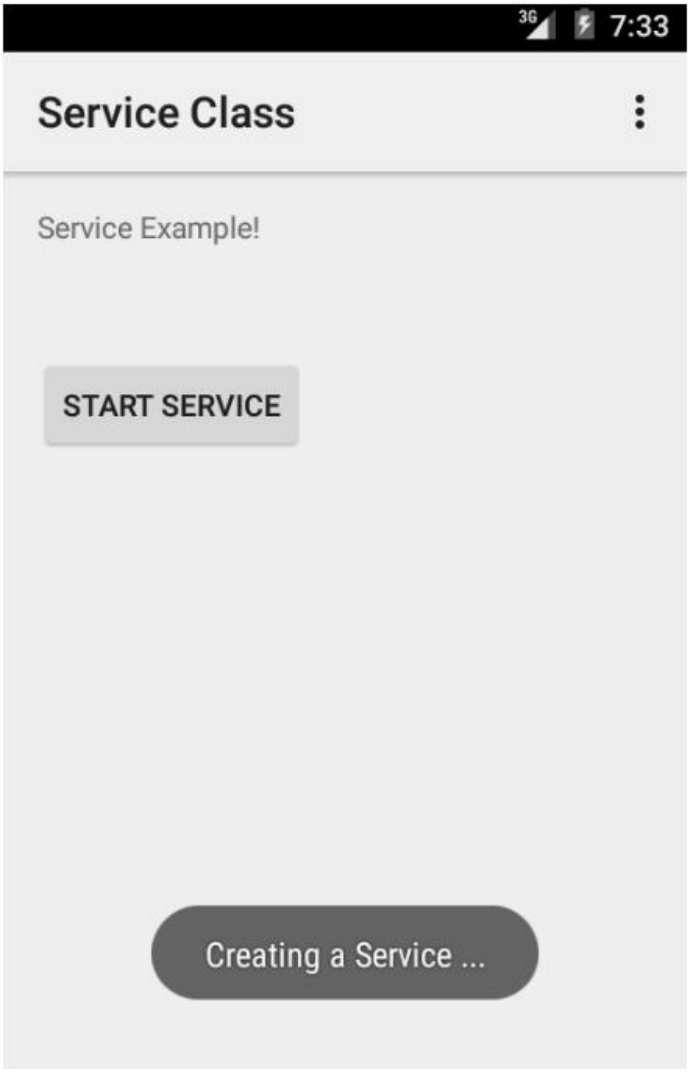


Figure 8.1: MyIntentService Output

- **Creating a Service Extending the Service class**

Using IntentService class makes the implementation of a started service easy. However, if the user requires the service to perform multi-threading, then the developer can extend it from the Service class to handle each intent.

The section explains in detail the implementation of the Service class where the user will start and stop the music using service.

The steps to create a ServiceDemoExample project are as follows:

1. Create a new project named ServiceDemoExample in Android Studio.
2. Navigate to src → com.example.servicedemoexample package.
3. Create a new class called ServiceExample.
4. Modify the code in ServiceExample class as shown in Code Snippet 7.

Code Snippet 7:

```
package com.example.servicedemoexample;
import android.app.Service;
import android.content.Intent;
import android.media.MediaPlayer;
import android.os.IBinder;
import android.widget.Toast;
public class ServiceExample extends Service {
MediaPlayer mp;
@Override
public IBinderonBind(Intent Intent) {
            // TODO Auto-generated method stub
            Return null;
      }
      @Override
      public void onCreate() {
      Toast.makeText(this, "Service call oncreate() event once",
Toast.LENGTH_SHORT).show();
            mp = new MediaPlayer();
             FileInputStream iS = (FileInputStream)
getAssets().open("flysong.mp3");
            mp.setDataSource(iS.getFD());
            mp.setLooping(false); // Set looping
      mp.prepare();
```

```
      }
      @Override
      public void onDestroy() {
            Toast.makeText(this, "Service Stop",Toast.LENGTH_
SHORT).show();
mp.stop();
      }
      @Override
      public void onStart(Intent Intent, int startid) {
            Toast.makeText(this, "Service started",Toast.LENGTH_
SHORT).show();
      mp.start();
      }
}
```

5. Navigate to src → com.example.servicedemoexample → MainActivity.java file.
6. Modify the code in MainActivity.java file as shown in Code Snippet 8.

Code Snippet 8:

```
package com.example.servicedemoexample;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.content.Intent;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.Toast;
public class MainActivity extends Activity {
Button startService, stop_Service;
@Override
```

```
protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
            setContentView(R.layout.activity_main);
            startService = (Button) findViewById(R.id.button1);
            stop_Service = (Button) findViewById(R.id.button2);
            startService.setOnClickListener
            (newOnClickListener() {
                  @Override
public void onClick(View v) {
                        // TODO Auto-generated method stub
                        Toast.makeText(getApplicationContext(),
                        "Service call to start",
                  Toast.LENGTH_SHORT).show();
                  startService(new Intent(MainActivity.this,
                                    ServiceExample.class));
                  }
            });
      stop_Service.setOnClickListener(newOnClickList
      ener() {
            @Override
public void onClick(View v) {
                  // TODO Auto-generated method stub
                  Toast.makeText(MainActivity.this,
            "Service call to stop",
Toast.LENGTH_SHORT).show();
                  stopService(new Intent(MainActivity.
                  this, ServiceExample.class));
                  }
```

```
            });
      }
      @Override
      public boolean onCreateOptionsMenu(Menu menu) {
            // Inflate the menu; this adds items to the action bar
            //if it is present.
            getMenuInflater().inflate(R.menu.main, menu);
            return true;
      }
}
```

7. Right-click app and navigate to New → Folder and select Assets Folder.
8. Click Finish.
9. Copy any song inside the assets folder. For example, it is flysong.mp3 as shown in figure 8.2.

Note - The song name should be in lowercase and without space.

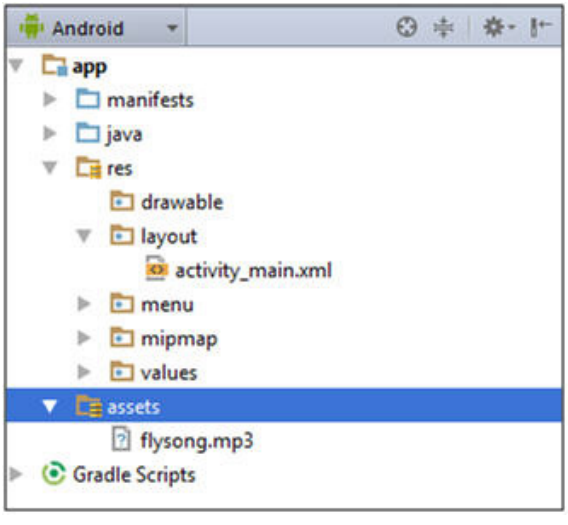


Figure 8.2: Song

10. Click res → layout → activity_main.xml file to open the file for editing.

11. Modify the code in activity_main.xml file as shown in Code Snippet 9.

Code Snippet 9:

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/
android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:paddingBottom="@dimen/activity_vertical_margin"
android:paddingLeft="@dimen/activity_horizontal_margin"
android:paddingRight="@dimen/activity_horizontal_margin"
android:paddingTop="@dimen/activity_vertical_margin"
tools:context=".MainActivity">
<TextView
            android:id="@+id/textView1"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/hello_world"/>
      <Button
            android:id="@+id/button2"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignRight="@+id/button1"
            android:layout_centerVertical="true"
            android:text="Stop Music by Service"/>
      <Button
            android:id="@+id/button1"
            android:layout_width="wrap_content"
```

```
            android:layout_height="wrap_content"
            android:layout_below="@+id/textView1"
            android:layout_centerHorizontal="true"
            android:layout_marginTop="52dp"
            android:text="Start Music by Service"/>
</RelativeLayout>
```

12. Navigate to res → values → strings.xml file.
13. Modify the code in strings.xml file as shown in Code Snippet 10.

Code Snippet 10:

```
<?xmlversion="1.0"encoding="utf-8"?>
<resources>
<stringname="app_name">ServiceDemoExample</string>
<stringname="action_settings">Settings</string>
<stringname="hello_world">Hello using Service class!
</string>
</resource>
```

14. Open AndroidManifest.xml file and add the <Service> element as shown in Code Snippet 11.

Code Snippet 11:

```
...
<service android:name=".ServiceExample"></service>
...
```

On execution of the program the output will be as shown in figure 8.3.

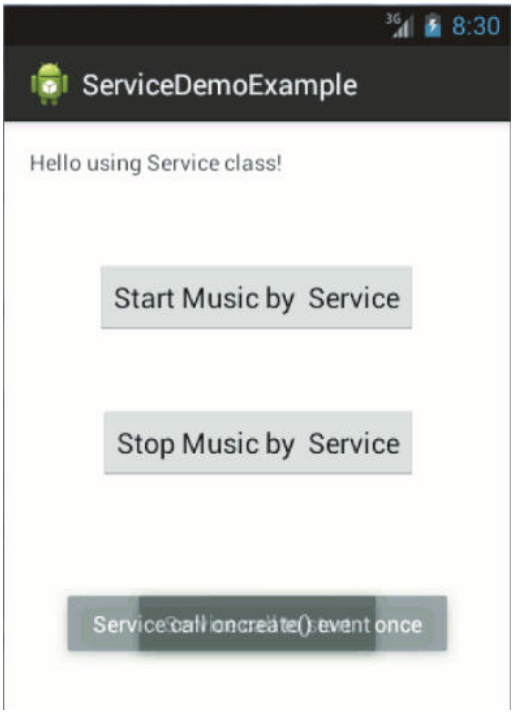


Figure 8.3: Start Music by Service

Figure 8.4 displays the output when the Stop Music by Service button is clicked.

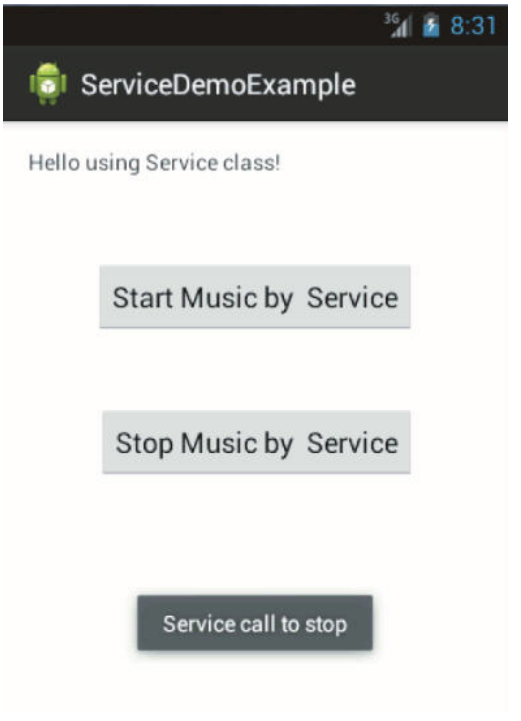


Figure 8.4: Stop Music by Service

8.2.4 Starting a Service

A service can be started from an activity or a component by passing an Intent to the startService() method. The onStartCommand() method of the Service class is then invoked by the Android system and the Intent is passed.

Note - onStartCommand() method should never be invoked directly.

For example, in Code Snippet 12, an activity can start the example service using an explicit intent with startService() method.

Code Snippet 12:

```
startService(newIntent(MainActivity.this, ServiceExample.
class)); ...
```

The startService() method returns instantly and the Android system calls the service's onStartCommand() method. In case, the service is not running already, the system will first invoke onCreate() method followed by the invocation of the onStartCommand() method.

When multiple requests are sent to start a service, it results in multiple calls to the service's onStartCommand() method, but it requires the invocation of the method stopService() method only once to stop the service.

8.2.5 Stopping a Service

A started service must stop either by invoking itself the stopSelf() method or by invoking the stopService() method by another component. The moment it is requested to stop using either ways, the system destroys the service.

However, if the service is handling multiple requests by using onStartCommand(), then it should not be stopped as a new start request might have been received. To avoid this issue, stopSelf(int) method can be used to ensure that the request to stop the service is based on the most recent start request.

Code Snippet 13 demonstrates how to stop a started service using Intents.

Code Snippet 13:

```
...
stopService(new Intent(MainActivity.this,ServiceExample.class));
...
```

Note -To avoid wastage of system resources and consumption of battery power, it is important that the application stops its services when the work is complete. If required, other components can stop the service by invoking the stopService() method.

8.2.6 Creating a Bound Service

When components of an application bind to a service by calling bindService(), it is called a bound service. Binding to a service helps in creating long-standing connection. Developer can bind the service with Activity, Service, and Content Provider. The bindService() method is invoked when the developer interact with activity, service, or to expose some of the application functionality to other process using Android Interface Definition Language (AIDL).

To use a bound service, the onBind() callback method must be implemented. It returns an IBinder that specifies the interface for communication with the service. The service exists only till it serves the application component that it is bound to. However, if there are no components bound to the service, the system destroys it.

While creating a bound service, the interface that specifies how a client can communicate with the service must be defined. This interface should be an implementation of IBinder and must extend the Service class from the onBind() callback method.

Several clients can bind to the service at once. When the interaction is complete, the client invokes the method unbindService() to unbind. The moment there are no clients bound to the service, the system destroys the service.

When using a bound service, the developer must use an IBinder which provides the programming layer in which client can interact with the service. There are three ways where the developer can define layer and they are as follows:

- By using a Messenger
- By extending a Binder Class
- By using an AIDL

In the following example, the developer can start, stop, bind and unbind the service with a button click.

The steps to create a BoundServiceExample project are as follows:

1. Create a project named BoundServiceExample in Android Studio.
2. Navigate to src → com.example.boundserviceexample package.
3. Create a new class called MyBoundService.

4. Modify the code in MyBoundService class as shown in Code Snippet 14.

Code Snippet 14:

```
package com.example.boundserviceexample;
import android.app.Service;
import android.content.Intent;
import android.os.Bundle;
import android.os.Handler;
import android.os.IBinder;
import android.os.Message;
import android.os.Messenger;
import android.os.RemoteException;
import android.util.Log;
import java.util.ArrayList;
import java.util.Timer;
import java.util.TimerTask;
public class MyBoundService extends Service {
    private Timer timer = new Timer();
    private int counter = 0, incrementby = 100;
    private static boolean isRunning = false;
    ArrayList<Messenger> mClients = new ArrayList<Messenger>();
    int mValue = 0; // Holds last value set by a client.
    static final int MSG_REGISTER_CLIENT = 1;
    static final int MSG_UNREGISTER_CLIENT = 2;
    static final int MSG_SET_INT_VALUE = 3;
    static final int MSG_SET_STRING_VALUE = 4;
final Messenger mMessenger = new Messenger (new
IncomingHandler()); // Target
```

```
    @Override
    public IBinder onBind(Intent Intent) {
        return mMessenger.getBinder();
    }
    class IncomingHandler extends Handler {
        @Override
        public void handleMessage(Message msg) {
            switch (msg.what) {
                case MSG_REGISTER_CLIENT:
                mClients.add(msg.replyTo);
                break;
                case MSG_UNREGISTER_CLIENT:
                mClients.remove(msg.replyTo);
                break;
                case MSG_SET_INT_VALUE:
                incrementby = msg.arg1;
                break;
                default:
                    super.handleMessage(msg);
            }
        }
    }
private void sendMessageToUI(int intvaluetosend) {
        for (int i = mClients.size() - 1; i >= 0; i--) {
            try {
                // Send data as an Integer
                mClients.get(i).send(
```

```
  Message.obtain(null, MSG_SET_INT_VALUE, intvaluetosend,0));
                // Send data as a String
                Bundle b = new Bundle();
                b.putString("str1", "ab" + intvaluetosend +"cd");
                Message msg = Message.obtain(null, MSG_SET_
STRING_VALUE);
                msg.setData(b);
                mClients.get(i).send(msg);
            } catch (RemoteException e) {
     // The client is dead. Remove it from the list; we are
going
    // through the list from back to front so this is safe to do
                // inside the loop.
                mClients.remove(i);
            }
        }
    }
    @Override
    public void onCreate() {
        super.onCreate();
        timer.scheduleAtFixedRate(new TimerTask() {
            public void run() {
                onTimerTick();
            }
        }, 0, 100L);
        isRunning = true;
    }
```

```
@Override
    public int onStartCommand(Intent Intent, int flags, int
startId) {
        Log.i("MyService", "Received start id " + startId + ": "
+ Intent);
        return START_STICKY; // run until explicitly stopped.
    }
    public static boolean isRunning() {
        return isRunning;
    }
    private void onTimerTick() {
        try {
            counter += incrementby;
            sendMessageToUI(counter);
        } catch (Throwable t) {
// you should always ultimately catch all
            // exceptions in timer tasks.
        }
    }
    @Override
    public void onDestroy() {
        super.onDestroy();
        if (timer != null) {
            timer.cancel();
        }
        counter = 0;
        isRunning = false;
    }
}
```

5. Navigate to src → com.example.boundserviceexample → MainActivity.java file.
6. Modify the code in MainActivity.java file as shown in Code Snippet 15.

Code Snippet 15:

```
package com.example.boundserviceexample;
import android.app.Activity;
import android.content.ComponentName;
import android.content.Context;
import android.content.Intent;
import android.content.ServiceConnection;
import android.os.Bundle;
import android.os.Handler;
import android.os.IBinder;
import android.os.Message;
import android.os.Messenger;
import android.os.RemoteException;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;
public class MainActivity extends Activity {
    Button Servicestart, Servicestop, btnBindService,
btnUnbindService,btnaddby100;
    TextView textStatus, textIntValue, textStrValue;
    Messenger mServicecall = null;
    boolean mIsBound;
    final Messenger mMessenger = new Messenger (new
IncomingHandler());
    class IncomingHandler extends Handler {
        @Override
```

```
public void handleMessage(Message msg) {
            switch (msg.what) {
                case MyBoundService.MSG_SET_INT_VALUE:
textIntValue.setText("Status : " + msg.arg1);
                break;
                case MyBoundService.MSG_SET_STRING_VALUE:
                String str1 = msg.getData().getString("str1");
                textStrValue.setVisibility(View.GONE);
                textStrValue.setText("Str Message: " + str1);
                break;
                default:
                    super.handleMessage(msg);
            }
        }
    }
   private ServiceConnection mConnection = new
ServiceConnection() {
        @Override
        public void onServiceConnected(ComponentName className,
IBinder Service) {
            mServicecall = new Messenger(Service);
            textStatus.setText("Attached.");
            try {
                Message msg = Message.obtain(null,
                        MyBoundService.MSG_REGISTER_CLIENT);
                msg.replyTo = mMessenger;
                mServicecall.send(msg);
            } catch (RemoteException e) {
   // In this case the Service has crashed before we could even
// do  anything with it
```

```
                    }
        }
        public void onServiceDisconnected (ComponentName
className) {
   // This is called when the connection with the Service
// has been unexpectedly disconnected - process crashed.
            mServicecall = null;
            textStatus.setText("Disconnected.");
        }
    };
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Servicestart = (Button) findViewById (R.id.Servicestart);
        Servicestop = (Button) findViewById(R.id.Servicestop);
        btnBindService = (Button) findViewById (R.id.
btnBindService);
        btnUnbindService = (Button) findViewById (R.id.
btnUnbindService);
        textStatus = (TextView) findViewById(R.id.textStatus);
        textIntValue = (TextView) findViewById (R.id.
textIntValue);
        textStrValue= (TextView) findViewById(R.id.textStrValue);
        btnaddby100 = (Button) findViewById(R.id.btnaddby100);
    Servicestart.setOnClickListener(new View.OnClickListener() {
           @Override
            public void onClick(View v) {
```

```
// TODO Auto-generated method stub

startService(new Intent(MainActivity.this, MyBoundService.
class));

            }

        });

       Servicestop.setOnClickListener(new View.OnClickListener()
{

            @Override

                    public void onClick(View v) {

                doUnbindService();

                stopService(new Intent(MainActivity.this,
MyBoundService.class));

            }

        });

        btnBindService.setOnClickListener (new View.
OnClickListener() {

            @Override

                    public void onClick(View v) {

                doBindService();

            }

        });

        btnUnbindService.setOnClickListener (new View.
OnClickListener() {

            @Override

                    public void onClick(View v) {

                doUnbindService();

            }

  });

      btnaddby100.setOnClickListener(new View.OnClickListener()
{

@Override
```

```
                    public void onClick(View v) {
                sendMessageToService(100);
            }
        });
        restoreMe(savedInstanceState);
        CheckIfServiceIsRunning();
    }
    @Override
    protected void onSaveInstanceState(Bundle outState) {
        super.onSaveInstanceState(outState);
        outState.putString("textStatus", "Method called:" +
textStatus.getText().toString());
        outState.putString("textIntValue", textIntValue.
getText().toString());
        outState.putString("textStrValue", textStrValue.
getText().toString());
    }
    private void restoreMe(Bundle state) {
        if (state != null) {
   textStatus.setText(state.getString("textStatus"));
textIntValue.setText(state.getString("textIntValue"));
textStrValue.setText(state.getString("textStrValue"));
        }
    }
   private void CheckIfServiceIsRunning() {
 // If the Service is running when the activity starts, we want
to
    // automatically bind to it.
        if (MyBoundService.isRunning()) {
            doBindService();
}
```

```
     }
    private void sendMessageToService(int intvaluetosend) {
        if (mIsBound) {
            if (mServicecall != null) {
                try {
                    Message msg = Message.obtain(null,
MyBoundService. MSG_SET_INT_VALUE,intvaluetosend, 0);
                    msg.replyTo = mMessenger;
                    mServicecall.send(msg);
                } catch (RemoteException e) {
                }
            }
        }
    }
    void doBindService() {
        bindService(new Intent(this, MyBoundService.class),
mConnection,
                Context.BIND_AUTO_CREATE);
        mIsBound = true;
        textStatus.setText("Binding.");
    }
void doUnbindService() {
        if (mIsBound) {
// If we have received the service, and hence registered with
it,
     // then now is the time to unregister.
            if (mServicecall != null) {
                try {
                    Message msg = Message.obtain(null,
MyBoundService.MSG_UNREGISTER_CLIENT);
```

```
                    msg.replyTo = mMessenger;
                    mServicecall.send(msg);
                } catch (RemoteException e) {
    // There is nothing special we need to do if the service has
          // crashed.
                }
            }
            // Detach our existing connection.
            unbindService(mConnection);
            mIsBound = false;
            textStatus.setText("Unbinding.");
        }
    }
    @Override
    protected void onDestroy() {
        super.onDestroy();
        try {
            doUnbindService();
        } catch (Throwable t) {
        }
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if
//it is present.
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
   }
}
```

7. Click res → layout → activity_main.xml file to open the file for editing.
8. Modify the code as shown in Code Snippet 16.

Code Snippet 16:

```
<LinearLayout xmlns:android=”http://schemas.android.com/apk/res/
android”
    xmlns:tools=”http://schemas.android.com/tools”
android:layout_width=”match_parent”
    android:layout_height=”match_parent” android:paddingLeft=”@
dimen/activity_horizontal_margin”
    android:paddingRight=”@dimen/activity_horizontal_margin”
    android:paddingTop=”@dimen/activity_vertical_margin”
    android:paddingBottom=”@dimen/activity_vertical_margin”
tools:context=”.MainActivity”
    android:orientation=”vertical”>
    <TextView
        android:layout_width=”wrap_content”
        android:layout_height=”wrap_content”
        android:text=”CallBack Used “
        android:id=”@+id/textStatus”
        android:layout_weight=”1”
        android:textSize=”20dp” />
    <LinearLayout
        android:orientation=”horizontal”
        android:layout_width=”match_parent”
        android:layout_height=”wrap_content”
        android:layout_weight=”1”>
        <Button
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Start Service”
            android:id=”@+id/Servicestart”
            android:layout_weight=”1” />
```

```
        <Button
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Stop Service”
            android:id=”@+id/Servicestop”
            android:layout_weight=”1” />
    </LinearLayout>
    <TextView
        android:layout_width=”wrap_content”
        android:layout_height=”wrap_content”
        android:text=”String Value will be displayed here”
        android:id=”@+id/textIntValue”
        android:layout_weight=”1”
        android:textSize=”20dp” />
    <TextView
        android:layout_width=”wrap_content”
        android:layout_height=”wrap_content”
        android:text=”Output”
        android:id=”@+id/textStrValue”
        android:layout_weight=”1”
        android:textSize=”20dp” />
  <TextView
        android:layout_width=”wrap_content”
        android:layout_height=”wrap_content”
        android:text=”Notice the onBound() Effect”
        android:id=”@+id/showvalue”
        android:layout_weight=”1”
        android:textSize=”20dp” />
```

```
    <Button
        android:layout_width=”match_parent”
        android:layout_height=”wrap_content”
        android:text=”Add by 100”
        android:id=”@+id/btnaddby100”
        android:layout_weight=”0” />
    <LinearLayout
        android:orientation=”horizontal”
        android:layout_width=”match_parent”
        android:layout_height=”wrap_content”
        android:layout_weight=”1”>
        <Button
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Bind Service”
            android:id=”@+id/btnBindService”
            android:layout_weight=”1” />
        <Button
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Unbind Service”
            android:id=”@+id/btnUnbindService”
            android:layout_weight=”1” />
</LinearLayout>
</LinearLayout>
```

9. Navigate to manifests and open AndroidManifest.xml file.

10. Modify the code to add the <Service> element as shown in Code Snippet 17.

Code Snippet 17:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/
android"
    package=" com.example.boundserviceexample " >
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <activity
            android:name=".MainActivity"
            android:label="@string/app_name" >
            <Intent-filter>
            <action android:name="android.Intent.action.MAIN" />
                <category android:name="android.Intent.category.
LAUNCHER" />
            </Intent-filter>
        </activity>
        <Service android:name=".MyBoundService"></Service>
    </application>
</manifest>
```

Once the application is executed the output will be as shown in figure 8.5.

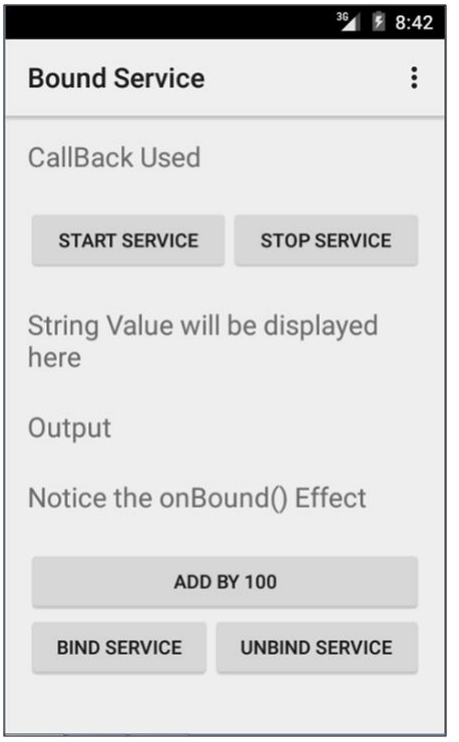


Figure 8.5: Bound Service Example

Click START SERVICE, then click ADD BY 100 and finally, click BIND SERVICE to get the output as shown in figure 8.6.

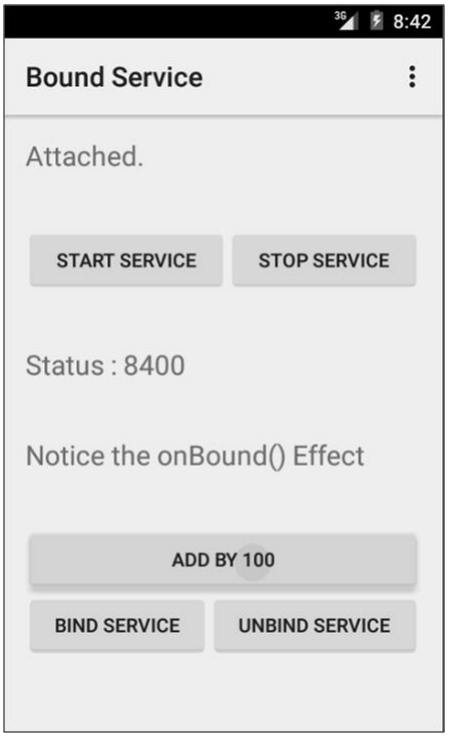


Figure 8.6: Service Bounded

The UNBIND SERVICE button will unbind the service. The counter can be increased by 100 by clicking the ADD BY 100 button. When the button is clicked, a message is sent to the service with the count to increment. Once received, the service increments the status count.

The STOP SERVICE button will stop the service and unbind the service. The output of stopping the service is shown in figure 8.7.

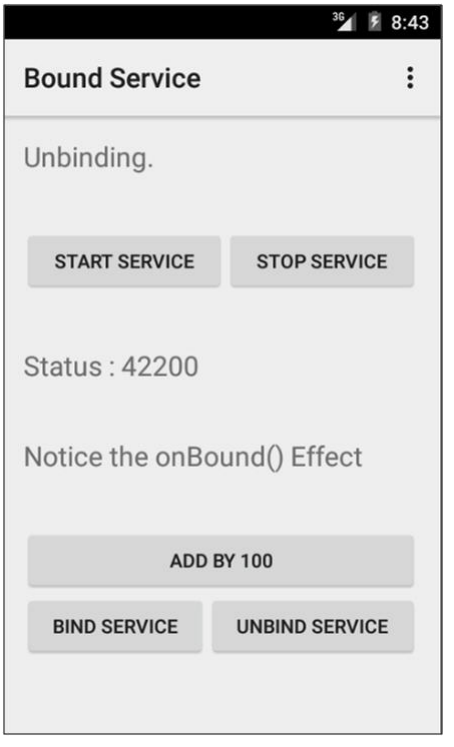


Figure 8.7: Service Stopped

8.3 Broadcast Receiver and Intent filters

Broadcast Receivers is also a type of component and plays an important part in building an Android application. It plays a specific role in the application development and is basically responsible for responding to system-wide broadcast announcements. On the other hand, Intents are responsible for binding the different individual components to each other at runtime.

8.3.1 Overview of Broadcast Receiver

A broadcast receiver is an Android component which allows the application to register for system or application events. The operating system and other applications broadcast announcements about the things that are happening. For example, a broadcast may be sent about the screen turning OFF, the change in battery status, Wi-Fi turning ON, or a picture that was captured. Broadcast receivers do not display a user interface; they might create a status bar notification to inform the user about a broadcast event. Each broadcast is delivered as Intent. In other words, the messages are sometimes called events or intents.

Applications can generate broadcasts to send events not knowing who will receive them. Receivers that require the information, subscribe to specific messages using filters. If the message matches a filter, the receiver is activated and notified of the message.

Broadcast receivers are great for performing small amounts of work, in response to an external stimulus, such as posting an announcement to the user after being sent a new GPS location report. They are active for a fixed amount of time; fairly strong guarantees can be made about not killing their process while running. However, they are not suitable for anything of indeterminate length, such as networking.

8.3.2 Implementing Broadcast Receivers

Following are the two steps to implement a broadcast receiver:

1. You have to create a subclass of Android's Broadcast Receiver.
2. You have to implement the onReceive() method.

Note - Broadcast receivers are registered in AndroidManifest.xml file using the <receiver> element or context.registerReceiver() method in Java class.

☐ Implementing the onReceive() Method

Android calls the onReceive() method on all registered broadcast receivers, when a matching broadcast Intent is identified. For example, a user wants to be notified when-ever the battery level is low. In this case, the receiver can be registered to the event Intent.ACTION_BATTERY_LOW. So, the moment battery level falls below a de¬fined threshold, the onReceive() method will be called.

The onReceive() method accepts the following two arguments:

- **Context**: This object is used to access additional information or to start services or activities.
- **Intent**: The Intent object is the action you used to register your receiver. This object contains additional information that can be used in the implementation of onReceive() method.

The Intent object passed to the onReceive() method often contains additional in-formation that can be used to determine more about the event. On the other hand, the Context object is required if the user wants to start an Activity or a Service.

The section explains BroadCastReceiverExample in detail where user will get the message from Broadcast Receiver whatever the user types in the text box.

The steps to create a BroadCastReceiverExample project are as follows:

1. Create a project named BroadCastReceiverExample in Android Studio.
2. Navigate to src → com.example.broadcastreceiverexample package.
3. Create a new class called MyBroadcast.
4. Modify the code in MyBroadcast class as shown in Code Snippet 18.

Code Snippet 18:

```
package com.example.broadcastreceiverexample;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.os.Vibrator;
import android.widget.Toast;
public class MyBroadcast extends BroadcastReceiver{
@Override
public void onReceive(Context context, Intent Intent) {
     Toast.makeText(context, "Congrats you learnt sending
Broadcast!!!!.",
          Toast.LENGTH_LONG).show();
}
}
```

5. Navigate to src → com.example.broadcastreceiverexample → MainActivity.java file.
6. Modify the code in MainActivity.java file as shown in Code Snippet 19.

Code Snippet 19:

```
package com.example.broadcastreceiverexample;
import android.os.Bundle;
import android.app.Activity;
import android.app.AlarmManager;
import android.app.PendingIntent;
```

```
import android.content.Intent;
import android.view.Menu;
import android.view.View;
import android.widget.EditText;
import android.widget.Toast;
public class MainActivity extends Activity {
      @Override
      protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
            setContentView(R.layout.activity_main);
      }
      public void startAlert(View view) {
      EditText text = (EditText) findViewById(R.id.time);
      int i = Integer.parseInt(text.getText(). toString());
            Intent Intent = new Intent(this, MyBroadcast. class);
            PendingIntent pendingIntent = PendingIntent.
getBroadcast(
this.getApplicationContext(), 234324243, Intent, 0);
            AlarmManager alarmManager = (AlarmManager)
getSystemService(ALARM_SERVICE);
            alarmManager.set(AlarmManager.RTC_WAKEUP, System.
currentTimeMillis() + (i * 1000), pendingIntent);
            Toast.makeText(this, "youwillgetthebroadcast
messageafter" + i+ " seconds",
            Toast.LENGTH_LONG).show();
      }
      @Override
      public boolean onCreateOptionsMenu(Menu menu) {
            // Inflate the menu; this adds items to the action bar
if it is present.
```

```
    getMenuInflater().inflate(R.menu.main, menu);
return true;
}
}
```

7. Navigate to Manifests and open AndroidManifest.xml file.
8. Modify the code to add the <receiver> element as shown in Code Snippet 20.

Code Snippet 20:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/ res/
android"
package="com.example.broadcastreceiverexample"
android:versionCode="1"
android:versionName="1.0" >
<uses-sdk
android:minSdkVersion="8"
android:targetSdkVersion="17" />
<uses-permission android:name="android.permission. VIBRATE" >
</uses-permission>
<application
android:allowBackup="true"
android:icon="@drawable/ic_launcher"
android:label="@string/app_name"
android:theme="@style/AppTheme" >
<activity
android:name="com.example.broadcastreceiverexample.
MainActivity"
android:label="@string/app_name" >
<intent-filter>
<action android:name="android.Intent.action.MAIN" />
<category android:name="android.Intent.category. LAUNCHER" />
```

```
</intent-filter>
</activity>
<receiver
android:name=".MyBroadcast"
android:enabled="true" >
</receiver>
</application>
</manifest>
```

9. Navigate to res → layout → activity_main.xml file.
10. Modify the code as shown in Code Snippet 21.

Code Snippet 21:

```
?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android. com/apk/
res/android"
android:layout_width="fill_parent"
android:layout_height="fill_parent"
android:orientation="vertical" >
<EditText
            android:id="@+id/time"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignParentLeft="true"
            android:layout_alignParentTop="true"
            android:layout_marginLeft="32dp"
            android:layout_marginTop="92dp"
            android:ems="10"
            android:hint="Number of seconds"
            android:inputType="numberDecimal" >
<requestFocus />
</EditText>
```

```
<Button
      android:id="@+id/ok"
     android:layout_width="wrap_content"
     android:layout_height="wrap_content"
     android:layout_below="@+id/time"
      android:layout_centerHorizontal="true"
      android:layout_marginTop="97dp"
      android:onClick="startAlert"
      android:text="Start Counter" />
      <TextView
           android:id="@+id/abcv"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:gravity="center"
            android:text="Broadcast Receiver Prompt Message
Example"
      android:textSize="18sp" />
</RelativeLayout>
```

Once the application is executed, the output will be as shown in figure 8.8.

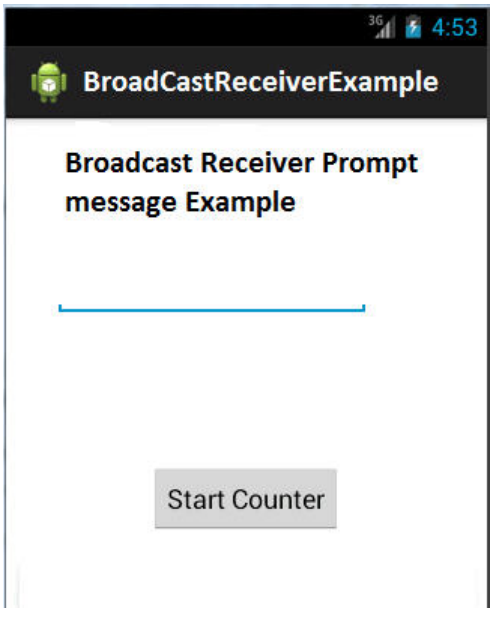


Figure 8.8: Broadcast Message Example

Enter 3 in the Edit box and the output will be as shown in figure 8.9.

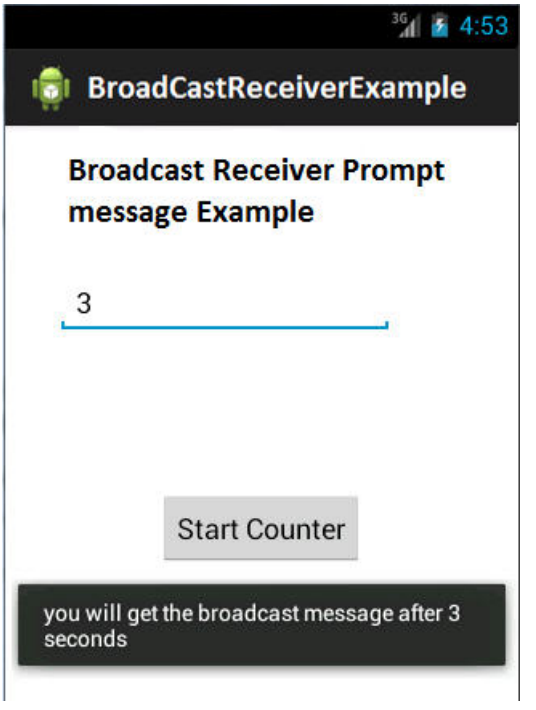


Figure 8.9: Broadcast Pending Request

After three seconds the message will be displayed as shown in figure 8.10.

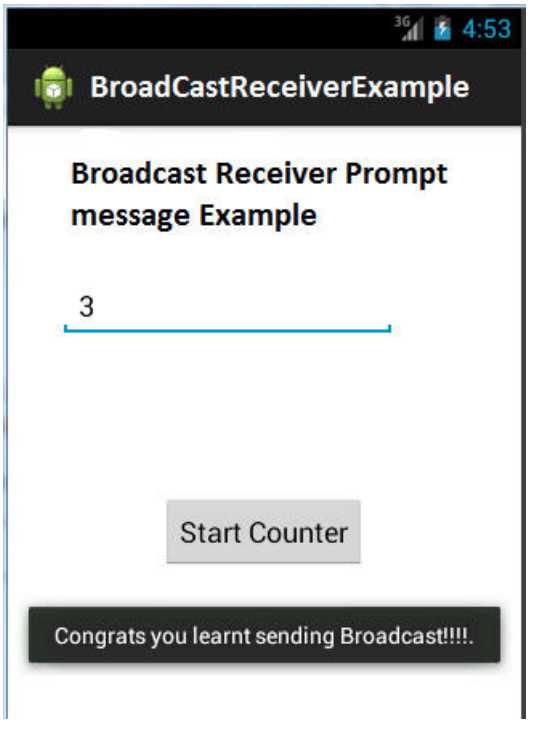


Figure 8.10: Broadcast Message

8.3.3 System Broadcast

Several system events are defined as final static fields in the Intent class. In the Application Program Interface (API), there are many classes that have specific broadcast events.

Table 8.1 lists some of the available events.

Event	Usage
Intent.ACTION_BATTERY_LOW	The battery level has dropped below a threshold.
Intent.ACTION_BATTERY_OKAY	The battery level has increased again.
Intent.ACTION_BOOT_COMPLETED	Android is up and running.
Intent.ACTION_DEVICE_STORAGE_LOW	Storage space on the device is becoming less.
Intent.ACTION_DEVICE_STORAGE_OK	The storage situation has improved again.
Intent.ACTION_HEADSET_PLUG	A headset was plugged in or a previously plugged headset was removed.
Intent.ACTION_LOCALE_CHANGED	The language of the device has been changed by the user.
Intent.ACTION_MY_PACKAGE_REPLACED	The application has been updated.
Intent.ACTION_PACKAGE_ADDED	A new application has been installed.
Intent.ACTION_POWER_CONNECTED	The device has been plugged in.
Intent.ACTION_POWER_DISCONNECTED	The device has been disconnected again.
KeyChain.ACTION_STORAGE_CHANGED	The key store has changed.
BluetoothDevice.ACTION_ACL_ CONNECTED	A Bluetooth ACL connection has been established.
AudioManager.ACTION_AUDIO_ BECOMING_ NOISY	The internal audio speaker is about to be used instead of other output means (like a headset).

Table 8.1: Intent Constants

8.3.4 Role of Filters

When Intent is sent to the Android system, it determines suitable applications for this Intent. In case several components have been registered for this type of Intent, Android offers the user the choice to open one of them.

This decision is based on IntentFilters. To inform the system which implicit Intents they can handle, activities, services, and broadcast receivers can have more than one Intent filter. Each filter declares the capability of a component and a set of Intents that a component can handle. It filters in the Intents of a desired type, while filtering out the unwanted Intents. However, only unwanted implicit Intents that do not name a target class are filtered out. An explicit Intent is always delivered to its target without consulting the filter.

Consider an example of Notepad sample application which contains two filters, where a component has separate filter for each jobs it can do; one for starting up with a specific note that a user can view or edit and second for starting a new, blank note that a user can fill in, and save.

An Intent filter is an instance of the IntentFilter class. However, the Android system requires pre-defined knowledge about the capabilities of a component before launching it. Therefore, Intent filters are usually not set up in Java code, but defined in the AndroidManifest.xml file as <Intent-filter> elements.

Code Snippet 22 shows an example of registering an activity. Following code will register an Activity for the Intent which is triggered when a user wants to view.

Code Snippet 22:

```
<activity android:name=".BrowserActivitiy"
android:label="@string/app_name">
<Intent-filter>
<action android:name="android.Intent.action.VIEW" />
<category android:name="android.Intent.category.DEFAULT" />
<data android:scheme="http"/>
</Intent-filter>
</activity>
```

A filter has fields that parallel the action, data, and category fields of an Intent object. An implicit intent is tested against the filter in all three categories. In order to be delivered to a component that owns the filter, an implicit intent must pass all three tests. In case it fails even one of them, the Android system will not deliver it to the component. However, a component can have multiple intent filters, hence, an intent that could not pass through one of the component's filters might pass through another.

Caution: An Intent filter is not meant for security. While it opens a component to receive only a certain kind of implicit Intent, it does not prevent explicit Intents from attacking the component.

8.3.5 Intent Matching

Matching of intents with intent filters is performed to discover a target component that is to be activated and also to get information about the set of components available on the device. For instance, the Android system populates the application launcher that is available for the user to launch, by finding all the activities with intent filters that specify the android.Intent.action.MAIN action and android.Intent.category.LAUNCHER category. In the AndroidManifest.xml file, it also shows the icons and labels of the activities in the launcher. In the same way, it discovers the home screen by looking for the activity with android.Intent.category.HOME in its filter.

An intent filter can match against the three characteristics – actions, data, and categories in an Intent. To order multiple matching filters, intent filters include a 'priority' value. Intent filter objects are generally created in XML as part of a package's AndroidManifest.xml file, using intent filter tags.

- **Intent-Matching Rules**

  An intent matching is based on three rules. To match an IntentFilter with an Intent, three conditions must be fulfilled - the action, the category, and the data (both the data type and data scheme+authority+path, if specified) must match.

  Action matches if any of the given values match the Intent action or if no action were specified in the filter.

  The Data characteristic is divided into four attributes: types, schemes, authority, and path. Data Type matches if any of the given values match the Intent type. The Intent type is determined by invoking resolveType(ContentResolver). A wildcard can be used for the MIME sub-type, in both the Intent and IntentFilter, so that the type 'audio/*' will match 'audio/mpeg', 'audio/aiff', 'audio/*', and so on. MIME type matching here is case sensitive. Therefore, only lower case letters should be used for the MIME types.

  Data Scheme matches when any of the given values match the Intent data's scheme. The Intent scheme is determined by calling getData() and getScheme() on that URI. Scheme matching is case sensitive. Therefore, only lower case letters should be used.

  Data Authority matches the given values with Intent data's authority and any one of the data's schemes in the filter matches the Intent or no authorities are supplied to the filter. The Intent authority is determined by calling getData() and getAuthority() on that URI. Authority matching is case sensitive and only lower case letters can be used.

  Data Path matches any given values with the Intent's data path and when both a scheme and an authority in the filter matches the Intent or no paths are supplied in the filter. The Intent authority is determined by calling getData() and getPath() on the URI.

8.3.6 Filters in Manifest

This section explains how filters in Android are reflected in the AndroidManifest.xml file.

Intents activate core components of an application, which are its activities, Services, and broadcast receivers. Intent is a collection of information that describes a desired action—including the data that needs to be acted upon, the category of component that performs the action, and other important instructions. Android searches for appropriate component to reply to the intent, launches a new instance of the component if required and passes it to the Intent object.

By using intent filters, components specify their capability - that is the kinds of Intents they can respond to. The Android system needs to know which intents can be handled by a component before it launches the component. Intent filters are specified in the manifest as <Intent-filter> elements. In other words, using Intent Filters an application component tells Android that they can service action requests from others. A component can have any number of filters, every filter describing a diverse capability. To register an application component as an Intent handler, the <intent-filter> tag is used within the components node in the AndroidManifest.xml file.

When an intent explicitly names a target component, it activates that component; the filter does not play any role. However, when Intent does not specify a target by name, it activates a component only if it can pass through one of the component's filters.

The default intent filter declaration in AndroidManifest.xml file is as shown in Code Snippet 23.

Code Snippet 23:

```
<intent-filterandroid:icon="resource path from
drawables" android:label="resource path of string"
android:priority="integer"> </intent-filter>
```

8.3.7 Filters in Broadcast Receivers

When the user registers a BroadcastReceiver to be executed in the main activity thread, the receiver is called with any broadcast Intent that matches the filter, in the main application thread.

The system can broadcast Intents that are 'sticky'—the Intents that stay around even after the broadcast is complete—to be sent to any later registrations. If the IntentFilter matches any of these sticky Intents, that Intent will be returned and sent to the receiver as if it had just been broadcast.

There might be times when multiple sticky Intents may match the filter and then in that case each of the Intents will be sent to the receiver. Only one of these Intents can be returned directly by the function and that is randomly decided by the system.

When the user knows that the registered Intent is sticky, then null can be supplied for the receiver. In case when no receiver is registered, the function simply returns the sticky Intent that matches the filter. The same rules apply when there are multiple matches.

Code Snippet 24 explains the usage of filters in broadcast receivers.

Code Snippet 24:

```
<receiverandroid:name=".BCastReceiver">
<intent-filter>
<action android:name="android.Intent.action.MAIN"/>
<category android:name="android.Intent.category.LAUNCHER"/>
<action android:name="android.Intent.action.ACTION_POWER_
CONNECTED"/>
</intent-filter>
</receiver>
...
```

The intent-filter here is used to specify the action the receiver has to perform. Here in this Code Snippet, ACTION_POWER_CONNECTED, tells the broadcast receiver to notify when power cable connected to device and is handled by the onReceive() method in the BCastReceiver class.

Note - This method cannot be called from a BroadcastReceiver component, that is, from a BroadcastReceiver that is declared in an application's manifest. However, this method can be called from another BroadcastReceiver that has itself been registered at run time with registerReceiver(BroadcastReceiver, IntentFilter), since the duration of such a registered BroadcastReceiver depends on the object that registered it.

Check Your Progress

1. A service can be created by extending which of the following classes?

1.	IntentService	2.	ServiceManager
3.	ServiceInterface	4.	Service

(A)	1, 3	(B)	2, 3
(C)	3, 4	(D)	1, 4

2. Which of the following statements are true?

1.	The Service class name needs to be mentioned in the Manifest file.
2.	Intent.ACTION_LOCALE_CHANGED indicates that the user has moved out of service area.
3.	A broadcast receiver allows the application to register for system or application events.
4.	The onReceive() method is called once a service is started.

(A)	3, 4	(B)	1, 2
(C)	2, 4	(D)	1, 3

3. Which of the following method is called to start a service?

(A)	runService()	(B)	beginService()
(C)	startService()	(D)	loadService()

4. A service can be started from an activity or a component by passing a _____ to startService() method.

(A)	Method	(B)	Class
(C)	Intent	(D)	Filter

5. Match the following methods with the corresponding description.

	Method		Description
a.	onBind()	1.	Components can use this to stop a service.
b.	stopSelf()	2.	System initiates a component.
c.	stopService()	3.	Used to create a bound service.
d.	onCreate()	4.	A started service stops itself.

(A)	a-3, b-4, c-1, d-2	(B)	a-3, b-1, c-4, d-2
(C)	a-2, b-4, c-1, d-3	(D)	a-1, b-3, c-4, d-2

Answers

1.	D
2.	D
3.	C
4.	C
5.	A

Session

08

Services, Broadcast Receivers, and Intent Filters

Summary

- A service is an application component that is able to carry out long-running operations in the background. It does not provide a user interface.
- In order to interact with the components, a service can let other components bind to it and perform Inter Process Communication (IPC). A service runs in the main thread of the application that hosts it.
- A Broadcast Receiver is a component that responds to system-wide broadcast announcements.
- Activities, Services, and Broadcast Receivers have intent filters to inform the system which intents they can handle.
- Intent filters can handle two types of intents: implicit and explicit.
- Matching of intents with intent filters is done to discover a target component to be activated, and also to get information about the set of components available on the device.
- To match an intent filter with an Intent, three conditions must be fulfilled: the action, category, and the data.

Onlinevarsity

INDUSTRY BEST PRACTICES

SYNC WITH THE INDUSTRY

09

Google API

Welcome to the Session, **Google API.**

This session discusses about the various components that are required to use Google API. It explains the process to create map based activities using Google Maps as a user interface element. The session further proceeds and explains location based services that help the user to find the current location of the device. Utilizing Play Services is explained in the later sections.

In this Session, you will learn to:

- Explain API and its uses
- Explain Google API
- Explain location based service

Session

09

Google API

9.1 Introduction

Every programming language has its own set of APIs. So, what is an API? It can be explained as follows:

- API stands for Application Programming Interface.
- It is a specification used by software components to communicate with each other.
- It is a library containing specifications for objects, classes, variables, and so on.
- It describes the ways in which a particular task is performed.

The Google API stands for 'Application Programmable Interface'. It is an interface that helps the programmers in the development of their applications by querying the Google database. Google API's basically consist of specialized Web services, programs, and specialized scripts. These are used by Internet application developers to find and process information on the Web. In other words, Google APIs are used as an added resource in their applications.

9.2 Google API

Google as an organization provides several services such as Google Drive, Google App Engine, Google Translate, Google Maps, and so on. Developers can integrate these services provided by Google into their application free of cost. Each of these services has a different use and the developer can choose which service will be used in their application. The developer needs to keep in mind that by using Google API, the application will rely on a network connection in order to function properly. Offline functionality needs to be implemented manually by the developer.

The session will discuss how Google API works with the following:

- Location Based Services
- Google Maps

9.2.1 Working with Location Based Services

Location Based Service (LBS) is an information service and has a number of uses in social networking. Location Based Services can be defined as follows:

- A set of applications that exploit the knowledge of the geographical position of a mobile device in order to provide services based on that information.
- Provides personalized services to mobile clients according to their current location.

With the incorporation of Global Positioning System (GPS) devices in Smartphones, LBS have become important in the past few years. iPhone was the first to give a huge boost to this kind of applications and now Android is following the same path.

There are different classes available in the location API to retrieve the location information of the user. Most important classes and interface that are to be used are as follows:

- LocationManager class provides access to location service. It includes string constants that return the providers name for two common location providers. The two common location providers are LocationManager.GPS_PROVIDER and LocationManager.NETWORK_PROVIDER.
- LocationProvider class is an abstract class for all the location providers. It provides information on the geographic location of the device by using different technologies.
- Location class is a data class that represents the geographic location such as latitude, longitude, and so on.
- LocationListener is an interface that provides the callback methods. These methods are invoked when the location changes. The listener object is required to be registered with the location manager.
- Criteria is a class that helps the application to select the suitable location provider.

Building your First LBS Application for Android

The first step in LBS is retrieving the user's current location and then using the coordinates to provide data for that location.

In general, the user analytics on the map are feasible in one of the following ways:

- Using the GPS device that comes with the mobile.
- Using the ID of the cell that the user is currently being served by.

The first approach is much easier and more accurate (since, the second provides only an approximation) as these days a large number of mobile phones have GPS devices incorporated within. The Android SDK's emulator can emulate changes in the user's location and provide dummy data for his coordinates.

To understand the concept, perform the following steps:

1. Create a new project named AndroidLbsGeocodingProject1 in Android Studio.
2. Navigate to res → layout folder.
3. Modify the code in activity_main.xml file as shown in Code Snippet 1.

Code Snippet 1:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/
                                                    android"
   android:layout_width="fill_parent"
```

```
    android:layout_height="fill_parent"
    android:orientation="vertical" >
<Button
    android:id="@+id/retrieve_location_button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Retrieve Location" />
</LinearLayout>
```

A button is added which will result in an event that will trigger the retrieval of the current location coordinates from a location provider.

4. Navigate to src → com.example.androidlbsgeocodingproject1.
5. Modify the code in MainActivity.java file as shown in Code Snippet 2.

Code Snippet 2:

```
package com.example.androidlbsgeocodingproject1;
import android.os.Bundle;
import android.app.Activity;
import android.content.Context;
import android.view.Menu;
import android.location.Location;
import android.location.LocationListener;
import android.location.LocationManager;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.Toast;
public class MainActivity extends Activity {
private static final long MINIMUM_DISTANCE_CHANGE_FOR_ UPDATES = 1;
// in Meters
```

```
private static final long MINIMUM_TIME_BETWEEN_UPDATES = 1000;
// in Milliseconds
       protected LocationManager locationManager;
       protected Button retrieveLocationButton;
       @Override
       protected void onCreate(Bundle savedInstanceState) {
           super.onCreate(savedInstanceState);
           setContentView(R.layout.activity_main);
           retrieveLocationButton = (Button) findViewById(R.id.
                                    retrieve_location_button);
           locationManager = (LocationManager) getSystemService
                             (Context.LOCATION_SERVICE);
           locationManager.requestLocationUpdates(LocationMana
ger.GPS_PROVIDER, MINIMUM_TIME_BETWEEN_UPDATES, MINIMUM_
DISTANCE_CHANGE_FOR_UPDATES, new MyLocationListener());
     retrieveLocationButton.setOnClickListener(new
           OnClickListener() {
           @Override
           public void onClick(View v) {
                 showCurrentLocation();
           }
     });
}
          protected void showCurrentLocation() {
         Location location = locationManager.
        getLastKnownLocation(LocationManager.GPS_ PROVIDER);
         if (location != null) {
           String message = String.format("Current Location \n
Longitude: %1$s \n Latitude: %2$s", location.getLongitude(),
location. getLatitude());
Toast.makeText(MainActivity.this, message, Toast.LENGTH_LONG).
                                                    show();
```

```
}
}
        @Override
        public boolean onCreateOptionsMenu(Menu menu) {
      // Inflate the menu; this adds items to the action bar if it
is present.
        getMenuInflater().inflate(R.menu.main, menu);
        return true;
}
    private class MyLocationListener implements LocationListener
{
   public void onLocationChanged(Location location) {
      String message = String.format("New Location \n Longitude:
%1$s \n Latitude: %2$s", location.getLongitude(), location.
getLatitude());
           Toast.makeText(MainActivity.this, message, Toast.
                                        LENGTH_LONG).show();
}
   public void onStatusChanged(String s, int i, Bundle b) {
   Toast.makeText(MainActivity.this, "Provider status changed",
           Toast.LENGTH_LONG).show();
}
   public void onProviderDisabled(String s) {
      Toast.makeText(MainActivity.this, "Provider disabled by the
user. GPS turned off", Toast.LENGTH_LONG).show();
}
   public void onProviderEnabled(String s) {
      Toast.makeText(MainActivity.this,
      "Provider enabled by the user. GPS turned on", Toast.
                                        LENGTH_ LONG).show();
}
}
}
```

In order to get started with the location capabilities of the Android API, the first step is to obtain a reference of the LocationManager class. An instance of this class will provide access to the system location services. This is achieved by invoking the getSystemService() method which it inherits from the Context parent class. Then, to obtain the location of the device the developer uses the requestLoca¬tionUpdates() method. The requestLocationUpdates() method accepts the name of the preferred location provider (in our case GPS), the minimum time interval for notifications (in milliseconds), the minimum distance interval for notifications (in meters), and finally a class implementing the LocationListener interface.

The interface declares methods for handling changes in the user's location as well as changes in the location provider's status. To implement the LocationListener interface, a private inner class named MyLocationListener is declared. The class overrides the abstract methods of the interface, LocationListener. Toasts mes¬sages are used in the method to provide information about the GPS status or any lo¬cation changes. The only UI element in the application is a button to which a listener is attached which when clicked invokes the onClick() method. The onClick() method in turn invokes the user defined showCurrentLocation() method. In the showCurrentLocation() method, the getLastKnownLocation() method of the LocationManager instance is executed which returns the last known location and stores it in a Location instance. From a Location instance, the developer obtains information regarding the user's altitude, latitude, longitude, speed, and so on. In order to execute the application, necessary permissions are required to be granted in the AndroidManifest.xml file. The permissions to be granted are as follows:

- ACCESS_FINE_LOCATION
- ACCESS_MOCK_LOCATION
- ACCESS_COARSE_LOCATION

6. Modify the code in AndroidManifest.xml file as shown in Code Snippet 3.

Code Snippet 3:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
xmlns:android="http://schemas.android.com/apk/res/ android"
package="com.example.androidlbsgeocodingproject1"
android:versionCode="1"
android:versionName="1.0" >
     <application
```

```
            android:allowBackup="true"
            android:icon="@drawable/ic_launcher"
            android:label="@string/app_name"
            android:theme="@style/AppTheme" >
      <activity
            android:name="com.example.
            androidlbsgeocodingproject1.MainActivity"
            android:label="@string/app_name" >
       <intent-filter>
      <action android:name="android.intent.action.MAIN" />
      <category android:name="android.intent.category. LAUNCHER" />
      </intent-filter>
       </activity>
</application>
<uses-permission android:name="android.permission.ACCESS_ FINE_
                                                    LOCATION" />
<uses-permission android:name="android.permission. ACCESS_MOCK_
                                                  LOCATION" />
<uses-permission android:name="android.permission.ACCESS_
                                        COARSE_LOCATION" />
</manifest>
```

7. Navigate to Tools menu → Android and select Android Device Monitor.
8. Click Emulation Control tab. The Location Controls section sends mock location data to the emulator.

9. Click Send in the Manual tab of the Location Controls section as there are already some dummy coordinates as shown in figure 9.1.

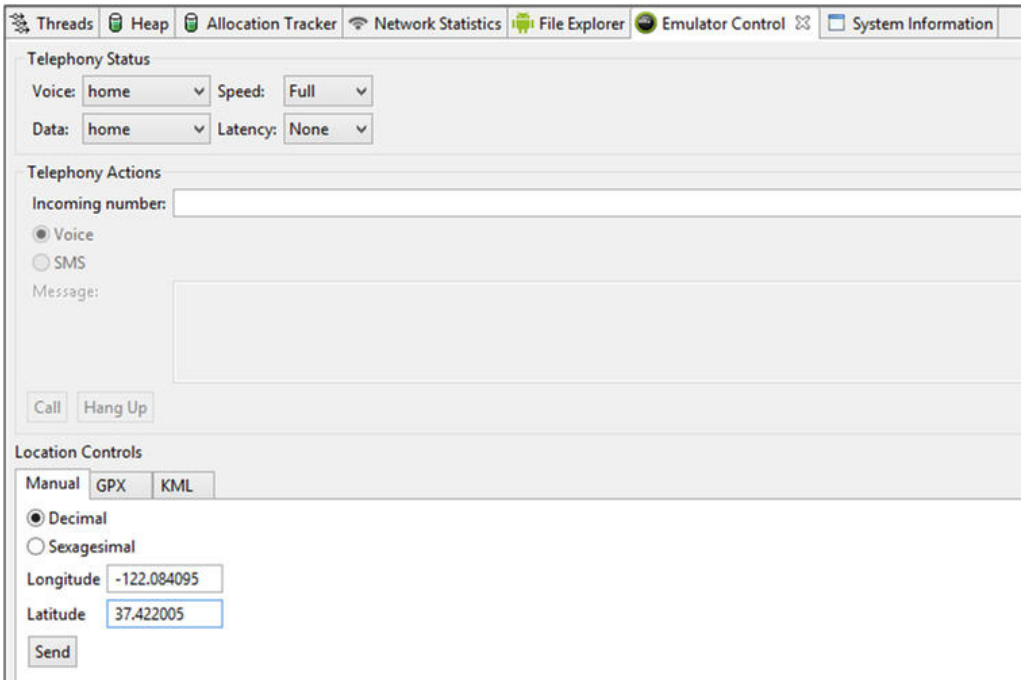


Figure 9.1: DDMS Showing Dummy Coordinates

When the data is sent to the emulator's GPS device by clicking Send, the listener who has been added will be triggered and the current location will be printed on the screen in the form of a Toast notification as shown in figure 9.2.

Figure 9.2: Retrieve Location Value

Since, the last known location exists, when the user clicks Retrieve Location, a not null location will be fetched and the same coordinates will again be displayed in the screen as shown in figure 9.3.

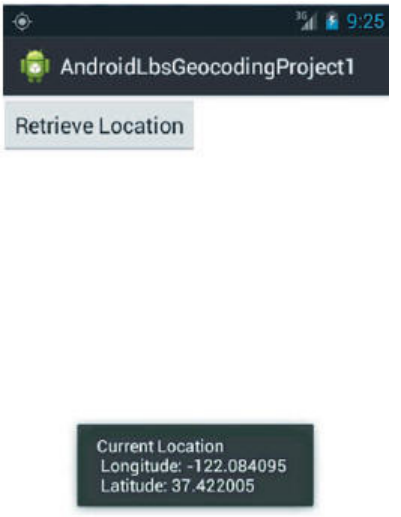


Figure 9.3: Location Show

Thus, the user can change the latitude and longitude in the DDMS and retrieve the updated values. In real-time, the data is automatically updated from the GPS.

9.2.2 Setting up of Android SDK for Google API

To work with Google Maps, ensure that the Android SDK has the necessary components installed. A key component is called the Google API's Add-on, which according to Google: "Is an extension to the Android SDK development environment that lets you develop applications for devices that include Google's set of custom applications, libraries, and services."

To install Google API, perform the following steps:

1. Navigate to Tools → Android and select SDK Manager.
2. Select Google APIs under the Android API being used. In this case it is Android 5.0.1.
3. Select Google Play services under Extras.
4. Click Install.

The selected packages will be installed as shown in figure 9.4.

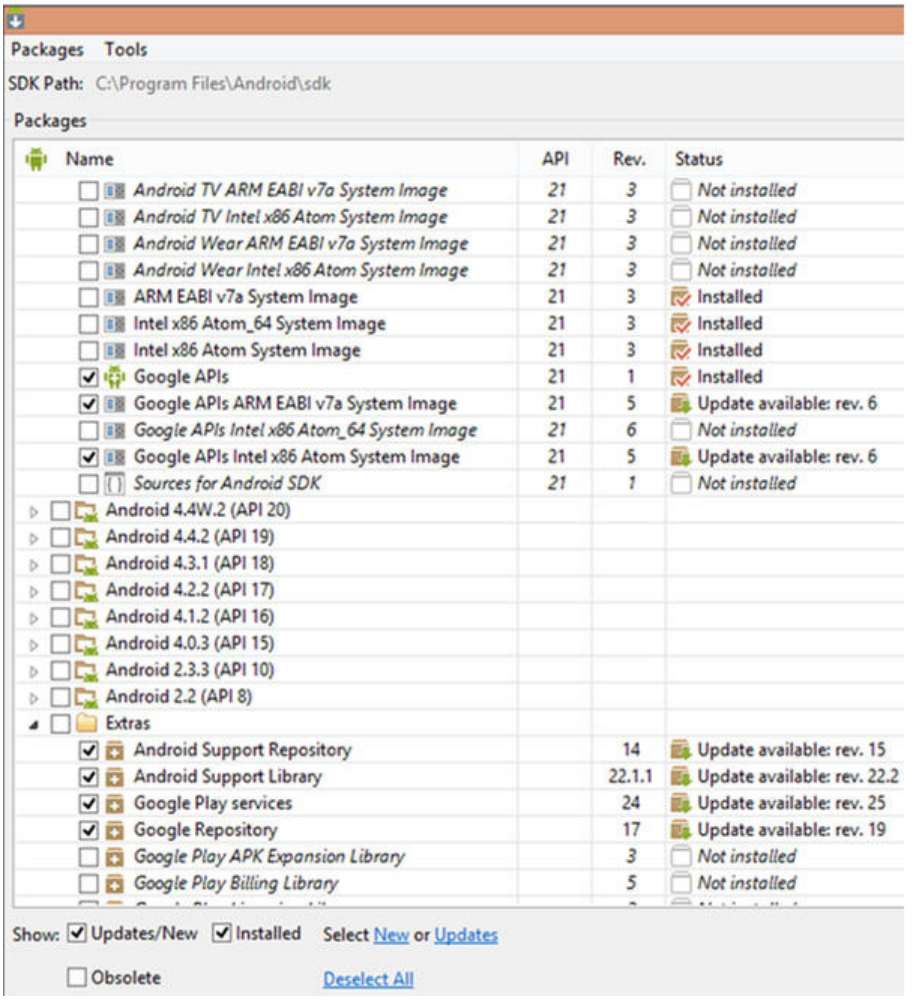


Figure 9.4: Google APIs and Google Play Services

9.2.3 Working with Google Maps

Google Maps allows the users to explore the world with rich maps provided by Google. It helps to identify locations with custom markers, augment the map data with image overlays, embed one or more maps as fragments, and much more.

In order to use Google Maps in an Android application, you need to make sure that the application is properly setup to be able to host an Activity Fragment.

Although, it only takes a couple of steps to prepare your application for using Google Maps, failing to complete or skipping one of these steps can quickly turn this into an error-prone process.

In short, the following points need to be taken care of to enable the Android application to work with Google Maps:

- The Android SDK should have libraries for Google Maps installed.
- The Activity class that will be responsible for showing the map needs to extend from ActivityFragment class.
- The application manifest file needs to be setup with the 'android.permission. INTERNET' permission.
- The application manifest file is also required to be setup with the Maps key.

Adding Dependencies for the Application

Now that Android SDK has been set up with the Google APIs and Google Play services, you can create Android Projects that target these APIs. To develop an application, perform the following steps:

1. Create a project named GoogleProject in Android Studio.
2. During the activity selection screen, select Google Maps Activity as shown in figure 9.5.

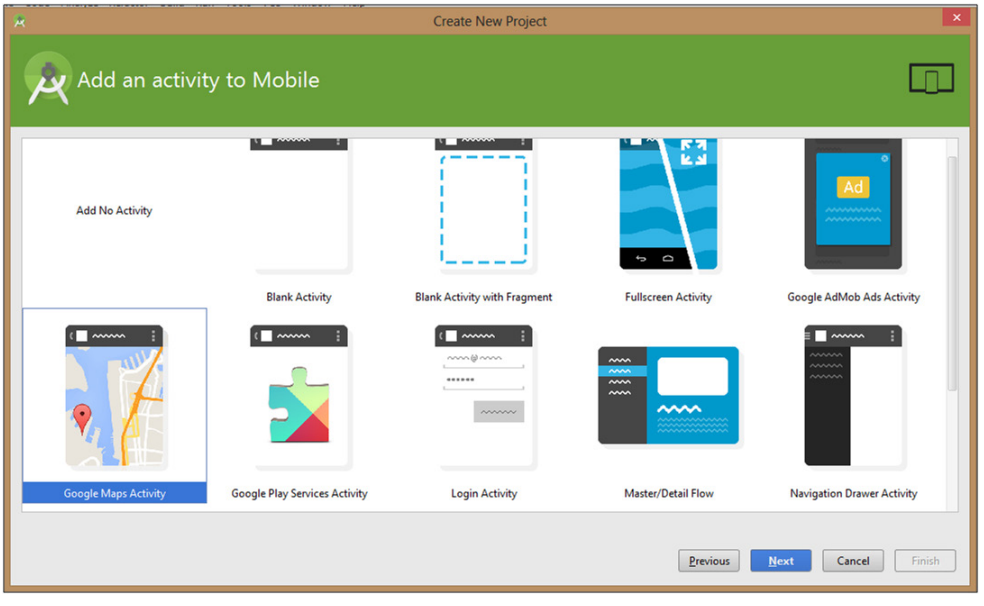


Figure 9.5: Selecting Google Maps Activity

If this step is done, then steps 3 to 7 can be skipped.

3. If the project is created using a Blank Activity, right-click app and select Open Module Settings.

4. From the Project Structure dialog box, select the Dependencies tab as shown in figure 9.6.

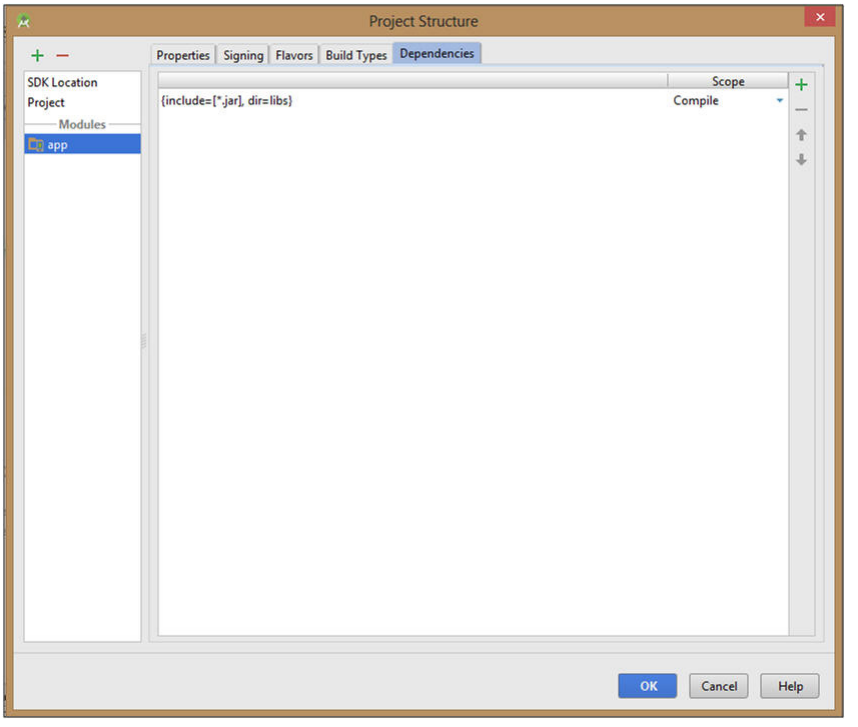


Figure 9.6: App Module Settings

5. Click the '+' sign and select Library Dependency to open Library Selection dialog box as shown in figure 9.7.

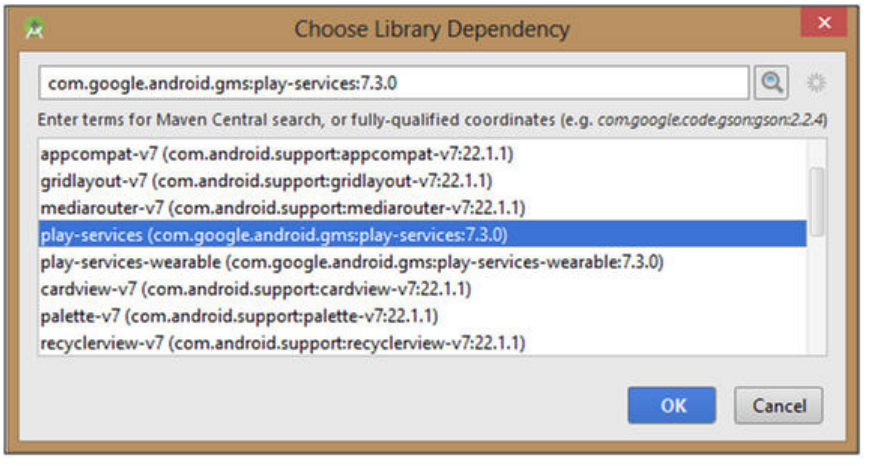


Figure 9.7: Library Selection Dialog Box

6. Select the play-services library and click OK.
7. Click OK again to confirm the dependencies.
8. Navigate to app → java → com.example.googleproject and add the import for the package com.google.android.gms.maps.GoogleMap.

Generating a key for the Application

1. Navigate to Build menu and select Generate Signed APK.
2. The Generate Signed APK dialog box is displayed as shown in figure 9.8.

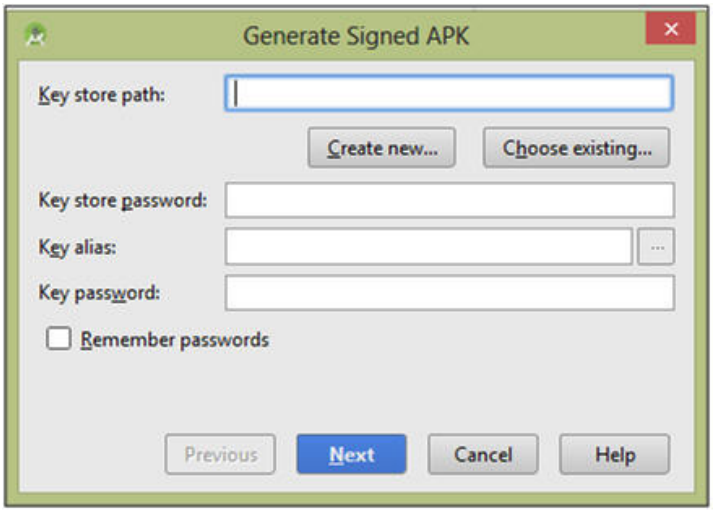


Figure 9.8: Generate Signed APK Dialog Box

3. If it is the first time key generation, click Create New under Key store path to open the New Key Store dialog box as shown in figure 9.9.

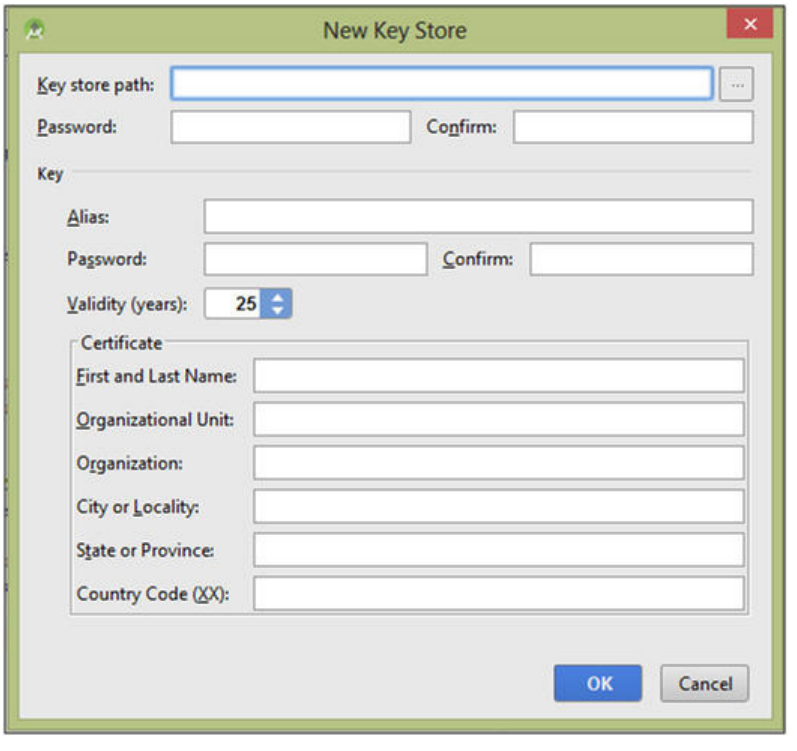


Figure 9.9: New Key Store Dialog Box

4. Enter the details as shown in figure 9.10.

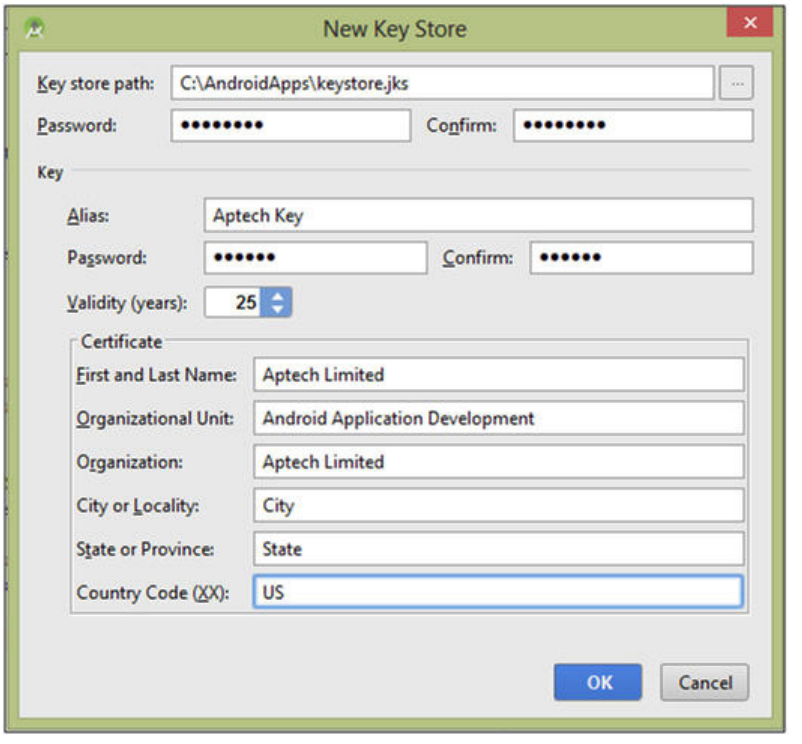


Figure 9.10: Keystore Details

5. Click OK to return to the Generate Signed APK dialog box with the details already entered as shown in figure 9.11.

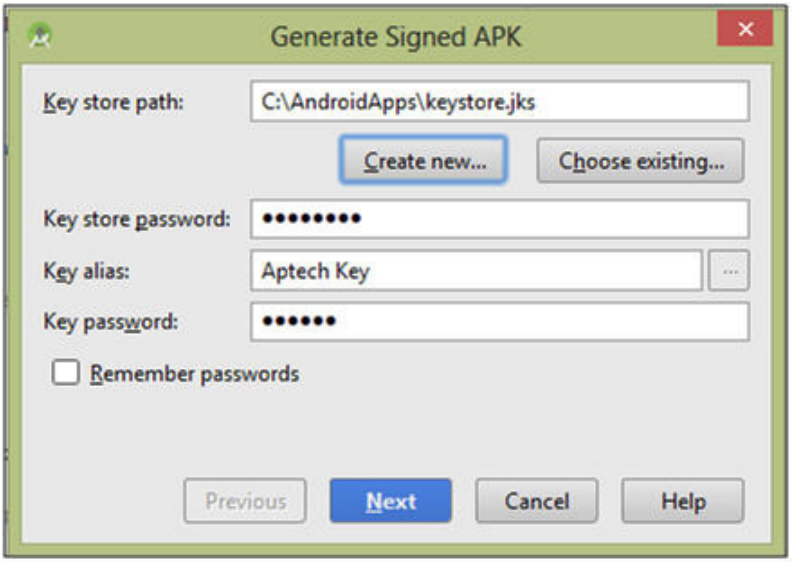


Figure 9.11: Generate Signed APK with the Details

6. Click Next to display the destination details screen as shown in figure 9.12.

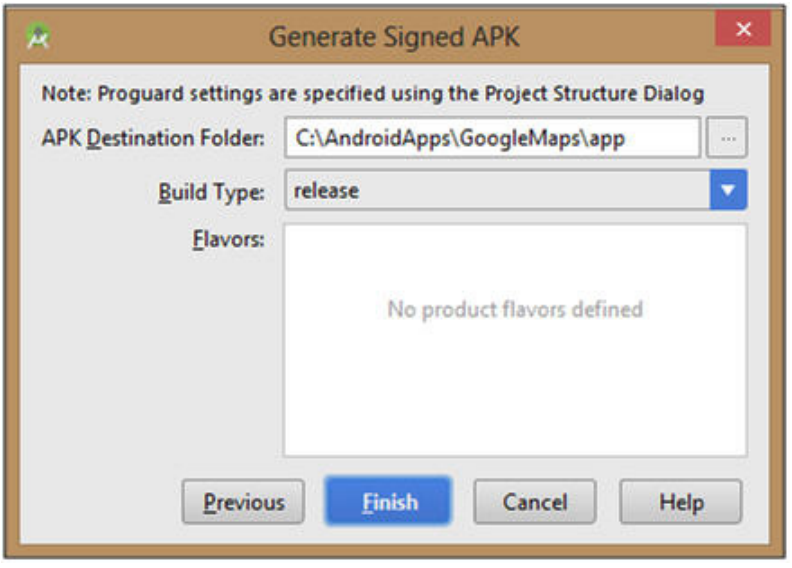


Figure 9.12: Destination Details

7. Click Finish. The application key is now setup.

Retrieve a Google Maps API Key

Before using maps in the Android application, you need to obtain a Maps API Key.

Signing up for such an API key is fairly straightforward, and requires the developer to register the SHA1 fingerprint of the certificate that will be used to sign the application. The Maps registration service then provides a Maps API Key that is associated with the application's signer certificate.

The developer needs to execute a command line tool called keytool to display the certificate fingerprint. The keytool command is distributed with any Java JDK/JRE and can be located under %JAVA_HOME%/bin directory.

Steps to follow to extract SHA1 fingerprint are as follows:

1. The keytool.exe is present in the C:\Program Files\Java\<JDK_version_number>\ bin folder.
2. Type the following command to extract the SHA1 fingerprint:
   `keytool -list -v -keystore C:\AndroidApps\keysotre.jks`
   When prompted, enter the password.

Figure 9.13 displays the command in the command prompt and displays the generated SHA1 key.

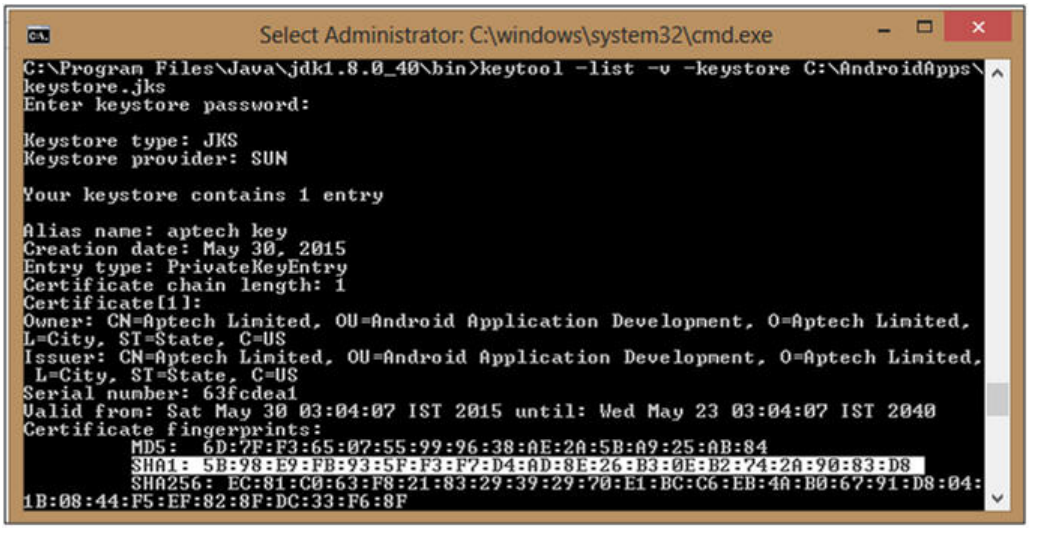


Figure 9.13: Google SHA1 Keys

3. Enter https://console.developers.google.com/ in the address bar of a browser. This opens the Google console to register your application for the Maps API.
4. Sign-in with the Gmail ID of the developer. If you are using the Google APIs Console for the first time, then click Create Project to create a new project named API Project as shown in figure 9.14.

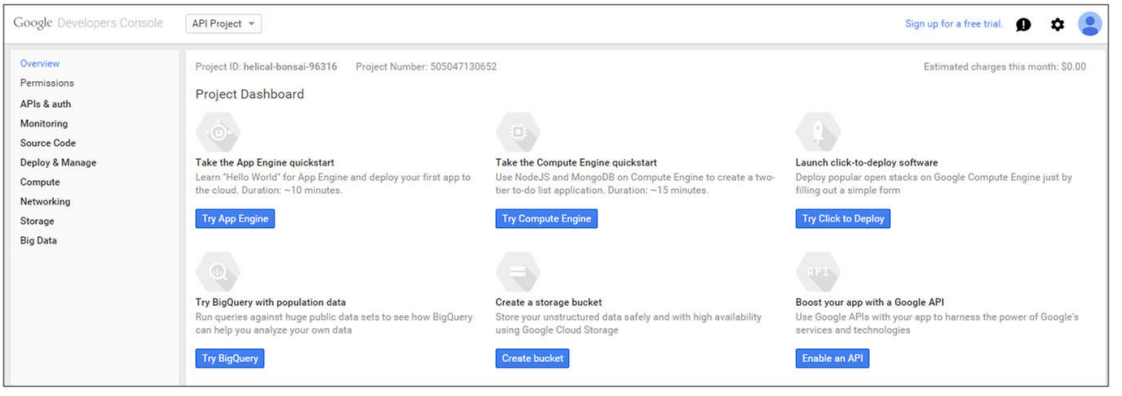


Figure 9.14: Google Console

5. Click APIs & auth and select APIs from the left pane as shown in figure 9.15.

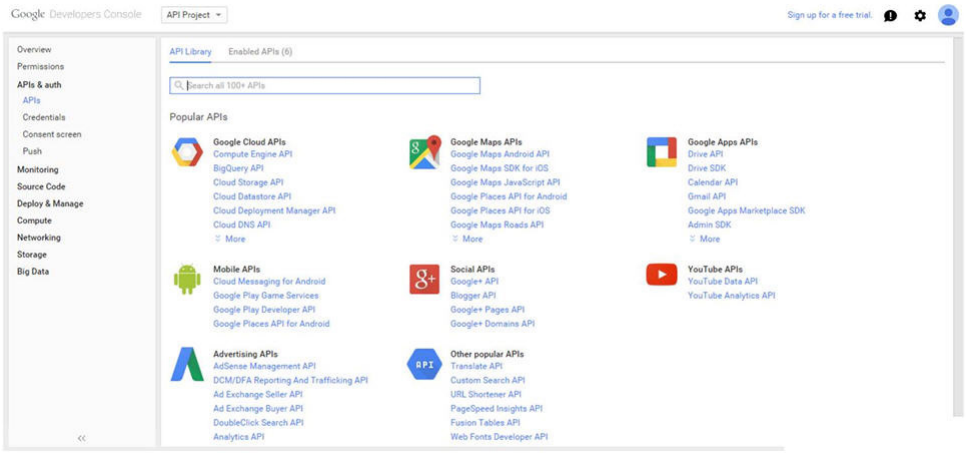


Figure 9.15: Developer Console APIs Page

6. Select Google Maps Android API under Google Maps APIs and click Enable API. The output will be as shown in figure 9.16.

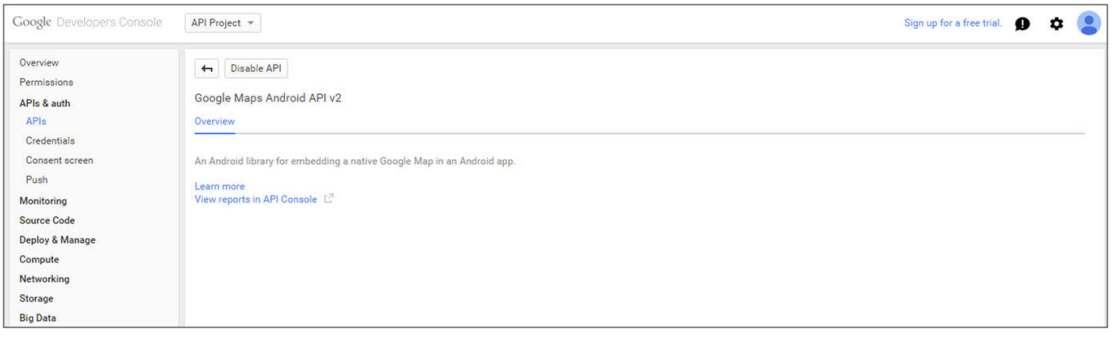


Figure 9.16: Activate Google Maps Android API

7. From the left pane, click Credentials as shown in figure 9.17.

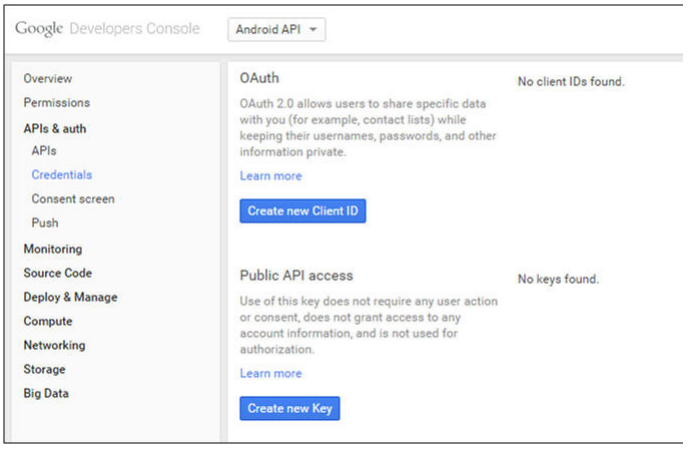


Figure 9.17: Credentials Page

8. Click Create new Key.
9. Click Android key to obtain an API key from the popup as shown in figure 9.18.

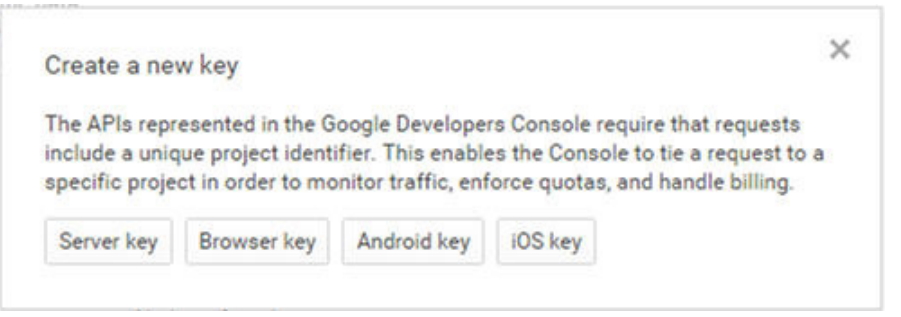


Figure 9.18: New Key Selection

10. Enter the application key followed by a semicolon and the package name as shown in figure 9.19.

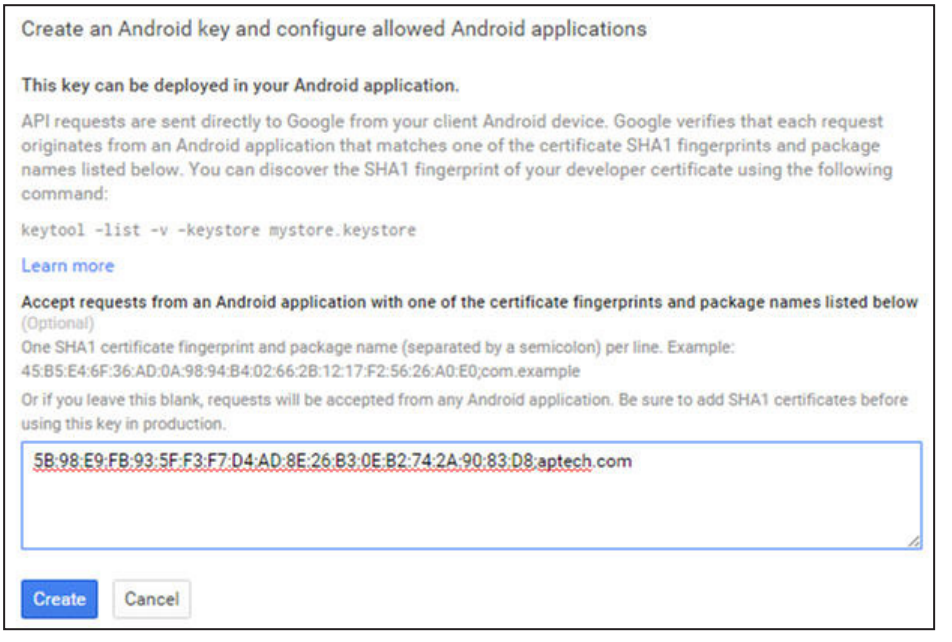


Figure 9.19: Create Key

11. Click Create to generate an API key. The API key is created and displayed on the Credentials page as shown in figure 9.20.

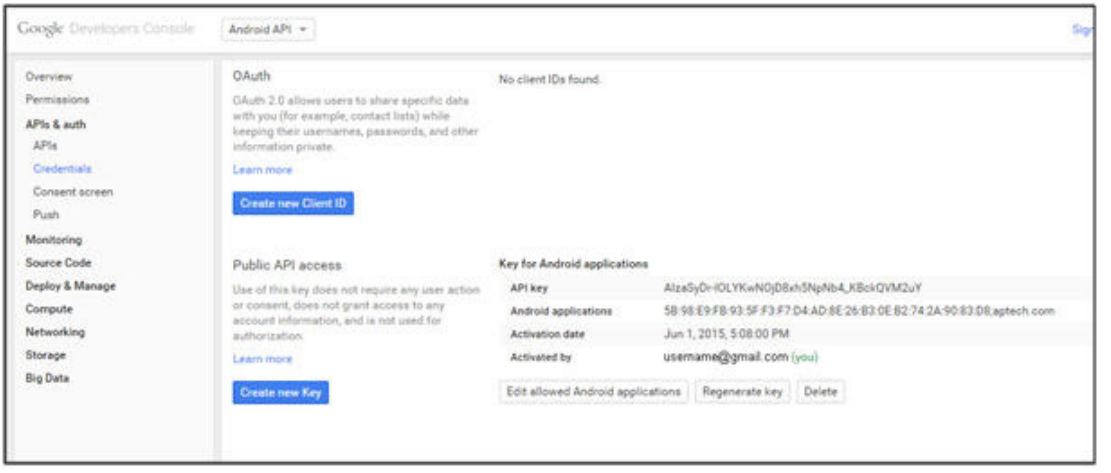


Figure 9.20: Created API Key

Coding the Application

Once the API key has been generated, the API key needs to be added to the application.

1. Modify the AndroidManifest.xml file with the settings to add the key and set the permissions for your application to access Android features and Google Maps servers as shown in Code Snippet 4.

Code Snippet 4:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
xmlns:android="http://schemas.android.com/apk/res/ android"
package="com.example.googleproject"
android:versionCode="1"
android:versionName="1.0" >
<permission
            android:name="com.trafficapp.permission.MAPS_ RECEIVE"
            android:protectionLevel="signature"></permission>
<uses-permission
           android:name="com.trafficapp.permission.MAPS_ RECEIVE"/>
 <uses-permission android:name="android.permission. INTERNET"/>
<uses-permission android:name="com.google.android.
```

```
      providers.gsf.permission.READ_GSERVICES”/>
      <uses-permission android:name=“android.permission. ACCESS_
       NETWORK_STATE”/>
      <uses-permission android:name=“android.permission. ACCESS_
       COARSE_LOCATION”/>
      <uses-permission android:name=“android.permission. ACCESS_
       FINE_LOCATION”/>
     <uses-feature
      android:name=“android.hardware.location”
      android:required=“false” />
     <uses-feature
      android:name=“android.hardware.location.network”
      android:required=“false” />
<uses-feature android:name=“android.hardware.location. gps” />
      <uses-feature
      android:name=“android.hardware.wifi”
      android:required=“false” />
     <uses-feature
     android:glEsVersion=“0x00020000”
      android:required=“true”/>
     <application
      android:allowBackup=“true”
      android:icon=“@drawable/ic_launcher”
      android:label=“@string/app_name”
      android:theme=“@style/AppTheme” >
<activity android:name=“com.example.googleproject.
MainActivity”
      android:label=“@string/app_name” >
      <intent-filter>
<action android:name=“android.intent. action.MAIN” />
```

```
 <category android:name="android.intent.category.LAUNCHER" />
      </intent-filter>
 </activity>
<meta-data
      android:name="com.google.android.maps.v2.API_ KEY"
      android:value=" AIzaSyDr-IOLYKwNOjD8xh5NpNb4_KBckQVM2uY"/>
</application>
</manifest>
```

As can be seen in the manifest file, the android.permission.INTERNET permission has been granted and the use of the library, com.google.android.maps has been specified.

Important: The API key generated is linked to the Android SDK keystore that one is currently using. This means that if you execute your application on another development environment, a different Android SDK keystore is installed and the same API key will not work.

2. If the project is made using the Google Maps activity, a string resource is used instead. In this case, navigate to res → values and open google_maps_api.xml for editing and modify the code as shown in Code Snippet 5.

Code Snippet 5:

```
<resources>
    <!--
    TODO: Before you run your application, you need a Google
Maps API key.


    To get one, follow this link, follow the directions and
press "Create" at the end:

https://console.developers.google.com/flows/enableapi?
apiid=maps_android_backend&keyType=CLIENT_SIDE _
ANDROID&r=26:46:5B:B4:04:E9:A7:AD:10:00:DF:0C: 43:5A:06:7C:A6:60
:F8:46%3Baptech.com.googlemaps

You can also add your credentials to an existing key, using this
line:

26:46:5B:B4:04:E9:A7:AD:10:00: DF:0C:43:5A:06:7C:A6:60:F8:46;apt
ech.com.googlemaps
```

```
    Once you have your key (it starts with "AIza"), replace the
"google_maps_key"
    string in this file.
    -->
    <string name="google_maps_key" translatable="false"
templateMergeStrategy="preserve">
        AIzaSyDr-IOLYKwNOjD8xh5NpNb4_KBckQVM2uY
    </string>
</resources>
```

3. Modify the code in activity_main.xml file as shown in Code Snippet 6.

Code Snippet 6:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
xmlns:android="http://schemas.android.com/ apk/res/android"
      android:layout_width="fill_parent"
      android:layout_height="fill_parent"
      android:orientation="vertical" >
<LinearLayout
      android:layout_width="fill_parent"
      android:layout_height="fill_parent" >
<fragment
      android:id="@+id/map"
      android:layout_width="match_parent"
      android:layout_height="match_parent"
      class="com.google.android.gms.maps.SupportMapFragment" />
</LinearLayout>
</RelativeLayout>
```

The fragment class is set to 'com.google.android.gms.maps.SupportMapFragment', indicating that this fragment is a Google Maps fragment used to display the map layout.

4. Also, modify the code in the MainActivity file as shown in Code Snippet 7.

Code Snippet 7:

```
package com.example.googleproject;
import com.google.android.gms.maps.GoogleMap;
import com.google.android.gms.maps.SupportMapFragment;
import android.os.Bundle;
import android.app.Activity;
import android.support.v4.app.FragmentActivity;
import android.view.Menu;
public class MainActivity extends FragmentActivity {
private GoogleMap map;
@Override
protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_main);
      /*map =((SupportMapFragment) getSupportFragmentManager()
                              .findFragmentById(R.id.map)).getMap();
       map.setMapType(GoogleMap.MAP_TYPE_NORMAL); */
}
@Override
public boolean onCreateOptionsMenu(Menu menu) {
      // Inflate the menu; this adds items to the action bar if
it is present.
      getMenuInflater().inflate(R.menu.main, menu);
      return true;
}
}
```

The commented code demonstrates how to set the map type.

5. Navigate to the Build menu and select Generate Signed APK.
6. Click Next.

7. Click Finish.
8. Connect the mobile device through USB cable.
9. Enable USB to connect to storage/device with developer mode enabled.
10. Install the adb drivers for the device.
11. Copy the generated signed apk from the Project Folder → app to the device.
12. Open the APK on the device and select install.
13. Once the application is installed, run the application from the App Drawer.

The output of the application is shown in figure 9.21.

Figure 9.21: Google Map on Real Time Device

Note - This application will run only on a real device.

Key Developer Features

- **Add maps to your app:** With version 2 of the Google Maps Android API, the developer can embed maps into an activity as a fragment with a simple XML snippet. The new Maps offer exciting fea¬tures such as 3D maps, indoor, satellite, terrain, and hybrid maps; and much more.
- **Customize the map:** Markers can be added to the map to indicate special locations of interest for the us¬ers. Besides these, the developer can also define custom colors or icons for the map markers to match the application's look and feel. The developer can draw polylines and polygons to indicate the path or region, or provide complete image overlays and thus, enhance the maps further.
- **Control the user's view:** Users can be provided a different view of the world with the ability to control the rotation, tilt, zoom, and pan properties of the 'camera' perspective of the map.

9.3 Google Play Services

This section explains Google Play Services, the core component, making Google API possible in Android along with an example. Play Services is a client library available for use by other applications to access the Google API. Play Services is installed as an APK on devices by default. It is automatically updated on a regular basis. On older devices, where play services is not pre-installed, the Play Store will make an automatic install. The developer can assume that almost all devices have Play Services installed and freely use the services provided by the library.

Play Services need to be installed alongside Google APIs using the procedure explained earlier.

9.3.1 Relevant Classes

This section describes all the relevant classes for utilizing Play Services.

- **GoogleApiClient.Builder**

  The builder class is used to retrieve a reference to a GoogleApiClient object. The API that will be utilized by the application can be specified using the addAPI() method. If OAuth is being used, the scope must be mentioned using the addScope() method. The addConnectionCallbacks() method is used to set the callback listener. The addOnConnectionFailedListener() method is used to set a call back listener if the connection to the service fails. The build() method is used to retrieve an instance of GoogleApiClient.

- **GoogleApiClient**

  This class is the main entry point for Google Play Services integration. The blockingConnect() and connect() methods are used to make a blocking and a non-blocking connect respectively. A blocking connect halts further execution until a connection is established. A non-blocking connect returns immediately. The disconnect() and reconnect() methods are used to disconnect and reconnect respectively. The getSessionId() method is used to get the session id for the connection. The registerConnectionCallbacks() is used to register callback listeners for non-blocking connect and unregisterConnectionCallbacks() method is used to unregister the listener.

- **GooglePlayServicesUtil**

  A utility class which can be used to verify that Google Play Services are available on the devices and to check the version to ensure that they are up to date. The static method isGooglePlayServicesAvailable() is used to check if Google Play Services is installed.

9.3.2 Integrating Google+ Login using Play Services

This section explain the process of using Google Play Services to integrate Google+ authentication.

- **Adding Dependencies for the Application**

  Google APIs and Google Play services need to be installed using the SDK manager before creating an application using them. To develop an application, perform the following steps:

1. Create a project named PlayServices in Android Studio.
2. During the activity selection screen, select Google Play Services Activity as shown in figure 9.22.

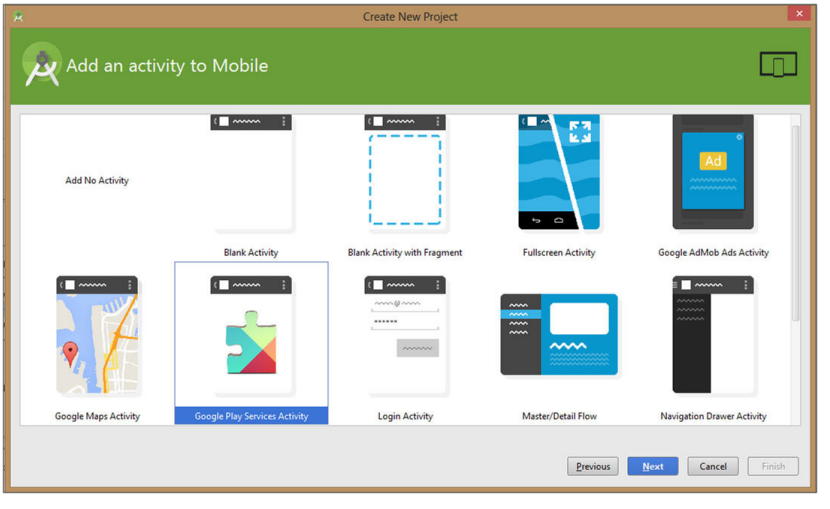


Figure 9.22: Selecting Google Play Services Activity

3. Click Next.
4. Select the Enable Google + APIs check box as shown in figure 9.23.

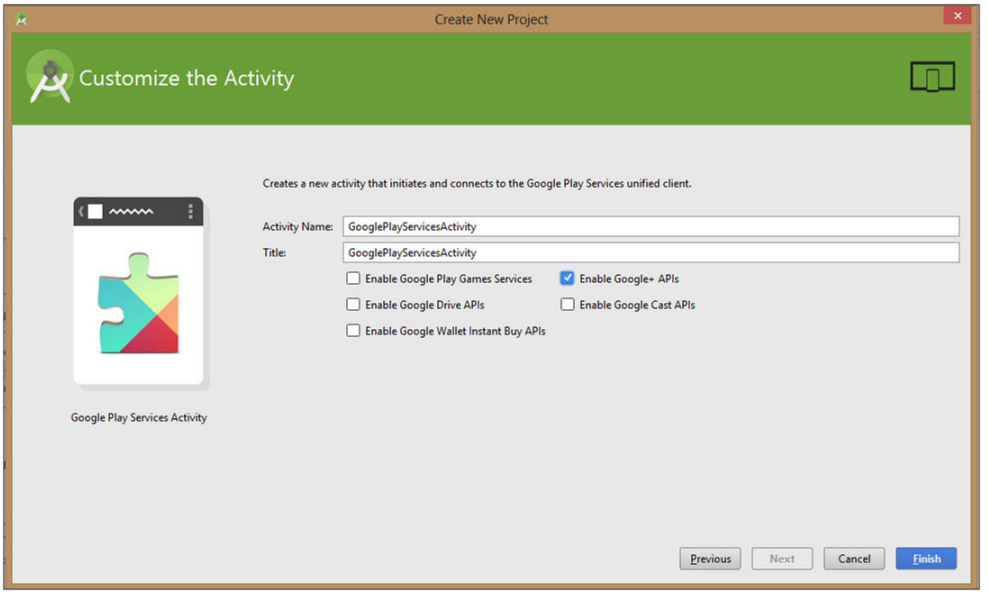


Figure 9.23: API Selection Screen

If the steps 3 and 4 are done, then steps 5 to 9 can be skipped.

5. If the project is created using a Blank Activity, right-click **app** and **select Open Module Settings.**

> **Note** - If using a Blank Activity, remember to change the name to GooglePlayServicesActivity for consistency with the rest of this section.

6. From the Project Structure dialog box, select the Dependencies tab as shown in figure 9.24.

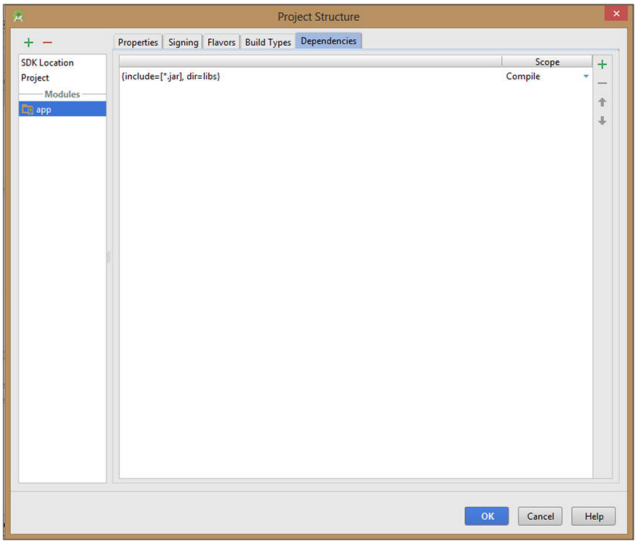


Figure 9.24: App Module Settings

7. Click the '+' sign and select Library Dependency to open Choose Library Dependency dialog box as shown in figure 9.25.

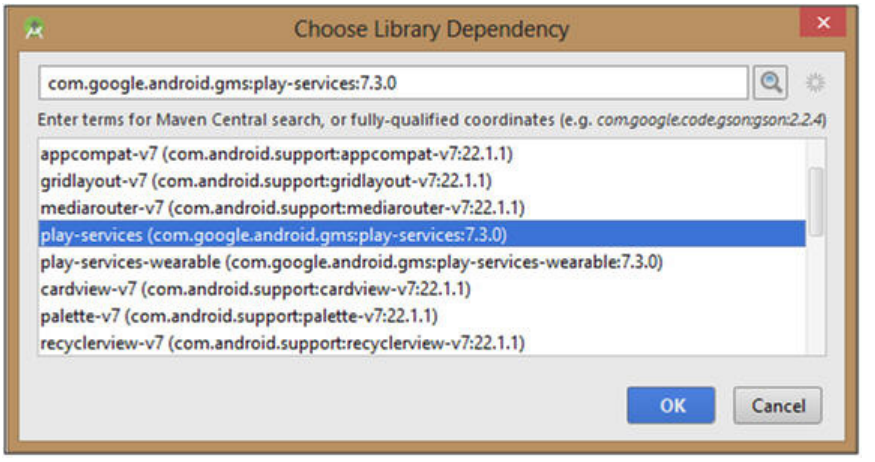


Figure 9.25: Library Selection Dialog Box

8. Select the play-services library and click OK.
9. Click OK again to confirm the dependencies.
10. Navigate to app → java → aptech.com.playservices and add the imports as shown in Code Snippet 8.

Code Snippet 8:

```
import com.google.android.gms.common.ConnectionResult;
import com.google.android.gms.common.GooglePlayServicesUtil;
import com.google.android.gms.common.api.GoogleApiClient;
```

☐ Generating a key for the Application

1. Navigate to Build Menu and select **Generate Signed APK**.
2. The Generate Signed Application Dialog box is displayed as shown in figure 9.26.

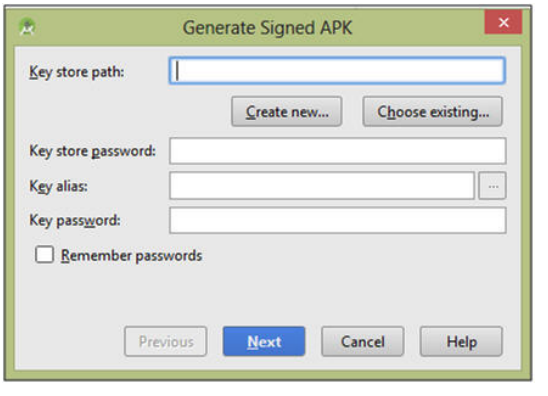


Figure 9.26: Generate Signed Application Dialog Box

3. If a keystore and a key were already created in section 9.2.3, then steps 3 and 4 can be skipped.

4. If it is the first time, click Create New under Key store path to open the New Key Store dialog box as shown in figure 9.27.

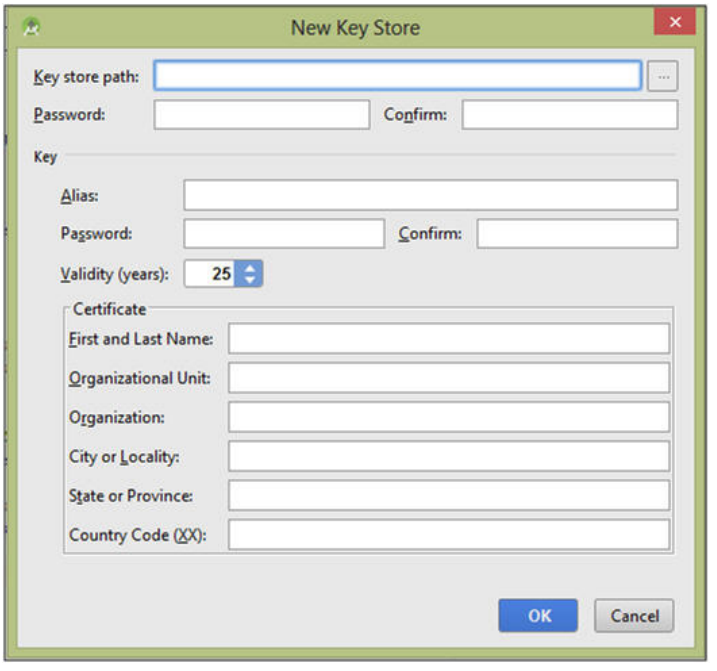


Figure 9.27: New Key Store Dialog Box

5. Enter the details as shown in figure 9.28.

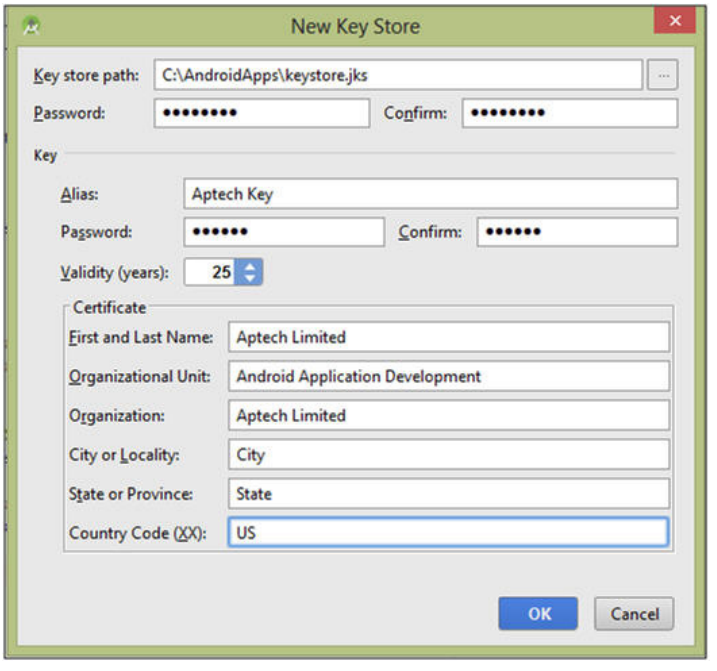


Figure 9.28: Keystore Details

6. Click OK to return to the Generate Signed APK dialog box with the details already entered as shown in figure 9.29.

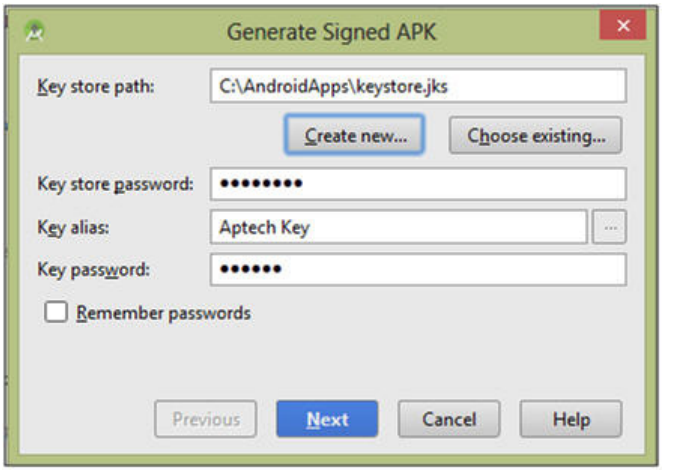


Figure 9.29: Generate Signed APK with the Details

7. Click Next to display the Destination dialog box as shown in figure 9.30.

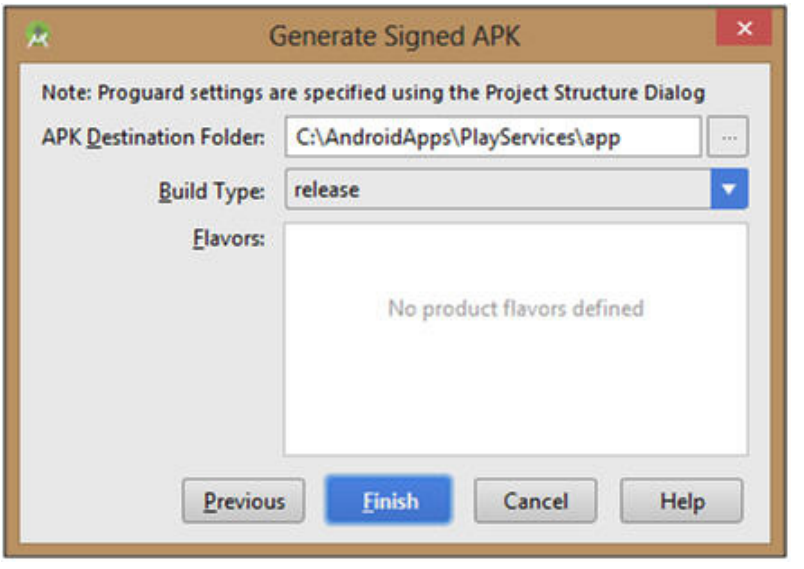


Figure 9.30: Generate Dialog Box Destination Details

8. Click Finish. An application key has been generated.

Retrieve a Google+ API Key

Identical to the Maps API Key, a Google + API key needs to be obtained before the API can be used. If an API key was already created in the section 9.2.3, then the steps in this section can be skipped.

Signing up for such an API key is fairly straightforward, and requires the developer to perform the following:

- Registering the SHA1 fingerprint of the certificate that you will use to sign your application.

The developer needs to execute a command line tool called keytool to display the certificate fingerprint. The keytool command is distributed with any Java JDK/JRE and can be located under %JAVA_HOME%/bin directory.

Steps to follow to extract SHA1 Fingerprint are as follows:

1. The keytool.exe is present in the C:\Program Files\Java\<JDK_version_number>\ bin folder.
2. Type the following command to extract the SHA1 fingerprint:
   `keytool -list -v -keystore C:\AndroidApps\keysotre.jks`
   When prompted, enter the Password.
3. Enter https://console.developers.google.com/ in the Address bar of a browser. This opens the Google console to register your application for the Maps API.
4. Sign-in with the Gmail id of the developer. If you are using the Google APIs Console for the first time, then click Create Project to create a new project named API Project.
5. Click API & auth and select APIs.
6. Select Google+ API and click Enable API.
7. From the Left pane, click Credentials.
8. Click Create new key.
9. Click Create new Android Key to obtain an API key from the popup.
10. Enter the application key followed by a semicolon and the package name as shown in figure 9.31.

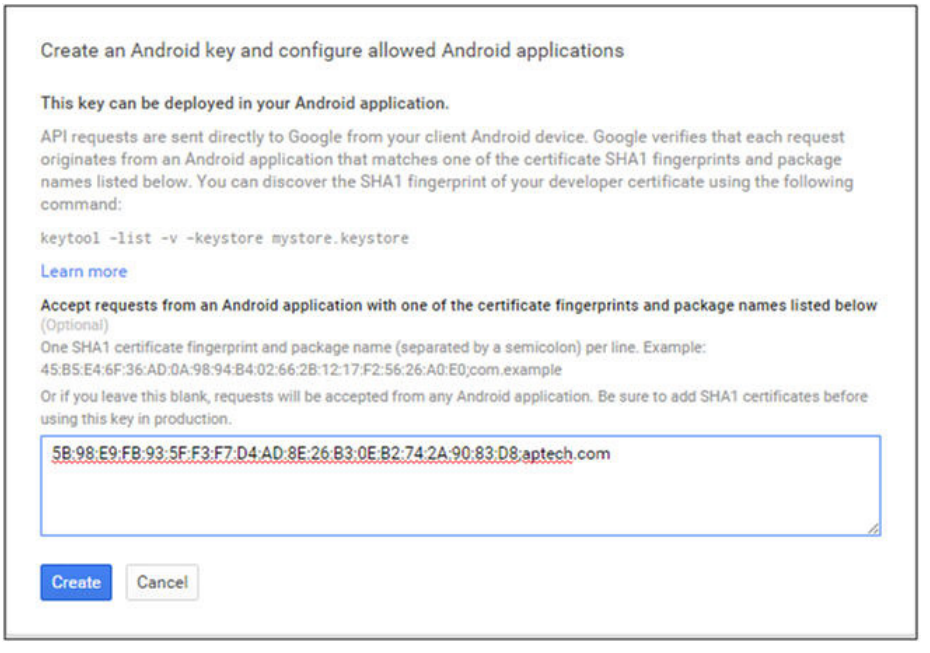


Figure 9.31: Create Key

11. Click Create to generate an API key. The API key is created and displayed on the Credentials page as shown in figure 9.32.

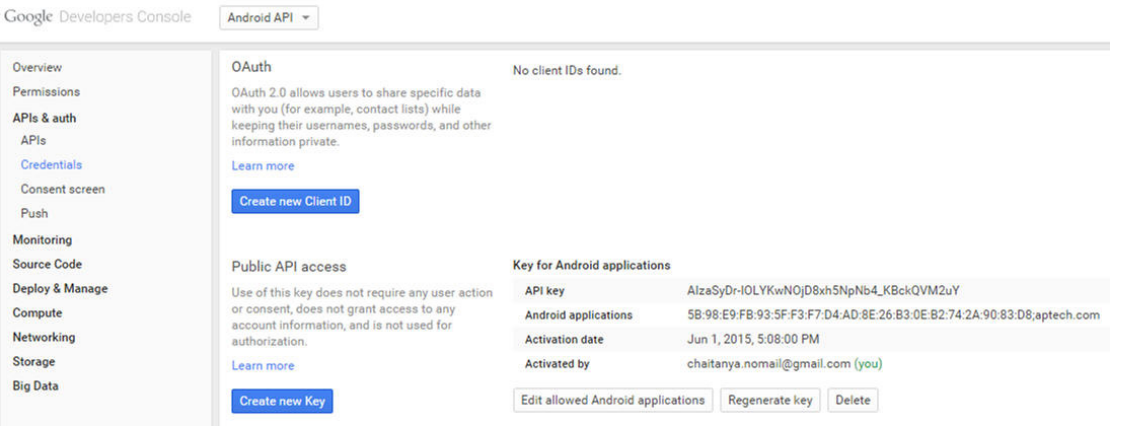


Figure 9.32: Created API Key

☐ Coding the Application

Once the API Key has been generated, the steps to create the application are as follows:

1. Navigate to app → java → aptech.com.playservices and open GooglePlayServicesActivity.java for editing.
2. Modify the code as shown in Code Snippet 9.

Code Snippet 9:

```
package aptech.com.playservices;

import android.app.Activity;

import android.content.DialogInterface;

import android.content.DialogInterface.OnCancelListener;

import android.content.Intent;

import android.content.IntentSender.SendIntentException;

import android.os.Bundle;

import android.util.Log;

import android.view.View;

import android.widget.Button;

import com.google.android.gms.common.ConnectionResult;

import com.google.android.gms.common.GooglePlayServicesUtil;
```

```
import com.google.android.gms.common.api.GoogleApiClient;

import com.google.android.gms.plus.Plus;

import com.google.android.gms.plus.model.people.Person;

public class GooglePlayServicesActivity extends Activity
implements

        GoogleApiClient.ConnectionCallbacks,

        GoogleApiClient.OnConnectionFailedListener {

    private static final String TAG = "GooglePlayServicesActiv";

    private static final String KEY_IN_RESOLUTION = "is_in_
                                                    resolution";

    /**

  * Request code for auto Google Play Services error resolution.

  */protected static final int REQUEST_CODE_RESOLUTION = 1;

    /**

     * Google API client.

     */

    private GoogleApiClient mGoogleApiClient;

    /**

     * Determines if the client is in a resolution state, and

     * waiting for resolution intent to return.

     */

   private boolean mIsInResolution;

   Button gPlusLogin;

   private boolean gplusClicked;

   private boolean gplusInProgress;

    /**

     * Called when the activity is starting. Restores the
activity state.
```

```
     */
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        if (savedInstanceState != null) {
            mIsInResolution = savedInstanceState.getBoolean(KEY_
                                        IN_RESOLUTION, false);
        }
        setContentView(R.layout.login_layout);
        gPlusLogin  = (Button)findViewById(R.id.gpluslogin);
        gplusClicked = false;
        gplusInProgress = false;
    }
    /**
    * Called when the Activity is made visible.
     * A connection to Play Services need to be initiated as
     * soon as the activity is visible. Registers {@code
       ConnectionCallbacks}
     * and {@code OnConnectionFailedListener} on the
     * activities itself.
     */
    @Override
    protected void onStart() {
        super.onStart();
        if (mGoogleApiClient == null) {
        mGoogleApiClient = new GoogleApiClient.Builder(this)
                .addApi(Plus.API).
        addScope(Plus.SCOPE_PLUS_LOGIN)
// Optionally, add additional APIs and scopes if required.
                    .addConnectionCallbacks(this)
```

```
                    .addOnConnectionFailedListener(this)
                    .build();
        }
//mGoogleApiClient.connect();
        if(getIntent().getStringExtra("logout")=="GPlus")
        {
            logoutGPlus();
        }
    }
    /**
     * Called when activity gets invisible. Connection to Play
       Services needs to
     * be disconnected as soon as an activity is invisible.
     */
    @Override
    protected void onStop() {
        if (mGoogleApiClient != null) {
        mGoogleApiClient.disconnect();
       }
        super.onStop();
    }
    /**
     * Saves the resolution state.
     */
    @Override
    protected void onSaveInstanceState(Bundle outState) {
       super.onSaveInstanceState(outState);
       outState.putBoolean(KEY_IN_RESOLUTION, mIsInResolution);
```

```
    }
    /**
     * Handles Google Play Services resolution callbacks.
     */
    @Override
    protected void onActivityResult(int requestCode, int
                                        resultCode, Intent data)
{
        super.onActivityResult(requestCode, resultCode, data);
        switch (requestCode) {
        case REQUEST_CODE_RESOLUTION:
              if (resultCode != RESULT_OK) {
                                              gplusClicked = false;
                }
               gplusInProgress = false;
               retryConnecting();
               break;
        }
   }
    private void retryConnecting() {
        mIsInResolution = false;
        if (!mGoogleApiClient.isConnecting()) {
            mGoogleApiClient.connect();
        }
   }
    /**
     * Called when {@code mGoogleApiClient} is connected.
    */
    @Override
```

```
public void onConnected(Bundle connectionHint) {
    Log.i(TAG, "GoogleApiClient connected");
    loadGPlusInfo();
}
private void loadGPlusInfo() {
    try {
        if (Plus.PeopleApi.getCurrentPerson
                                (mGoogleApiClient) != null)
          {
        Person user = Plus.PeopleApi.getCurrentPerson
                                        (mGoogleApiClient);
        String userName = Plus.AccountApi. getAccountName
                                        (mGoogleApiClient);
        String userRealName = user.getDisplayName();
        String userPhotoUrl = user.getImage().getUrl();
        String userCoverUrl = user.getCover().
                                  getCoverPhoto().getUrl();
        Intent i = new Intent(GooglePlayServicesActivity.
                                 this,GPlusActivity.class);
            i.putExtra("userName",userName);
            i.putExtra("userRealName",userRealName);
            i.putExtra("userPhotoUrl",userPhotoUrl);
            i.putExtra("userCoverUrl",userCoverUrl);
            startActivity(i);    }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
/**
* Called when {@code mGoogleApiClient} connection is
  suspended.
```

```
    */
    @Override
    public void onConnectionSuspended(int cause) {
    Log.i(TAG, "GoogleApiClient connection suspended");
        retryConnecting();
    }
    /**
     * Called when {@code mGoogleApiClient} is trying to connect
but failed.
     * Handle {@code result.getResolution()} if there is a
resolution
     * available.
     */
    @Override
    public void onConnectionFailed(ConnectionResult result) {
      Log.i(TAG, "GoogleApiClient connection failed: " + result.
                                                    toString());
        if (!result.hasResolution()) {
            // Show a localized error dialog.
           GooglePlayServicesUtil.getErrorDialog(result.
                 getErrorCode(), this, 0, new OnCancelListener()
               {
    @Override
     public void onCancel(DialogInterface dialog) {
       retryConnecting();
    }
       }).show();
        return;
        }
// If there is an existing resolution error being displayed or a
//resolution activity has started before, do nothing and wait
```

```
//for resolution progress to be completed.
        if (mIsInResolution) {
            return;
        }
        if (!gplusInProgress) {
            mIsInResolution = true;
         if (gplusClicked && result.hasResolution()) {
                try {
                   result.startResolutionForResult(this,
REQUEST_CODE_RESOLUTION);
                    gplusInProgress = true;
                } catch (SendIntentException e) {
                    gplusInProgress = false;
                    mGoogleApiClient.connect();
                }
            }
        }
    }
   public void onGPlusClicked(View view) {
        if(!mGoogleApiClient.isConnecting())
        {
            gplusClicked=true;
            mGoogleApiClient.connect();
        }
    }
   private void logoutGPlus()
    {
        if (mGoogleApiClient.isConnected()) {
          Plus.AccountApi.clearDefaultAccount(mGoogleApiClient);
```

```
            mGoogleApiClient.disconnect();
            mGoogleApiClient.connect();
        }
    }
}
```

Once the user clicks the Sign In button, the user is authenticated using the Google Account. The details are then retrieved using the loadGPlusInfo() method. The details are then passed on as extras to the GPlus Activity.

3. Create a new directory under res named layout, if it does not already exist.
4. Right-click layout and select New Layout Recourse and create a new layout called login_layout.xml.
5. Modify the code as shown in Code Snippet 10.

Code Snippet 10:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
            android:orientation="vertical"
            android:layout_width="match_parent"
            android:layout_height="match_parent">
    <LinearLayout
            android:orientation="horizontal"
            android:layout_width="match_parent"
            android:layout_height="wrap_content">
    <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Username"
            android:id="@+id/textView" />
    <EditText
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
```

```
            android:inputType="textEmailAddress"
            android:ems="10"
            android:id="@+id/editText"
            android:layout_weight="1" />
    </LinearLayout>
    <LinearLayout
            android:orientation="horizontal"
            android:layout_width="match_parent"
            android:layout_height="wrap_content" >
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Password"
            android:id="@+id/textView2" />
        <EditText
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:inputType="textPassword"
            android:ems="10"
            android:id="@+id/editText2"
            android:layout_weight="1" />
    </LinearLayout>
        <Button
            android:layout_width="fill_parent"
            android:layout_height="wrap_content"
            android:text="Login "
            android:id="@+id/button"
            android:layout_gravity="center_horizontal" />
        <com.google.android.gms.common.SignInButton
```

```
        android:id="@+id/gpluslogin"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:onClick="onGPlusClicked"
        android:layout_marginBottom="20dp"/>
    </LinearLayout>
```

A login button is provided to allow the user to authenticate using a Google Account.

6. Create a new Blank Activity called GPlusActivity.
7. Navigate to res → layout and open activity_gplus.xml for editing.
8. Modify the code as shown in Code Snippet 11.

Code Snippet 11:

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:paddingLeft="0dp"
        android:paddingRight="0dp"
        android:paddingTop="0dp"
        android:paddingBottom="0dp"
        tools:context="aptech.com.playservices.GPlusActivity"
        android:orientation="vertical"
        android:padding="0dp">
    <LinearLayout
        android:orientation="horizontal"
        android:layout_width="match_parent"
        android:layout_height="50dp"
        android:background="@android:color/holo_green_light"
        android:padding="10dp">
 <TextView
        android:layout_width="match_parent"
```

```
            android:layout_height=”match_parent”
            android:text=”user@gmail.com”
            android:id=”@+id/userNameView”
            android:gravity=”center_vertical|right”
            android:textColor=”@android:color/white” />
    </LinearLayout>
    <LinearLayout
            android:orientation=”horizontal”
            android:layout_width=”match_parent”
            android:layout_height=”150dp”>
      <ImageView
            android:layout_width=”wrap_content”
            android:layout_height=”fill_parent”
            android:id=”@+id/userCoverView”
            android:layout_gravity=”center_horizontal”
            android:layout_weight=”1”/>
    </LinearLayout>
    <LinearLayout
            android:orientation=”horizontal”
            android:layout_width=”match_parent”
            android:layout_height=”wrap_content”
            android:layout_gravity=”center_horizontal”
            android:padding=”10dp”>
      <ImageView
            android:layout_width=”100dp”
            android:layout_height=”100dp”
            android:id=”@+id/userPhotoView”
            android:padding=”5dp”
            android:layout_gravity=”top” />
```

```
    <LinearLayout
            android:orientation=”vertical”
            android:layout_width=”wrap_content”
            android:layout_height=”match_parent”
            android:padding=”5dp”
            android:gravity=”center_vertical|right”
            android:layout_weight=”3”>
       <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:textAppearance=”?android:attr/
                                        textAppearanceLarge”
            android:text=”Welcome “
            android:id=”@+id/textView3” />
       <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:textAppearance=”?android:attr/
                                        textAppearanceLarge”
            android:text=”Aptech Limited”
            android:id=”@+id/userRealNameView”
            android:textColor=”@android:color/holo_green_
                                                    light”/>
        </LinearLayout>
    </LinearLayout>
</LinearLayout>
```

9. Navigate to app → java → aptech.com.playservices and open GplusActivity.java for editing.

10. Modify the code as shown in Code Snippet 12.

Code Snippet 12:

```
package aptech.com.playservices;
import android.app.Activity;
import android.content.Intent;
import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.widget.ImageView;
import android.widget.TextView;
import java.io.IOException;
import java.net.URL;
public class GPlusActivity extends Activity {
    String userName;
    String userRealName;
    String userPhotoUrl;
    String userCoverUrl;
    TextView userNameView;
    TextView userRealNameView;
    ImageView userCoverView;
    ImageView userPhotoView;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_gplus);
        Intent i = getIntent();
```

```
        userName = i.getStringExtra("userName");
        userRealName = i.getStringExtra("userRealName");
        userPhotoUrl = i.getStringExtra("userPhotoUrl");
        userCoverUrl = i.getStringExtra("userCoverUrl");
        userNameView = (TextView)findViewById(R.id.userNameView);
        userRealNameView = (TextView)findViewById(R.id.
                                                userRealNameView);
        userCoverView = (ImageView)findViewById(R.userCoverView);
        userPhotoView = (ImageView)findViewById(R.id.
                                                id.userPhotoView);
        userNameView.setText(userName);
        userRealNameView.setText(userRealName);
        try {
        URL url = new URL(userCoverUrl);
        Bitmap bmp= BitmapFactory.decodeStream(url.
                            openConnection().getInputStream());
        userCoverView.setImageBitmap(bmp);
        url = new URL(userPhotoUrl);
        bmp= BitmapFactory.decodeStream(url.openConnection().
                                                getInputStream());
        userPhotoView.setImageBitmap(bmp);
        } catch (IOException e) {
          e.printStackTrace();
        }
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
// Inflate the menu; this adds items to the action bar
//if it is present.
```

```
        getMenuInflater().inflate(R.menu.menu_gplus, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
// Handle action bar item clicks here. The action bar will
// automatically handle clicks on the Home/Up button, so long
   // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        if(id == R.id.action_logout) {
            Intent i = new Intent(GPlusActivity.this,
                            GooglePlayServicesActivity.class);
            i.putExtra("logout","GPlus");
            startActivity(i);
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
}
```

The Logged in user information is retrieved using the Intent Extras and displayed to the user. The user can return to the Home Screen by selecting the Logout button.

11. Navigate to res → menu and open menu_gplus.xml.

12. Modify the code as shown in Code Snippet 13.

Code Snippet 13:

```
<menu
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
tools:context="aptech.com.playservices.GPlusActivity">
    <item android:id="@+id/action_settings"
        android:title="@string/action_settings"
        android:orderInCategory="100"
        android:showAsAction="never" />
    <item android:id="@+id/action_logout"
        android:title="Logout"
        android:orderInCategory="100"
        android:showAsAction="never" />
</menu>
```

13. Navigate to manifests and open AndroidManifest.xml for editing.

14. Modify the code as shown in Code Snippet 14.

Code Snippet 14:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
xmlns:android="http://schemas.android.com/apk/res/android"
   package="aptech.com.playservices" >
  <uses-permission android:name="android.permission.INTERNET" />
  <uses-permission android:name="android.permission.GET_
                                                ACCOUNTS" />
  <uses-permission android:name="android.permission.USE_
                                              CREDENTIALS" />
   <application
        android:allowBackup="true"
```

```
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
    <activity
        android:name=".GooglePlayServicesActivity"
        android:label="@string/app_name" >
          <intent-filter>
          <action android:name="android.intent.action.MAIN" />
          <category android:name="android.intent.category.
                                                    LAUNCHER" />
          </intent-filter>
          </activity>
    <meta-data
        android:name="com.google.android.gms.version"
        android:value="@integer/google_play_services_version" />
    <activity
        android:name=".GPlusActivity"
        android:label="@string/title_activity_gplus"
        android:theme="@style/Google">
        </activity>
     </application>
</manifest>
```

15. Navigate to res → values and open styles.xml for editing.

16. Modify the code as shown in Code Snippet 15.

Code Snippet 15:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="AppTheme" parent="android:Theme.Material.
                                                    Light">
    </style>
    <style name="Google" parent="android:Theme.Material">
     <item
       name="android:colorPrimary">@android:color/holo_green_
                                                     light
     </item>
     <item name="android:colorPrimaryDark">@android:color/holo_
                                          green_dark </item>
        <item name="android:colorAccent">@android:color/holo_
                                          blue_light</item>
        <item name="android:textColorPrimary">@android:color/
                                              white</item>
        <item name="android:navigationBarColor">@android:color/
                                       holo_green_light</item>
    </style>
</resources>
```

17. Navigate to the Build menu and select Generate Signed APK.
18. Click Next.
19. Click Finish.
20. Connect the mobile device through USB cable.
21. Enable USB to connect to storage/device with developer mode enabled.
22. Install the adb drivers for the device.
23. Copy the generated signed apk from the Project Folder → app to the device.
24. Open the APK on the device and select install.
25. Once the application is installed, run the application from the App Drawer.

The output of the application is shown in figure 9.33.

Figure 9.33: Play Services Application

Click the 'g+' sign in the SIGN IN button and sign in to your Google+ account.

Once the user signs in, the Google+ Home page is displayed as shown in figure 9.34.

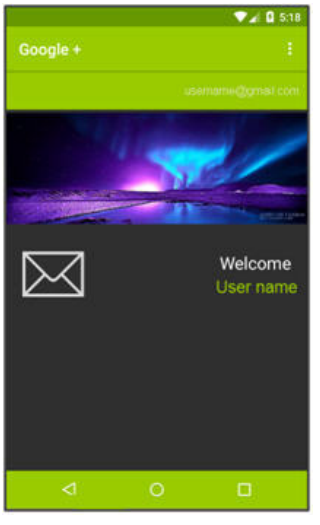


Figure 9.34: Google+ Post Login Homepage

The users can logout from the action menu and return to the login page as shown in figure 9.35.

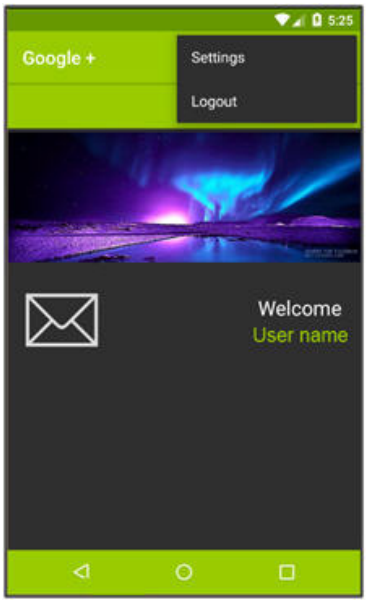


Figure 9.35: Logout Option in Action Menu

The application now uses Google API to integrate Google+ Authentication using Play Services.

Check Your Progress

1. Which of the following permissions are needed to use Play Services?

1.	INTERNET	2.	GET_ACCOUNTS
3.	WRITE_EXTERNAL_STORAGE	4.	NETWORK_STATE

(A)	1, 2	(B)	2, 3
(C)	3, 4	(D)	1, 4

2. Which of the following statements about Google API are true?

1.	There is a registration fee for Google API
2.	Google API can be accessed offline
3.	Working with Google Maps requires INTERNET permission
4.	A Google Account is required to use Google API

(A)	1, 3	(B)	1, 2
(C)	3, 4	(D)	2, 3

3. Which of the following information is displayed by the Location class?

(A)	Data Type and Size	(B)	Latitude and Longitude
(C)	Name and Key	(D)	Key and Value

4. The dependency that needs to be added for Play Services is _____.

(A)	play-services-wearable	(B)	play-services
(C)	google-services	(D)	google-api

5. Arrange the following steps to use Play Services in the correct sequence.

1.	Retrieve API Key
2.	Adding Dependencies
3.	Generate Application key
4.	Deploy Application

(A)	1, 2, 3, 4	(B)	2, 3, 1, 4
(C)	2, 1, 4, 3	(D)	1, 2, 4, 3

Answers

1.	A
2.	C
3.	B
4.	B
5.	B

Session

09

Google API

Summary

- Google provides API for developers to access their product services such as search, drive, App Engine and so on.
- Location-based Services (LBS) refer to a set of applications that exploit the knowledge of the geographical position of a mobile device in order to provide services based on that information.
- Google Maps allow the developers to develop applications of world with rich maps provided by Google.
- Google API Add on is an extension to Android SDK development environment that lets the developer develop applications for devices that include Google's set of custom applications, libraries, and services.
- The GoogleAPIClient and GooglePlayServicesUtil classes are used to access Google API.
- Google Play Services is a client side library for accessing Google services.

Onlinevarsity

10

Web Services in Android

Welcome to the Session, **Web Services in Android.**

This session describes Web Services in detail. The various standards for communication with Web Services along with the process for using third party API in applications are explained. Further, it explains Text-to-Speech (TTS) and Voice Recognition classes along with their subclasses and examples.

In this Session, you will learn to:

- Explain and use Web Services
- Explain standards for Web Services Communication
- Use third party API
- Create Web Services
- Understand the correct use of Web Service
- Explain and use Text to Speech and Voice Recognition

Session

10

Web Services in Android

10.1 Introduction

Web Services can enhance the application by providing computational offloading and information services. Text to Speech and Voice Recognition provide unique and efficient ways for interacting with the application. Coupled together, the developer can make an extremely user friendly and easy to use applications.

10.2 Web Services

Web Service is an application component which allows communication between devices over a network. There are several platforms which provide useful resources which can be utilized in developing Android Applications. This section describes the process of utilizing third party Web services, along with the various standards.

10.2.1 Types of Web Services

In order to communicate with a Web server providing a Web service, a common standard that is understandable by both the server and client needs to be established. There are several standards available for use with Web services. This section explains some of the more frequently used standards.

- **ReSTful**

  ReST is the abbreviation for Representational State Transfer. ReSTful is the most commonly used standard for providing Web services. Its simplicity, extendibility, adaptability, and robustness have allowed for it to rise as the prime access mode for Web services. There are no concrete standards for implementing a Web service using ReST. The developer can choose any format that fits the requirement. Requests are sent as standard HTTP 'GET' requests and the reply can be in any format that is mutually understandable among the communicating parties.

  Consider the following sample link that demonstrates a ReSTFul address consisting of the Web Service name followed by API Domain, subdomain, and so on.

  http://www.Webservice.com/APIDomain/APIsubdomain/getDetails.php?req=XXX&key=Ab123asf

  An example ReSTful service interaction is shown in Code Snippet 1.

Code Snippet 1:

```
<Details>
     <name>John Pattrick</name>
</Details>
```

The details were requested in the link and these are details returned by the ReSTful service.

- **SOAP**

  SOAP is an abbreviation for Simple Object Access Protocol. SOAP is an XML based Remote Procedure Call protocol. It relies on application layer protocols such as HTTP and SMTP. The client executes a remote method (procedure) on the Web server using an XML envelope to send the request. The Web service generates a response and sends it back to the client in an XML envelope. This is considered to be more secure than ReSTful and it can be used to create more complex interactions than what ReSTful can allow.

- **Custom/Proprietary**

  Besides the two open standards, several Web services provide custom formats for communication with the Web service. These Web services usually come with extensive documentation on using these services or with libraries for use with the platforms.

10.2.2 Using Third Party API

This section explains the process of using a third party API to access a Web service. For demonstration, create an application using Google Play Services to integrate Google+ authentication and extend it with Facebook Login using Facebook API.

- **Facebook**

  Facebook is currently the leading social platform. The number of users is increasing by millions. If the application is using authentication, then it will be a wise decision to integrate Facebook login into applications. This reduces the hassle for registration and provides a trustable authentication system for the application. It is also possible to use social features from Facebook such as friend's lists and post to wall. However, for the scope of this session, the Login feature will be demonstrated.

- **Generating Hash Key**

  1. Generate an Application key for signing the application.
  2. Download OpenSSL for Windows from the URL https://code.google.com/p/openssl-for-windows/downloads/detail?name=openssl-0.9.8k_WIN32.zip&can=2&q
  3. Extract the contents of the zip file to C:\openssl.
  4. Navigate to the keytool.
  5. Enter the following command shown in Code Snippet 2.

Code Snippet 2:

```
keytool -exportcert -alias "aptech key" -keystore "C:\
AndroidApps\keystore.jks" | "C:\openssl\bin\openssl" sha1
-binary | "C:\openssl\bin\openssl" base64
```

6. Enter the password.
7. The key is displayed as shown in figure 10.1.

Figure 10.1: Command Prompt Command for the Application Key

8. Store the key value.

Generating an API Key

Similar to the process for accessing Google API, Facebook also requires an API key to be generated for use in the applications. The steps to achieve these are as follows:

1. Open https://developers.facebook.com and enable developer account.
2. From the My Apps menu, select Add a New App.

   The app type selection screen is displayed as shown in figure 10.2.

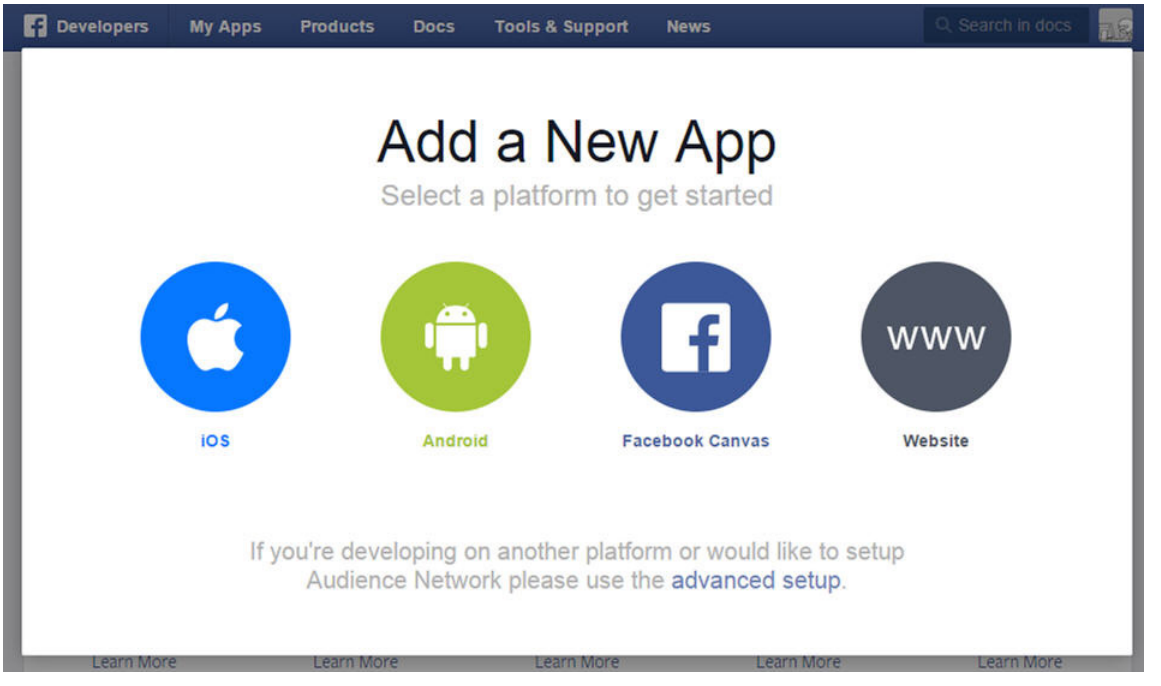


Figure 10.2: App Type Screen

3. Select Android and enter the app name as Aptech Auth and select Create New Facebook App ID as shown in figure 10.3.

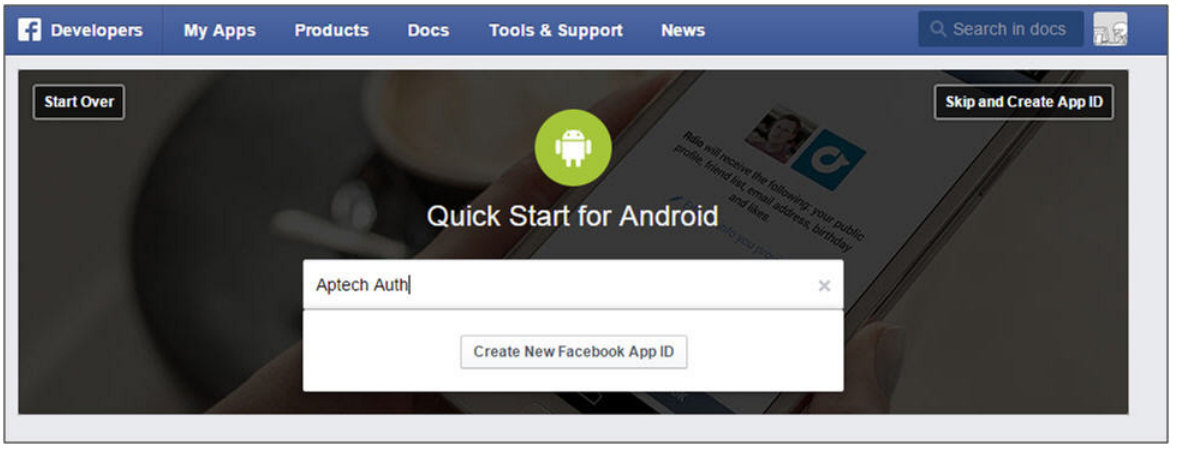


Figure 10.3: App Name Screen

4. Select a category and click Create a New App ID as shown in figure 10.4.

Figure 10.4: App Category Dialog Box

5. Download the SDK (If using the Eclipse IDE) and the APK file (If the Facebook App is not installed on your device) and navigate to the bottom of the page.

6. Enter the app details as shown in figure 10.5.

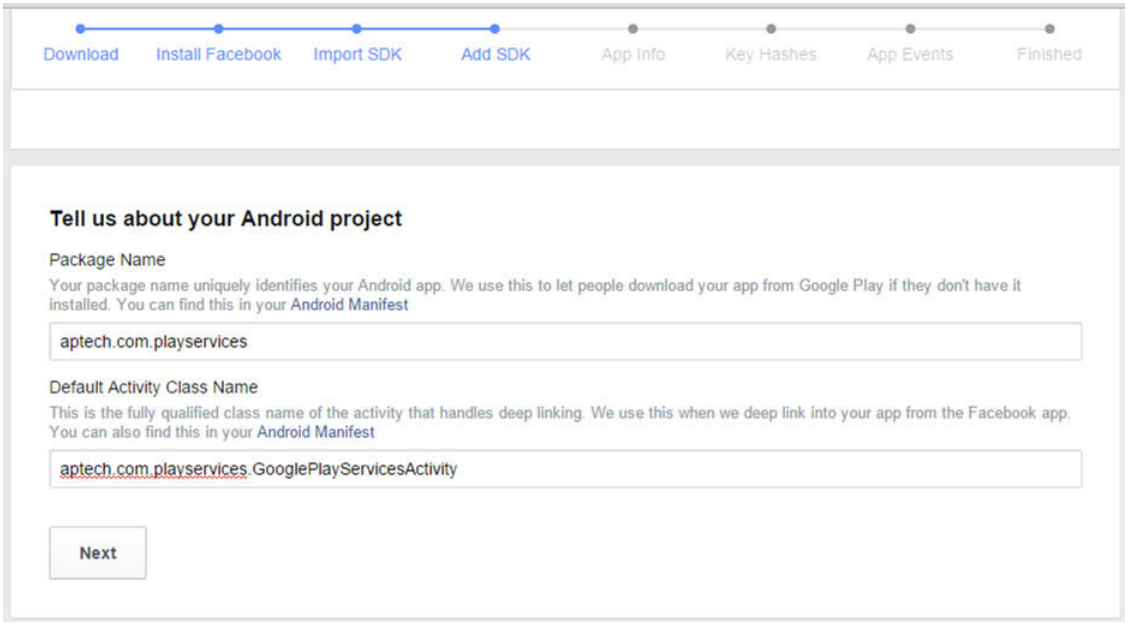


Figure 10.5: Android Project Details

7. Click Next. If a warning pops up, click Use this package name as shown in figure 10.6.

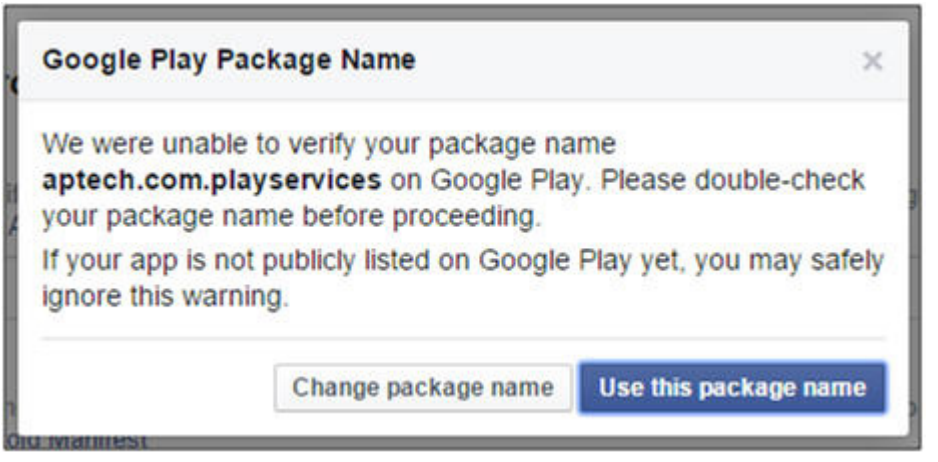


Figure 10.6: Package Name Confirmation Popup

8. Enter the hash key generated earlier as shown in figure 10.7.

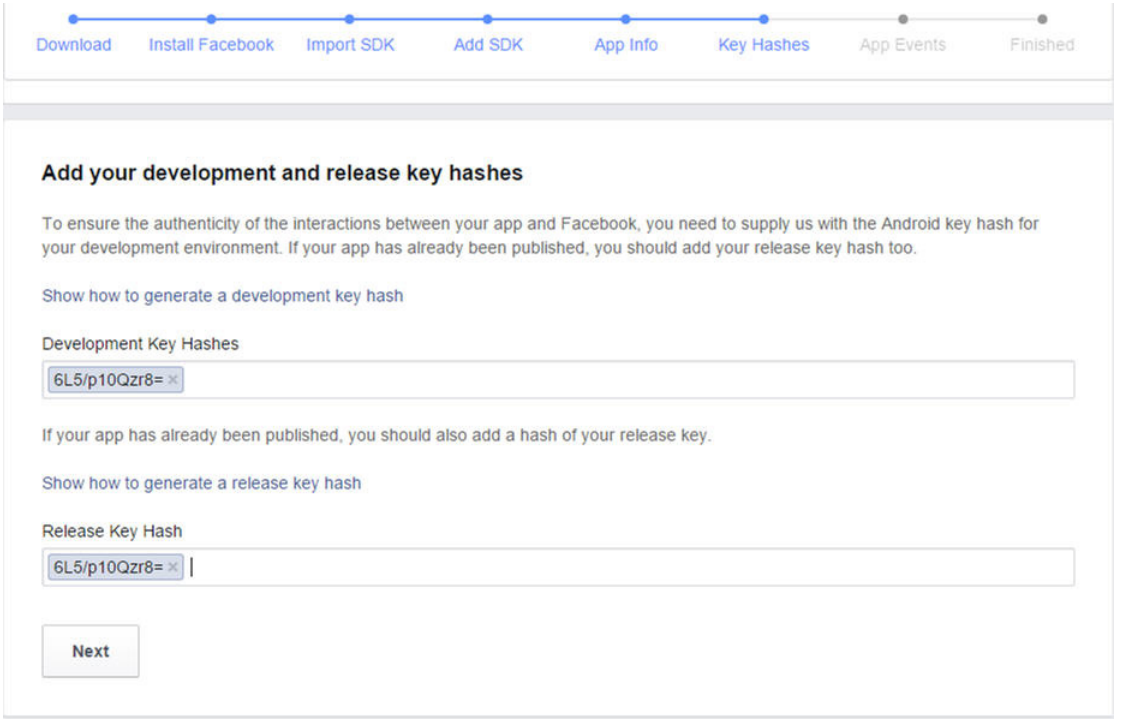


Figure 10.7: Application Hash Key

9. Click Next.

   The process for setting up the Facebook application is completed.

10. Scroll down to the bottom of the page to see the finished image.
11. Navigate to My Apps menu and select the created application Aptech Auth.
12. Copy the App id displayed as shown in figure 10.8.

Figure 10.8: App Dashboard

Setting up Android Studio

1. Open the project PlayServices.
2. Right-click app and select Open Module Settings.
3. Navigate to the Dependencies tab and click the '+' sign.
4. Select Library dependency as shown in figure 10.9.

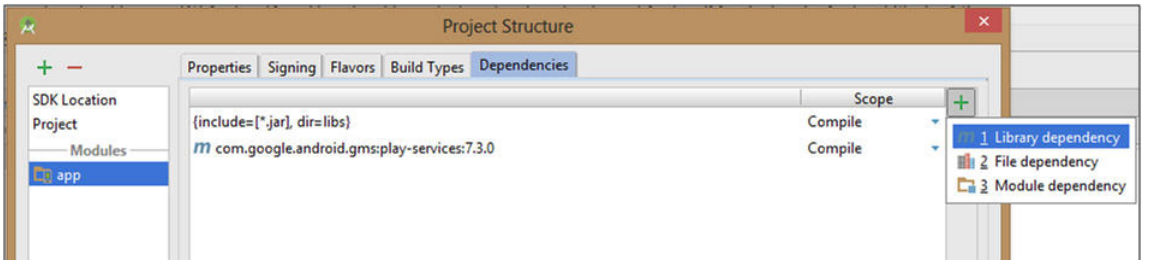


Figure 10.9: Adding a Library Dependency

5. In the popup box, enter the name com.facebook.android:facebook-android-sdk: and select the Facebook library as shown in figure 10.10.

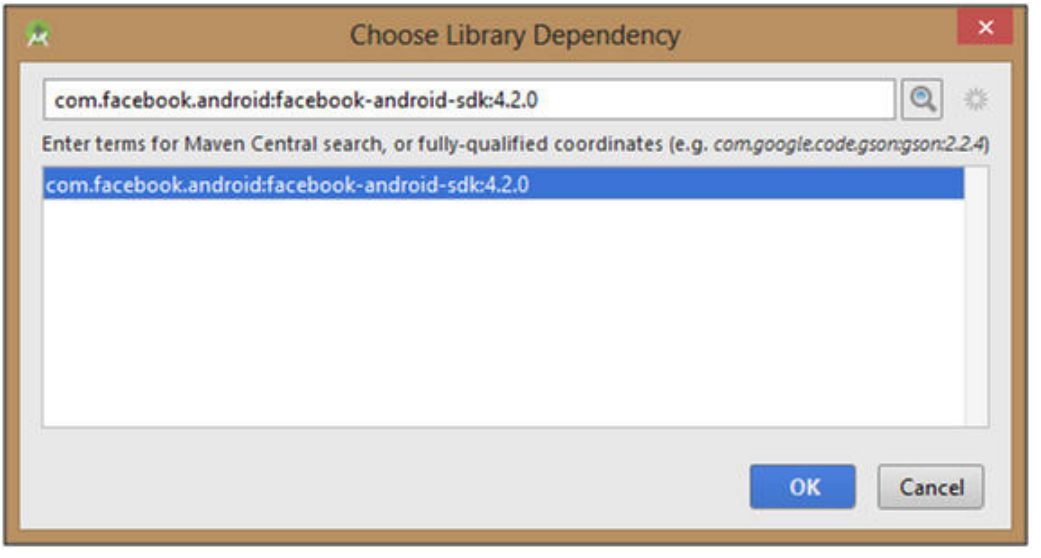


Figure 10.10: Adding the Facebook SDK

Note - Do not enter the version number during search as it is subject to change. It is auto filled.

6. Click OK.
7. Click OK again in the Dependencies tab.

   If it is the first time Facebook API is being used, Android Studio will automatically download and build the Facebook SDK for Android.

Setting up the Target Device

In order to use the Facebook API, the target device needs to have Facebook application installed. Most devices come with social applications pre-installed. If the application is not installed, then it can be installed by two means.

1. Navigate to the Play Store and search for the Facebook application and click Install. The app will be downloaded and installed on the device.
2. Download the Facebook application from the Developer Page as described in the previous section. Connect the target device using USB with adb. Execute the command as shown in Code Snippet 3.

Code Snippet 3:

```
adb install C:\Path_here\Facebook-{version_here}.apk
```

Coding the Application

With the dependencies added, and the Facebook SDK installed, now proceed with coding the application. The steps are as follows:

1. Open the PlayServices.
2. Create a new Blank Activity called FacebookActivity.
3. Navigate to res → layout.
4. Open login_layout.xml for editing and modify the code as shown in Code Snippet 4.

Code Snippet 4:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/
                                                        android"
    android:orientation="vertical" android:layout_width="match_
                                                         parent"
    android:layout_height="match_parent"
    android:weightSum="1"
    android:padding="10dp">
   <LinearLayout
        android:orientation="horizontal"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">
```

```
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Username”
            android:id=”@+id/textView” />
        <EditText
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:inputType=”textEmailAddress”
            android:ems=”10”
            android:id=”@+id/editText”
            android:layout_weight=”1”
          android:textColor=”@android:color/holo_red_dark” />
    </LinearLayout>
    <LinearLayout
        android:orientation=”horizontal”
        android:layout_width=”match_parent”
        android:layout_height=”wrap_content” >
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Password”
            android:id=”@+id/textView2” />
        <EditText
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:inputType=”textPassword”
            android:ems=”10”
```

```
            android:id="@+id/editText2"
            android:layout_weight="1"
         android:textColor="@android:color/holo_red_dark" />
    </LinearLayout>
    <Button
        android:layout_width="fill_parent"
        android:layout_height="50dp"
        android:text="Login "
        android:id="@+id/button"
        android:layout_gravity="center_horizontal" />
    <com.google.android.gms.common.SignInButton
        android:id="@+id/gpluslogin"
        android:layout_width="fill_parent"
        android:layout_height="50dp"
        android:onClick="onGPlusClicked"/>
    <com.facebook.login.widget.LoginButton
        android:id="@+id/fbLoginButton"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:layout_gravity="center_horizontal"
        android:layout_weight="0.11"
        android:minHeight="50dp"
        android:padding="10dp" />
</LinearLayout>
```

A Login with Facebook button is provided to allow the user to authenticate using Facebook.

5. Open activity_facebook.xml for editing and modify the code as shown in Code Snippet 5.

Code Snippet 5:

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/
android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="0dp"
    android:paddingRight="0dp"
    android:paddingTop="0dp"
    android:paddingBottom="0dp"
    tools:context=".FacebookActivity"
    android:orientation="vertical"
    android:padding="0dp">
    <LinearLayout
        android:orientation="horizontal"
        android:layout_width="match_parent"
        android:layout_height="50dp"
        android:background="@color/com_facebook_blue">
        <TextView
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:text="user@gmail.com"
            android:id="@+id/userIdViewfb"
            android:gravity="center_vertical|right"
            android:textColor="@android:color/white" />
    </LinearLayout>
   <LinearLayout
        android:orientation="horizontal"
        android:layout_width="match_parent"
```

```
android:layout_height=”wrap_content”
android:layout_gravity=”center_horizontal”
android:padding=”10dp”>
<ImageView
    android:layout_width=”100dp”
    android:layout_height=”100dp”
    android:id=”@+id/userPhotoViewfb”
    android:padding=”5dp”
    android:layout_gravity=”top” />
<LinearLayout
    android:orientation=”vertical”
    android:layout_width=”wrap_content”
    android:layout_height=”match_parent”
    android:padding=”5dp”
    android:gravity=”center_vertical|right”
    android:layout_weight=”3”>
    <TextView
        android:layout_width=”wrap_content”
        android:layout_height=”wrap_content”
        android:textAppearance=”?android:attr/
                        textAppearanceLarge”
        android:text=”Welcome “
        android:id=”@+id/textView3”
        android:textColor=”@android:color/black” />
    <TextView
        android:layout_width=”wrap_content”
        android:layout_height=”wrap_content”
        android:textAppearance=”?android:attr/
                        textAppearanceLarge”
        android:text=”Aptech Limited”
```

```
                android:id="@+id/userRealNameViewfb"
                android:textColor="@color/com_facebook_blue" />
        </LinearLayout>
    </LinearLayout>
</LinearLayout>
```

6. Navigate to app → java → aptech.com.playservices.
7. Open GooglePlayServicesActivity.java and modify the code as shown in Code Snippet 6.

Code Snippet 6:

```
package aptech.com.playservices;
import android.app.Activity;
import android.content.DialogInterface;
import android.content.DialogInterface.OnCancelListener;
import android.content.Intent;
import android.content.IntentSender.SendIntentException;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.Button;
import android.widget.Toast;
import com.facebook.AccessToken;
import com.facebook.CallbackManager;
import com.facebook.FacebookCallback;
import com.facebook.FacebookException;
import com.facebook.FacebookSdk;
import com.facebook.Profile;
import com.facebook.login.LoginManager;
import com.facebook.login.LoginResult;
```

```
import com.facebook.login.widget.LoginButton;
import com.google.android.gms.common.ConnectionResult;
import com.google.android.gms.common.GooglePlayServicesUtil;
import com.google.android.gms.common.api.GoogleApiClient;
import com.google.android.gms.plus.Plus;
import com.google.android.gms.plus.model.people.Person;
import java.util.ArrayList;
import java.util.List;
public class GooglePlayServicesActivity extends Activity
                                                implements
        GoogleApiClient.ConnectionCallbacks,
        GoogleApiClient.OnConnectionFailedListener {
    private static final String TAG = "GooglePlayServicesActiv";
    private static final String KEY_IN_RESOLUTION = "is_in_
                                                  resolution";
    /**
     * Request code for auto Google Play Services error
                                                 resolution.
     */
    protected static final int REQUEST_CODE_RESOLUTION = 1;
    /**
     * Google API client.
     */
    private GoogleApiClient mGoogleApiClient;
    /**
     * Determines if the client is in a resolution state, and
     * waiting for resolution intent to return.
     */
    private boolean mIsInResolution;
```

```
Button gPlusLogin;
private boolean gplusClicked;
private boolean gplusInProgress;
LoginButton fbLoginButton;
CallbackManager callbackManager;
List<String> fbPermissions;
AccessToken accessToken;
/**
 * Called when the activity is starting. Restores the
                                        activity state.
 */
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    if (savedInstanceState != null) {
        mIsInResolution = savedInstanceState.getBoolean(KEY_
                                        IN_RESOLUTION, false);
    }
    setContentView(R.layout.login_layout);
    setTitle("Login");
    gPlusLogin  = (Button)findViewById(R.id.gpluslogin);
    gplusClicked = false;
    gplusInProgress = false;
    FacebookSdk.sdkInitialize(getApplicationContext());
    callbackManager = CallbackManager.Factory.create();
    fbLoginButton = (LoginButton)findViewById(R.
                            id.fbLoginButton);
    fbPermissions = new ArrayList<String>();
    fbPermissions.add("public_profile");
```

```
        fbPermissions.add("email");
        fbPermissions.add("user_friends");
        fbLoginButton.setReadPermissions(fbPermissions);
        fbLoginButton.registerCallback(callbackManager, new
                        FacebookCallback<LoginResult>() {
            @Override
            public void onSuccess(LoginResult loginResult)
{//Alternate method to retrieve user ID of the logged in user
                /*String userId = loginResult.getAccessToken().
                                                    getUserId();*/
                accessToken = loginResult.getAccessToken();
                loadFbInfo();
            }
            @Override
            public void onCancel() {
            }
            @Override
            public void onError(FacebookException e) {
                Toast.makeText(getApplicationContext(),"Facebook
                        Login failed",Toast.LENGTH_LONG).show();
            }
        });
    }
   /**
     * Called when the Activity is made visible.
     * A connection to Play Services need to be initiated as
     * soon as the activity is visible. Registers {@code
ConnectionCallbacks}
     * and {@code OnConnectionFailedListener} on the
     * activities itself.
```

```
     */
    @Override
    protected void onStart() {
        super.onStart();
        if (mGoogleApiClient == null) {
            mGoogleApiClient = new GoogleApiClient.Builder(this)
                    .addApi(Plus.API)
                    .addScope(Plus.SCOPE_PLUS_LOGIN)
// Optionally, add additional APIs and scopes if required.
                    .addConnectionCallbacks(this)
                    .addOnConnectionFailedListener(this)
                    .build();
        }
        //mGoogleApiClient.connect();
        if(getIntent().getStringExtra("logout").equals("GPlus"))
        {
            logoutGPlus();
        }
        if(getIntent().getStringExtra("logout").equals("fb"))
        {
            logoutFb();
        }
    }
    /**
     * Called when activity gets invisible. Connection to Play
                                                Services needs to
     * be disconnected as soon as an activity is invisible.
     */
```

```
@Override
protected void onStop() {
    if (mGoogleApiClient != null) {
        mGoogleApiClient.disconnect();
    }
    super.onStop();
}
/**
 * Saves the resolution state.
 */
@Override
protected void onSaveInstanceState(Bundle outState) {
    super.onSaveInstanceState(outState);
    outState.putBoolean(KEY_IN_RESOLUTION, mIsInResolution);
}
/**
 * Handles Google Play Services resolution callbacks.
 */
@Override
protected void onActivityResult(int requestCode, int
                            resultCode, Intent data) {
    super.onActivityResult(requestCode, resultCode, data);
    switch (requestCode) {
        case REQUEST_CODE_RESOLUTION:
            if (resultCode != RESULT_OK) {
                gplusClicked = false;
            }
           gplusInProgress = false;
```

```
                retryConnecting();
                break;
            }
        }
        private void retryConnecting() {
            mIsInResolution = false;
            if (!mGoogleApiClient.isConnecting()) {
                mGoogleApiClient.connect();
            }
        }
        /**
         * Called when {@code mGoogleApiClient} is connected.
         */
        @Override
        public void onConnected(Bundle connectionHint) {
            Log.i(TAG, "GoogleApiClient connected");
            loadGPlusInfo();
        }
        private void loadGPlusInfo() {
            try {
                if (Plus.PeopleApi.getCurrentPerson
(mGoogleApiClient) != null) {
                    Person user = sPlus.PeopleApi.getCurrentPerson
(mGoogleApiClient);
                   String userName = Plus.AccountApi.
getAccountName(mGoogleApiClient);
                    String userRealName = user.getDisplayName();
                    String userPhotoUrl = user.getImage().getUrl();
```

```
                String userCoverUrl = user.getCover().
                            getCoverPhoto().getUrl();
                Intent i = new Intent(GooglePlayServicesActivity.
                            this,GPlusActivity.class);
                i.putExtra("userName",userName);
                i.putExtra("userRealName",userRealName);
                i.putExtra("userPhotoUrl",userPhotoUrl);
                i.putExtra("userCoverUrl",userCoverUrl);
                startActivity(i);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
    private void loadFbInfo()
    {
        Profile userProfile = Profile.getCurrentProfile();
        String userId = userProfile.getId();
        String userRealName = userProfile.getName();
        String userPhotoUrl = userProfile.
                        getProfilePictureUri(100,100).toString();
        Intent i = new Intent(GooglePlayServicesActivity.
                                    this,GPlusActivity.class);
        i.putExtra("useId",userId);
        i.putExtra("userRealName",userRealName);
        i.putExtra("userPhotoUrl",userPhotoUrl);
        startActivity(i);
    }
    /**
```

```
     * Called when {@code mGoogleApiClient} connection is
                                                  suspended.
     */
    @Override
    public void onConnectionSuspended(int cause) {
        Log.i(TAG, "GoogleApiClient connection suspended");
        retryConnecting();
    }
    /**
     * Called when {@code mGoogleApiClient} is trying to connect
                                                      but failed.
     * Handle {@code result.getResolution()} if there is a
                                                    resolution
     * available.
     */
    @Override
    public void onConnectionFailed(ConnectionResult result) {
        Log.i(TAG, "GoogleApiClient connection failed: " +
                                        result.toString());
        if (!result.hasResolution()) {
            // Show a localized error dialog.
            GooglePlayServicesUtil.getErrorDialog(
                    result.getErrorCode(), this, 0, new
                                    OnCancelListener() {
            @Override
           public void onCancel(DialogInterface dialog) {
                            retryConnecting();
                        }
                    }).show();
           return;
```

```
        }
  // If there is an existing resolution error being displayed
                                                  //or a resolution
// activity has started before, do nothing and wait for
                                                     //resolution
        // progress to be completed.
        if (mIsInResolution) {
            return;
        }
        if (!gplusInProgress) {
            mIsInResolution = true;
            if (gplusClicked && result.hasResolution()) {
                try {
             result.startResolutionForResult(this, REQUEST_CODE_
                                                      RESOLUTION);
                    gplusInProgress = true;
                } catch (SendIntentException e) {
                    gplusInProgress = false;
                    mGoogleApiClient.connect();
                }
            }
        }
    }
    public void onGPlusClicked(View view) {
        if(!mGoogleApiClient.isConnecting()) {
            gplusClicked=true;
            mGoogleApiClient.connect();
        }
    }
```

```
    private void logoutGPlus(){
        if (mGoogleApiClient.isConnected()) {
            Plus.AccountApi.clearDefaultAccount
                                        (mGoogleApiClient);
            mGoogleApiClient.disconnect();
            mGoogleApiClient.connect();
        }
    }
    private void logoutFb(){
        LoginManager.getInstance().logOut();
    }
}
```

Once the activity is created, the permissions required for the application are initialized. When the Login button is clicked, the Login process is started and a callback listener is registered for a successful login. The listener retrieves the logged in user information and displays it to the user.

8. Open FacebookActivity.java and modify the code as shown in Code Snippet 7.

Code Snippet 7:

```
package aptech.com.playservices;
import android.app.Activity;
import android.content.Intent;
import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.widget.ImageView;
import android.widget.TextView;
import java.io.IOException;
```

```
import java.net.URL;
public class FacebookActivity extends Activity {
    String userId;
    String userRealName;
    String pictureUrl;
    TextView userIdView;
    TextView userRealNameView;
    ImageView userPhotoView;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_facebook);
        setTitle("Facebook");
        Intent i = getIntent();
        userId = i.getStringExtra("userId");
        userRealName = i.getStringExtra("userRealName");
        pictureUrl = i.getStringExtra("userPhotoUrl");
        userIdView = (TextView)findViewById(R.id.userIdViewfb);
        userRealNameView = (TextView)findViewById(R.id.
                                    userRealNameViewfb);
        userPhotoView = (ImageView)findViewById(R.id.
                                        userPhotoViewfb);
        userIdView.setText(userId);
        userRealNameView.setText(userRealName);
       try {
            URL url = new URL(pictureUrl);
            Bitmap bmp= BitmapFactory.decodeStream(url.
                        openConnection().getInputStream());
           userPhotoView.setImageBitmap(bmp);
```

```
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
 // Inflate the menu; this adds items to the action bar if it is
//present.
        getMenuInflater().inflate(R.menu.menu_facebook, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
   // Handle action bar item clicks here. The action bar will
  // automatically handle clicks on the Home/Up button, so long
  // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
       if(id == R.id.action_logout){
            Intent i = new Intent(FacebookActivity.
                        this,GooglePlayServicesActivity.class);
            i.putExtra("logout","fb");
            startActivity(i);
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
}
```

The activity receives user data through the use of Intent Extras. The information is then displayed to the user.

9. Navigate to res → menu and open menu_facebook.xml for editing.

10. Modify the code as shown in Code Snippet 8.

Code Snippet 8:

```
<menu
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    tools:context="aptech.com.playservices.FacebookActivity">
    <item android:id="@+id/action_settings"
          android:title="@string/action_settings"
          android:orderInCategory="100"
          android:showAsAction="never"/>
    <item android:id="@+id/action_logout"
          android:title="Logout"
          android:orderInCategory="100"
          android:showAsAction="never"/>
</menu>
```

11. Navigate to manifests and open AndroidManifest.xml for editing.

12. Modify the code as shown in Code Snippet 9.

Code Snippet 9:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
xmlns:android="http://schemas.android.com/apk/res/
                                                    android"
    package="aptech.com.playservices" >
 <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.GET_
                                                  ACCOUNTS" />
    <uses-permission android:name="android.permission.USE_
                                                CREDENTIALS" />
   <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
```

```
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <activity
            android:name=".GooglePlayServicesActivity"
            android:label="@string/app_name"
            android:theme="@style/Main">
            <intent-filter>
                <action android:name="android.intent.action.
                                                    MAIN" />
                <category android:name="android.intent.category.
                                                    LAUNCHER" />
            </intent-filter>
        </activity>
        <meta-data
            android:name="com.google.android.gms.version"
            android:value="@integer/google_play_services_
                                                version" />
    <activity
            android:name=".GPlusActivity"
            android:label="@string/title_activity_gplus"
            android:theme="@style/Google">
        </activity>
        <activity
            android:name=".FacebookActivity"
            android:label="@string/title_activity_facebook"
            android:theme="@style/Facebook">
        </activity>
    <provider
            android:authorities="com.facebook.app.
                        FacebookContentProvider935989213130894"
            android:name="com.facebook.FacebookContentProvider"
           android:exported="true" />
```

```
        <meta-data android:name="com.facebook.sdk.ApplicationId"
                        android:value="@string/facebook_app_id"/>
    </application>
</manifest>
```

Note that the <provider> tag has the Application id from the previous step appended to the provider authority.

13. Navigate to res → values.
14. Open styles.xml for editing and modify the code as shown in Code Snippet 10.

Code Snippet 10:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="AppTheme" parent="android:Theme.Material.Light">
    </style>
    <style name="Google" parent="android:Theme.Material">
       <item name="android:colorPrimary">@android:color/holo_
                                              green_light</item>
        <item name="android:colorPrimaryDark">@android:color/holo_
                                              green_dark </item>
        <item name="android:colorAccent">@android:color/holo_blue_
                                                 light</item>
        <item name="android:textColorPrimary">@android:color/
                                                 white</item>
        <item name="android:navigationBarColor">@android:color/
                                          holo_green_light</item>
    </style>
    <style name="Facebook"
      parent="android:Theme.Material.Light">
        <item name="android:colorPrimary">@color/com_facebook_
                                                  blue</item>
        <item name="android:colorPrimaryDark">@color/fbdark</item>
        <item name="android:colorAccent">@android:color/holo_blue_
                                                 light</item>
        <item name="android:textColorPrimary">@android:color/
                                                 white</item>
```

```
        <item name="android:navigationBarColor">@color/com_
                    facebook_button_background_color</item>
    </style>
    <style name="Main" parent="android:Theme.Material">
    </style>
</resources>
```

15. Open strings.xml for editing.
16. Modify the code as shown in Code Snippet 11.

Code Snippet 11:

```
<resources>
    <string name="app_name">Play Services</string>
    <string name="title_activity_gplus">GPlusActivity</string>
    <string name="hello_world">Hello world!</string>
    <string name="action_settings">Settings</string>
    <string name="title_activity_facebook">FacebookActivity</
                                                    string>
    <string name="facebook_app_id">935989213130894</string>
</resources>
```

17. Navigate to the Build menu and select Generate Signed APK.
18. Click Next.
19. Click Finish.
20. Connect the mobile device through USB cable.
21. Enable USB to connect to storage/device with developer mode enabled.
22. Install the adb drivers for the device.
23. Copy the generated signed apk from the Project Folder → app to the device.
24. Open the APK on the device and select install.
25. Once the application is installed, run the application from the App Drawer.

Once the application is run on the device, the output is as shown in figure 10.11.

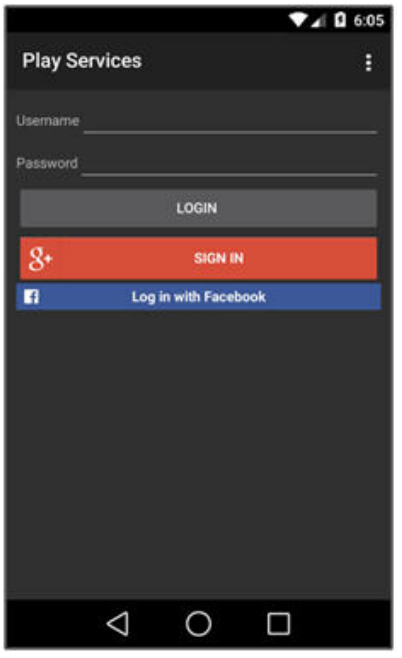


Figure 10.11: Play Services Application with Facebook

Click the Log in with Facebook button and login to your Facebook account. Once logged in, the user is redirected to the Facebook login homepage as shown in figure 10.12.

Figure 10.12: Facebook Homepage

Users can logout from the action menu and return to the login page as shown in figure 10.13.

Figure 10.13: Logout Option in Action Menu

Note - Facebook provides an Authentication Web service. It is not made by Google (meaning it is third party). The application demonstrated the use of third party API to access a Web Service.

10.2.3 Creating Web Services

In this section, the process to create a custom Web service with ReSTful API will be explained. For demonstration, the application developed earlier will now be extended to support custom Web service login as well.

☐ Creating the Web Service

Before we create an application using the Web service, the Web service itself needs to be created. For demonstration purposes, a simple PHP Web service is created running on an Apache Server. The PHP code is as shown in Code Snippet 12.

Code Snippet 12:

```
<?php
$username = $_GET['uname'];
$password = $_GET['pass'];
if($username == "admin" && $password == "adminpass")
{ echo "success"; }
else
{ echo "failure"; }
?>
```

The service checks the user name and password and if it matches the values, success is returned. If not, failure is returned.

- **Coding the Application**

Now that the Web Service is available, the application can be prepared. The steps are as follows:

1. Open the PlayServices project.
2. Create a new Blank Activity called WebServiceActivity.
3. Navigate to res → layout.
4. Open login_layout.xml for editing and modify the code as shown in Code Snippet 13.

Code Snippet 13:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:weightSum="1"
    android:padding="10dp">
   <LinearLayout
        android:orientation="horizontal"
        android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Username"
            android:id="@+id/textView" />
        <EditText
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:inputType="textEmailAddress"
            android:ems="10"
            android:id="@+id/username"
            android:layout_weight="1"
         android:textColor="@android:color/holo_red_dark" />
    </LinearLayout>
    <LinearLayout
        android:orientation="horizontal"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" >
       <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Password"
            android:id="@+id/textView2" />
       <EditText
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:inputType="textPassword"
```

```
            android:ems="10"
            android:id="@+id/password"
            android:layout_weight="1"
            android:textColor="@android:color/holo_red_dark" />
    </LinearLayout>
    <Button
        android:layout_width="fill_parent"
        android:layout_height="50dp"
        android:text="Login "
        android:onClick="onWebServiceLoginClicked"
        android:id="@+id/button"
        android:layout_gravity="center_horizontal" />
    <com.google.android.gms.common.SignInButton
        android:id="@+id/gpluslogin"
        android:layout_width="fill_parent"
        android:layout_height="50dp"
        android:onClick="onGPlusClicked"/>
    <com.facebook.login.widget.LoginButton
        android:id="@+id/fbLoginButton"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:layout_gravity="center_horizontal"
        android:layout_weight="0.11"
        android:minHeight="50dp"
        android:padding="10dp" />
</LinearLayout>
```

A login button has been added which can be used for user authentication using a custom created Web Service.

5. Add two images in res → drawable folder for the cover and profile picture for the Web service with the names aptech and admincover.
6. Open activity_web_service.xml for editing and modify the code as shown in Code Snippet 14.

Code Snippet 14:

```
<LinearLayout
    xmlns:android=”http://schemas.android.com/apk/res/android”
    xmlns:tools=”http://schemas.android.com/tools”

    android:layout_width=”match_parent”
    android:layout_height=”match_parent”
    android:paddingLeft=”0dp”
    android:paddingRight=”0dp”
    android:paddingTop=”0dp”
    android:paddingBottom=”0dp”
    tools:context=”.WebServiceActivity”
    android:orientation=”vertical”
    android:padding=”0dp”>
    <LinearLayout
        android:orientation=”horizontal”
        android:layout_width=”match_parent”
        android:layout_height=”50dp”
        android:background=”@android:color/holo_red_light”
        android:padding=”10dp”>
       <TextView
            android:layout_width=”match_parent”
            android:layout_height=”match_parent”
            android:text=”Admin”
            android:id=”@+id/userIdView”
            android:gravity=”center_vertical|right”
            android:textColor=”@android:color/white” />
    </LinearLayout>
```

```
<LinearLayout
    android:orientation=”horizontal”
    android:layout_width=”match_parent”
    android:layout_height=”150dp”>
    <ImageView
        android:layout_width=”wrap_content”
        android:layout_height=”fill_parent”
        android:id=”@+id/userCoverView”
        android:layout_gravity=”center_horizontal”
        android:layout_weight=”1”
        android:src=”@drawable/admincover” />
 </LinearLayout>
 <LinearLayout
    android:orientation=”horizontal”
    android:layout_width=”match_parent”
    android:layout_height=”wrap_content”
    android:layout_gravity=”center_horizontal”
    android:padding=”10dp”>
    <ImageView
        android:layout_width=”200dp”
        android:layout_height=”100dp”
        android:id=”@+id/userPhotoView”
        android:padding=”5dp”
        android:src=”@drawable/aptech”
        android:layout_gravity=”top”
        android:longClickable=”true” />
 <LinearLayout
    android:orientation=”vertical”
```

```
            android:layout_width="wrap_content"
            android:layout_height="match_parent"
            android:padding="5dp"
            android:gravity="center_vertical|right"
            android:layout_weight="3">

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:textAppearance="?android:attr/
                                        textAppearanceLarge"
                android:text="Welcome "
                android:id="@+id/textView3"
                android:textColor="@android:color/black" />

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:textAppearance="?android:attr/
                                       textAppearanceLarge"
                android:text="Aptech Limited"
                android:id="@+id/userRealNameView"
                android:textColor="@android:color/holo_red_dark"/>
        </LinearLayout>
    </LinearLayout>
</LinearLayout>
```

The Post Login Screen displays the user information.

7. Navigate to app → java → aptech.com.playservices.

8. Open GooglePlayServicesActivity.java and modify the code as shown in Code Snippet 15.

Code Snippet 15:

```
package aptech.com.playservices;
import android.app.Activity;
import android.content.DialogInterface;
import android.content.DialogInterface.OnCancelListener;
import android.content.Intent;
import android.content.IntentSender.SendIntentException;
import android.net.Uri;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;
import com.facebook.AccessToken;
import com.facebook.CallbackManager;
import com.facebook.FacebookCallback;
import com.facebook.FacebookException;
import com.facebook.FacebookSdk;
import com.facebook.Profile;
import com.facebook.login.LoginManager;
import com.facebook.login.LoginResult;
import com.facebook.login.widget.LoginButton;
import com.google.android.gms.common.ConnectionResult;
import com.google.android.gms.common.GooglePlayServicesUtil;
import com.google.android.gms.common.api.GoogleApiClient;
import com.google.android.gms.plus.Plus;
```

```
import com.google.android.gms.plus.model.people.Person;
import org.apache.http.HttpResponse;
import org.apache.http.HttpStatus;
import org.apache.http.StatusLine;
import org.apache.http.client.HttpClient;
import org.apache.http.client.methods.HttpGet;
import org.apache.http.impl.client.DefaultHttpClient;
import java.io.BufferedReader;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.InputStreamReader;
import java.util.ArrayList;
import java.util.List;
public class GooglePlayServicesActivity extends Activity
                                                    implements
        GoogleApiClient.ConnectionCallbacks,
        GoogleApiClient.OnConnectionFailedListener {
    private static final String TAG = "GooglePlayServicesActiv";
    private static final String KEY_IN_RESOLUTION = "is_in_
                                                    resolution";
    /**
     * Request code for auto Google Play Services error
                                                    resolution.
     */
    protected static final int REQUEST_CODE_RESOLUTION = 1;
    /**
     * Google API client.
     */
    private GoogleApiClient mGoogleApiClient;
```

```
    /**
     * Determines if the client is in a resolution state, and
     * waiting for resolution intent to return.
     */
    private boolean mIsInResolution;
    Button gPlusLogin;
    private boolean gplusClicked;
    private boolean gplusInProgress;
    LoginButton fbLoginButton;
    CallbackManager;
    List<String> fbPermissions;
    AccessToken;
    /**
     * Called when the activity is starting. Restores the
                                            activity state.
     */
    @Override
    protected void onCreate(Bundle savedInstanceState) {
       super.onCreate(savedInstanceState);
        if (savedInstanceState != null) {
            mIsInResolution = savedInstanceState.getBoolean(KEY_
                                        IN_RESOLUTION, false);
        }
setContentView(R.layout.login_layout);
        setTitle("Login");
        gPlusLogin  = (Button)findViewById(R.id.gpluslogin);
        gplusClicked = false;
        gplusInProgress = false;
        FacebookSdk.sdkInitialize(getApplicationContext());
```

```
        callbackManager = CallbackManager.Factory.create();
        fbLoginButton =
(LoginButton)findViewById(R.id.fbLoginButton);
        fbPermissions = new ArrayList<String>();
        fbPermissions.add("public_profile");
        fbPermissions.add("email");
        fbPermissions.add("user_friends");
        fbLoginButton.setReadPermissions(fbPermissions);
        fbLoginButton.registerCallback(callbackManager, new
                        FacebookCallback<LoginResult>() {
            @Override
            public void onSuccess(LoginResult loginResult) {
 //Alternate method to retrieve user ID of the logged in user
          /*String userId =
loginResult.getAccessToken().getUserId();*/
                accessToken = loginResult.getAccessToken();
                loadFbInfo();
            }
            @Override
            public void onCancel() {
}
            @Override
            public void onError(FacebookException e) {
            Toast.makeText(getApplicationContext(),"Facebook
Login failed",Toast.LENGTH_LONG).show();
            }
        });
}
```

```
    /**
     * Called when the Activity is made visible.
     * A connection to Play Services need to be initiated as
     * soon as the activity is visible. Registers {@code
                                          ConnectionCallbacks}
     * and {@code OnConnectionFailedListener} on the
     * activities itself.
     */
    @Override
    protected void onStart() {
        super.onStart();
        if (mGoogleApiClient == null) {
            mGoogleApiClient = new GoogleApiClient.Builder(this)
                    .addApi(Plus.API)
                    .addScope(Plus.SCOPE_PLUS_LOGIN)
// Optionally, add additional APIs and scopes if required.
                    .addConnectionCallbacks(this)
                    .addOnConnectionFailedListener(this)
                    .build();
}
//mGoogleApiClient.connect();
        if(getIntent().getStringExtra("logout").equals("GPlus"))
        {
          logoutGPlus();
        }
        if(getIntent().getStringExtra("logout").equals("fb"))
        {
            logoutFb();
```

```
                }
    }
    /**
     * Called when activity gets invisible. Connection to Play
                                                Services needs to
     * be disconnected as soon as an activity is invisible.
     */
    @Override
    protected void onStop() {
        if (mGoogleApiClient != null) {
            mGoogleApiClient.disconnect();
        }
        super.onStop();
    }
    /**
     * Saves the resolution state.
     */
    @Override
    protected void onSaveInstanceState(Bundle outState) {
        super.onSaveInstanceState(outState);
        outState.putBoolean(KEY_IN_RESOLUTION, mIsInResolution);
    }
     /**
     * Handles Google Play Services resolution callbacks.
     */
```

```
    @Override
    protected void onActivityResult(int requestCode, int
                                    resultCode, Intent data) {
        super.onActivityResult(requestCode, resultCode, data);
        switch (requestCode) {
            case REQUEST_CODE_RESOLUTION:
                if (resultCode != RESULT_OK) {
                    gplusClicked = false;
                }
                gplusInProgress = false;
                retryConnecting();
                break;
        }
    }
private void retryConnecting() {
        mIsInResolution = false;
        if (!mGoogleApiClient.isConnecting()) {
            mGoogleApiClient.connect();
        }
    }
    /**
     * Called when {@code mGoogleApiClient} is connected.
     */
        @Override
        public void onConnected(Bundle connectionHint) {
        Log.i(TAG, "GoogleApiClient connected");
```

```
                        loadGPlusInfo();
    }
    private void loadGPlusInfo() {
        try {
            if (Plus.PeopleApi.
                getCurrentPerson(mGoogleApiClient) != null) {
                Person user = Plus.PeopleApi.
                getCurrentPerson(mGoogleApiClient);
                String userName = Plus.AccountApi.
                 getAccountName(mGoogleApiClient);
                String userRealName = user.getDisplayName();
                String userPhotoUrl = user.getImage().getUrl();
                String userCoverUrl = user.getCover().
                getCoverPhoto().getUrl();
               Intent i = new Intent(GooglePlayServicesActivity.
                this,GPlusActivity.class);
                i.putExtra("userName",userName);
                i.putExtra("userRealName",userRealName);
                i.putExtra("userPhotoUrl",userPhotoUrl);
                i.putExtra("userCoverUrl",userCoverUrl);
                startActivity(i);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
private void loadFbInfo(){
        Profile userProfile = Profile.getCurrentProfile();
        String userId = userProfile.getId();
```

```
        String userRealName = userProfile.getName();
        String userPhotoUrl = userProfile.
            getProfilePictureUri(100,100).toString();
        Intent i = new Intent(GooglePlayServicesActivity.
             this,GPlusActivity.class);
        i.putExtra("useId",userId);
        i.putExtra("userRealName",userRealName);
        i.putExtra("userPhotoUrl", userPhotoUrl);
        startActivity(i);
   }
    /**
     * Called when {@code mGoogleApiClient} connection is
                                                suspended.
*/
    @Override
    public void onConnectionSuspended(int cause) {
       Log.i(TAG, "GoogleApiClient connection suspended");
        retryConnecting();
    }
    /**
     * Called when {@code mGoogleApiClient} is trying to connect
                                                but failed.
     * Handle {@code result.getResolution()} if there is a
                                                resolution.
     * available.
     */
    @Override
     public void onConnectionFailed(ConnectionResult result) {
        Log.i(TAG, "GoogleApiClient connection failed: " +
                                        result.toString());
```

```
        if (!result.hasResolution()) {
            // Show a localized error dialog.
             GooglePlayServicesUtil.getErrorDialog(
  result.getErrorCode(), this, 0, new OnCancelListener() {
            @Override
          public void onCancel(DialogInterface dialog) {
                              retryConnecting();
                          }
                      }).show();
             return;
        }
// If there is an existing resolution error being displayed
//or a resolution activity has started before, do nothing and
//wait for resolution
         // progress to be completed.
        if (mIsInResolution) {
            return;
        }
        if (!gplusInProgress) {
            mIsInResolution = true;
            if (gplusClicked && result.hasResolution()) {
                try {
                    result.startResolutionForResult(this,
                    REQUEST_CODE_RESOLUTION);
                    gplusInProgress = true;
                } catch (SendIntentException e) {
                    gplusInProgress = false;
                    mGoogleApiClient.connect();
                }
```

```
            }
        }
    }
    public void onGPlusClicked(View view) {
        if(!mGoogleApiClient.isConnecting()){
            gplusClicked=true;
            mGoogleApiClient.connect();
        }
    }
   public void onWebServiceLoginClicked(View view) {
        String userName = ((EditText)findViewById(R.
                        id.username)).getText().toString();
        String passWord =
((EditText)findViewById(R.id.password)).getText().toString();
        webLogin(userName,passWord);
    }
    private void webLogin(String userName, String passWord) {
        Uri.Builder builder = new Uri.Builder();
        builder.scheme("http")
                .authority("192.168.1.19")
                .appendPath("webservice")
                .appendPath("index.php")
                .appendQueryParameter("uname", userName)
                .appendQueryParameter("pass", passWord);
        Uri webService = builder.build();
        try {
        HttpClient = new DefaultHttpClient();
        HttpResponse response = httpclient.execute(new
                        HttpGet(webService.toString ()));
```

```
        StatusLine = response.getStatusLine();
        if (statusLine.getStatusCode() == HttpStatus.SC_OK) {
            BufferedReader bufferedReader=new BufferedReader(new
         InputStreamReader(response.getEntity().getContent()));
            String line="";
            StringBuffer stringBuffer=new StringBuffer();
            while ((line=bufferedReader.readLine())!=null)
            {
stringBuffer.append(line);
            }
checkResponse(stringBuffer.toString());
} else {
                response.getEntity().getContent().close();
            }
        }
        catch (IOException e) {
            e.printStackTrace();
        }
    }
        private void checkResponse(String response) {
        if(response.equals("success"))
        {
            loadWebServceinfo();
        }
        else{
            Toast.makeText(getApplicationContext(),"Web Service
                        Login Failed",Toast.LENGTH_LONG).show();
        }
    }
```

```
        private void loadWebServceinfo() {
        Intent i = new Intent(GooglePlayServicesActivity.
        this,GPlusActivity.class);
        startActivity(i);
    }
private void logoutGPlus(){
  if (mGoogleApiClient.isConnected()) {
Plus.AccountApi.clearDefaultAccount(mGoogleApiClient);
            mGoogleApiClient.disconnect();
            mGoogleApiClient.connect();
}
    }
    private void logoutFb() {
        LoginManager.getInstance().logOut();
    }
}
```

9. Open WebServiceActivity.java and modify the code as shown in Code Snippet 16.

Code Snippet 16:

```
package aptech.com.playservices;
import android.app.Activity;
import android.content.Intent;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
public class WebServiceActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
       super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.activity_web_service);
        setTitle("Web Services");
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
  // Inflate the menu; this adds items to the action bar if it
                                                    //is present.
        getMenuInflater().inflate(R.menu.menu_web, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
  // Handle action bar item clicks here. The action bar will
 // automatically handle clicks on the Home/Up button, so long
  // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        if(id == R.id.action_logout){
            Intent i = new Intent(WebServiceActivity.
                 this,GooglePlayServicesActivity.class);
            startActivity(i);
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
}
```

The activity adds a Post Login screen which displays the logged in user information. The user can logout using the menu and return to the Home Screen.

10. Navigate to res → menu and open menu_web.xml for editing.
11. Modify the code as shown in Code Snippet 17.

Code Snippet 17:

```
<menu
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    tools:context="aptech.com.playservices.WebServiceActivity">
    <item android:id="@+id/action_settings"
          android:title="@string/action_settings"
          android:orderInCategory="100"
          android:showAsAction="never" />
    <item android:id="@+id/action_logout"
          android:title="Logout"
          android:orderInCategory="100"
          android:showAsAction="never" />
</menu>
```

12. Navigate to manifests and open AndroidManifest.xml for editing.
13. Modify the code as shown in Code Snippet 18.

Code Snippet 18:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
xmlns:android="http://schemas.android.com/apk/res/android"
    package="aptech.com.playservices" >
 <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.GET_
                                                ACCOUNTS" />
    <uses-permission android:name="android.permission.USE_
                                              CREDENTIALS" />
<application
        android:allowBackup="true"
```

```
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <activity
            android:name=".GooglePlayServicesActivity"
            android:label="@string/app_name"
            android:theme="@style/Main" >
            <intent-filter>
                <action android:name="android.intent.action.
                                                    MAIN" />
                <category android:name="android.intent.category.
                                                   LAUNCHER" />
            </intent-filter>
        </activity>
        <meta-data
            android:name="com.google.android.gms.version"
            android:value="@integer/google_play_services_
                                                version" />
        <activity
            android:name=".GPlusActivity"
            android:label="@string/title_activity_gplus"
            android:theme="@style/Google" >
        </activity>
        <activity
            android:name=".FacebookActivity"
            android:label="@string/title_activity_facebook"
            android:theme="@style/Facebook" >
        </activity>
        <provider
```

```
            android:name="com.facebook.FacebookContentProvider"
            android:authorities="com.facebook.app.
        FacebookContentProvider935989213130894"
            android:exported="true" />
        <meta-data
            android:name="com.facebook.sdk.ApplicationId"
            android:value="@string/facebook_app_id" />
        <activity
            android:name=".WebServiceActivity"
            android:label="@string/title_activity_web_service" >
        </activity>
    </application>
</manifest>
```

Note that the <provider> tag has the Application id from the previous step appended to the provider authority.

14. Navigate to res → values.
15. Open styles.xml for editing and modify the code as shown in Code Snippet 19.

Code Snippet 19:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="AppTheme" parent="android:Theme.Material.
                                                    Light">
    </style>
    <style name="Google" parent="android:Theme.Material">
    <item name="android:colorPrimary">@android:color/holo_green_
                                                    light</item>
    <item name="android:colorPrimaryDark">@android:color/holo_
                                                green_dark </item>
    <item name="android:colorAccent">@android:color/holo_blue_
                                                    light</item>
```

```
        <item name="android:textColorPrimary">@android:color/
                                                white</item>
        <item name="android:navigationBarColor">@android:color/
                                          holo_green_light</item>
    </style>
    <style name="Facebook" parent="android:Theme.Material.
                                                  Light">
        <item name="android:colorPrimary">@color/com_facebook_
                                                  blue</item>
        <item name="android:colorPrimaryDark">@color/fbdark</
                                                       item>
        <item name="android:colorAccent">@android:color/holo_
                                              blue_light</item>
        <item name="android:textColorPrimary">@android:color/
                                                 white</item>
        <item name="android:navigationBarColor">@color/com_
                    facebook_button_background_color</item>
    </style>
    <style name="WebService" parent="android:Theme.Material">
        <item name="android:colorPrimary">@android:color/holo_
                                              red_light</item>
        <item name="android:colorPrimaryDark">@android:color/
                                          holo_red_dark</item>
        <item name="android:colorAccent">@android:color/holo_
                                              red_light</item>
        <item name="android:navigationBarColor">@android:color/
                                          holo_red_light</item>
        <item name="android:windowBackground">@android:color/
                                        background_light</item>
    </style>
    <style name="Main" parent="android:Theme.Material">
    </style>
</resources>
```

16. Navigate to the Build menu and select Generate Signed APK.
17. Click Next.
18. Click Finish.
19. Connect the mobile device through USB cable.
20. Enable USB to connect to storage/device with developer mode enabled.
21. Install the adb drivers for the device.
22. Copy the generated signed apk from the Project Folder → app to the device.
23. Open the APK on the device and select install.
24. Once the application is installed, run the application from the App Drawer.

    The output is as shown in figure 10.14.

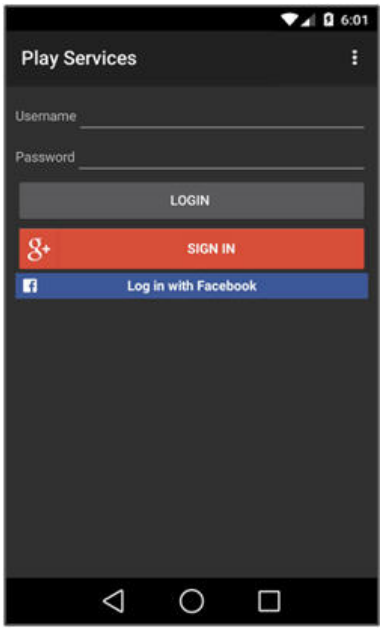


Figure 10.14: Play Services Application with Web Services

Enter the Username as admin and Password as adminpass and click LOGIN as shown in figure 10.15.

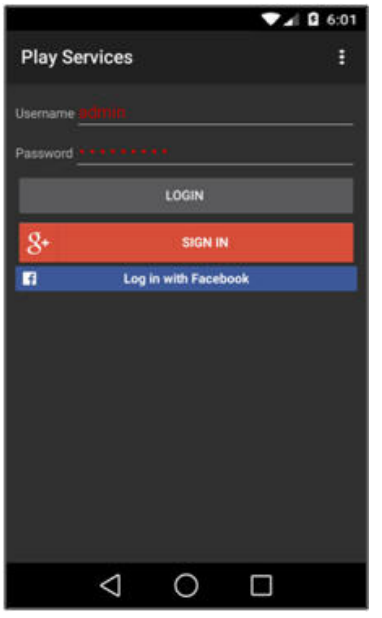


Figure 10.15: User Details

Once logged in, the user is redirected to the Web Service home screen as shown in figure 10.16.

Figure 10.16: Web Services Home Screen

Users can logout from the Action menu to return to the home screen as shown in figure 10.17.

Figure 10.17: Logout Option in Action Menu

10.2.4 Web Service Implementation

There are service providers for almost all kinds of applications. There are also storage providers such as Google Drive, Computation providers such as Amazon EC2, and so on. In most cases, creating custom Web Services can be avoided. In the event that an application requires a specialized solution, the first step is to decide the communication standard that will be used for the Web Service. ReSTful is an architectural style whereas SOAP is a protocol implementation. The design and development of a Web Service, is beyond the scope of an Android Developer and it needs to be done by professional of the respective field. All the requirements need to be communicated beforehand. Once the development is complete, the Android Developer is provided with the API and the associated documentation of the Web Service.

10.2.5 Utilizing Web Services

Web Services offer promising features and utilities. For a novice, Web Services appear to be an insta–include in all applications. In reality, this is not the case. Web Services can severely affect the overall performance of the application, if not implanted properly.

Web Services should not be implemented under the following situations:

1. If the solution can be implemented locally on the device.
2. If battery usage is a concern.
3. If application is expected to respond quickly.
4. If data access is required.
5. When Sensitive data is involved.
6. If the number of requests is expected to be low.
7. If the application is expected to function offline.

Using Web Services just for the sake of using them is considered lazy programming.

Web Services can be utilized in the following situations:

1. If a platform is already established.
2. If cross-platform services need to be provided.
3. When scaling is a concern. Web Services can scale up very easily.
4. Data Encapsulation is a concern. The mode and structure of storage of information is not revealed to the user.
5. When fault tolerance is desired. Web Services can quickly recover from major errors/crashes without revealing it to the user.
6. If services are planned to be changed/tuned. Web Services provide the option to completely rewrite the implementation without affecting the service.
7. If a task is intensive and offloading it can save energy and time on the device.

10.3 Text-to-Speech and Voice Recognition

Text-to-Speech is process of narrating text to the user in audio form. Voice Recognition is process of taking the user's audio input and generating text. Android readily provides API for both these operations. This section explains the process of implementing these features.

10.3.1 Relevant Classes

This section covers the classes that the developer needs to be familiar before developing apps using the respective API.

- **Voice Recognition**
  - **RecognizerIntent:** A broadcast intent that starts an activity for accepting user voice input. The extras for this Intent can be used for setting additional settings such as expected input language, custom text for the pop up dialog box, and a context for the voice input (that is, Web search or generic).

- **Text-to-Speech**

  - **TextToSpeech:** The central class used in text to speech systems. It is responsible for generating speech from input text. The setPitch() method is used to set the pitch of the voice and the setSpeechRate() method is used to set the speed of the voice. The setVoice() method can be used to set the voice of the speech if multiple voice packs are available. The synthesizeToFile() method can be used to save the speech output to a file.
  - **TextToSpeech.OnInitListener:** The event listener class for TextToSpeech. The onInit() method is called once TextToSpeech has been initialized. The method can be implemented to set the base settings for TTS.
  - **Voice:** The class representing a voice pack installed on the device. The name of the voice can be retrieved by using getName() method. The getQuality(), getLatency(), and getLocale() methods are used to retrieve the quality, latency, and locale information respectively.
  - **Locale:** The class representing localization information of the speech. This includes the language, sub type, and the accent combination. For example, US English differs from UK English both in terms of spelling variations and pronunciation.

10.3.2 Implementing Text-to-Speech

In this section, we will create an app which creates text based on user voice command and then narrates it to the user. The steps to achieve this are as follows:

1. Create a new project called TTSVoiceRecognition in Android Studio.
2. Navigate to res → layout and open activity_main.xml for editing.
3. Modify the code as shown in Code Snippet 20.

Code Snippet 20:

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
                          tools:context=".MainActivity"
```

```
        android:orientation="vertical"
        android:weightSum="1">
    <Button
        android:layout_width="150dp"
        android:layout_height="wrap_content"
        android:text="Voice"
        android:id="@+id/voiceRecord"
        android:onClick="onVoiceRecordClicked"
        android:layout_gravity="center_horizontal"
        android:background="@android:color/holo_green_light"
        android:layout_margin="20dp" />
    <LinearLayout
        android:orientation="horizontal"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="10dp"
        android:layout_marginTop="10dp">
        <TextView
          android:layout_width="wrap_content"
          android:layout_height="wrap_content"
          android:text="Text"
          android:id="@+id/textView4"
          android:layout_weight="1"
          android:textColor="@android:color/holo_green_light"/>
        <EditText
          android:layout_width="wrap_content"
          android:layout_height="wrap_content"
```

```
            android:inputType="textMultiLine"
            android:ems="10"
            android:lines="4"
            android:minLines="3"
            android:maxLines="6"
            android:id="@+id/editText"
            android:editable="true"
            android:enabled="true"
            android:layout_weight="1"
            android:textColor="@android:color/holo_green_light"
                                                        />
    </LinearLayout>
    <LinearLayout
        android:orientation="horizontal"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:layout_marginBottom="30dp">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="fill_parent"
            android:text="Voice"
            android:id="@+id/textView3"
            android:layout_gravity="center"
            android:gravity="center_vertical"
            android:textColor="@android:color/holo_red_light" />
       <Spinner
            android:layout_width="0dp"
            android:layout_height="fill_parent"
```

```
            android:id="@+id/spinner"
            android:spinnerMode="dialog"
            android:layout_weight="1"
            android:layout_gravity="right"
            android:gravity="center_vertical" />
    </LinearLayout>
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Pitch"
        android:id="@+id/textView"
        android:textColor="@android:color/holo_red_light" />
    <SeekBar
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:id="@+id/seekBar"
        android:progress="1"
        android:max="3" />
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Speed"
        android:id="@+id/textView2"
        android:textColor="@android:color/holo_red_light" />
    <SeekBar
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:id="@+id/seekBar2"
```

```
        android:progress="1"
        android:max="4" />
    <Button
        android:layout_width="150dp"
        android:layout_height="wrap_content"
        android:text="Text To Speech"
        android:id="@+id/TTS"
        android:layout_gravity="center_horizontal"
        android:onClick="onTTSClicked"
        android:background="@android:color/holo_red_light"
        android:layout_margin="20dp" />
</LinearLayout>
```

The activity has two sliders for adjusting the pitch and voice for the TTS conversion. A Spinner list is provided for selecting the desired Voice pack. An EditText is provided for entering data that needs to be converted to text or to display the Voice Recognized text. Buttons are provided for the desired functionality.

4. Navigate to app → java → aptech.com.ttsvoicerecognition and open MainActivity.java for editing.
5. Modify the code as shown in Code Snippet 21.

Code Snippet 21:

```
package aptech.com.ttsvoicerecognition;
import android.app.Activity;
import android.content.ActivityNotFoundException;
import android.content.Intent;
import android.os.Bundle;
import android.speech.RecognizerIntent;
import android.speech.tts.TextToSpeech;
import android.speech.tts.Voice;
```

```
import android.util.Log;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.ArrayAdapter;
import android.widget.EditText;
import android.widget.SeekBar;
import android.widget.Spinner;
import android.widget.Toast;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.Locale;
import java.util.Set;
public class MainActivity extends Activity {
    TextToSpeech;
    ArrayAdapter<String> dataAdapter;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        textToSpeech = new TextToSpeech(this, new TextToSpeech.
                                            OnInitListener() {
            @Override
            public void onInit(int status) {
                if(status != TextToSpeech.ERROR) {
                    textToSpeech.setLanguage(Locale.US);
```

```
    List<String> voiceList = new ArrayList<String>();
                    try {
                        Set<Voice> voices = textToSpeech.
                                                getVoices();
                if (voices.size() > 0) {
                 Iterator<Voice> iterator = voices.iterator();
                  while (iterator.hasNext()) {
                                voiceList.add(iterator.next().
                                                 getName());
                            }
                        }
                    }
                    catch(NullPointerException e) {
                        voiceList.add("Default");
                    }
    Spinner = (Spinner)findViewById(R.id.spinner);
                    dataAdapter = new
ArrayAdapter<String>(getApplicationContext(),
                android.R.layout.simple_spinner_item,voiceList);
                    spinner.setAdapter(dataAdapter);
                }
            }
        });
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
    // Inflate the menu; this adds items to the action bar if it
    //is present.
        getMenuInflater().inflate(R.menu.menu_main, menu);
```

```
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
   // Handle action bar item clicks here. The action bar will
  // automatically handle clicks on the Home/Up button, so long
  // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
    public void onVoiceRecordClicked(View view) {
        getVoiceInput();
    }
    private void getVoiceInput() {
        Intent intent = new Intent(RecognizerIntent.ACTION_
                                            RECOGNIZE_SPEECH);
        intent.putExtra(RecognizerIntent.EXTRA_LANGUAGE_
           MODEL,RecognizerIntent.LANGUAGE_MODEL_FREE_FORM);
        intent.putExtra(RecognizerIntent.EXTRA_LANGUAGE, "en-
                                                        US");
        intent.putExtra(RecognizerIntent.EXTRA_PROMPT, "Speak a
                                        few words for TTS");
         try {
            startActivityForResult(intent,1);
        } catch (ActivityNotFoundException a) {
```

```
            Toast.makeText(getApplicationContext(),"Voice
Recognition not supported on this Device / Currently
Unavailable",Toast.LENGTH_SHORT).show();

        }

    }

    @Override

    protected void onActivityResult(int requestCode, int
                                    resultCode, Intent data) {

        super.onActivityResult(requestCode, resultCode, data);

        switch (requestCode) {

            case 1: {

                if (resultCode == RESULT_OK && null != data) {

                    ArrayList<String> result = data.
getStringArrayListExtra(RecognizerIntent.EXTRA_RESULTS);

                    EditText ttsText = (EditText)findViewById(R.
                                                    id.editText);

                    ttsText.setText(result.get(0));

                }

                break;

            }

        }

    }

   public void onTTSClicked(View view) {

        String text = ((EditText) findViewById(R.id.editText)).
                                        getText().toString();

        float pitch = ((SeekBar) findViewById(R.id.seekBar)).
                                                getProgress();

        float speed = ((SeekBar) findViewById(R.id.seekBar2)).
                                                 getProgress();

        Spinner spinner = (Spinner) findViewById(R.id.spinner);
```

```
        try {
            Set<Voice> voices = textToSpeech.getVoices();
            if (voices.size() > 0) {
                Iterator<Voice> iterator = voices.iterator();
                int position = spinner.
                                    getSelectedItemPosition();
                for (int i = 0; i < position - 1; i++, iterator.
                                                    next()) ;
                Voice voice = iterator.next();
                textToSpeech.setVoice(voice);
            }
        }
        catch(NullPointerException e){}
        textToSpeech.setPitch(pitch);
        textToSpeech.setSpeechRate(speed);
        textToSpeech.speak(text, TextToSpeech.QUEUE_FLUSH,
                                                null,null);
    }
}
```

Once the Voice Recognition button is clicked, the application receives user input over the microphone and displays the converted text inside the EditText control. If the TEXT TO SPEECH button is clicked, the text in the EditText is converted to speech using the settings selected by the user.

6. Navigate to res → values and open styles.xml for editing.

7. Modify the code as shown in Code Snippet 22.

Code Snippet 22:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="AppTheme" parent="android:Theme.Material">
        <item name="android:colorAccent">@android:color/holo_
                                          red_light</item>
    </style>
</resources>
```

8. Navigate to manifests and open AndroidManifest.xml for editing.
9. Modify the code as shown in Code Snippet 23.

Code Snippet 23:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
xmlns:android="http://schemas.android.com/apk/res/android"
    package="aptech.com.ttsvoicerecognition" >
    <uses-permission android:name="android.permission.INTERNET"
                                                          />
    <uses-permission android:name="android.permission.ACCESS_
                                            NETWORK_STATE" />
    <uses-permission android:name="android.permission.WRITE_
                                          EXTERNAL_STORAGE" />
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <activity
        android:name=".MainActivity"
        android:label="@string/app_name" >
        <intent-filter>
```

```
    <action
        android:name="android.intent.action.MAIN" />
<category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

10. Connect the mobile device through USB cable.
11. Enable USB to connect to storage/device with developer mode enabled.
12. Install the adb drivers for the device.
13. Navigate to Run menu and select Run 'app'.
14. From the list of Connected Devices, select the device to install the apk on.
15. Click OK.

    The application is deployed on the device and the output is shown in figure 10.18.

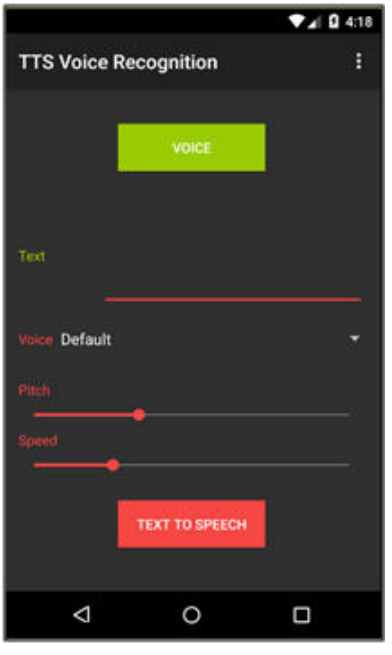


Figure 10.18: TTS Voice Recognition Application

Click the voice button and speak few words. Once completed, the voice is converted into text and the output is displayed in the text box as shown in figure 10.19.

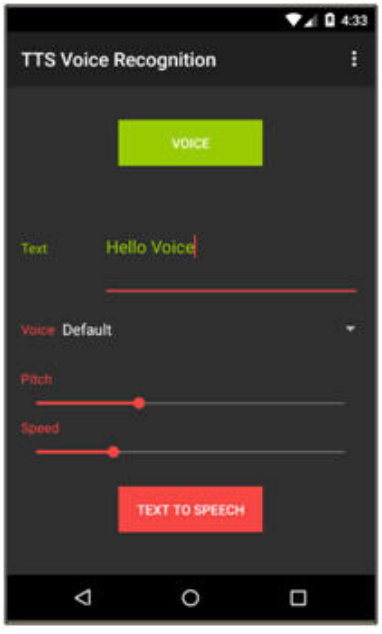


Figure 10.19: Recorded Text Converted to Speech

Adjust the voice, pitch, and speed to suit your liking and change the text if necessary. Click the TEXT TO SPEECH button to hear the speech generated.

Check Your Progress

1. Which of the following is a standard of communication with Web Services?

1.	HTTP	2.	SOAP
3.	FTP	4.	ReSTful

(A)	1, 2	(B)	3, 4
(C)	1, 3	(D)	2, 4

2. Which of the following statements are true?

1.	ReSTful cannot be used for Web Service communication
2.	Web Services cannot be used offline
3.	Web Services are always beneficial
4.	Text-to-Speech supports Multiple Voice packs

(A)	1, 3	(B)	1, 2
(C)	2, 4	(D)	2, 3

3. Which of the following classes is not related to Text-to-Speech?

(A)	TextToSpeech	(B)	Voice
(C)	GoogleApiClient	(D)	Locale

4. Registering a Facebook application requires a _____ key.

(A)	Hash	(B)	API
(C)	Vendor	(D)	Developer

5. Arrange the following steps to use third party API in the correct sequence.

1.	Generate API Key
2.	Setup Target Device
3.	Setup Android Studio
4.	Setup Hash Key

(A)	1, 2, 3, 4	(B)	4, 1, 3, 2
(C)	2, 4, 1, 3	(D)	3, 1, 2, 4

Answers

1.	D
2.	C
3.	C
4.	A
5.	B

Summary

- Web Services are application components that allow communicating devices to use features that are not available on the device locally.
- An application can use both Google API as well as third party Web Services.
- ReSTful and SOAP are well established communication standards for Web Services.
- Unnecessary use of Web Services can hurt the application performance and usability.
- Text-to-Speech (TTS) is the process of narration of text to the user. Android has built-in support for TTS which can be accessed via the TextToSpeech class.
- Voice Recognition is the converse of TTS. It is the process of converting user audio input into text.

Onlinevarsity

11 Wireless and Networking

Welcome to the Session, **Wireless and Networking.**

This session explains the Wireless Android features that can be used in Android applications. It explains the low level communication APIs by describing the Bluetooth, network, Wi-Fi, and NFC packages. Bluetooth APIs help to manage and monitor Bluetooth devices whereas Wi-Fi APIs helps to create hotspots and modify the configuration settings. NFC beam API facilitates more reliable file transfer at better speeds.

In this Session, you will learn to:

- Explain Bluetooth, Network, Wi-Fi, and NFC
- Explain the process of using Bluetooth
- Explain network management and Internet connectivity
- Explain working with Wi-Fi
- Explain working with NFC

Session

11

Wireless and Networking

11.1 Introduction

With Android and other Smartphones taking over the mobile market, the need for high speed data transfer began to rise. Wireless was the chosen medium as mobile devices cannot be tied down with a wired connection. This session explains about the primary means of wireless networking systems supported in Android.

11.2 Bluetooth

Bluetooth is one of the two networks that operate in the 2.4 GHz wireless spectrum (along with Wi-Fi). It provides a means for short distance data transfer at reasonable speeds. As a result, it became the means for peer to peer data transfer and communication channel for wireless peripheral devices such as headsets, Input controls such as keyboard and mouse, and so on.

Android provides Bluetooth APIs, which can perform the following functionalities:

- Scan for Bluetooth devices which are present nearby.
- Query for paired Bluetooth devices using the local Bluetooth adapter.
- Establishing the RFCOMM channels.
- Connect through service discovery to other devices.
- Exchange data to and from other devices.
- Manage multiple connections.

Android provides all the Bluetooth APIs' under android.bluetooth package.

The main classes and interfaces required to create Bluetooth connections are as follows:

- BluetoothAdapter
- BluetoothDevice
- BluetoothSocket
- BluetoothServerSocket
- BluetoothClass
- BluetoothProfile
- BluetoothHandset
- BluetoothA2dp
- BluetoothHealth
- BluetoothHealthCallback
- BluetoothHealthAppConfiguration
- BluetoothProfile.ServiceListener

The main functionalities of the classes are explained in detail.

- **BluetoothAdapter:**

  Is a representation of the local Bluetooth adapter and helps to perform the following functionalities:

  - Discover all Bluetooth devices.
  - Query a list of devices.
  - Create a Bluetooth device instance using the MAC address.
  - Create a BluetoothServerSocket which will enable it to listen for communications from other Bluetooth connected devices.

- **BluetoothDevice:**

  Is a representation of a remote Bluetooth device and performs the following functionalities:

  - Request a connection with remote device using BluetoothSocket.
  - Retrieve information about the device such as name, address and state of the device.

- **BluetoothSocket:**

  Is a representation of the interface like TCP socket and performs the following functionalities:

  - Allows data exchange between the Bluetooth connected devices via InputStream and OutputStream can occur through BluetoothSocket.

- **BluetoothServerSocket:**

  Is a representation of an open server socket and performs the following functionalities:

  - Listens for the incoming requests.
  - Responds to the received requests by returning a Bluetooth socket when the connection is accepted.

- **Bluetooth Class:**

  Is a read only set of properties that describe the characteristics and capabilities of a Bluetooth device and all Bluetooth profiles and services supported by the particular device.

- **Bluetooth Profile:**

  Is a wireless interface specification for communication between Bluetooth based devices.

- **Bluetooth Handset:**

  It provides support for Bluetooth headsets that are connected with the mobile device.

- **BluetoothA2dp:**

  It stands for Advanced Audio Distribution Profile and performs the following functionality:

  - Defines the way of streaming a high quality audio from one Bluetooth connected device to another.

- **Bluetooth Health:**

  It represents a Health Device profile proxy and is used for controlling the Bluetooth service.

- **BluetoothHealthCallback:**

  Is an abstract class that the callback methods must be implemented for the following functionality:

  - Receive updates about the changes in the Bluetooth channel state and application registration.

- **BluetoothHealthAppConfiguration:**

  It represents a configuration of the Bluetooth Health third party application that is used for communicating with a remote Bluetooth health device.

- **BluetoothProfile.Servicelistener:**

  Is an interface that performs the following functionality:

  - Notifies the IPC clients whenever they are connected or disconnected from the service.

Following application turns ON and OFF the Bluetooth, searches for the nearest Bluetooth connected devices, and displays them in list view.

1. Create a project named BlueToothExample in Android Studio.
2. Navigate to res → layout folder.
3. Modify the code in activity_main.xml file as shown in Code Snippet 1.

Code Snippet 1:

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
```

```
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:paddingLeft="@dimen/activity_horizontal_margin"
        android:paddingRight="@dimen/activity_horizontal_
                                                margin"
        android:paddingTop="@dimen/activity_vertical_margin"
        android:paddingBottom="@dimen/activity_vertical_margin"
        tools:context=".MainActivity"
        android:orientation="vertical">
    <ToggleButton
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:textOff="Bluetooth Off"
        android:textOn="Bluetooth On"
        android:id="@+id/tb"
        android:layout_weight="0"
        android:checked="false" />
    <Button
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="Search"
        android:id="@+id/buttonSearch"
        android:layout_weight="0" />
    <ListView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:id="@+id/listView"
        android:layout_weight="0" />
</LinearLayout>
```

4. Navigate to src → com.example.bluetoothexample folder.
5. Modify the code in MainActivity file as shown in Code Snippet 2.

Code Snippet 2:

```
package com.example.bluetoothexample;
import java.util.ArrayList;
import java.util.Set;
import android.os.Bundle;
import android.app.Activity;
import android.bluetooth.BluetoothAdapter;
import android.bluetooth.BluetoothDevice;
import android.content.Intent;
import android.view.Menu;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.ListView;
import android.widget.ToggleButton;
public class MainActivity extends Activity implements
OnClickListener {
private ToggleButton tb;
private BluetoothAdapter bl;
Button search;
ListView listView;
@Override
protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
            setContentView(R.layout.activity_main);
            tb = (ToggleButton) findViewById(R.id.tb);
            tb.setOnClickListener(this);
            bl = BluetoothAdapter.getDefaultAdapter();
```

```
            search = (Button) findViewById(R.id.buttonSearch);
            listView = (ListView) findViewById(R.id.listView1);
            search.setOnClickListener(new OnClickListener() {
      @Override
      public void onClick(View v) {
      // TODO Auto-generated method stub
            searchDevices();
            }
      });
      }
      @Override
      public boolean onCreateOptionsMenu(Menu menu) {
      // Inflate the menu; this adds items to the action bar if it
is present.
      getMenuInflater().inflate(R.menu.main, menu);
        return true;
      }
      @Override
      public void onClick(View v) {
            if (v == tb) {
                  if ((tb.isChecked())) {
                        try {
    IntentdiscoverableIntent= newIntent(BluetoothAdapter.ACTION_
                                          REQUEST_DISCOVERABLE);
    discoverableIntent.putExtra(BluetoothAdapter.EXTRA_
                                    DISCOVERABLE_DURATION,300);
    startActivity(discoverableIntent);
      // b1.enable();
      } catch (Exception e) {
```

```
}
            } else {
                  try {
                        bl.disable();
                  } catch (Exception ee) {
                  }
      }
}
}
      public void searchDevices() {
            BluetoothAdapter mBluetoothAdapter =
                              BluetoothAdapter.getDefaultAdapter();
            Set<BluetoothDevice> pairedDevices =
                              mBluetoothAdapter.getBondedDevices();
            ArrayList<String> s = new ArrayList<String>();
            for (BluetoothDevice bt : pairedDevices) s.add(bt.
                                                getName());
            listView.setAdapter(new ArrayAdapter<String>
                  (this,android.R.layout.simple_list_item_1,s));
      }
}
```

6. Modify the code in AndroidManifest.xml file and grant the permissions for Bluetooth as shown in Code Snippet 3.

Code Snippet 3:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
xmlns:android="http://schemas.android.com/apk/res/android"
package="com.example.bluetoothexample"
android:versionCode="1"
android:versionName="1.0" >
<uses-sdk
```

```
            android:minSdkVersion="8"
            android:targetSdkVersion="21" />
            <uses-permission android:name="android.permission.
                                                BLUETOOTH" />
            <uses-permission android:name="android.permission.
                                              BLUETOOTH_ADMIN" />
      <application
            android:allowBackup="true"
            android:icon="@drawable/ic_launcher"
            android:label="@string/app_name"
            android:theme="@style/AppTheme" >
      <activity
            android:name="com.example.bluetoothexample.
                                            MainActivity"
            android:label="@string/app_name" >
      <intent-filter>
      <action android:name="android.intent.action.MAIN" />
      <category android:name="android.intent.category.LAUNCHER"
                                                                  />
      </intent-filter>
      </activity>
</application>
</manifest>
```

To deploy the Android application on an Android-enabled mobile device, perform the following steps:

1. Connect the mobile device through USB cable.
2. Enable USB to connect to storage/device.
3. Install the adb drivers for the device.
4. Navigate to Run menu and select Run 'app'.
5. From the list of Connected Devices, select the device to install the apk on.
6. Click OK.

Figure 11.1 displays the output when connected with an actual Android device.

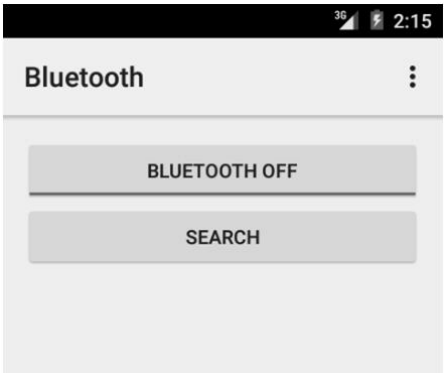


Figure 11.1: Bluetooth

Figure 11.2 is displayed when the BLUETOOTH OFF button is clicked.

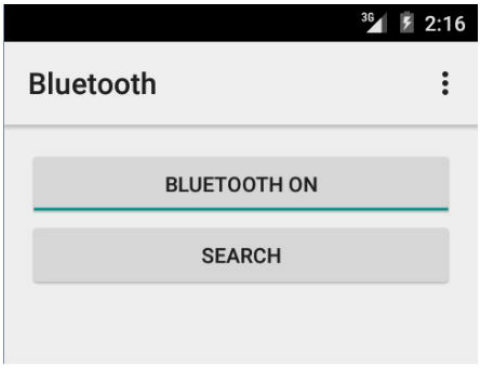


Figure 11.2: Bluetooth ON

Figure 11.3 Bluetooth Search shows the list of devices displayed when SEARCH button is clicked. It searches for nearest Bluetooth paired connected devices and displays them.

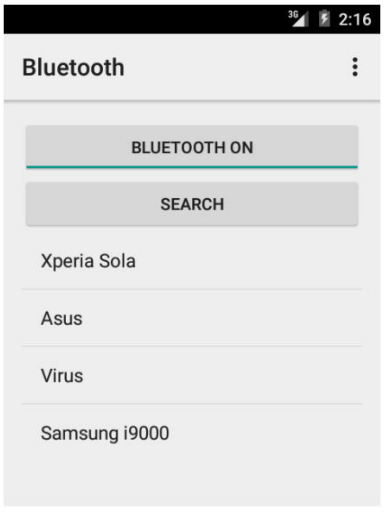


Figure 11.3: Bluetooth Search

Note - This particular application can be run only in real time Android devices since, emulator cannot check the availability of Bluetooth devices.

11.3 Mobile Networks

This section explains the process of utilizing a network connection. In Android, the type of the connection is irrelevant and hidden from the application. The network may be a mobile data connection, a Wi-Fi network connection, a wired connection, or even a Bluetooth tether. As long as a network connection is established, it will be available to access using the classes HttpClient and HttpUrlConnection.

Most of Android applications performing network operations use HTTP to send and receive data. Android uses following two HTTP clients:

- **HttpClient:**

  HttpClient encapsulate a smorgasbord objects which are required to execute HTTP requests to send and receive the data over the network. HttpClient simplifies the handling of Http Requests. The thread safety of HttpClient depends on the way the configuration of a specific client is being made.

- **HttpUrlConnection:**

  HttpUrlConnection is used to make a single request to the Http Server. Connection for the particular URL can be established by invoking the openConnection() method and the data from the URL can be read by invoking the method getInputStream() of the InputStreamReader class. It is used for sending and receiving data over the Web.

  The class uses the GET and POST method to send and receive data. By default it uses the GET method. The POST method is used if the method setDoOutput(true) method has been invoked. The other HTTP methods such as HEAD, PUT, DELETE, and so on are used with setRequest() method.

  Consider an application that checks whether the network connection is available and downloads the JSON data.

  To develop the application, perform the following steps:

  1. Create a project named NetworkConnection in Android Studio.
  2. Navigate to res → layout folder.

3. Modify the code in activity_main.xml file as shown in Code Snippet 4.

Code Snippet 4:

```
<RelativeLayout
xmlns:android="http://schemas.android. com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:paddingBottom="@dimen/activity_vertical_ margin"
android:paddingLeft="@dimen/activity_horizontal_ margin"
android:paddingRight="@dimen/activity_horizontal_ margin"
android:paddingTop="@dimen/activity_vertical_ margin"
tools:context=".MainActivity" >
      <Button
           android:id="@+id/buttonCheck"
           android:layout_width="wrap_content"
           android:layout_height="wrap_content"
           android:layout_alignParentTop="true"
           android:layout_centerHorizontal="true"
           android:layout_marginTop="26dp"
           android:text="Check Network Availability" />
     <Button
           android:id="@+id/buttonDownload"
           android:layout_width="wrap_content"
           android:layout_height="wrap_content"
           android:layout_below="@+id/buttonCheck"
           android:layout_centerHorizontal="true"
           android:layout_marginTop="28dp"
           android:text="Download data" />
```

```
      <ScrollView
            android:layout_width="fill_parent"
            android:layout_height="wrap_content"
            android:layout_alignParentBottom="true"
            android:layout_alignParentLeft="true"
            android:layout_alignParentRight="true"
            android:layout_below="@+id/buttonDownload" >
       <TextView
            android:id="@+id/textView1"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            android:text=""
            android:textAppearance="?android:attr/
            textAppearanceMedium" />
      </ScrollView>
</RelativeLayout>
```

4. Navigate to src → com.example.networkconnection folder.
5. Modify the code in MainActivity file as shown in Code Snippet 5.

Code Snippet 5:

```
package com.example.networkconnection;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.net.HttpURLConnection;
import java.net.URL;
import org.json.JSONArray;
```

```
import org.json.JSONException;
import org.json.JSONObject;
import android.content.Context;
import android.net.ConnectivityManager;
import android.net.NetworkInfo;
import android.os.AsyncTask;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;
public class MainActivity extends Activity {
      Button checkNetwork;
      Button downloadData;
      TextView showResult;
      URL url;
      HttpURLConnection connection = null;
      OutputStreamWriter request = null;
      @Override
      protected void onCreate(Bundle savedInstanceState) {
           super.onCreate(savedInstanceState);
           setContentView(R.layout.activity_main);
            checkNetwork = (Button) findViewById (R.id.
                                        buttonCheck);
            downloadData = (Button) findViewById (R.id.
                                        buttonDownload);
     showResult = (TextView) findViewById (R.id.textView1);
     checkNetwork.setOnClickListener (new OnClickListener() {
      @Override
                public void onClick(View v) {
```

```
// TODO Auto-generated method stub
      if (checkNetwork()) {
      Toast.makeText(MainActivity.this,
      "Network connection available", Toast.LENGTH_LONG).show();
                  } else {
      Toast.makeText(MainActivity.this,
     "Network connection not available", Toast.LENGTH_LONG).
                                                      show();
                        }
                  }
            });
      downloadData.setOnClickListener(new OnClickListener()
       {
            @Override
            public void onClick(View v) {
                  // TODO Auto-generated method stub
                  if (checkNetwork()) {
                   new DownloadTask().execute();
                        } else {
                  Toast.makeText (getApplicationContext(),
                 "Network connection not available", Toast.
                                          LENGTH_LONG).show();
                        }
                  }
            });
      }
      @Override
      public boolean onCreateOptionsMenu(Menu menu) {
            // Inflate the menu; this adds items to the action bar
if it is present.
```

```
getMenuInflater().inflate(R.menu.main, menu);
            return true;
      }
      public boolean checkNetwork() {
      boolean haveConnectedWifi = false;
      boolean haveConnectedMobile = false;
      boolean haveConnectedActive = false;
      ConnectivityManager connMgr;
      android.net.NetworkInfo wifi;
      android.net.NetworkInfo mobile;
      /*
      * Checking the availability of network
      */
      connMgr = (ConnectivityManager) MainActivity.this
              .getSystemService(Context. CONNECTIVITY_SERVICE);
      wifi = connMgr.getNetworkInfo(ConnectivityManag er.TYPE_
                                                    WIFI);
      mobile = connMgr.getNetworkInfo(ConnectivityMana ger.TYPE_
                                                   MOBILE);
      NetworkInfo activeNetworkInfo = connMgr.
                                      getActiveNetworkInfo();
      if (activeNetworkInfo != null) {
            haveConnectedActive = true;
      }
      if (wifi.isAvailable()) {
            haveConnectedWifi = true;
      }
      if (mobile.isAvailable()) {
            haveConnectedMobile = true;
      }
```

```
// if network is connected and either Wi-Fi or
// mobile available means
 // return true, else false
if ((haveConnectedWifi == true || haveConnectedMobile == true)
                                  && haveConnectedActive == true)
       {
            return true;
      } else {
            return false;
      }
      }
      public class DownloadTask extends AsyncTask<Context,
                                                  String, String>
       {
      String result;
      @Override
      protected String doInBackground(Context... arg0) {
            // TODO Auto-generated method stub
      try {
            url = new URL("http://free.worldweatheronline. com/
feed/weather.ashx?q="+ "bangalore"+ "&format=json&num_ of_
days=5&key=fc1372a610062229130602");
            connection =(HttpURLConnection) url.openConnection();
            connection.setDoOutput(true);
            connection. setRequestProperty("Content-Type",
                        "application/x-www-form-urlencoded");
            connection.setRequestMethod("POST");
            request = new OutputStreamWriter (connection.
                                              getOutputStream());
            request.flush();
```

```
request.close();
            StringBuilder sb = new StringBuilder();
            InputStreamReaderisr=newInputStreamReader(connection.
                                                  getInputStream());
BufferedReader reader = new BufferedReader(isr);
      String line;
            while ((line = reader.readLine()) != null) {
                  sb.append(line + "\n");
                  result = sb.toString();
            }
            } catch (Exception e) {
            e.printStackTrace();
}
return result;
}
       @Override
       protected void onProgressUpdate(String... values) {
            // TODO Auto-generated method stub
            super.onProgressUpdate(values);
      }
      @Override
      protected void onPreExecute() {
            // TODO Auto-generated method stub
            super.onPreExecute();
 Toast.makeText(getApplicationContext(), "Fetching data....",
      Toast.LENGTH_LONG).show();
}
      @Override
       protected void onPostExecute(String result) {
```

```
// TODO Auto-generated method stub
            super.onPostExecute(result);
            System.out.println("result is" + result);
            // response[0] = result;
      try {
            JSONObject rootObj = new JSONObject(result);
            JSONObject rootObj1 = rootObj. getJSONObject("data");
            if (!rootObj1.has("error")) {
            JSONArray rooms_array = rootObj1.getJSONArray
                                        ("current_ condition");
           JSONObject jsonObj = rooms_array. getJSONObject(0);
           String humidity = jsonObj. getString("humidity");
           String temperature = jsonObj. getString("temp_C");
           String pressure = jsonObj. getString("pressure");
           showResult.setText("\n\n" + "Temperature : " +
              temperature + "\n\n" + "Humidity : " + humidity +
                              "\n\n" +"Pressure : " + pressure);
      }
} catch (JSONException e) {
            Toast.makeText(getApplicationContext(),"Unable to
                      reach Server", Toast. LENGTH_SHORT).show();
            e.printStackTrace();
            finish();
} catch (NullPointerException e) {
            Toast.makeText (getApplicationContext(),"Unable
                    to reach Server", Toast. LENGTH_SHORT).show();
            e.printStackTrace();
                  finish();
                  }
            }
      }
}
```

The source code checks network connections, establishes network connections, downloads data, and parses data using JSON.

6. Modify the AndroidManifest.xml file as shown in Code Snippet 6. It is modified to grant the permissions required for accessing the network.

Code Snippet 6:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
xmlns:android="http://schemas.android.com/apk/ res/android"
package="com.example.networkconnection"
android:versionCode="1"
android:versionName="1.0" >
          <uses-permission android:name="android.permission.
                                          INTERNET"/>
          <uses-permission android:name="android.permission.
                                    ACCESS_NETWORK_STATE"/>
     <application
          android:allowBackup="true"
          android:icon="@drawable/ic_launcher"
          android:label="@string/app_name"
          android:theme="@style/AppTheme" >
     <activity
          android:name="com.example. networkconnection.
                                        MainActivity"
          android:label="@string/app_name" >
          <intent-filter>
     <action android:name="android.intent. action.MAIN" />
 <category android:name="android.intent. category.LAUNCHER" />
          </intent-filter>
          </activity>
</application>
</manifest>
```

Figure 11.4 Output - Network Application displays the output when the application is executed.

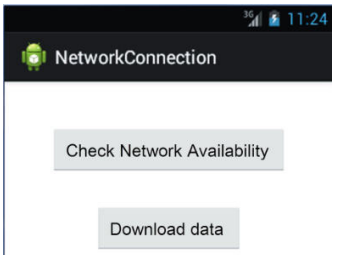


Figure 11.4: Output - Network Application

Click Check Network Availability to display the output as shown in figure 11.5.

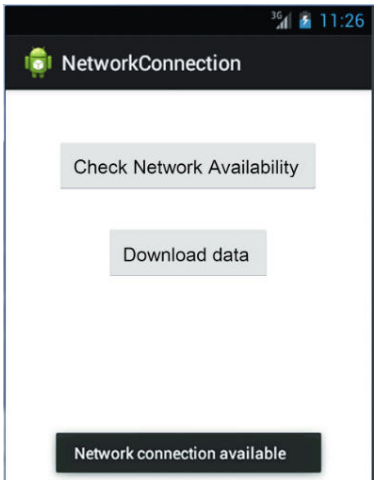


Figure 11.5: Check Network Availability

Figure 11.6 Response from Network Connection displays the response when Download Data is clicked.

Figure 11.6: Response from Network Connection

Figure 11.7 Downloaded Data displays the final output.

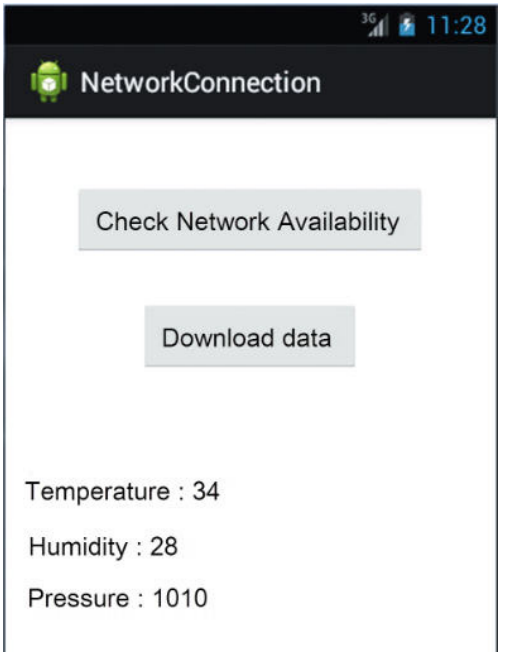


Figure 11.7: Downloaded Data

11.4 Wi-Fi

Wi-Fi is the most commonly used standard for wireless communication for high bandwidth transfer. As a result most network connections on an Android device are established using Wi-Fi. Wi-Fi runs in the same 2.4 Ghz spectrum as Bluetooth. It has a reasonably long range with high transfer rates reaching up to 300 Mbps and higher with the 802.11 n standard. It is also possible to share a mobile data connection with other devices using Android's 'tether' feature. The process to utilize a Wi-Fi connection is already explained earlier. This section explains the process to establish and manage the Wi-Fi connections.

Android provides Wi-Fi APIs through which applications can communicate with the wireless stack that provides Wi-Fi network access. These APIs also have the ability to scan, add, save, terminate, and initiate connections. Some of the classes of the APIs are as follows:

- **WifiManager:**

  This class is used to manage the Wi-Fi connectivity by invoking the method, Context. getSystemService(Context.WIFI_SERVICE). In other words, this class is used specifically when one is making Wi-Fi specific operations.

  The Wi-Fi Manager can deal with the following:

  - List of configured networks.
  - Currently available active networks.
  - Provide enough information to the developer regarding which network one has to choose and connect to.

- **ConnectivityManager:**

ConnectivityManager notifies the application whenever any network changes happen. An instance of ConnectivityManager can be created by invoking the method, Context.getSystemService(Context.CONNECTIVITY_SERVICE).

Some of the tasks performed by the ConnectivityManager are as follows:

- Scans Wi-Fi, GPRS and UMTS, and other network connections.
- Sends broadcast intents whenever the connectivity changes.
- Queries the state of available network using an API.

Following example demonstrates how to enable and disable the Wi-Fi using the WifiManager. To develop the application, perform the following steps:

1. Create a project named WifiExample in Android Studio.
2. Navigate to res → layout folder.
3. Modify the code in activity_main.xml file as shown in Code Snippet 7.

Code Snippet 7:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/ apk/res/android"
            android:layout_width="fill_parent"
            android:layout_height="fill_parent"
            android:orientation="vertical" >
<TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_gravity="center_horizontal"
            android:text="Wifi Manager"
            android:textSize="40dp" />
<Button
            android:id="@+id/onwifi"
            android:layout_width="200dp"
            android:layout_height="wrap_content"
```

```
            android:layout_gravity="center_horizontal"
            android:layout_marginTop="20dp"
            android:text="Turn Wi-Fi On" />
<Button
            android:id="@+id/offwifi"
            android:layout_width="200dp"
           android:layout_height="wrap_content"
            android:layout_gravity="center_horizontal"
            android:text="Turn Wi-Fi Off" />
      <TextView
            android:id="@+id/wifistate"
            android:layout_width="fill_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="50dp"
            android:textSize="30dp"
             android:textStyle="bold" />
</LinearLayout>
```

4. Navigate to src → com.example.wifiexample folder.
5. Modify the code in MainActivity as shown in Code Snippet 8.

Code Snippet 8:

```
package com.example.wifiexample;
import android.os.Bundle;
import android.app.Activity;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.content.IntentFilter;
import android.view.View;
```

```
import android.widget.Button;
import android.widget.TextView;
import android.net.wifi.WifiManager;
public class MainActivity extends Activity {
      TextView WifiState;
      @Override
      protected void onCreate(Bundle savedInstanceState) {
            super.onCreate(savedInstanceState);
            setContentView(R.layout.activity_main);
            WifiState = (TextView) findViewById(R.id.wifistate);
            Button OnWifi = (Button) findViewById(R.id.onwifi);
            Button OffWifi = (Button) findViewById (R.id.offwifi);
            this.registerReceiver(this. WifiStateChangedReceiver,
                  new IntentFilter(WifiManager.WIFI_STATE_CHANGED_
                                                        ACTION));
            OnWifi.setOnClickListener(new Button.OnClickListener()
                  {
                  @Override
                  public void onClick(View arg0) {
                  // TODO Auto-generated method stub
                  WifiManager wifiManager = (WifiManager)
                      getBaseContext().getSystemService(Context.
                                                  WIFI_SERVICE);
                  wifiManager.setWifiEnabled(true);
                  }
               });
           OffWifi.setOnClickListener(new Button. OnClickListener()
              {
             @Override
             public void onClick(View arg0) {
             // TODO Auto-generated method stub
```

```
        WifiManager wifiManager = (WifiManager) getBaseContext()
                    .getSystemService(Context. WIFI_SERVICE);
                wifiManager.setWifiEnabled(false);
            }
    });
}
    private BroadcastReceiver WifiStateChangedReceiver = new
                                    BroadcastReceiver() {
    @Override
    public void onReceive(Context context, Intent intent) {
        // TODO Auto-generated method stub
        int extraWifiState = intent.getIntExtra(WifiManager.
          EXTRA_WIFI_STATE,WifiManager.WIFI_STATE_UNKNOWN);
    switch (extraWifiState) {
        case WifiManager.WIFI_STATE_DISABLED:
        WifiState.setText("WIFI STATE DISABLED");
    break;
    case WifiManager.WIFI_STATE_DISABLING:
        WifiState.setText("WIFI STATE DISABLING");
    break;
    case WifiManager.WIFI_STATE_ENABLED:
        WifiState.setText("WIFI STATE ENABLED");
    break;
    case WifiManager.WIFI_STATE_ENABLING:
        WifiState.setText("WIFI STATE ENABLING");
    break;
     case WifiManager.WIFI_STATE_UNKNOWN:
        WifiState.setText("WIFI STATE UNKNOWN");
    break;
    }
    }
    };
}
```

6. Modify the code in AndroidManifest.xml file and add the permission as shown in Code Snippet 9.

Code Snippet 9:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
xmlns:android="http://schemas.android.com/apk/ res/android"
package="com.example.wifiexample"
android:versionCode="1"
android:versionName="1.0" >
<uses-sdk
          android:minSdkVersion="8"
          android:targetSdkVersion="17" />
          <uses-permission android:name="android.permission.
                                        CHANGE_WIFI_STATE"/>
     <application
          android:allowBackup="true"
          android:icon="@drawable/ic_launcher"
          android:label="@string/app_name"
          android:theme="@style/AppTheme" >
     <activity android:name="com.example.wifiexample.
                                        MainActivity"
          android:label="@string/app_name" >
     <intent-filter>
     <action android:name="android.intent. action.MAIN" />
     <category android:name="android.intent.category.LAUNCHER"/>
           </intent-filter>
          </activity>
     </application>
</manifest>
```

Figure 11.8 displays the output after executing the code.

Figure 11.8: WifiExample – Output

Figure 11.9 displays the output when Turn Wi-Fi On button is clicked.

Figure 11.9: Wi-Fi State Enabled

11.5 NFC

NFC is the abbreviation for Near Field Communication. It is a new wireless communication standard emerging to be a good replacement for other short range communication standards. It operates at 13.56 Mhz frequency and can achieve speeds up to 424 Kbit/s. NFC devices are activated on proximity using a touch or tap gesture among the devices.

NFC primarily consists of three standards/utilities:

- **Card emulation mode** – Allows the device to act as a smart card for authentication/ identification.
- **Read/Write mode** – This standard allows the device to read information from called NFC 'tags'.
- **Peer-to-Peer mode** – The mode is similar to the P2P mode of Bluetooth. It allows two devices to communicate with each other.

Card emulation standards are closely regulated for security reasons. Google Wallet is a prime example of this standard in usage. Read/Write mode is already handled by Android natively. Hence, this section will describe the Peer-to-Peer standard termed by Android as the Beam API.

NDEF Content Type

NDEF is the data format used by Android for NFC messages and records. The data formats are implemented in the classes NdefMessage and NdefRecord.

An NDEF object can be retrieved using the get(Tag) method. The tag type needs to be mentioned before the NDEF object is retrieved. There are currently four supported tag types:

- NFC_FORUM_TYPE_1
- NFC_FORUM_TYPE_2
- NFC_FORUM_TYPE_3
- NFC_FORUM_TYPE_

Relevant Classes

This section describes the relevant classes needed to use the NFC API.

- **NdefMessage**: It represents an NFC message in NDEF format.
- **NdefRecord**: Each NdefMessage contains one or more NdefRecords.
- **NfcAdapter**: It is a class used to represent the local NFC adapter. The getDefaultAdapter(Context) method can be used to retrieve a reference to the adapter.
- **NfcEvent**: A wrapper class for NFC events and notifications.
- **NfcManager**: A high level manager used to obtain references to NfcAdpater.

- **Permissions**

  The application needs to explicitly request permission to access NFC hardware. The manifest code is shown in Code Snippet 10.

Code Snippet 10:

```
<uses-permission android:name="android.permission.NFC" />
```

11.5.1 Verifying Support

It is a good practice to ensure that the device has the hardware and API support to perform NFC operation. This can be done using one of the following methods:

- **Manifest File**

  The uses-feature tag can be used to ensure that the application is prevented from being installed on devices that do not support NFC. The code for this is as shown in Code Snippet 11.

Code Snippet 11:

```
<uses-feature
android:name="android.hardware.nfc" android:required="true" />
```

- **System Feature**

  The getSystemFeature() method of the package manager can be used to query feature support inside the application. The code to achieve this is as shown in Code Snippet 12.

Code Snippet 12:

```
boolean systemSupport = getPackageManager.
getSystemFeature(PackageManager.FEATURE_NFC;
```

- **API Support**

  The API level requirement is already mentioned in the Manifest file. In the event that the application is force installed, the API level can be retrieved using the code as shown in Code Snippet 13.

Code Snippet 13:

```
int level = Build.VERSION.SDK_INT;
```

The API level can then be compared to the minimum level requirement of 16 to decide if the device is compatible.

11.5.2 File Transfer Using the Beam API

Consider an application to transfer files using NFC. The steps to create the application are as follows:

1. Create a new project called NFC Transfer in Android Studio.
2. Navigate to app → java → aptech.com.nfctransfer.
3. Open the file MainActivity.java for editing and modify the code as shown in Code Snippet 14.

Code Snippet 14:

```
package aptech.com.nfctransfer;
import android.app.Activity;
import android.net.Uri;
import android.nfc.NfcAdapter;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.Toast;
import java.io.File;
public class MainActivity extends Activity {
    private NfcAdapter nfcAdapter;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        nfcAdapter = NfcAdapter.getDefaultAdapter(this);;
    }
   @Override
    public boolean onCreateOptionsMenu(Menu menu) {
```

```
// Inflate the menu; this adds items to the action bar if it is
present.
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
   // Handle action bar item clicks here. The action bar will
   // automatically handle clicks on the Home/Up button, so long
   // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
    public void onSendClicked(View view) {
        transferFile();
    }
    private void transferFile() {
        if(!nfcAdapter.isEnabled()){
        Toast.makeText(this,"NFC not enabled",Toast.LENGTH_
                                                LONG).show();
            return;
        }
        File transferFile = new File(getExternalFilesDir(null)
                                      ,"media/nfcimage.jpg");
        Uri[] filesURIList = new Uri[]{Uri.
                                    fromFile(transferFile)};
        nfcAdapter.setBeamPushUris(filesURIList,this);
    }
}
```

4. Navigate to res → layouts and open main_acitivity.xml for editing.
5. Modify the code as shown in Code Snippet 15.

Code Snippet 15:

```
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
    android:layout_height="match_parent" android:paddingLeft="@
dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
tools:context=".MainActivity">
    <Button
        android:layout_width="fill_parent"
        android:layout_height="fill_parent"
        android:text="Start Transfer"
        android:id="@+id/button"
        android:onClick="onSendClicked"
        android:layout_centerHorizontal="true"
        android:layout_alignParentTop="true"
        android:textColor="@android:color/white"
        android:background="@android:color/holo_red_light"
        android:textSize="20dp" />
</RelativeLayout>
```

6. Navigate to manifests and open AndroidManifest.xml for editing.

7. Modify the code as shown in Code Snippet 16.

Code Snippet 16:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/
android"
    package="aptech.com.nfctransfer" >
    <uses-permission android:name="android.permission.NFC" />
    <uses-permission android:name="android.permission.READ_
EXTERNAL_STORAGE" />
    <uses-feature android:name="android.hardware.nfc"
android:required="true" />
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <activity
            android:name=".MainActivity"
            android:label="@string/app_name" >
            <intent-filter>
           <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.
                                                    LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

8. Navigate to res → values and open styles.xml for editing.

9. Modify the code as shown in Code Snippet 17.

Code Snippet 17:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
 <style name="AppTheme" parent="android:Theme.Material.Light">
        <item name="android:colorPrimary">@android:color/holo_
blue_light</item>
        <item name="android:colorPrimaryDark">@android:color/
holo_blue_dark </item>
        <item name="android:colorAccent">@android:color/holo_
blue_light</item>
        <item name="android:textColorPrimary">@android:color/
white</item>
        <item name="android:windowBackground">@color/
background</item>
        <item name="android:navigationBarColor">@android:color/
holo_blue_dark</item>
    </style>
</resources>
```

10. Connect the mobile device through USB cable.
11. Enable USB to connect to storage/device.
12. Copy an image to the media directory inside the SD card with the name nfcimage.jpg.
13. Install the adb drivers for the device.
14. Navigate to Run menu and select Run 'app'.
15. From the list of Connected Devices, select the device to install the apk On.
16. Click OK.
17. Bring two devices supporting NFC to close proximity. Use an activation gesture such as tap or touch if necessary.
18. Click the START TRANSFER button to start the transfer.

The output of the application running is as shown in figure 11.10.

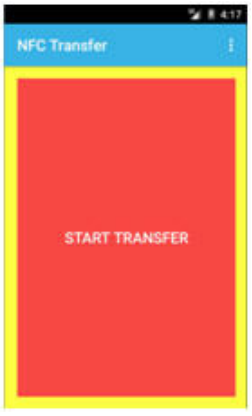


Figure 11.10: NFC File Transfer Application

The transferred file can be seen viewed on the device as shown in figure 11.11.

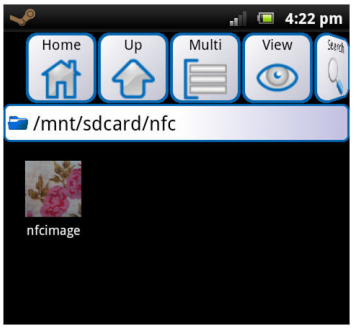


Figure 11.11: Transferred File on the Device

The file transfer and storage is managed by Android's NFC Beam service. The developer is not required to worry about it. The receiving device will show a progress notification and store the file once the transfer is complete similar to Bluetooth file transfer functionality.

Note - This particular application can be run only in real time Android devices since, emulator does not support NFC emulation.

Check Your Progress

1. Which of the following share the same wireless frequency spectrum?

1.	Bluetooth	2.	NFC
3.	Mobile Networks	4.	Wi-Fi

(A)	1, 3	(B)	2, 3
(C)	3, 4	(D)	1, 4

2. Which of the following statements are true?

1.	Data transfer can be done using Bluetooth.
2.	Wi-Fi is a short range wireless standard.
3.	NFC can be used to replace smart cards.
4.	Wireless networks are always available.

(A)	1, 3	(B)	1, 2
(C)	2, 4	(D)	2, 3

3. Which of the following method is called to get an NDEF object?

(A)	retrieve()	(B)	content()
(C)	get()	(D)	getNDEFtype()

4. Wi-Fi networks are managed by the _____ service.

(A)	NetworkManager	(B)	ConnectivityManager
(C)	WifiManager	(D)	WirelessInterface

5. Match the following classes with the corresponding functionality.

	Class		**Functionality**
a.	HttpUrlConnection	1.	Start a connection to another device
b.	BluetoothDevice	2.	Query list of Bluetooth devices
c.	BluetoothAdapter	3.	Manage Wi-Fi connectivity
d.	WifiManager	4.	Make a single request to the Http Server

(A)	a-4, b-1, c-2, d-3	(B)	a-3, b-1, c-4, d-2
(C)	a-2, b-4, c-1, d-3	(D)	a-1, b-3, c-4, d-2

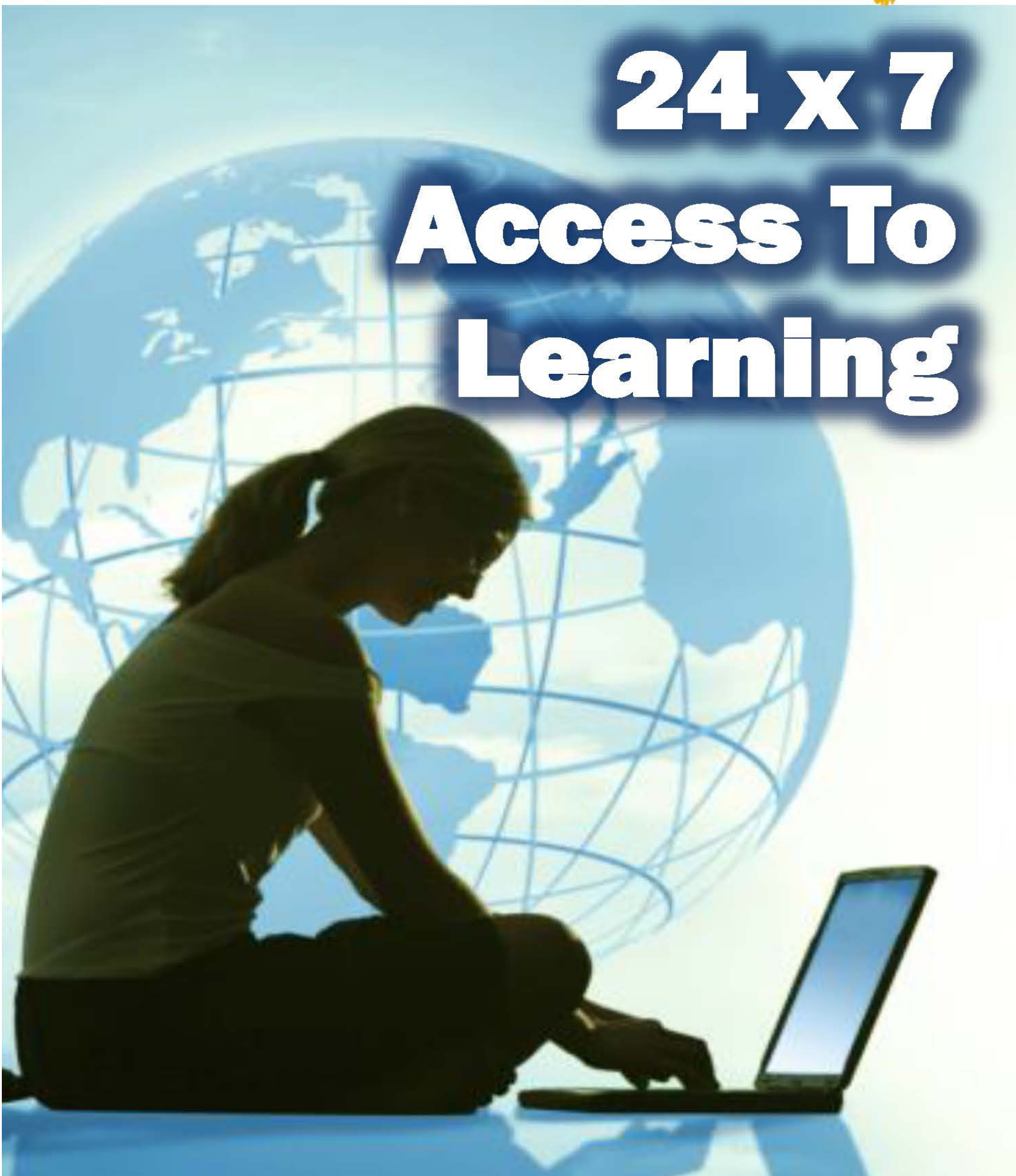
Answers

1.	D
2.	A
3.	C
4.	C
5.	A

Summary

- Bluetooth is a network stack that transfers data between devices wirelessly. Android provides all the Bluetooth APIs' under android.bluetooth package.
- Bluetooth Adapter plays the role of discovering all the Bluetooth enabled devices and querying for request.
- Android applications performing network operations uses HTTP to send and receive data.
- HttpClient can be used to send and receive data from the server by creating a DefaultHttpClient() which would help in sending and receiving of data.
- Android provides Wi-Fi APIs, through which applications can communicate with the wireless stack that provides Wi-Fi network access.
- NFC is a short range wireless data transfer protocol.
- The API for NFC is encapsulated in the android.nfc package.

Onlinevarsity



Session
12

Telephony, SMS, and VoIP

Welcome to the Session, **Telephony, SMS, and VoIP.**

This session introduces the concept of Telephony, Short Messaging Service (SMS), and Voice over Internet Protocol (VoIP) where the learners would learn how to send and receive SMS and also, the key importance of the Telephony. The basics of Session Initiation Protocol (SIP) and VoIP will also be explained.

In this Session, you will learn to:

- Explain Telephony
- Explain TelephonyManager
- Explain SMS
- Explain the process of sending SMS messages programmatically from within your application
- Explain the process of receiving incoming SMS messages
- Explain SIP and VoIP
- Use the SIP Framework to make a VoIP application

V 1.0 © Aptech Limited

12.1 Introduction

The quest for developing or inventing something new does not end. In fact, it increases as the days pass by. As soon as the basic concept of developing an Android application is clear and all applications are running, the developer might think of developing something interesting, such as adding the capability of communication with the outside world. A developer might wish to develop an application that will have the capability of sending an SMS message to another phone. This session explains how to send and receive SMS messages programmatically from within the Android application.

12.2 Telephony in Android

First the developer should learn how Android internally handles SMS and telephony services using the telephony framework and the Radio Interface Layer (RIL). The telephony architecture in Android is shown in figure 12.1.

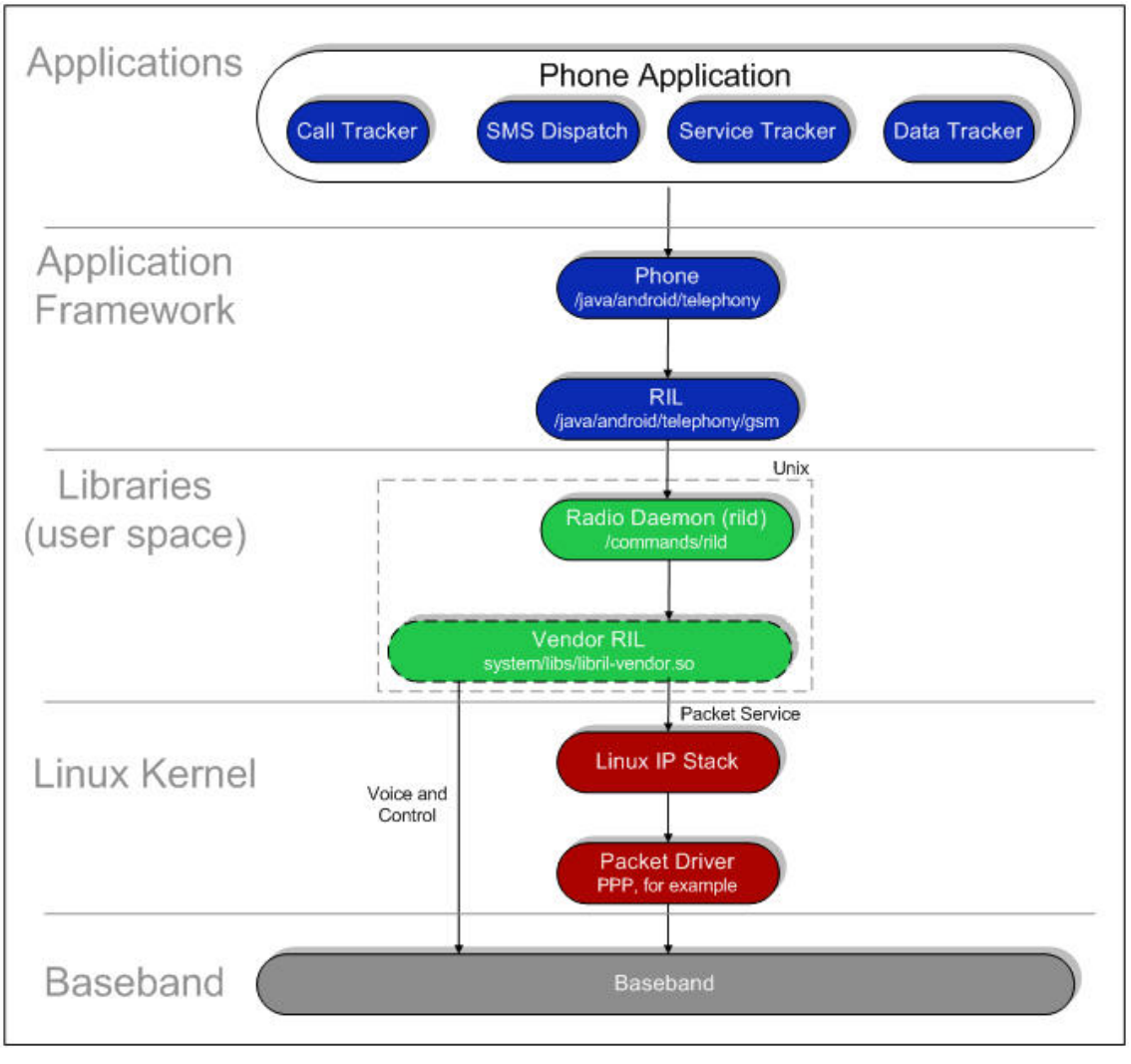


Figure 12.1: Android Telephony Architecture

12.2.1 Radio Hardware

Android devices supporting telephony services have a modem installed within them either as a separate module or as part of SoC (System on Chip). Regardless of either, Android views the modem as an autonomous device.

The communication with the modem is done with a serial port or an emulated serial port using Hayes AT Command Set. The AT commands are not unique to Android or any platform. The same operations can be performed on modem connected to a PC using a program such as Telnet. Communication using AT commands is done using a request-response model. An example of communication session is shown in Code Snippet 1.

Code Snippet 1:

```
Machine : ATD +123214563

Modem : OK

Modem : Ring

Modem : Ring

… (Other modem issues an ATA answer command)

Modem: Connect
```

The reader is encouraged to try to dial a number using data connection modems connected to a PC using telnet and AT commands.

Voice implementation varies from one modem to another. Some devices directly accept analog audio data; some accept digital PCM data where as some accept raw audio data transferred using an additional serial port.

12.2.2 Telephony Framework

The telephony framework provides services to the telephony application similar to the services provided by the Wi-Fi Manager to the application written earlier. The use of this framework to manage phone calls and SMS information is described in the following sessions. This is the API that is exposed to the developer for creating applications. The API is the same across all the devices irrespective of the modem type or the AT commands supported by the device.

12.2.3 RIL Layer

The telephony manager does not directly interact with the modem hardware. It uses the API provided by the RIL Layer. The RIL layer is where the AT commands and the modem interaction is executed.

It consists of the RIL Daemon which listens for requests and the RIL library which has the implementation to handle the requests. Due to the non-homogenous nature of the modem hardware in devices, the RIL implementation varies from one device to another. The RIL is one of the major areas of Android code that is written in C++. The RIL layer translates the application requests into AT commands understandable by the modem. For example, when the user initiates voice call from the telephony application, the telephony framework sends a request to the RIL. RIL resolves the type of the request and then sends an ATD <number> command to the modem.

12.2.4 Linux Kernel Stack

The Kernel stack is responsible for the drivers which support the modem device. It is also responsible to provide a channel of communication for the RIL layer to send the AT commands. In modern devices with SoCs, the Kernel stack is also responsible for initializing the modem hardware on startup.

12.2.5 Flow of commands

Following are the steps performed by Android whenever a user initiates a telephony request:

1. A request is initiated from the caller application or the SMS application or a custom application using the telephony manager.
2. The request is received by the telephony framework. The telephony framework then creates a request for the RIL layer using a JNI interface.
3. RIL layer resolves the type of the request and executes the necessary Hayes commands.
4. The commands are sent over a serial port or an emulated port.
5. The Kernel layer translates the data from the emulated channel into the mode of communication used by the Modem (such as USB).

12.3 Telephony

Telephony provides access to information about the telephony services on the devices. Applications can use the methods present in this class to determine the telephony services and states, as well as have access to some types of subscriber information. Applications can also register to a listener to receive notification of telephony state changes. Apart from this, the telephony API is also used for activities such as monitoring phone information including the current states of the phone, connections, and network. Android's telephony APIs help to monitor mobile voice and data connections. It helps the applications to access the underlying hardware stack so that the developer can create their own dialer and integrate them within the applications for call handling and phone state monitoring.

12.3.1 TelephonyManager

The Android TelephonyManager provides information about the Android telephony system. In other words, it provides information about the available telephony services and states on the device. To use TelephonyManager, first you need to obtain an instance of the Telephony Service by invoking the method Context.getSystemService(TELEPHONY_SERVICE), since an instance of this class cannot be directly instantiated. This particular telephony service can be used to retrieve call state, cell location, and operator name.

The access to information in telephony is set with permission and the application should have the required permission specified in the AndroidManifest.xml file to access the information.

A change to the state of the phone is monitored using the PhoneStateListener class. The listener can listen for respond to call state, phone service change, and so on. To react to the change in events of the phone state, the PhoneStateListener is created and the respective event handler of the event is overridden.

Following example displays the use of monitoring call states such as ringing, off-hook, and many more. To develop the application, perform the following steps:

1. Create a new Android application project named TelephonyExample in Android Studio.
2. Navigate to res → layout folder.

3. Modify the code in activity_main.xml file as shown in Code Snippet 2.

Code Snippet 2:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="fill_parent"
android:layout_height="fill_parent"
android:orientation="vertical" >
<TextView
     android:layout_width="fill_parent"
     android:layout_height="wrap_content"
     android:text="Telephony Demo"
      android:textSize="22sp" />
<TextView
      android:id="@+id/textViewOut"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Telephony Output" >
</TextView>
</LinearLayout>
```

4. Navigate to AndroidManifest.xml file.
5. Modify and add the code under <uses-sdk> tag as shown in Code Snippet 3.

Code Snippet 3:

```
<uses-permission
android:name="android.permission.READ_PHONE_STATE"/>
```

6. Navigate to src → com.example.telephonyexample folder.

7. Add the code in MainActivity.java file as shown in Code Snippet 4.

Code Snippet 4:

```
package com.example.telephonyexample;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.content.Context;
import android.telephony.PhoneStateListener;
import android.telephony.TelephonyManager;
import android.widget.TextView;
public class MainActivity extends Activity {
TextView Tv;
TelephonyManager telephonyManager;
PhoneStateListener phoneStateListener;
@Override
protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_main);
      // Get the UI
      Tv = (TextView) findViewById(R.id.textViewOut);
      // Get the telephony manager
      telephonyManager = (TelephonyManager)
            getSystemService(Context.TELEPHONY_SERVICE);
      // Create a new PhoneStateListener
      phoneStateListener = new PhoneStateListener() {
            @Override
            public void onCallStateChanged(int state,
                              String incomingNumber) {
                  String stateString = "N/A";
                  switch (state) {
```

```
            case TelephonyManager.CALL_STATE_IDLE:
           stateString = "Idle";
            break;
            case TelephonyManager.CALL_STATE_OFFHOOK:
            stateString = "Off State";
            break;
            case TelephonyManager.CALL_STATE_RINGING:
            stateString = "Ringing";
            break;
      }
      Tv.append(String.format("\nonCallStateChanged: %s",
                                    stateString));
      }
};
// Register the listener with the telephony manager
telephonyManager.listen(phoneStateListener,
PhoneStateListener.LISTEN_CALL_STATE);
}
}
```

Figure 12.2 displays the output in the emulator when the application is executed.

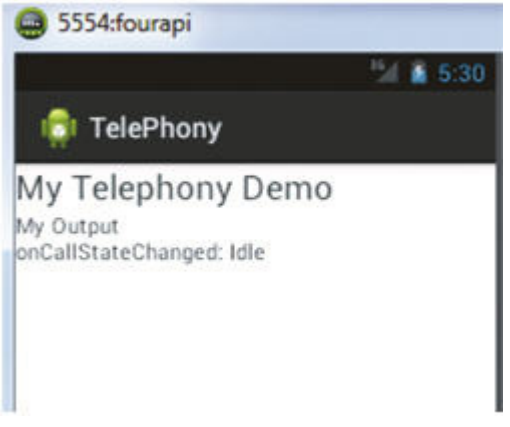


Figure 12.2: TelephonyExample - Output

12.4 SMS

SMS stands for Short Messaging Service. SMS is one of the most important and frequently used applications that are executed on mobile devices. Today, any mobile phone that users buy have SMS messaging capability and nearly all the mobile users of any age group know how to send and receive messages. Android comes with the built-in SMS application that enables you to send and receive SMS messages.

This section explains how developers can programmatically send and receive SMS messages in an Android application. The good news for the Android developers is that you do not need a real-time device to test the SMS messaging as the emulator is more than enough for testing since it has that capability built-in.

The SmsManager manages the SMS operations, such as sending text and data messages. Similarly, ConnectivityManager manages the Internet connectivity in a mobile phone. This class supervises network connections and also, deals with sending broadcast intents in case network connectivity changes.

12.4.1 Sending SMS Messages from Application

The application can send SMS to another phone, when an event takes place. In an Android application, the developer can access the SMS application using the SmsManager. To obtain the default instance of SmsManager class, the developer needs to invoke the SmsManager. getDefault() method. The class cannot be directly instantiated, unlike the other classes. The sendTextMessage() function is used to send an SMS message.

The syntax of sendTextMessage() method is as follows:

Syntax:

```
public void sendTextMessage (String destinationAddress, String scAddress, String text, PendingIntentsentIntent, PendingIntent deliveryIntent)
```

where,

destinationAddress : Denotes the recipient address.

scAddress : Gets the service centre address.

text : Represents the body of the message to be sent.

sentIntent : Broadcasted when the message is sent successfully.

deliveryIntent : Broadcasted when the message is delivered successfully.

Code Snippet 5 displays the use of SmsManager class.

Code Snippet 5:

```
…
SmsManager smsMgr = SmsManager.getDefault();
smsMgr.sendTextMessage("phone_num", null, "msg_text", null,
null);
…
```

To develop an application that sends SMS, perform the following steps:

1. Start Android Studio.
2. Create a project named SMSApp.
3. Navigate to res → layout folder.
4. Modify the code in activity_main.xml file as shown in Code Snippet 6.

Code Snippet 6:

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/ res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical" >
<Button
      android:id="@+id/buttonSendSMS"
      android:layout_width="fill_parent"
      android:layout_height="wrap_content"
      android:text="Send SMS" />
</LinearLayout>
```

5. Navigate to AndroidManifest.xml file and add the code under <uses-sdk> tag as shown in Code Snippet 7.

Code Snippet 7:

```
<uses-permission android:name="android.permission.SEND_SMS" />
```

6. Navigate to src → com.example.smsapp folder.
7. Modify the code in MainActivity.java file as shown in Code Snippet 8.

Code Snippet 8:

```
package com.example.smsapp;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.telephony.SmsManager;
public class MainActivity extends Activity {
Button button;
@Override
protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
     setContentView(R.layout.activity_main);
     button = (Button) findViewById(R.id.buttonSendSMS);
      button.setOnClickListener(newView.OnClickListener() {
            public void onClick(View v) {
                        sendSMS("5556", "Hello World!!");
            }
 private void sendSMS(String phoneNumber, String
                                              message) {
```

```
                SmsManagersmsmanager= SmsManager.
                                      getDefault();

                smsmanager.sendTextMessage(phoneNumber,
                              null, message, null, null);

      }

});

}

@Override

public boolean onCreateOptionsMenu(Menu menu) {

      // Inflate the menu; this adds items to the action bar if it
is present.getMenuInflater().inflate(R.menu.main, menu);

      return true;

}

}
```

In the code, the sendSMS ("5556","Hello World!!") method has been used. In this method, the number of the emulator to which the developer is sending the SMS message along with the message has been passed as an argument. Every emulator is identified by a number which appears on the top left corner.

Launch two different emulator devices and notice the emulator numbers appearing on the top left corner. For example, consider you have two emulators named emulator A and emulator B. The application will be executed on one of the emulators, for example, on emulator A. Then, in the application code, the developer needs to write emulator B's number. Once you execute the application in your emulator A and click Send SMS, emulator B's notification bar will display a message, "Hello world" which has been received from emulator A.

Figure 12.3 displays the output when the application is executed.

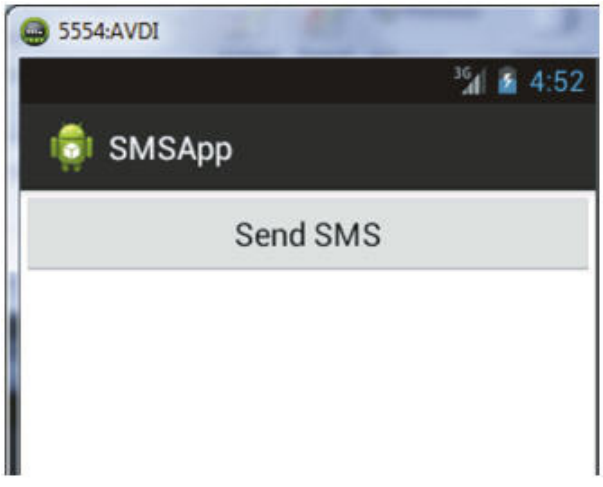


Figure 12.3: SMSApp Application – Output

Figure 12.4 displays the output in the second emulator when the Send SMS is clicked.

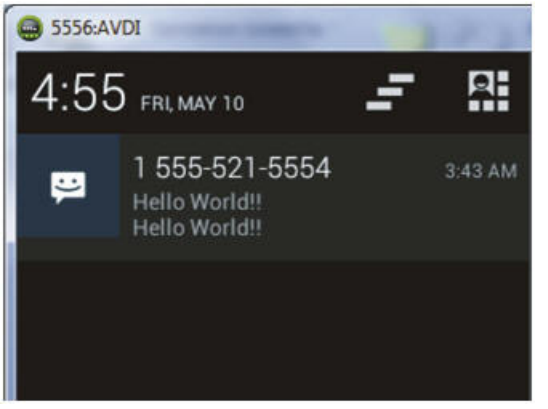


Figure 12.4: Output in the Second Emulator

In this application the message which has been written in the code is delivered if and only if the developer specifies the exact emulator number to which the message is being sent. If the application is tried in the AVD emulator then the SMS size is very restricted and it should not be more than 160 characters.

12.4.2 Receiving Incoming SMS Messages

Apart from sending SMS messages from within your Android applications, the developer can also receive incoming SMS messages from within the application using BroadCastReceiver object. This is of help when the application wants to perform an action on receipt of SMS.

To receive an SMS, perform the following steps:

1. Create a project named SMSReceiverApp in Android Studio.
2. Navigate to res → layout folder.
3. Modify the code in activity_main.xml file as shown in Code Snippet 9.

Code Snippet 9:

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/ res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical" >
<Button
android:id="@+id/buttonSendSMS"
android:layout_width="fill_parent"
```

```
android:layout_height="wrap_content"
android:text="Receive SMS" />
</LinearLayout>
```

4. Navigate to project's AndroidManifest.xml file and add the code under <uses-sdk> tag as shown in Code Snippet 10.

Code Snippet 10:

```
<uses-permission android:name="android.permission.SEND_SMS" >
</uses-permission>
<uses-permission android:name="android.permission.RECEIVE_SMS" >
</uses-permission>
```

5. Add the following code before the </application> tag as shown in Code Snippet 11.

Code Snippet 11:

```
<receiver android:name=".SMSReceiver" >
<intent-filter>
<action android:name="android.provider.Telephony.SMS_RECEIVED"
/>
</intent-filter>
</receiver>
```

6. Navigate to src → com.example.smsreceiverapp folder.
7. Modify the code in MainActivity.java file as shown in Code Snippet 12.

Code Snippet 12:

```
package com.example.smsreceiverapp;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.view.Menu;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.telephony.SmsManager;
```

```
public class MainActivity extends Activity {
Button button;
@Override
protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_main);
      button = (Button) findViewById(R.id.buttonSendSMS);
            button.setOnClickListener(new View.
                                    OnClickListener() {
                  public void onClick(View v) {
// Here write the emulator's number to which your sending the
                  // SMS
                  sendSMS("5556", "Hello World!!");
            }
private void sendSMS(String phoneNumber, String message) {
            SmsManager smsmanager = SmsManager.
                                    getDefault();
            smsmanager.sendTextMessage(phoneNumber, null,
                                    message, null, null);
            }
       });
}
@Override
public boolean onCreateOptionsMenu(Menu menu) {
      // Inflate the menu; this adds items to the action bar if it
is present.
      getMenuInflater().inflate(R.menu.main, menu);
      return true;
}
}
```

8. Create a new class named SMSReceiver in src → com.example.smsreceiverapp folder.

9. Add the code as shown in Code Snippet 13.

Code Snippet 13:

```
package com.example.smsreceiverapp;
import android.os.Bundle;
import android.app.Activity;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.telephony.SmsMessage;
import android.view.Menu;
import android.widget.Toast;
public class SMSReceiver extends BroadcastReceiver {
@Override
public void onReceive(Context context, Intent intent) {
      Bundle bundle = intent.getExtras();// --get the SMS message
passed in
            SmsMessage[] msgs = null;
            String str = " ";
            if (bundle != null) {
                  // --retrieve the SMS message received---
                  Object[] obj = (Object[]) bundle.
                                                get("obj");
                  msgs = new SmsMessage[obj.length];
                  for (int i = 0; i < msgs.length; i++) {
                  msgs[i] = SmsMessage.
                          createFromPdu((byte[]) obj[i]);
                  str += "SMS from " + msgs[i].
                                    getOriginatingAddress();
                  str += ":";
                  str += msgs[i].getMessageBody(.
                                        toString();
```

```
            str += "\n";
              }
                    // ---display the new SMS message
                    Toast.makeText(context, str, Toast.LENGTH_
                                                    SHORT).show();
              }
}
}
```

10. Navigate to Run menu and select Run 'app'.
11. With the emulator running, Navigate to Tools → Android and select Android Device Monitor.
12. Select the Emulator Control tab from the right pane.
13. Select SMS.
14. Type the number, 12980097 in the Incoming number box. It indicates the DDMS number.
15. Type the message "Hello SMS Receiver" in the Message box.
16. Click Send as shown in figure 12.5.

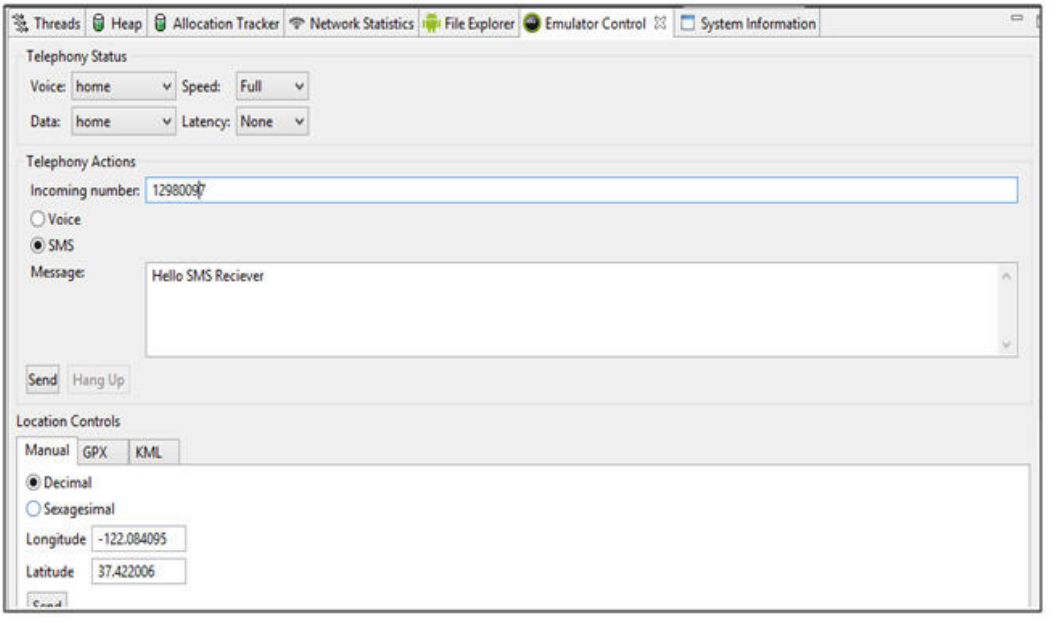


Figure 12.5: Setting the SMS Message

Start the application installed on the AVD. After executing the application the output will be as shown in figure 12.6.

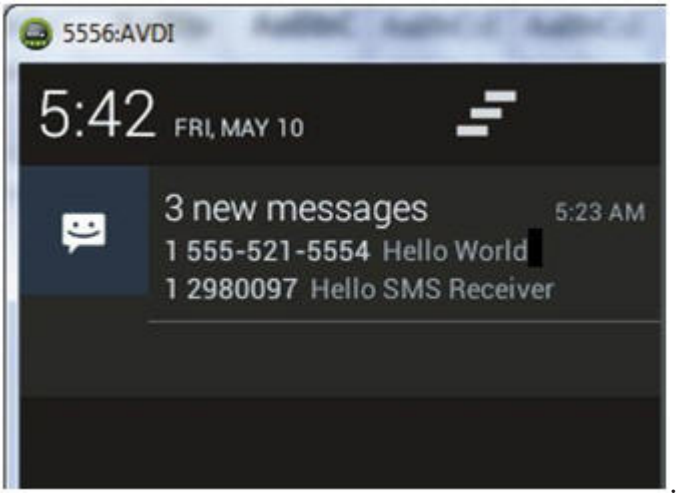


Figure 12.6: SMS Received

12.4.3 Sending SMS and Tracking It

There are several ways to track an SMS. Some of these are as follows:

- For tracking SMS, use broadcast receiver which checks the status of the message. You have to develop your own BroadcastReceiver and pass the string using an object of the PendingIntent class.
- Also, ContentObserver listening on the content://sms to track SMS. This class will invoke the onChange() method when the SMS database changed.

12.5 SIP and VoIP

Not all Android devices have modem hardware. However, they may still have all the other requirements to transmit voice and data. This section describes the alternate means of telephony supported by Android using network connections.

12.5.1 VoIP

VoIP is the abbreviation for Voice over Internet Protocol. With the costs of high speed data bandwidth reducing, transfer of voice over network has proven to be cheaper than the traditional telephone networks. In fact, most providers have a free service for VoIP as long as you have a data connection to their servers. Services such as Skype and Google Hangouts have grown to support millions of users.

Most of these services have mobile clients to provide functionality similar to the telephone application in Android at a reduced rate or free of cost. These clients, in most cases, utilize the SIP protocol for signalling and data transfer. The next sections explain the process of utilizing SIP support API provided by Android to develop VoIP applications.

12.5.2 SIP

SIP is the abbreviation for Session Initiation Protocol. SIP uses message based stateless protocol similar to HTTP and SMTP for session control. Streaming protocols such as Real-time Streaming Protocol (RTP) and Secure Real-time Streaming Protocol (SRTP) are used for data transfer such as voice and video. On a network level, SIP can utilize TCP, UDP, or SCTP making it the very robust and easy to deploy in any environment.

12.5.3 SIP Architecture

This section describes the underlying architecture for SIP model. The model is similar to the SMTP mail transfer architecture.

- **SIP User Agent Client**

  UAC is the client application which creates SIP messages and communicates with the SIP server end. From a developer's perspective, the developed application using the Android's SIP API will serve this purpose.

- **SIP User Agent Server (UAS)**

  In the event that a Proxy Server or a Redirect Server or the Registrar is not directly reachable, the message is delivered to the SIP UAS. The SIP UAS receives the SIP request and generates a SIP response.

- **Proxy Servers**

  Proxy Servers are responsible for receiving incoming messages and then re-transmitting them to the intended destination. They can act as both clients and servers. In addition to routing the messages, proxy servers can also be used for enforcing security and restricting access.

- **Redirection Servers**

  It is an optional component of the SIP architecture. The redirection server is responsible for handling messages which generate a direct request such as 301 (Permanently moved), 307 (Temporary redirect), or 308 (Permanent Redirect) messages.

- **Gateways**

  Gateways are used to interconnect networks. They function similar to network gateways with the purpose to route messages from one network to another and to translate protocols.

- **Registrar**

  A registrar is the central management server in the SIP protocol. It is responsible for responding to SIP requests, registering the users, managing users, and establishing sessions. Every SIP client needs to register with the registrar for it to be discoverable by the remaining peers.

12.5.4 SIP Requests

SIP uses commands to generate requests. SIP requests are listed in table 12.1.

Request	Description
INVITE	Indicates a client is being invited to participate in a call session.
ACK	Confirms that the client has received a final response to an INVITE request.
BYE	Terminates a call and can be sent by either the caller or the callee.
CANCEL	Cancels any pending request.
OPTIONS	Queries the capabilities of servers.
REGISTER	Registers the address listed in the To header field with a SIP server.
PRACK	Provisional acknowledgement.
SUBSCRIBE	Subscribes for an Event of Notification from the Notifier.
NOTIFY	Notify the subscriber of a new Event.
PUBLISH	Publishes an event to the Server.
INFO	Sends mid-session information that does not modify the session state.
REFER	Asks recipient to issue SIP request (call transfer).
MESSAGE	Transports instant messages using SIP.
UPDATE	Modifies the state of a session without changing the state of the dialog.

Table 12.1: SIP Requests

12.5.5 SIP Responses

SIP uses numeric response codes as replies similar to the response codes used in HTTP. The code families are outlined in table 12.2.

Response	Description
1xx	Status Response codes.
2xx	Success Response code.
3xx	Redirection Response code.
4xx	Client Side Error.
5xx	Server Side Error.
6xx	Protocol not supported.

Table 12.2: SIP Response Codes

12.5.6 SIP Permissions

In order to utilize the SIP framework, permissions specified in Code Snippet 14 need to be used.

Code Snippet 14:

```
<uses-permission android:name="android.permission.USE_ SIP" />
<uses-permission android:name="android.permission.INTERNET" />
```

The INTERNET permission is mandatory as SIP cannot function without a network connection.

12.5.7 SIP Framework in Android

Android framework provides API which handles session establishment and audio/video data. This section explains the procedure to connect to a SIP server and send messages. The package android.net.sip holds the classes for the SIP API.

Relevant Classes

- **SipAudioCall**: This class is used to handle a voice call.
- **SipSession**: This class is used to handle a SIP session.
- **SipErrorCode**: This class is used to contain the error data.
- **SipProfile**: This class holds connection information such as the server, user name,and password for the account.
- **SipManager**: This class is responsible for establishing connection and provides access to SIP services.

Support Classes

- **SipAudioCall.Listener**: The listener class for SipAudioCall. The methods for handling incoming and outgoing call events can be handled here. These events are also received by the SipSession.Listener class.
- **SipSession.Listener**: The listener class for an SIP session.
- **SipProfile.Builder**: The builder class for creating SipProfile objects.
- **SipSession.State**: An encapsulation class holding integer values for call states.

12.5.8 Connecting to an SIP Server

This section explains creation of an application that connects to an SIP server.

Prerequisites:

- A device running Android Lollipop with a data connection. The SIP application needs to be deployed on a real device as AVD does not allow network connections.
- A registered SIP account for use with the application.
- Access to a SIP server.

Steps:

1. Create a new application called SIPApplication in Android Studio.
2. Navigate to res → layout and open activity_main.xml for editing.
3. Modify the code as shown in Code Snippet 15.

Code Snippet 15:

```
<LinearLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools"
  android:layout_width="fill_parent"
  android:layout_height="match_parent"
  android:paddingLeft="@dimen/activity_horizontal_margin"
   android:paddingTop="@dimen/activity_vertical_margin"
   android:paddingBottom="@dimen/activity_vertical_margin"
                         tools:context=".MainActivity"
   android:orientation="vertical">
    <LinearLayout
        android:orientation="horizontal"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_below="@+id/textView"
        android:layout_alignEnd="@+id/textView"
        android:layout_weight="0">
```

```
        <TextView
            <TextView android:text="Status : "
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:id="@+id/textView"
            android:textSize="20dp" />
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Not Connected"
            android:id="@+id/statusView"
            android:textSize="20dp" />
    </LinearLayout>
    <LinearLayout
        android:orientation="horizontal"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_weight="0"
        android:id="@+id/">
       <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="SIP Server"
            android:id="@+id/textView3" />
        <EditText
            android:layout_width="fill_parent"
            android:layout_height="wrap_content"
            android:id="@+id/editText"
            android:layout_weight="1" />
```

```
</LinearLayout>
 <LinearLayout
     android:orientation=”horizontal”
     android:layout_width=”match_parent”
     android:layout_height=”wrap_content”
     android:layout_gravity=”center_horizontal”
     android:layout_weight=”0”>
     <TextView
         android:layout_width=”wrap_content”
         android:layout_height=”wrap_content”
         android:text=”User Name”
         android:id=”@+id/textView4” />
     <EditText
         android:layout_width=”fill_parent”
         android:layout_height=”wrap_content”
         android:id=”@+id/editText2”
         android:layout_weight=”0” />
 </LinearLayout>
 <LinearLayout
     android:orientation=”horizontal”
     android:layout_width=”match_parent”
     android:layout_height=”wrap_content”
     android:layout_weight=”0”>
     <TextView
         android:layout_width=”wrap_content”
         android:layout_height=”wrap_content”
         android:text=”Password”
         android:id=”@+id/textView5”
         android:layout_weight=”0” />
```

```
        <EditText
            android:layout_width=”match_parent”
            android:layout_height=”wrap_content”
            android:inputType=”textPassword”
            android:ems=”10”
            android:id=”@+id/editText3”
            android:layout_weight=”0” />
    </LinearLayout>
    <Button
        android:layout_width=”fill_parent”
        android:layout_height=”wrap_content”
        android:text=”Login”
        android:id=”@+id/button”
        android:layout_gravity=”center_horizontal”
        android:onClick=”onConnectClick”
        android:layout_weight=”0” />
</LinearLayout>
```

4. Navigate to app → java → aptech.com.sipapplication and open MainActivity.java for editing.
5. Modify the code as shown in Code Snippet 16.

Code Snippet 16:

```
package aptech.com.sipapplication;
import android.app.Activity;
import android.app.PendingIntent;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.content.IntentFilter;
import android.net.sip.SipException;
import android.net.sip.SipManager;
```

```
import android.net.sip.SipProfile;
import android.net.sip.SipRegistrationListener;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.EditText;
import android.widget.TextView;
import java.text.ParseException;
public class MainActivity extends Activity {
    private SipManager sipManager;
    private SipProfile profile;
    SipRegistrationListener listener;
    IntentFilter filter;
    BroadcastReceiver receiver;
    @Override
    protected void onCreate(Bundle savedInstanceState){
         super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        sipManager = SipManager.newInstance(this);
        listener = new SipRegistrationListener() {
       public void onRegistering(String localProfileUri) {
                setStatus("Connecting");
            }
            public void onRegistrationDone(String
                           localProfileUri, long expiryTime) {
                setStatus("Connected");
            }
```

```
            public void onRegistrationFailed(StringlocalProfileUri,
                                    int errorCode, String errorMessage) {
                    setStatus("Registration failed. Check
                                                credentials.");
                }
            };
            receiver = new BroadcastReceiver() {
                @Override
         public void onReceive(Context context, Intent intent) {
                                        setStatus(" Incoming Call ");
                }
            };
            filter = new IntentFilter();
            filter.addAction("sipApp.intent.IncomingCall");

            this.registerReceiver(receiver, filter);
        }
@Override
    public boolean onCreateOptionsMenu(Menu menu) {
   // Inflate the menu; this adds items to the action bar if //it
is present.
            getMenuInflater().inflate(R.menu.menu_main, menu);
            return true;
        }
        @Override
        public boolean onOptionsItemSelected(MenuItem item) {
  // Handle action bar item clicks here. The action bar will
// automatically handle clicks on the Home/Up button, so long
 // as you specify a parent activity in AndroidManifest.xml.
```

```
        int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
    public void onConnectClick(View view) {
        String domain = ((EditText)findViewById(R.id.editText)).
                        getText().toString();
        String userName = ((EditText)findViewById(R.
                          id.editText2)).getText().toString();
        String passWord = ((EditText)findViewById(R.
                          id.editText3)).getText().toString();
        connectToServer(domain,userName,passWord);
    }
private void connectToServer(String domain,String userName,
String
                             passWord) {
        SipProfile.Builder builder = null;
        try {
          builder = new SipProfile.Builder(userName,
                                           domain);
            builder.setPassword(passWord);
            builder.setAutoRegistration(true);
            profile = builder.build();

       // If AutoRegistration is set to false, uncomment the
       //below line of code
       //sipManager.register(profile,9999,listener);
```

```
            Intent intent = new Intent();
            intent.setAction("sipApp.intent.    IncomingCall");
            PendingIntent pendingIntent = PendingIntent.
getBroadcast(this, 0, intent, Intent.FILL_IN_DATA);
       // If AutoRegistration is set to true, then register is
      //automatically called when open() is called
         sipManager.open(profile,pendingIntent,listener);

        } catch (ParseException e) {
            e.printStackTrace();
        } catch (SipException e) {
            e.printStackTrace();
        }
    }
       private void setStatus(String status){
        TextView statusView = (TextView)findViewById(R.
                              id.statusView);
        statusView.setText(status);
    }
}
```

Once the user clicks the Login button, the server address and the credentials are retrieved. They are then used to establish a connection using the SipManager class. Once a connection is established a Broadcast Receiver is registered for an Incoming call. The receiver updates the status whenever there is an incoming VoIP call to the logged in user.

6. Navigate to Manifests and open AndroidManifest.xml for editing.

7. Modify the code as shown in Code Snippet 17.

Code Snippet 17:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
xmlns:android="http://schemas.android.com/apk/res/android"
  package="aptech.com.sipapplication" >
<uses-permission android:name="android.permission.USE_SIP" />
<uses-permission android:name="android.permission.INTERNET" />
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <activity
            android:name=".MainActivity"
            android:label="@string/app_name" >
            <intent-filter>
                <action android:name="android.intent.
                                      action.MAIN" />
                <category
                android:name="android.intent.category.
                                          LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

8. Navigate to res → values and open styles.xml for editing.

9. Modify the code as shown in Code Snippet 18.

Code Snippet 18:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="AppTheme" parent="android:Theme.
                                        Material">
    </style>
</resources>
```

10. Connect the mobile device through USB cable.
11. Enable USB to connect to storage/device.
12. Install the adb drivers for the device.
13. Navigate to Run menu and select Run 'app'.
14. From the list of Connected Devices, select the device to install the apk on.
15. Click OK.

The SIP Application is deployed on the device as show in figure 12.7.

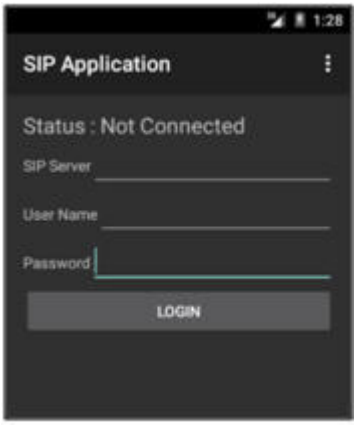


Figure 12.7: SIP Application Output

Once the connect button is clicked, the application registers with the SIP registrar the status is changed to Connected as shown in figure 12.8.

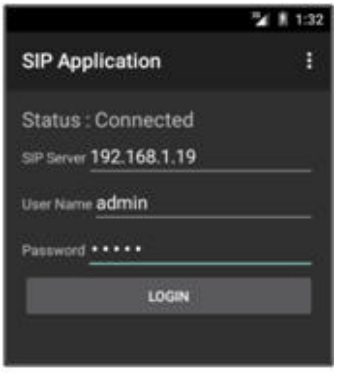


Figure 12.8: Connected to a SIP Server

If the account receives an incoming call, the status is set to Incoming Call as shown in figure 12.9.

Figure 12.9: Incoming Call

Note - The application needs to be run on a real device running Lollipop or newer with a data connection using Wi-Fi or a mobile data connection with good bandwidth. Account credentials of a SIP server are also necessary.

Note - For the purpose of this example a custom server running OpenSIPS has been setup. The reader can either do the same using OpenSIPS, Kamilio, or Asterik. Alternatively, the reader can register with a free SIP provider.

12.5.9 Alternatives to SIP

Alternatives to SIP protocols are available such as Extensible Messaging and Presence Protocol (XMPP) and Standard Interface for Multiple Platform Link Evaluation (SIMPLE). XMPP is an xml based instant messaging protocol that also supports voice and video data. SIMPLE is an emerging standard closely based on the SIP standard. Neither of these two protocols is natively supported by Android. However, third party libraries are available to make the process of development much easier. XMPP, in particular, has gained a huge market share in the desktop market and continues to rise in usage.

Check Your Progress

1. Which of the following permissions are needed for a VoIP application?

1.	<uses-permission android:name="android.permission.READ_PHONE_STATE"/>
2.	<uses-permission android:name="android.permission.USE_SIP" />
3.	<uses-permission android:name="android.permission.INTERNET" />
4.	<uses-permission android:name="android.permission .SEND_SMS" />

(A)	1, 3	(B)	2, 3
(C)	3, 4	(D)	1, 4

2. Which of the following statements are true?

1.	RIL library is a part of Kernel Layer.
2.	AT commands are also called Hayes commands.
3.	All devices have modems.
4.	Permissions are needed to use the telephony API.

(A)	1, 3	(B)	1, 2
(C)	2, 4	(D)	2, 3

3. Which of the following method retrieves an instance of SmsManager?

(A)	SmsManager.getDefault()	(B)	Context.getSmsManager()
(C)	getSmsManager()	(D)	Context.getSystemService(SMS_MANAGER)

4. The telephony API is also used for other activities such as monitor phone information including the current states of the phone, connections, and _____ information.

(A)	Battery	(B)	Storage
(C)	Network	(D)	Connectivity

Check Your Progress

5. Match the following permissions with their respective purpose.

	Permission		Purpose
a.	android.permission.READ_PHONE_STATE	1.	Notification of incoming SMS
b.	Android.permission.USE_SIP	2.	Permission to send SMS
c.	Android.permission.SEND_SMS	3.	Phone status information
d.	android.permission.RECEIVE_SMS	4.	To use SIP API

(A)	a-4, b-1, c-2, d-3	(B)	a-3, b-1, c-4, d-2
(C)	a-3, b-4, c-2, d-1	(D)	a-1, b-3, c-4, d-2

Answers

1.	B
2.	C
3.	A
4.	C
5.	C

Summary

- Telephony provides access to information about the telephony services on the devices.
- The Android TelephonyManager provides information about the Android telephony system. In other words, it provides information about the available telephony services and states on the device.
- SMS stands for Short Messaging Service. SMS messaging is one of the most important and frequently used applications that is executed on the mobile devices.
- The SmsManager manages the SMS operations, such as sending text and data messages. Similarly, ConnectivityManager manages the Internet connectivity in a mobile phone.
- Apart from sending SMS messages from Android applications, the developer can also receive incoming SMS messages from within the application using BroadCastReceiver object.
- VoIP is the process of utilizing network access to provide telephony services such as voice and video.
- SIP is the protocol natively supported by Android for VoIP and similar applications.

LEARN BETTER @ Onlinevarsity

13

Sensors

Welcome to the Session, **Sensors.**

This session explains various types of Sensors supported by Android and describes the API(s) to utilize them. It also describes how the information from sensors can be used to make context aware services.

In this Session, you will learn to:

- Explain various types of Sensors
- Use Motion Sensors
- Use Position Sensors
- Use Status Sensors
- Explain Context aware services based on sensor information

Session

13

Sensors

13.1 Introduction

Sensors are hardware devices that record and measure physical data. Sensors in Android play a key role in the core operating system as well as the platform as a whole. Android Compatibility Definition Document (CDD) specifies that the vendors implement basic sensors and any additional sensors can be implemented at the discretion of the vendor. Sensors in Android can be broadly classified into following three types:

- Motion Sensors
- Environmental Sensors
- Position Sensors

13.2 Sensor Framework

Android Sensor Framework is responsible for translating the input received from the hardware devices to readings usable by the applications. Raw data received from the sensors can also be accessed using the Sensor Framework.

Not all sensors in Android are hardware devices. Certain sensors can be implemented by software emulation using data received from other sensors. In this section the basics of utilizing the sensor framework will be explained.

13.2.1 Relevant Classes

- **SensorManager**

  This class is the central component of the sensor framework. It is responsible for retrieving sensor instances and setting event listeners for the sensor. It is also responsible for identifying the sensor support on the device. The process of getting a reference to the SensorManager instance is shown in Code Snippet 1.

Code Snippet 1:

```
SensorManager
sManager=(SensorManager)getSystemService(Context.SENSOR_SERVICE);
```

- **Sensor**

  This class is used to reference a specific sensor on the device. An instance of a sensor class can be retrieved by using the getDefaultSensor(int type) or the getSensorList(int type) method of SensorManager.

- **SensorEvent**

  This class stores the event information whenever there is a change in status of the sensor.

- **SensorEventListener**

  This class provides an interface to implement event listeners for the sensors. The methods onSensorChanged() and onAccuracyChanged() need to be implemented.

13.2.2 Working with Sensor Data

In order to retrieve and utilize sensor data, the following steps need to be performed. These steps are common for all types of sensors.

1. Obtain a reference to the SensorManager instance.
2. Ensure that the sensor type is supported by the device.
3. Get a reference to the specific type of the sensor.
4. Create an event listener for the sensor(s) by implementing the SensorEventListener interface.
5. Implement the onSensorChanged() method to handle changes in sensor data.
6. Implement the onAccuracyChanged() method to handle accuracy changes.
7. Register the listener with the sensor manager for the type of the sensor.
8. Implement the onPause() method of the activity and unregister the listener.
9. Implement the onResume() method of the activity to re-register the listener. This saves CPU cycles that are wasted unnecessarily in processing the sensor changes.

An example code to achieve this is shown in Code Snippet 2.

Code Snippet 2:

```
public class MainActivity extends Activity {

    SensorManager;

    private SensorEventListener listener = new
                                          SensorEventListener() {

       @Override

        public void onSensorChanged(SensorEvent event) {

            int value1 = event.values[0];

            int value2 = event.values[1];

            int value3 = event.values[2];

            }

       }
```

```
     @Override
       public void onAccuracyChanged(Sensor sensor, int accuracy) {
            // Code here
        }
    };
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        sensorManager = (SensorManager)getSystemService(SENSOR_
                                                        SERVICE);
        List<Sensor> sensorList = sensorManager.
                                  getSensorList(Sensor.TYPE_ALL);
       if(sensorList.size() =0 ){
          //No sensor support
        }
        Sensor = sensorManager.getDefaultSensor(Sensor.TYPE_
                                                PROXIMITY);
        sensorManager.registerListener(listener,SensorManager.
                                       SENSOR_DELAY_NORMAL);
    }
    @Override
    protected void onResume() {
        super.onResume();
sensorManager.registerListener(listener,sensorManager
getDefaultSensor(Sensor.TYPE_PROXIMITY),SensorManager.SENSOR_
                                        DELAY_NORMAL);
    }
   @Override
```

```
    protected void onPause() {
        super.onPause();
        sensorManager.unregisterListener(listener);
    }
}
```

13.2.3 Event Data

The onSensorChanged(SensorEvent event) and onAccuracyChanged(SensorEvent event) methods are passed an event object reference. The event float matrix holds the sensor reading values. The values represent the following data:

- For most sensors, the values of the indices 0, 1, and 2 measure the values along the X, Y, and Z axis respectively.
- Uncalibrated sensors have three additional fields in the indices 3, 4, and 5 which measure the drift values along the X, Y, and Z axis respectively.
- Environment sensors have only one value in the index 0, which measures the reading of the sensor.

Note - All the examples in this session need to be run on a real device running Android 5.0 Lollipop or higher. The emulator will not be sufficient for this.

13.3 Motion Sensors

Motion Sensors help monitor the movement of the device. Some motion sensors are hardware only and the remaining may be implemented in software by using input from the hardware sensors. Motion sensors are primarily used in games and gimmicky multimedia applications.

13.3.1 Android Motion Sensors

The types of motion sensors supported by Android along with their implementation option and utility are shown in table 13.1.

Sensor Type	Implementation	Description
TYPE_ACCELEROMETER	Hardware	Accelerometer measures the acceleration or movement of the device in a direction.

Sensor Type	Implementation	Description
TYPE_GRAVITY	Hardware or Software	Gravity Sensor measure the gravitation force experienced by the device.
TYPE_GYROSCOPE	Hardware	Gyroscope measures the rotational force of the device in a direction.
TYPE_GYROSCOPE_ UNCALIBRATED	Software	The software sensor provides raw uncalibrated data from the Gyroscope.
TYPE_LINEAR_ ACCELERATION	Hardware or Software	Linear Acceleration scope provides the acceleration force experienced by the device without the Gravitation force.
TYPE_ROTATION_VECTOR	Hardware or Software	Provides the angle and inclination of the device.
TYPE_STEP_COUNTER	Hardware or Software	Calculates the number of steps measured by the step sensor since time of activation.

Table 13.1: Motion Sensors

Note - The TYPE_STEP_COUNTER sensor has only one field at index 0 which returns the step count.

13.3.2 Measuring Sensor Data

In this section the process of writing an application for understanding and measuring sensor data will be explained. The steps are as follows:

1. Create a new project called MotionSensors in Android Studio.
2. Right-click app, navigate to app → New Activity, and select Blank Activity.
3. Create a new Activity called SensorDetails.
4. Navigate to res → layout and open acitivty_main.xml for editing.

5. Modify the code as shown in Code Snippet 3.

Code Snippet 3:

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
                          tools:context=".MainActivity"
    android:orientation="vertical">
    <ListView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:id="@+id/listView" />
</LinearLayout>
```

6. Open acitivity_sensor_details.xml and modify the code as shown in Code Snippet 4.

Code Snippet 4:

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
```

```
    tools:context="aptech.com.motionsensors.SensorDetails"
    android:orientation="vertical">
    <LinearLayout
        android:orientation="horizontal"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Sensor readings for  : "
            android:id="@+id/textView"
            android:layout_weight="1" />
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Sensor"
            android:id="@+id/sensorName" />
    </LinearLayout>
   <LinearLayout
        android:orientation="horizontal"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" >
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="The measured value is"
            android:id="@+id/textView7"
            android:layout_weight="1" />
```

```
        <TextView
             android:layout_width=”wrap_content”
             android:layout_height=”wrap_content”
             android:text=”acceleration”
             android:id=”@+id/valueType” />
    </LinearLayout>
    <LinearLayout
        android:orientation=”horizontal”
        android:layout_width=”match_parent”
        android:layout_height=”wrap_content” >
        <TextView
             android:layout_width=”wrap_content”
             android:layout_height=”wrap_content”
             android:text=”The units of measurement are : “
             android:id=”@+id/textView2”
             android:layout_weight=”1” />
        <TextView
             android:layout_width=”wrap_content”
             android:layout_height=”wrap_content”
             android:text=”m/s”
             android:id=”@+id/units” />
    </LinearLayout>
    <LinearLayout
        android:orientation=”vertical”
        android:layout_width=”match_parent”
        android:layout_height=”30dp”>
   </LinearLayout>
   <LinearLayout
```

```
        android:orientation=”horizontal”
        android:layout_width=”match_parent”
        android:layout_height=”wrap_content”>
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”X - Axis”
            android:id=”@+id/textView3”
            android:layout_weight=”1”
            android:textAlignment=”center” />
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Y-Axis”
            android:id=”@+id/textView4”
            android:layout_weight=”1”
            android:textAlignment=”center” />
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Z-Axis”
            android:id=”@+id/textView5”
            android:layout_weight=”1”
            android:textAlignment=”center” />
    </LinearLayout>
    <LinearLayout
        android:orientation=”horizontal”
        android:layout_width=”match_parent”
```

```
        android:layout_height=”wrap_content” >
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”0”
            android:id=”@+id/xValue”
            android:layout_weight=”1”
            android:textAlignment=”center” />
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”0”
            android:id=”@+id/yValue”
            android:layout_weight=”1”
            android:textAlignment=”center” />
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”0”
            android:id=”@+id/zValue”
            android:layout_weight=”1”
            android:textAlignment=”center” />
    </LinearLayout>
    <LinearLayout
        android:orientation=”vertical”
        android:layout_width=”match_parent”
        android:layout_height=”30dp” />
    <LinearLayout
```

```
        android:orientation="horizontal"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" >
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Scalar Component Value"
            android:id="@+id/scalarLabel"
            android:layout_weight="1" />
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="0"
            android:id="@+id/scalarValue" />
    </LinearLayout>
</LinearLayout>
```

7. Navigate to app → java → aptech.com.motionsensors and open MainActivity.java for editing.
8. Modify the code as shown in Code Snippet 5.

Code Snippet 5:

```
package aptech.com.motionsensors;
import android.app.Activity;
import android.content.Context;
import android.content.Intent;
import android.hardware.Sensor;
import android.hardware.SensorManager;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
```

```
import android.widget.AdapterView;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import android.widget.Toast;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
public class MainActivity extends Activity {
    private SensorManager;
    List<String> sensorNameList;
    HashMap<Integer, Integer> typesList;
    int motionSensorTypes[] = {
           Sensor.TYPE_ACCELEROMETER,
            Sensor.TYPE_GRAVITY,
            Sensor.TYPE_GYROSCOPE,
            Sensor.TYPE_GYROSCOPE_UNCALIBRATED,
            Sensor.TYPE_LINEAR_ACCELERATION,
            Sensor.TYPE_ROTATION_VECTOR,
            Sensor.TYPE_STEP_COUNTER
    };
    private AdapterView.OnItemClickListener listViewListener =
                          new AdapterView.OnItemClickListener() {
        public void onItemClick(AdapterView<?> parent, View, int
                                position, long id) {
           Intent i = new Intent(MainActivity.this,
                                 SensorDetails.class);
            i.putExtra("type", typesList.get(position));
            startActivity(i);
        }
    };
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    sensorManager = (SensorManager) getSystemService
                    (Context.SENSOR_SERVICE);
    List<Sensor> sensorsList = sensorManager.getSensorList
                               (Sensor.TYPE_ALL);
    sensorNameList = new ArrayList<String>();
    typesList = new HashMap<Integer, Integer>();
    if (!(sensorsList.size() > 0)) {
        Toast.makeText(this, "No sensors supported", Toast.
                               LENGTH_LONG).show();
    } else {
        for (Sensor : sensorsList) {
            String sensorName = sensor.getName();
            int sensorType = sensor.getType();
            if (isMotionSensor(sensorType)) {
                typesList.put(sensorNameList.size(),
                                        sensorType);
                sensorNameList.add(sensorName);
            }
        }
    }
    ListView listView = (ListView) findViewById(R.id.
                                                listView);
    ArrayAdapter<String> adapter = new ArrayAdapter
              <String>(this, android.R.layout.simple_list_
                                    item_1, sensorNameList);
    listView.setAdapter(adapter);
```

```
        listView.setOnItemClickListener(listViewListener);
    }
    private boolean isMotionSensor(int sensorType) {
        boolean found = false;
        for (int i : motionSensorTypes) {
            if (i == sensorType) {
                found = true;
                break;
            }
        }
        return found;
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if
it is present.
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
// Handle action bar item clicks here. The action bar will
// automatically handle clicks on the Home/Up button, so long
  // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
       //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
```

```
        return super.onOptionsItemSelected(item);
    }
}
```

The code retrieves the list of all the sensors supported by the device. It then checks the sensors which are motion sensors and adds them to the sensor list. Once the user selects a sensor from this list, the sensor information is displayed.

9. Open the activity, SensorDetails.java and modify the code as shown in Code Snippet 6.

Code Snippet 6:

```
package aptech.com.motionsensors;
import android.app.Activity;
import android.content.Context;
import android.hardware.Sensor;
import android.hardware.SensorEvent;
import android.hardware.SensorEventListener;
import android.hardware.SensorManager;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.TextView;
import android.widget.Toast;
import java.util.HashMap;
public class SensorDetails extends Activity {
    private SensorManager sensorManager;
    private int sensorType;
    private Sensor sensor;
    private TextView sensorNameView;
    private TextView sensorTypeView;
    private TextView unitsView;
```

```
private TextView xValue;
private TextView yValue;
private TextView zValue;
private HashMap<Integer,String> namessMap;
private HashMap<Integer,String> typesMap;
private HashMap<Integer,String> unitsMap;
private SensorEventListener listener = new
SensorEventListener() {
    @Override
    public void onSensorChanged(SensorEvent event) {
        xValue.setText(Float.toString(event.values[0]));
        yValue.setText(Float.toString(event.values[1]));
        zValue.setText(Float.toString(event.values[2]));
        if(sensorType==Sensor.TYPE_ROTATION_VECTOR) {
            ((TextView) findViewById(R.id.scalarLabel)
                            .setVisibility(View.VISIBLE);
            ((TextView) findViewById(R.id.scalarValue))
                             setVisibility(View.VISIBLE);
            ((TextView) findViewById(R.id.scalarValue))
                    .setText(Float.toString(event.values[3]));
        }
    }
 @Override
  public void onAccuracyChanged(Sensor sensor, int accuracy) {
    }
 };
@Override
 protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
```

```
setContentView(R.layout.activity_sensor_details);
sensorManager = (SensorManager)
                getSystemService(Context.SENSOR_SERVICE);
sensorType = getIntent().getIntExtra("type", Sensor
                                    TYPE_ACCELEROMETER);
namessMap = new HashMap<Integer,String>();
typesMap = new HashMap<Integer,String>();
unitsMap = new HashMap<Integer,String>();
sensorNameView = (TextView)findViewById(R.id.sensorName);
sensorTypeView = (TextView)findViewById(R.id.valueType);
unitsView = (TextView)findViewById(R.id.units);
xValue = (TextView)findViewById(R.id.xValue);
yValue = (TextView)findViewById(R.id.yValue);
zValue = (TextView)findViewById(R.id.zValue);
((TextView)findViewById(R.id.scalarLabel))
                        .setVisibility(View.GONE);
((TextView)findViewById(R.id.scalarValue))
                      .setVisibility(View.GONE);
sensor = sensorManager.getDefaultSensor(sensorType);
if (sensor != null){
    initMaps();
    sensorNameView.setText(namessMap.get(sensorType));
    sensorTypeView.setText(typesMap.get(sensorType));
    unitsView.setText(unitsMap.get(sensorType));
    sensorManager.registerListener(listener, sensor,
                  SensorManager.SENSOR_DELAY_NORMAL);
}
else {
   Toast.makeText(this,"The Sensor type is not available
          on this device",Toast.LENGTH_LONG).show();
```

```
        }
    }
    private void initMaps() {
        namessMap.put(Sensor.TYPE_ACCELEROMETER,"Acceleromoter");
        namessMap.put(Sensor.TYPE_GRAVITY,"Gravity Sensor");
        namessMap.put(Sensor.TYPE_GYROSCOPE,"Gyroscope");
        namessMap.put(Sensor.TYPE_GYROSCOPE_UNCALIBRATED,
                                    "Gyroscope Uncalibrated");
        namessMap.put(Sensor.TYPE_LINEAR_ACCELERATION,"Linear
                                                Acceleration");
        namessMap.put(Sensor.TYPE_ROTATION_VECTOR,"Rotation
                                                 Vector");
        namessMap.put(Sensor.TYPE_STEP_COUNTER,"Step Counter");
        typesMap.put(Sensor.TYPE_ACCELEROMETER,"Acceleration
                                                  Force");
        typesMap.put(Sensor.TYPE_GRAVITY,"Gravitation Force");
        typesMap.put(Sensor.TYPE_GYROSCOPE,"Rate of Rotation");
        typesMap.put(Sensor.TYPE_GYROSCOPE_UNCALIBRATED,"Raw
                              data recieved from gyroscope");
        typesMap.put(Sensor.TYPE_LINEAR_ACCELERATION,"Acceleration
                          Force excluding Gravitational Force");
        typesMap.put(Sensor.TYPE_ROTATION_VECTOR,"Vector
                 representing the current angle of the device");
        typesMap.put(Sensor.TYPE_STEP_COUNTER,"Integer value
                           representing number of steps");
        unitsMap.put(Sensor.TYPE_ACCELEROMETER,"m/s2");
        unitsMap.put(Sensor.TYPE_GRAVITY,"m/s2");
        unitsMap.put(Sensor.TYPE_GYROSCOPE,"rad/s");
        unitsMap.put(Sensor.TYPE_GYROSCOPE_UNCALIBRATED,"rad/s");
        unitsMap.put(Sensor.TYPE_LINEAR_ACCELERATION,"m/s2");
        unitsMap.put(Sensor.TYPE_ROTATION_VECTOR,"Unitless");
```

```
        unitsMap.put(Sensor.TYPE_STEP_COUNTER,"Unitless");
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
    // Inflate the menu; this adds items to the action bar if it
is //present.
        getMenuInflater().inflate(R.menu.menu_sensor_details,
                                                        menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
   // Handle action bar item clicks here. The action bar will
   / automatically handle clicks on the Home/Up button, so long
   // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
   @Override
    protected void onResume() {
        super.onResume();
        sensorManager.registerListener(listener, sensor,
                            SensorManager.SENSOR_DELAY_NORMAL);
    }
   @Override
```

```
    protected void onPause() {
        super.onPause();
        sensorManager.unregisterListener(listener);
    }
}
```

The code registers a listener for the sensor type selected by the user. It then initializes the sensor related information such as name and units. Whenever there is a change in sensor information, the new values are updated in the respective TextViews.

10. Connect the mobile device through USB cable.
11. Enable USB to connect to storage/device.
12. Copy an image to the media directory inside the SD card with the name nfcimage.jpg.
13. Install the adb drivers for the device.
14. Navigate to Run menu and select Run 'app'.
15. From the list of Connected Devices, select the device to install the apk on.
16. Click OK.

The application is deployed on the device and the output is as shown in figure 13.1.

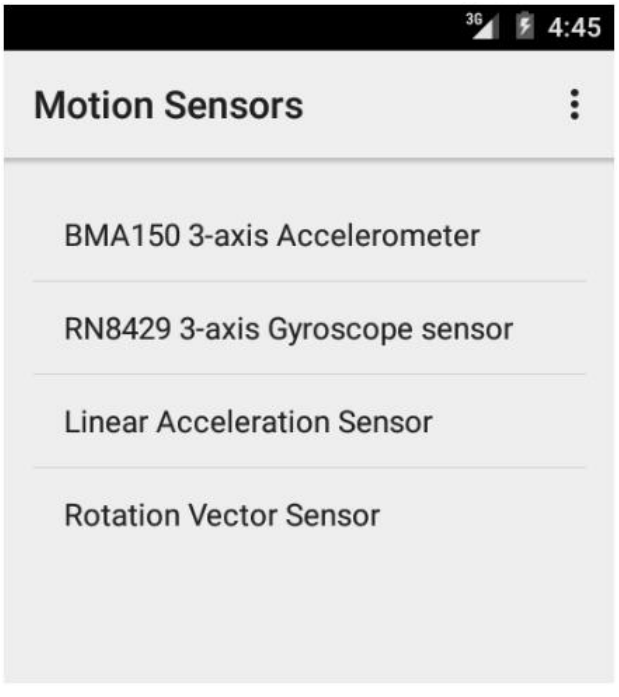


Figure 13.1: Motion Sensors Application

The list of Motion Sensors is displayed. Select a sensor, for example, BMA 150 3-axis Accelerometer and move the device in any direction to view the data as shown in figure 13.2.

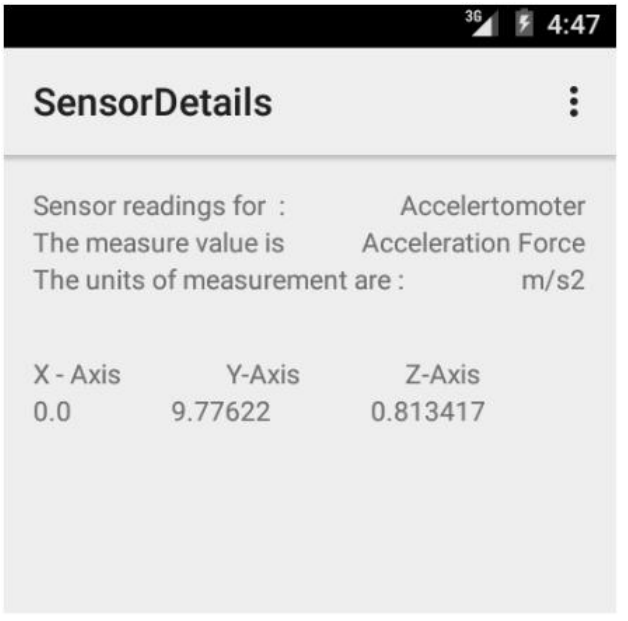


Figure 13.2: Sensor Details

13.3.3 Detecting a Shake Motion

A shake or jerk motion has been staple to mobile phone even before Android came into existence. A shake motion is used to signify a 'Next' gesture. This section explains the process of detecting and handling a Shake motion.

- **Reading Sensor Input**

  The main sensor that is responsible for detecting a shake motion is the accelerometer. The Linear Acceleration sensor is the ideal sensor for this purpose, but not all devices have this sensor. Hence, the accelerometer will be used as it is guaranteed to be supported on all devices.

  A shake motion in terms of sensor reading is characterized by a sudden increase in the acceleration experienced by the device in a short period of time. This is implemented programmatically as follows:

  - Create a SensorEventListener object for receiving changes in the accelerometer readings.
  - Save the initial time of measuring the acceleration.
  - Calculate the Root Mean Square (RMS) values of the initial acceleration of the device, that is, Square root of X2+Y2+Z2.
  - Store the initial acceleration value.

- Whenever new acceleration values are experienced, check the time difference from the time of the initial measurement and time of changed values.
- If the time difference is greater than a threshold (in this case, 300 milliseconds), then calculate the new RMS acceleration value.
- Calculate the change in acceleration using the RMS value.
- If the change in acceleration is greater than a threshold (in this case, 6), execute the code to handle a shake event. This implies a rapid change in acceleration in a short period time.

13.3.4 Implementation of Motion Sensors

This section demonstrates a real-world application of the input from motion sensors.

Revisiting the Quotes Application

For demonstrating the use of Motion Sensors in real life applications, create a Quotes application with support for Shake motion gesture.

1. Create a project named Widgets in Android Studio.
2. Navigate to res → layout and open activity_main.xml for editing.
3. Modify the code as shown in Code Snippet 7.

Code Snippet 7:

```
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
                                tools:context=".MainActivity">
   <TextView android:text="@string/hello_world" android:layout_
                                              width="wrap_content"
    android:layout_height="wrap_content"
    android:id="@+id/quoteView" />
```

```
    <Button
        android:layout_width=”wrap_content”
        android:layout_height=”wrap_content”
        android:text=”New Quote”
        android:id=”@+id/refresh”
        android:onClick=”refreshQuote”
        android:layout_below=”@+id/quoteView”
        android:layout_alignParentStart=”true” />
</RelativeLayout>
```

4. Create a new class called QuoteGenerator in the com.aptech.widgets package.

   This class contains a list of quotes and returns one at random when requested using the getQuote() method.

5. Modify the code as shown in Code Snippet 8.

Code Snippet 8:

```
package aptech.com.widgets;
import java.util.ArrayList;
import java.util.List;
import java.util.Random;
public class QuoteGenerator {
    private List<String> quotes;
    QuoteGenerator(){
        quotes = new ArrayList<String>();
        quotes.add(“Success is a Sum of small efforts day in
                                                and day out”);
        quotes.add(“The old believe everything”);
        quotes.add(“Slow and steady wins the race”);
        quotes.add(“Health is wealth”);
        quotes.add(“A friend in need is a friend indeed”);
        quotes.add(“Every cause produces more than one effect”);
```

```
        quotes.add("Practice makes a man perfect");
        quotes.add("Every story I create, creates a part of me");
        quotes.add("One touch of nature makes the world kin");
        quotes.add("Don't forget to love yourself");
        quotes.add("Knowledge is divine");
    }
    public String getQuote(){
        Random rand = new Random();
        int randomNum = rand.nextInt(11);
        return quotes.get(randomNum);
    }
}
```

6. Navigate to app → java → aptech.com.widgets and open MainActivity.java for editing.
7. Modify the code as shown in Code Snippet 9.

Code Snippet 9:

```
package aptech.com.shaketochange;
import android.app.Activity;
import android.hardware.Sensor;
import android.hardware.SensorEvent;
import android.hardware.SensorEventListener;
import android.hardware.SensorManager;
import android.os.Bundle;
import android.text.Html;
import android.view.Menu;
import android.view.MenuItem;
import android.widget.TextView;
public class MainActivity extends Activity {
   SensorManager sensorManager;
```

```
QuoteGenerator quoteGen;
float accelLast=0;
float accelCurrent=0;
float accelerationChange =0;
long lastTime=0;
private SensorEventListener shakeDetector = new
                                        SensorEventListener() {
    @Override
    public void onSensorChanged(SensorEvent event) {
        long now = System.currentTimeMillis();
        if(now - lastTime > 300) {
            lastTime = now;
            float x = event.values[0];
            float y = event.values[1];
            float z = event.values[2];
            accelLast = accelCurrent;
            accelCurrent = (float) Math.sqrt((double) (x * x
                                        + y * y + z * z));
            float delta = accelCurrent - accelLast;
      accelerationChange = accelerationChange * 0.9f + delta;
            if (accelerationChange > 6) {
                loadQuote();
            }
        }
    }
 @Override
  public void onAccuracyChanged(Sensor sensor, int accuracy) {
    }
};
```

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        quoteGen = new QuoteGenerator();
        loadQuote();
        sensorManager = (SensorManager)getSystemService(SENSOR_
                                                    SERVICE);
        sensorManager.registerListener(shakeDetector,
                sensorManager.getDefaultSensor(Sensor.TYPE_
                ACCELEROMETER),SensorManager.SENSOR_DELAY_NORMAL);
}
    private void loadQuote() {
        TextView quoteBox = (TextView)findViewById(R.id.textView);
        quoteBox.setText(Html.fromHtml("<b>" + quoteGen.
                                        getQuote() + "</b>"));
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar
        if it is present.
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
   @Override
   public boolean onOptionsItemSelected(MenuItem item) {
   // Handle action bar item clicks here. The action bar will
  // automatically handle clicks on the Home/Up button, so long
  / as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
```

```
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
    @Override
    protected void onResume() {
        super.onResume();
        sensorManager.registerListener(shakeDetector,
            sensorManager.getDefaultSensor(Sensor.TYPE_
              ACCELEROMETER),SensorManager.SENSOR_DELAY_NORMAL);
    }
    @Override
    protected void onPause() {
        super.onPause();
        sensorManager.unregisterListener(shakeDetector);
    }
}
```

The code detects a shake motion using the procedure explained earlier and loads a new quote each time a shake motion is detected.

8. Connect the mobile device through USB cable.
9. Enable USB to connect to storage/device.
10. Install the adb drivers for the device.
11. Navigate to Run menu and select Run 'app'.
12. From the list of Connected Devices, select the device to install the apk on.
13. Click OK.

The application is deployed on the device and the output is as shown in figure 13.3.

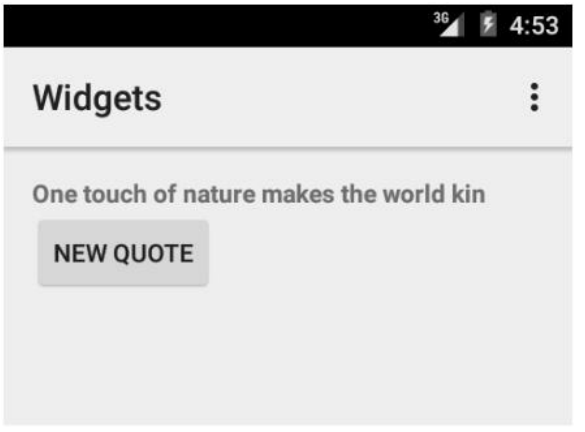


Figure 13.3: Widgets Application

By shaking the device, a new quote is displayed as shown in figure 13.4.

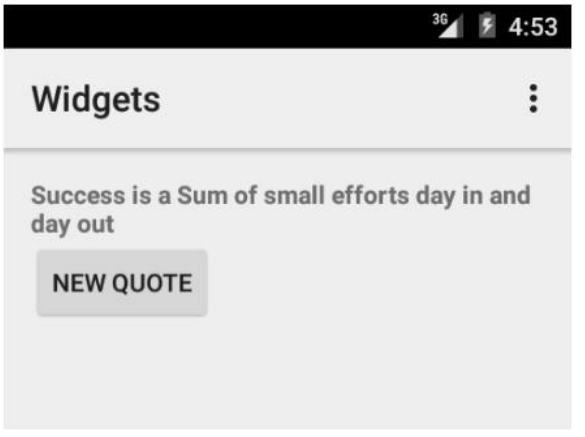


Figure 13.4: New Quote

Note - If the application detects a shake motion too frequently or too rarely, change the threshold value for the acceleration change in MainActivity.java (Initially 6).

13.4 Position Sensors

Position Sensors help monitor the position and orientation of the device with respect to its surroundings. The Geomagnetic Sensor and the Proximity Sensor are implemented in almost all Android Devices. In this section, the process to utilize the position sensors will be explained.

13.4.1 Android Position Sensors

The types of position sensors supported by Android along with their implementation option and utility are shown in table 13.2.

Sensor Type	Implementation	Description
TYPE_GAME_ROTATION_VECTOR	Software	A specialized rotation vector that is useful for developing games. It is the rotational vector value excluding the geomagnetic readings. This sensor is used to implement 'Steering Wheel' or 'Tilt' functionality in games.
TYPE_GEOMAGNETIC_ROTATION_VECTOR	Hardware or Software	It provides a rotation angle and inclination information of the device similar to the rotation vector. The key difference being that it uses the Geomagnetic Sensor instead of the Gyroscope.
TYPE_MAGNETIC_FIELD	Hardware or Software	Geomagnetic Sensor helps monitor changes in the Earth's magnetic field. The readings are reported in terms of field strength along the x, y, and z axis.
TYPE_MAGNETIC_FIELD_UNCALIBRATED	Software	Provides Raw uncalibrated data from the Geomagnetic sensor.
TYPE_PROXIMITY	Hardware or Software	The proximity sensor measures the distance of the device from a nearby object or the device user. This information can be in the form of exact distance measured in centimetres (cm) or just a binary value indicating far or near.

Table 13.2: Position Sensors

Note - The TYPE_ PROXIMITY sensor has only one field at index 0 which returns the distance in cm or a binary value representing far or near.

13.4.2 Measuring Data of Position Sensors

In this section the process of writing an application for understanding and measuring sensor data will be explained. The steps are as follows:

1. Create a new project called PositionSensors in Android Studio.
2. Right-click app, navigate to app → New Activity, and select Blank Activity.
3. Create a new Activity called SensorDetails.
4. Navigate to res → layout and open acitivty_main.xml for editing.
5. Modify the code as shown in Code Snippet 10.

Code Snippet 10:

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
                          tools:context=".MainActivity"
    android:orientation="vertical">
    <ListView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:id="@+id/listView" />
</LinearLayout>
```

6. Open acitivity_sensor_details.xml and modify the code as shown in Code Snippet 11.

Code Snippet 11:

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
```

```
    android:layout_width=”match_parent”
    android:layout_height=”match_parent”
    android:paddingLeft=”@dimen/activity_horizontal_margin”
    android:paddingRight=”@dimen/activity_horizontal_margin”
    android:paddingTop=”@dimen/activity_vertical_margin”
    android:paddingBottom=”@dimen/activity_vertical_margin”
    tools:context=”aptech.com.positionsensors.SensorDetails”
    android:orientation=”vertical”>
    <LinearLayout
        android:orientation=”horizontal”
        android:layout_width=”match_parent”
        android:layout_height=”wrap_content”>
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Sensor readings for  : “
            android:id=”@+id/textView”
            android:layout_weight=”1” />
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Sensor”
            android:id=”@+id/sensorName” />
    </LinearLayout>
    <LinearLayout
        android:orientation=”horizontal”
        android:layout_width=”match_parent”
        android:layout_height=”wrap_content” >
      <TextView
```

```
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”The measured value is”
            android:id=”@+id/textView7”
            android:layout_weight=”1” />
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”acceleration”
            android:id=”@+id/valueType” />
    </LinearLayout>
    <LinearLayout
        android:orientation=”horizontal”
        android:layout_width=”match_parent”
        android:layout_height=”wrap_content” >
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”The units of measurement are : “
            android:id=”@+id/textView2”
            android:layout_weight=”1” />
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”m/s”
            android:id=”@+id/units” />
    </LinearLayout>
    <LinearLayout
        android:orientation=”vertical”
```

```
        android:layout_width=”match_parent”
        android:layout_height=”30dp”>
    </LinearLayout>
    <LinearLayout
        android:orientation=”horizontal”
        android:layout_width=”match_parent”
        android:layout_height=”wrap_content”>
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”X - Axis”
            android:id=”@+id/textView3”
            android:layout_weight=”1”
            android:textAlignment=”center” />
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Y-Axis”
            android:id=”@+id/textView4”
            android:layout_weight=”1”
            android:textAlignment=”center” />
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”Z-Axis”
            android:id=”@+id/textView5”
            android:layout_weight=”1”
            android:textAlignment=”center” />
    </LinearLayout>
```

```
    <LinearLayout
        android:orientation=”horizontal”
        android:layout_width=”match_parent”
        android:layout_height=”wrap_content” >
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”0”
            android:id=”@+id/xValue”
            android:layout_weight=”1”
            android:textAlignment=”center” />
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”0”
            android:id=”@+id/yValue”
            android:layout_weight=”1”
            android:textAlignment=”center” />
        <TextView
            android:layout_width=”wrap_content”
            android:layout_height=”wrap_content”
            android:text=”0”
            android:id=”@+id/zValue”
            android:layout_weight=”1”
            android:textAlignment=”center” />
    </LinearLayout>
</LinearLayout>
```

7. Navigate to app → java → aptech.com.positionsensors and open MainActivity.java for editing.

8. Modify the code as shown in Code Snippet 12.

Code Snippet 12:

```
package aptech.com.positionsensors;
import android.app.Activity;
import android.content.Context;
import android.content.Intent;
import android.hardware.Sensor;
import android.hardware.SensorManager;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.AdapterView;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import android.widget.Toast;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
public class MainActivity extends Activity {
    private SensorManager sensorManager;
    List<String> sensorNameList;
    HashMap<Integer,Integer> typesList;
    int positionSensorTypes[]={
            Sensor.TYPE_GAME_ROTATION_VECTOR,
            Sensor.TYPE_GEOMAGNETIC_ROTATION_VECTOR,
            Sensor.TYPE_MAGNETIC_FIELD,
            Sensor.TYPE_MAGNETIC_FIELD_UNCALIBRATED,
            Sensor.TYPE_ORIENTATION,
```

```
        Sensor.TYPE_PROXIMITY
};
private AdapterView.OnItemClickListener listViewListener = new
                        AdapterView.OnItemClickListener() {
    public void onItemClick(AdapterView<?> parent, View view,
                        int position,long id) {
        Intent i = new Intent(MainActivity.this,SensorDetails.
                        class);
        i.putExtra("type",typesList.get(position));
        startActivity(i);
    }
};
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    sensorManager = (SensorManager) getSystemService(Context.
                        SENSOR_SERVICE);
   List<Sensor> sensorsList = sensorManager.
                        getSensorList(Sensor.TYPE_ALL);
    sensorNameList = new ArrayList<String>();
    typesList = new HashMap<Integer,Integer>();
   if(!(sensorsList.size() > 0 )){
        Toast.makeText(this,"No sensors supported",Toast.
                        LENGTH_LONG).show();
    }
    else{
       for(Sensor sensor : sensorsList){
            String sensorName = sensor.getName();
           int sensorType = sensor.getType();
```

```
                if(isPositionSensor(sensorType)) {
                typesList.put(sensorNameList.size(),sensorType);
                    sensorNameList.add(sensorName);
                }
            }
        }
        ListView listView = (ListView) findViewById(R.
                                            id.listView);
        ArrayAdapter<String> adapter = new ArrayAdapter
                  <String>(this,android.R.layout.simple_list_
                           item_1,sensorNameList);
        listView.setAdapter(adapter);
        listView.setOnItemClickListener(listViewListener);
}
   private boolean isPositionSensor(int sensorType){
        boolean found = false;
        for(int i: positionSensorTypes){
            if(i==sensorType) {
                found = true;
                break;
            }
        }
        return found;
   }
  @Override
   public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if
        it is present.
```

```
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
  // Handle action bar item clicks here. The action bar will
  // automatically handle clicks on the Home/Up button, so long
   // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
}
```

The code is functionally similar to that of the MotionSensors application created earlier.

9. Open the activity SensorDetails.java and modify the code as shown in Code Snippet 13.

Code Snippet 13:

```
package aptech.com.positionsensors;
import android.app.Activity;
import android.content.Context;
import android.hardware.Sensor;
import android.hardware.SensorEvent;
import android.hardware.SensorEventListener;
import android.hardware.SensorManager;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
```

```
import android.widget.TextView;
import android.widget.Toast;
import java.util.HashMap;
public class SensorDetails extends Activity {
    private SensorManager sensorManager;
    private int sensorType;
    private Sensor sensor;
    private TextView sensorNameView;
    private TextView sensorTypeView;
    private TextView unitsView;
    private TextView xValue;
    private TextView yValue;
    private TextView zValue;
    private HashMap<Integer,String> namesMap;
    private HashMap<Integer,String> typesMap;
    private HashMap<Integer,String> unitsMap;
   private SensorEventListener listener = new
                                        SensorEventListener() {
        @Override
        public void onSensorChanged(SensorEvent event) {
            xValue.setText(Float.toString(event.values[0]));
            yValue.setText(Float.toString(event.values[1]));
            zValue.setText(Float.toString(event.values[2]));
        }
   @Override
     public void onAccuracyChanged(Sensor sensor, int accuracy)
{
       }
    };
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_sensor_details);
    sensorManager = (SensorManager)getSystemService
                                (Context.SENSOR_SERVICE);
    sensorType = getIntent().getIntExtra("type", Sensor.
                                        TYPE_ACCELEROMETER);
    namesMap = new HashMap<Integer,String>();
    typesMap = new HashMap<Integer,String>();
    unitsMap = new HashMap<Integer,String>();
    sensorNameView = (TextView)findViewById(R.
                                         id.sensorName);
    sensorTypeView = (TextView)findViewById(R.id.valueType);
    unitsView = (TextView)findViewById(R.id.units);
    xValue = (TextView)findViewById(R.id.xValue);
    yValue = (TextView)findViewById(R.id.yValue);
    zValue = (TextView)findViewById(R.id.zValue);
    sensor = sensorManager.getDefaultSensor(sensorType);
    if (sensor != null){
        initMaps();
        sensorNameView.setText(namesMap.get(sensorType));
        sensorTypeView.setText(typesMap.get(sensorType));
        unitsView.setText(unitsMap.get(sensorType));
       sensorManager.registerListener(listener, sensor,
                            SensorManager.SENSOR_DELAY_NORMAL);
    }
    else {
        Toast.makeText(this,"The Sensor type is not available
                     on this device",Toast.LENGTH_LONG).show();
```

```
        }
    }
    private void initMaps() {
        namesMap.put(Sensor.TYPE_GAME_ROTATION_VECTOR, "Game
                                                Rotation Vector");
        namesMap.put(Sensor.TYPE_GEOMAGNETIC_ROTATION_VECTOR,
                                            "Geo Magnetic Sensor");
        namesMap.put(Sensor.TYPE_MAGNETIC_FIELD, "Magnetic
                                                    Field");
        namesMap.put(Sensor.TYPE_MAGNETIC_FIELD_UNCALIBRATED,
                                "Magnetic Field Uncalibrated");
        namesMap.put(Sensor.TYPE_PROXIMITY, "Proximity Sensor");
        typesMap.put(Sensor.TYPE_GAME_ROTATION_VECTOR,"Rotation
                                vector without Magnetic field");
        typesMap.put(Sensor.TYPE_GEOMAGNETIC_ROTATION_VECTOR,
                            "Rotation vector with Magnetic field");
        typesMap.put(Sensor.TYPE_MAGNETIC_FIELD,"Geo Magnetic
                            Force");
        typesMap.put(Sensor.TYPE_MAGNETIC_FIELD_UNCALIBRATED,
                                "Raw Magnetic Force values");
        typesMap.put(Sensor.TYPE_PROXIMITY,"Distance from the
                                            device");
        unitsMap.put(Sensor.TYPE_GAME_ROTATION_
                                            VECTOR,"Unitless");
        unitsMap.put(Sensor.TYPE_GEOMAGNETIC_ROTATION_
                                             VECTOR,"Unitless");
        unitsMap.put(Sensor.TYPE_MAGNETIC_FIELD,"uT");
        unitsMap.put(Sensor.TYPE_MAGNETIC_FIELD_
                                    UNCALIBRATED,"uT");
        unitsMap.put(Sensor.TYPE_PROXIMITY,"cm OR binary");
    }
    @Override
```

```
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if
it is present.
        getMenuInflater().inflate(R.menu.menu_sensor_details, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
    // Handle action bar item clicks here. The action bar will
    // automatically handle clicks on the Home/Up button, so long
    // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
    @Override
    protected void onResume() {
        super.onResume();
        sensorManager.registerListener(listener, sensor,
                                SensorManager.SENSOR_DELAY_NORMAL);
    }
    @Override
   protected void onPause() {
        super.onPause();
        sensorManager.unregisterListener(listener);
    }
}
```

The code is functionally similar to the MotionSensors application created earlier.

10. Connect the mobile device through USB cable.

11. Enable USB to connect to storage/device.
12. Install the adb drivers for the device.
13. Navigate to Run menu and select Run 'app'.
14. From the list of Connected Devices, select the device to install the apk on.
15. Click OK.

The application is deployed on the device and the output is shown in figure 13.5.

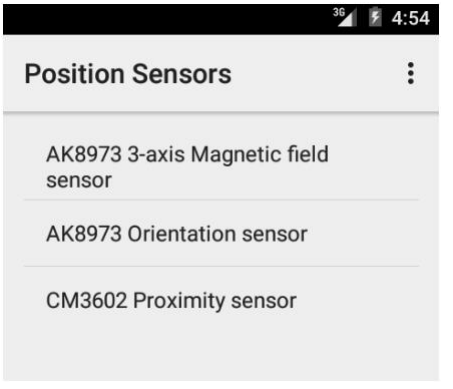


Figure 13.5: Position Sensors Application

The list of Position Sensors is displayed. Select a sensor, for example, AK8973 3-axis Magnetic field sensor and move the device in any direction to view the data as shown in figure 13.6.

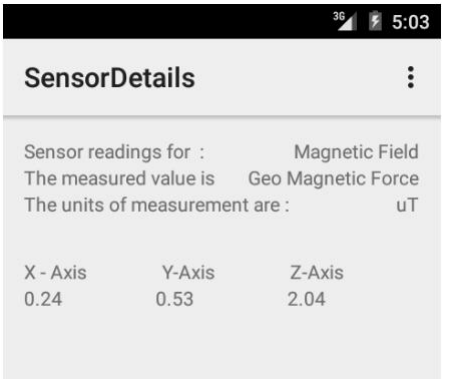


Figure 13.6: Sensor Details

13.4.3 Turning OFF the Device Screen

Before creating a real-time application of Position Sensors, one needs to understand the process of turning OFF the device screen.

Note - A practical use of the proximity sensor is to turn OFF the device screen when it is placed near the face (in call position). Hence, the process to turn OFF the screen must be understood.

Unfortunately, there is no standard procedure in Android for doing this. In this section, some workaround to achieve this behavior is explained. The success rate varies and the procedure which works on one device may not work on another.

- **Device Policy Administrator**

  This process involves adding the application as a Device Administrator and then, using the device policy manager to lock the screen of the device essentially switching OFF the screen. This is similar to emulating a Power button press. The code for this is shown in Code Snippet 14.

Code Snippet 14:

```
DevicePolicyManager dpm = (DevicePolicyManager)getSystemService
                          (Context.DEVICE_POLICY_SERVICE);

dpm.lockNow();
```

- **Power Manager Sleep Mode**

  The Power Manager's goToSleep() method can be used to put the device in sleep mode. The code for this is shown in Code Snippet 15.

Code Snippet 15:

```
PowerManager manager = (PowerManager) getSystemService(Context.
                                      POWER_SERVICE);

manager.goToSleep(1000);
```

- **Using a Wake Lock**

  Another implementation using the power manager involves getting hold of a wake lock momentarily and then releasing it, to put the device to sleep. The code for this is shown in Code Snippet 16.

Code Snippet 16:

```
PowerManager.WakeLock wl = manager.newWakeLock(PowerManager.
                                  PARTIAL_WAKE_LOCK, "Your Tag");

wl.acquire();

wl.release();
```

- **Setting the Brightness to Extremely Low**

  Most Android devices turn OFF the screen entirely if the brightness value of the screen is set to an impossibly low value. The code for this is shown in Code Snippet 17.

Code Snippet 17:

```
WindowManager.LayoutParams params = getWindow().getAttributes();
params.flags |= LayoutParams.FLAG_KEEP_SCREEN_ON;
params.screenBrightness = 0;
getWindow().setAttributes(params);
```

- **Reducing the Screen Timeout Value**

  The timeout value for the display to power OFF when there is no interaction can be set to a value low enough that the display powers OFF immediately. The code is shown in Code Snippet 18.

Code Snippet 18:

```
Settings.System.putInt(getContentResolver(), Settings.System.
                                        SCREEN_OFF_TIMEOUT, 10);
```

13.4.4 Real-time Example

As a real-time example, in this section, the steps to make an application to auto turn OFF the screen when an object is in proximity using the proximity sensor will be explained. The behavior is similar to that shown by the built-in 'Phone or Dialer' application, which turns OFF the screen during a call when the device is held near the face by the user. The steps are as follows:

1. Create a new project called ProximityOff in Android Studio.
2. Navigate to res → layout and open acitivty_main.xml for editing.
3. Modify the code as shown in Code Snippet 19.

Code Snippet 19:

```
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
```

```
    android:paddingBottom="@dimen/activity_vertical_margin"
                                tools:context=".MainActivity">

    <TextView

        android:layout_width="fill_parent"

        android:layout_height="fill_parent"

        android:textAppearance="?android:attr/textAppearanceLarge"

        android:text="Put your hand close to the screen to Turn
                                                Off the screen"

        android:id="@+id/textView"

        android:layout_alignParentTop="true"

        android:layout_alignParentStart="true"

        android:textColor="@android:color/holo_blue_light" />
</RelativeLayout>
```

4. Navigate to app → java → com.aptech.proximityoff and open MainActivity.java for editing.
5. Modify the code as shown in Code Snippet 20.

Code Snippet 20:

```
package aptech.com.proximityoff;

import android.app.Activity;

import android.app.admin.DevicePolicyManager;

import android.content.ComponentName;

import android.content.Context;

import android.content.Intent;

import android.hardware.Sensor;

import android.hardware.SensorEvent;

import android.hardware.SensorEventListener;

import android.hardware.SensorManager;

import android.os.Bundle;

import android.view.Menu;
```

```
public class MainActivity extends Activity {

    private SensorManager sensorManager;

    private SensorEventListener listener = new
                                          SensorEventListener() {

       @Override

        public void onSensorChanged(SensorEvent event) {

            float distance = event.values[0];

            if(distance < 2)

                turnOffScreen();

        }

      @Override

       public void onAccuracyChanged(Sensor sensor, int accuracy)
{

        }

    };

   private void turnOffScreen() {

        //Method to turn off screen

        int result=1;

        Intent intent = new

        Intent(DevicePolicyManager.ACTION_ADD_DEVICE_ADMIN);

        intent.putExtra(DevicePolicyManager.EXTRA_DEVICE_ADMIN,
                  new ComponentName(this, MainActivity.class));

        intent.putExtra(DevicePolicyManager.EXTRA_ADD_
     EXPLANATION,"Permissions required for locking the screen");

        startActivityForResult(intent, result);

        DevicePolicyManager dpm = (DevicePolicyManager)
               getSystemService(Context.DEVICE_POLICY_SERVICE);

        dpm.lockNow();

        //Alternate Method 1
```

```
        /*
        PowerManager manager = (PowerManager) getSystemService
                                        (Context.POWER_SERVICE);
        manager.goToSleep(1000);
         */
        //Alternate Method 2
        /*
        PowerManager.WakeLock wl = manager.newWakeLock
                    (PowerManager.PARTIAL_WAKE_LOCK, "Your Tag");
        wl.acquire();
        wl.release();
         */
       //Alternate Method 3
        /*
        WindowManager.LayoutParams params = getWindow().
                                            getAttributes();
       params.flags |= LayoutParams.FLAG_KEEP_SCREEN_ON;
        params.screenBrightness = 0;
        getWindow().setAttributes(params);
         */
        //Alternate Method
        /*
        Settings.System.putInt(getContentResolver(), Settings.
                                  System.SCREEN_OFF_TIMEOUT, 10);
         */
    }
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
```

```
        sensorManager = (SensorManager)getSystemService(Context.
                                                    SENSOR_SERVICE);
        Sensor proximitySensor = sensorManager.getDefaultSensor
                                    (Sensor.TYPE_PROXIMITY);
       sensorManager.registerListener(listener,proximitySensor,
                            SensorManager.SENSOR_DELAY_NORMAL);
    }
   @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if
//it is present.
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
   @Override
    public boolean onOptionsItemSelected(MenuItem item) {
  // Handle action bar item clicks here. The action bar will
  // automatically handle clicks on the Home/Up button, so long
 // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
}
```

The code registers a listener for the proximity sensor. Once an object is detected at a distance less than two units, the screen is turned OFF using one of the methods described earlier in section 13.4.3.

6. Navigate to res → values and open styles.xml for editing.

7. Modify the code as shown in Code Snippet 21.

Code Snippet 21:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="AppTheme" parent="android:Theme.Material">
    </style>
</resources>
```

8. Navigate to manifests and open AndroidManifest.xml for editing.
9. Modify the code as shown in Code Snippet 22.

Code Snippet 22:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
    xmlns:android="http://schemas.android.com/apk/res/android"
                            package="aptech.com.proximityoff" >
  <uses-permission android:name="android.permission.WAKE_LOCK" />
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <activity
            android:name=".MainActivity"
            android:label="@string/app_name" >
          <intent-filter>
             <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.
                                                    LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

10. Connect the mobile device through USB cable.
11. Enable USB to connect to storage/device.
12. Install the adb drivers for the device.
13. Navigate to Run menu and select Run 'app'.
14. From the list of Connected Devices, select the device to install the apk on.
15. Click OK.

Note - If the screen does not turn OFF on the device, uncomment the alternate method and try again.

The application is deployed on the device and the output is as shown in figure 13.7.

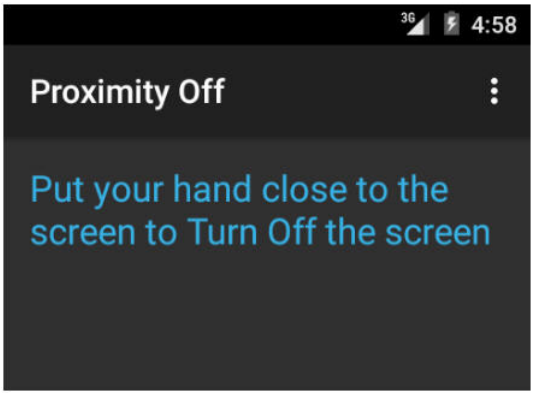


Figure 13.7: Proximity Off Application

The screen will be turned OFF if a hand is placed near the screen or if the device is held close to the face in an in-call position.

13.5 Environment Sensors

Environment sensors help monitor the properties of the surroundings of the device. All environment sensors are completely optional to implement by the device vendor. All the Environment sensors are hardware based only.

13.5.1 Android Environment Sensors

The types of Environment sensors supported by Android along with their implementation option and utility are shown in table 13.3.

Sensor Type	Implementation	Description
TYPE_AMBIENT_ TEMPERATURE	Hardware	Temperature sensor measures the room temperature that the device is placed in.

Sensor Type	Implementation	Description
TYPE_LIGHT	Hardware	Light sensor measures the ambient light falling on the device. This information is used in devices to adjust the brightness of the screen automatically.
TYPE_PRESSURE	Hardware	Pressure sensor measures the air pressure around the device.
TYPE_RELATIVE_ HUMIDITY	Hardware	Relative Humidity sensor measures the relative humidity in the atmosphere.

Table 13.3: Motion Sensors

13.5.2 Calculating Additional Information from Sensors

The information received from Environment sensors can be used to further calculate more environmental measurements as explained in this section.

- **Absolute Humidity**

  Absolute humidity is the total mass of water vapor present in a given volume of air. Absolute humidity is measured in grams/meter3. It can be calculated using the equation shown in figure 13.8.

$$d_v(t,RH) = \frac{216.7 \cdot (RH/100\%) \cdot A \cdot \exp(m \cdot t/(T_n+t)}{273.15 + t}$$

Figure 13.8: Equation to Calculate Absolute Humidity

In the equation,

d_v = absolute humidity in grams/meter^3

t = actual temperature in degrees C

RH = actual relative humidity in percent (%)

m = 17.62

T_n = 243.12 degrees C

A = 6.112 hPa

The code for this is shown in Code Snippet 23.

Code Snippet 23:

```
double numerator = 216.7f * ((relativeHumidity / 100) *
6.112 * Math.exp((17.62f * roomTemperature) / (243.12f +
roomTemperature)));

absoluteHumidity = (float) (numerator / (273.15f +
                    roomTemperature));
```

- **Dew Point**

The dew point is the temperature at which the water vapor in a sample of air at constant barometric pressure condenses into liquid water at the same rate at which it evaporates. At temperatures below the dew point, water will leave the air. It can be calculated using the equation shown in figure 13.9.

$$t_d(t,RH) = \frac{T_n \cdot \ln(RH/100\%) + m \cdot t/(T_n+t)}{m - [\ln(RH/100\%) + m \cdot t/(T_n+t)]}$$

Figure 13.9: Equation to Calculate Dew Point

In the equation,

t_d = dew point temperature in degrees C

t = actual temperature in degrees C

RH = actual relative humidity in percent (%)

m = 17.62

T_n = 243.12

The code for this is shown in Code Snippet 24.

Code Snippet 24:

```
double numerator =  (Math.
log(relativeHumidity/100f))+((17.62f*roomTemperature)/( 243.12f
+ roomTemperature));

double denominator = 17.62f-numerator;

dewPoint = (float)(numerator/denominator);
```

13.5.3 Implementing Environment Sensors

The basic API and code to access sensor information is demonstrated in the previous sections. This section explains the use of Environment sensors to make trivial weather predictions. The steps are as follows:

1. Create a new project called WeatherForecast in Android.
2. Navigate to res → layout and open acitivty_main.xml for editing.
3. Modify the code as shown in Code Snippet 25.

Code Snippet 25:

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
                          tools:context=".MainActivity"
    android:orientation="vertical">
    <ImageView
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:id="@+id/imageView"
        android:layout_weight="1" />
    <TextView
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="Weather Forecast Here"
        android:id="@+id/textView"
        android:layout_weight="0" />
```

```
    <Button
          android:layout_width="fill_parent"
          android:layout_height="wrap_content"
          android:text="Get Forcast"
          android:onClick="onGetForecast"
          android:id="@+id/button"
          android:layout_weight="0" />
</LinearLayout>
```

4. Navigate to app → java → com.aptech.weatherforecast and open MainActivity.java for editing.
5. Modify the code as shown in Code Snippet 26.

Code Snippet 26:

```
package aptech.com.weatherforecast;
import android.app.Activity;
import android.content.Context;
import android.hardware.Sensor;
import android.hardware.SensorEvent;
import android.hardware.SensorEventListener;
import android.hardware.SensorManager;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.ImageView;
import android.widget.TextView;
public class MainActivity extends Activity {
    private SensorManager sensorManager;
    float roomTemperature = 0;
    float illuminance = 0;
    float pressure = 0;
```

```
float relativeHumidity = 0;
float dewPoint = 0;
float absoluteHumidity = 0;
private SensorEventListener listener = new
                                        SensorEventListener() {
    @Override
    public void onSensorChanged(SensorEvent event) {
        int type = event.sensor.getType();
        if(type == Sensor.TYPE_AMBIENT_TEMPERATURE)
            roomTemperature = event.values[0];
        if(type == Sensor.TYPE_LIGHT)
            illuminance = event.values[0];
        if(type == Sensor.TYPE_PRESSURE)
            pressure = event.values[0];
        if(type == Sensor.TYPE_RELATIVE_HUMIDITY)
            relativeHumidity = event.values[0];
    }
 @Override
public void onAccuracyChanged(Sensor sensor, int accuracy) {
    }
};
    @Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    sensorManager = (SensorManager) getSystemService
                    (Context.SENSOR_SERVICE);
   sensorManager.registerListener(listener, sensorManager.
       getDefaultSensor(Sensor.TYPE_AMBIENT_TEMPERATURE),
                    SensorManager.SENSOR_DELAY_NORMAL);
```

```
        sensorManager.registerListener(listener, sensorManager.
getDefaultSensor(Sensor.TYPE_LIGHT), SensorManager.SENSOR_DELAY_
                                                        NORMAL);

        sensorManager.registerListener(listener, sensorManager.
getDefaultSensor(Sensor.TYPE_PRESSURE), SensorManager.SENSOR_
                                                DELAY_NORMAL);

        sensorManager.registerListener(listener, sensorManager.
getDefaultSensor(Sensor.TYPE_RELATIVE_HUMIDITY), SensorManager.
                                            SENSOR_DELAY_NORMAL);

    }

    @Override

    public boolean onCreateOptionsMenu(Menu menu) {

        // Inflate the menu; this adds items to the action bar if
//it is present.

        getMenuInflater().inflate(R.menu.menu_main, menu);

        return true;

    }

    @Override

    public boolean onOptionsItemSelected(MenuItem item) {

   // Handle action bar item clicks here. The action bar will
// automatically handle clicks on the Home/Up button, so long

   // as you specify a parent activity in AndroidManifest.xml.

        int id = item.getItemId();

        //noinspection SimplifiableIfStatement

        if (id == R.id.action_settings) {

            return true;

        }

        return super.onOptionsItemSelected(item);

    }

   private void generateForecast()

    {
```

```
boolean rainChance = false;
boolean hot = false;
TextView forecast = (TextView)findViewById(R.id.textView);
ImageView forecastImage = (ImageView)findViewById(R.
                          id.imageView);
generateDewPoint();
generateAbsoluteHumidity();
if(roomTemperature > dewPoint && absoluteHumidity > 10 &&
                                 pressure < 1013.25)
    rainChance = true;
if(roomTemperature > 40)
    hot = true;
if(hot && rainChance) {
    forecast.setText("It's very hot with a high chance of
                     rain. Carry an Umbrella");
    forecastImage.setImageResource(R.drawable.sunnyRain);
}
else if(!hot && rainChance){
    forecast.setText("Rain season. Carry an Umbrella");
    forecastImage.setImageResource(R.drawable.rain);
}
else if(hot && !rainChance)
{
    forecast.setText("Beat the Heat.");
    forecastImage.setImageResource(R.drawable.Sunny);
}
else {
   forecast.setText("Sweater weather");
    forecastImage.setImageResource(R.drawable.cold);
```

```
        }
    }
    private void generateDewPoint() {
        if(relativeHumidity!=0 && roomTemperature !=0)
        {
            double numerator =  (Math.
log(relativeHumidity/100f))+((17.62f*roomTemperature)/( 243.12f
                            + roomTemperature));
            double denominator = 17.62f-numerator;
            dewPoint = (float)(numerator/denominator);
        }
    }
    private  void generateAbsoluteHumidity(){
        if(relativeHumidity!=0 && roomTemperature !=0) {
            double numerator = 216.7f * ((relativeHumidity /
100) * 6.112 * Math.exp((17.62f * roomTemperature) / (243.12f +
                                roomTemperature)));
            absoluteHumidity = (float) (numerator / (273.15f +
                                    roomTemperature));
        }
    }
    public void onGetForecast(View view) {
        generateForecast();
    }
}
```

The code generates a weather prediction based on the values reported by the Environment Sensors. Additional information is calculated using methods described in section 13.5.2. Finally, a weather prediction is displayed to the user based on the values.

6. Add four images in the res → drawable folder representing weather conditions as shown in figure 13.10.

Figure 13.10: Weather Images

7. Connect the mobile device through USB cable.
8. Enable USB to connect to storage/device.
9. Install the adb drivers for the device.
10. Navigate to Run menu and select Run 'app'.
11. From the list of Connected Devices, select the device to install the apk on.
12. Click OK.

Note - Normal Weather Apps generally retrieve the forecast from a Web server. The organizations use sophisticated equipment to make these predictions. An Android device cannot match the quality of these predictions. This process can only serve as an alternative to generate results in offline mode or to send the data back to the main server to generate better predictions based on the ground conditions. The second usage is more common among these applications.

Note - This application can only be run on Real devices which implement relative humidity, pressure, and ambient temperature sensors and running Android version Lollipop or higher. In the given example, Moto turbo has been used to run the application. However, any other device with similar capabilities can be used.

The application is deployed on the device and the output is shown in figure 13.11.

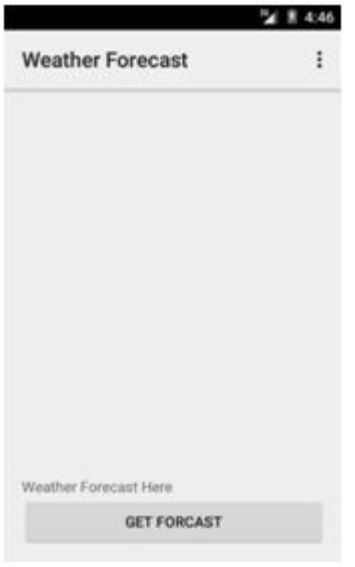


Figure 13.11: Environment Sensor Output

By clicking GET FORECAST, a weather forecast is generated as shown in figure 13.12.

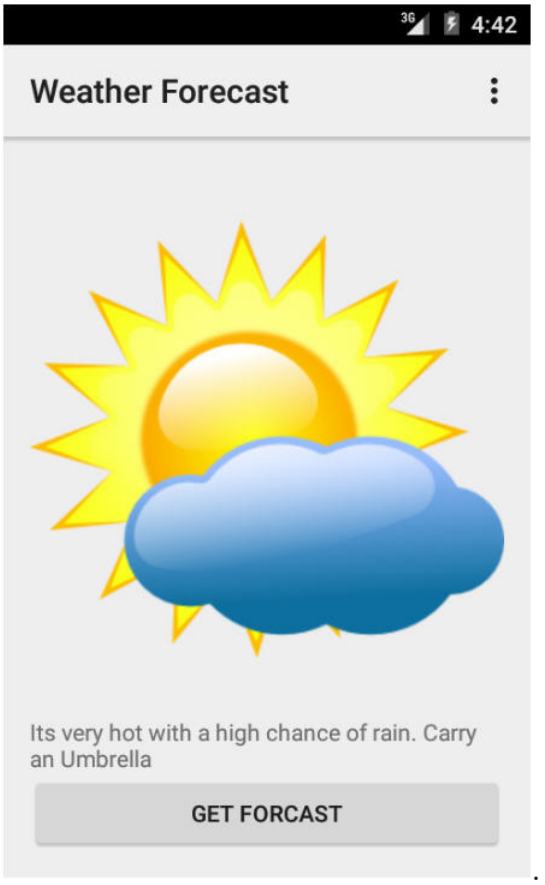


Figure 13.12: Weather Forecast Displayed

The output may vary depending on the current weather conditions.

13.6 Other Status Related Sensors

Besides the three major types of sensors, there are additional sensors in Android devices which can be accessed by other means or optional and specific to certain device types. This section explains some of them.

13.6.1 Battery Status

Battery status can be retrieved using a broadcast receiver with an intent filter for the intent Intent.ACTION_BATTERY_CHANGED. The receiver will receive an update whenever there is a change in the status of the battery (that is, charging, discharging, and so on.) and also, when there is a change in the battery level. Example code to achieve this is shown in Code Snippet 27.

Code Snippet 27:

```
public class MainActivity extends Activity {
    private TextView mPlugged;
    private TextView mStatus;
    private TextView mLevel;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        mPlugged = (TextView) findViewById(R.id.plugged_text);
        mStatus = (TextView) findViewById(R.id.status_text);
        mLevel = (TextView) findViewById(R.id.level_text);
       registerReceiver(batterReceiver, new IntentFilter(Intent.
                                        ACTION_BATTERY_CHANGED));
    }
   @Override
    protected void onStop() {
       super.onStop();
        unregisterReceiver(batterReceiver);
   }
```

```
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if
it is present.
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
    // Handle action bar item clicks here. The action bar will
    // automatically handle clicks on the Home/Up button, so
long
    // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
    private BroadcastReceiver batterReceiver = new
                                          BroadcastReceiver() {
        Intent intent = null;
        @Override
        public void onReceive(Context context, Intent intent) {
            this.intent = intent;
            SimpleDateFormat dateFormat = new SimpleDateFormat
                                      ("yyyy-MM-dd HH:mm:ss");
            String currentTimeStamp = dateFormat.format(new
                                                    Date());
```

```
        BatteryStatus currentInfo = new BatteryStatus
                (currentTimeStamp, getStatus(),Integer.
                                    valueOf(getLevel()));
        mPlugged.setText("Battery Plugged : " + getPlugged());
        mStatus.setText("Battery Status : " + getStatus());
        mLevel.setText("Battery Level : " + getLevel());
        db.addInfo(currentInfo);
    }
    public String getPlugged() {
        int plugged = intent.getIntExtra(BatteryManager.EXTRA_
                                                PLUGGED, -1);
        String pluggedStr = "";
        switch (plugged) {
            case BatteryManager.BATTERY_PLUGGED_AC:
                pluggedStr = "Charging from AC";
                break;
            case BatteryManager.BATTERY_PLUGGED_USB:
                pluggedStr = "Charging from USB";
                break;
            default:
                break;
        }
        return pluggedStr;
    }
    public String getStatus() {
    int status = intent.getIntExtra(BatteryManager.EXTRA_
                                    STATUS, -1);
       String statusStr = "";
        switch (status) {
```

```
                case BatteryManager.BATTERY_STATUS_CHARGING:
                    statusStr = "Battery Charging";
                    break;
                case BatteryManager.BATTERY_STATUS_DISCHARGING:
                    statusStr = "Battery Discharging";
                    break;
                case BatteryManager.BATTERY_STATUS_FULL:
                    statusStr = "Battery Full";
                    break;
                case BatteryManager.BATTERY_STATUS_NOT_CHARGING:
                    statusStr = "Battery not Charging";
                    break;
                case BatteryManager.BATTERY_STATUS_UNKNOWN:
                    statusStr = "Battery Status Unknown";
                    break;
                default:
                    break;
            }
            return statusStr;
        }
        public String getLevel() {
            int level = intent.getIntExtra(BatteryManager.EXTRA_
                                           LEVEL, -1);
            int scale = intent.getIntExtra(BatteryManager.EXTRA_
                                           SCALE, -1);
            if(level == -1 || scale == -1) {
                return "0%";
            }
```

```
            return "" + ((float)level / (float)scale) * 100.0f + "%";
        }
    };
}
```

The code creates a BroadcastReciever for any changes in battery status. Once a change has been reported, the battery information is retrieved using the getLevel(), getPlugged(), and getStatus() methods.

13.6.2 Heart Rate

The Heart Rate sensor is a heart rate monitoring sensor that is unique to the Android Wear devices such as the Galaxy Gear and the Moto 360. The device then communicates with another device running a proprietary app using Bluetooth or NFC. The application using this sensor needs to be targeted specifically for a wear device. The requirements for this are small screens with circular layouts and limited resolution. The information reported by the sensor is a single event value at the index 0. The data represents the heart rate in beats per minute. Heart Rate Monitor Sensor API is newly introduced and subject to change. The code to access the heart rate information is shown in Code Snippet 28.

Code Snippet 28:

```
public class SensorDetails extends Activity {
    private SensorManager sensorManager;
    private int sensorType;
    private Sensor sensor;
    private SensorEventListener listener = new
                                          SensorEventListener() {
        @Override
        public void onSensorChanged(SensorEvent event) {
            float heartrate = event.values[0];
            // Use the heart rate value here
        }
     @Override
      public void onAccuracyChanged(Sensor sensor, int accuracy) {
```

```
        }
    };
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_sensor_details);
        sensorManager = (SensorManager) getSystemService
                                    (Context.SENSOR_SERVICE);
        sensor = sensorManager.getDefaultSensor(Sensor. TYPE_
                                                    HEART_RATE);
        if (sensor != null){
            Toast.makeText(this,"The device supports a Heart
                        Rate Sensor",Toast.LENGTH_LONG).show();
            sensorManager.registerListener(listener, sensor,
                            SensorManager.SENSOR_DELAY_NORMAL);
        }
        else {
            Toast.makeText(this,"The Sensor type is not
            available on this device",Toast.LENGTH_LONG).show();
        }
    }
   @Override
    protected void onResume() {
        super.onResume();
        sensorManager.registerListener(listener, sensor,
                          SensorManager.SENSOR_DELAY_NORMAL);
    }
   @Override
    protected void onPause()
{
```

```
        super.onPause();

        sensorManager.unregisterListener(listener);

    }

}
```

The code to access Hear Rate sensor is similar to other types of sensors as it uses the same Sensor Framework.

13.7 Context Sensing and Location Based Services

Sensor information can be used to create interactive multimedia applications and games. Another more powerful use of sensors is to identify the current situation or context-aware and location based services.

Sensors contain a lot of information which can be further processed to generate even more information. For example, consider that a shopping app has been developed for a Website. A traditional approach to this application would be to just act as an interface for the store front. A more powerful context-aware approach would have the app record the user preferences and the current conditions to predict the user requirement. A simple example would be to use the ambient temperature sensor to record the temperature of the area. If the temperatures are cold, then sweaters and winter wear can be suggested to the user automatically. The user is more likely to purchase these products than other fashion accessories.

Location also plays a key role in identifying the context. Consider a restaurants app as an example. Suggesting a restaurant that is three hours away to the user, even if it is the best restaurant in the city, may not be the right approach. The user is likely to ignore the suggestion all together. Localized suggestions near the user will yield better results.

Marketing is a prime area of use for location based services, followed by news and infotainment. Applications belonging to this category almost mandatorily implement Context sensing to some degree. Context sensing not only makes the user feel that he is getting relevant output from the application but also builds trust and makes the app appear more useful than it really is.

In order to implement context sensing, the developer can either write a specialized sensing framework from scratch that fits their needs. An alternative would be to use existing context sensing services such as Intel's Context Sensing SDK. The SDK supports Android at the moment with Windows support coming very soon. Using this framework, the developer need not worry about the sensors that are supported by the device. The SDK framework automatically detects the available sensors and generates context data over time.

Google's own applications such as Google Now and products such as Google Ads implement context sensing to generate suggestions. However, before implementing context sensing in the application, the developer needs to take in account the privacy concerns involved in the process. If the data is being sent over to another remote server to process, then this information needs to be clearly communicated with the user. All the data and statistics need to be maintained with utmost security and usage statistics should always be kept anonymous. The privacy policy needs to specify the exact type of data that is being used for this purpose and how it is being handled. Finally, the developer also needs to take into account the computational overhead caused due to context sensing and decide if it is worth the extra performance hit.

Check Your Progress

1. Which of the following sensors can be implemented only in hardware for Android?

1.	Accelerometer	2.	Proximity Sensor
3.	Gyroscope	4.	Ambient Light Sensor

(A)	1, 3	(B)	2, 3
(C)	3, 4	(D)	1, 4

2. Which of the following sensors are only Hardware based?

1.	Other Status Related Sensors	2.	Position sensors
3.	Environment sensors	4.	Motion sensors

(A)	1, 3	(B)	1, 2
(C)	2, 4	(D)	2, 3

3. Which of the following code retrieves an instance of SensorManager?

(A)	new SensorManager()	(B)	Context.getSensorManager()
(C)	(SensorManager) getSystemService(Context. SENSOR_SERVICE);	(D)	Context.getSystemService(SENSOR_ MANAGER)

4. Sensor data values are encapsulated in the _____ class.

(A)	Sensor	(B)	SensorManager
(C)	SensorEvent	(D)	SensorListener

5. Match the following sensors with their corresponding type.

	Sensor		Type
a.	Accelerometer	1.	Other Status Related Sensors
b.	Temperature	2.	Position Sensor
c.	Magnetic Field	3.	Environment Sensor
d.	Battery Status	4.	Motion Sensor

(A)	a-4, b-3, c-2, d-1	(B)	a-3, b-1, c-4, d-2
(C)	a-2, b-4, c-3, d-1	(D)	a-1, b-3, c-4, d-2

Answers

1.	A
2.	A
3.	C
4.	C
5.	A

Summary

- Sensors are devices that record physical data.
- Android supports primarily three types of sensors.
- Motion Sensors monitor the movement of the device.
- Position Sensors monitor the position and orientation of the device.
- Environment Sensors monitor the surroundings of the device.
- Other miscellaneous or vendor specific sensors are also supported by Android.
- Sensors can be used for context sensing in applications.

MANY COURSES ONE PLATFORM

14

Google Play Store

Welcome to the Session**, Google Play Store.**

This session introduces the developers to Google Play Store. It also walks the developer through the basic requirements needed for the Android app development and publishing of app in the Google Play Store.

In this Session, you will learn to:

- Explain the requirement for Google Play Store
- Explain the different versions and feature set
- Explain the share of Android in the market
- Explain different kinds of devices available in the market
- Describes the making of an .apk file
- Explain the process of publishing the .apk file in Google Play Store
- Explain the best practices to be followed
- Explain good marketing and promotion strategies

Session

14

Google Play Store

14.1 Introduction

Google Play Store is Google's market place for Android Applications, music, movies, and books. Google Play Store comes pre-installed in all Android Devices. It serves as the main source for application for more than 70 % of Android users. Hence, it is in the best interest of a developer to publish the app on Google Play Store in order to maximize the target audience and the associated revenue.

14.2 What is Play Store?

Google Play Store (formerly, Android Market) is an online electronic store or digital application distribution platform for Android powered devices. It is a robust publishing platform. The services offered by Play Store to the users includes downloading free and paid applications, books, movies, music, magazines, and so on. Users can also purchase mobile devices such as Google-Nexus, Chromebook, and so on through Play Store. The initial release of Android market was on October 23, 2008. At the initial beta release Android market provided only free applications. Later on, by February 2009, Google implemented the purchase system, which enabled the users to download applications only if the user made the payment while downloading the app from the Android market. At that time, Android market contained only 2300 applications. Currently, there are over 1.5 million applications and with over 60 billion downloads.

The advantage of distributing applications over the Play Store is tremendous, since the developer has access to global users. Also, the developer will have access to revenue generating tools, such as in-app-billing and application-licensing. The developer will also know the sale trends and understand the end users.

14.3 Market Shares and Targeting the Right Version

The Android market is fragmented. The hardware and the OS version vary from one device to another. This enables innovation and rapid progress on the platform. However, at the same time, it adds the burden on the developer to maintain compatibility. Targeting the latest and greatest may give you better quality application, but the potential market of the application will be reduced drastically. This section, explains the guidelines the developer can follow to ensure the application is compatible with most of the popular devices.

14.3.1 Hardware Requirements

The developer needs to try to utilize the least common denominator in terms of hardware. Do not try to use hardware that is not essential to the application. Support alternatives when the hardware is not available. For example, a file transfer application does not necessarily need NFC support. Try to use Bluetooth or network connections. If the application is using location based services and GPS hardware is not available, the application uses the network location to generate an estimate. If GPU acceleration is being used, try to make sure that the application runs properly in a device from two generations earlier (Nexus 4 if Nexus 6 is the current generation).

14.3.2 Operating System Market Shares

Operating System upgrades on the Android platform are much slower than any other platform. This is mainly due to the control of operating system choice with the device vendor. The user cannot decide when/if he can upgrade to a new OS version. The device will receive the update only if the vendor pushes an update. The vendor may also choose to discontinue updates for the device leaving the device forever at the same OS version.

The updates themselves are delayed even if the vendor has planned an update. Most vendors make several changes to stock Android code to make it look unique. Only the Nexus devices receive the update on day one. Hence, the developer needs to make sure that by targeting the latest version, he is not alienating potential customer until the update or even forever. Most applications have no particular use of the latest API. Unless it is absolutely mandatory to make the switch, try to use the old API.

As of May 2015, the number of devices executing particular version of Android are displayed in table 14.1.

Code Name	Version	API Level	Share
Lollipop	5.1	22	0.7 %
Lollipop	5.0	21	9 %
Kitkat	4.4	19	39.8 %
JellyBean	4.2	18, 17, 16	39.2 %
Ice Cream Sandwich	4.0	15	5.3 %
Gingerbread	2.3	9	5.7 %
Froyo	2.2	8	0.3 %

Table 14.1: Market Share of Different Platforms

The market shares are represented in graphic format in figure 14.1.

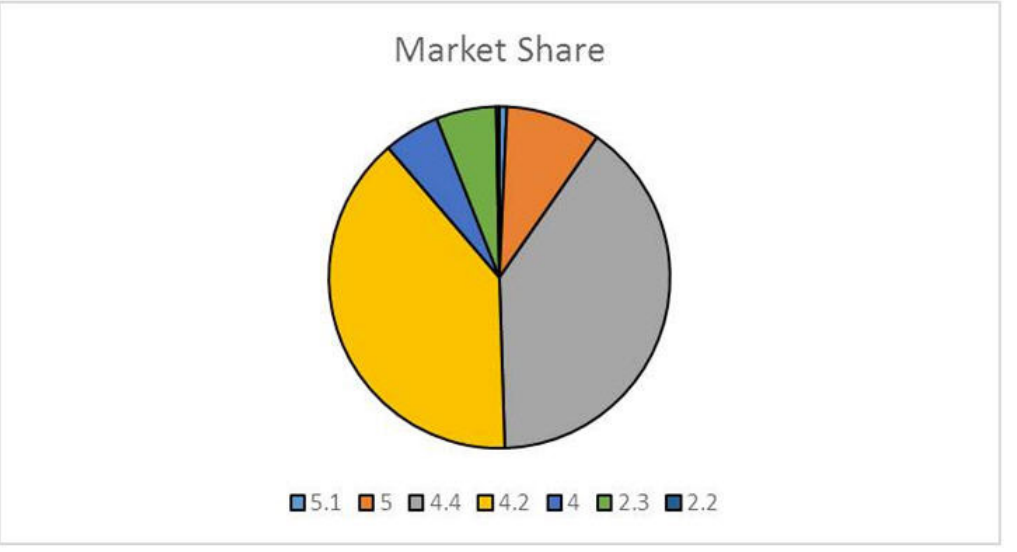


Figure 14.1: Market Shares for Android Versions

Note - The data is subject to change. It is advisable to verify the market shares at the time of starting the development.

It must be ensured that the application supports 'at least 80% of the market share for it to be successful. In that case, the minimum SDK version suitable is JellyBean. This data is displayed when creating an application in Android Studio as shown in figure 14.2.

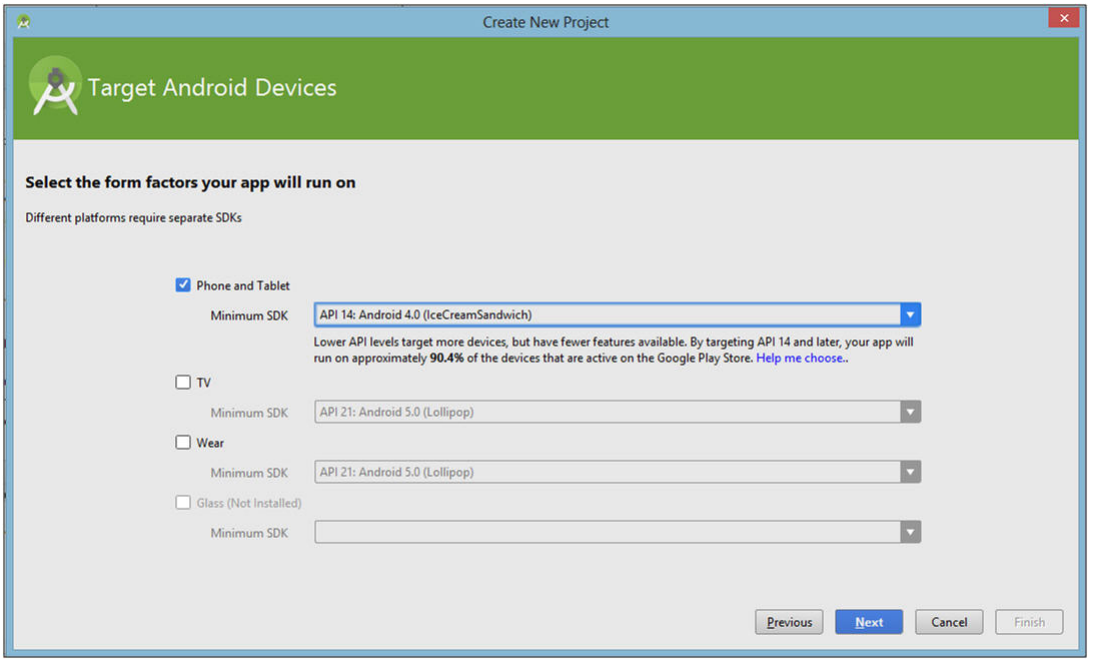


Figure 14.2: New Project Dialog Box with Compatibility Information

14.3.3 Support Library

In order to ease the issue with variation of API, Android has introduced Support Libraries to use in applications. By using these libraries, the developer can continue to use the newer API and still make sure that the application will be compatible with older devices. The Library chooses the appropriate implementation for the developer. The packages android.support.v4 and android.support.v7 should to be imported for the v4 and v7 support libraries respectively. The classes in this library have a prefix of AppCompat with the same API. For example, the support library replacement for Activity class is AppCompatActivity.

Support libraries can be installed from the SDK Manager as shown in figure 14.3.

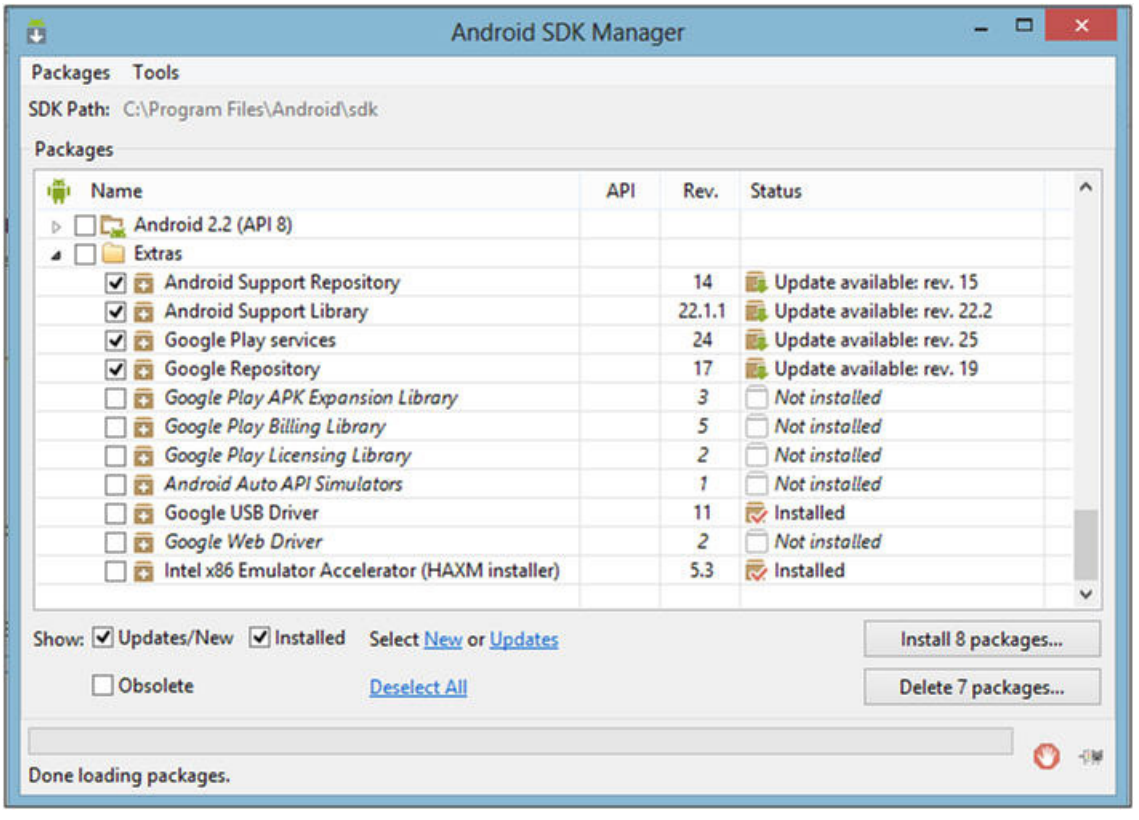


Figure 14.3: SDK Manager with Support Libraries Installed

14.4 Prerequisites to Publishing the Application

Before uploading the application in Android market, the basic points to be noted are as follows:

1. Testing the application.
2. Checking the application performance.
3. SDK Compatibility.

14.4.1 Testing the Application

The application developed must be tested well before uploading it to the Play Store so that the uploaded application is not only error/bug free but also does not crash at users' end. A stable application and good reviews are key to the success of the application.

The output of the application when tested on an emulator may vary when tested on real devices. The output also varies from one device to another and also from one version of Android to another. For this reason, the application is required to be tested on real devices of different types before being uploaded to the Play Store for the users to download.

The resolutions supported and the layouts for the supported resolutions need to be thoroughly tested. A broken UI is a huge put off for the user. Currently, the primary screen resolutions are as follows:

- 800 px x 480 px
- 1280 px x 720 px
- 1920 px x 1080 px

Note - px stands for pixels. It is advisable to check the commonly used resolution at the time of publishing as they are subject to change over time.

To augment the support for different screen sizes, Android provides a different drawable folder for each application to place different resolution of images. Android will choose the images from the respective folder depending on the screen size.

14.4.2 Preparations

To prepare your application for releasing, the developer typically needs to perform five tasks which are as follows:

- **Collect Material for Release**: This includes preparation of End User License Agreement (EULA) for the application. This will help to protect not only the developer and the organization but also the intellectual property rights. It also includes the process of obtaining an encrypted key for digitally signing the application and creation of an icon for the application.
- **Arrange Application for Resource**: This includes gathering of all the materials required for configuring the applications which includes configuration changes made to the source code, resource files, and the manifest file of the application. Besides this the developer has to clean up the project, update the manifest file setting, update the URLs for servers and services, and so on.
- **Build Application for Resource**: This includes signing the application and building the application for release.
- **Prepare Remote Server**: This includes the process of ensuring that if a remote server is used it is secure and configured for use.

- **Test Application for Resource**: This includes testing the application to ensure that the application works properly under real device and varying conditions.

14.4.3 Application Performance

The performance of the application is one of the most important concerns to the developer. A sluggish application is destined to receive bad reviews followed by less downloads. Code optimization tools such as lint and code clean up tools (part of the IDE) can be used to improve the performance of the application.

14.4.4 Pre-Publish Tasks

Following are some of the tasks that need to be performed before uploading the package:

1. Set permissions.
2. Establish version, set icon, and application label.
3. Set compatibility options.
4. Remove log data.
5. Export the project and create the key using Eclipse.

- **Permissions:**

Following details should be set correctly in the AndroidManifest.xml file:

- **<uses-permission> element**

  In this tag, specify only those permissions that are relevant and required for executing the application.

- **android:icon and android:label attributes**

  In the <application> element, specify values for these attributes as these are displayed to the user. The default icon needs to be replaced with a proper icon representing the application.

- **android:versionCode and android:versionName attributes**

  In the <manifest> element, specify values for these attributes.

There are several additional manifest elements that can be set by the developer if the application is being released on Google Play. For example, the android:minSdkVersion and android:targetSdkVersion attributes, which are located in the <uses-sdk> element. Code Snippet 1 demonstrates the use of some of these attributes.

Code Snippet 1:

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_
NETWORK_ STATE" />
<uses-permission android:name="android.permission.WRITE_
EXTERNAL_ STORAGE"/>
<uses-permission android:name="android.permission.MOUNT_UNMOUNT_
FILESYSTEMS" />
<uses-permission android:name="android.permission.WAKE_LOCK"/>
```

- **Version:**

The version names have to be specified in the AndroidManifest file that is written within the <manifest> tag. Code Snippet 2 demonstrates the use of the tag.

Code Snippet 2:

```
<manifestxmlns:android="http://schemas.android.com/apk/res/
android"
package="com.example.helloandroid"
android:versionCode="1"
android:versionName="1.0" >
```

In the Code Snippet, the version code starts from 1 and is required to be incremented by 1 for each upload. Version name specifies the release version and always should be in ascending order.

- **Specifying Icon, Application Label, and SDK Version:**

The icon to be set for the application and the name of the application have to be specified inside the <application> tag in AndroidManifest file as shown in Code Snippet 3.

Code Snippet 3:

```
<application
   android:allowBackup="true"
    android:icon="@drawable/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/AppTheme" >
```

```
<activity
        android:name="com.helloworld.MainActivity"
        android:label="@string/app_name" >
        <intent-filter><action android:name="android.intent.action.
                                            MAIN" />
      <category android:name="android.intent.category.LAUNCHER" />
       </intent-filter>
       </activity>
</application>
```

Code Snippet 4 demonstrates the process of specifying the SDK version.

Code Snippet 4:

```
<uses-sdk android:minSdkVersion="8"
android:targetSdkVersion="10"
android:maxSdkVersion="16"/>
```

The minSdkVersion, specifies the minimum sdk version the application will support.

The targetSdkVersion, specifies the version to which the application is built for.

The maxSdkVersion, specifies the maximum sdk version the application can support.

- **Feature Restriction**

Features that are essential to the functionality of the application can be specified using the Uses-feature tag. This ensures that the application is not installed on devices that do not have the feature. The code to achieve this is shown in Code Snippet 5.

Code Snippet 5:

```
<uses-feature android:name="android.hardware.nfc"
android:required="true" />
```

- **Compatibility Options:**

The Android system will decide on which platform the application will be installed depending on the target specified in the manifest file. The play store automatically identifies the compatible devices and prevents unsupported devices from installing the application. The publisher can change this from the Google Play Developer Console.

- **Remove Log data:**

  Remove the calls to logs, as it will display output data in LogCat always. This will not only increase the size of the log file but also, increase the time required to be spent on to print huge data each time. Unnecessary resource files are required to be removed as it will lead to increase in size of the application. This can also lead to security vulnerabilities and leak key information about the source code of the application.

14.4.5 Signing the Application

Before an application can be uploaded to any application store, the application needs to be signed by the developer. There is no need for a Certification Authority that is, the developers can generate their own certificates for signing the application. The purpose of signing the application is to verify the authenticity of the application and to ensure that it is not a counterfeit or tampered with.

The steps to sign an application are as follows:

1. Start Android Studio and open the desired project.
2. Navigate to the Build Menu and select Generate Signed APK.
3. The Generate Signed APK dialog box is displayed as shown in figure 14.4.

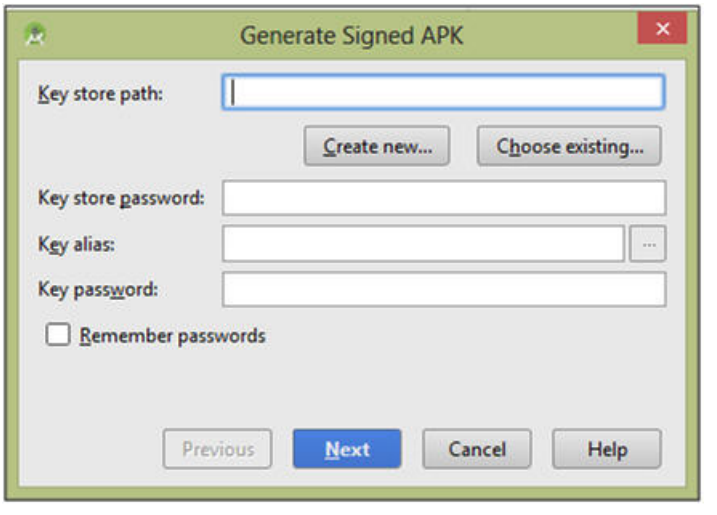


Figure 14.4: Generate Signed APK Dialog Box

4. If it is the first time signing, click Create new to open the New Key Store dialog box as shown in figure 14.5.

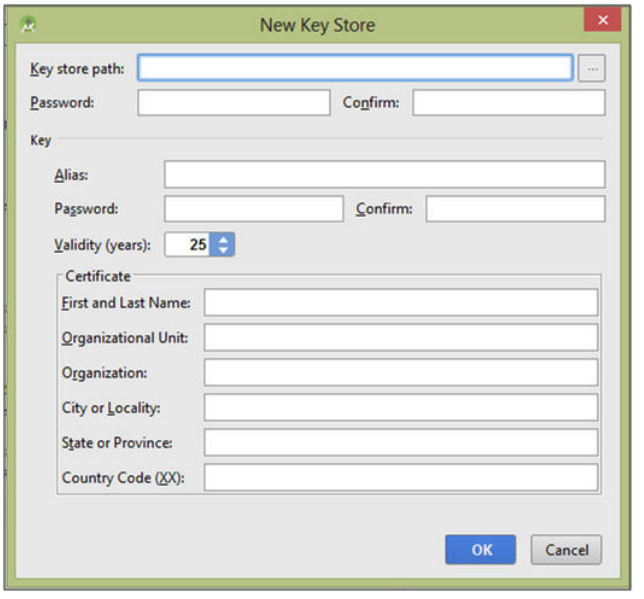


Figure 14.5: New Key Store Dialog Box

5. Enter the details as shown in figure 14.6.

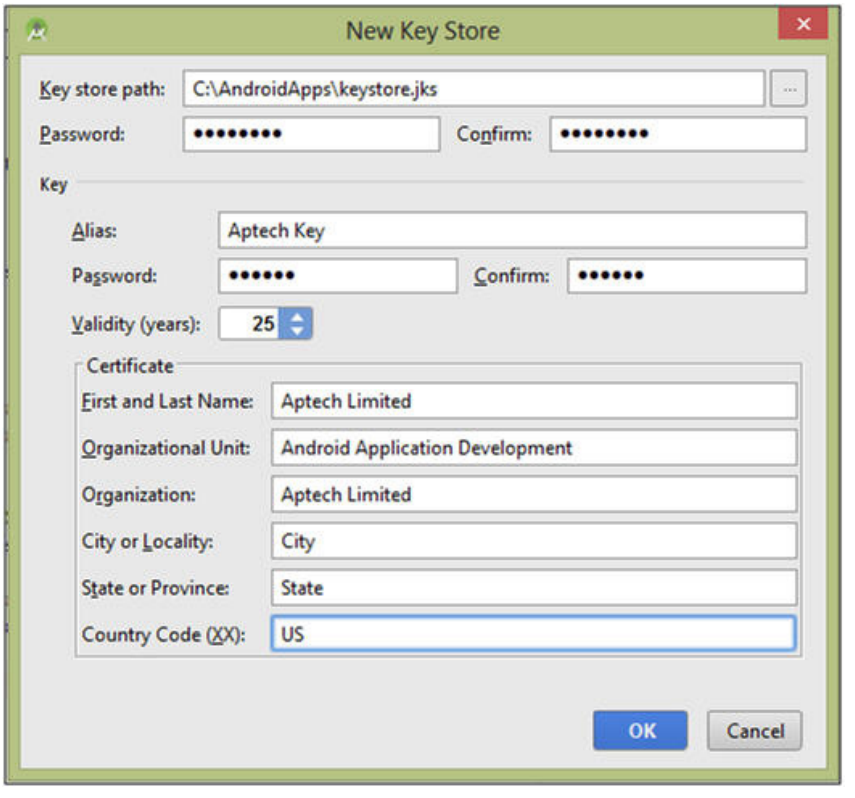


Figure 14.6: Key Store Details

6. Click OK to return to the Generate Signed APK dialog box with the details already entered as shown in figure 14.7.

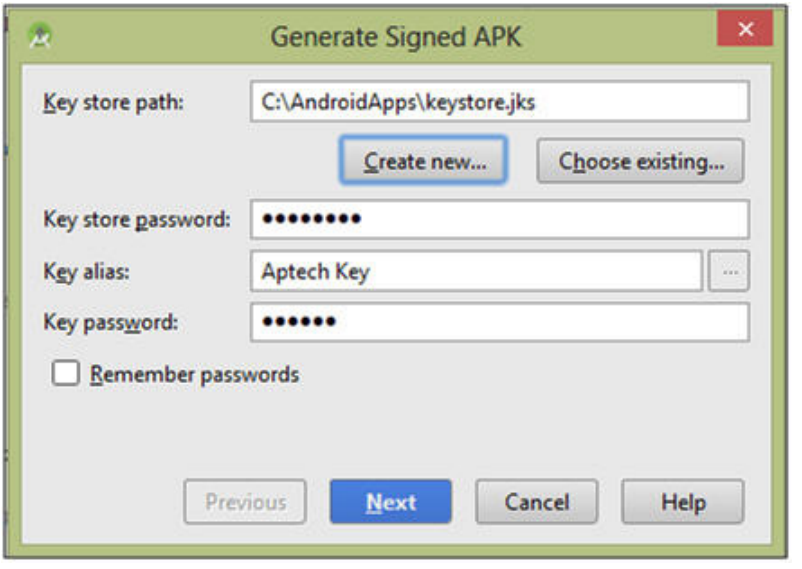


Figure 14.7: Generate Signed APK with the Details

7. Click Next to display the Destination dialog box as shown in figure 14.8.

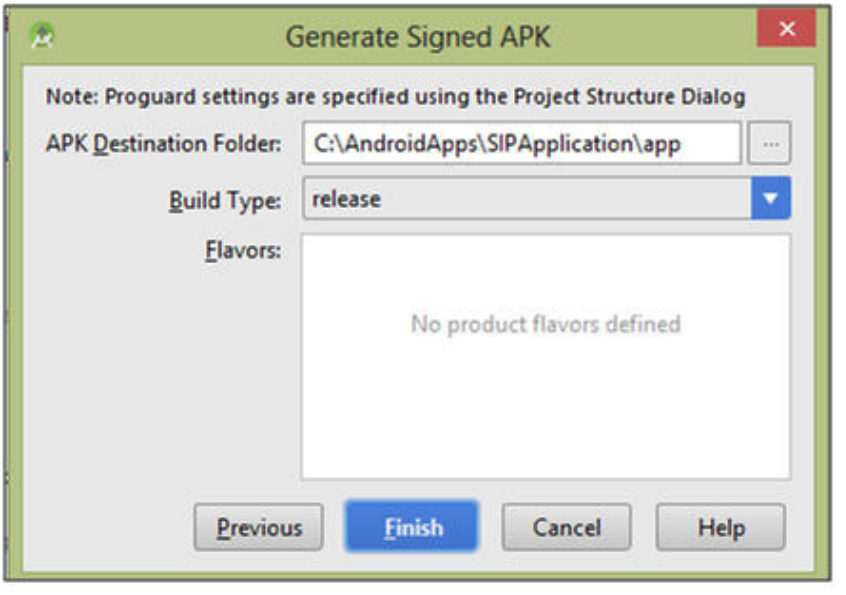


Figure 14.8: Destination Details

Note - The application will expire in 25 years as validity was specified as 25 years. A minimum validity of 25 years is required.

8. Click Finish. A new app-release.apk file is generated by Android Studio in the specified target directory. This .apk file can be renamed and uploaded to the Play Store.

14.5 Publishing Android Application to Play Store

This section explains the procedure for publishing an application on the Play Store. The steps for publishing an Android application to Play Store are as follows:

1. **Create a Signed .apk File**

   As discussed, the developer needs to create an .apk file with 25 years validity. The Play Store will only allow application which is at least valid for 25 years.

2. **Sign in to the Play Store Account**

   The developer needs to register for a publisher account by visiting the Google Play Developer console at https://play.google.com/apps/publish/. The basic details of the developer such as name, email address, and so on are accepted. The developer distribution agreement related to the developer's country is accepted. If the developer is registering for the first time, then US $25 should be paid as registration fee using Google wallet.

   The Sign in page for Google Play Store is shown in figure 14.9.

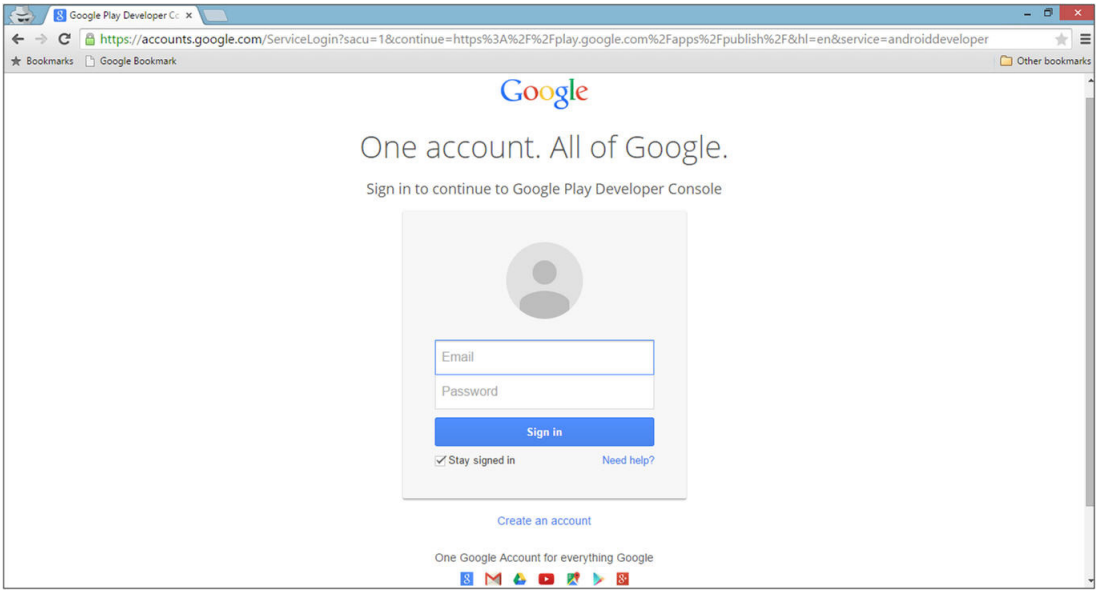


Figure 14.9: Sign in Page

The home screen is displayed as shown in figure 14.10.

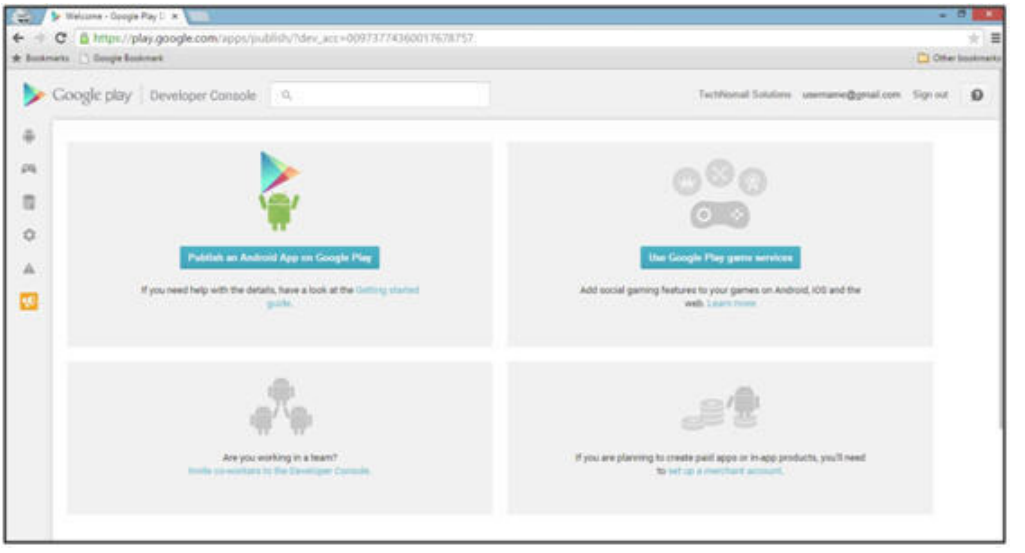


Figure 14.10: Home Page

3. **Add a New Application**

Select the 'ALL APPLICATIONS' tab from the left pane. Add your application to play store by clicking Add new application as shown in figure 14.11.

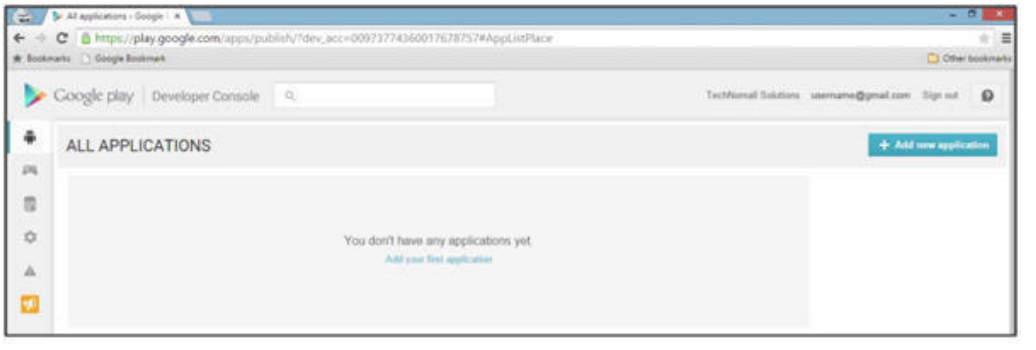


Figure 14.11: Adding a New Application

The Add New Application screen is displayed as shown in figure 14.12.

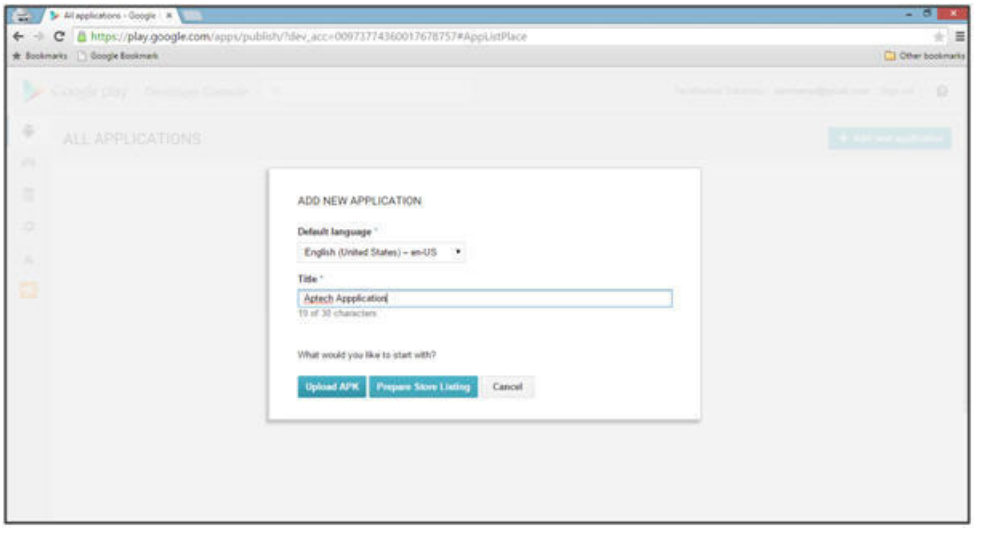


Figure 14.12: Add New Application

Language can be selected from the Default language list.

Title of the application is specified in the Title box.

Click Prepare Store Listing.

4. **Prepare Store Listing**

Store listing displays a page which allows the developer to enter the description of the application, Promo text, Recent changes, and so on as shown in figure 14.13.

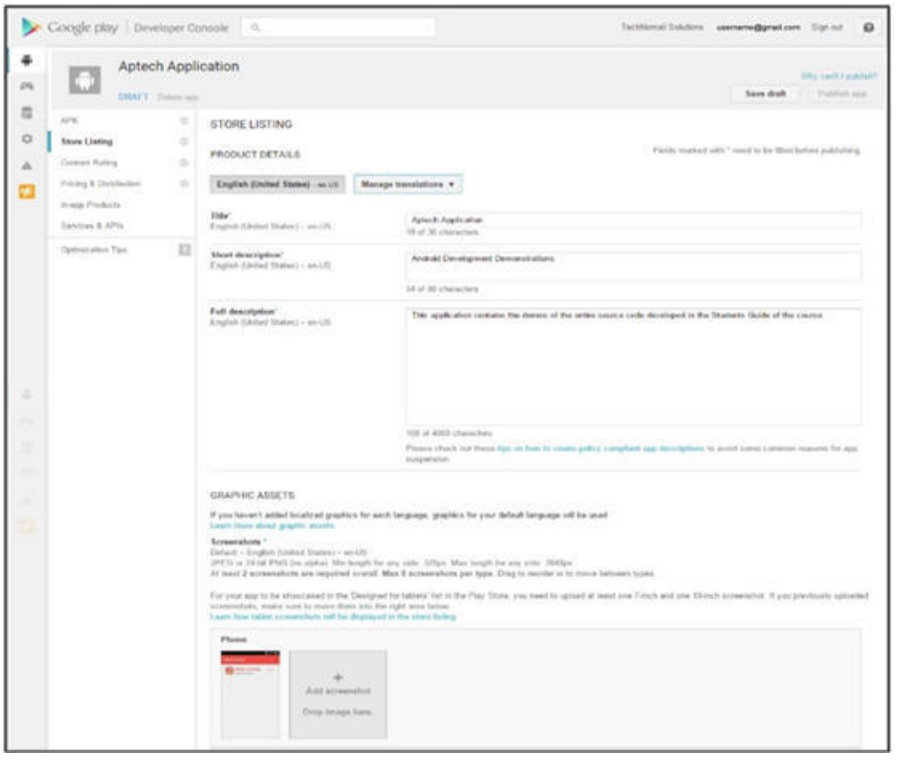


Figure 14.13: Store Listing

5. **Upload the Application Screenshots**

Screen shots of the application is taken using either Emulator or Real device and uploaded as shown in figure 14.14.

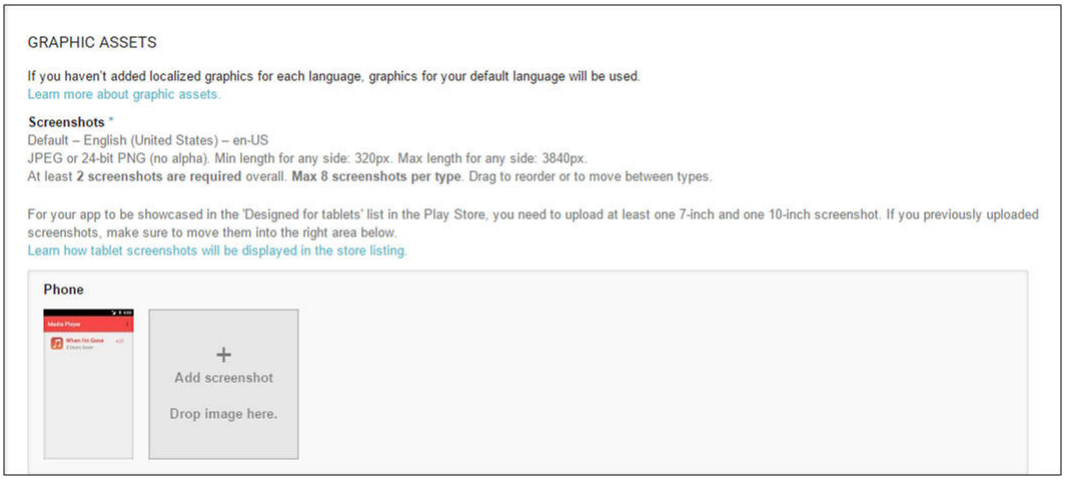


Figure 14.14: Application Screenshots

Different screenshots can be uploaded for different screen sizes as specified.

In the same pane, the developer needs to fill in the Categorization Details.

6. **Specify Categorization Details**

Categorization details mainly specify the application type and category. The Application type and Category helps the developer to specify the type of application that is getting uploaded as shown in figure 14.15.

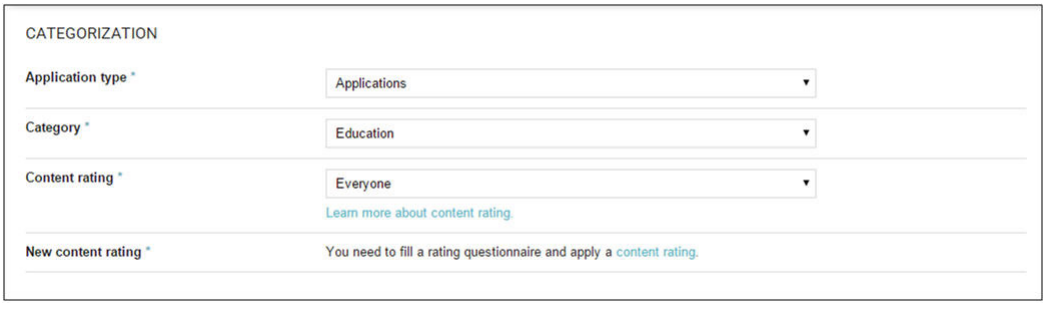


Figure 14.15: Category Details

Note - Application type can be Social, Entertainment, Puzzle, and so on.

7. **Add Contact Information**

Play Store allows the developer to enter the contact information as shown in figure 14.16. The contact information includes details such as Website, Email, and Phone.

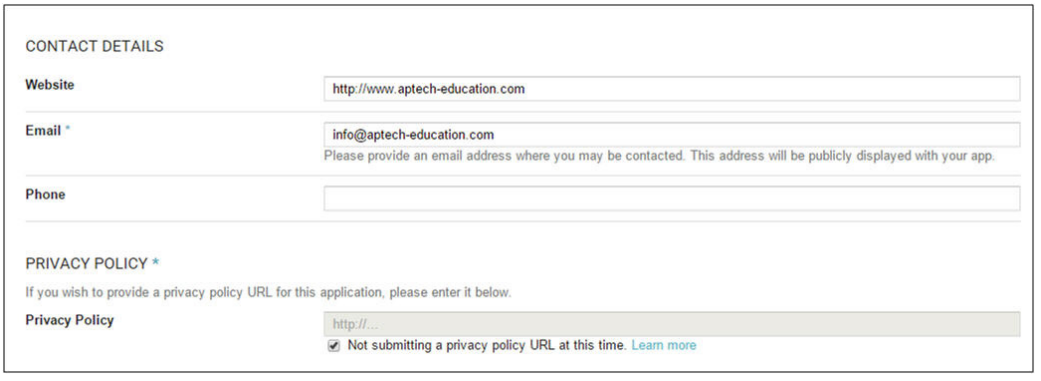


Figure 14.16: Contact Information

After adding all the information, click Save.

8. **Upload the .apk File**

Finally, the developer needs to click 'Upload your first APK to Production' as shown in figure 14.17.

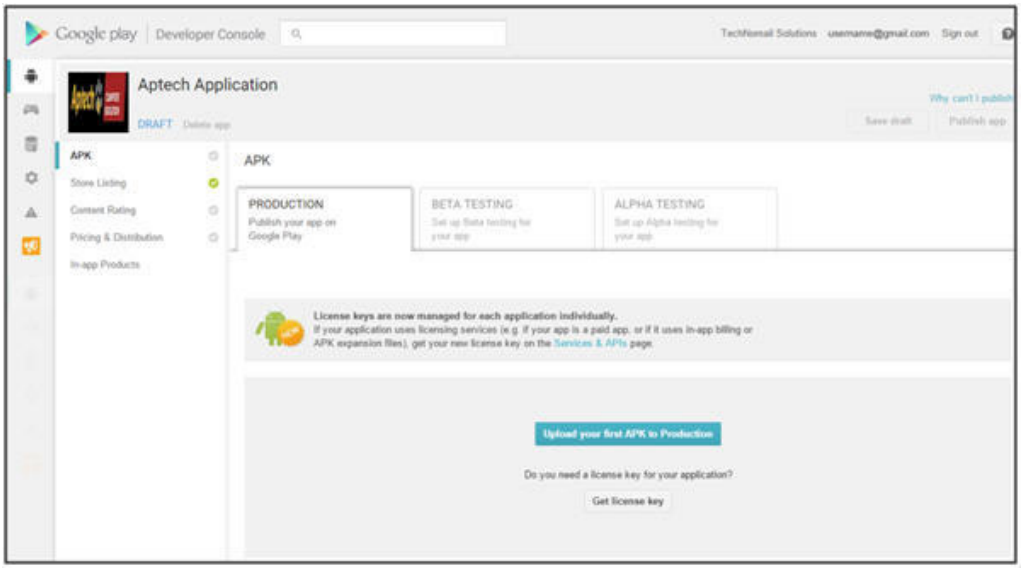


Figure 14.17: Uploading the .apk File

The developer needs to navigate to the desired folder and select the .apk file and the uploading process starts as shown in figure 14.18.

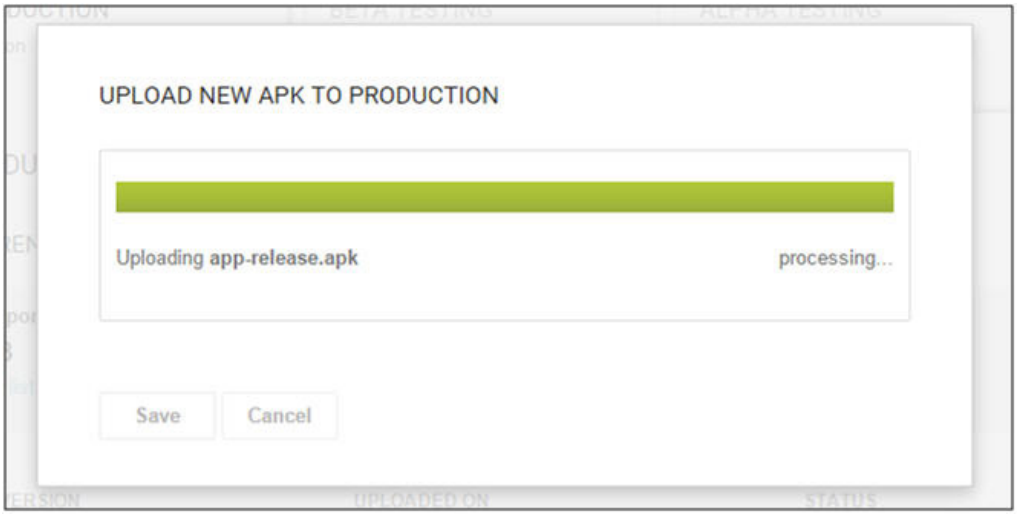


Figure 14.18: Upload of APK File

9. **Complete the Content Rating Questionnaire**

The developer needs to complete a content rating questionnaire before the application is published. Click the Content Rating tab as shown in figure 14.19.

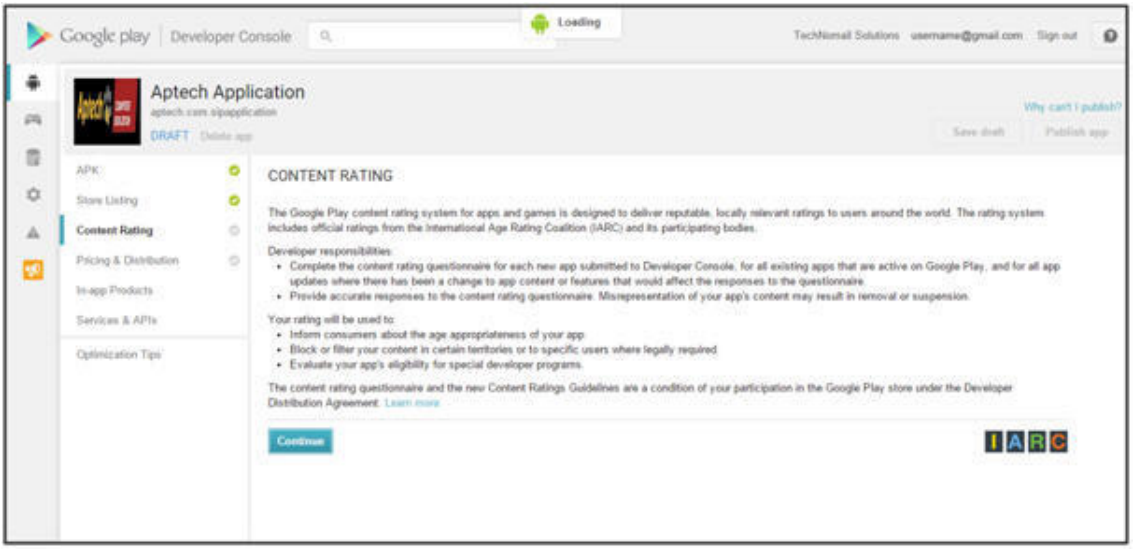


Figure 14.19: Content Rating Screen

Click Continue, enter the email address, and select the app category as shown in figure 14.20.

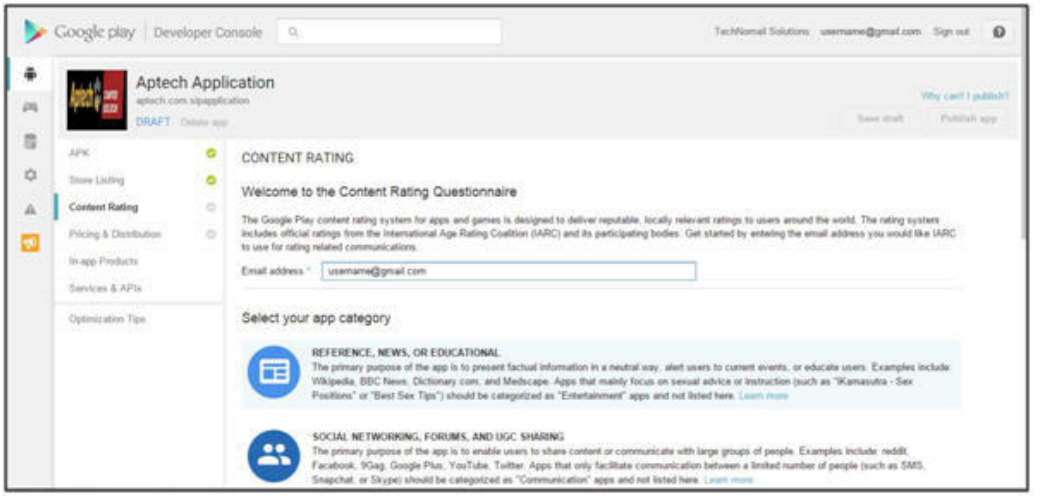


Figure 14.20: Content Rating Questionnaire

Finish the questionnaire and click save questionnaire as shown in figure 14.21.

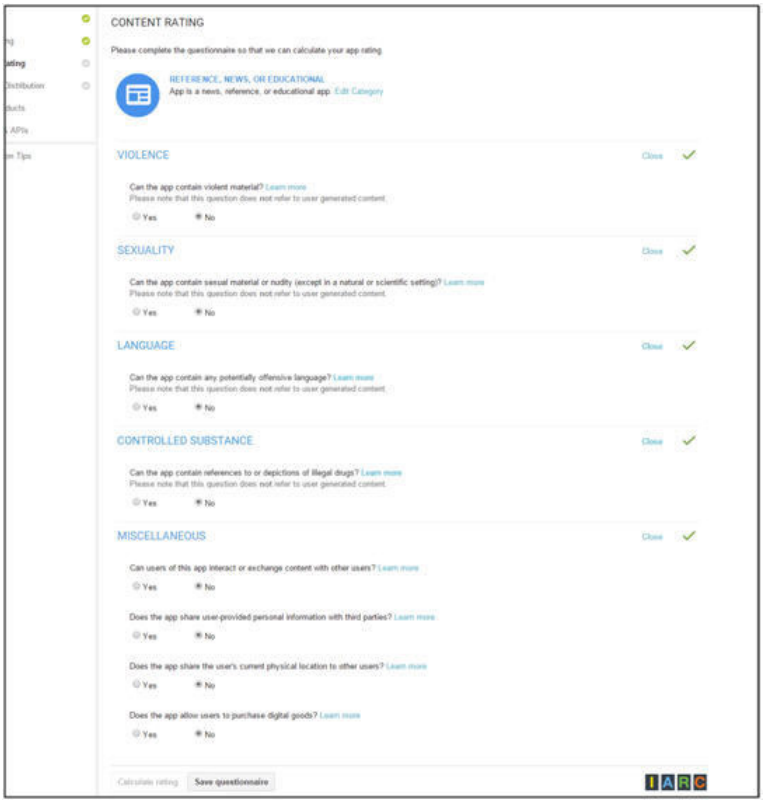


Figure 14.21: Answered Questionnaire

Click Calculate rating. The confirmation screen is displayed as shown in figure 14.22.

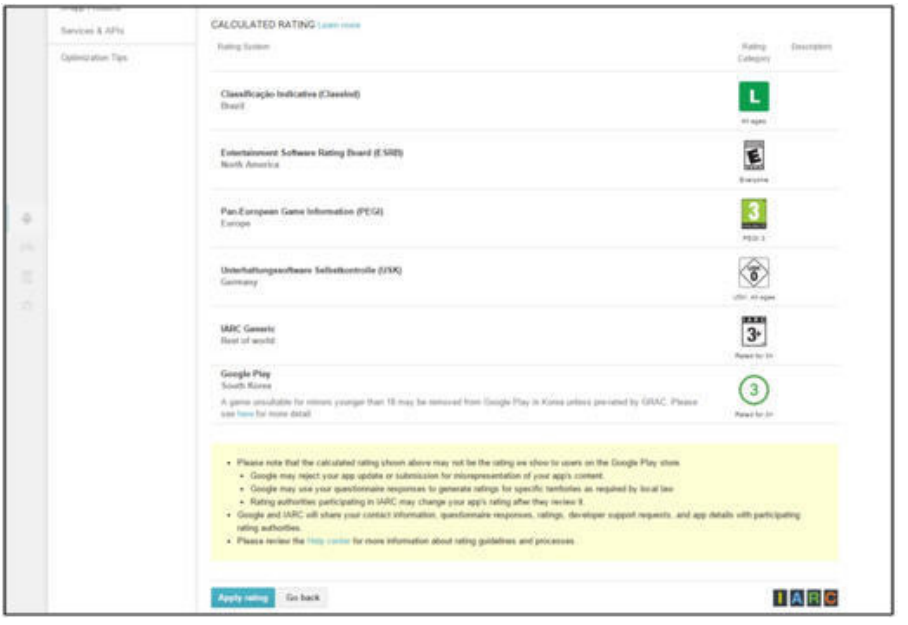


Figure 14.22: Generated Rating Output

Click Apply rating.

10. **Pricing and Distribution**

Click Pricing & Distribution tab in the left pane of the screen as shown in figure 14.23.

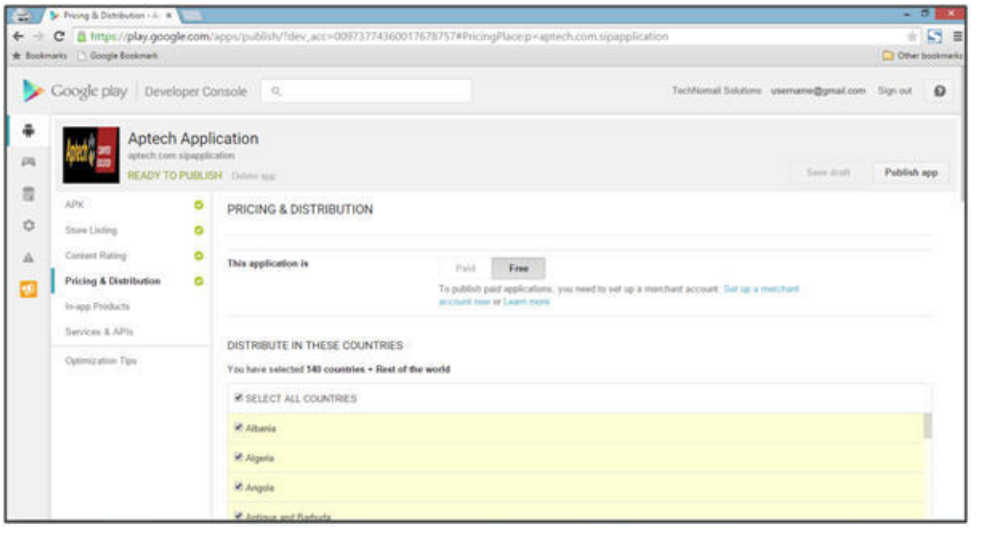


Figure 14.23: Pricing and Distribution Tab

This tab helps to set the price and name of the countries where the application can be distributed.

11. **Publish the Application**

Once all these steps are completed, the developer is ready to publish the application. Click Publish app. The App is published on the Play Store.

14.6 Best Practices

This section explains the steps that are advised to be followed by the developer before the application is published.

14.6.1 Setup a Developer Page

Play Store allows the developer to set up a developer page which displays developer/ publisher information and promotional material. This page can help to increase visibility and priority of the application in search results. Navigate to the SETTINGS → DEVELOPER PAGE as shown in figure 14.24.

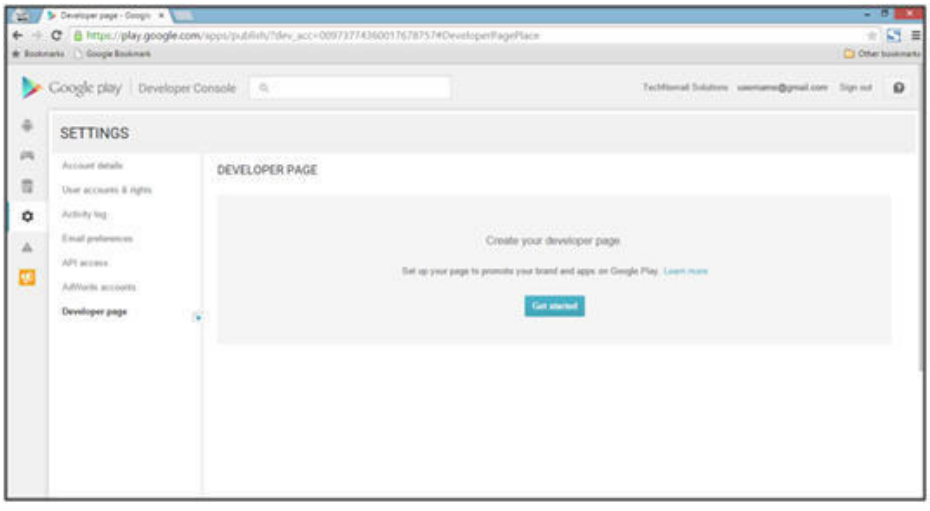


Figure 14.24: Developer Page

Click 'Get started' and enter the required information as shown in figure 14.25.

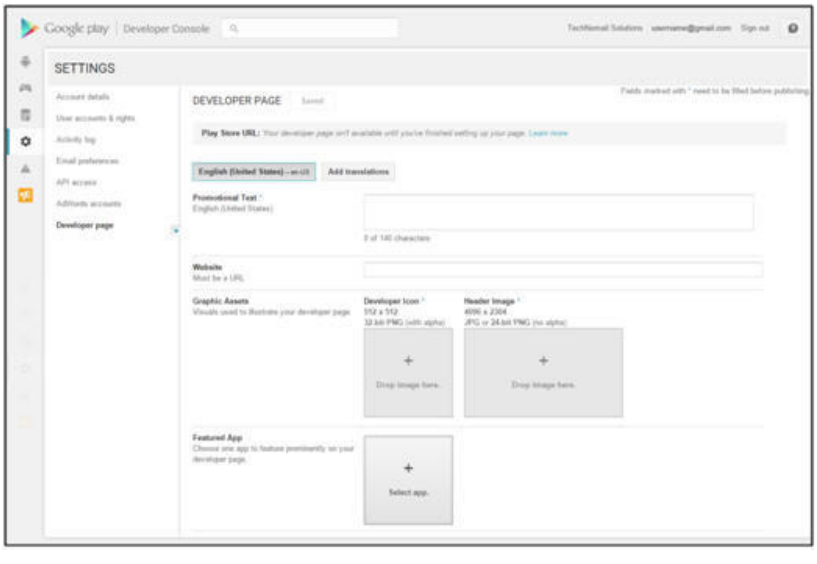


Figure 14.25: Developer Details

Click Save.

14.6.2 Upload and Publish for Testing

Play Store allows the developer/publisher to upload and publish app in Alpha or Beta testing phase before making it available to all users. The list of users who can access and test the application can be limited in the testing phases. Rating is also disabled during these phases. Participants can be added from within the organization during alpha phase. The developer can implement an open beta for all users. These user details can be collected using a Website or an on request basis. In order to upload an application in testing phase, during the Upload APK phase, choose the appropriate tab to upload the application for testing as shown in figure 14.26.

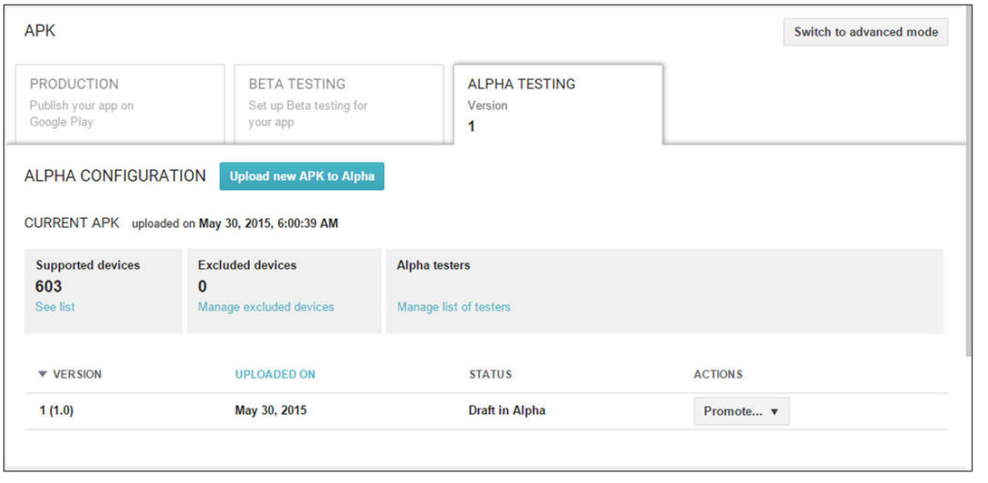


Figure 14.26: Testing Phase

Participants can be added by clicking Manage list of testers under the Alpha testers section. The screen as shown in figure 14.27 is displayed.

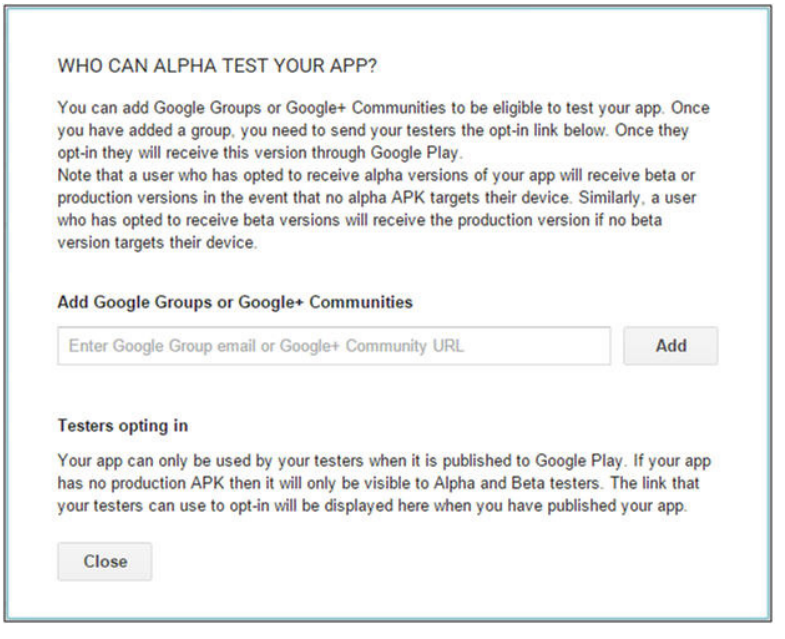


Figure 14.27: Adding Participants

14.6.3 Revenue Stream

Ultimately, the developer wants the effort put into the application to be profitable. However, setting a price on the app may not be the best way to achieve this. Payments are to be made on the app store and are not enabled in all countries. As a result, the market in those countries will be alienated. It is also recommended to offer a 'trial' for a limited time or a 'Free Version' with limited functionality. This builds reputation, increases potential customers, and increases the overall installation of the application. Note that 'In app' purchases for free apps is on a rise in the market. It may appear as if the revenue from this form of payment is less compared to premium applications. However, the publisher needs to keep in mind that the revenue is also limitless. While onetime payment on premium apps is only received once, 'In app' purchases continue to generate revenue forever.

14.7 Marketing and Promoting

In order for the application to be successful, it is very important to promote the application properly. The application market is pretty similar to any other market, where brand name has a huge impact. Building a reputable brand is a matter of hard work, dedicated support, and consistent quality.

14.7.1 Quality Assurance

The developer/publisher is required to pass the application through multiple rounds of Quality Assurance before the application is ready to be uploaded to the play store. All bugs need to be eliminated.

14.7.2 Customer Support

Customer support can go a long way towards building a good relationship with the user. Always try to have dedicated staff working on support tickets. Try to solve the issue as soon as possible.

14.7.3 Google AdWords

Google AdWords is Google's advertising platform. It allows the publisher to display ads for the application to target audience based on their interests. The payment is on a per-click basis. Hence, money invested into AdWords is guaranteed to have an impact.

14.7.4 Setup a Website

Setup a Website for the developer/publisher where all the applications developed have been listed. This makes the brand look sophisticated and builds confidence and trust. A support ticket system can also be implemented on the Website. The Website itself can be promoted using Google AdWords and Search Engine Optimization, which will in turn help to advertise the applications.

14.7.5 Reputation and Reviews

The most effective way to market a product is word of mouth. Nothing draws in customers like an advice from a good friend/relative. Try to maintain good relationships with your customers. Encourage them to submit reviews for the application on the Play Store. Whenever you receive a negative review, try to communicate with the user and try to improve. Well rated applications are also featured on the Play Store, which is the most effective marketing for your application.

Check Your Progress

1. Which of the following media are sold on the Play Store?

1.	Music	2.	E-Books
3.	Apps	4.	Newspapers

(A)	1, 2, 3	(B)	2, 3, 4
(C)	1, 3, 4	(D)	1, 2, 4

2. Which of the following statements about Google Play Store are true?

1.	Play Store was formerly known as Android Market.
2.	Play Store is operated by Google.
3.	There is no set up fee for publishing on Google Play Store.
4.	Play Store needs to be downloaded and installed separately on Android Devices.

(A)	1, 3	(B)	1, 2
(C)	2, 4	(D)	2, 3

3. Which of the following is not required to publish an application on the play store?.

(A)	Play Developer Console	(B)	Application
(C)	Privacy Policy	(D)	License Agreement

4. Google Play Store (in its original name) was started in the year _____.

(A)	2006	(B)	2008
(C)	2009	(D)	2011

Check Your Progress

5. Arrange the following steps to publish an application in the correct sequence.

1.	Create a signed .apk
2.	Upload Screen shots
3.	Specify Pricing
4.	Categorize details

(A)	1, 2, 3, 4	(B)	4, 2, 1, 3
(C)	2, 1, 4, 3	(D)	1, 2, 4, 3

Answers

1.	A
2.	B
3.	C
4.	B
5.	D

Session

14

Google Play Store

Summary

- The Google Play Store (formerly Android market) is known to be the very own repository of Google for Android applications.
- Play Store also sells music, movies, e-Books, and audio books.
- In order to publish applications on the play store, a Google Play Developer Console account is needed. A payment of 25 USD is required for this.
- The application needs to be signed and tested before uploading to the Play Store.
- Marketing and good reputation are key factors for the success of an application.
- In order to maximize the audience, the market shares and distributions need to be researched thoroughly.

15 Android Studio 2.1 and Android 6.0 Marshmallow

Welcome to the Session, **Android Studio 2.1 and Android 6.0 Marshmallow**.

This session explains the new features in Android Studio 2.1. It also explains the new features of Android 6.0 Marshmallow.

In this Session, you will learn to:

- Identify the updated features of Android Studio 2.1
- Explain the updated and new features of Android 6.0 Marshmallow

Session

15

Android Studio 2.1 and Android 6.0 Marshmallow

15.1 Introduction

Android Studio 2.1 is the latest release of Android Studio that enables developers to build good quality, high performance apps for the Android platform, including phones, tablets, Android Auto, and Android TV. This new and stable version of Android Studio has an improved interface and many enhanced features, such as fast build speeds and an accelerated emulator with support for the latest Android version and Google Play Services. Let us take a look at the updates and new features of Android Studio 2.1.

15.1.1 Updated Features in Android Studio 2.1

Android Studio 2.1 is a minor platform release after Android 2.0. This release includes new API changes and bug fixes. Table 15.1 describes some of the updated features.

Feature	Description
API Level	Level 7
Bluetooth	Bluetooth 2.1 New BT profiles: Object Push Profile (OPP) and Phone Book Access Profile (PBAP)
Built-in Applications	☐ Alarm Clock ☐ Browser ☐ Calculator ☐ Camera ☐ Contacts ☐ Custom Locale (developer app) ☐ Dev Tools (developer app) ☐ Email Gallery ☐ IMEs for Japanese, Chinese, and Latin text input ☐ Messaging ☐ Music ☐ Phone ☐ Settings ☐ Spare Parts (developer app)
Framework APIs	Android 2.1 includes several new developer APIs. For an overview of new APIs, see the Android 2.0 version notes.
Live Wallpapers	Following additions provide APIs for developing animated wallpapers: ☐ New android.service.wallpaper package. ☐ New WallpaperInfo class. ☐ Updated WallpaperManager.

Feature	Description
Media Framework	Revamped graphics architecture for improved performance that enables better hardware acceleration.
Telephony	☐ To get information about the device's current network signal, use the new SignalStrength class. This can be acquired from the new onSignalStrengthsChanged(SignalStrength) callback. ☐ New onDataConnectionStateChanged(int, int) callback.
Views	☐ New View methods: isOpaque()and onDrawScrollBars(Canvas). ☐ New RemoteViews methods: addView(int, RemoteViews)and removeAllViews(int). ☐ New ViewGroup methods: isChildrenDrawingOrderEnabled() and setChildrenDrawingOrderEnabled(boolean).
WebKit	☐ New WebStorage methods to manipulate Web storage databases. ☐ New GeolocationPermissions methods to get Geolocation permissions from, and set them on the WebView. ☐ New WebSettings methods to manage settings for app cache, Web storage, and zooming based on screen density. ☐ New WebChromeClient methods for handling video, browsing history, custom Views, app cache limits, and more.

Table 15.1: Updated Features of Android Studio 2.1

In addition, the platform provides following new features that Android developers can use to develop and build applications faster and more efficiently:

- **Instant Run**: This feature makes changes made to the app live instantly in the running app. This ensures faster builds for developers.
- **Android Emulator**: The new emulator is at least three times faster than the previous emulator. The enhanced Android Debug Bridge (ADB) ensures that the developers can deploy apps and data to the emulator at a much faster rate. In addition, the built-in Google Play Services enable the developer to test out more API functionality. Finally, the new emulator has rich new features to manage calls, battery, network, GPS, and more.
- **Cloud Test Lab Integration**: After writing the app code, developers can run and test

the app on a wide range of physical Android devices in the Cloud Test Lab from within Android Studio. This helps to make the apps more robust and foolproof.

- **App Indexing Code Generation and Test**: After the app is out in the market, developers would want their app to be visible in Google Search for users. The new App Indexing feature in Android Studio can help developers to add indexable URL links that they can test all within the IDE.
- **GPU Debugger Preview**: While developing OpenGL ES-based games or apps, the new GPU debugger displays each frame and the GL state. This helps a great deal in diagnosing GL rendering issues by capturing and analyzing the GPU stream from the Android device.
- **IntelliJ 15 Update**: Android Studio 2.1 comes with new features from IntelliJ, a world class coding platform packed with useful tools for code navigation, code inspection, and refactoring.

Let us next look at some of these features in more detail.

15.1.2 Instant Run

This feature of Android Studio 2.1 is a leap towards reducing the need to build and deploy an app every time changes are made. Instant Run provides pushing of updates, including code and resources, to a running instance of an app on a device or emulator without requiring a full reinstall.

While an app is running, the lightning bolt beside the Run button, as shown in figure 15.1, indicates that the Instant Run feature is available.

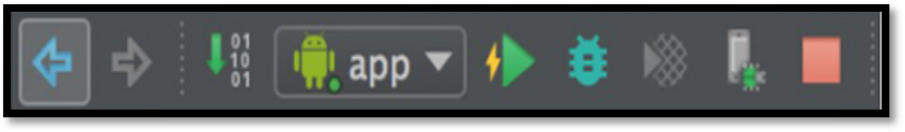


Figure 15.1: Instant Run

The Instant Run feature is enabled by default. To view this setting or to enable the feature manually, perform following steps:

1. Select Preferences → Build, Execution, Deployment → Instant Run.
2. Ensure that the Enable Instant Run to hot swap code/resource changes on deploy check box is selected.
3. Ensure that the Restart activity on code changes check box is clear. Refer to figure 15.2.

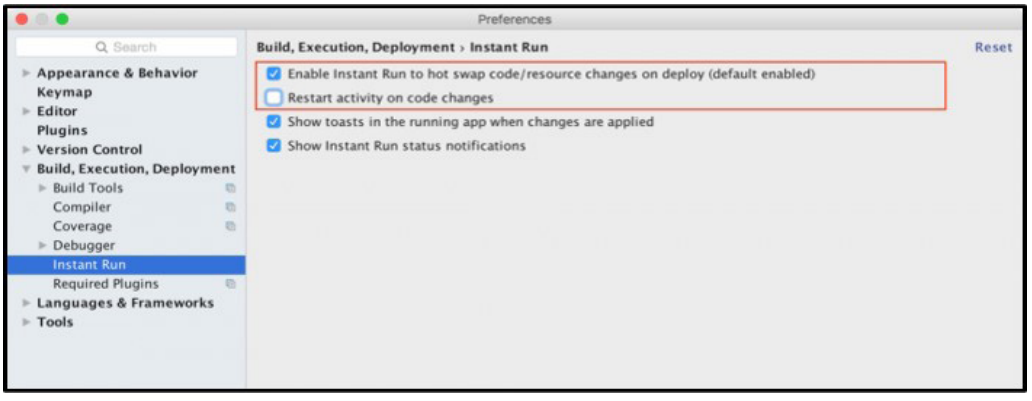


Figure 15.2: Viewing/Setting Instant Run Feature

Depending upon the extent and type of changes made to the app, Instant Run implements updates in three ways:

- **A hot swap**: In case changes are made to a method, the app will continue to run and use the changes next time that method is called. This process is called a hot swap.
- **A warm swap**: If the resources are modified, the app continues to run but restarts the current activity. This is called warm swap.
- **A cold swap**: In case of structural code changes, such as to inheritance or method signatures, the app will restart but will not reinstall. This is called cold swap.

To test the Instant Run feature, let us create an application named ViewFlipperExample that has two screens as shown in figure 15.3.

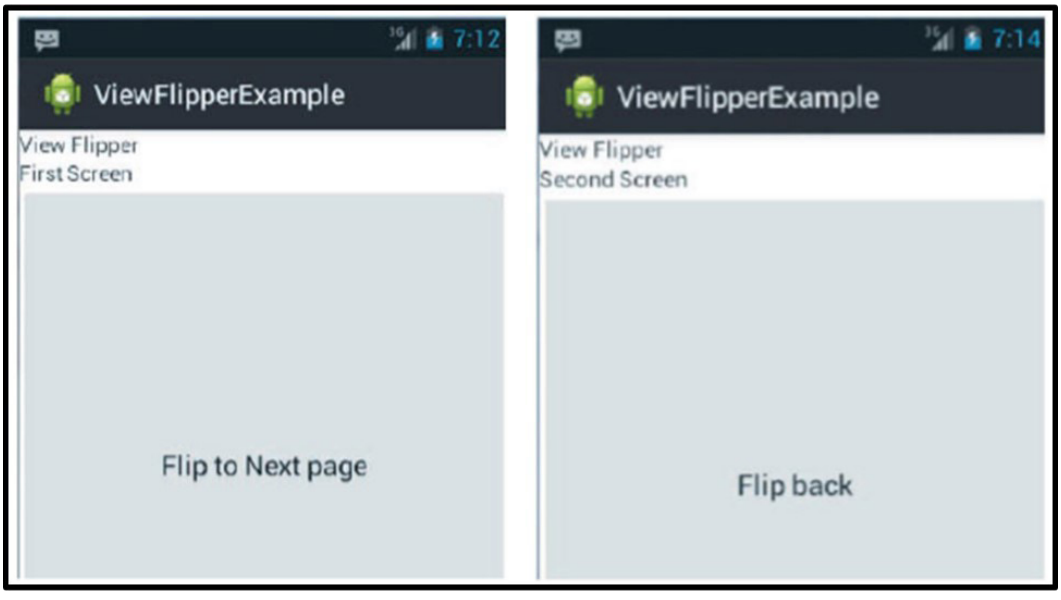


Figure 15.3: App Output

To create this application, perform the following steps:

1. Create a project named ViewFlipperExample in Android Studio.
2. Navigate to res folder.
3. Create a new folder under it named anim.
4. Create a new Android XML file named flipin under this.
5. Add the code in flipin.xml file as shown in Code Snippet 1.

Code Snippet 1:

```
<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/android"
android:interpolator="@android:anim/decelerate_interpolator" >
<translate
android:duration="500"
android:fromXDelta="-100%"
android:toXDelta="0%" />
</set>
```

6. Create another new Android XML file named flipout under res → xml.
7. Modify the code in flipout.xml file as shown in Code Snippet 2.

Code Snippet 2:

```
<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/android"
android:interpolator="@android:anim/decelerate_interpolator" >
<translate
android:duration="500"
android:fromXDelta="0%"
android:toXDelta="100%" /> </set>
```

8. Navigate to res → layout folder.
9. Modify the code in activity_main.xml file as shown in Code Snippet 3. The code uses ViewFlipper widget.

Code Snippet 3:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/
android" android:layout_width="fill_parent"
android:layout_height="fill_parent"
android:orientation="vertical">
<TextView android:layout_width="fill_parent"
android:layout_height="wrap_content" android:text="View Flipper"
/>
<ViewFlipper android:id="@+id/viewflipper"
android:layout_width="fill_parent"
android:layout_height="fill_parent" >
<LinearLayout android:layout_width="fill_parent"
android:layout_height="fill_parent"
android:orientation="vertical" >
<TextView android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="First Screen" />
<Button android:id="@+id/button1" android:layout_width=
"fill_parent" android:layout_height="fill_parent"
android:text="Flip to Next page" />
</LinearLayout>
<LinearLayout android:layout_width="fill_parent"
android:layout_height="fill_parent"
android:orientation="vertical" >
<TextView android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Second Screen" /><Button android:id="@+id/button2"
android:layout_width="fill_parent"
android:layout_height="fill_parent"
android:text="Flipback" /></LinearLayout></ViewFlipper>
</LinearLayout>
```

10. Add the code given in Code Snippet 4 into the MainActivity.java file.

Code Snippet 4:

```
package com.example.viewflipperexample;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.view.View;
import android.view.animation.Animation;
import android.view.animation.AnimationUtils;
import android.widget.Button;
import android.widget.ViewFlipper;
public class MainActivity extends Activity
{
@Override
protected void onCreate(Bundle savedInstanceState)
{
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
final ViewFlipper FlipIt = (ViewFlipper)
findViewById(R.id.viewflipper);


Button flip1 = (Button) findViewById(R.id.button1);
Button flip2 = (Button) findViewById(R.id.button2);
Animation FlipIn = AnimationUtils.loadAnimation (this, R.anim.
flipin);
Animation FlipOut = AnimationUtils.loadAnimation(this, R.anim.
flipout);
FlipIt.setInAnimation(FlipIn);
```

```
FlipIt.setOutAnimation(FlipOut);
flip1.setOnClickListener(new Button. OnClickListener()
{
@Override
public void onClick(View arg0)
{
// TODO Auto-generated method stub
FlipIt.showNext();
}
});
flip2.setOnClickListener(new Button. OnClickListener()
{
@Override
public void onClick(View arg0)
{
FlipIt.showPrevious();
}
});
@Override
public boolean onCreateOptionsMenu(Menu menu)
{
getMenuInflater().inflate(R.menu.main, menu);
return true;
}
}
```

Let us test a hot swap update first. To do this, in Code Snippet 4, replace the code for the onClick() method with the code given in Code Snippet 5.

Code Snippet 5:

```
public void onClick(View v)
{
//add code to display new message is displayed on selecting
// the action button
. . .
```

Add code in Code Snippet 5 that will ensure that the flip take place when the user clicks the START button. The flip back also happens when the user clicks the START button on the Flip back screen. Refer to figure 15.4.

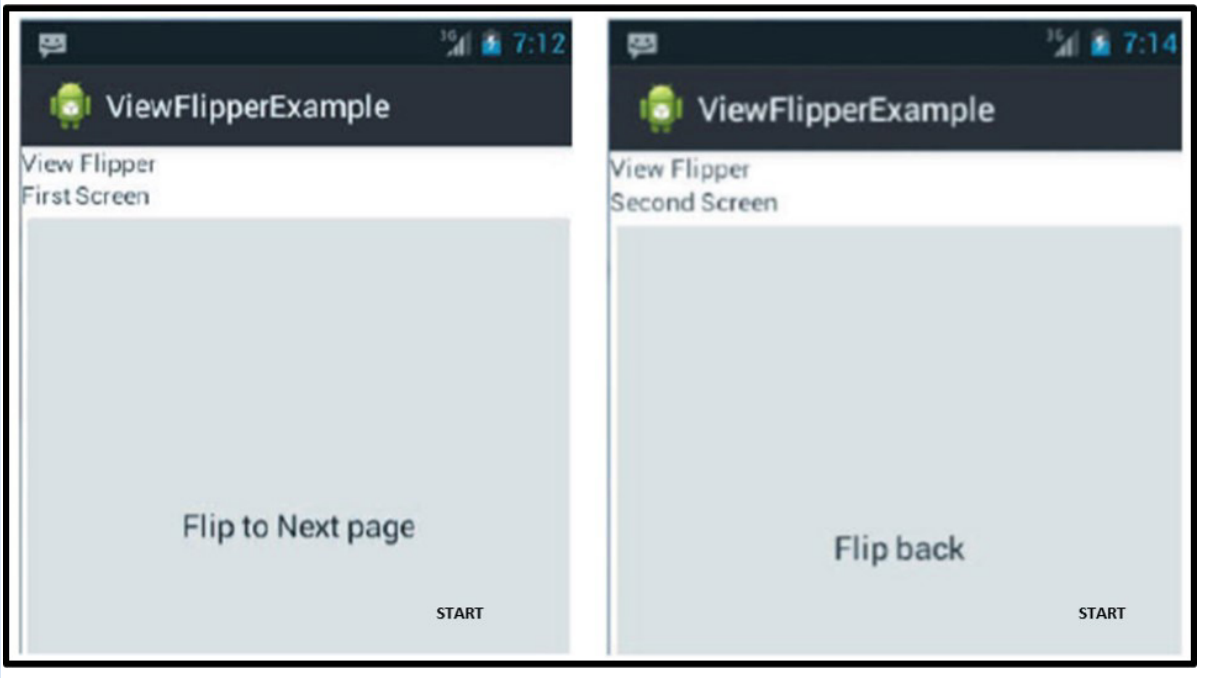


Figure 15.4: START Button

When the developer clicks the Run button, a toast will appear to notify that Instant Run applied code changes without activity restart.

Instant Run helps to make the app layout and styling more efficient, because the developer can instantly see how code changes result in UI changes. Let us look at an example using the same ViewFlipperExample app.

Suppose, the developer wants to change the gray color of the app background to pale yellow. This can be done using the code given in Code Snippet 6.

Code Snippet 6:

```
import android.view.View.OnClickListener;
import android.widget.Button;
import android.graphics.Color;
@Override
protected void onCreate(Bundle savedInstanceState)
{
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_new_contact);
      Button button = (Button)findViewById(R.id.button);
button.setOnClickListener(addButtonListener);
      RelativeLayout colorLayout = (RelativeLayout) findViewById(R.
id.colorLayout);
      colorLayout.setBackgroundColor(Color.YELLOW);
}
```

After adding this code and clicking the Run button, the screen will flicker, and a toast will be displayed notifying the developer that the activity has restarted. The background color of the app will change to yellow without a full build and install.

Note: Here, the activity restarted; therefore, this is a warm swap update.

For a cold swap update, the code given in Code Snippet 7 must be added to the code given in Code Snippet 6, after the onCreate() method.

Code Snippet 7:

```
/ A complete restart is triggered by a cold swap.
//Just add a new method after onCreate()
//for a complete restart of currently running application
public void demoMethod()
{
   }
/
```

15.1.3 Android Emulator

The new Android Emulator provides the Instant Run feature. This feature helps to instantly view the appearance of the app as changes are made to the code. It also populates builds much faster as compared to previous versions. The apps can be developed at a faster rate on the official Android Emulator as compared to a real device.

When compared with the previous versions of android emulator, the latest Android Emulator is up to three times faster in terms of I/O, RAM and CPU, thereby, providing enhanced performance. In addition, ADB push speeds are at least 10 times faster resulting in app being built at a much rapid rate.

The Android Emulator has an enhanced user interface. Some of the new interface features are as follows:

- The emulator can be resized by dragging the corner of the emulator window.
- The new emulator allows the developers to easily use the various multi-touch actions, such as zoom, pinch, pan, tilt, and rotate.
- A new toolbar is added to the emulator which provides easy-access controls for the emulator, a screenshot button and a screen rotation button.
- APKs can be quickly installed by simply dragging and dropping them.
- Some extra features that were available only through command line in the earlier versions are now available in GUI version. Some of these features are: SMS Sending, Fingerprint, Phone Calling, and so on.

One of the new Extended Control Features in the new Android emulator is the battery status and charging state.

Code Snippet 8 displays the battery status in percentage on screen.

Code Snippet 8:

```
package com.example.test;

import java.io.IOException;

import android.media.MediaPlayer;

import android.os.Bundle;

import android.app.Activity;

import android.content.BroadcastReceiver;

import android.content.Context;
```

```
import android.content.Intent;
import android.content.IntentFilter;
import android.content.res.AssetFileDescriptor;
import android.widget.ProgressBar;


 setContentView(R.layout.activity_main);
     registerReceiver(mbcr,new  IntentFilter(Intent.ACTION_BATTERY_
CHANGED));
   }


}
```

To display the battery status on screen, add the code in Code Snippet 9 to the XML file.

Code Snippet 9:

```
<LinearLayout  xmlns:android="http://schemas.android.com/apk/res/
android"
  android:layout_width="fill_parent"
  android:layout_height="fill_parent"
  android:orientation="vertical"
  android:background="#abc" >
<TextView
  android:id="@+id/textView1"
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:text=""
android:textSize="25sp"
  android:layout_gravity="center"
/>
```

```
<ProgressBar
  android:id="@+id/progressBar1"
  style="?android:attr/progressBarStyleHorizontal"
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:layout_gravity="center"
  android:minHeight="100dp"
  android:minWidth="200dp" />
</LinearLayout>
```

To make changes in the battery status:

1. Open the Extended controls window and choose the Battery tab. This tab includes various options for changing the battery status.

2. Select the Charge level slider to change the battery level. There is an immediate refection of the new battery status change in the status bar of the emulator.

3. Choose Full from the Battery status drop-down list for setting the battery status explicitly. The app detects the change and displays the new status immediately.

Developers can use the Location tab in the Extended controls window, to manually update the device location.

15.1.4 Cloud Test Lab Integration

Google data centers host a wide range of device configurations and physical devices. A new service called Cloud Test Lab helps developers to test their apps across those device configurations and physical devices. It enables the developers to conduct basic set of crash tests, directly from the Android Studio, even if they have not written tests explicitly for it.

The Run/Debug Configurations interface is a new interface that can be used to configure the portfolio of tests when the developers want to run it on the Cloud Test Lab, in Android Studio, as shown in figure 15.5.

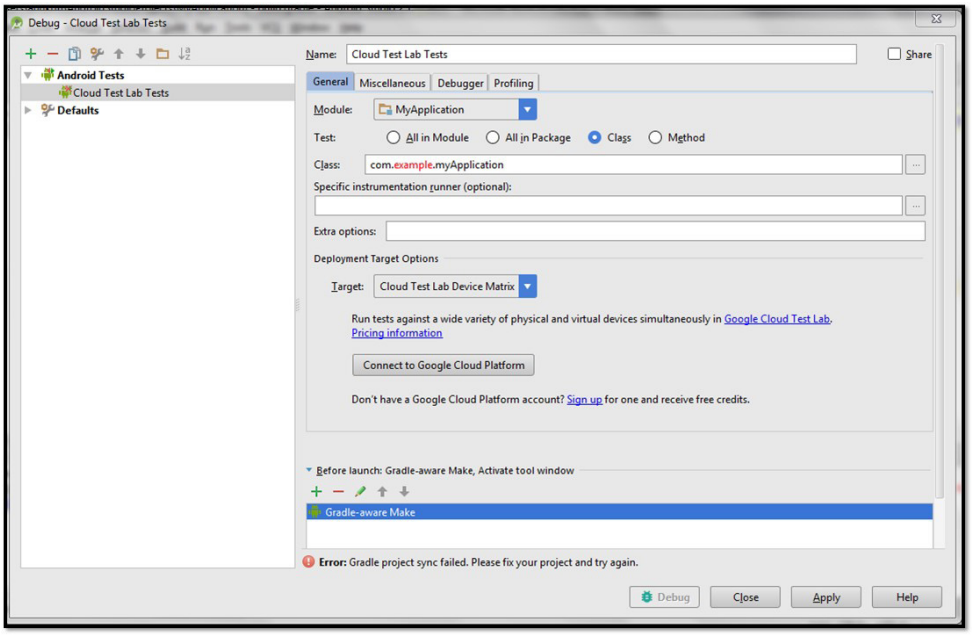


Figure 15.5: Run/Debug Configurations Interface

This interface also shows the results of the tests.

15.1.5 App Indexing Code Generation and Test

An app can be made visible to users in Google Search with the help of App Indexing API. To do this, attributes needs to be added to the AndroidManifest.xml file. The Google App Indexing service helps to list the app in the search results. This service directly works with the AndroidManifest.xml file.

Developers can add the correct URL structure in their app code.

To validate the app indexing code, developers need to perform following steps:

1. Click Tools
2. Select Android and choose Google App Indexing Test option, as shown in figure 15.6.

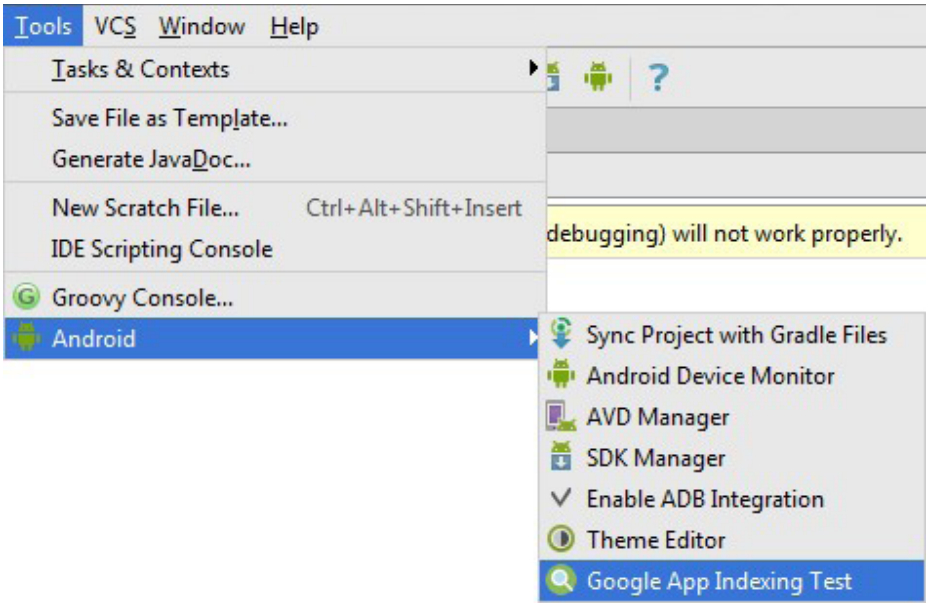


Figure 15.6: Google App Indexing Test Option

15.2 Android 6.0 Marshmallow

Released on September 29, 2015, Android 6.0 Marshmallow refines and extends the core features and functionality of Android Lollipop. However, it also includes a variety of system changes and API behavior changes.

15.2.1 Updated Features

Following features highlight some of the key changes that developers of Android apps must be aware of:

- **Doze**: To save power during idle time, Android 6.0 Marshmallow ensures that if a user leaves the device stationary, unplugged and with its screen off, for a period of time, the device goes into Doze or sleep mode. While in the doze mode, the device system only allows app syncing and any pending system operations to take place for brief periods of time.
- **App standby**: This feature allows the system to disable network access and suspend syncs and jobs for any apps that the user has not used a certain period of time. This helps to save battery as well as network bandwidth.
- **Enhanced HTTP support**: Android 6.0 release replaces the support for the Apache HTTP client with a more efficient API using the HttpURLConnection class. This API reduces network use through transparent compression and response caching, and minimizes power consumption.
- **BoringSSL**: Android 6.0 Marshmallow uses BoringSSL library instead of OpenSSL. To avoid linking to libraries that may have changed or been removed and to avoid any security vulnerabilities, the developer must either statically link against a cryptography library or modify the native code to call the Java cryptography APIs

via JNI.

- **Access to hardware identifier**: To ensure enhanced data protection, Marshmallow does not allow programmatic access to the device's local hardware identifier for apps using the Wi-Fi and Bluetooth APIs any longer. If the app needs to access the hardware identifiers of nearby external devices via Bluetooth and Wi-Fi scans, it needs to have the ACCESS_FINE_LOCATION or ACCESS_COARSE_LOCATION permissions:
  - WifiManager.getScanResults()
  - BluetoothDevice.ACTION_FOUND
  - BluetoothLeScanner.startScan()

15.2.2 New Features

In addition to the updates discussed so far, Android 6.0 Marshmallow has come with a variety of new features. Let us learn about few important ones here.

Android Run Time Permissions

Android 6.0 Marshmallow enables the users to directly manage app permissions at runtime. Users can now grant or revoke permissions individually for installed apps.

Apps that target Android 6.0 or higher, should be tested under the new permissions model. As per the new permission model:

- If the app lists a normal permission in its manifest, the system grants the permission automatically. To determine if the app has been granted a permission, the developer must call the new checkSelfPermission() method. If the app has the permission, the method returns PackageManager.PERMISSION_GRANTED, and the app can proceed with the operation. If the app does not have the permission, the method returns PERMISSION_DENIED, and the app has to explicitly ask the user for permission.
- If the app asks for dangerous permission, that is, access to the user's confidential data, the user has to explicitly give approval to the app. To request permissions for the app at runtime, the developer must call the new requestPermissions() method.

The code given in Code Snippet 10 checks if the app has permission to read the user's contacts and requests the permission if necessary. Note that the app cannot configure or alter the dialog box that is shown to the user when the app calls requestPermissions() method. If the developers need to provide any information or explanation to the user, they should do that before calling the requestPermissions() method.

Code Snippet 10:

```
// Here, thisActivity is the current activity

if (ContextCompat.checkSelfPermission(thisActivity, Manifest.
permission.READ_CONTACTS)!= PackageManager.PERMISSION_GRANTED)

{

/* Check whether explanation needs to be shown */

     if(ActivityCompat.
shouldShowRequestPermissionRationale(thisActivity, Manifest.
permission.READ_CONTACTS))

     {

/* Show an explanation to the user and wait for the user's
response

After user has noticed the explanation, request permission
again.*/

    }

     else

     {

// Request permission directly without explanation.

        ActivityCompat.requestPermissions(thisActivity, new
String[]{Manifest.permission.READ_CONTACTS}, GRANT_REQUESTS_TO_
READ_CONTACTS);

/* GRANT_REQUESTS_TO_READ_CONTACTS is an app-defined int
constant.

The callback method gets the result of the request. */

    }

}

/* When user responds, the system passes response to the app's
onRequestPermissionsResult() method. */

@Override

public void onRequestPermissionsResult(int codeRequest, String
accessRights[], int[] accessResults)

{

     switch (codeRequest)
```

```
      {
            case GRANT_REQUESTS_TO_READ_CONTACTS:
            {
// If request is cancelled, the result arrays are empty.
             if (accessResults.length > 0 && accessResults[0] ==
PackageManager.PERMISSION_GRANTED)
                  {
/* The app got the permission. Continue with the functionality
that depends on this permission. */
                  }
                  else
                  {
/* The app did not get the permission. Disable the functionality
that depends on this permission. */
                  }
                  return;
            }
       }
}
```

By default, the user only needs to grant permission once for each permission group. If the app requests any other permissions in that group, the system automatically grants them. However, the app must still explicitly request every permission it needs, even if the user has already granted another permission in the same group. This is because the app code should not rely on any assumed permission groups that may change in future Android releases.

Android Data Binding Library

Data Binding Library is a support library that helps developers to build rich and responsive user layouts and experiences with minimal effort. These libraries have pre-defined sets of boilerplate code for common layouts and functionality, which an app developer can use to plug those features in their apps with minimal code.

Developers can use the Data Binding Library with all Android platform versions back to Android 2.1 (API level 7+). To use data binding, the developers will require Android Plugin for Gradle 1.5.0-alpha1 or higher.

To configure the app to use data binding, the developer must perform following steps:

1. Download the library from the Support repository in the Android SDK manager.
2. Add the dataBinding element to the build.gradle file in the app module.

3. Use the code given in Code Snippet 11 to configure data binding.

Code Snippet 11:

```
android
{
      ....
      dataBinding
      {
         enabled = true
      }
}
```

Code Snippet 12 shows a layout file without data binding.

Code Snippet 12:

```
<LinearLayout .....">
      <TextView
              android:layout_width="wrap_content"
              android:layout_height="wrap_content"
              android:text="DataBound"
              android:id="@+id/dataBoundText"
              android:layout_centerVertical="true"
              android:layout_centerHorizontal="true"/>
</LinearLayout>
```

To configure data binding, the developer needs to add a new class to the project, called DataBindingHelper, as shown in Code Snippet 13. This class will bind elements with data.

Code Snippet 13:

```
package com.databindingexample;
public class DataBindingHelper
{
      private String information;
      public static DataBindingHelper get(String information)
```

```
      {
            return new DataBindingHelper(information);
      }
      public DataBindingHelper(String information)
      {
            this.information = information;
      }
       public String getMessage()
      {
            return String.format("Welcome %s", information);
      }
}
```

The Data binding library is used. The layout files will have the root tag of layout and this is accompanied by a view root element and data element, as shown in Code Snippet 14.

Code Snippet 14:

```
<?xml version="1.0" encoding="utf-8" ?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/
android"
    android:id="@+id/Layout01"
    android:layout_width="match_parent"
    android:layout_height="match_parent" >
    <data>
        <variable
         name="databindingidentifier"
            type="com.databindingexample"
        />
    </data>
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@{databindingidentifier.message}"
```

```
        android:layout_centerVertical="true"
        android:layout_centerHorizontal="true"
    />
</LinearLayout>
```

The androidmanifest.xml must declare the activities. Data Binding allows a developer to write expressions for handling events that are initiated from the layout; for example, one such event is onClick().

There are two ways to handle an event:

- **Method References**: In expressions, a developer can reference methods that conform to the signature of the listener method. For example, android:onClick() can be assigned to a method in an Activity. The expression is processed at compile time, so if the method does not exist or its signature is not correct, a compile-time error is thrown. The parameters of the method must match the parameters of the event listener. When an expression evaluates to a method reference, Data Binding wraps the method reference and owner object in a listener, and sets that listener on the target view. If the expression evaluates to null, Data Binding sets a null listener. Code Snippet 15 shows how method references can be used. The code needs to be added to the XML file of the activity where the button is present.

Code Snippet 15:

```
<Button …
        android:onClick="@{isTrainer  ?  handler.trainerClick  :
handler.studentClick}" />
// First search the setOnClick() method whenever event is fired by
clicking on the button.
@BindingMethods
({@BindingMethod(type = View.class, attribute = "android:onClick",
method = "setOnClickListener"})
// Search the OnClickListener  for abstract method
void onClick(View v1);
// Search the View for setOnClickListener
void setOnClickListener(View.OnClickListener l)
```

- **Listener Bindings**: Unlike in case of method references, in this case, Data Binding always creates a listener, which it sets on the view. When the event is dispatched, the listener evaluates the binding or lambda expression. The return value must match the expected return value of the listener, unless it is expecting void.

By extending the code shown in Code Snippet 15, we can say that onClick implies setOnClickListener. This listener is used for handling events.

Code Snippet 16 shows how Listener bindings can be used.

Code Snippet 16:

```
static class Implemantion1OnClickListener implements
OnClickListener
{
      public Handler mouseHandler;
      @Override
      public void onClick(android.view.View arg0)
      {
            mouseHandler.trainerClick(arg0);
      }
}
static class Implemantion2OnClickListener implements
OnClickListener
{
      public Handler mouseHandler;
      @Override
      public void onClick(android.view.View arg0)
      {
            mouseHandler.studentClick(arg0);
      }
}
```

Finger Print and Payments

These APIs help developers to incorporate fingerprint scan-based authentication and accelerate payments and sign-in operations in their app.

Let us assume that a developer wants to include the Fingerprint feature in the app so that users can use fingerprint authentication for making an online payment.

To implement this feature, the developer needs to perform following steps:

1. Ensure that the app is running on a compatible device with a fingerprint sensor.
2. Include the standard Android fingerprint icon in the UI.
3. Add the USE_FINGERPRINT permission in the manifest, as shown in Code Snippet 17.

Code Snippet 17:

```
<uses-permission
        android:name="android.permission.USE_FINGERPRINT" />
```

Code Snippet 18 explains fingerprint authentication for making online purchase.

Code Snippet 18:

```
package com.example.android.fingerprintdialog;
import android.app.Activity;
import android.app.DialogFragment;
import android.content.SharedPreferences;
import android.hardware.fingerprint.FingerprintManager;
import android.os.Bundle;
import android.view.KeyEvent;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.view.inputmethod.EditorInfo;
import android.view.inputmethod.InputMethodManager;
import android.widget.Button;
import android.widget.CheckBox;
import android.widget.EditText;
import android.widget.ImageView;
import android.widget.TextView;
import javax.inject.Inject;
public class FingerprintAuthenticationDialogFragment extends
DialogFragment
implements TextView.OnEditorActionListener, FingerprintUiHelper.
Callback {
    private Button mCancelButton;
    private Button mSecondDialogButton;
    private View mFingerprintContent;
    private View mBackupContent;
```

```
    private EditText mPassword;
    private CheckBox mUseFingerprintFutureCheckBox;
    private TextView mPasswordDescriptionTextView;
    private TextView mNewFingerprintEnrolledTextView;
    private Stage mStage = Stage.FINGERPRINT;
    private FingerprintManager.CryptoObject mCryptoObject;
    private FingerprintUiHelper mFingerprintUiHelper;
    private MainActivity mActivity;
    @Inject FingerprintUiHelper.FingerprintUiHelperBuilder
mFingerprintUiHelperBuilder;
    @Inject InputMethodManager mInputMethodManager;
    @Inject SharedPreferences mSharedPreferences;
    @Inject
    public FingerprintAuthenticationDialogFragment() {}
    @Override
    public void onCreate(Bundle savedInstanceState) {
// During re-instantiated of activity do not a create a new
fragment
        setRetainInstance(true);
         super.onCreate(savedInstanceState);
           setStyle(DialogFragment.STYLE_NORMAL, android.R.style.
Theme_Material_Light_Dialog);
    }
    @Override
      public View onCreateView(LayoutInflater inflater, ViewGroup
container,
   Bundle savedInstanceState) {
   getDialog().setTitle(getString(R.string.sign_in));
   View v = inflater.inflate(R.layout.fingerprint_dialog_container,
```

```
container, false);
        mCancelButton = (Button) v.findViewById(R.id.cancel_
button);
        mCancelButton.setOnClickListener(new View.
OnClickListener() {
            @Override
            public void onClick(View view) {
                dismiss();
            }
        });
        mSecondDialogButton = (Button) v.findViewById(R.id.second_
dialog_button);
        mSecondDialogButton.setOnClickListener(new View.
OnClickListener() {
            @Override
            public void onClick(View view) {
                if (mStage == Stage.FINGERPRINT) {
                    goToBackup();
                } else {
                    verifyPassword();
                }
            }
        });
        mFingerprintContent = v.findViewById(R.id.fingerprint_
container);
        mBackupContent = v.findViewById(R.id.backup_container);
        mPassword = (EditText) v.findViewById(R.id.password);
        mPassword.setOnEditorActionListener(this);
        mPasswordDescriptionTextView = (TextView) v.findViewById(R.
id.password_description);
```

```
        mUseFingerprintFutureCheckBox = (CheckBox)
         v.findViewById(R.id.use_fingerprint_in_future_check);
              mNewFingerprintEnrolledTextView = (TextView)


                 v.findViewById(R.id.new_fingerprint_enrolled_
description);
         mFingerprintUiHelper = mFingerprintUiHelperBuilder.
build(
                 (ImageView) v.findViewById(R.id.fingerprint_icon),
                 (TextView) v.findViewById(R.id.fingerprint_
status), this);
         updateStage();


         // If fingerprint authentication is not available, switch
immediately to the backup
         // (password) screen.
         if (!mFingerprintUiHelper.isFingerprintAuthAvailable())
{
             goToBackup();
         }
         return v;
     }


     @Override
     public void onResume() {
         super.onResume();
         if (mStage == Stage.FINGERPRINT) {
             mFingerprintUiHelper.startListening(mCryptoObject);
         }
     }
```

```
    public void setStage(Stage stage) {
        mStage = stage;
    }

    @Override
    public void onPause() {
        super.onPause();
        mFingerprintUiHelper.stopListening();
    }

    @Override
    public void onAttach(Activity activity) {
        super.onAttach(activity);
        mActivity = (MainActivity) activity;
    }

    /**
     * Sets the crypto object to be passed in when authenticating
with fingerprint.
     */
    public void setCryptoObject(FingerprintManager.CryptoObject
cryptoObject) {
        mCryptoObject = cryptoObject;
    }
    // After Unsccessfull attempt for authenticating lets switch to
the other authentication method
    private void goToBackup() {
        mStage = Stage.PASSWORD;
        updateStage();
```

```
        mPassword.requestFocus();
        // Enable keyboard.
        mPassword.postDelayed(mShowKeyboardRunnable, 500);
       // Fingerprint is not used anymore. Stop listening for it.
        mFingerprintUiHelper.stopListening();
    }
    //After Successful authentication dismisses the dialog
    private void verifyPassword() {
        if (!checkPassword(mPassword.getText().toString())) {
            return;
        }
        if (mStage == Stage.NEW_FINGERPRINT_ENROLLED) {
            SharedPreferences.Editor editor = mSharedPreferences.
edit();
             editor.putBoolean(getString(R.string.use_fingerprint_
to_authenticate_key),
                    mUseFingerprintFutureCheckBox.isChecked());
            editor.apply();

            if (mUseFingerprintFutureCheckBox.isChecked()) {
               // Re-create the key so that fingerprints including
new ones are validated.
                mActivity.createKey();
                mStage = Stage.FINGERPRINT;
            }
        }
        mPassword.setText("");
        mActivity.onPurchased(false /* without Fingerprint */);
        dismiss();
```

```
    }
    /**
     * @return true if {@code password} is correct, false otherwise
     */
    private boolean checkPassword(String password) {
       // Authentication process needs to be verified at server side.
        return password.length() > 0;
    }

    private final Runnable mShowKeyboardRunnable = new Runnable() {
        @Override
        public void run() {
            mInputMethodManager.showSoftInput(mPassword, 0);
        }
    };
    private void updateStage() {
        switch (mStage) {
            case FINGERPRINT:
                mCancelButton.setText(R.string.cancel);
               mSecondDialogButton.setText(R.string.use_password);
                mFingerprintContent.setVisibility(View.VISIBLE);
                mBackupContent.setVisibility(View.GONE);
                break;
            case NEW_FINGERPRINT_ENROLLED:
                // Intentional fall through
            case PASSWORD:
               mCancelButton.setText(R.string.cancel);
mSecondDialogButton.setText(R.string.ok);
                mFingerprintContent.setVisibility(View.GONE);
```

```
                                    mBackupContent.
setVisibility(View.VISIBLE);
                if (mStage == Stage.NEW_FINGERPRINT_ENROLLED) {
                    mPasswordDescriptionTextView.
setVisibility(View.GONE);
                    mNewFingerprintEnrolledTextView.
setVisibility(View.VISIBLE);
                    mUseFingerprintFutureCheckBox.
setVisibility(View.VISIBLE);
                }
                break;
        }
    }
    @Override
    public boolean onEditorAction(TextView v, int actionId,
KeyEvent event) {
        if (actionId == EditorInfo.IME_ACTION_GO) {
            verifyPassword();
            return true;
        }
        return false;
    }
    @Override
    public void onAuthenticated() {
        // After the authentication was successfull, call the
UiHelper
        mActivity.onPurchased(true /* withFingerprint */);
        dismiss();
    }
@Override
    public void onError() {
        goToBackup();
```

```
        }
    //Types of Authentication Methods which are used to authenticate
the user.
    public enum Stage {
        FINGERPRINT,
        NEW_FINGERPRINT_ENROLLED,
        PASSWORD
    }
}
```

When allowing the users to make payments through the app using Fingerprint authentication, the developers may want to include an additional security feature. The developers can have their apps authenticating users based on how recently they last unlocked their device by implementing a public or secret key for user authentication. To set the timeout duration for which the same key can be re-used after a user is successfully authenticated, call the new setUserAuthenticationValidityDurationSeconds() method when setting up a KeyGenerator or KeyPairGenerator.

After the timeout expires, use the createConfirmDeviceCredentialIntent() method to re-authenticate the user within the app.

Code Snippet 19 can be inserted by extending the earlier example to include Confirm Credential Functionality.

Code Snippet 19:

```
private void showAuthenticationScreen()
{
      // Description and Title are customizable, this
      // creates the Confirm Credentials screen Or if it is left
null,
      // a generic one shall be provided
      Intent i = mKeyguardManager.
createConfirmDeviceCredentialIntent(null, null);
      if (i != null)
      {  startActivityForResult(i,  REQUEST_CODE_CONFIRM_DEVICE_
CREDENTIALS);
      }
}
```

Direct Share

Marshmallow provides the ability to define direct share targets, such as contacts within other apps. This feature allows users to share content with a friend or community defined in another app, such as a social network app.

To enable direct share targets, developers need to perform following steps:

1. Define a class that extends the ChooserTargetService class. Declare this service in the manifest, as shown in Code Snippet 20.

Code Snippet 20:

```
<service android:name=".ChooserTargetService" android:label="@
string/service_name" android:permission="android.permission.
BIND_CHOOSER_TARGET_SERVICE">
     <intent-filter>        <action android:name="android.
service.chooser.ChooserTargetService" />
     </intent-filter>
</service>
```

Note: Within the declaration, the developer needs to specify the BIND_CHOOSER_TARGET_SERVICE permission and an intent filter using the SERVICE_INTERFACE action.

2. For each activity that needs to be exposed to ChooserTargetService, add a <meta-data> element with the name "android.service.chooser.chooser_target_service" in the app manifest, as shown in Code Snippet 21.

Code Snippet 21:

```
<activity android:name=".MyShareActivity"
        android:label="@string/share_activity_label">
    <intent-filter>
        <action android:name="android.intent.action.SEND" />
    </intent-filter>
    <meta-data
            android:name="android.service.chooser.chooser_target_
service"
        android:value=".ChooserTargetService" />
</activity>
```

Check Your Progress

1. A developer has modified a method in the app and clicked the Instant Run button. What will happen?

(A)	The app will immediately show the changes	(B)	The app will show the changes the next time that method is called
(C)	The app will restart the current activity to show the changes	(D)	The app will restart and reinstall before showing the changes

2. In which of the following update types, will the app restart on clicking Instant Run?

(A)	Cold swap	(B)	Warm swap
(C)	Hot swap	(D)	Extra hot swap

3. Which of the following features or services of Android Studio 2.1 enables developers to test their apps across a wide range of physical devices and device configurations hosted in Google's data centers?

(A)	Instant Run	(B)	Android Emulator
(C)	Cloud Test Lab Integration	(D)	App Indexing Code Generation and Test

4. A developer wants an app to be visible in Google Search for the users. Which of the following features or services of Android Studio 2.1 enables the developer to do that?

(A)	Cloud Test Lab Integration	(B)	Android Emulator
(C)	Instant Run	(D)	App Indexing Code Generation and Test

Answers

1.	B
2.	A
3.	C
4.	D

Session

15

Android Studio 2.1 and Android 6.0 Marshmallow

Summary

- Android Studio 2.1 has many new features to develop applications faster and efficiently.
- The Instant Run feature makes changes made to the app live instantly in the running app.
- The new emulator has rich new features to manage calls, battery, network, GPS, and more.
- The Cloud Test Lab Integration feature allows developers to run and test the app on a wide range of physical Android devices in the Cloud Test Lab from within Android Studio.
- The new App Indexing feature in Android Studio allows the developers to add indexable URL links that they can test all within the IDE.
- Android 6.0 Marshmallow has doze and app standby features to save power and resources.
- Fingerprint and Confirm Credential APIs help developers to incorporate fingerprint scan-based authentication and accelerate payments and sign-in operations in the app.
- The Direct Share feature of Marshmallow allows developers to define direct share targets, such as contacts within other apps.

16

Material Design

Welcome to the Session, **Material Design**.

This session explores material design, its features, and guidelines. The session also describes the various material design elements such as themes, list and cards, view shadows, and animations.

In this Session, you will learn to:

- Explain material design
- Describe the material design environment elements
- Describe various material design elements

Session

16

Material Design

16.1 Introduction

Material design combines good and classic design principles with some advanced tools to create a single underlying system across various platforms and mobile devices. It is also considered as a visual language or design language developed by Google for the application developers. It explains the relation between visual, motion, and interaction design. Material design uses various grid based layouts, animations, lighting effects, and shadows.

16.2 Introduction to Material and Material Environment

Google introduced a new UI design philosophy called Material Design. The material design specifies a set of design guidelines to maintain a look, consistent with the base operating system. Material design has a three-dimensional environment. It contains light, material, and cast shadows.

Table 16.1 lists the material design environment elements.

Element	Description
Objects	☐ Material objects have three dimensions such as x, y, and z. ☐ Material sheets within the objects have a standard thickness of 1dp (density-independent pixel).
Shadows	☐ Virtual lights illuminate the material environment. ☐ Difference in the elevations between the overlapping material objects creates shadows in the environment. ☐ Key lights create directional shadows. ☐ Ambient lights create soft shadows in the environment.

Table 16.1: Environment Elements

Materials have certain built-in characteristics such as:

- ☐ They are solid by nature.
- ☐ They move along any axis in the plane.
- ☐ They never bend or fold.
- ☐ They can change their shape.
- ☐ They can cast shadows.
- ☐ They have uniform thickness of **1 dp**.
- ☐ They have varying x and y dimensions.
- ☐ They cannot be penetrated.
- ☐ They can easily join with other materials in the environment.
- ☐ They can be created.
- ☐ They can be destroyed.

Session 16

Material Design

Material design objects or elements possess elevations. Elevations indicate the distance between the depth of the surface and the shadow.

Following are the different types of elevations in material objects:

- **Resting elevation**: The default elevation of an object that does not change under any circumstance is known as resting elevation.
- **Component elevation**: The consistent resting elevation maintained by the objects is known as component elevation.
- **Responsive elevation**: The elevation that can change as a response to user input is known as responsive elevation.

16.2.1 Motion

Material motion describes hierarchical and spatial relationships between objects. Following are the characteristics of material motion:

- It describes the movement of materials.
- It provides good transition to the user.
- It organizes the visual appearance of apps.
- It guides the user to have the right focus.
- It is natural and responsive.

The developers can use easing curves such as standard curve, deceleration curve, acceleration curve, and sharp curve to create smooth and consistent motion in materials.

16.2.2 Style

Table 16.2 lists the style elements that can be used in material design to customize the apps.

Element	Content	Description
Color	Color schemes	Primary color Secondary color Accent color Text and background colors
	Themes	Light theme Dark theme
Icons	Product icons	Design approach Product icon grid Keyline shapes Density Independent Pixel (DP) unit grid Geometry Product icon anatomy Product icon metrics Product icon patterns Human iconography

Element	Content	Description
	☐ System icons	☐ Design principles ☐ Grid, proportion, and size ☐ Icon grid ☐ Content area ☐ Dense layouts ☐ Keyline shapes ☐ Geometry ☐ System icon anatomy ☐ Corners ☐ Strokes ☐ Optical corrections ☐ Clearance area
Imagery	☐ Principles	☐ Personal relevance ☐ Information ☐ Delight ☐ Dynamic and context-relevant ☐ Immersive
	☐ Best Practices	☐ Multiple mediums ☐ No stock photos or clipart ☐ Iconic point of focus in imagery ☐ Immersive story
	☐ UI Integration	☐ Resolution ☐ Scale ☐ Text protection ☐ Avatars and thumbnails ☐ Hero images ☐ Gallery images
Typography	☐ Roboto ☐ Nato	☐ App bar ☐ Buttons ☐ Subheading ☐ Body 1 ☐ Text contrast ratios
Writing	☐ Clear ☐ Easy to understand ☐ Concise ☐ Accurate text	☐ Tone ☐ Capitalization ☐ Punctuation ☐ UI button text ☐ Guidelines

Table 16.2: Style Elements

16.2.3 Layout

Layout simplifies the process of creating scalable apps by using elements from print-based design. Table 16.3 lists the layout elements that can be used in material design.

Element	Description
Seam	Two sheets of material sharing a common edge is called a seam.
Step	Two overlapping sheets of material with different depths is called a step.
Floating action button	A circular sheet that is separate from a toolbar is a floating action button.
Pixel density	The number of pixels that can accommodate in an inch is pixel density.
Density-independent pixels(DP)	The uniform display of user interface components on screen having different densities is termed as density-independent pixels.
Metrics and keylines	This layout includes the baseline grids, ratio keylines, sizing by increments, and touch target size for the screen elements.
Structure	This layout includes the UI regions, toolbars, app bar, system bars, slide nav, and whiteframes.
Responsive UI	The responsive UI layout determine the consistency, adaptability, and scalability of apps on different screen sizes. This layout includes the breakpoints, grid, surface behaviors, and patterns.
Split screen	This layout allows the user to view two tasks or activities on the screen simultaneously.

Table 16.3: Layout Elements

16.2.4 Components

There are various user interface components in material design. Apart from the common ones such as buttons, dialog boxes, grid lists, lists, menus, sliders, progress bars, tabs, text fields, toolbars, and tooltips, there are specific components in material design.

Table 16.4 lists the specific components.

Component	Description
Bottom navigation	It provides quick navigation between top-level views in an app.
Bottom sheet	It reveals more screen content by sliding up from the bottom of the screen. There are two types of bottom sheets; modal and persistent.
Card	It displays information composed of different elements such as photo or text.
Chip	It can contain photos, text, or icons.
Divider	It helps to organize the page into different tiles and thus separate the content.
Picker	It helps to pick a single value from a pre-determined set of values. There are two types of pickers; date and time.
Snackbar	It provides a feedback on the performed operation at the bottom of the screen.
Toast	It provides system messages at the bottom of the screen.
Stepper	It displays the progress of an operation with the help of logical and numbered steps.

Table 16.4: Components

16.2.5 Patterns

There are different types of patterns in material design. Table 16.5 lists some of the patterns.

Component	Description
Empty states	This pattern occurs when an action or operation does not display any content.
Fingerprint	The user can use this pattern to unlock the device or sign into apps.
Gestures	This includes touch mechanics and touch activities.
Launch screens	The initial screen that is launched in an app.
Permissions	This pattern explains the need for each permission request.
Swipe to refresh	This pattern manually refreshes the screen content based on user's action.

Table 16.5: Patterns

16.2.6 Growth and Communications

The growth and communications guidelines define best practices that the developer can adopt for developing the apps. The guidelines explain the features of onboarding, feature discovery, and gesture education.

Some of these features are as follows:

- **Onboarding**: This feature help the users to get started with the app's user interface. There are three onboarding models in material design:
  - **Self-select**: The user can customize the user interface using this model.
  - **Quickstart**: The user can quickly get started with the app's functionality.
  - **Top User Benefits**: The user can know the major user interface benefits when using the app using this model.
- **Feature Discovery**: This feature facilitates the user to gain awareness about the new app features in the right context.
- **Gesture Education**: This feature support the users to understand the functionality of the elements thereby enabling maximum interaction with the interface.

16.2.7 Usability

There are two aspects to usability in material design. These are as follows:

- **Accessibility**: This aspect allows all kinds of users, even those with accessibility issues, to access the app's user interface successfully. Some users may have low vision, may be afflicted with blindness and hearing impairments. There are different elements such as color and contrast, sound, motion, layout, and writing that provide effective accessibility to the user interface for those users.
- **Bidirectionality**: Material design considers the feature of bidirectionality. Apps can include bidirectional scripts such as Arabic, Hebrew, and Persian. In bidirectional scripts, text is read and written from right-to-left unlike the English language.

16.2.8 Resources

There are various downloadable resources available in material design. Table 16.6 lists the resources.

Component	Description
Color palette	It includes the Adobe Photoshop and Adobe Illustrator color swatches.
Devices	It includes the device metrics.

Component	Description
Layout templates	It includes the mobile layout, tablet layout, desktop layout, and whiteframes.
Roboto and Noto fonts	Roboto is designed for mobile and Web usage. Noto is the standard typeface on Android. It covers all the languages not dealt by Roboto.
Sticker sheets and icons	Sticker sheets are used by components and include elements that make up layouts. Icons include system and product icons.

Table 16.6: Resources

16.3 Material Design Elements

Let us now look at the material design elements.

16.3.1 Themes

One of the primary design elements is themes.

Themes provide new styles for the apps. They offer uniform or logical styling of apps through surface shades, shadow depth, and ink opacity. The developer can set various themes for the apps.

The developer can set the color palette and touch animations for the system widgets using material themes. The developer can also set themes for the action bars and status bars, as well.

The material theme is defined as follows:

- @android:style/Theme.Material (dark version)
- @android:style/Theme.Material.Light (light version)
- @android:style/Theme.Material.Light.DarkActionBar

Applying Material Themes

The material theme is only available in Android 5.0 and later versions. The developer can use the material theme API to customize the color palette. Code Snippet 1 shows how to customize the theme's base colors or define custom colors by updating the Style.xml file.

Code Snippet 1:

```
<resources>
<!-- Base application theme. -->
<style name="AppTheme" parent="Theme.AppCompat.Light.
DarkActionBar">
<!-- Customize your theme here. -->
<item name="colorPrimary">@color/colorPrimary</item>
<item name="colorPrimaryDark">@color/colorPrimaryDark</item>
<item name="colorAccent">@color/colorAccent</item>
</style>
<style name="AppTheme.NoActionBar">
<item name="windowActionBar">false</item>
<item name="windowNoTitle">true</item>
</style>
<style name="AppTheme.AppBarOverlay" parent="ThemeOverlay.
AppCompat.Dark.ActionBar" />
<style name="AppTheme.PopupOverlay" parent="ThemeOverlay.
AppCompat.Light" />
<style name="CustomMaterialTheme" parent="android:Theme.Material"
>
<item name="android:colorPrimaryDark">@color/customcolor1</item>
</style>
</resources>
```

This code applies the material theme named **CustomMaterialTheme** to the mobile app.

Alternatively, the developer can use the Theme Editor of Android Studio to customize the theme. The Theme Editor can be accessed by clicking **Tools** → **Android** → **Theme Editor**. The Theme Editor window opens as shown in figure 16.1.

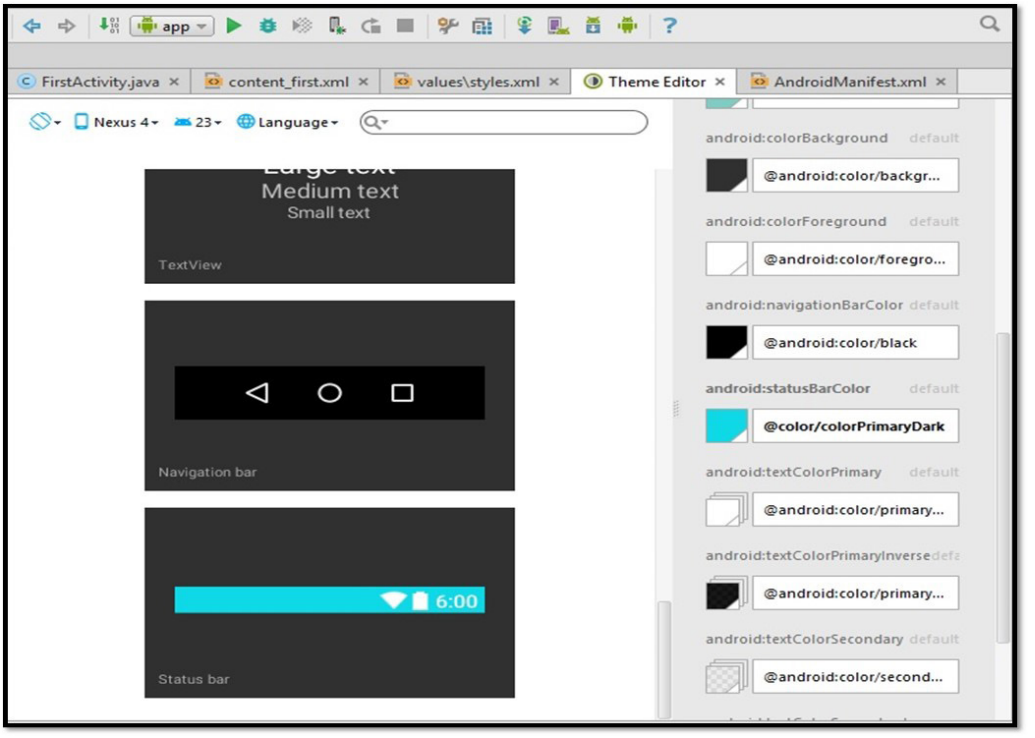


Figure 16.1: Theme Editor Window

The developer can also use the material theme API to customize the status bar. The developer can draw behind the status bar, set custom animations, and set custom colors. Code Snippet 2 shows the code of assigning the PrimaryDark color to the status bar.

Code Snippet 2:

```
<item name="android:statusBarColor">@color/colorPrimaryDark</
item>
```

16.3.2 List and Cards

List and cards are the design elements available in material design. Lists display multiple line items in a vertical manner as a single continuous element. Cards are material sheets that function as entry points to display detailed information.

There are two widgets in Android to display lists and cards with styles and animations.

The app developer can create complex lists and cards using the following widgets:

- **RecyclerView**: This widget is a pluggable version of ListView. It supports different types of layouts. It improves and enhances the app's performance.
- **CardView**: This widget allows the developers to display important information inside cards. There is a consistency maintained in the look and feel of the cards.

Create a List

The developer can use the **RecyclerView** widget to display and handle huge amount of data with the help of scroll bars. This widget can be used to display data that changes at runtime based on user inputs or actions. The widget provides layout managers for data items. It also provides default animations for items. Figure 16.2 illustrates the RecyclerView widget.

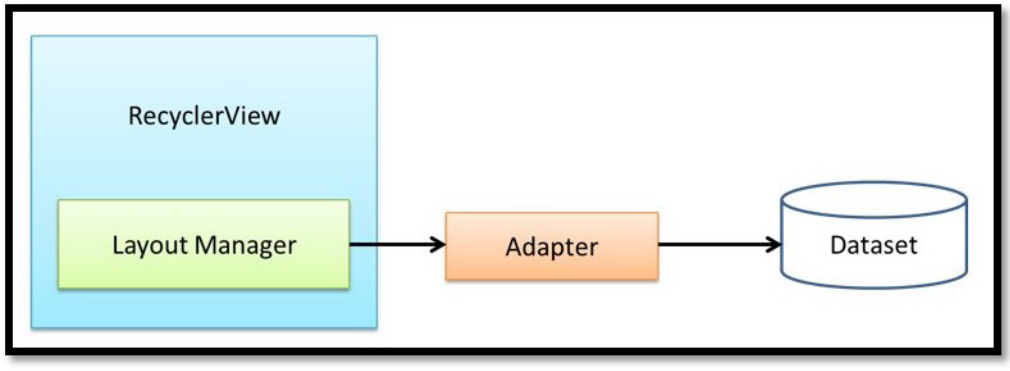


Figure 16.2: RecyclerView Widget

As shown in figure 16.2, the RecyclerView widget has an adapter and a layout manager. The layout manager arranges the items inside the RecyclerView. It reuses or recycles the views with the help of the adapter.

There are three built-in layout managers in RecyclerView widget. These are as follows:

- **LinearLayoutManager**: This layout manager shows items in a vertical or horizontal manner with a scroll list.
- **GridLayoutManager**: This layout manager shows items in a grid.
- **StaggeredGridLayoutManager**: This layout manager shows items in a staggered grid.

To add a **RecyclerView** widget to a layout, a custom view needs to be created as shown in figure 16.3.

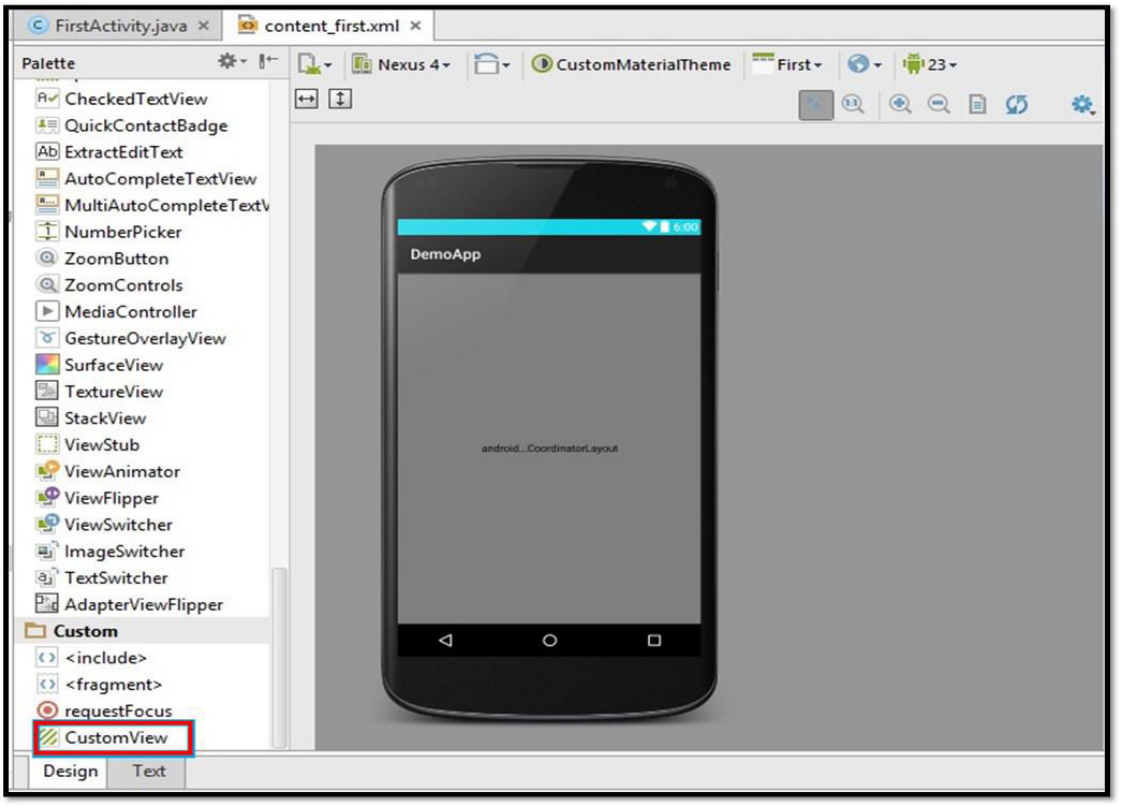


Figure 16.3: Adding Custom View

The list of views that can be added is displayed. The developer can search for the required view by specifying search string.

The **RecyclerView** widget is displayed as shown in figure 16.4.

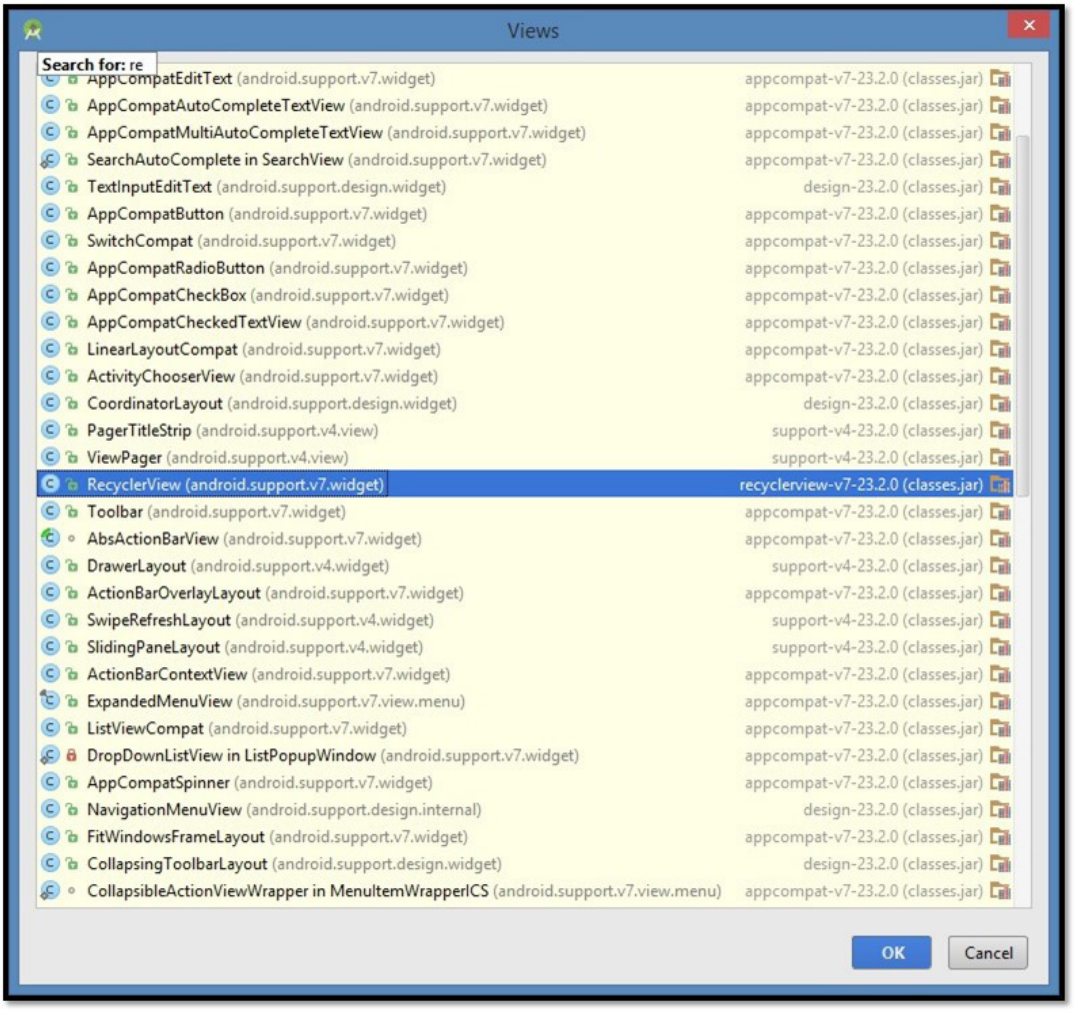


Figure 16.4: List of Views

Code Snippet 3 demonstrates how to add the **RecyclerView** widget using code to a layout by updating the content_first.xml file.

Code Snippet 3:

```
<!--A RecyclerView-->
<android.support.v7.widget.RecyclerView
 android:id="@+id/view"
 android:scrollbars="vertical"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"/>
```

Create a Card

The developer can use the **CardView** widget to create cards. The developer can use various properties to customize the appearance of the **CardView** widget. To add a CardView widget, the developer needs to drag the widget from the list of widgets to the work area in the Design mode as shown in figure 16.5.

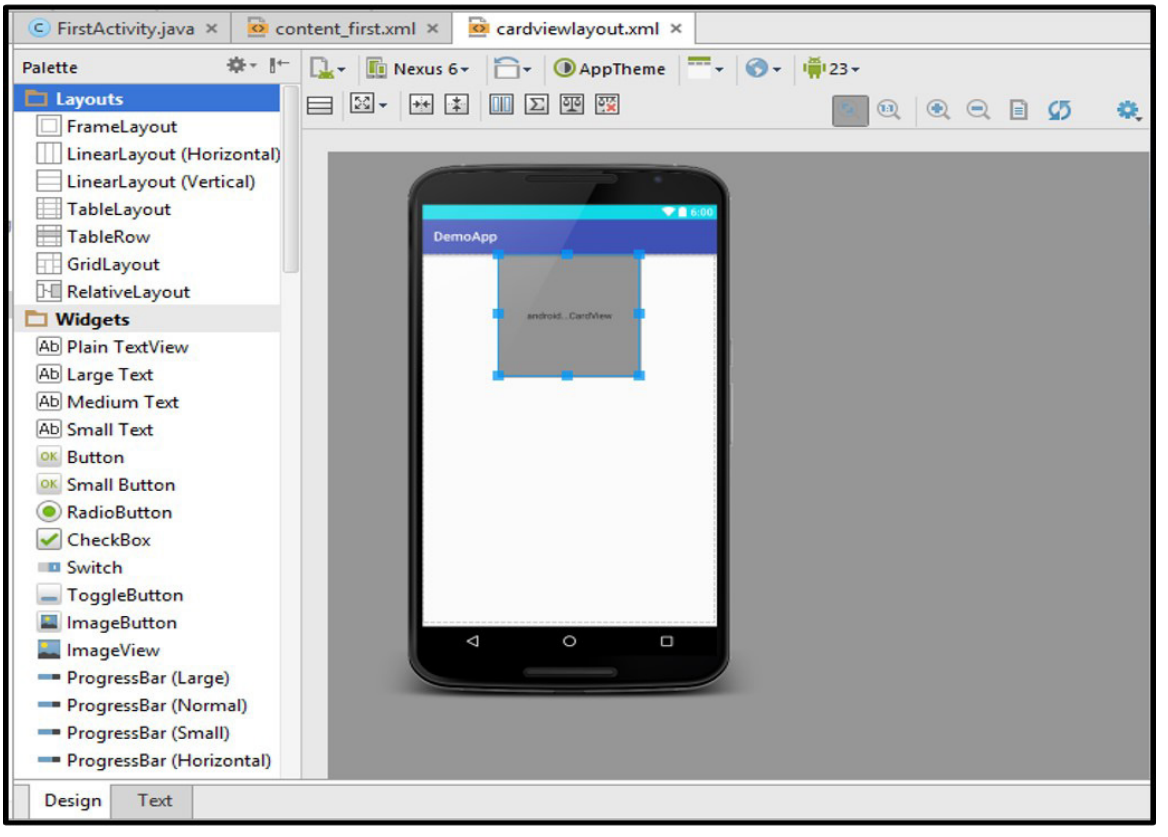


Figure 16.5: Design Mode of cardviewlayout.xml File

Code Snippet 4 demonstrates how to add the CardView widget using code to a layout in the CardViewLayout.xml file.

Code Snippet 4:

```
<LinearLayoutxmlns:android="http://schemas.android.com/apk/res/
android"
 xmlns:tools="http://schemas.android.com/tools"
 xmlns:card_view="http://schemas.android.com/apk/res-auto"
 ...>
 <!--A CardView with a TextView-->
 <android.support.v7.widget.CardView
  xmlns:card_view="http://schemas.android.com/apk/res-auto"
  android:id="@+id/mycardview"
android:layout_gravity="center"
  android:layout_width="200dp"
  android:layout_height="200dp"
  mycardview:cardCornerRadius="4dp">
  <TextView
   android:id="@+id/cardtext"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:text="This is a Card" />
 </android.support.v7.widget.CardView>
</LinearLayout>
```

16.3.3 View Shadows

There is a **Z** property for views in Android. This is in addition to the existing X and Y properties. The **Z** property represents the view's elevation. It determines the shadow's size, appearance, and the drawing order of the views. It is measured in dp.

Assigning Elevations to Views

There are two components for the Z value of the view. These are as follows:

- **Elevation**: It is the static component. To set the elevation of the view in a layout definition, use the **android:elevation** attribute. To animate the elevations, use **ViewPropertyAnimator.z()** and **ViewPropertyAnimator.translationZ()** methods.
- **Translation**: It is the dynamic component used for animations. To set the translation of the view, use **View.setTranslationZ()** method.

Customizing View Shadows

Code Snippet 5 customizes view shadows.

Code Snippet 5:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayoutxmlns:android="http://schemas.android.com/apk/
res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:paddingBottom="@dimen/activity_vertical_margin"
```

```
android:paddingLeft="@dimen/activity_horizontal_margin"
android:paddingRight="@dimen/activity_horizontal_margin"
android:paddingTop="@dimen/activity_vertical_margin"
app:layout_behavior="@string/appbar_scrolling_view_behavior"
tools:context="com.syskan.demoapp.FirstActivity"
tools:showIn="@layout/activity_first">
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:elevation="3dp"
android:textColor="@color/green"
android:text="@string/hello_world_welcome_to_android_6_0" />
</RelativeLayout>
```

This code customizes the shadow by adding padding to the bottom, left, right, and top of the activity. It also adds a text view that displays the string "hello world welcome to android 6.0" in green font.

The View Shadow appears as shown in figure 16.6.

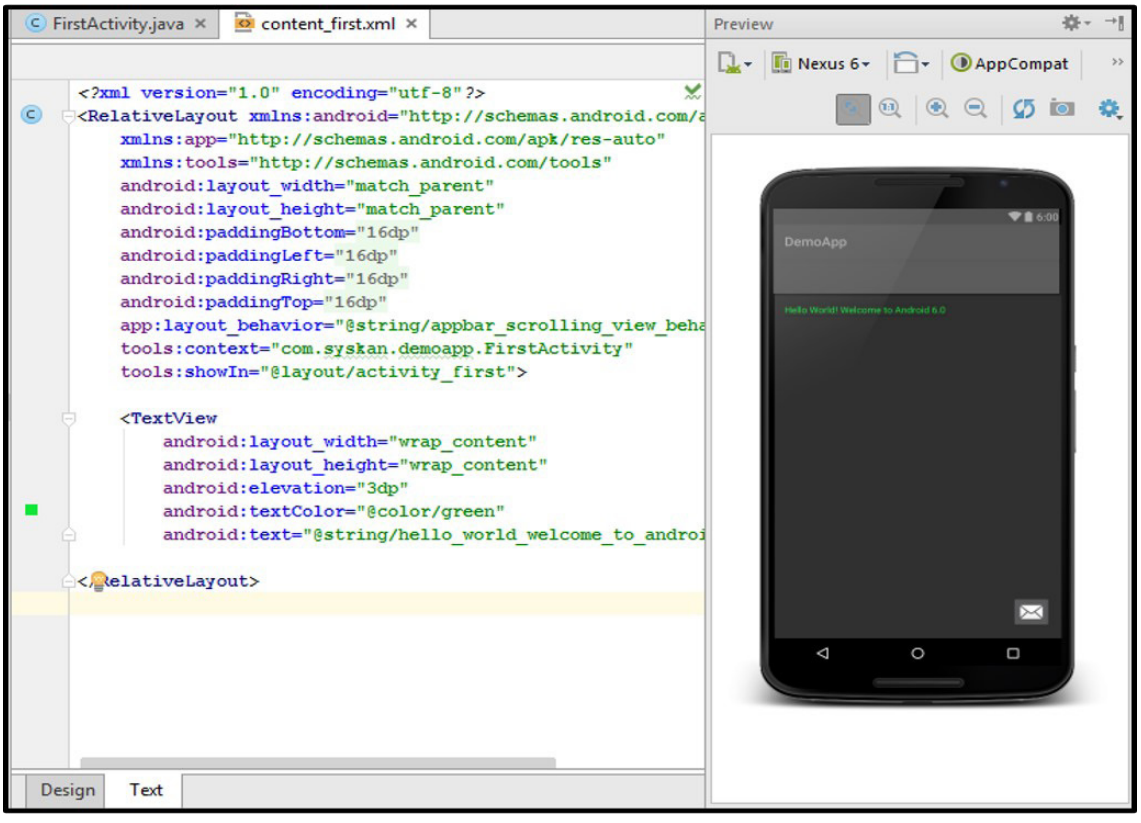


Figure 16.6: View Shadow

Clipping Views

Clip views are used to change the shape of the views. The developer can clip a view to its outline area using the **View.setClipToOutline()** method or the a**ndroid:clipToOutline** attribute. All outlines do not support clipping except rectangle, circle, and round rectangle.

16.3.4 Animations

The material themes provide default animations for buttons and activities transitions. Let us look at these in detail.

Introduction

Android 5.0 and later versions have various animation APIs that let the developers to create touch based custom animations for the user interface components. The custom animations can be added to the custom views.

The developer can customize the following animations:

- Touch feedback
- Circular Reveal
- Activity transitions
- Curved motion

- View state change

Customizing Touch Feedback

The touch feedback provides instant feedback to developers during the interaction with the user interface elements. The buttons have default touch feedback animations. The **RippleDrawable** class provides default animations with a ripple effect.

Using Reveal Animations

The developers can show or hide a group of user interface elements. The reveal animations provide visual continuity to the users during the process of showing or hiding elements. Code Snippet 6 demonstrates how to use the hide and reveal animation effect on a view.

Code Snippet 6:

```
View testView = findViewById(R.id.mycardview);
// get the center of the clipping circle
int width = testView.getWidth() / 2;
int height = testView.getHeight() / 2;
float newRadius = (float) Math.hypot(width, height);
Animator animator =
   ViewAnimationUtils.createCircularReveal(testView, width,
height, 0, newRadius);
testView.setVisibility(View.VISIBLE);
animator.start();
// previously visible view
final View testView1 = findViewById(R.id.mycardview);
int width2 = testView1.getWidth() / 2;
int height2 = testView1.getHeight() / 2;
float oldRadius = (float) Math.hypot(width2, height2);
// create the animation with the final radius as zero
Animator animator1 =
   ViewAnimationUtils.createCircularReveal(testView1, width2,
height2, oldRadius, 0);
// make the view invisible at the end of the animation
animator1.addListener(new AnimatorListenerAdapter() {
 @Override
 public void onAnimationEnd(Animator animation) {
  super.onAnimationEnd(animation);
  testView1.setVisibility(View.INVISIBLE);
```

```
  }
});
animator1.start();
```

This code uses the CircularReveal option to reveal and hide the view.

Customizing Transitions

Material design provides activity transitions. They provide visual connections between different states of views. The developers can specify custom animations for enter and exit transitions and for transitions of shared elements between activities.

The enter transition determines the way views enter the scene. For example, in the explode enter transition, the views enter from outside and fly in towards the screen's center area.

The exit transition determines the way views exit the scene. For example, in the explode exit transition, views leave the scene away from the center area. Android 5.0 supports **explode**, **slide**, and fade enter and exit transitions.

The shared elements transition determines the sharing of views between activities transitions. For example, if two activities have the same image but are of different sizes, the shared element transition ensures smooth transition between the activities with respect to the image. Android 5.0 supports shared elements transitions such as **changeBounds**, **changeclipBounds**, **changeTransform**, and **changeImageTransform**.

Code snippet 7 demonstrates custom transitions.

Code Snippet 7:

```
<style name="BaseAppTheme" parent="android:Theme.Material">
<!-- enable window content transitions -->
<item name="android:windowActivityTransitions">true</item>
<item name="android:windowEnterTransition">@transition/burst</
item>
<item name="android:windowExitTransition">@transition/burst</
item>
<!-- specify shared element transitions -->
<item name="android:windowSharedElementEnterTransition">
   @transition/transform_image</item>
<item name="android:windowSharedElementExitTransition">
   @transition/transform_image</item>
</style>
```

This code applies the burst transition to a Windows shared element and transforms it into an image.

Customizing Timing Curves and Curve Motion Patterns

In Android 5.0, the developers can customize the timing curves and curve motion patterns. Code snippet 8 demonstrates the usage of curved motion.

Code Snippet 8:

```
<pathInterpolatorxmlns:android="http://schemas.android.com/apk/
res/android"
 android:controlX1="0.4"
 android:controlY1="0"
 android:controlX2="1"
 android:controlY2="1"/>
ObjectAnimatormyAnimator;
myAnimator=ObjectAnimator.ofFloat(view,View.X,View.Y,path);
...
myAnimator.start();
```

Animating Changes to View State

The developer can animate changes to view state in Android. The **StateListAnimator** class defines animators that run when the state of a view changes. Code snippet 9 demonstrates the usage of the **StateListAnimator** class.

Code Snippet 9:

```
<!--animate the translationZ property of a view when pressed-->
<selector xmlns:android="http://schemas.android.com/apk/res/
android">
 <item android:state_pressed="true">
 <selector xmlns:android="http://schemas.android.com/apk/res/
android">
 <item android:state_pressed="true"
  android:state_enabled="true"
  android:state_focused="true">
  <set>
   <objectAnimator android:propertyName="translationZ"
    android:duration="@android:integer/config_longAnimTime"
    android:valueTo="6dp"
    android:valueType="floatType"
     />
```

```
    </set>
  </item>
  <item android:state_enabled="true"
    android:state_pressed="false"
    android:state_focused="true" >
    <set>
      <objectAnimator android:propertyName="translationZ"
        android:duration="120"
        android:valueTo="0"
        android:valueType="floatType"/>
    </set>
  </item>
</selector>
```

Check Your Progress

1. Which of the following versions of Android was exclusive to Tablets?

(A)	Materials are solid by nature	(B)	Materials move along only X axis in the plane
(C)	Materials bend or fold	(D)	Materials cannot change their shape

2. The _____ class defines animators that run when the state of a view changes.

(A)	StateListAnimators	(B)	StateListAnimator
(C)	StateListsAnimator	(D)	StatesListAnimator

3. Material design has a _____ dimensional environment.

(A)	Two	(B)	Multi
(C)	Four	(D)	Three

4. Material design has a _____ dimensional environment.

(A)	ViewPropertyAnimators.x()	(B)	ViewPropertyAnimator.y()
(C)	ViewPropertyAnimator.z()	(D)	ViewPropertyAnimator.x()

5. Material design has a _____ dimensional environment.

(A)	Roboto	(B)	Noto
(C)	Adobe Illustrator	(D)	Themes

Answers

1.	A
2.	B
3.	D
4.	C
5.	A

Summary

- Material design combines good and classic design principles with some advanced tools to create a single underlying system across various platforms and mobile devices.
- The difference in elevations between overlapping material objects creates shadows in the environment.
- Material motion describes hierarchical and spatial relationships between objects.
- The layout simplifies the process of creating scalable apps by using elements from print-based design.
- Growth and communications guidelines define the best practices that the developers can adopt for developing the apps.
- RecyclerView widget is a pluggable version of ListView and supports different types of layouts.
- Activity transitions provide visual connections between different states of views.
- The shared elements transition determines the sharing of views between activities transitions.

17 Android Interface Definition Language

Welcome to the Session, **Android Interface Definition Language**.

This session provides an introduction to the Android Interface Definition Language (AIDL). It also explains how to create an AIDL file and how to implement the interface. In addition to this, the session explains how the interface helps in client and server interaction with the help of Interprocess Communication (IPC).

In this Session, you will learn to:

- Explain the AIDL Interface
- Explain how to create an AIDL file
- Explain how to implement the interface
- Explain the process of communication through IPC
- Explain how the IPC method is invoked

Session

17

Android Interface Definition Language

17.1 Introduction

Consider a scenario of a mobile-based application. App1 is a school application pertaining to a certain city, which consists of data related to the schools in the city, students in these schools, their age, stream, and so on. It also consists of data related to the teachers, their qualification, years of experience, and other personal information. This application can be viewed as a large data bank containing all essential information about schools of the particular city along with all information about students and teachers. Now, consider another school-based application, App2. This application processes the App1 student information and suggests a suitable course or career choice for them.

In this scenario, App1 is a service provider that shares student information with App2. It means that the processes of App2 communicate with processes of App1 to gain access to the required information. AIDL is an Interface Definition Language which helps processes of an application to communicate with processes of another application, in a way that is similar to a client-server communication. In the scenario described, App1 is the server that provides information and App2 is the client that requires information. AIDL facilitates the communication between App1 and App2.

17.2 AIDL Interface

To enable communication between the client and service provider interfaces, the developer has to define the AIDL interface. This interface determines the methods and fields that should be available for client access. The developer can define the AIDL interface in the .aidl file using the Java programming language syntax. The .aidl file must be saved in the source code of the service hosting application and the application that requires the information from the service.

Designing the AIDL interface involves following three steps:

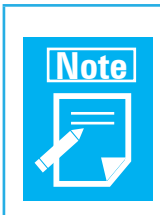
1. Creating an .aidl file to define the interface
2. Implementing the interface generated from the .aidl
3. Exposing the interface to the client applications

17.2.1 Creating an AIDL file to define the interface

The AIDL file is created using the Java programming language and consists of an interface with one or more methods. These methods can take parameters and return values. The parameters and return values can be of any type and can also refer to another aidl interface. Every .aidl file must define a single interface. The definition consists of interface declaration and method signatures.

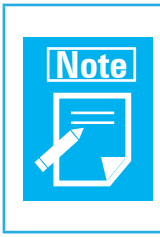
AIDL supports following data types by default:

- All primitive data types supported in Java, such as int, long, and Boolean
- String
- CharSequence
- List
- Map



All the elements in the List and Map data types should belong to the other data types mentioned in the given list.

- Other AIDL-generated interfaces that can be passed by reference
- Custom classes that implement the Parcelable protocol



An import statement has to be declared to use the AIDL interfaces and custom classes.

Defining a Service Interface

Some of the important factors that developers need to understand before defining the service interface are as follows:

- Parameters of the methods can be zero.
- The methods can return a value or can be void.
- The non-primitive data types should have a directional tag indicating the direction of data.
- AIDL does not support static fields.
- The code comments in .aidl file are included in the IBinder interface.

To add an AIDL interface, right-click the app name in the Navigator pane, point to **New** → **AIDL** and then, click **AIDL File** as shown in figure 17.1.

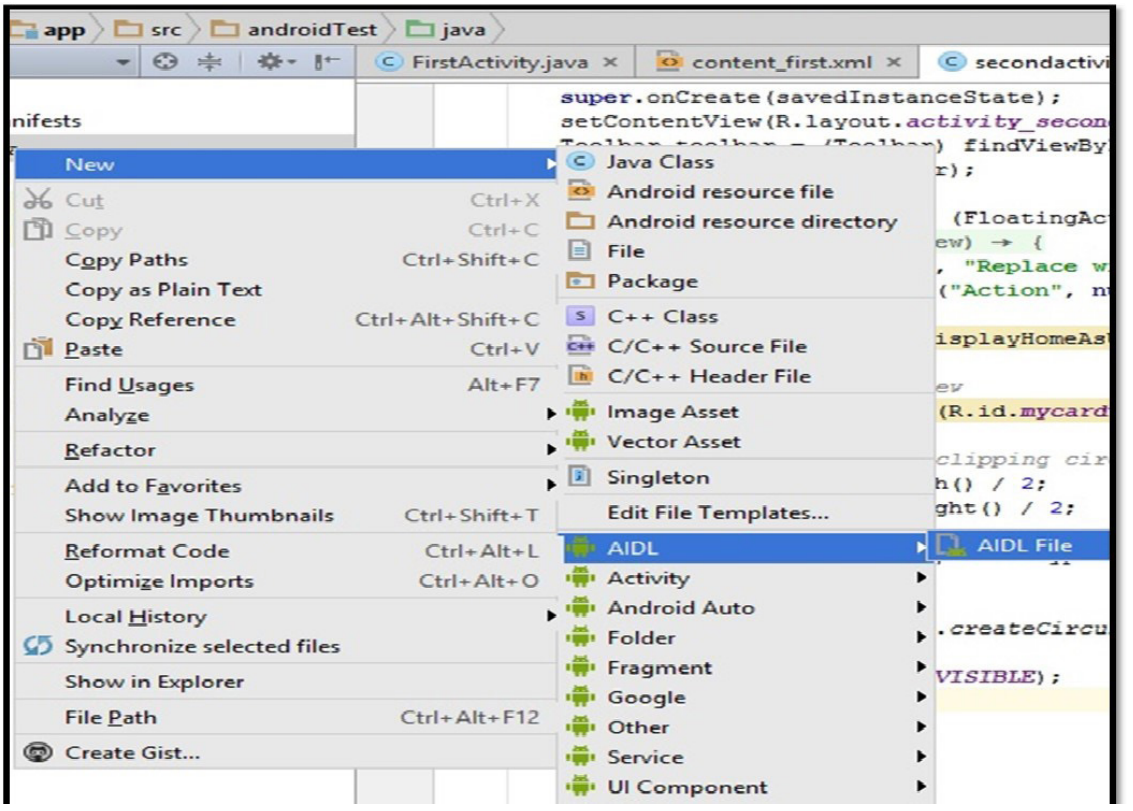


Figure 17.1: Creating an AIDL File

The New Android Component dialog box appears. Specify the name for the interface and the target source set of the interface and then click Finish, as shown in figure 17.2.

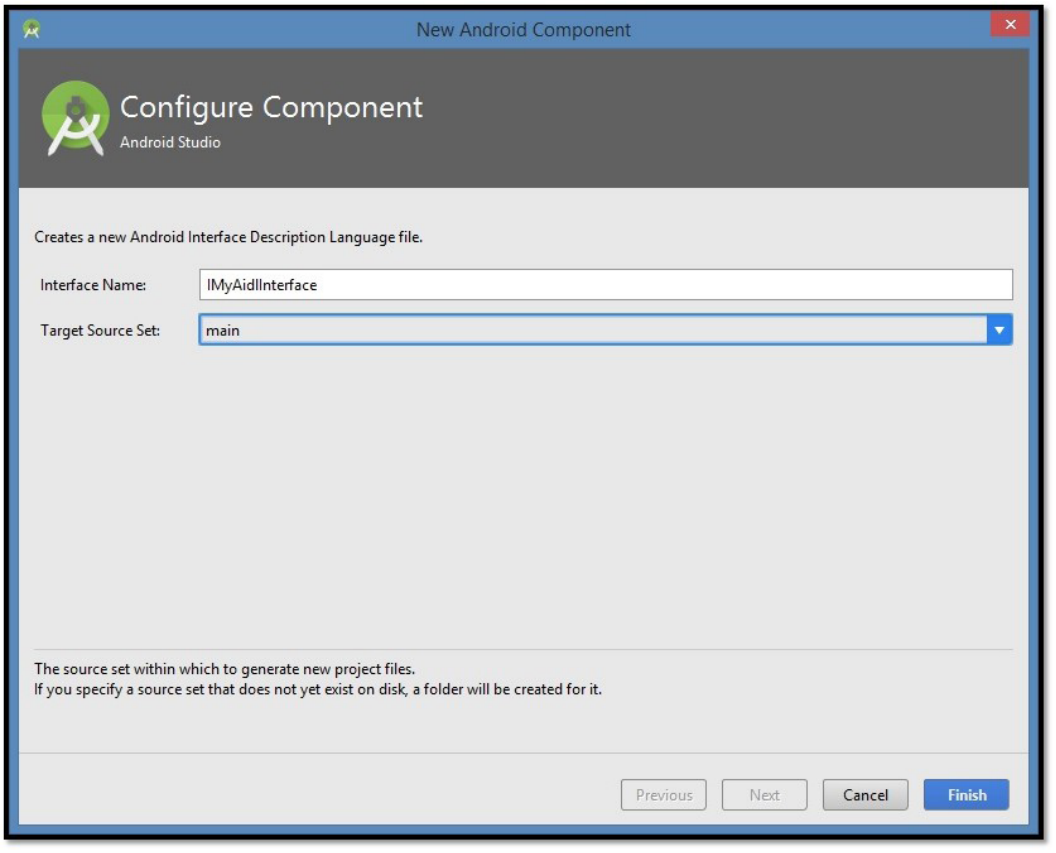


Figure 17.2: Specifying Name and Target Source Set for Interface

The .aidl file is created with autogenerated code as shown in figure 17.3.

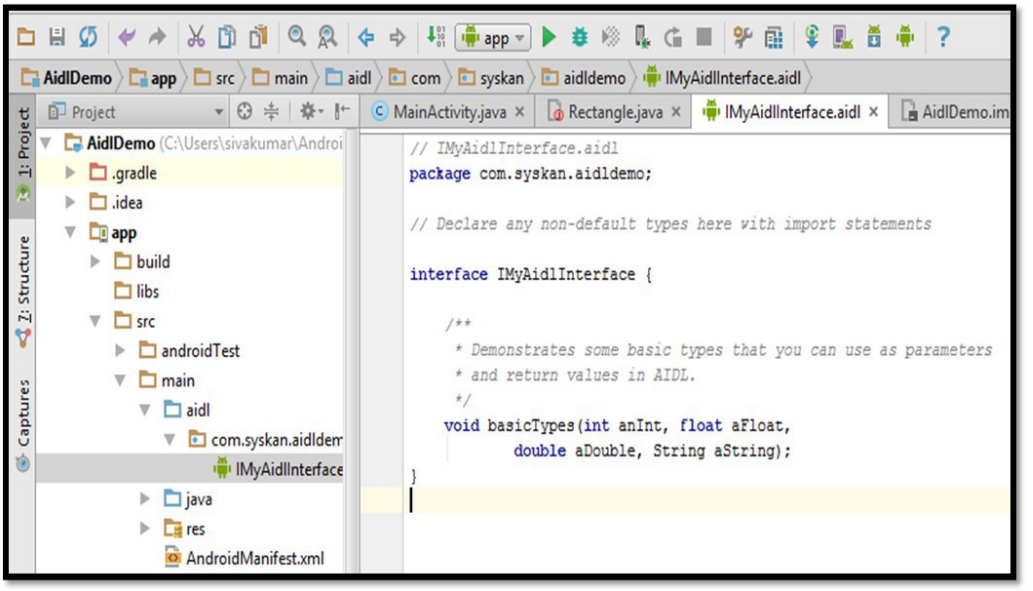


Figure 17.3: Autogenerated AIDL File

The .aidl file is saved in the project's src directory of the server as well as the client applications. The SDK tools generate the IBinder interface when the application is built. The IBinder interface is saved in the 'gen' or 'generated' directory and has the same name as the .aidl file, however with a .java extension.

Figure 17.4 shows the IBinder interface autogenerated for the IMyAidlInterface AIDL interface.

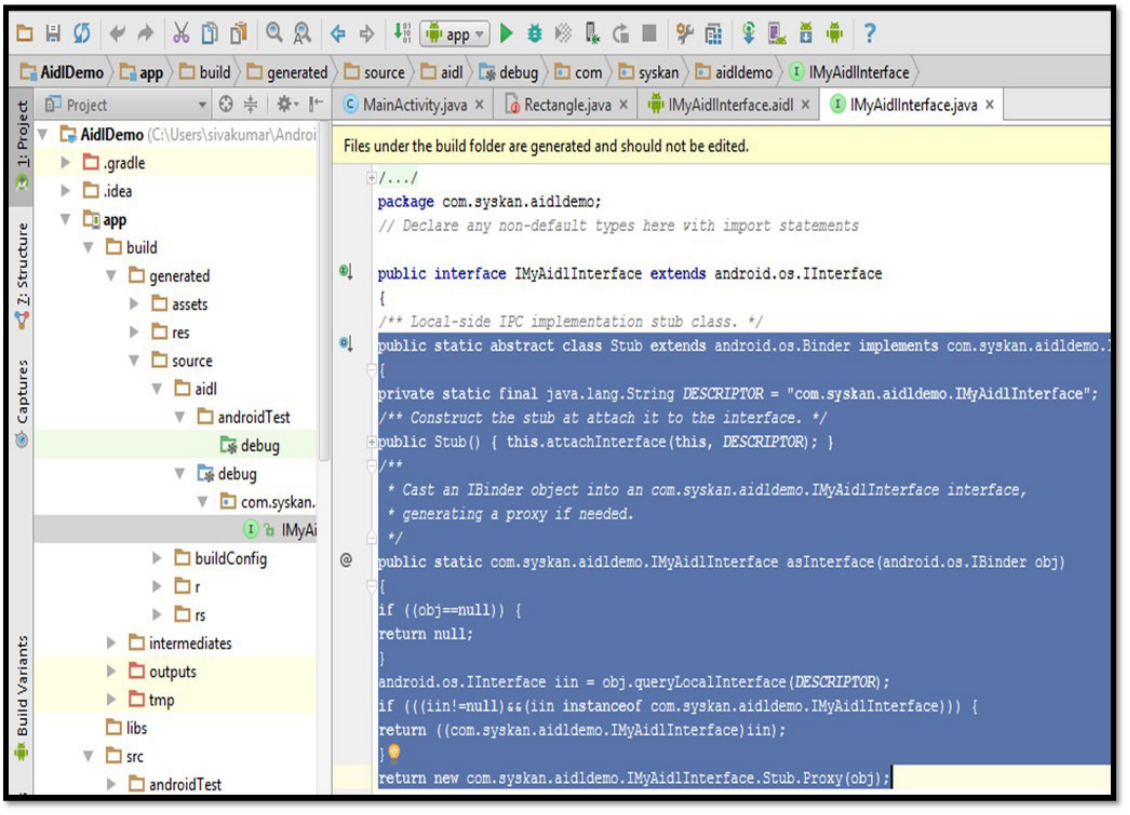


Figure 17.4: Automatically Generated IBinder Interface

17.2.2 Implementing the AIDL Interface Generated

The next step is to implement the interface generated by the .aidl file. The interface generated by the .aidl file with .java extension includes a subclass Stub that declares all the methods from the .aidl file. Stub consists of helper methods such as the asInterface() method which implements the IBinder interface. The method returns the instance of the interface, which is used to call the RPC methods.

The Binder interface can be extended and implemented by creating a new Java file that extends Service or in any activity.java file as an anonymous instance. Code Snippet 1 shows how to extend the Binder interface and implement the interface using an anonymous interface.

Code Snippet 1:

```
private final IMyAidlInterface.Stub myBinder = new IMyAidlInterface.
Stub() {

  public void basicTypes(int anInt, float aFloat,
```

```
    double aDouble, String aString) {
   // Does nothing
  }
};
```

The code defines an instance of the stub class, which defines the Remote Procedure Call (RPC) for the service. However, the developer should consider following important factors before the implementation of the interface:

- The service should be thread safe or that is, it should support multithreading.
- The RPC calls are synchronous. RPC calls should use a separate thread, if the processing is time consuming. This avoids forced shut down of application.
- The exceptions thrown are not diverted back to the caller.

17.2.3 Exposing the Interface to Clients

If the developer wants to expose the interface or publish the service to the client, the user needs to inherit Service and implement the Service.onBind () method. This method returns an instance of the class that implements the generated Stub. Code Snippet 2 shows how to publish the service to the client by exposing the IMyAidlInterface interface. Ensure that any other import statements that may be required are added. IMyAidlInterface should be used only for Stub instantiation, not for this class.

Code Snippet 2:

```
import android.app.Service;
. . . // Other import statements
import com.syskan.aidldemo.IMyAidlInterface;


public class MyFirstRemoteService extends Service {
 @Override
 public void onCreate() {
  super.onCreate();
 }
 @Override
 public IBinder onBind(Intent intent) {
 // Return the interface
  return myBinder;
 }
```

```
   private final IMyAidlInterface.Stub myBinder = new
IMyAidlInterface.Stub() {
   public void basicTypes(int IntegerData, long LongData, boolean
BooleanData, float FloatData, double DoubleData, String StringData) {
    // Does nothing
   } };
}
```

Consider again the scenario discussed earlier about the school applications, App1 and App2. App2's process calls the bindService () method to connect with the service provided by App1 and the App2 onServiceConnected () callback receives the myBinder instance. This instance is returned by the onBind () method.

The .aidl file should also be copied to the src directory of the client (App2 in this case), if the client and service (App1) are in separate applications.

Code Snippet 3 shows how the client calls the **IMyAidlInterface** .Stub.asInterface (service) when it receives the IBinder in the onServiceConnected () method. This code must be added to the activity.java file.

Code Snippet 3:

```
IMyAidlInterface iMyAidlInterface;
protected ServiceConnection mConnection = new ServiceConnection() {
 // Called when the connection with the service is established
 public void onServiceConnected(ComponentName className, IBinder
service) {
   //Following the example given earlier for an AIDL interface,
   // this gets an instance of the IRemoteInterface, which
// we can use to call on the service
   iMyAidlInterface = IMyAidlInterface.Stub.asInterface
(service);
 }
String TAG;
 // Called when the connection with the service disconnects
// unexpectedly
 public void onServiceDisconnected(ComponentName className) {
   Log.e(TAG, "Service has unexpectedly disconnected");
   iMyAidlInterface = null;
 }};
```

17.3 Communication through IPC

IPC interface facilitates communication between processes. This communication includes sending class from one process to another. However, to implement IPC between the processes, the class code should be available to the other side of the IPC channel. In addition, the class must also support Parcelable interface. Parcelable interfaces allow the Android system to decompose the objects of a class into primitives. This primitive form of the objects can be easily interfaced across various other processes.

To facilitate communication through IPC and to create a class that supports parcelable protocol, the developer needs to perform following steps:

- Implement the Parcelable interface in the class.
- To write the current state of the object to Parcel, implement writetoParcel. To implement the Parcelable.Creator interface, use a static field CREATOR in the class.
- Ensure that the .aidl file that is created, declares the Parcelable class.

Code Snippet 4 shows how to create a parcelable class in the existing or a new .aidl file.

Code Snippet 4:

```
// Declare Triangle so AIDL can find it & knows that it implements
// the parcelable protocol.
parcelable Triangle;
```

Code Snippet 5 shows declaration of the class created in the previous code snippet, which is parcelable.

Code Snippet 5:

```
import android.os.Parcel;
import android.os.Parcelable;
public final class Triangle implements Parcelable {
 public int leftpos;
 public int toppos;
 public int rightpos;
 public static final Parcelable.Creator<Triangle> CREATOR = new
    Parcelable.Creator<Triangle>() {
     public Triangle createFromParcel(Parcel in) {
       return new Triangle(in);   }
     public Triangle[] newArray(int size) {       return new
Triangle[size];
}
};
```

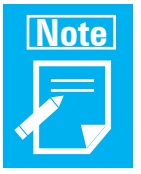
```
 public Triangle() {  }
 private Triangle(Parcel in) {
   readFromParcel(in);
 }
 public void writeToParcel(Parcel out) {
   out.writeInt(leftpos);
   out.writeInt(toppos);
   out.writeInt(rightpos);
 }
 public void readFromParcel(Parcel in) {
   leftpos = in.readInt();
   toppos = in.readInt();
   rightpos = in.readInt();
 }
}
```

This code snippet shows the implementation of the Parcelable protocol. This class assists when sending a parcelable object in an IPC call. Declaring the parcelable class name in .aidl file, enables sending object in an IPC. In addition, the parcelable interface allows android system to decompose the object into primitives and assist the marshalling process.

17.4 IPC Method Invocation

To call a method defined in the .aidl interface using IPC, the calling class performs certain steps. These steps are as follows:

1. Copying the .aidl file in the src directory of the project.
2. Creating an instance of the IBinder interface.
3. Implementing ServiceConnection.
4. Calling context.bindservice () in the ServiceConnection implementation.
5. Calling IMyAidlInterface.Stub.asInterface (IBinder) service to cast the returned parameter to IMyAidlInterface type. Calling the methods defined in the interface.
6. Calling the Context.unbindService () method with the instance of the service to disconnect.



- While calling an IPC service, object references are counted across processes.
- The developer can send anonymous objects as method arguments.

Code Snippet 6 shows how to call an AIDL service.

Code Snippet 6:

```
import android.content.ComponentName;
import android.content.Intent;
import android.content.ServiceConnection;
import android.os.Bundle;
import android.os.IBinder;
import android.os.RemoteException;
import android.support.v7.app.AppCompatActivity;
import android.widget.TextView;
public class MyFirstBinding extends AppCompatActivity {
 private IMyAidlInterface mService;
 private TextView mLog;
 @Override
 protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
   setContentView(R.layout.activity_main);
   mLog = (TextView) findViewById(R.id.log);
   Intent serviceIntent = new Intent()
      .setComponent(new ComponentName(
         "com.syskan.aidlexamplereceiver",
"com.syskan.aidlexamplereceiver.MainService"));
   mLog.setText("Starting service…\n");
   startService(serviceIntent);
   mLog.append("Binding service…\n");
   bindService(serviceIntent, mConnection, BIND_AUTO_CREATE);
}
 private ServiceConnection mConnection = new ServiceConnection() {
```

```
    @Override
  public void onServiceConnected(ComponentName className, IBinder
service) {
    mLog.append("Service binded!\n");
    mService = IMyAidlInterface.Stub.asInterface(service);
    performListing();
  }
  @Override
  public void onServiceDisconnected(ComponentName className) {
    mService = null;
    // This method is only invoked when the service quits from the other
end or gets killed
    // Invoking exit() from the AIDL interface makes the Service kill
itself, thus invoking this.
    mLog.append("Service disconnected.\n");
  }
 };
 private void performListing() {
  mLog.append("Requesting file listing…\n");
 long start = System.currentTimeMillis();
  long end = 0;
  try {
   MainObject[] results = mService.listFiles("/sdcard/testing");
   end = System.currentTimeMillis();
   int index = 0;
   mLog.append("Received " + results.length + " results:\n");
   for (MainObject o : results) {
    if (index > 20) {
     mLog.append("\t -> Response truncated!\n");
     break;
    }
    mLog.append("\t -> " + o.getPath() + "\n");
index++;
   }
```

```
        } catch (RemoteException e) {
    e.printStackTrace();
   }
   mLog.append("File listing took " + (((double) end - (double) start) /
1000d) + " seconds, or " + (end - start) + " milliseconds.\n");
   try {
    mService.exit();
   } catch (RemoteException e) {
    e.printStackTrace();
   }
  }
}
```

Check Your Progress

1. What does AIDL stand for?

(A)	Android Interface Design Language	(B)	Android Interface Definition Language
(C)	Android Interactive Definition Language	(D)	Android Interactive Design Language

2. Which directory should the .aidl file be saved?

(A)	src/	(B)	gen/
(C)	url/	(D)	bin/

3. Which subclass does the Android SDK generated .java interface contain?

(A)	Stud	(B)	Stub
(C)	Interface	(D)	Binder

4. Which interface must a class support to enable communication through IPC?

(A)	Parcelable	(B)	.java interface
(C)	Serializable	(D)	.aidl interface

5. For a class to call the remote AIDL interface, it must _____.

(A)	Include the .aidl file in the project's gen/directory	(B)	Declare an instance of Binder interface
(C)	Include the .aidl file in the project's lib/directory	(D)	Declare an instance of IBinder interface

Answers

1.	B
2.	A
3.	B
4.	A
5.	D

Session

17

Android Interface Definition Language

Summary

- Android Interface Definition Language allows communication between the client and service provider through IPC.
- AIDL interface is defined in the .aidl file using Java programming language.
- The .aidl file defines the programming interface that includes method signatures.
- The programming interface contains an inner abstract class Stub that implements methods from the AIDL interface.
- To publish the service to the clients, one needs to expose the interface to the clients.
- IPC interface allows a parcelable class to be sent from one process to another.
- The calling class must follow certain steps to call the remote AIDL interface.

Onlinevarsity

18 Android Native Development Kit (NDK)

Welcome to the Session, **Android Native Development Kit (NDK)**

This session explains the basic concepts of Android Native Development Kit (NDK). It describes the main components that are used in native app development. It explains the process to install the NDK and create a sample app. Finally, the session describes the steps to run the sample app.

In this Session, you will learn to:

- Describe app components
- Explain the process to install the NDK
- Explain the steps to create and test a sample native app

Session

18

Android Native Development Kit (NDK)

18.1 Introduction

The Native Development Kit (NDK) is a set of tools. It allows the developers to use or embed the native code of C and C++ to build and debug Android apps. Embedding C or C++ native code in apps facilitates porting of apps across platforms and reuse of existing libraries in apps.

18.2 App Components

Developers can use various components while building native applications for Android devices. Table 18.1 describes the main components.

Component	Description
Application Binary Interface (ABI)	It acts as the interface between the system and the app code. The ABI is CPU-specific and includes the instruction set for a specific CPU. Therefore, the ABI facilitates the interaction of the app code with the system at runtime. The ABI helps the same app to work on systems with different CPUs and instruction sets. However, the app code must specify the ABI for each platform on which the application must run.
Java	When an android application is built, all the Java files included in the application is converted into .dex (Dalvik Executable) files. These .dex files run in the Dalvik Virtual Machine (DVM). If the application does not include any Java code, a .dex file is generated for the native component of the app to run.
Java Native Interface (JNI)	It is the programming framework that enables Java code and C or C++ code contained in an application to interact with each other.
Manifest	The developer must declare the **NativeActivity** class in the manifest if there is no Java code in the app.
ndk-build	It is a shell script which runs the required NDK build scripts. It generates binary files which are then copied to the app's project path. **Application.mk** is also required for building apps using the ndk-build file. Inside the jni folder, the developer needs to add an **Android.mk** configuration file for building the app.
Native Shared Libraries	These are built from the app's native source code and have an extension of .so.
Native Static Libraries	The NDK builds static libraries that can be linked with other existing libraries. The native static library files have an extension of .a.

Table 18.1: App Components

Let us take a look at the general series of steps that the developer needs to follow while developing an Android app. The steps are as follows:

1. Design the app.
2. Decide the Java and native code components that need to be used in the app.
3. Create an Android app project.
4. For a native-only app, declare the **NativeActivity** class in **AndroidManifest.xml**.
5. Create an **Android.mk** file.
6. Add the native source code under the project's **jni** directory.
7. Use **ndk-build** to compile the native libraries.
8. Build the Java component.
9. Package all the components into an **Android Application Package (APK)** file.

Native Activity

The Android NDK has a helper class called the **NativeActivity** class that allows the developer to build a native activity. This class handles the communication between the Android framework and the app's native code. The developer must declare the application as '**native**' in the **AndroidManifest.xml** file before developing it.

18.3 Installing the NDK

Let us learn the procedure to install the NDK. There are two ways you can do this:

- Install via https://developer.android.com/ndk/downloads/index.html and then configure the IDE to use this manually.

OR

- Install it via the Android Studio IDE.

The second approach is explored here through the following steps:

1. Launch Android Studio and click **Tools → Android → SDK Manager**. Figure 18.1 shows the options.

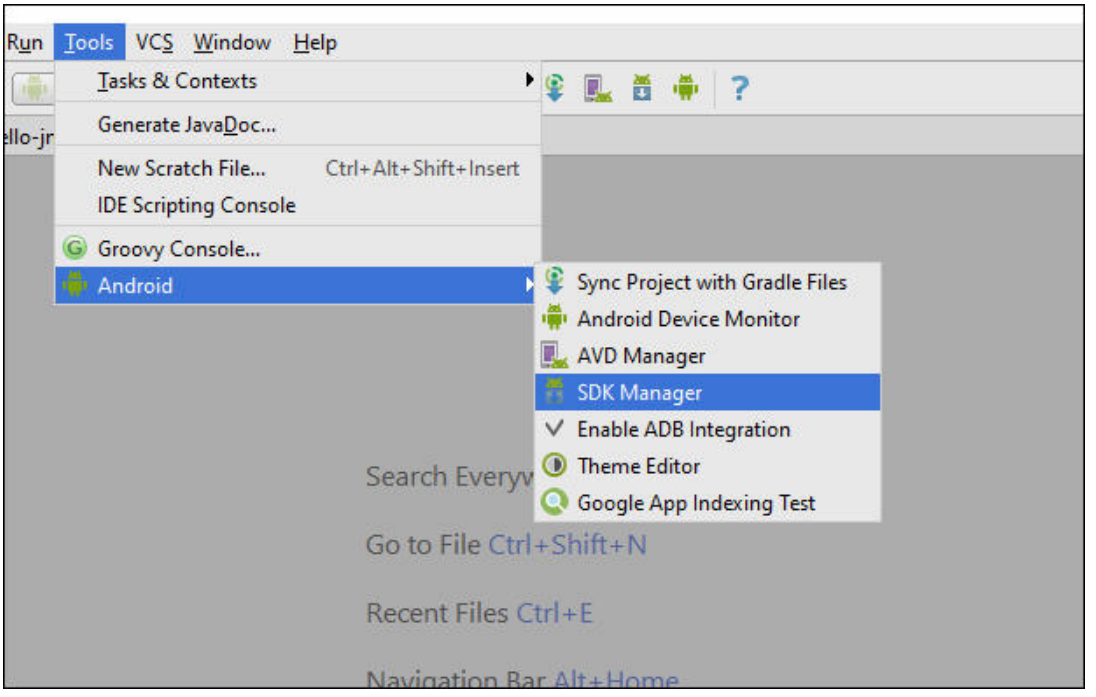


Figure 18.1: Launching the SDK Manager

2. In the dialog box that is displayed, select **SDK Tools** tab. You will observe NDK as one of the tools with status **Not installed** beside it as shown in figure 18.2.

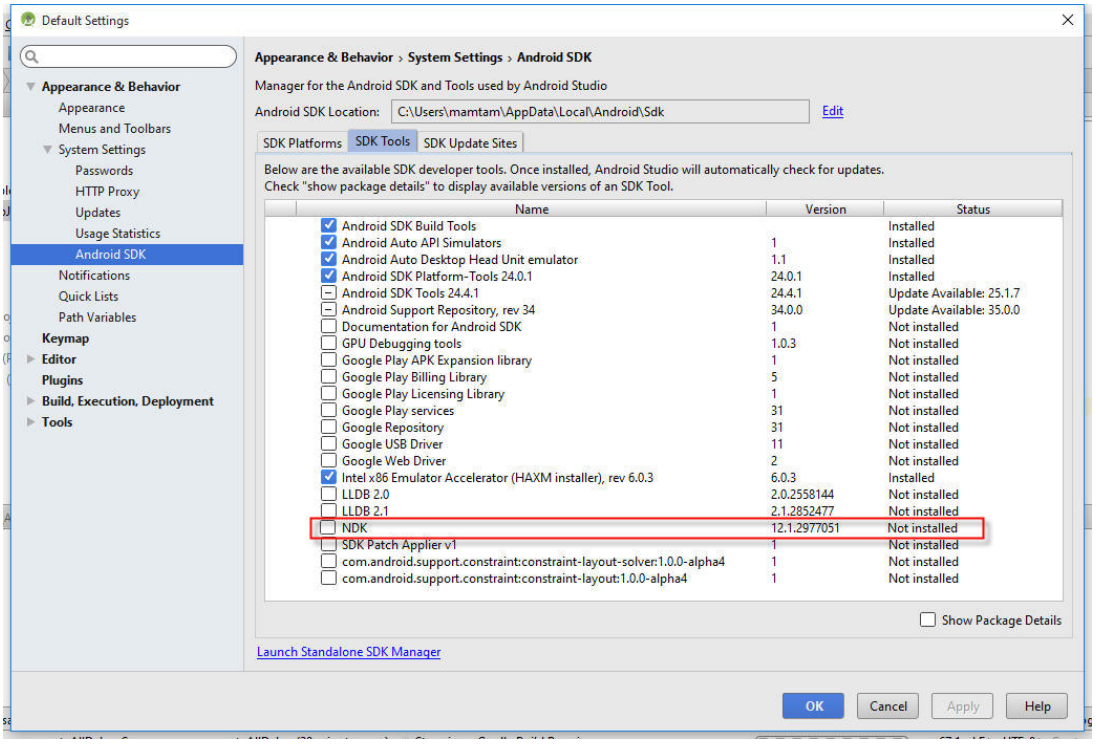


Figure 18.2: NDK

3. Click the check box beside NDK and then click **OK**. Wait until the tools are installed.

Now you are ready to create native apps with NDK.

18.4 Creating and Testing a Sample App

There are two ways for the developers to create applications using the NDK. They are as follows:

- The developers can build the application in Java or XML using the Android NDK.
- Then they can use JNI to access the Application Programming Interfaces (APIs) implemented in C or C++ using the Android NDK.

OR

- The developers can develop a native activity in C or C++.

Consider the first approach. Instead of creating a project from scratch, let us reuse an existing project sample from the Android Studio to understand native app development.

The steps to do this are as follows:

1. Launch Android Studio and select the **Import an Android code sample** option as shown in figure 18.3.

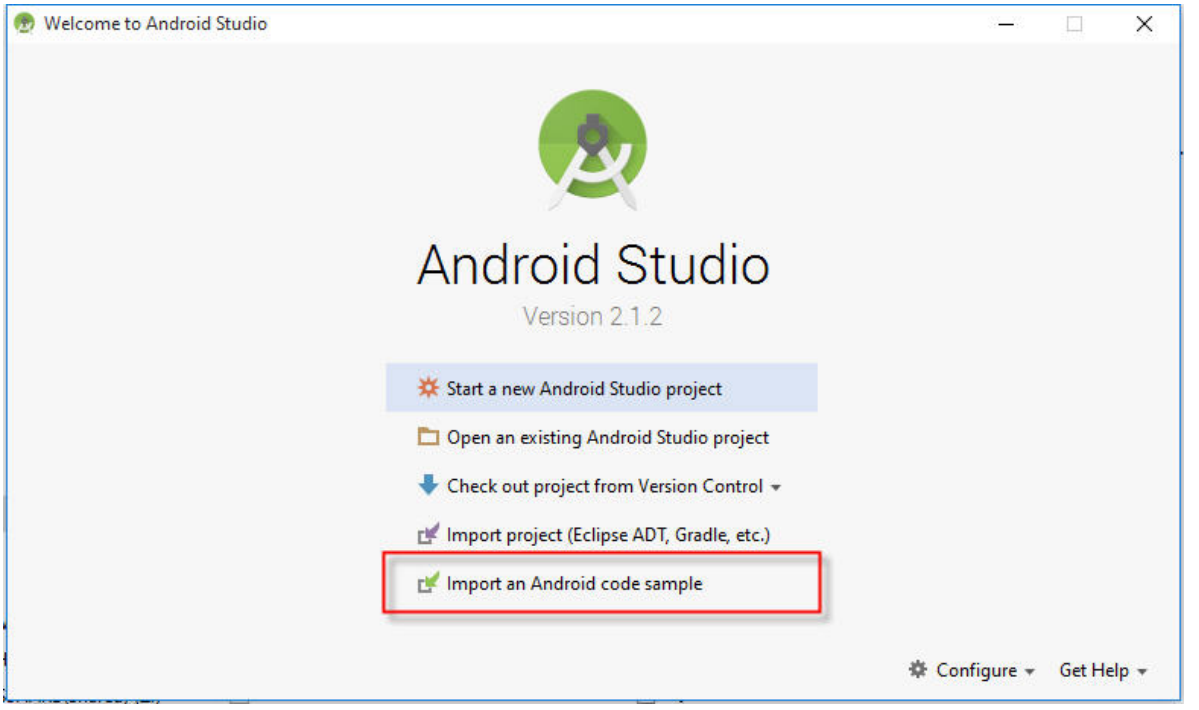


Figure 18.3: Importing a Code Sample

The **Import Sample** dialog box allows you to specify which samples to import.

2. Scroll down until you see **Ndk** and under that, select **Hello JNI**. This a readymade sample application that will call C code within an Android Java activity. Figure 18.4 shows this step.

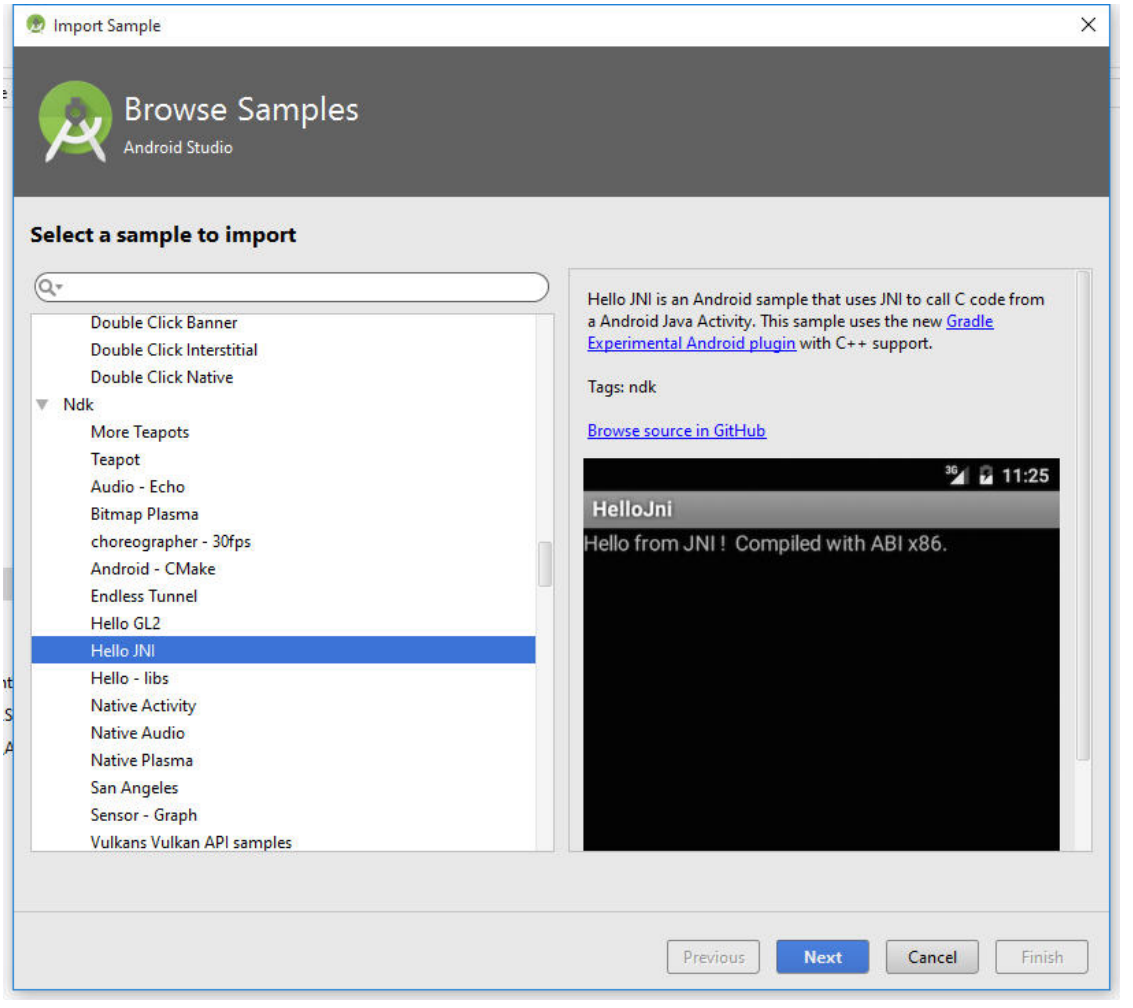


Figure 18.4: Selecting Hello JNI

3. Click **Next**.
4. Specify a name for the application as shown in figure 18.5 and then click **Finish**.

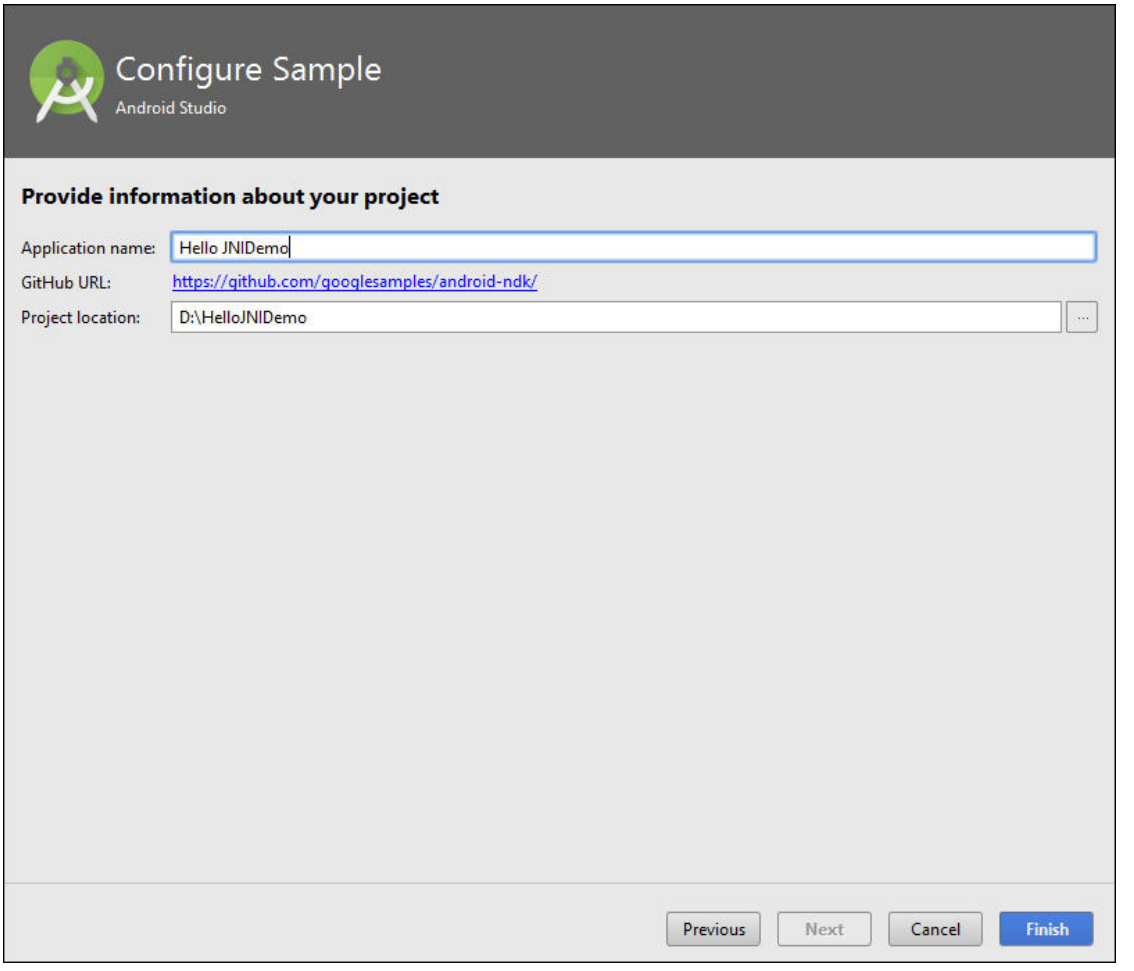


Figure 18.5: Specifying Application Name

The application will be created. Figure 18.6 displays the files that are imported as part of the project.

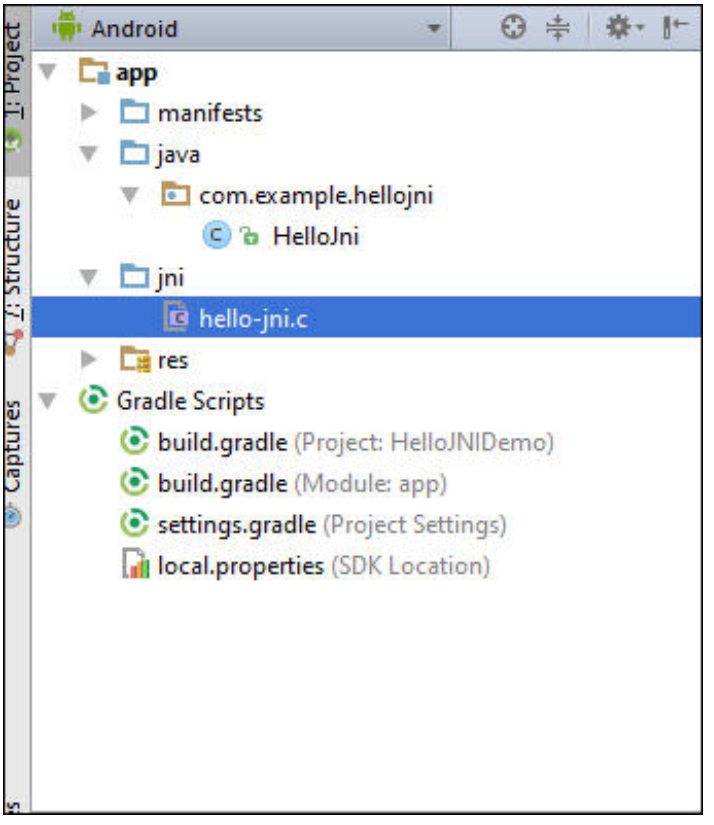


Figure 18.6: Files Created in HelloJNIDemo

Code Snippet 1 shows the code that is present in HelloJni.java.

Code Snippet 1:

```
public class HelloJni extends Activity
{
    /** Called when the activity is first created. */
    @Override
    public void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        /* Create a TextView and set its content.
         * the text is retrieved by calling a native
         * function.
         */
```

```
        TextView  tv = new TextView(this);
         tv.setText( stringFromJNI() );
         setContentView(tv);
    }
    /* A native method that is implemented by the
     * 'hello-jni' native library, which is packaged
     * with this application.
     */
    public native String  stringFromJNI();
    /* This is another native method declaration that is *not*
     * implemented by 'hello-jni'. This is simply to show that
     * you can declare as many native methods in your Java code
     * as you want, their implementation is searched in the
     * currently loaded native libraries only the first time
     * you call them.
     *
     * Trying to call this function will result in a
     * java.lang.UnsatisfiedLinkError exception !
     */
    public native String  unimplementedStringFromJNI();
    /* this is used to load the 'hello-jni' library on application
     * startup. The library has already been unpacked into
     * /data/data/com.example.hellojni/lib/libhello-jni.so at
     * installation time by the package manager.
     */
    static {
        System.loadLibrary("hello-jni");
       }
}
```

Java Native Interface (JNI) enables Java applications to interact with native code. The JNI and the NDK work in tandem to provide native support to Android apps. NDK is part of the Android framework whereas JNI is not specific to Android and is available to any Java application.

Through the JNI, C/C++ code is able to call Java code, including the standard Android libraries, and the Java code can call native functions defined in C/C++ code.

To run the app on an Android device emulator, perform steps similar to any other Android application and execute it. Alternatively, you can create an apk file and test it on an actual phone device.

18.5 Building an .apk File

An APK file is used to distribute and install the application on Android devices. The APK file has the source code, manifest, assets, resources, and certificates of the app project. To create an APK file for a project, in the Android Studio, click Build menu and select either Build APK or Generate Signed APK. Figure 18.7 shows the options.

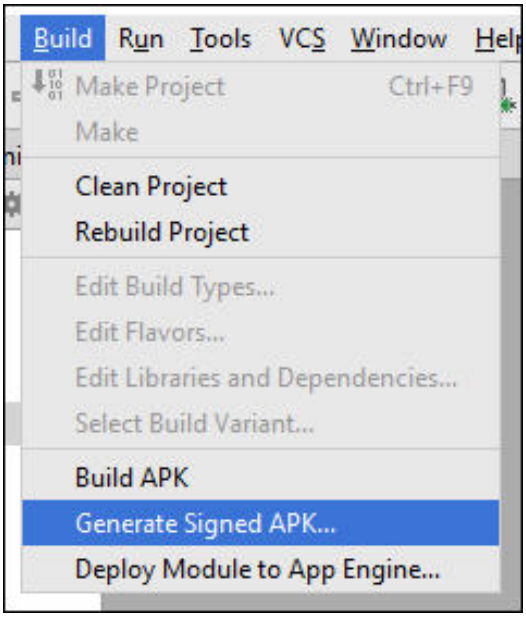


Figure 18.7: Building APK File

Once the file is generated, it can be deployed to an actual device.

When an app interacts with native code, it must support different CPU architectures. This means it must be compiled separately for each platform it has to be run on. Each ABI corresponds to a target architecture.

Check Your Progress

1. Which of the following files automatically identifies the project that needs to be built?

(A)	sdk-build	(B)	ABI
(C)	JNI	(D)	ndk-build

2. The Android build process generates _____files from the Java source code of the app.

(A)	.dex	(B)	.exe
(C)	.so	(D)	.a

3. Which of the following components acts as an interface between two programs that are at different levels?

(A)	Java	(B)	ABI
(C)	JNI	(D)	Native shared libraries

4. Which of the following components does the NDK build from the app's native source code?

(A)	ABI	(B)	JNI
(C)	Native static libraries	(D)	Native shared libraries

Answers

1.	D
2.	A
3.	B
4.	D

Session

18

Android Native Development Kit (NDK)

Summary

- The ndk-build file automatically identifies the project that needs to be built.
- The NDK builds the native shared libraries from the app's native source code.
- The NDK builds static libraries that can be linked with other existing libraries.
- JNI is the programming framework that enables Java code and C or C++ applications to interact with each other.
- Android Studio provides support to install NDK.
- ABI is the interface between two programs that are at different levels.
- The Android NDK has a helper class called the NativeActivity class that allows the developer to build a native activity.